代码随想录知识星球精华（最强八股文）第五版（计算机基础篇）

代码随想录知识星球精华（最强八股文）第五版为九份PDF，分别是：

- 代码随想录知识星球八股文概述
- C++篇
- Go篇
- Java篇
- 前端篇
- 算法题篇
- 计算机基础篇
- 问答精华篇
- 面经篇

本篇为最强八股文之计算机基础篇。

计算机网络

> 计算机网络是面试中的重点。无论是后端还是前端都会涉及到计算机网络的问题，其中三次握手、四次挥手、https、http缓存、tcp、udp被问到更是家常便饭，但是这些问题都有一定的复杂度，如果不给出过程的图解/讲解，录友们会很难理解，所以也就很难背下来，但是只给出解析过程，就很难抓住重点，不知道该具体回答什么内容，所以整理的八股会分为两个系列，讲解高频重点和一问一答区，高频重点区会深入讲解面试题，而一问一答区会补充高频重点之外的细碎的知识点，给出一个面试要回答的基本要点。

基础知识讲解

> 大部分内容来自 代码随想录知识星球，部分参考 小林coding、 牛客网，还有一些博客资料。

在浏览器中输入URL并按下回车之后会发生什么

> 超高频面试题，从这道面试题开始可以延伸到计算机网络有关的很多问题，如果这道题答的很好，可以把面试官引导到你更熟悉的地方。

1. 第一步：输入URL并解析

> 这里可以引申到URL的组成部分，这里不做过多的展开。

输入`URL`后，浏览器会解析出协议、主机、端口、路径等信息，并构造一个HTTP请求（浏览器会根据请求头判断是否有`HTTP缓存`,并根据是否有缓存决定是从服务器获取资源还是使用缓存资源，具体内容会在HTTP缓存章节讲解）

2. 第二步：DNS域名解析, 将域名解析成对应的IP地址

在发送HTTP请求之前，浏览器需要知道想要访问网页(url)对应的IP地址，这就需要使用到`DNS域名解析`（DNS域名解析的具体内容也会在后面章节中讲解）。

3. 第三步：建立起TCP连接之三次握手

> 这里是重中之重，可以扩展很多问题，比如为什么是三次，不是两次、四次？如果第一次握手丢失了会发生什么？三次握手过程中可以携带数据吗?

客户端和服务器之间进行HTTP请求和HTTP响应的过程中，需要建立起TCP连接，TCP连接需要进行三次握手（具体内容在后面章节中讲解）。

4. 第四步：浏览器发送HTTP/HTTPS请求到web服务器

> 这一步也可以扩展很多问题，比如HTTP/HTTPS的区别？请求的方式？请求的状态码，凡是与HTTP请求的问题都可以问。

5. 第五步：服务器处理HTTP请求并返回HTTP报文

> HTTP响应报文和状态码也是常考的内容。

服务器会接受请求并将其传递给请求处理程序并发送HTTP响应，一般响应报文包含：请求的网页以及状态码，压缩类型，如何缓存的页面，设置的cookie;

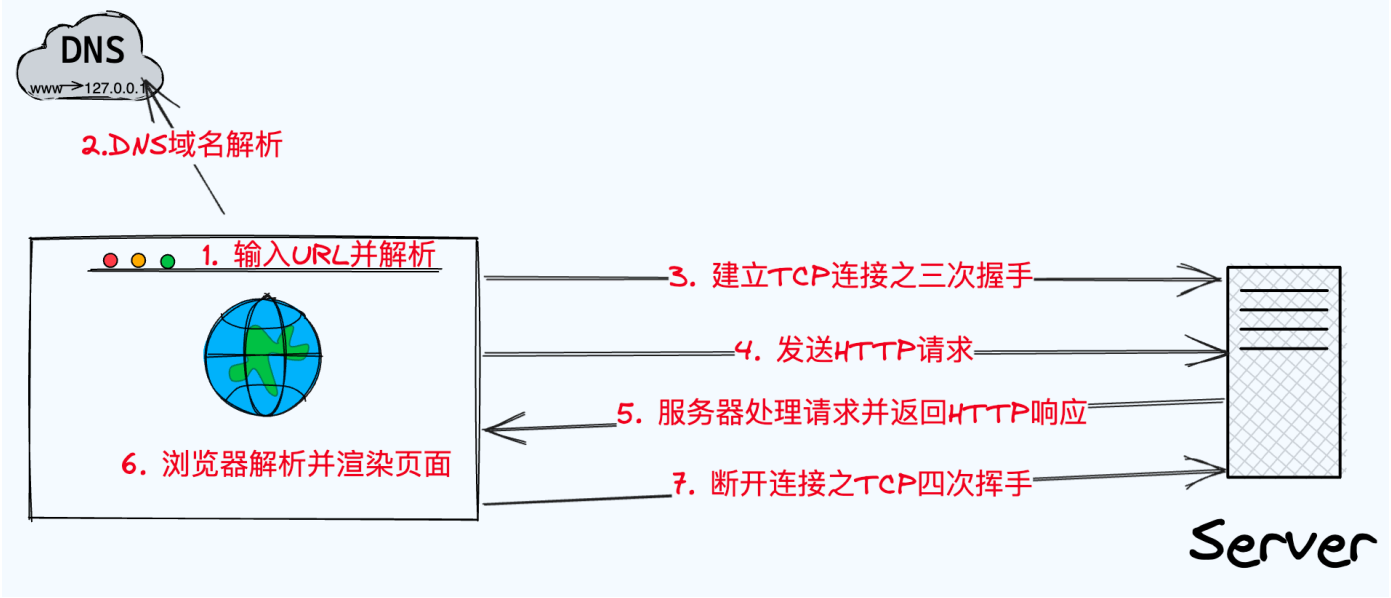
5. 第六步：浏览器渲染页面

> 浏览器渲染页面的流程对于前端同学来说也是必会的、还会牵扯到回流和重绘的问题。

7. 第七步：断开连接之TCP四次挥手

> 这里也是特别重要的知识点，四次挥手的过程，为什么是四次?

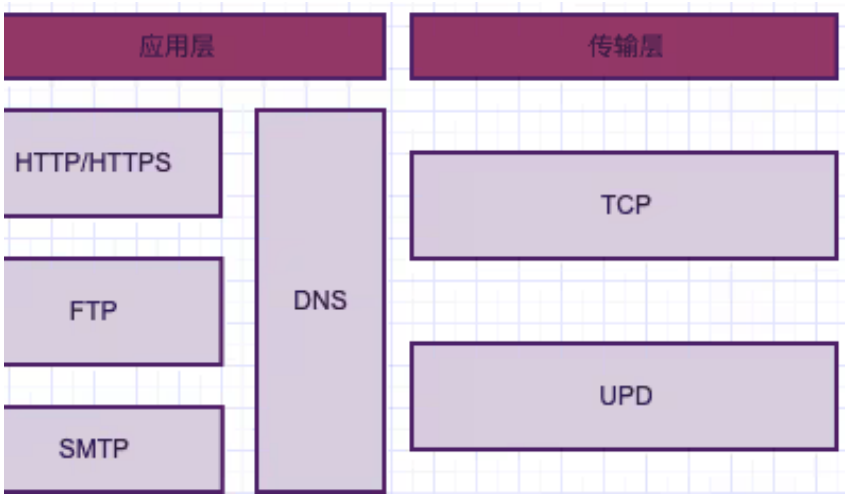
客户端和服务器之间断开连接需要进行四次挥手（具体内容在后面章节汇总讲解）。



DNS

DNS是什么

`DNS（Domain Name System）`是一种用于将域名（例如www.baidu.com）转换为IP地址（例如`220.181.111.188`）的**分布式**系统。在互联网上，计算机和其他网络设备使用IP地址来相互识别和通信。然而，IP地址是一串数字，不太方便人们使用和记忆，所以就使用了域名来代替复杂的IP地址



对于DNS服务器若采用集中式的设计有以下问题

1、单点故障

如果 DNS 服务器崩溃，那么整个网络随之瘫痪。通信容量(traaffic volume)，单个 DNS 服务器不得不处理所有的 DNS 查询，这种查询级别可能是上百万上千万级，一台服务器很难满足;

2、远距离集中式数据库

单个 DNS 服务器不可能 邻近 所有的用户，假设在美国的 DNS 服务器不可能临近让澳大利亚的查询使用，其中查询请求势必会经过低速和拥堵的链路，造成严重的时延;

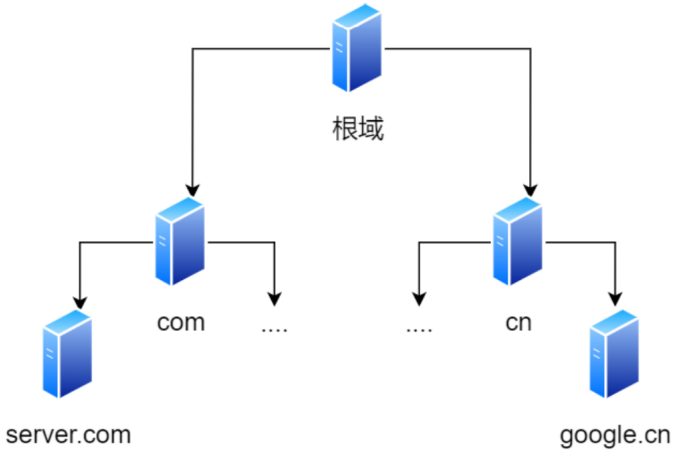
3、维护

维护成本巨大，而且还需要频繁更新。

域名的层级关系

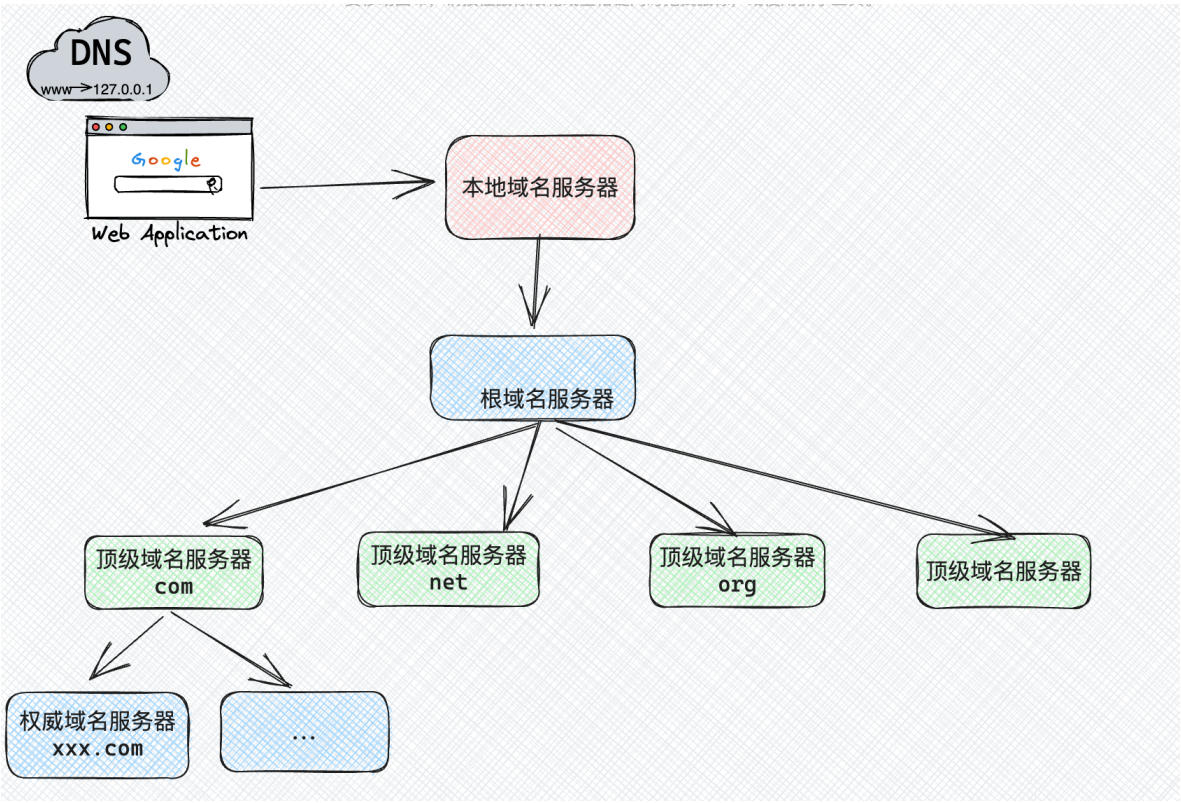
DNS 中的域名都是用句点来分隔的，比如 www.server.com ，这里的句点代表了不同层次之间的界限。

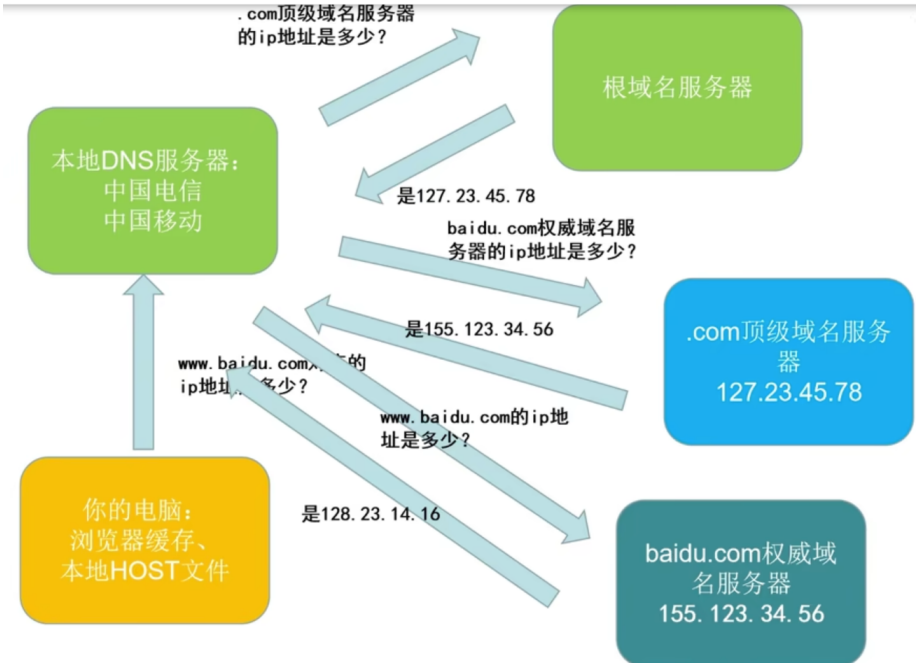
在域名中，越靠右的位置表示其层级越高。

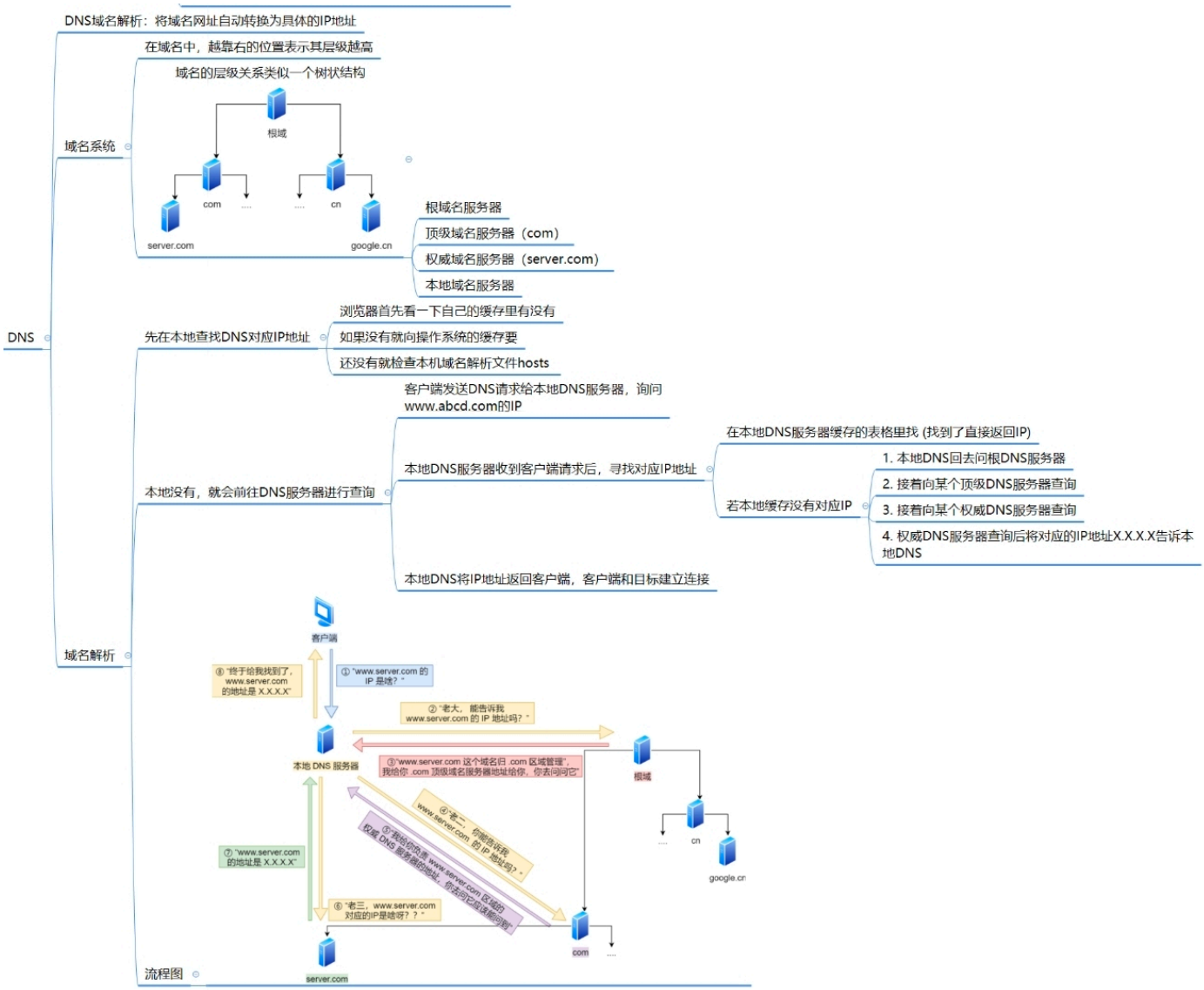


DNS解析过程

1. 先查询浏览器缓存是否有该域名对应的IP地址。
2. 如果浏览器缓存中没有，会去计算机本地的Host文件中查询是否有对应的缓存。
3. 如果Host文件中也没有则会向**本地的DNS服务器**（通常由你的互联网服务提供商（ISP）提供， 比如中国移动）发送一个DNS查询请求。
4. 如果本地DNS解析器有该域名的ip地址，就会直接返回，如果没有缓存该域名的解析记录，它会向**根DNS服务器**发出查询请求。根DNS服务器并不负责解析域名，但它能告诉本地DNS解析器应该向哪个顶级域（.com/.net/.org）的DNS服务器继续查询。
5. 本地DNS解析器接着向指定的**顶级域名DNS服务器**发出查询请求。顶级域DNS服务器也不负责具体的域名解析，但它能告诉本地DNS解析器应该前往哪个权威DNS服务器查询下一步的信息。
6. 本地DNS解析器最后向**权威DNS服务器**发送查询请求。 权威DNS服务器是负责存储特定域名和IP地址映射的服务器。当权威DNS服务器收到查询请求时，它会查找"example.com"域名对应的IP地址，并将结果返回给本地DNS解析器。
7. 本地DNS解析器将收到的IP地址返回给浏览器，并且还会将域名解析结果缓存在本地，以便下次访问时更快地响应。
8. **浏览器发起连接：** 本地DNS解析器已经将IP地址返回给您的计算机，您的浏览器可以使用该IP地址与目标服务器建立连接，开始获取网页内容。


DNS
www→127.0.0.1
Google
Web Application
本地域名服务器
根域名服务器
顶级域名服务器
com
顶级域名服务器
net
顶级域名服务器
org
顶级域名服务器
权威域名服务器
xxx.com
...


.com顶级域名服务器的ip地址是多少？
根域名服务器
本地DNS服务器：
中国电信
中国移动
是127.23.45.78
baidu.com权威域名服务器的ip地址是多少？
是155.123.34.56
.com顶级域名服务器
127.23.45.78
www.baidu.com的ip地址是多少？
www.baidu.com的ip地址是多少？
你的电脑：
浏览器缓存、
本地HOST文件
是128.23.14.16
baidu.com权威域名服务器
155.123.34.56



递归查询和迭代查询

递归查询和迭代查询是在DNS解析过程中用于获取域名解析信息的两种不同方法。

1. 递归查询

在递归查询中，DNS客户端（通常是本地DNS解析器）向上层DNS服务器（如根域名服务器、顶级域名服务器）发起查询请求，并要求这些服务器直接提供完整的解析结果。递归查询的特点是，DNS客户端只需要发送一个查询请求，然后等待完整的解析结果。上层DNS服务器会自行查询下一级的服务器，并将最终结果返回给DNS客户端。

2. 迭代查询

在迭代查询中，DNS客户端向上层DNS服务器发起查询请求，但不要求直接提供完整的解析结果。相反，DNS客户端只是询问上层服务器一个更高级的域名服务器的地址，然后再自行向那个更高级的服务器发起查询请求，以此类推，直到获取完整的解析结果为止。

递归查询适合普通用户和客户端，而迭代查询适用于DNS服务器之间的通信。

HTTP特性与简述

特性：简单、灵活、易于扩展、应用广泛和跨平台。

简述：

Web 上的通信都是建立在 HTTP 协议上的

1. 客户端发起 HTTP 请求；
2. 服务器做出响应处理后，返回 HTTP 响应报文

最初设想的基本理念是：借助多文档之间相互关联形成的超文本（HyperText），连成可相互参阅的 WWW（World Wide Web，万维网）

WWW 构建技术（3 项）：

1. HTML (HyperText Markup Language)：作为页面的文本标记语言
2. HTTP (HyperText Transfer Protocol)：文档传递协议；
3. URL (Uniform Resource Locator)：指定文档所在地址

HTTP 版本：

1. 1996 年的 5 月，HTTP/1.0，并记载于 RFC1945。现在 HTTP/1.0，仍然被使用在服务器端；
2. 1997 年 1 月，发布 HTTP/1.1 ，是目前主流的 HTTP 协议版本
3. 现今，HTTP/2.0 正在制订中，但还未得到广泛的使用

特性：

1. **简单**

基本报文格式为header+body，头部信息也是key-value简单文本的形式，易于理解。

2. **灵活和易于扩展**

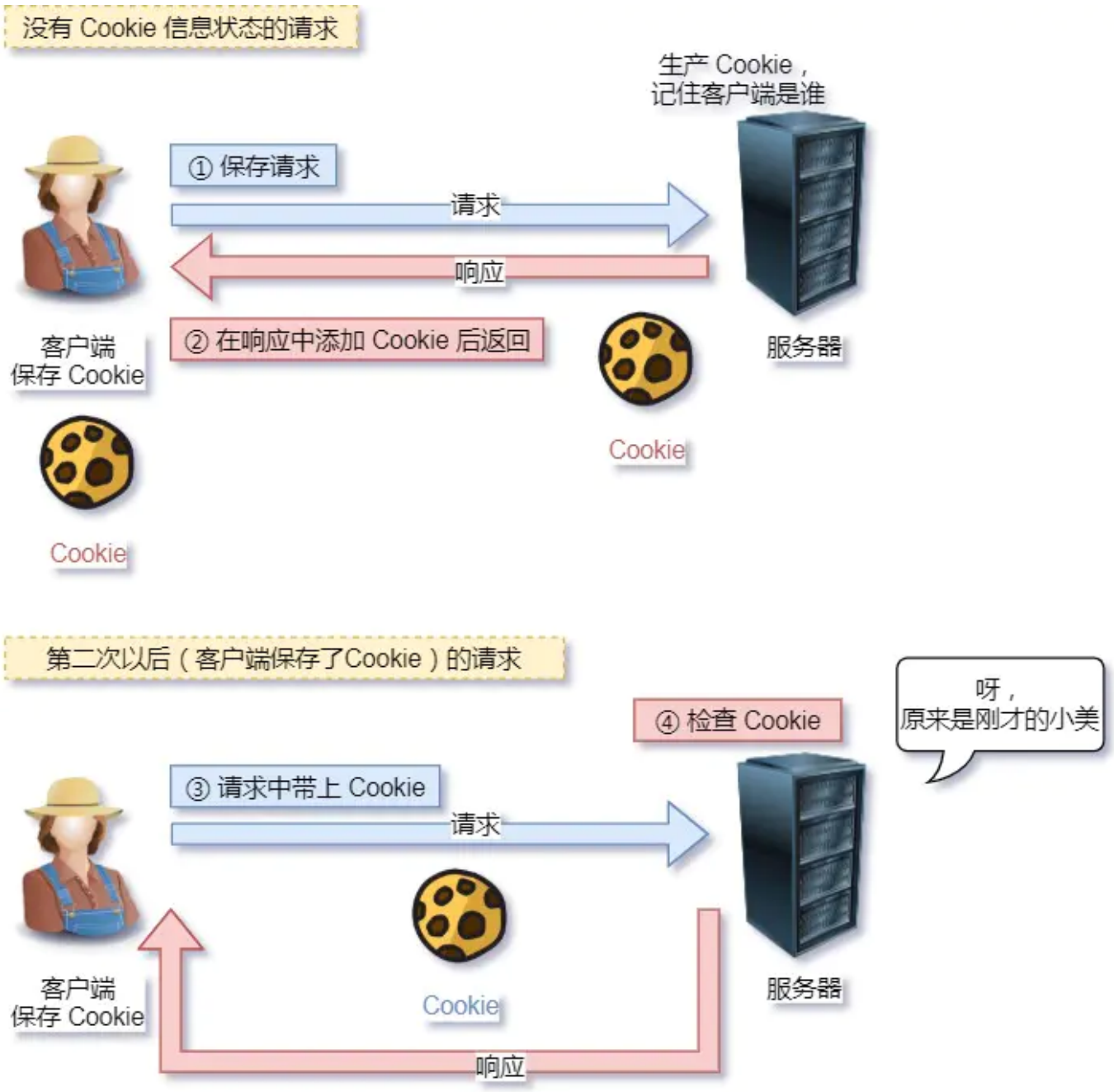
- HTTP协议里的各种请求方法、URI/URL、状态码、头字段等每个组成要求都没有被固定死，允许开发人员自定义和扩充;
- HTTP工作在应用层（OSI第七层），下层可以随意变化;
- HTTPS就是在HTTP与TCP之间增加了SSL/TSL安全传输层，HTTP/3把TCP换成了基于UDP的QUIC。

3. 无状态、明文传输、不安全

无状态：

服务器不会去记忆HTTP的状态，所以不需要额外的资源来记录状态信息，这能减轻服务器的负担。但它在完成有关联性的操作时会非常麻烦。

> 对于无状态的问题，解决方案友很多种，其中比较简单的方式的是 **Cookie** 技术，`Cookie` 通过在请求和响应报文中写入 Cookie 信息来控制客户端的状态。



明文传输：

传输过程中的信息，是可方便阅读的，通过浏览器的F12控制台或Wireshark抓包都可以直接肉眼查看

明文传输为我们调试工作带来了极大的便利性，但信息透明，容易被窃取。

不安全：

1. 通信使用明文（不加密），内容可能被窃听
2. 不验证通信方的身份，因此有可能遭遇伪装
3. 无法证明报文的完整性，所以有可能已遭篡改

可以用 HTTPS 的方式解决，也就是通过引入 `SSL/TLS` 层，使得在安全上达到了极致。

HTTP版本演变

目前为止，HTTP 常见的版本有 `HTTP/1.1`， `HTTP/2.0`， `HTTP/3.0`，不同版本的 HTTP 特性是不一样的。

HTTP/0.9

`HTTP/0.9`是最早的HTTP版本，在1991年就已经发布，只支持`GET`方法，也没有请求头，服务器只能返回`HTML`格式的内容。

HTTP/1.0

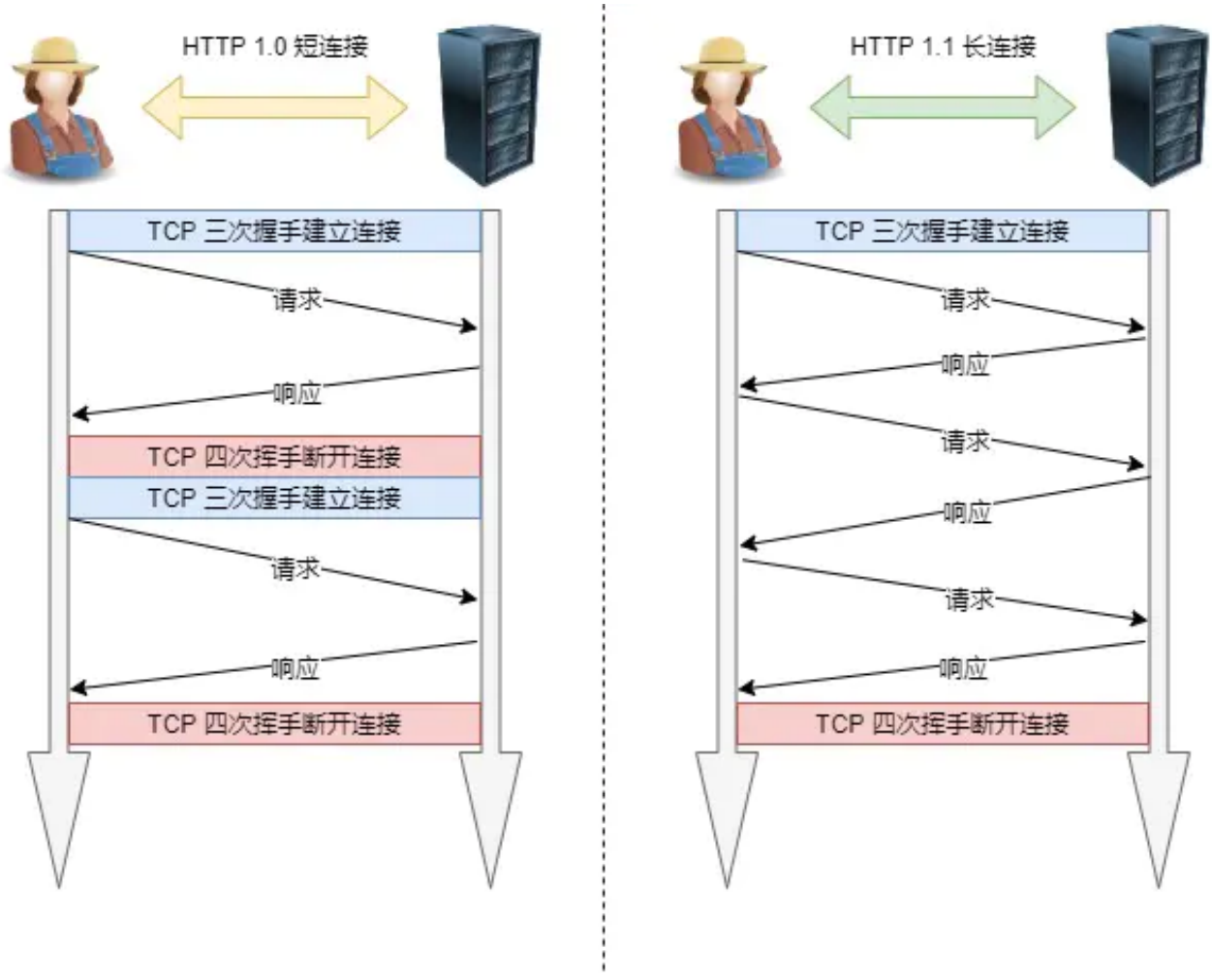
HTTP/1.0 是HTTTP 协议的第一个正式版本，主要具有以下特性：

- **引入了请求头和响应头，支持多种请求方法和状态码**
- 不支持持久连接，每次请求都需要建立新的连接

HTTP/1.1

1. 长连接

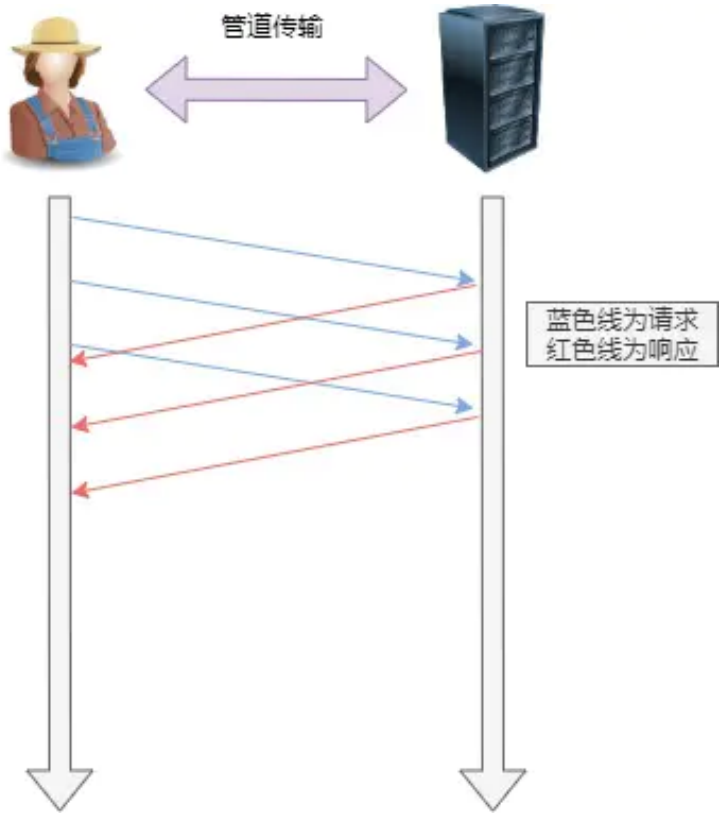
为了解决`HTTP/1.0`每次请求都需要建立新的连接的问题， `HTTP/1.1`提出了**长连接（持久连接）**，只要客户端和服务器任意一端没有明确提出断开连接，则保持TCP连接状态。



2. 管道网络传输

在同一个 TCP 连接里面，客户端可以发起多个请求，只要第一个请求发出去了，不必等其回来，就可以发第二个请求出去，可以**减少整体的响应时间。**

客户端需要请求两个资源。以前的做法是，在同一个 TCP 连接里面，先发送 A 请求，然后等待服务器做出回应，收到后再发出 B 请求。那么，管道机制则是允许浏览器同时发出 A 请求和 B 请求



但是**服务器必须按照接收请求的顺序发送对这些管道化请求的响应。**

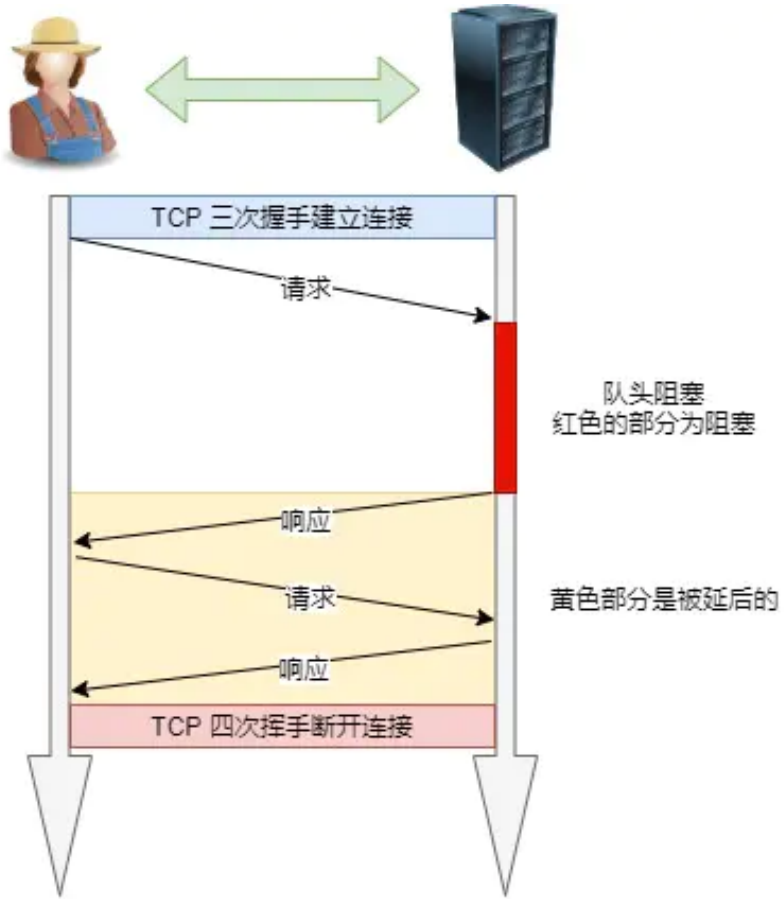
如果服务端在处理 A 请求时耗时比较长，那么后续的请求的处理都会被阻塞住，这称为「队头堵塞」。

所以，**HTTP/1.1 管道解决了请求的队头阻塞，但是没有解决响应的队头阻塞。**

> ⚠️ HTTP/1.1 管道化技术不是默认开启，而且浏览器基本都没有支持

3. 队头阻塞

当顺序发送的请求序列中的一个请求因为某种原因被阻塞时，在后面排队的所有请求也一同被阻塞了，会招致客户端一直请求不到数据，这也就是「**队头阻塞**」

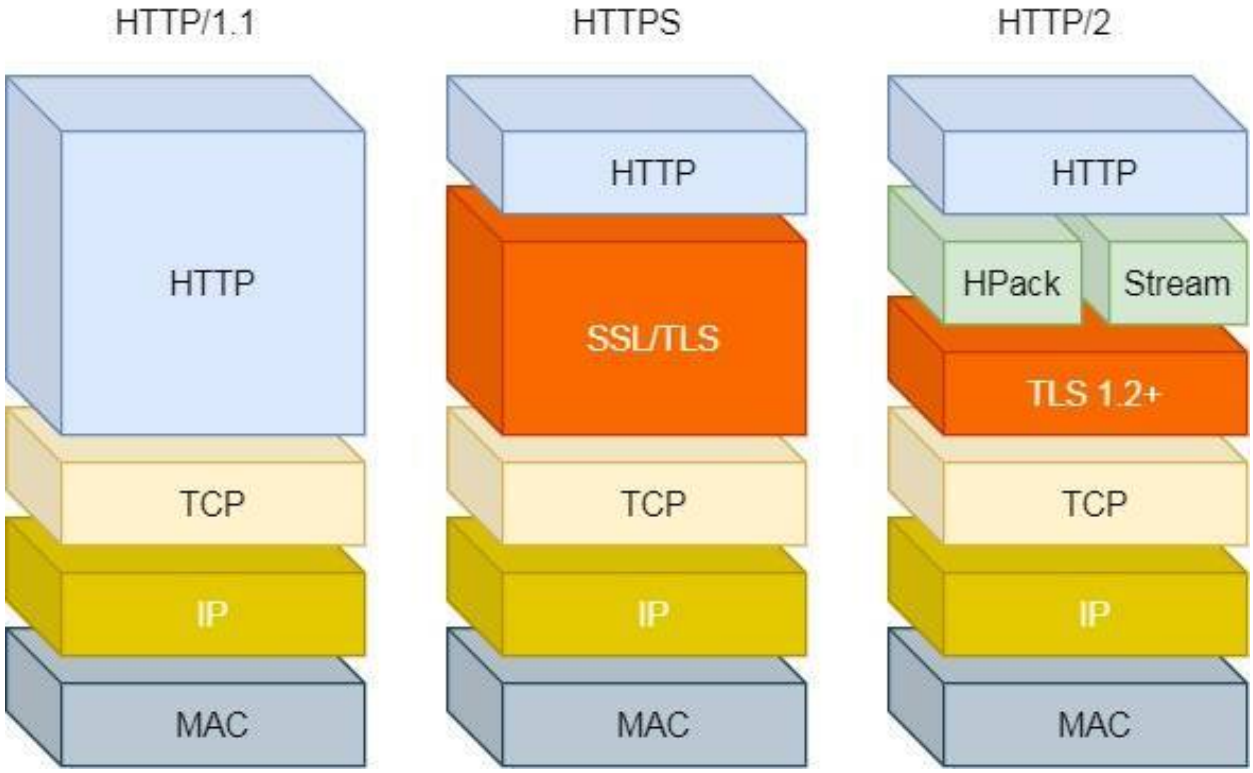


但是`HTTP1.1`仍然存在着不少问题：

- 头部冗余：每个请求和响应都需要带有一定的头部信息，每次互相发送相同的首部造成的浪费较多；
- 服务器是按请求的顺序响应的，如果服务器响应慢，会招致客户端一直请求不到数据，也就是队头阻塞；
- 没有请求优先级控制；
- 请求只能从客户端开始，服务器只能被动响应。

HTTP/2

HTTP/2 协议是基于 HTTPS 的, 所以HTTP/2的安全性也是有保障的



1. 头部压缩：HTTP/2 使用 `HPACK` 压缩算法对请求和响应头部进行压缩，减少了传输的头部数据量，降低了延迟。
2. 二进制帧：HTTP/2 将数据分割成二进制帧进行传输，分为**头信息帧（Headers Frame）和数据帧（Data Frame）**，增加了数据传输的效率。
3. 并发传输：引出了 Stream 概念，多个 Stream 复用在一条 TCP 连接，针对不同的 HTTP 请求用独一无二的 Stream ID 来区分，接收端可以通过 Stream ID 有序组装成 HTTP 消息，不同 Stream 的帧是可以乱序发送的，因此可以并发不同的 Stream ，也就是 HTTP/2 可以并行交错地发送请求和响应。
4. 服务器推送：在 `HTTP/2` 中，服务器可以对客户端的一个请求发送多个响应，即服务器可以额外的向客户端推送资源，而无需客户端明确的请求。

但是`HTTP/2`仍然存在着队头阻塞的问题，只不过问题是在传输层。

HTTP/2 是基于 TCP 协议来传输数据的，TCP 是字节流协议，TCP 层必须保证收到的字节数据是完整且连续的，这样内核才会将缓冲区里的数据返回给 HTTP 应用，那么当「前 1 个字节数据」没有到达时，后收到的字节数据只能存放在内核缓冲区里，只有等到这 1 个字节数据到达时，HTTP/2 应用层才能从内核中拿到数据，这就是 HTTP/2 队头阻塞问题。

HTTP1.1协议的性能问题

1、HTTP1.1常见性能问题

- *延迟难以下降*，虽然现在网络的「带宽」相比以前变多了，但是延迟降到一定幅度后，就很难再下降了，说白了就是到达了延迟的下限；
- *并发连接有限*，谷歌浏览器最大并发连接数是 6 个，而且每一个连接都要经过 TCP 和 TLS 握手耗时，以及 TCP 慢启动过程给流量带来的影响；
- *队头阻塞问题*，同一连接只能在完成一个 HTTP 事务（请求和响应）后，才能处理下一个事务；
- *HTTP 头部巨大且重复*，由于 HTTP 协议是无状态的，每一个请求都得携带 HTTP 头部，特别是对于有携带 cookie 的头部，而 cookie 的大小通常很大；
- *不支持服务器推送消息*，因此当客户端需要获取通知时，只能通过定时器不断地拉取消息，这无疑浪费大量了带宽和服务器资源。

2、为解决HTTP1.1性能问题而提出的常见优化手段

- 将多张小图合并成一张大图供浏览器 JavaScript 来切割使用，这样可以将多个请求合并成一个请求，但是带来了新的问题，当某张小图片更新了，那么需要重新请求大图片，浪费了大量的网络带宽；
- 将图片的二进制数据通过 base64 编码后，把编码数据嵌入到 HTML 或 CSS 文件中，以此来减少网络请求次数；
- 将多个体积较小的 JavaScript 文件使用 webpack 等工具打包成一个体积更大的 JavaScript 文件，以一个请求替代了很多个请求，但是带来的问题，当某个 js 文件变化了，需要重新请求同一个包里的所有 js 文件；
- 将同一个页面的资源分散到不同域名，提升并发连接上限，因为浏览器通常对同一域名的 HTTP 连接最大只能是 6 个；

但是上述优化手段都只是“外部”优化，效果不够好。

兼容HTTP1.1

HTTP2，为改善HTTP1.1而提出，同时兼容了HTTP1.1，具体做法如下：

> 第一点，HTTP/2 没有在 URI 里引入新的协议名，仍然用「http://」表示明文协议，用「https://」表示加密协议，于是只需要浏览器和服务器在背后自动升级协议，这样可以让用户意识不到协议的升级，很好的实现了协议的平滑升级。

> 第二点，只在应用层做了改变，还是基于 TCP 协议传输，应用层方面为了保持功能上的兼容，HTTP/2 把 HTTP 分解成了「语义」和「语法」两个部分，「语义」层不做改动，与 HTTP/1.1 完全一致，比如请求方法、状态码、头字段等规则保留不变。
>
> 但是，HTTP/2 在「语法」层面做了很多改造，基本改变了 HTTP 报文的传输格式。

头部压缩

1、HTTP1.1报文中Header部分存在的问题：

> - 含很多固定的字段，比如Cookie、User Agent、Accept 等，这些字段加起来也高达几百字节甚至上千字节，所以有必要**压缩**；
> - 大量的请求和响应的报文里有很多字段值都是重复的，这样会使得大量带宽被这些冗余的数据占用了，所以有必须要**避免重复性**；
> - 字段是 ASCII 编码的，虽然易于人类观察，但效率低，所以有必要改成**二进制编码**；

2、HTTP2的改造：

使用HPACK算法压缩头部。HPACK算法包含三个组成部分：

（i）静态字典
（ii）动态字典
（iii）Huffman编码（压缩算法）

客户端和服务器都会建立和维护【字典】，用长度较小的索引号表示重复的字符串，再用`Huffman`编码压缩数据。

（1）静态表编码：

HTTP2 为高频出现在头部的字符串和字段建立了一张静态表，它是写入到 HTTP2 框架里的，不会变化的，静态表里共有 61 组：

Index	Header Name	Header Value
1	:authority	
2	:method	GET
3	:method	POST
4	:path	/
5	:path	/index.html
6	:scheme	http
7	:scheme	https
8	:status	200
...	...	...
54	server	
55	set-cookie	
56	strict-transport-security	
57	transfer-encoding	
58	user-agent	
59	vary	
60	via	
61	www-authenticate	

「代码随想录」知识星球

（2）动态表编码：

使得动态表生效的一个前提：必须同一个连接上，重复传输完全相同的HTTP头部。

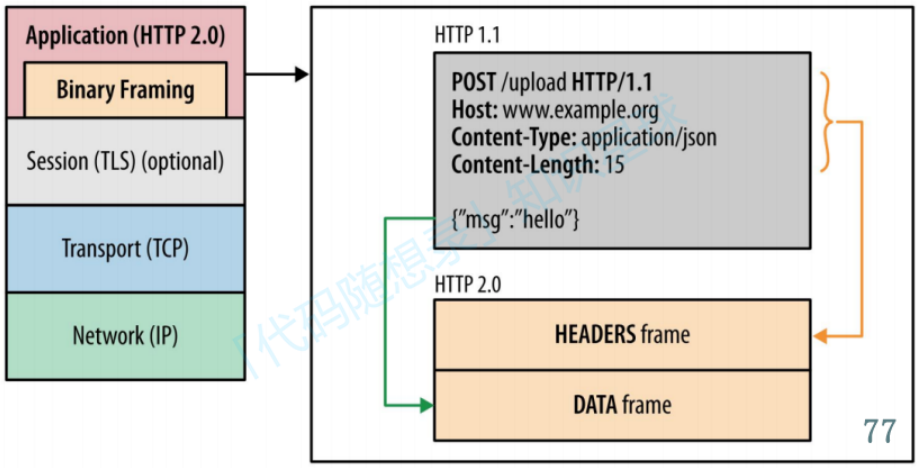
动态表越大，占用的内存也就越大，如果占用了太多内存，是会影响服务器性能的，因此 Web 服务器都会提供类似 `http2_max_requests` 的配置，用于限制一个连接上能够传输的请求数，避免动态表无限增大，请求数到达上限后，就会关闭 HTTP/2 连接来释放内存。

HTTP2头部的编码通过【静态表、动态表、Huffman编码】共同完成。

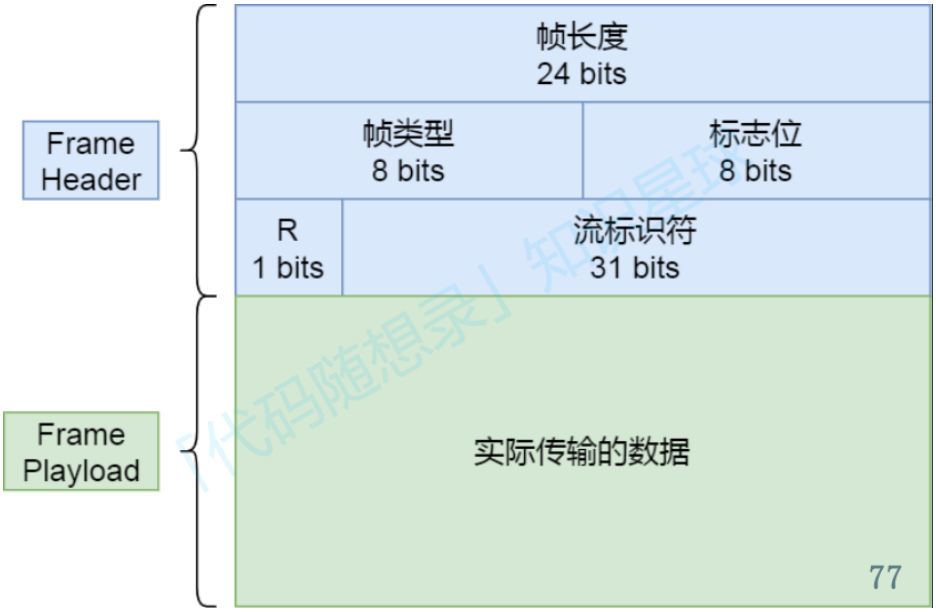
二进制帧

HTTP2更厉害的地方在于：将HTTP1的文本格式改成二进制格式传输数据，极大提高了HTTP传输效率，而且二进制数据使用位运算能高效解析。

两者响应区别（如下图）：



HTTP2二进制帧的结构：



HTTP2总共定义了10种类型的帧，分为两类，即**数据帧**和**控制帧**：

	帧类型	类型编码	用途
数据帧	DATA	0x0	传递HTTP包体
	HEADERS	0x1	传递HTTP头部
	PRIORITY	0x2	指定Stream流的优先级
控制帧	RST_STREAM	0x3	终止Stream流
	SETTINGS	0x4	修改连接或者Stream流的配置
	PUSH_PROMISE	0x5	服务端推送资源时描述请求的帧
	PING	0x6	心跳检测，兼具计算RTT往返时间的功能
	GOAWAY	0x7	优雅的终止连接或者通知错误
	WINDOW_UPDATE	0x8	实现流量控制
	CONTINUATION	0x9	传递较大HTTP头部时的持续帧

77

标志位用于携带简单的控制信息，比如：

- **END_HEADERS** 表示头数据结束标志，相当于 HTTP/1 里头后的空行（“\r\n”）；
- **END_STREAM** 表示单方向数据发送结束，后续不会再有数据帧。
- **PRIORITY** 表示流的优先级；

流标识符（Stream ID)：

用来标识该Fream属于哪个Stream。接收方可以根据这个信息从乱序的帧里找到相同Stream ID的帧，从而有序组装信息。

帧数据：

用于存放 通过`HPACK`算法压缩过的HTTP头部和包体。

并发传输

1、HTTP1.1与HTTP2的比较

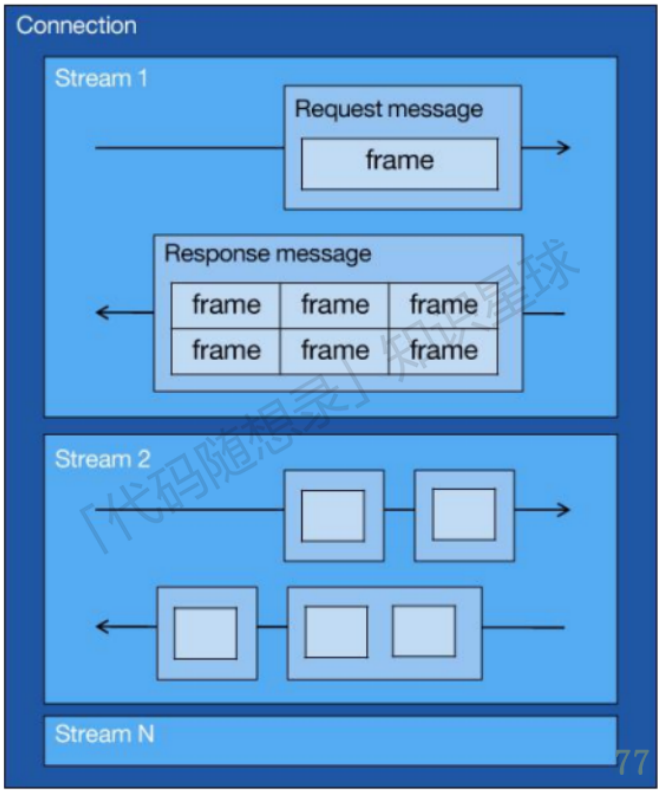
HTTP1.1 基于请求-响应模型。同一个连接中，HTTP完成一个事务（请求与响应），才能处理下一个事务。即：再发出请求等待响应的过程种是没办法做其他事情的，会造成【队头阻塞】问题。

HTTP2通过Stream这个设计（多个Stream复用一条TCP连接，达到并发的效果），解决了【队头阻塞】的问题，提高了HTTP传输的吞吐量。

2、HTTPS并发的实现

HTTP2并发是如何实现的呢?

先来理解三个概念，即：Stream、Message、frame。



从上图可以看到：

- 1 个 TCP 连接包含一个或者多个 Stream，Stream 是 HTTP/2 并发的关键技术；
- Stream 里可以包含 1 个或多个 Message，Message 对应 HTTP/1 中的请求或响应，由 HTTP 头部和包体构成；
- Message 里包含一条或者多个 Frame，Frame 是 HTTP/2 最小单位，以二进制压缩格式存放 HTTP/1 中的内容（头部和包体）；

1. HTTP消息可以由多个Frame构成
2. 一个Frame可以由多个TCP报文构成

在HTTP2连接上，不同Stream的帧可以乱序发送（因此可以并发不同的Stream），接收端可以通过Stream ID 有序组装HTTP消息。

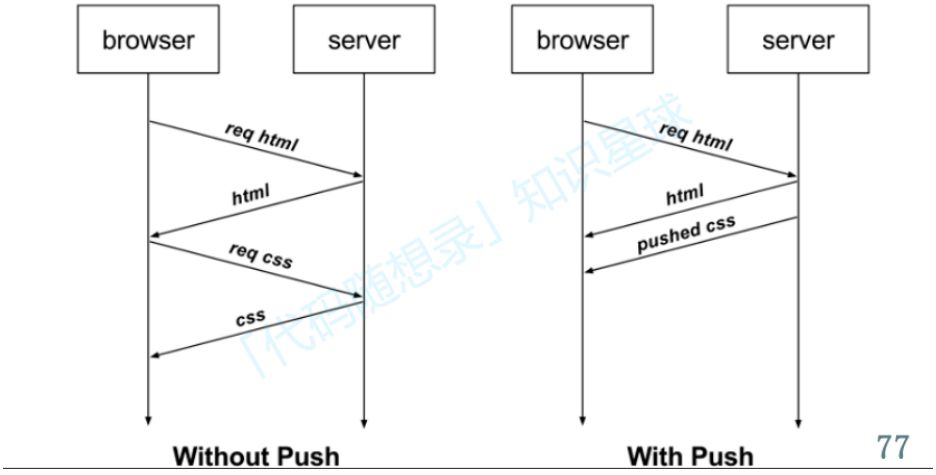
HTTP/2 通过 Stream 实现的并发，比 HTTP/1.1 通过 TCP 连接实现并发要牛逼的多，**因为当 HTTP/2 实现 100 个并发 Stream 时，只需要建立一次 TCP 连接，而 HTTP/1.1 需要建立 100 个 TCP 连接，每个 TCP 连接都要经过 TCP 握手、慢启动以及 TLS 握手过程，这些都是很耗时的。**

HTTP/2 还可以对每个 Stream 设置不同**优先级**，帧头中的「标志位」可以设置优先级，比如客户端访问 HTML/CSS 和图片资源时，希望服务器先传递 HTML/CSS，再传图片，那么就可以通过设置 Stream 的优先级来实现，以此提高用户体验。

服务器主动推送资源

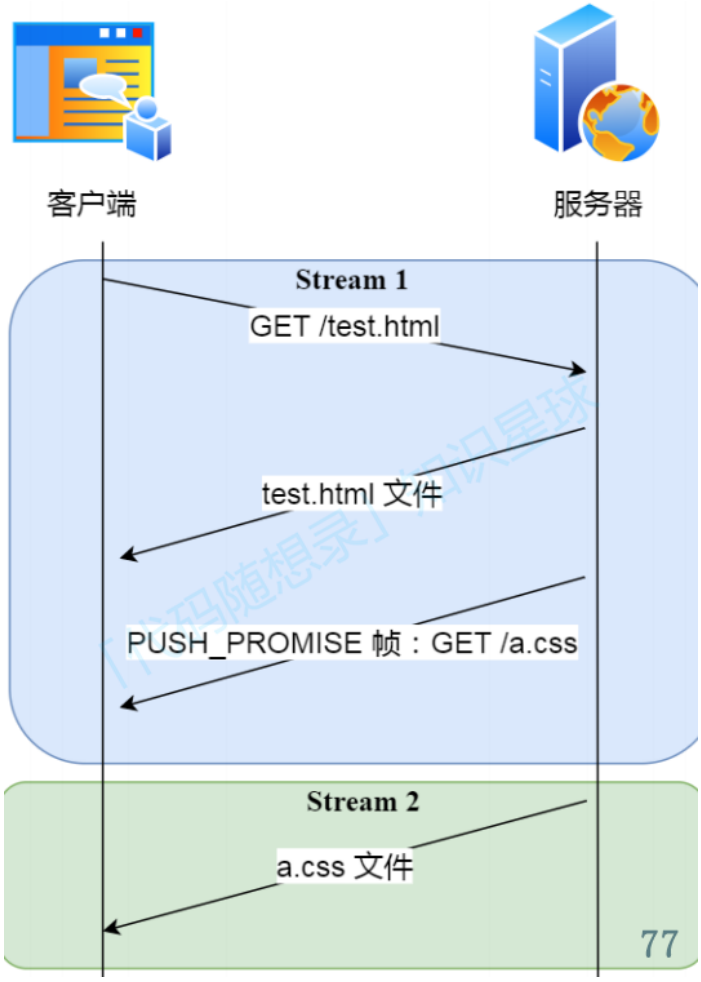
1、HTTP1.1与HTTP2的比较：

HTTP1.1不支持服务器主动推送资源给客户端，都是由客户端向服务器发起请求后，才能获取到服务器响应的资源。在HTTP2中，客户端在访问HTML时，服务器可以直接主动推送CSS文件，减少了消息传递的次数。



HTTP2推送的具体实现方式：

客户端发起的请求，必须使用的是奇数号 Stream，服务器主动的推送，使用的是偶数号 Stream。服务器在推送资源时，会通过 `PUSH_PROMISE` 帧传输 HTTP 头部，并通过帧中的 `Promised Stream ID` 字段告知客户端，接下来会在哪个偶数号 Stream 中发送包体。



总结

HTTP2协议还包括：流控制、流状态、依赖关系等。

本次主要介绍了关于HTTP如何提升性能：

第一点，对于常见的 HTTP 头部通过**静态表和 Huffman 编码**的方式，将体积压缩了近一半，而且针对后续的请求头部，还可以建立**动态表**，将体积压缩近 90%，大大提高了编码效率，同时节约了带宽资源。

不过，动态表并非可以无限增大， 因为动态表是会占用内存的，动态表越大，内存也越大，容易影响服务器总体的并发能力，因此服务器需要限制 HTTP/2 连接时长或者请求次数。

第二点，**HTTP/2 实现了 Stream 并发**，多个 Stream 只需复用 1 个 TCP 连接，节约了 TCP 和 TLS 握手时间，以及减少了 TCP 慢启动阶段对流量的影响。不同的 Stream ID 才可以并发，即时乱序发送帧也没问题，但是同一个 Stream 里的帧必须严格有序。

另外，可以根据资源的渲染顺序来设置 Stream 的**优先级**，从而提高用户体验。

第三点，**服务器支持主动推送资源**，大大提升了消息的传输性能，服务器推送资源时，会先发送 PUSH_PROMISE 帧，告诉客户端接下来在哪个 Stream 发送资源，然后用偶数号 Stream 发送资源给客户端。

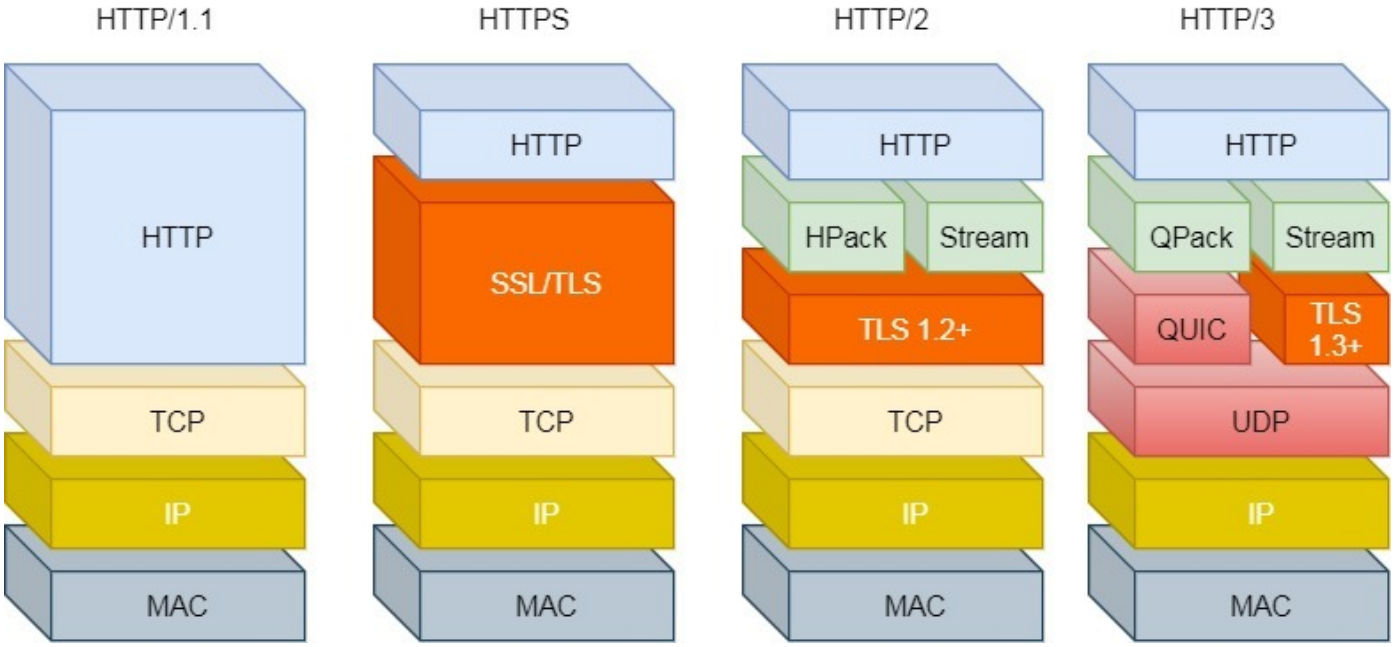
HTTP/2 通过 Stream 的并发能力，解决了 HTTP/1 队头阻塞的问题，看似很完美了，但是 HTTP/2 还是存在“队头阻塞”的问题，只不过问题不是在 HTTP 这一层面，而是在 TCP 这一层。

HTTP/2 是基于 TCP 协议来传输数据的，TCP 是字节流协议，TCP 层必须保证收到的字节数据是完整且连续的，这样内核才会将缓冲区里的数据返回给 HTTP 应用，那么当「前 1 个字节数据」没有到达时，后收到的字节数据只能存放在内核缓冲区里，只有等到这 1 个字节数据到达时，HTTP/2 应用层才能从内核中拿到数据，这就是 HTTP/2 队头阻塞问题。

有没有什么解决方案呢？既然是 TCP 协议自身的问题，那干脆放弃 TCP 协议，转而使用 UDP 协议作为传输层协议，这个大胆的决定， HTTP/3 协议做了！

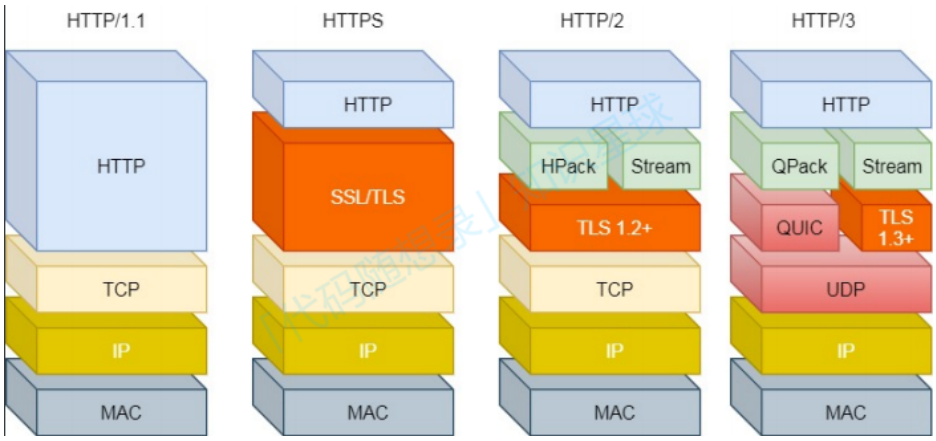
HTTP/3

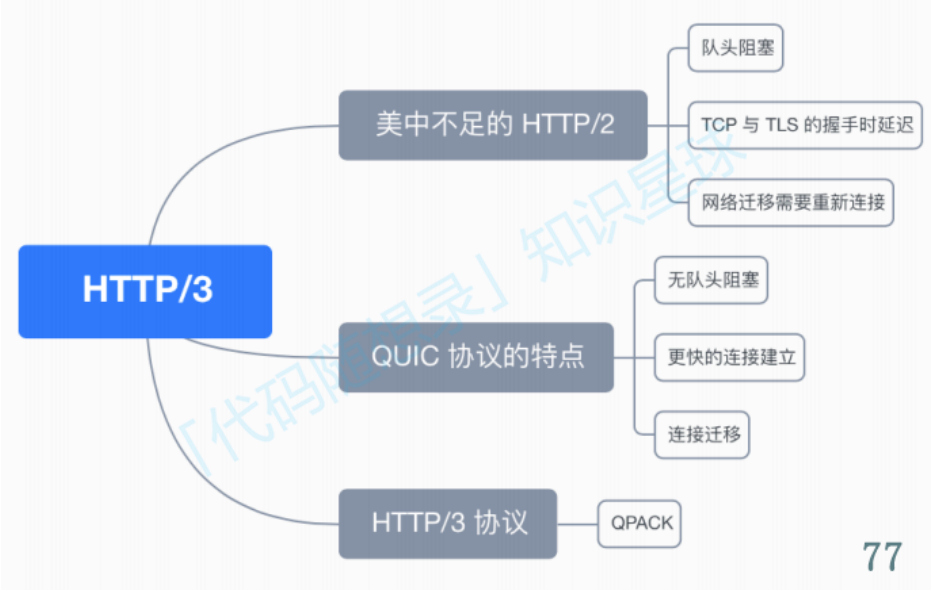
> HTTP/2 队头阻塞的问题是因为 TCP，所以 **HTTP/3 把 HTTP 下层的 TCP 协议改成了 UDP！**



HTTP/3 基于 `QUIC` 协议，具有以下特点：

- **零 RTT 连接建立**：QUIC 允许在首次连接时进行零往返时间（Zero Round Trip Time）连接建立，从而减少了连接延迟，加快了页面加载速度。
- 无队头阻塞：QUIC 使用 `UDP` 协议来传输数据。一个连接上的多个stream之间没有依赖, 如果一个stream丢了一个UDP包，不会影响后面的stream，不存在 TCP 队头阻塞
- **连接迁移**：QUIC 允许在网络切换（如从 Wi-Fi 到移动网络）时，将连接迁移到新的 IP 地址，从而减少连接的中断时间。
- 向前纠错机制：每个数据包除了它本身的内容之外，还包括了部分其他数据包的数据，因此少量的丢包可以通过其他包的冗余数据直接组装而无需重传。向前纠错牺牲了每个数据包可以发送数据的上限，但是减少了因为丢包导致的数据重传。





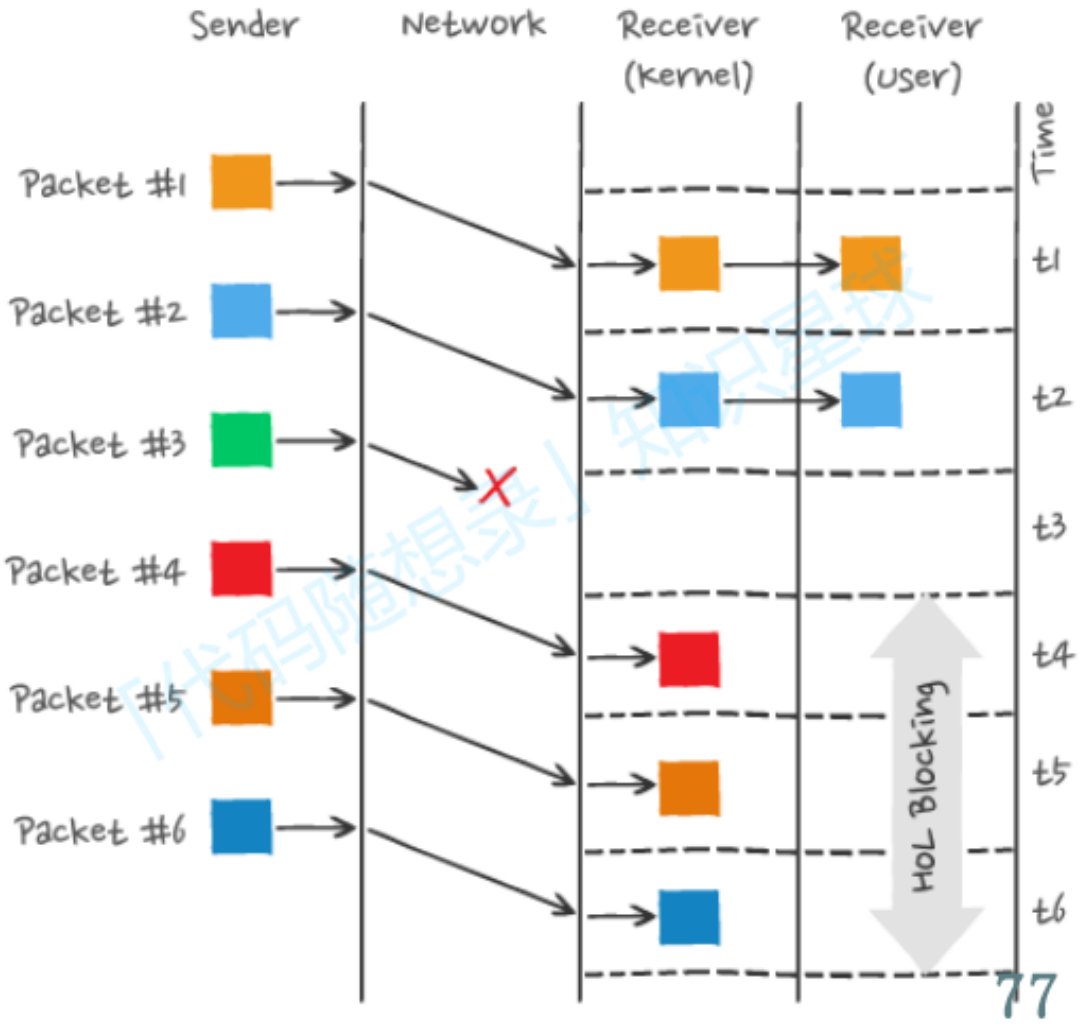
HTTP/2的缺点

HTTP2协议是基于TCP实现的，所以存在三个缺陷：

(i) 队头阻塞
(ii) TCP与TLS的握手时延迟
(iii)网络迁移需要重新连接

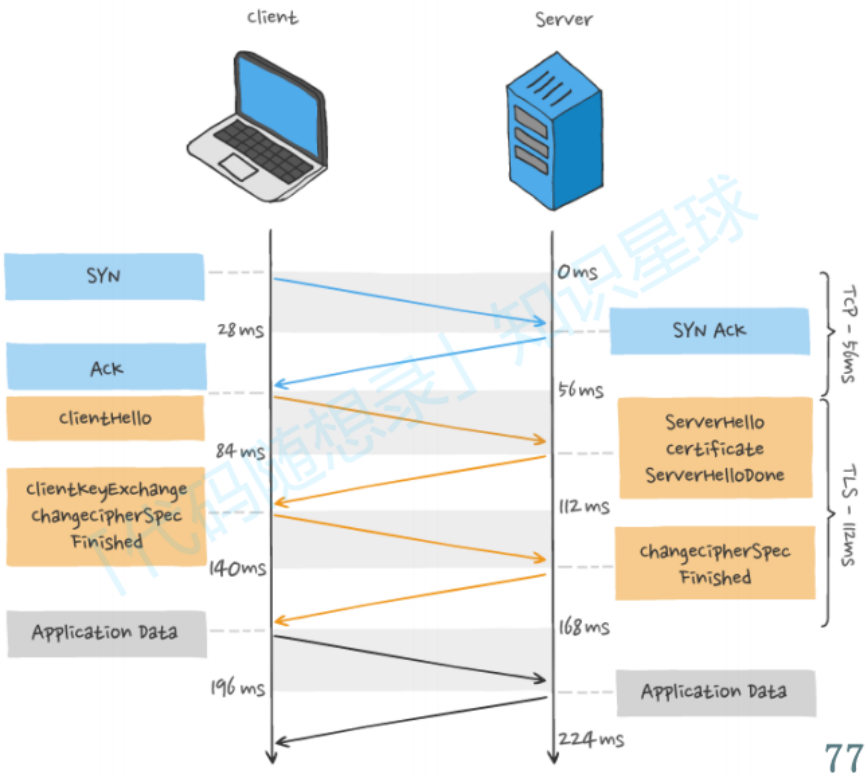
1、队头阻塞

TCP 是字节流协议，TCP 层必须保证收到的字节数据是完整且有序的，如果序列号较低的 TCP 段在网络传输中丢失了，即使序列号较高的 TCP 段已经被接收了，应用层也无法从内核中读取到这部分数据，从 HTTP 视角看，就是请求被阻塞了。



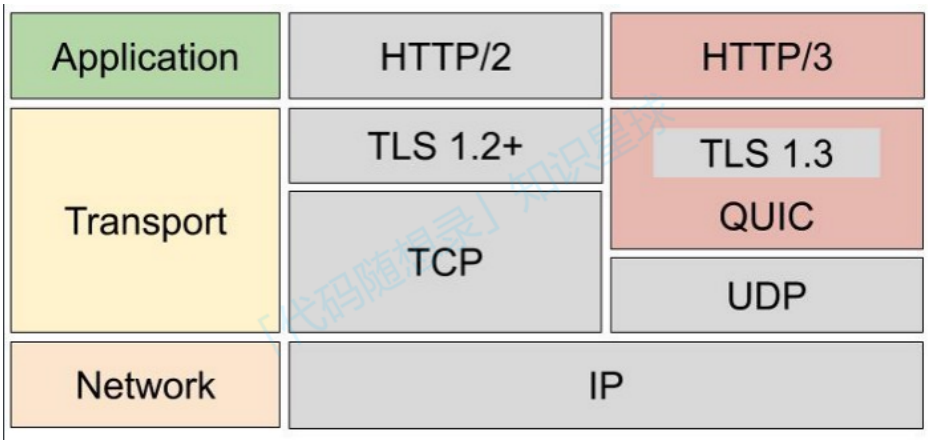
2、TCP与TLS的握手时延迟

发出HTTP请求时，需要经过TCP三次握手和TLS四次握手，共计3RTT的时延才能发出请求数据。



3、网络迁移需要重新连接

一个TCP连接由【源IP地址，源端口，目标IP地址，目标端口】确定。若IP地址或端口发生变暖，这需要重新进行连接。这不利于移动设备切换网络的场景。要解决该问题，就要修改传输层协议。在HTTP3中传输层协议修改为了UDP。



QUIC协议的特点

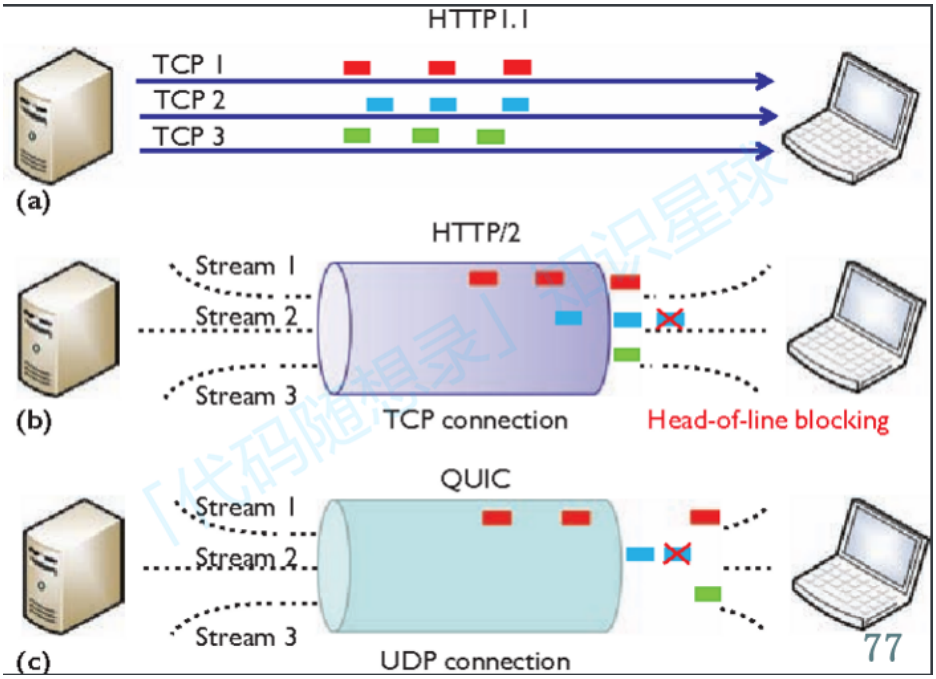
UDP是一个简单的、不可靠的传输协议，而且UDP包之间是无序的，也没有依赖关系。UDP也不需要连接。

HTTP3基于UDP协议在 应用层 实现了`QUIC`协议，它有类似TCP的连接管理、拥塞窗口、流量控制的网络特性，相当于将不可靠的UDP协议变成可靠的了，无需担心数据包丢包的问题。

1、无队头阻塞

HTTP2:只要某个流中的数据包丢失了，其他流也会因此受影响。

HTTP3:流与流（Stream)之间不影响。

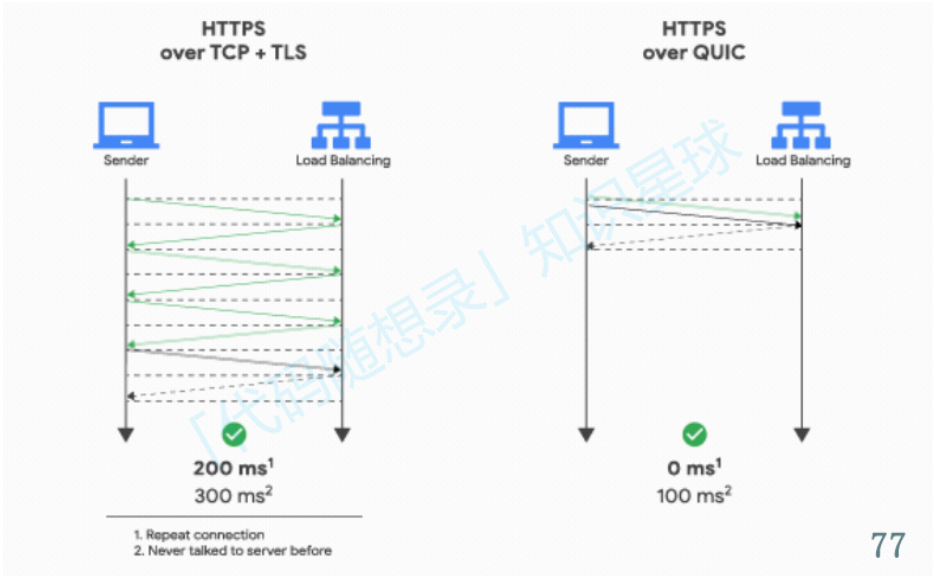


2、更快的连接建立

对于 HTTP/1 和 HTTP/2 协议，TCP 和 TLS 是分层的，分别属于内核实现的传输层、openssl 库实现的表示层，因此它们难以合并在一起，需要分批次来握手，先 TCP 握手，再 TLS 握手。

HTTP/3 在传输数据前虽然需要 QUIC 协议握手，这个握手过程只需要 1 RTT，握手的目的是为确认双方的「连接 ID」，连接迁移就是基于连接 ID 实现的。

但是 HTTP/3 的 QUIC 协议并不是与 TLS 分层，而是QUIC 内部包含了 TLS，它在自己的帧会携带 TLS 里的“记录”，再加上 QUIC 使用的是 TLS1.3，因此仅需 1 个 RTT 就可以「同时」完成建立连接与密钥协商，甚至在第二次连接的时候，应用数据包可以和 QUIC 握手信息（连接信息 + TLS 信息）一起发送，达到 0-RTT 的效果。



3、连接迁移

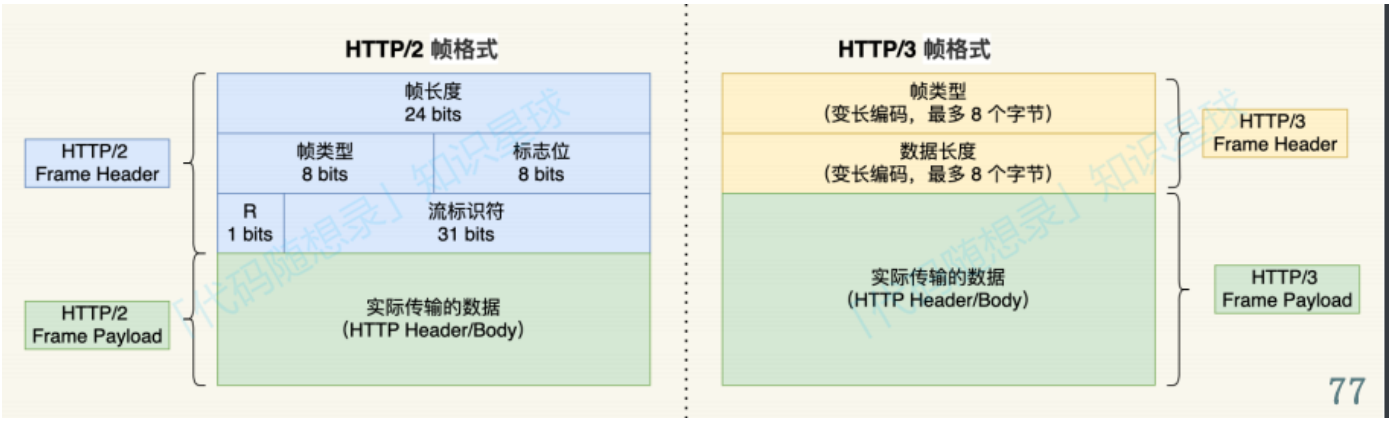
在前面我们提到，基于 TCP 传输协议的 HTTP 协议，由于是通过四元组（源 IP、源端口、目的 IP、目的端口）确定一条 TCP 连接，那么当移动设备的网络从 4G 切换到 WIFI 时，意味着 IP 地址变化了，那么就必须要断开连接，然后重新建立连接，而建立连接的过程包含 TCP 三次握手和 TLS 四次握手的时延，以及 TCP 慢启动的减速过程，给用户的感觉就是网络突然卡顿了一下，因此连接的迁移成本是很高的。

而 QUIC 协议没有用四元组的方式来“绑定”连接，而是通过**连接 ID**来标记通信的两个端点，客户端和服务器可以各自选择一组 ID 来标记自己，因此即使移动设备的网络变化后，导致 IP 地址变化了，只要仍保有上下文信息（比如连接 ID、TLS 密钥等），就可以“无缝”地复用原连接，消除重连的成本，没有丝毫卡顿感，达到了**连接迁移**的功能。

HTTP/3协议

HTTP/3 同 HTTP/2 一样采用二进制帧的结构，不同的地方在于 HTTP/2 的二进制帧里需要定义 Stream，而 HTTP/3 自身不需要再定义 Stream，直接使用 QUIC 里的 Stream，于是 HTTP/3 的帧的结构也变简单了。

1、对比图如下：



2、头部压缩算法：

HTTP2:HPACK
HTTP3:QPACK

3、静态表：

HTTP2:静态表61项
HTTP3:静态表91项

4、动态表：

HTTP2：动态表是具有时序性的，如果首次出现的请求发生了丢包，后续的收到请求，对方就无法解码出HPACK头部，因为对方还没建立好动态表，因此后续的请求解码会阻塞到首次请求中丢失的数据包重传过来。

HTTP3的QPACK解决了上述问题，具体方案如下：

QUIC 会有两个特殊的单向流，所谓的单项流只有一端可以发送消息，双向则指两端都可以发送消息，传输 HTTP 消息时用的是双向流，这两个单向流的用法：

- 一个叫 QPACK Encoder Stream， 用于将一个字典（key-value）传递给对方，比如面对不属于静态表的 HTTP 请求头部，客户端可以通过这个 Stream 发送字典；
- 一个叫 QPACK Decoder Stream，用于响应对方，告诉它刚发的字典已经更新到自己的本地动态表了，后续就可以使用这个字典来编码了。

这两个特殊的单向流是用来**同步双方的动态表**，编码方收到解码方更新确认的通知后，才使用动态表编码 HTTP 头部。

HTTP缓存

对于已经请求过的资源，客户端或代理服务器会将其副本保存在本地存储中，并且在下次请求同一资源时，首先检查缓存中是否存在有效的副本。如果存在有效的缓存，就直接读取本地的数据，不必在通过网络获取服务器的响应了，这就是`HTTP缓存`

HTTP 缓存有两种实现方式，分别是**强制缓存和协商缓存**

强制缓存

强缓存：浏览器判断请求的目标资源是否有效命中强缓存，如果命中，则可以直接从内存中读取目标资源，无需与服务器做任何通讯。

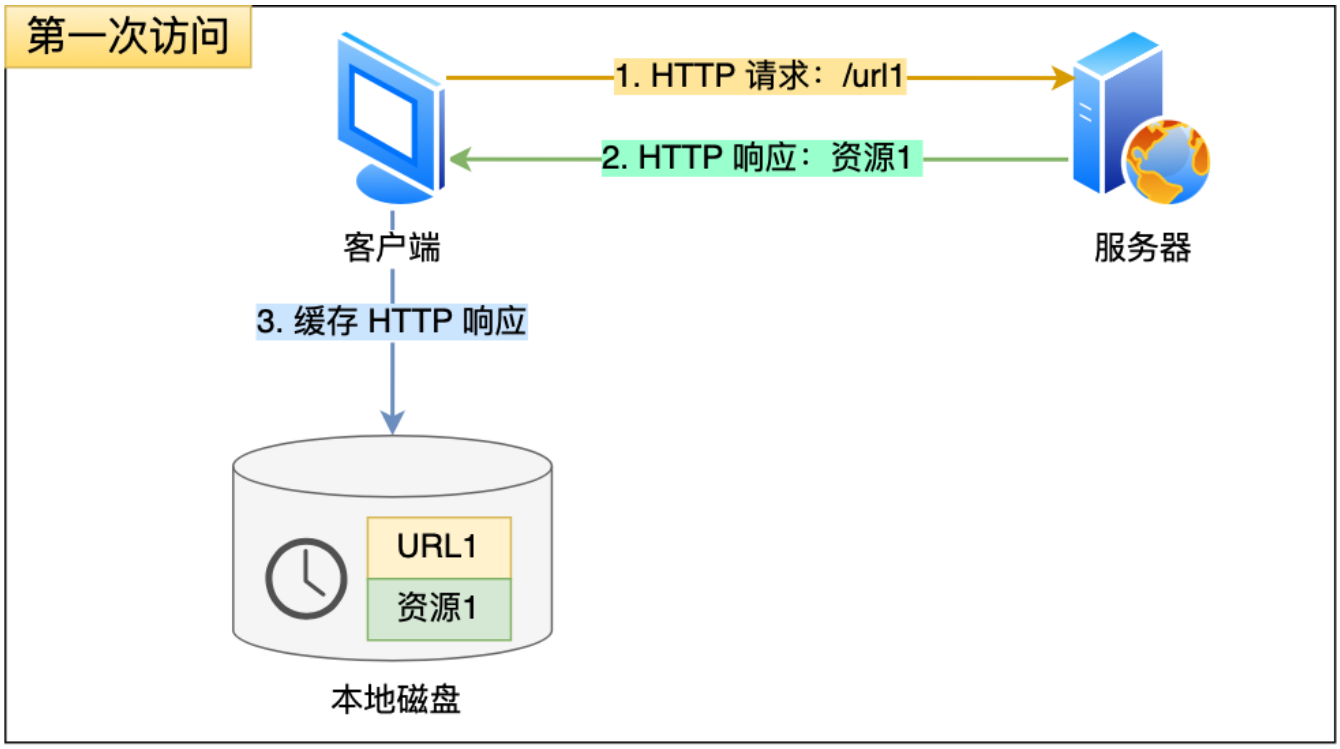
- `Expires强缓存`：设置一个强缓存时间，此时间范围内，从内存中读取缓存并返回，因为`Expires`判断强缓存过期的机制是**获取本地时间戳**，与之前拿到的资源文件中的`Expires`字段的时间做比较。来判断是否需要对服务器发起请求, 所以这里有一个巨大的漏洞：“如果我本地时间不准咋办？”正是因为这个原因，该字段目前已经基本上被废弃了。
- `Cache-Control强缓存`：`http1.1`中增加该字段，只要在资源的响应头上写上需要缓存多久就好了，单位是秒。`Cache-Control:max-age=N`，有**max-age**、**s-maxage**、**no-cache**、**no-store**、**private**、**public**这六个属性。
  - `max-age`决定客户端资源被缓存多久。
  - `s-maxage`决定代理服务器缓存的时长。
  - `no-cache`表示是**强制进行协商缓存。**
  - `no-store`是表示禁止任何缓存策略。
  - `public`表示资源既可以被浏览器缓存也可以被代理服务器缓存。
  - `private`表示资源只能被浏览器缓存，默认为private

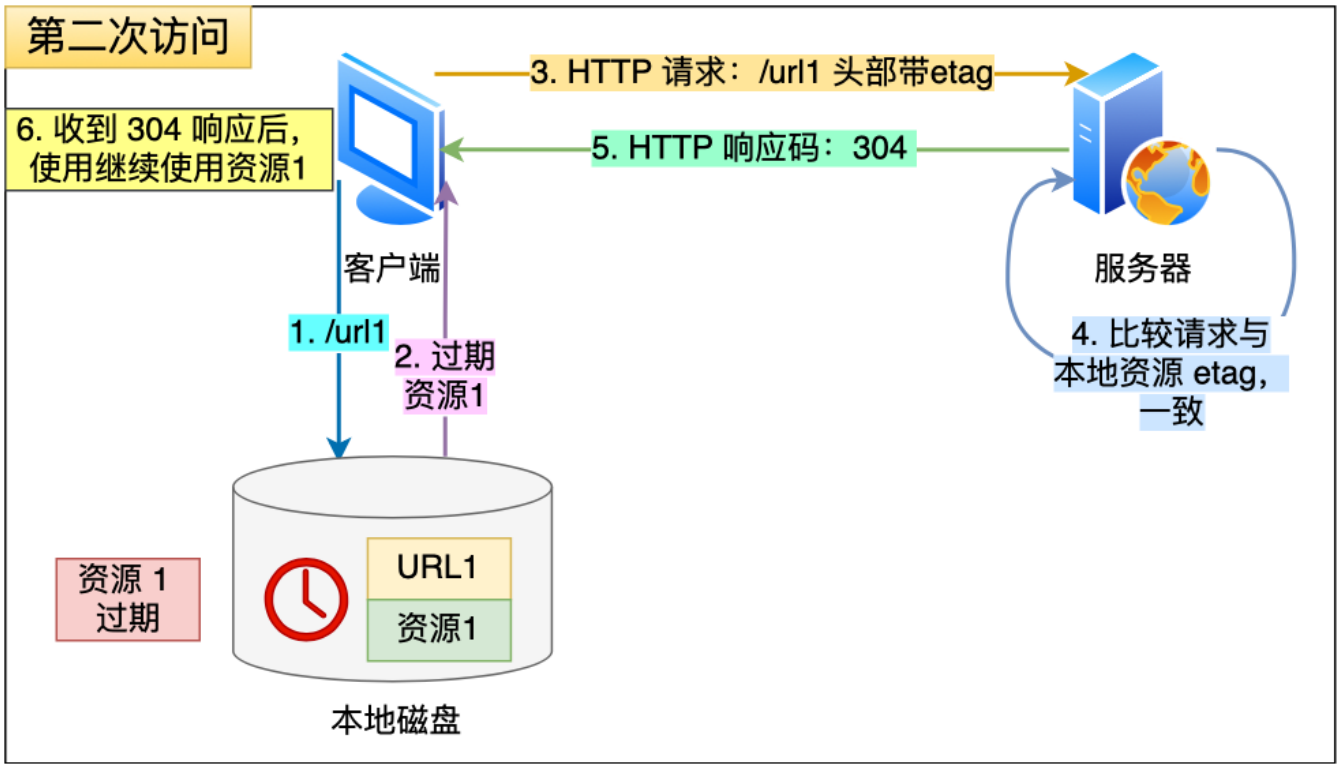
具体的流程如下：

- 当浏览器第一次请求访问服务器资源时，服务器会在返回这个资源的同时，在 Response 头部加上 Cache-Control，Cache-Control 中设置了过期时间大小；
- 浏览器再次请求访问服务器中的该资源时，会先**通过请求资源的时间与 Cache-Control 中设置的过期时间大小，来计算出该资源是否过期**，如果没有，则使用该缓存，否则重新请求服务器；
- 服务器再次收到请求后，会再次更新 Response 头部的 Cache-Control。

协商缓存

当我们在浏览器使用开发者工具的时候，你可能会看到过某些请求的响应码是 `304`，这个是告诉浏览器可以使用本地缓存的资源，通常这种通过服务端告知客户端是否可以使用缓存的方式被称为协商缓存。





1. 基于`Last-Modified`和`If-Modified-Since`的协商缓存

流程：

- 首先需要在服务器端读出文件修改时间，
- 将读出来的修改时间赋给响应头的`Last-Modified`字段。
- 最后设置`Cache-control:no-cache`

- 当客户端读取到`Last-Modified`的时候，会在下次的请求标头中携带一个字段:`If-Modified-Since`，而这个请求头中的`If-Modified-Since`就是服务器第一次修改时候给他的时间
- 之后每次对该资源的请求，都会带上`If-Modified-Since`这个字段，而服务端就需要拿到这个时间并再次读取该资源的修改时间，让他们两个做一个比对来决定是读取缓存还是返回新的资源
- 如果最后修改时间较新（大），说明资源又被改过，则返回最新资源，`HTTP 200 OK`如果最后修改时间较旧（小），说明资源无新修改，响应`HTTP 304`走缓存。

缺点：

- 因为是根据文件修改时间来判断的，所以，在文件内容本身不修改的情况下，依然有可能更新文件修改时间（比如修改文件名再改回来），这样，就有可能文件内容明明没有修改，但是缓存依然失效了。
- 当文件在极短时间内完成修改的时候（比如几百毫秒）。因为文件修改时间记录的最小单位是秒，所以，如果文件在几百毫秒内完成修改的话，文件修改时间不会改变，这样，即使文件内容修改了，依然不会 返回新的文件。

而基于`ETag`的协商缓存则解决了上面的两个问题，相比于`Last-Modified`，`ETag`的优先级更高。

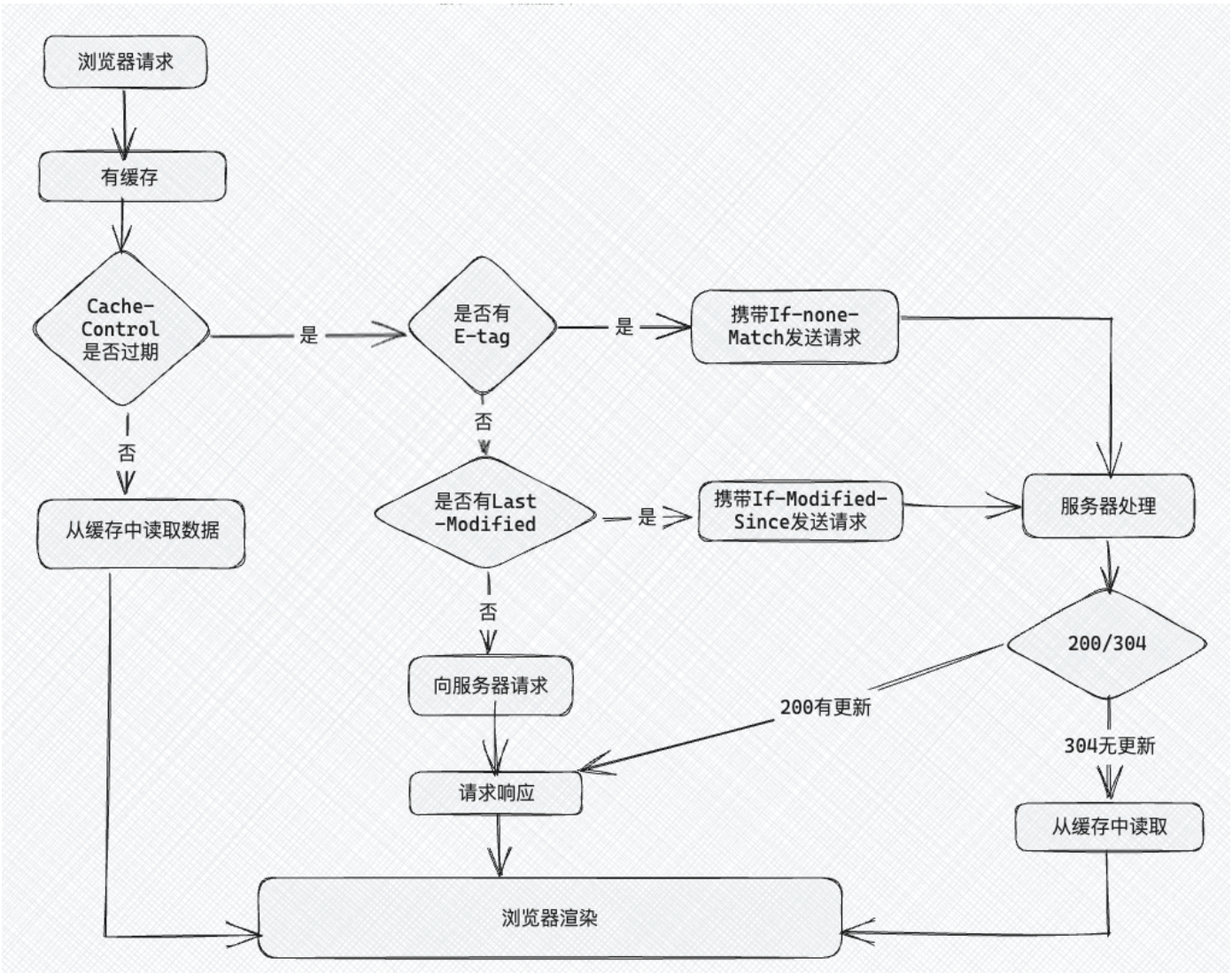
2. **基于`ETag`的协商缓存：**将原先协商缓存的比较**时间戳**的形式修改成了比较**文件指纹（根据文件内容计算出的唯一哈希值）**。

- 第一次请求某资源的时候，服务端读取文件并计算出文件指纹，将文件指纹放在响应头的`Etag`字段中跟资源一起返回给客户端。
- 第二次请求某资源的时候，客户端自动从缓存中读取出上一次服务端返回的`ETag`也就是文件指纹。并赋给请求头的`If-None-Match`字段，让上一次的文件指纹跟随请求一起回到服务端。
- 服务端拿到请求头中的`If-None-Match`字段值（也就是上一次的文件指纹），并再次读取目标资源并生成文件指纹，两个指纹做对比。如果两个文件指纹完全吻合，说明文件没有被改变，则直接返回`304`状态码和一个空的响应体并return。如果两个文件指纹不吻合，则说明文件被更改，那么将新的文件指纹重新存储到响应头的ETag中并返回给客户端

缺点：

- ETag需要计算文件指纹这样意味着，服务端需要更多的计算开销。如果文件尺寸大，数量多，并且计算频繁，那么ETag的计算就会影响服务器的性能。显然，ETag在这样的场景下就不是很适合。
- ETag有强验证和弱验证，所谓将强验证，ETag生成的哈希码深入到每个字节。哪怕文件中只有一个字节改变了，也会生成不同的哈希值，它可以保证文件内容绝对的不变。但是，强验证非常消耗计算量。ETag还有一个弱验证，弱验证是提取文件的部分属性来生成哈希值。因为不必精确到每个字节，所以他的整体速度会比强验证快，但是准确率不高。会降低协商缓存的有效性。

> **协商缓存这两个字段都需要配合强制缓存中 Cache-Control 字段来使用，只有在未能命中强制缓存的时候，才能发起带有协商缓存字段的请求。**



HTTPS

HTTPS协议

协议（protocol）：通信双方需要遵守的 统一规则。

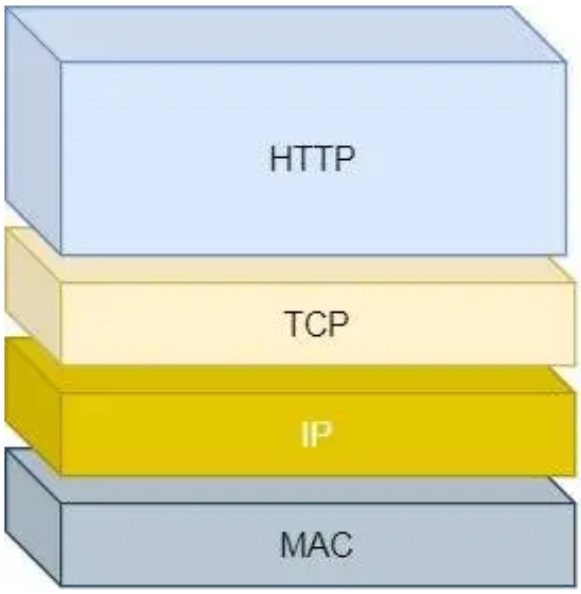
HTTP：Hyper Text Transfer Protocol：超文本传输协议

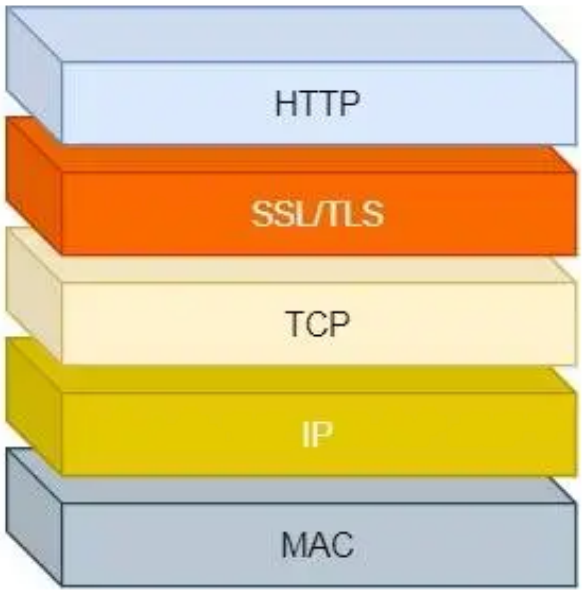
HTTPS：Hyper Text Transfer Protocol Secure：超文本安全传输协议

SSL：Secure Socket Layer 安全套接字

TSL：Transport Layer Security 安全传输层协议

HTTPS = HTTP+SSL/TSL





HTTPS的特点

1. 特点

- 信息加密：采用对称加密+非对称加密的混合加密的方式，对传输的数据加密，实现信息的机密性，解决了窃听的风险。
- 校验机制：用摘要算法为数据生成独一无二的「指纹」校验码，指纹用来校验数据的完整性，解决了被篡改的风险。
- 身份证书：将服务端的公钥放入到CA数字证书中，解决了服务端被冒充的风险。

2. 优点

- 在数据传输过程中，使用秘钥加密，安全性更高
- 可认证用户和服务器，确保数据发送到正确的用户和服务器

3. 缺点

- 握手阶段延时较高：在会话前还需进行SSL握手
- 部署成本高：需要购买CA证书；需要加解密计算，占用CPU资源，需要服务器配置或数目高

HTTPS和HTTP的区别

- **HTTP**： 以**明文**的方式在网络中传输数据，HTTPS 解决了HTTP 不安全的缺陷，在 TCP 和 HTTP 网络层之间加入了 `SSL/TLS` 安全协议，使得报文能够**加密传输**。
- HTTPS 在 TCP 三次握手之后，还需进行 `SSL/TLS` 的握手过程，才可进入加密报文传输。
- HTTP 的端口号是 80，HTTPS 的端口号是 443。
- HTTPS 协议需要向 `CA（证书权威机构）` 申请数字证书，来保证服务器的身份是可信的。

对称加密

对称加密也称为私钥加密，使用相同的密钥来进行加密和解密。

- 在加密过程中，明文数据通过应用特定的算法和密钥进行加密，生成密文数据。解密过程则是将密文数据应用同样的算法和密钥进行解密，恢复为明文数据。
- 由于加密和解密都使用相同的密钥，因此对称加密算法的速度通常较快，但密钥的安全性很重要。如果密钥泄漏，攻击者可以轻易地解密数据。

非对称加密

- 非对称加密也称为公钥加密，使用一对不同但相关的密钥：公钥和私钥。
- 公钥用于加密数据，私钥用于解密数据。如果使用公钥加密数据，只有拥有相应私钥的人才能解密数据；如果使用私钥加密数据，可以使用相应公钥解密。
- 除了加密和解密，非对称加密还用于【数字签名】，可以验证消息的来源和完整性。

混合加密

HTTPS采用**对称加密和非对称加密结合的【混合加密】**方式，保证信息的**机密性**，解决了窃听的风险。

通信建立前：

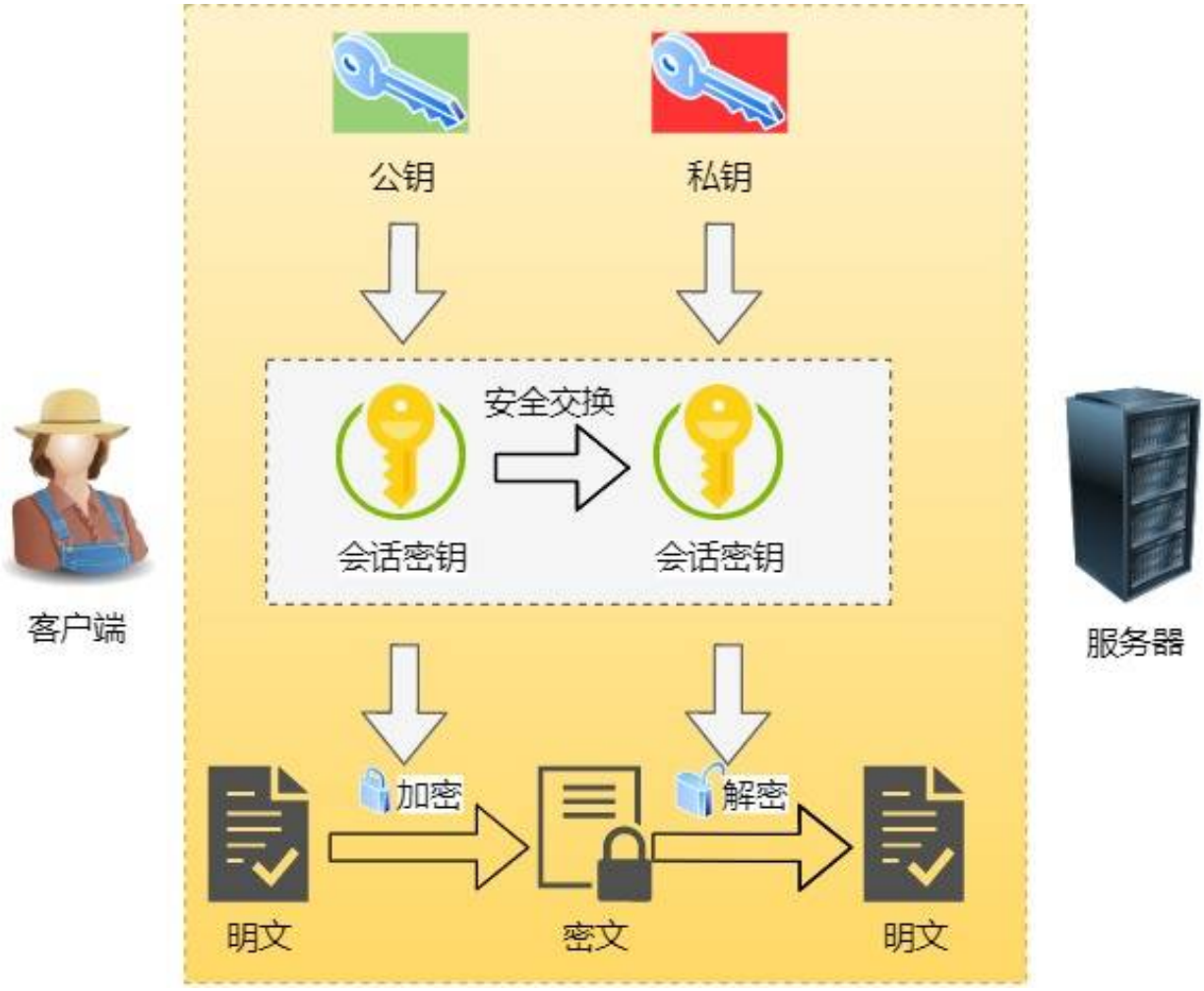
采用非对称加密的方式交换【会话密钥】，后续不再使用非对称加密;

通信过程中：

全部使用对称加密的【会话密钥】方式，加密明文数据。

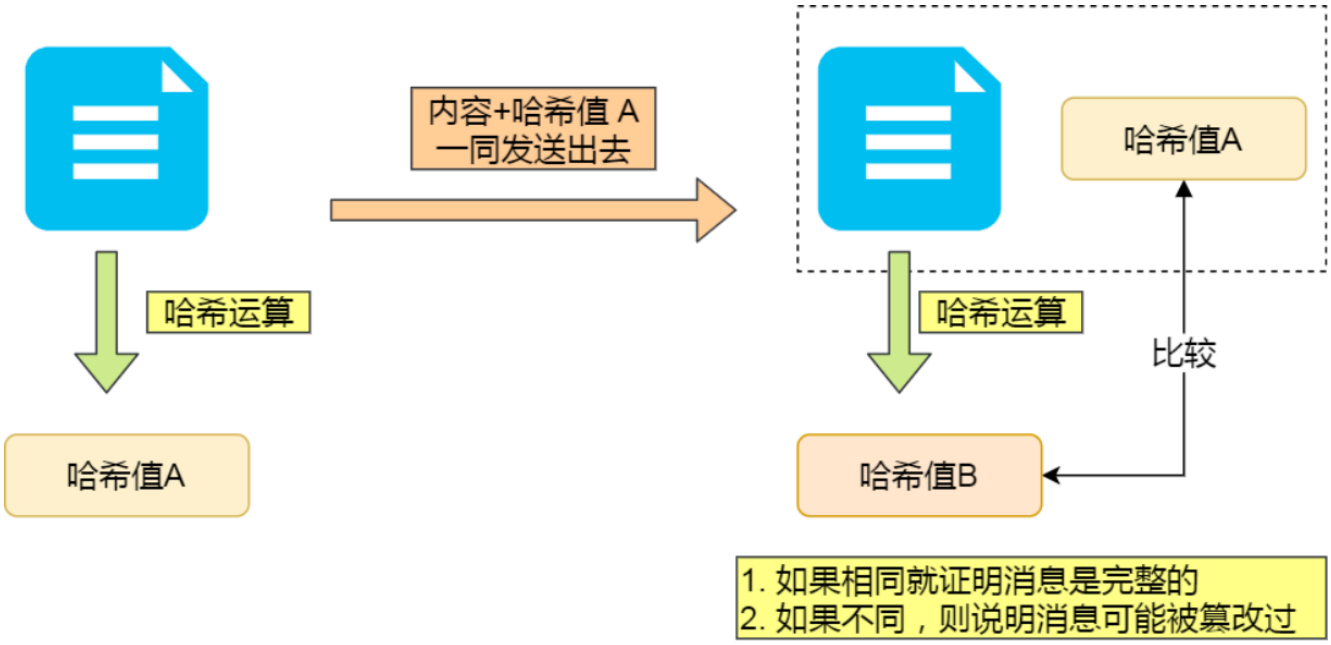
采用「混合加密」的方式的原因：

- 对称加密：只使用一个密钥，运算速度快，密钥必须保密，无法做到安全的密钥交换;
- 非对称加密：使用两个密钥，公钥可以任意分发而私钥保密，解决密钥交换问题，但速度慢。



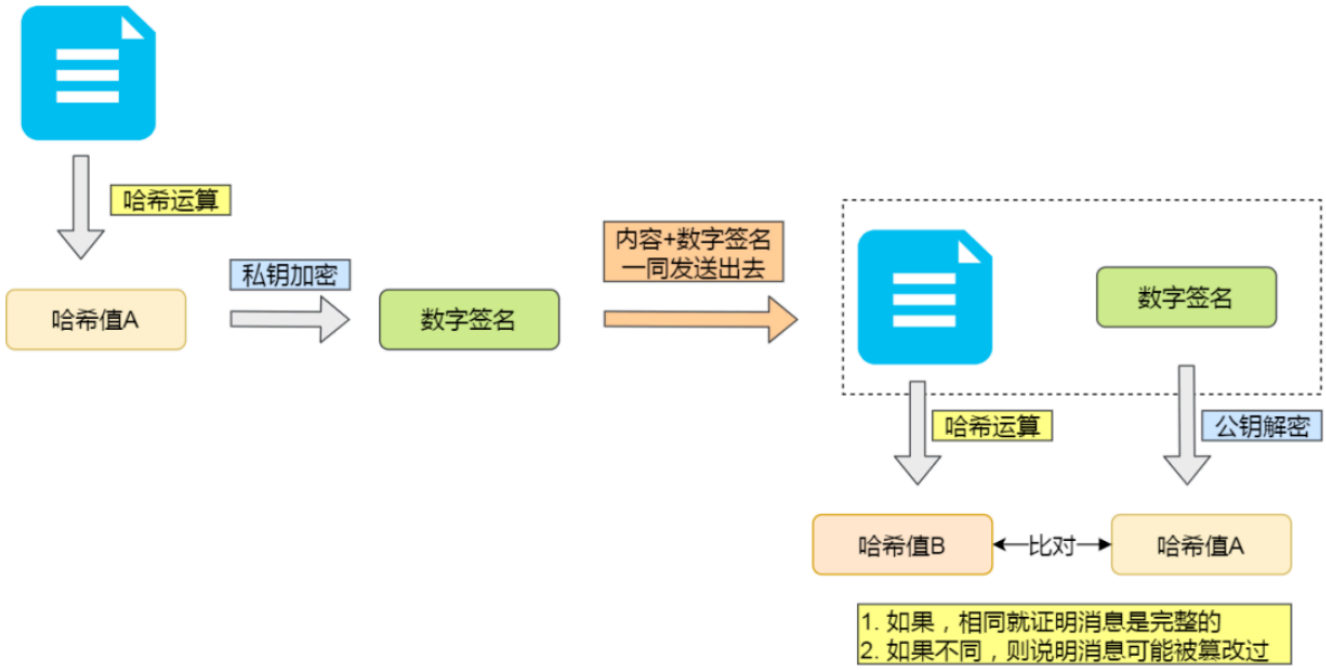
摘要算法+数字签名

为了保证传输的内容不被修改，可以将传输的内容计算出一个【指纹】，对方收到后，也把接收的内容计算出一个【指纹】，然后进行对比，如果【指纹】相同，则说明内容没有被篡改，常常会使用摘要算法（哈希函数）来计算出内容的哈希值，通过摘要算法可以生成数据的唯一标识，从而验证数据的完整性。



但是摘要算法只能保证内容不被修改，不能保证发送者的身份，为了避免这种情况，计算机里会用**非对称加密算法**来解决，共有两个密钥：公钥用于验证签名，私钥用于生成签名。私钥是由服务端保管，然后服务端会向客户端颁发对应的公钥。如果客户端收到的信息，能被公钥解密，就说明该消息是由服务器发送的。

1. **生成数字签名**：
    - 发送者使用私钥对消息的摘要（通常是通过哈希函数计算得到）进行加密，生成数字签名。
    - 数字签名是消息的哈希值经过私钥加密的结果。
2. **发送消息和数字签名**：
    - 发送者将原始消息和生成的数字签名一起发送给接收者。
3. **验证数字签名**：
    - 接收者收到消息和数字签名后，使用发送者的公钥对数字签名进行解密，得到摘要值。
    - 接收者再次计算收到的消息的摘要（使用相同的哈希函数），将其与解密得到的摘要值进行比较。
    - 如果两个摘要值相同，说明消息未被篡改过，数字签名有效，消息来源可信。



数字证书

根据上面的介绍，我们已经知道可以通过摘要算法保证内容不被修改，以及通过私钥和公钥保证消息来源的可靠性，但是却缺少身份验证的环节，如果此时在客户端和服务器之间有一个中间人，中间人把原本双方通信的公钥篡改成自己的公钥，这又该怎么办呢？这就需要使用到数字证书。

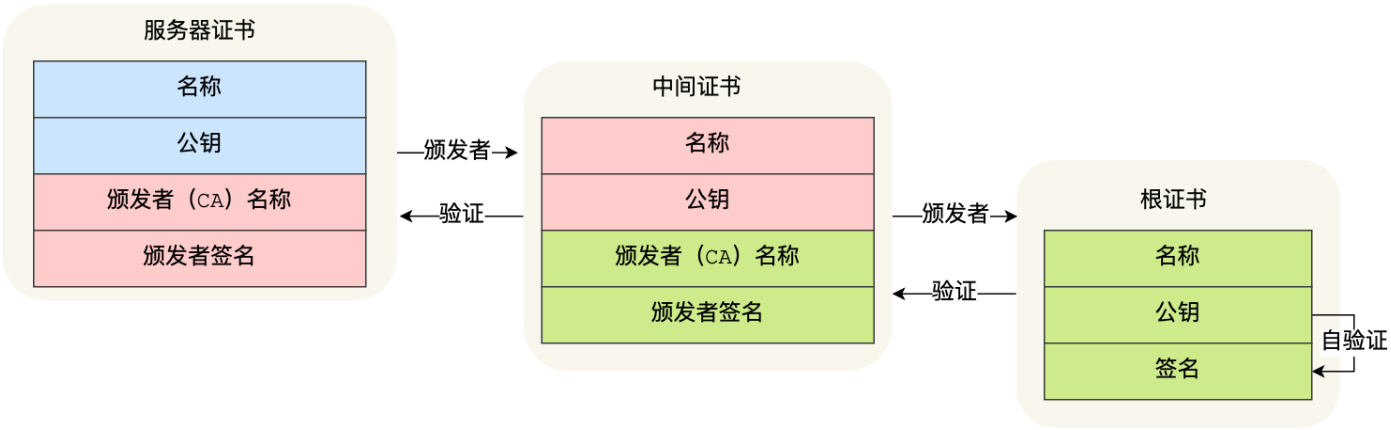
1. **密钥生成**：
    - 首先，实体（例如服务器、个人或组织）需要生成一对密钥：公钥和私钥。
    - 公钥是用于加密和验证的，可以被公开分享。
    - 私钥用于解密和签名，必须保密，只有持有者知道。
2. **证书请求（CSR - Certificate Signing Request）**：
    - 实体生成一个证书请求，其中包含公钥、实体信息（如名称、电子邮件等）和签名。
    - CSR是一个包含有关实体信息的文本块，可以被发送到`数字证书颁发机构（CA）`以获取数字证书。
3. **证书颁发**：
    - 实体将证书请求发送给数字证书颁发机构（CA）。
    - CA会验证请求者的身份，然后使用自己的私钥对请求中的信息进行签名，生成数字证书。
    - 数字证书包括公钥、实体信息、CA的信息和签名等内容。
4. **证书验证**：
    - 当实体收到数字证书时，它可以使用CA的公钥验证证书的签名，确保证书未被篡改且由合法的CA签发。
    - 接收者可以检查证书中的实体信息以及CA的信息，确保证书的合法性。
5. **数字证书使用**：
    - 接收者可以使用数字证书中的公钥来加密数据，然后发送给证书的持有者。
    - 持有者使用私钥解密数据，保护数据的机密性。
    - 持有者还可以使用私钥生成数字签名，接收者使用公钥验证签名，验证数据的来源和完整性。

证书信任链

证书信任链（Certificate Trust Chain）是数字证书的验证过程中的一部分，确保数字证书是由可信的证书颁发机构（CA）签发的，并最终连接到根证书，形成一条链条。

1. **根证书**：信任链的顶端是根证书（Root Certificate），也称为根CA证书。根证书是数字证书体系中的根基，它由操作系统或浏览器内置，作为信任的起点。
2. **中间证书颁发机构（Intermediate CA）**：在信任链中，根证书下方可能有一个或多个中间CA，也称为中间证书颁发机构。这些中间CA的数字证书由根CA签发，它们作为信任链中的中继，用于签发其他证书。
3. **服务器证书**：在最底层是服务器的数字证书，它是由中间CA签发的，包含了服务器的公钥和相关信息。服务器证书需要在客户端验证的过程中被验证。

证书验证过程中，客户端会依次验证服务器证书、中间CA的证书、根CA的证书。如果整个链条中的所有证书都可以被成功验证，那么数字证书被视为合法和可信的。相反，如果任何一个证书无法验证，整个信任链就会被打断，数字证书将被视为不合法或不可信的。



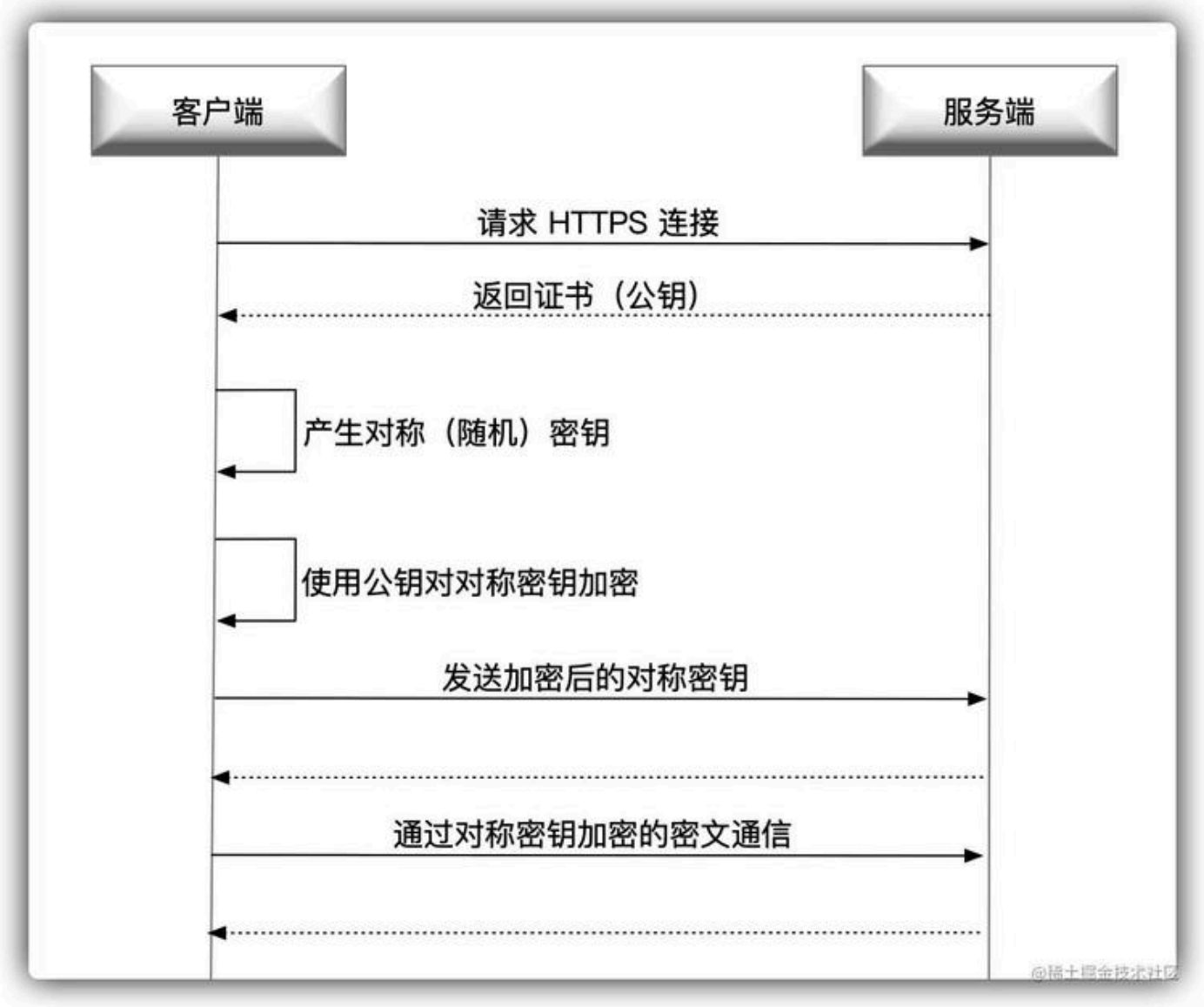
HTTPS是怎么建立连接的

SSL/TLS 协议基本流程：

- 客户端向服务器索要并验证服务器的公钥。
- 双方协商生产「会话秘钥」。
- 双方采用「会话秘钥」进行加密通信。

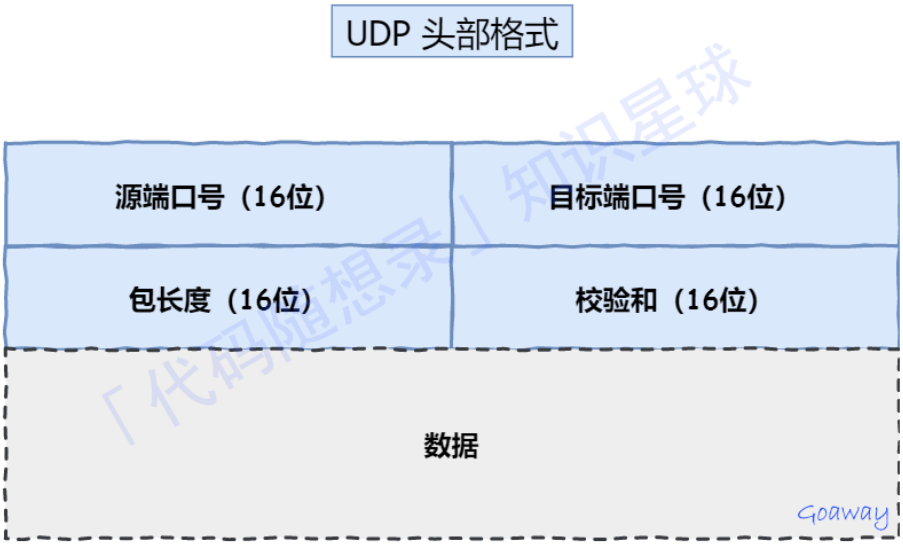
具体流程如下：

1. 首先，客户端向服务器端发送加密通信请求，也就是 `ClientHello` 请求，请求与服务端建立连接。
2. 服务端产生一**对公私钥**，然后将自己的公钥发送给CA机构，CA机构也有一对公私钥，然后CA机构使用自己的私钥将服务端发送过来的公钥进行加密，产生一个CA数字证书。
3. 服务端响应客户端的请求，也就是 `SeverHello`， 将CA机构生成的数字证书发送给客户端。
4. 客户端将服务端发送过来的数字证书进行解析(因为浏览器产商跟CA机构有合作，所以浏览器中已经保存了大部分CA机构的密钥，用于对服务端发送过来的数字证书进行解密)，验证这个CA数字证书是否合法，如果不合法，会发送一个警告。如果合法，从数字证书中取出服务端生成的公钥。
5. 客户端取出公钥并生成一个随机码key（其实就是对称加密中的密钥）
6. 客户端将加密后的随机码key发送给服务端，作为接下来的对称加密的密钥
7. 服务端接收到随机码key后，使用自己的私钥对它进行解密，然后获得到随机码key。
8. 服务端使用随机码key对传输的数据进行加密，在传输加密后的内容给客户端
9. 客户端使用自己生成的随机码key解密服务端发送过来的数据，之后，客户端和服务端通过对称加密传输数据，随机码Key作为传输的密钥。



TCP与UDP头部格式

UDP



目标端口和源端口：

主要是告诉 UDP 协议应该把报文发给哪个进程。

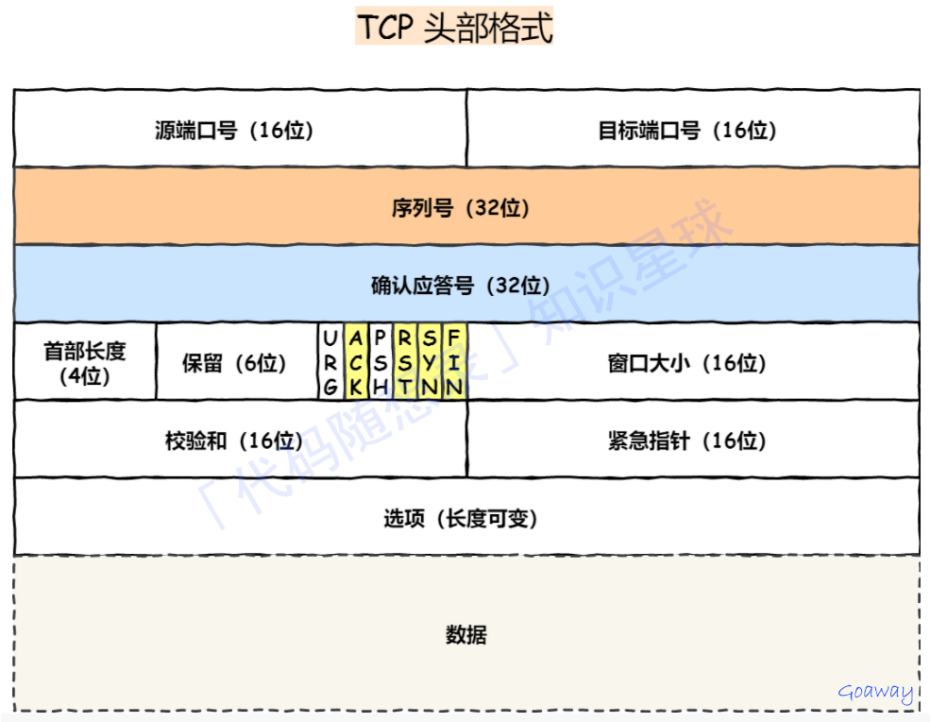
包长度：

该字段保存了 UDP 首部的长度跟数据的长度之和。

校验和：

校验和是为了提供可靠的 UDP 首部和数据而设计，接收方使用检验和来检查该报文段中是否出现差错。

TCP



源端口号和目的端口号：

用于多路复用/分解来自或送到上层应用的数据。告诉主机报文段来自哪里，传给哪个上层协议或应用程序。

序列号：

该报文段首字节的字节流编号，用来解决网络包乱序问题。确认应答号：对发送来的 TCP 报文段的响应，值是收到的 TCP 报文段的序号值加1，用来解决不丢包的问题。序列号和确认应答号都用于实现可靠数据传输。

首部长度：

标识 TCP 头部有多少字节，最长 60。

窗口大小：

接收窗口，告诉对方本端TCP缓冲区还有多少空间可以接收数据，用来做流量控制。

标志字段：

ACK：用于指示确认应答号值是否有效，置1表示包含一个对已成功接收报文段的确认；

RST：用于重置一个已经混乱的连接，或拒绝一个无效的数据段或者连接请求；

SYN：用于连接建立过程，请求建立一个连接；

FIN：用于断开连接，表示发送方没有数据要传输了。

检验和：

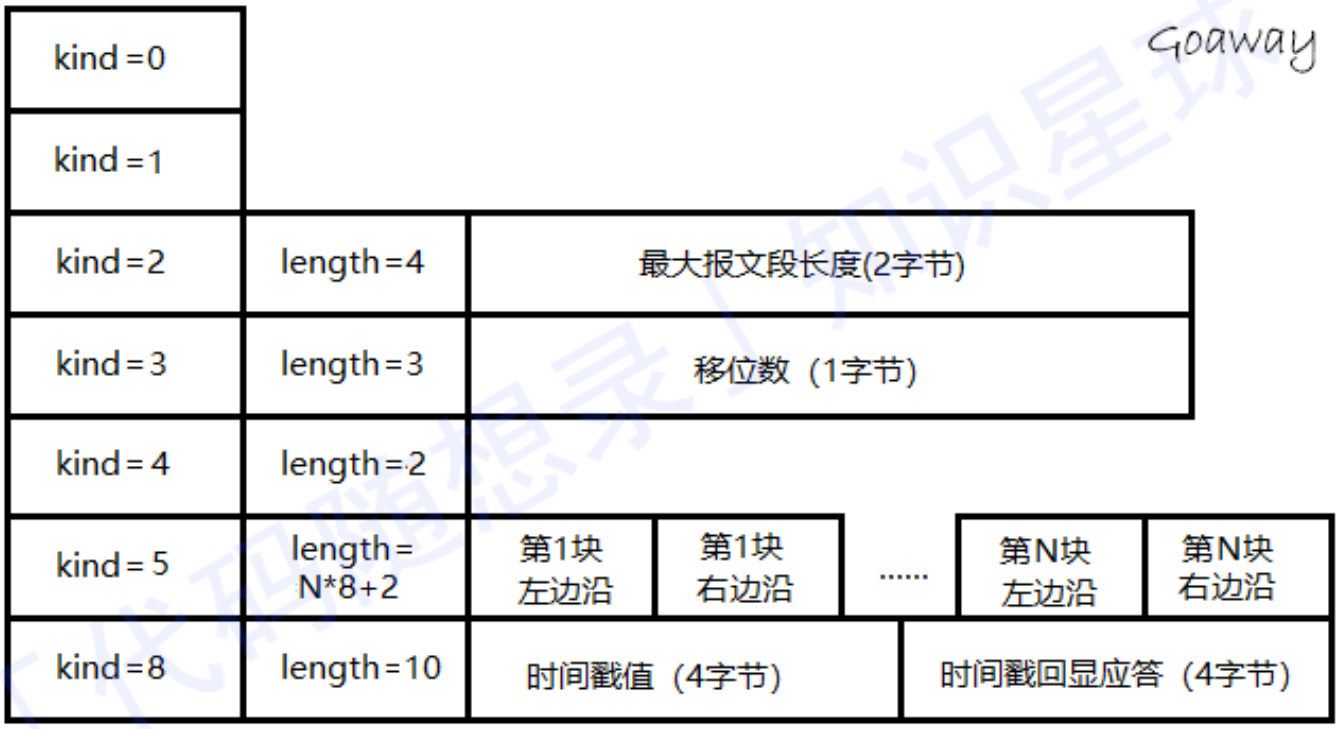
接收方使用检验和来检查该报文段(头部+数据)中是否出现差错（CRC算法），同 UDP。

列举TCP选项

TCP头部的最后一个选项字段（options）是可变长的可选信息。这部分最多包含40字节，因为TCP头部最长是60字节（其中还包含前面讨论的20字节的固定部分）。

- 选项的第一个字段kind说明选项的类型，有的TCP选项没有后面两个字段，仅包含1字节的kind字段。
- 第二个字段length（如果有的话）指定该选项的总长度。该长度包括kind字段和length字段占据的2字节。
- 第三个字段info（如果有的话）是选项的具体信息。

常见的TCP选项有7种：



1、kind=0，选项表结束（EOP）选项。

一个报文段仅用一次。放在末尾用于填充，用途是说明：首部已经没有更多的消息，应用数据在下一个32位字开始处。

2、kind=1，空操作（NOP）选项。

没有特殊含义，一般用于将TCP选项的总长度填充为4字节的整数倍。

3、kind=2，最大报文段长度（MSS）选项。

TCP连接初始化时，通信双方使用该选项来协商最大报文段长度。TCP模块通常将MSS设置为（MTU-40）字节（减掉的这40字节包括20字节的TCP头部和20字节的IP头部）。

这样携带TCP报文段的IP数据报的长度就不会超过MTU（假设TCP头部和IP头部都不包含选项字段，并且这也是一般情况），从而避免本机发生IP分片。对以太网而言，MSS值是1460（1500-40）字节。

4、kind=3，窗口扩大因子选项。

TCP连接初始化时，**通信双方使用该选项来协商接收窗口的扩大因子**。在TCP的头部中，接收窗口大小是用16位表示的，故最大为65535字节，但实际上TCP模块允许的接收窗口大小远不止这个数（为了提高TCP通信的吞吐量）。窗口扩大因子解决了这个问题。

假设 TCP 头部中的接收通告窗口大小是 N，窗口扩大因子（移位数）是 M，那么 TCP 报文段的实际接收通告窗口大小是 N*2M，或者说 N 左移 M 位。注意，M的取值范围是 0～14。

和 MSS 选项一样，窗口扩大因子选项只能出现在同步报文段中，否则将被忽略。但同步报文段本身不执行窗口扩大操作，即同步报文段头部的接收窗口大小就是该 TCP 报文段的实际接收窗口大小。当连接建立好之后，每个数据传输方向的窗口扩大因子就固定不变了。

5、kind=4，选择性确认（Selective Acknowledgment，SACK）选项。

TCP 通信时，如果某个 TCP 报文段丢失，则 TCP 会重传最后被确认的 TCP 报文段后续的所有报文段，这样原先已经正确传输的 TCP 报文段也可能重复发送，从而降低了 TCP 性能。

SACK 技术使 TCP 只重新发送丢失的 TCP 报文段，而不用发送所有未被确认的 TCP 报文段。选择性确认选项用在连接初始化时，表示是否支持 SACK 技术。

6、kind=5，SACK实际工作的选项。

该选项的参数告诉发送方本端已经收到并缓存的不连续的数据块，从而让发送端可以据此检查并重发丢失的数据块。

每个块边沿（edge of block）参数包含一个4字节的序号。其中块左边沿表示不连续块的第一个数据的序号，而块右边沿则表示不连续块的最后一个数据的序号的下一个序号。这样一对参数（块左边沿和块右边沿）之间的数据是没有收到的。因为一个块信息占用8字节，所以 TCP 头部选项中实际上最多可以包含4个这样的不连续数据块（考虑选项类型和长度占用的2字节）。

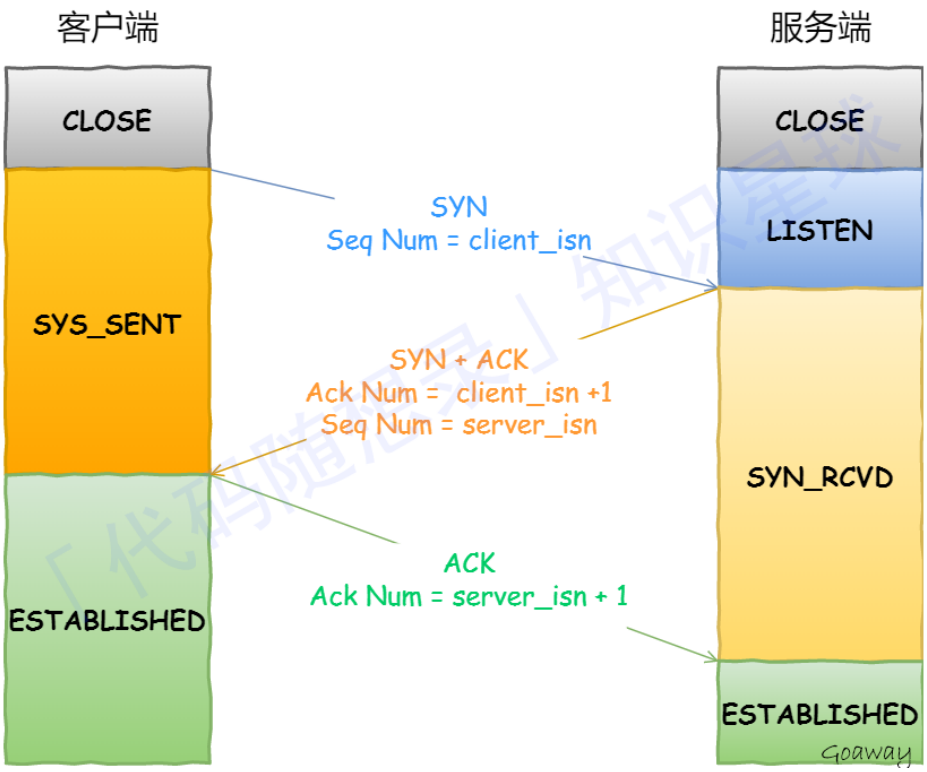
7、kind=8，时间戳选项。

该选项提供了较为准确的计算通信双方之间的回路时间（Round Trip Time，RTT）的方法，为TCP流量控制提供信息。

TCP三次握手

三次握手的过程

TCP 是面向连接的协议，所以使用 TCP 前必须先建立连接，而建立连接是通过三次握手来进行的。



1、第一次握手：SYN报文：

客户端随机初始化序列号`client_isn`，放进TCP首部序列号段，然后把SYN置1。把SYN报文发送给服务端，表示发起连接，之后客户端处于`SYN-SENT`状态。

2、第二次握手：SYN+ACK报文：

服务端收到客户端的SYN报文之后，会把自己随机初始化的序号`server_isn`放进TCP首部序列号段，**「确认应答号」**填入`client_isn + 1`，把SYN和ACK标志位置为1。把SYN+ACK报文发送给客户端，然后进入`SYN-RCVD`状态,表示服务器接受了客户端的请求，并希望建立连接。

3、第三次握手：ACK报文：

客户端收到服务端报文后，还要向服务端回应**最后一个应答报文**。首先该应答报文 TCP 首部 ACK 标志位置为 1，其次**「确认应答号」**字段填入`server_isn + 1`，最后把报文发送给服务端，**这次报文可以携带客户到服务器的数据**，之后客户端处于`ESTABLISHED`状态, 表示客户端已经准备好与服务器进行数据传输。

服务器收到客户端的应答报文后，也进入`ESTABLISHED`状态。

此时，TCP 连接已经建立起来，通信双方可以开始进行数据传输。

> ✏️ 第三次握手是可以携带数据的，但是前两次握手是不可以携带数据的。

为什么需要三次握手?

> 三次握手才能保证双方具有接收和发送的能力

总结：

1. 三次握手才可以阻止重复历史连接的初始化(主因)
2. 三次握手才可以同步双方的初始序列号
3. 三次握手才可以避免资源浪费

解释：

1. 阻止重复历史连接的初始化（主因）

1. 当因为网络阻塞原因，客户端向服务器发送了两次SYN报文
2. 旧的SYN报文先到达服务端，服务端回一个ACK+SYN报文
3. 客户端收到后可以根据自身的上下文，判断这是一个历史连接（序列号过期或超时），那么客户端就会发送 RST 报文给服务端，表示中止这一次连接。
4. 服务器收到RST报文，会释放连接
5. 新的SYN报文抵达之后，客户端和服务器之间进行正常的三次握手

如果只是两次握手，服务端在收到SYN报文之后，就进入到`ESTABLISHED`状态，服务器端并不知道这次是历史连接，直接与客户端建立连接并向客户端发送数据（资源浪费），但是客户端会判定这次连接是历史连接，从而发送RST报文来断开连接。

所以要想让服务器发送数据前，阻止掉历史连接，就需要三次握手。

2、同步双方的初始序列号

TCP 协议的通信双方， **都必须维护一个「序列号」**， 序列号是可靠传输的一个关键因素。

- 接收端可以去除重复数据。
- 接收端可以按照序列号顺序接收。
- 标识发送的数据包，哪些已经被收到。

过程

1. 客户端发送第一个报文，携带客户端初始序列号的SYN报文。
2. 服务器发送第二个报文，携带服务器初始序列号的ACK + SYN的应答报文，表示收到客户端的SYN报文。
3. 客户端发送第三个报文，携带服务器的ACK应答报文。

这样一来一回，才能确保双方的初始序列号能被可靠的同步。

两次握手只保证了一方的初始序列号能被对方成功接收，没办法保证双方的初始序列号都能被确认接收。四次握手也能保证双方的初始化序号同步，但是可以省略成三次。

3、避免资源浪费。

1. 只有两次握手时，如果客户端的SYN请求连接在网络中阻塞，客户端没有收到服务端的ACK报文，会重新发送SYN。
2. **由于没有第三次握手，服务器不清楚客户端是否收到了自己发送的建立连接的 ACK 确认信号，所以每收到一个 SYN 就只能先主动建立一个连接， 建立多个冗余的无效链接，造成不必要的资源浪费。**

所以通过TCP三次握手，能**能防止历史连接的建立，能减少双方不必要的资源开销，能帮助双方同步初始化序列号**

- 「两次握手」：无法防止历史连接的建立，会造成双方资源的浪费，也无法可靠的同步双方序列号；
- 「四次握手」：三次握手就已经理论上最少可靠连接建立，所以不需要使用更多的通信次数。

什么是半连接队列

半连接队列（SYN队列）用于存放已经发送了 SYN（同步）包，但还未完成三次握手的连接：服务器第一次收到客户端的 SYN 之后，就会处于 `SYN_RCVD`状态，此时双方还没有完全建立其连接，服务器会把此种状态下请求连接放在一个**队列**里，我们把这种队列称之为**半连接队列**。

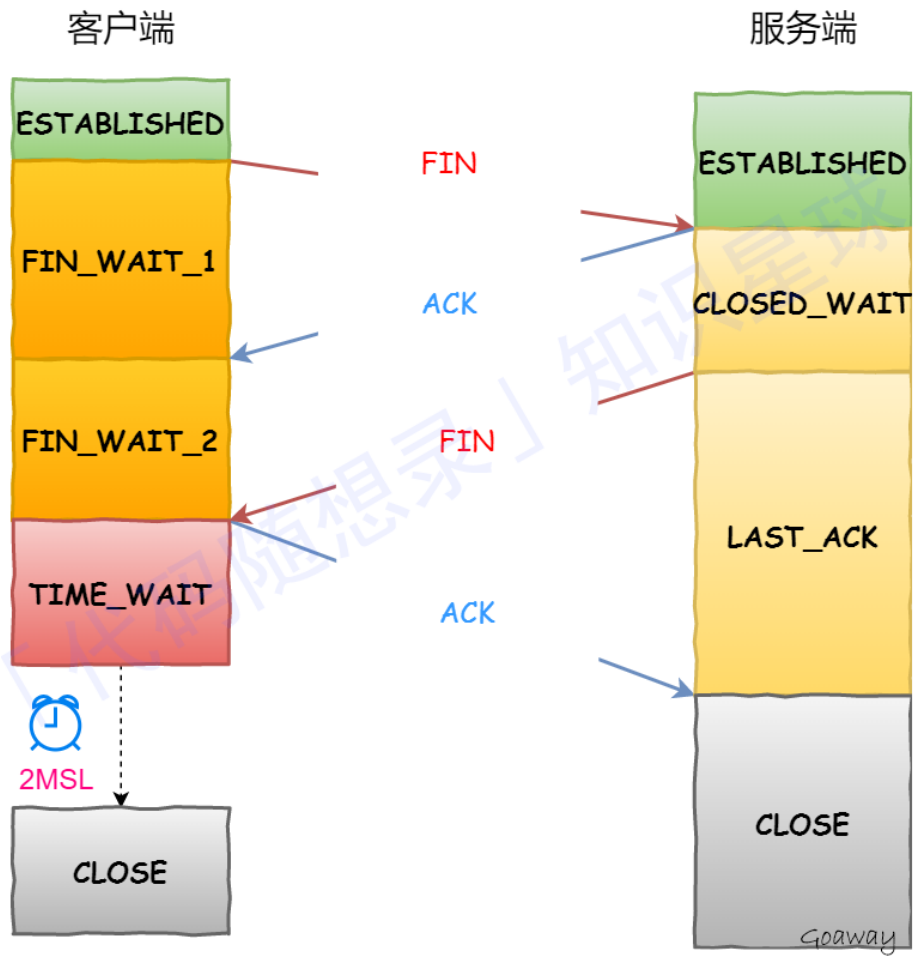
全连接队列（Accept队列）用于存放已经完成三次握手，处于完全建立连接状态的连接。

什么是SYN攻击

在 SYN 攻击中，攻击者发送大量伪造的 SYN 请求到目标服务器，但不完成后续的握手过程，从而让服务器一直等待确认，消耗服务器的资源（如半连接队列和系统资源），当半连接队列满了之后，后续再收到SYN报文就会丢弃，导致无法与客户端之间建立连接。

TCP四次挥手

TCP 断开连接是通过四次挥手来进行的，四次挥手的过程如下图：



四次挥手的过程

在挥手之前，客户端和服务器都处于`ESTABLISHED`状态

1. 第一次挥手：假设客户端打算关闭连接，发送一个TCP首部FIN被置1的`FIN`报文给服务端, 此时客户端处于`FIN_WAIT1`状态
2. 第二次挥手：服务端收到以后，向客户端发送ACK应答报文，且把客户端的序列号值+1作为ACK报文的序列号值，表明已经收到客户端的报文了，此时服务端处于`CLOSE_WAIT`状态
3. 第三次挥手：等待服务端处理完数据后，向客户端发送FIN报文。此时服务端处于`LAST_ACK`的状态
4. 第四次挥手：客户端接收到FIN报文后回一个ACK应答报文，之后客户端处于`TIME_WAIT`状态
5. 服务器收到ACK报文后，进入`CLOSE`状态，服务器完成连接关闭。
6. 客户端在经过 2MSL 一段时间后，自动进入`CLOSE`状态，客户端也完成连接的关闭。

为什么挥手需要四次

关闭连接时，客户端发送FIN报文，表示其不再发送数据，**但还可以接收数据**。

服务端收到FIN报文，可以直接发送SYN+ACK报文。其中ACK报文是用来应答的，SYN报文是用来同步的。但是关闭连接时，服务端可能还要数据需要处理和发送，所以先回一个ACK应答报文，等到其不再发送数据时，才发送FIN报文给客户端表示同意关闭连接。

从上面过程可知：服务端通常需要等待完成数据的发送和处理，**所以服务端的ACK和FIN一般都会分开发送**，从而比三次握手导致多了一次。

为什么需要 TIME_WAIT 状态

主动发起关闭连接的一方，才会有 TIME-WAIT 状态。

需要 TIME-WAIT 状态，主要是两个原因：

1. 防止历史连接中的数据，被后面相同四元组的连接错误的接收；

如果网络出现拥塞或延迟，数据包可能会在网络中滞留一段时间，甚至超过了原始连接关闭的时间。如果没有 TIME_WAIT 状态，客户端直接进入到CLOSE状态，这些滞留的数据包可能会被传递给新连接，导致新连接的数据被旧连接的数据干扰。

经过 2MSL 这个时间，**足以让两个方向上的数据包都被丢弃，使得原来连接的数据包在网络中都自然消失，再出现的数据包一定都是新建立连接所产生的。**

2. 保证「被动关闭连接」的一方能被正确的关闭，即保证最后的 ACK 能让被动关闭方接收，从而帮助其正常关闭

如果最后的一次ACK报文丢失（第四次挥手），客户端没有`TIME_WAIT`状态，直接进入ClOSE，服务端一直在等待ACK状态，一直没有等到，就会重发FIN报文，而客户端已经进入到关闭状态，在收到服务端重传的 FIN 报文后，就会回 RST 报文,服务端收到这个 RST 并将其解释为一个错误, 为了防止这种情况出现，客户端必须等待足够长的时间，确保服务端能够收到 ACK，如果服务端没有收到 ACK，那么就会触发 TCP 重传机制，服务端会重新发送一个 FIN，这样一去一来刚好两个 MSL 的时间。

如果 TIME-WAIT 等待足够长的情况就会遇到两种情况：

1. 服务端正常收到四次挥手的最后一个 ACK 报文，则服务端正常关闭连接。
2. 服务端没有收到四次挥手的最后一个 ACK 报文时，则会重发 FIN 关闭连接报文并等待新的 ACK 报文。

为什么 TIME_WAIT 等待的时间是 2MSL

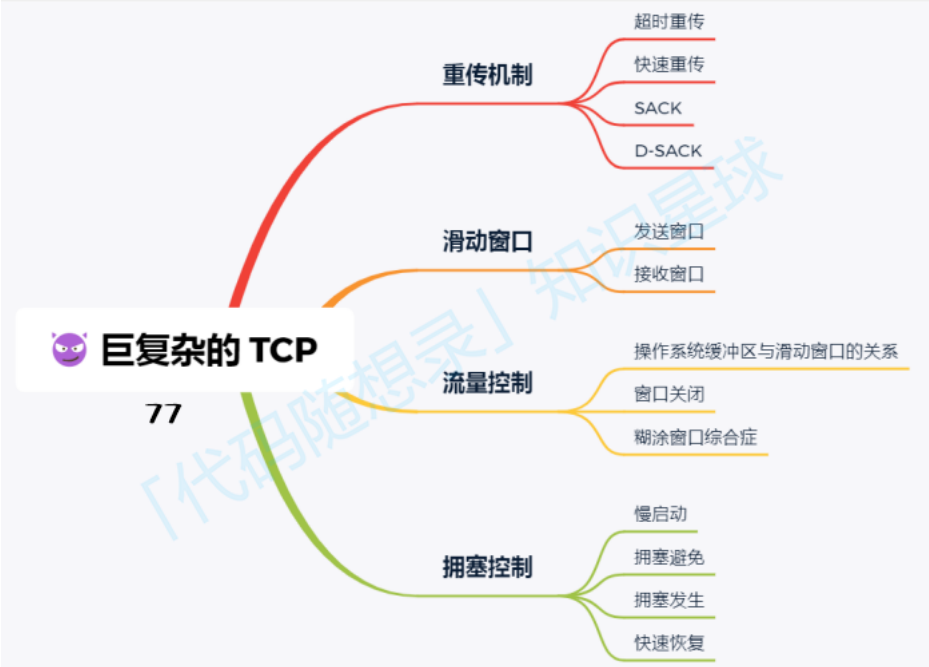
1. MSL是`Maximum Segment Lifetime`，报文最大生存时间，它是任何报文在网络上存在的最长时间，超过这个时间报文将被丢弃。
2. 等待MSL两倍：网络中可能存在发送方的数据包，当这些发送方的数据包被接收方处理后又会向对方发送响应，所以一来一回需要等待 2 倍的时间。
3. 1 个 MSL 确保四次挥手中主动关闭方最后的 ACK 报文最终能达到对端；1 个 MSL 确保对端没有收到 ACK 重传的 FIN 报文可以到达。
4. 2MSL 的时间是从客户端接收到 FIN 后发送 ACK 开始计时的。如果在 TIME-WAIT 时间内，因为客户端的 ACK 没有传输到服务端，客户端又接收到了服务端重发的 FIN 报文，那么 2MSL 时间将重新计时。

TIME_WAIT 过多有什么危害?

过多的TIME-WAIT 状态主要的危害有两种：

1. 内存资源占用；
2. 对端口资源的占用，一个 TCP 连接至少消耗一个本地端口；

如果发起连接一方的 `TIME_WAIT` 状态过多，占满了所有端口资源，则会导致无法创建新连接。



TCP的重传机制

TCP 实现可靠传输的方式之一，是通过序列号与确认应答。

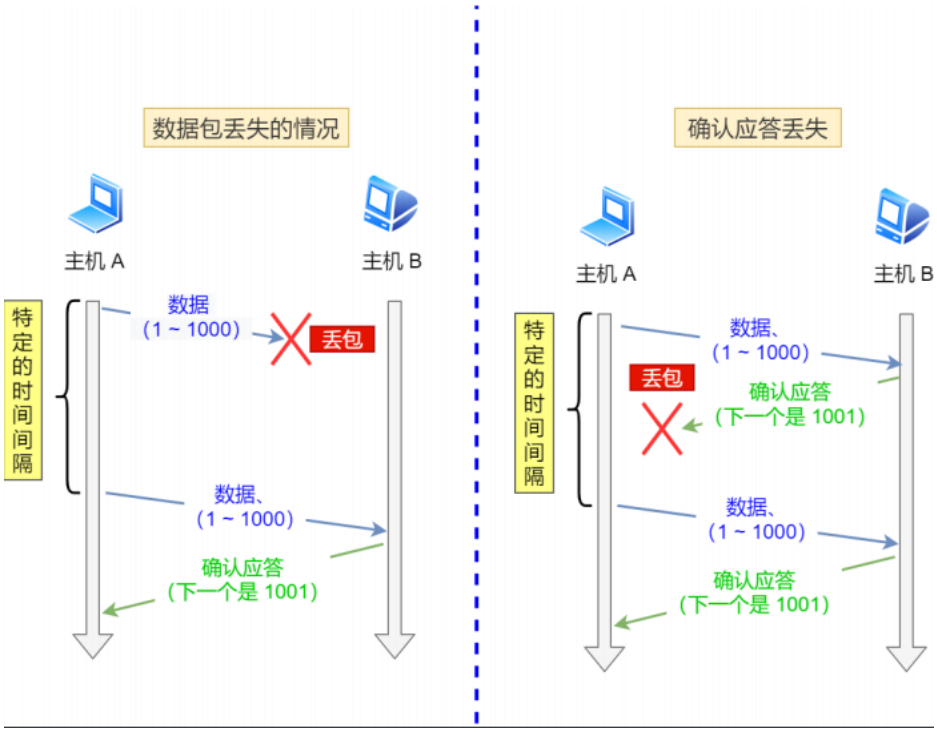
在 TCP 中，当发送端的数据到达接收主机时，接收端主机会返回一个确认应答消息，表示已收到消息。

但是如果传输的过程中，数据包丢失了，就会使用重传机制来解决。

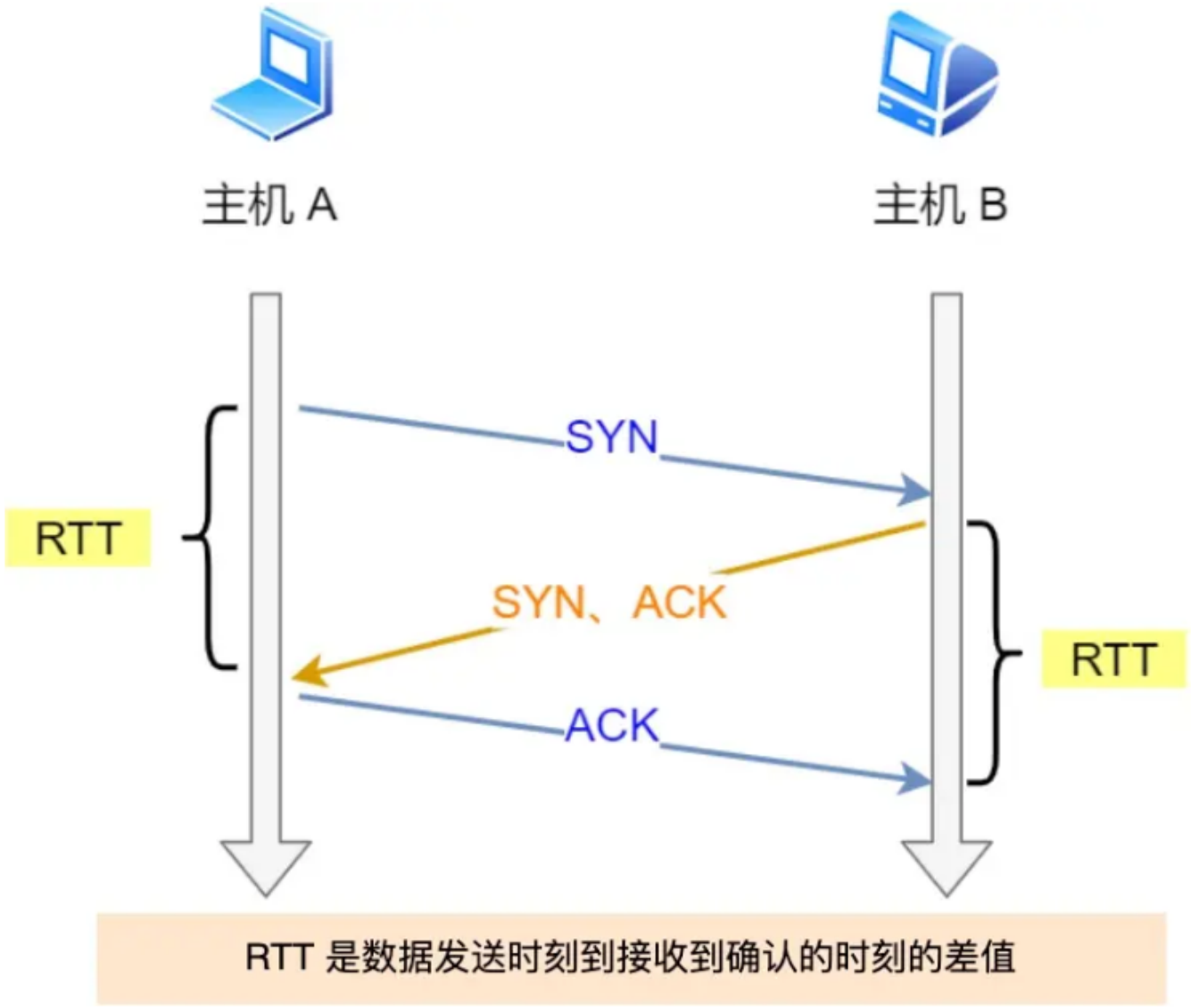
1、超时重传：

设定一个计时器，当超过指定的时间后，没有收到对方的确认ACK应答报文，就会重发该数据。

超时重传的两种情况如下：

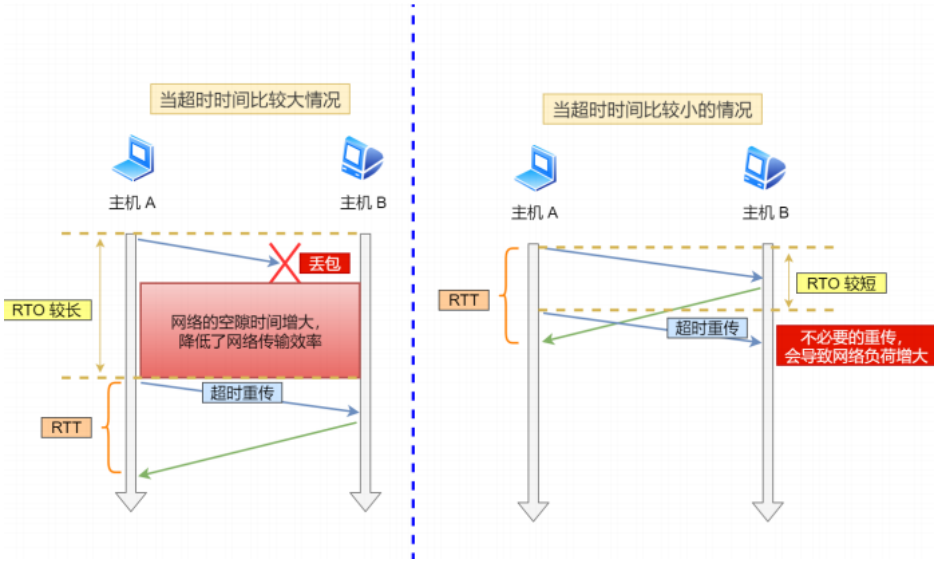


RTT(round-trip time):往返时延,指的是数据包的往返时间

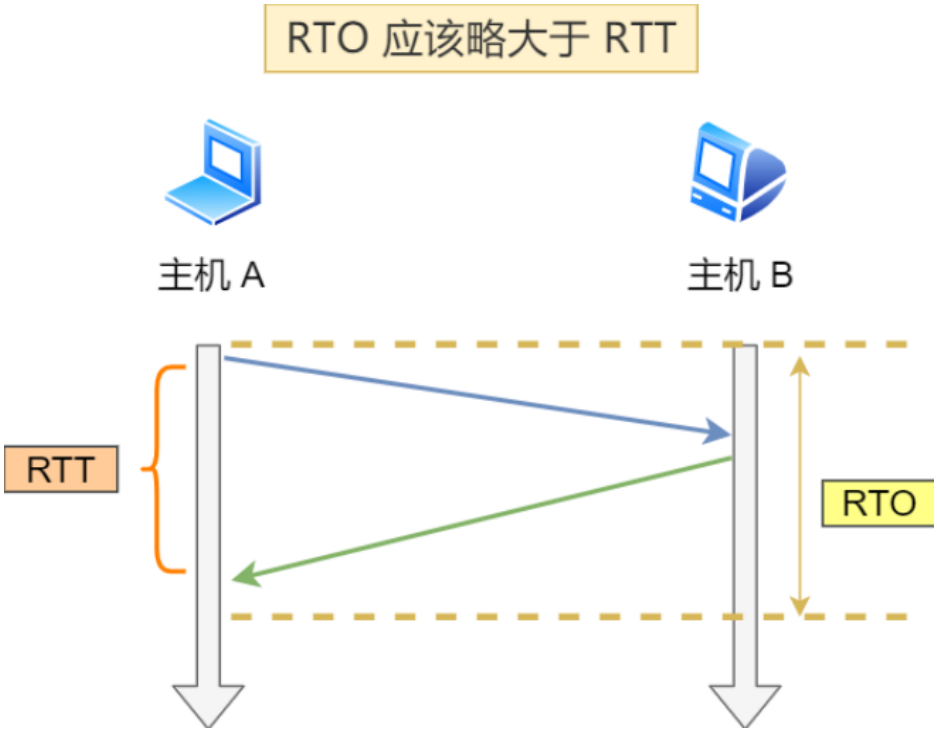


RTO（Retransmission Timeout）：超时重传时间

RTO较长或较短可能发生的情况如下：



由上图可知：**超时重传时间RTO的值 应该略大于 报文往返RTT的值，且是动态变化的。**



如果超时重发的数据，再次超时又要重传的时候，TCP的策略是将超时间隔加倍，也就是**每当遇到一次超时重传的时候，都会将下一次超时时间间隔设为先前值的两倍。**

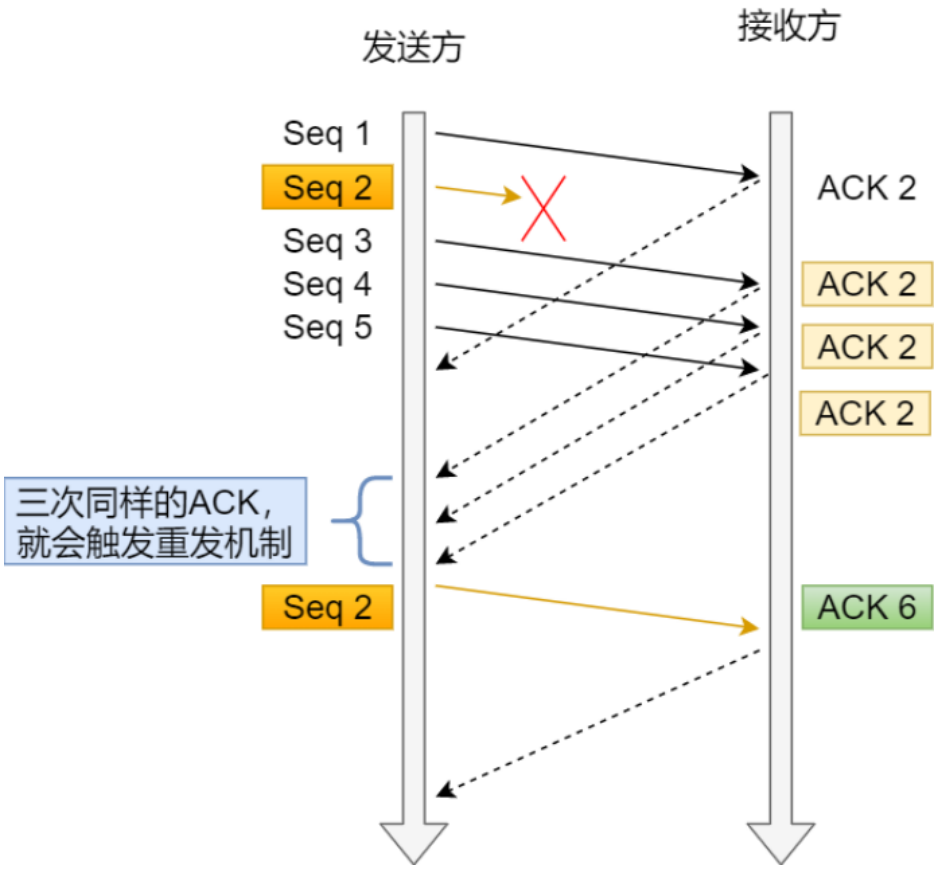
2、快速重传：

快速重传（Fast Retransmit）机制，它不以时间为驱动，而是以数据驱动重传。

工作原理：

当收到三个相同的ACK报文时，会在定时器过期之前，重传丢失的报文段。

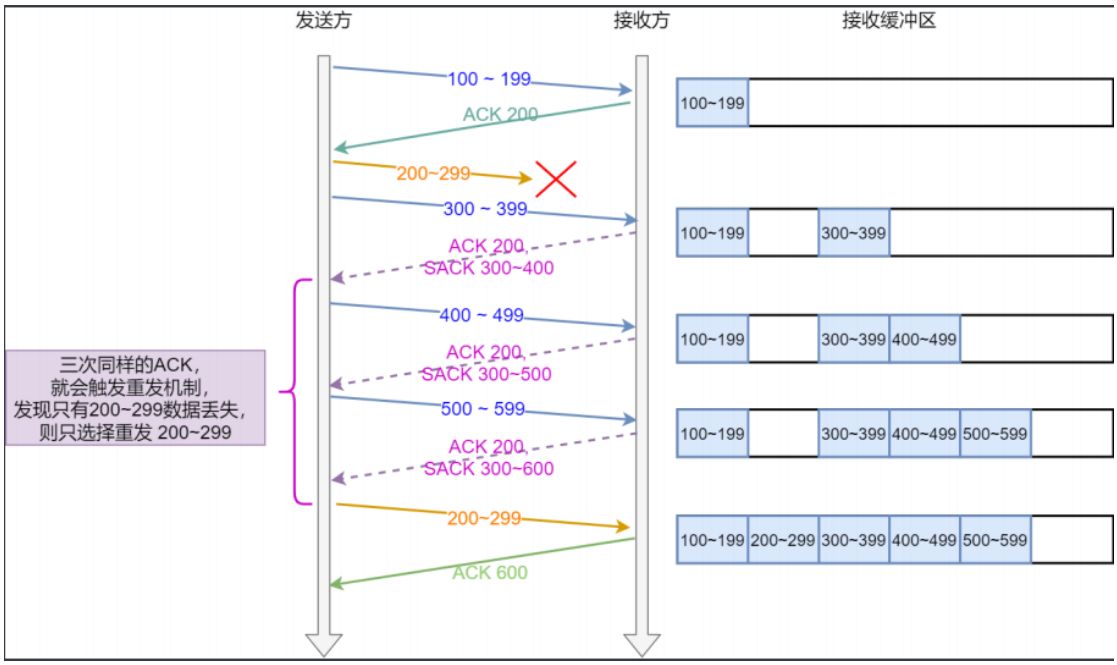
快速重传解决了超时时间的问题，但还面临另外一个问题：重传的时候，是重传之前的一个，还是重传所有的问题。



3、SACK（Selective Acknowledgment,选择性确认）：

为了解决 重传哪些报文的问题而提出。

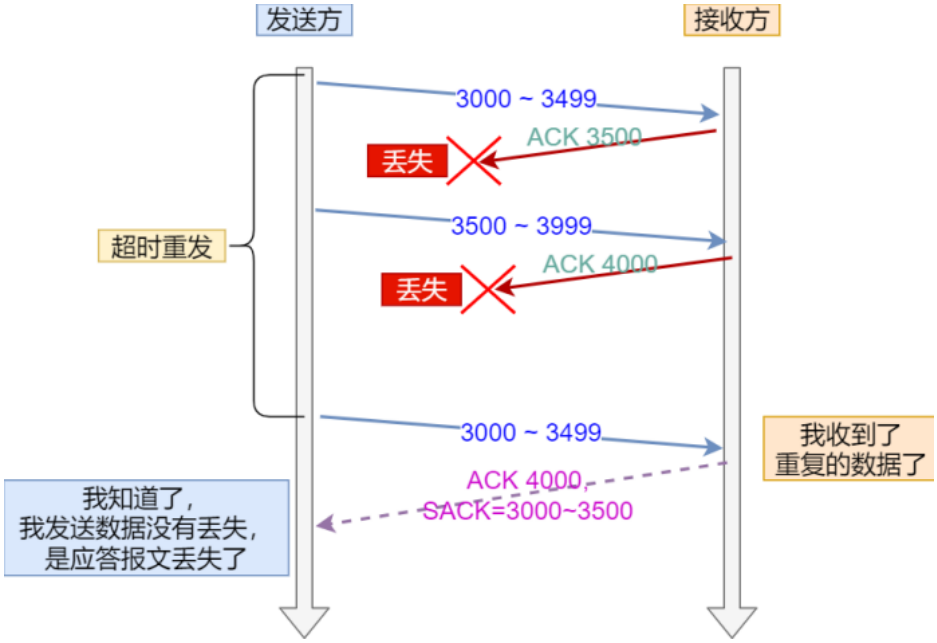
这种方式需要在 TCP 头部「选项」字段里加一个 `SACK` 的东西，**可以将已收到的数据的信息发送给「发送方」**，这样发送方就可以知道哪些数据收到了，哪些数据没收到，知道了这些信息，就可以**只重传丢失的数据**。



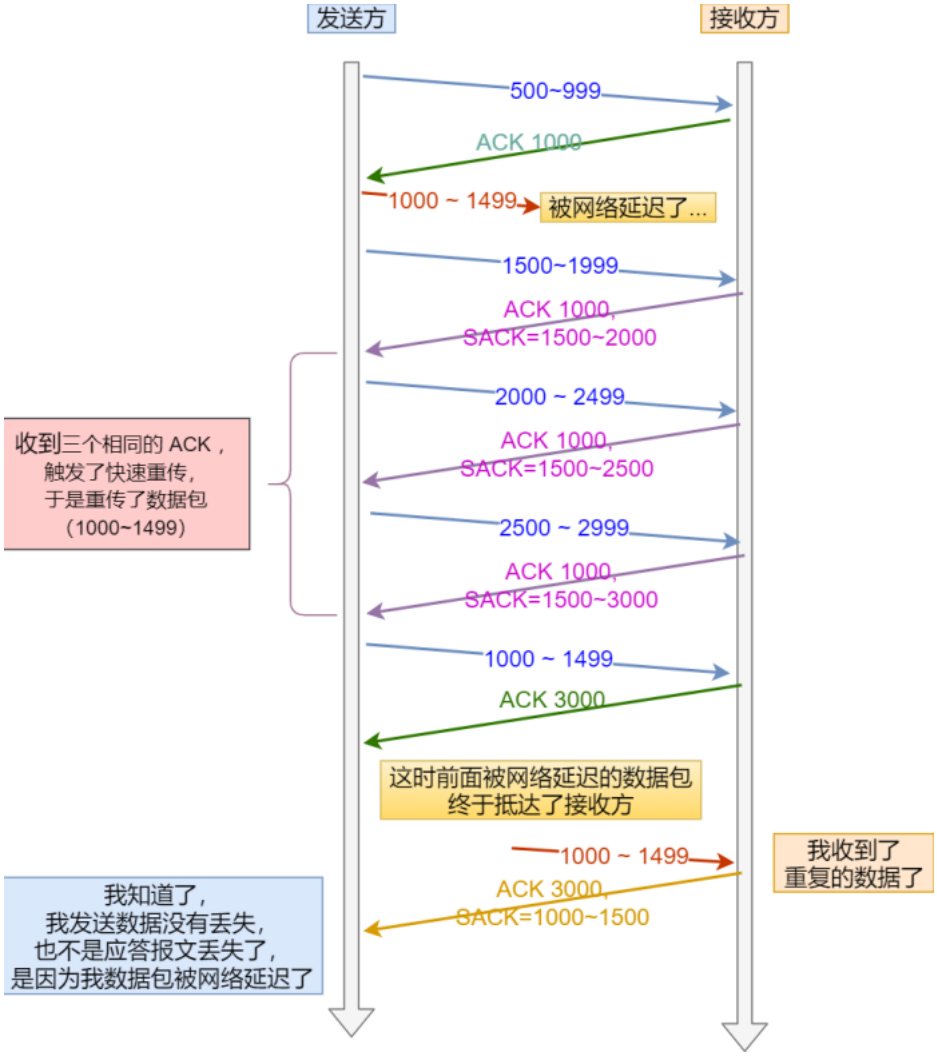
4、D-SACK（Duplicate SACK）：

主要使用了SACK来告诉【发送方】有哪些数据被重复接收了。下面举例来说明 `D-SACK` 的作用：

ACK丢包：



网络延时：



使用D-SACK的好处：

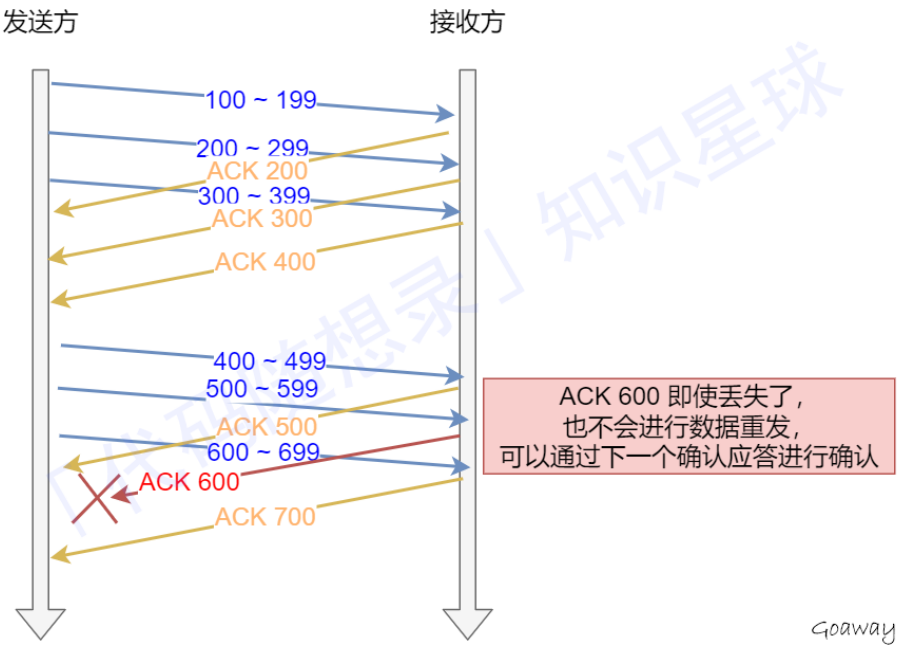
（1）可以让【发送方】知道，是发出去的包丢了，还是接收方回应的ACK包丢了；
（2）可以知道是不是【发送方】的数据包被网络演示了；
（3）可以知道网络中是不是把【发送方】的数据包给复制了。

TCP的滑动窗口

1. 什么是窗口

TCP每发送一个数据，都需要一次应答，然后继续发送，这样为每个数据包都进行确认应答，缺点是：数据往返时间越长，网络吞吐量越低。为了解决这个问题，TCP 引入了**窗口**这个概念。即使在往返时间较长的情况下，它也不会降低网络通信的效率。而窗口的大小呢，就是无需等待确认应答，可以继续发送数据的最大值。

假设窗口大小为 `3` 个 TCP 段，那么发送方就可以「连续发送」 `3` 个 TCP 段，并且中途若有 ACK 丢失，可以通过「下一个确认应答进行确认」。如下图：



窗口的实现就是操作系统开辟的一个缓存空间，发送方主机在等到确认应答返回之前，必须在缓冲区中保留已发送的数据。如果按期收到确认应答，此时数据就可以从缓存区清除。

图中的 ACK 600 确认应答报文丢失，也没关系，因为可以通过下一个确认应答进行确认，只要发送方收到了 ACK 700 确认应答，就意味着 700 之前的所有数据「接收方」都收到了。这个模式就叫**累计确认或者累计应答**。

2. 什么决定窗口大小

TCP头部有一个字段叫window，窗口大小。

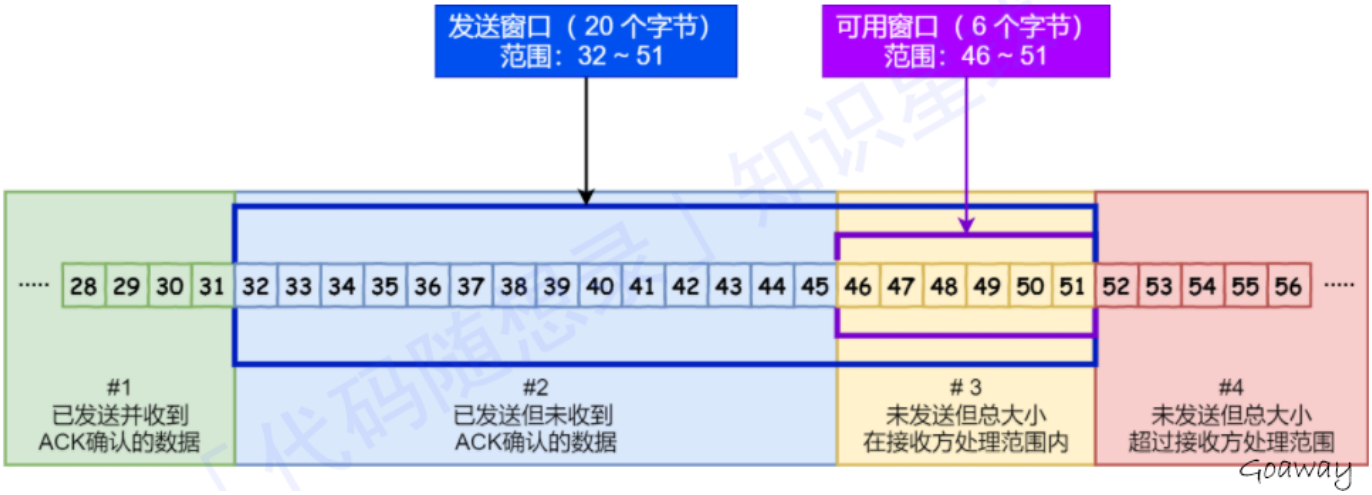
这个字段是接收端告诉发送端自己还有多少缓冲区可以接收数据。于是发送端就可以根据这个接收端的处理能力来发送数据，而不会导致接收端处理不过来

通常窗口的大小是由接收方的窗口大小来决定的。

发送方发送的数据大小不能超过接收方的窗口大小，否则接收方就无法正常接收到数据。

3. 发送方的滑动窗口

下图就是发送方缓存的数据，根据处理的情况分成四个部分：



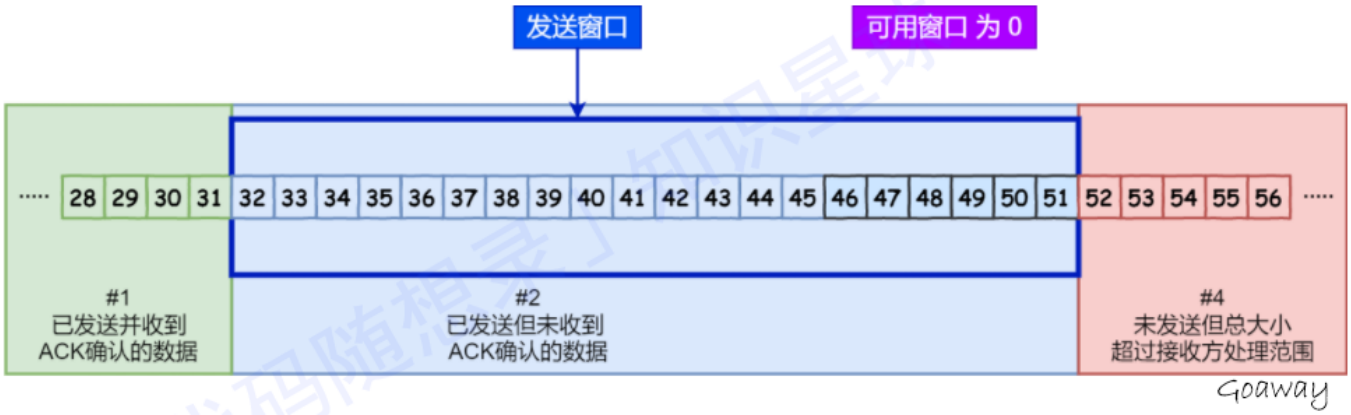
#1 是已发送并收到 ACK确认的数据：1到31 字节

#2 是已发送但未收到 ACK确认的数据：32到45 字节

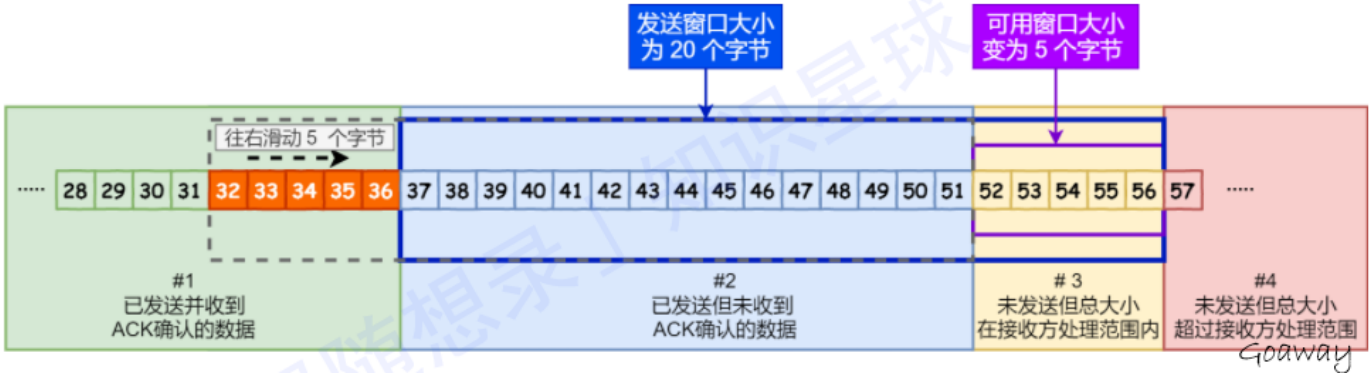
#3 是未发送但总大小在接收方处理范围内（接收方还有空间）：46到51字节

#4 是未发送但总大小超过接收方处理范围（接收方没有空间）：52字节以后

下图表示数据全都发送以后，可用窗口大小为0，在没收到ACK确认应答之前无法继续发送数据。



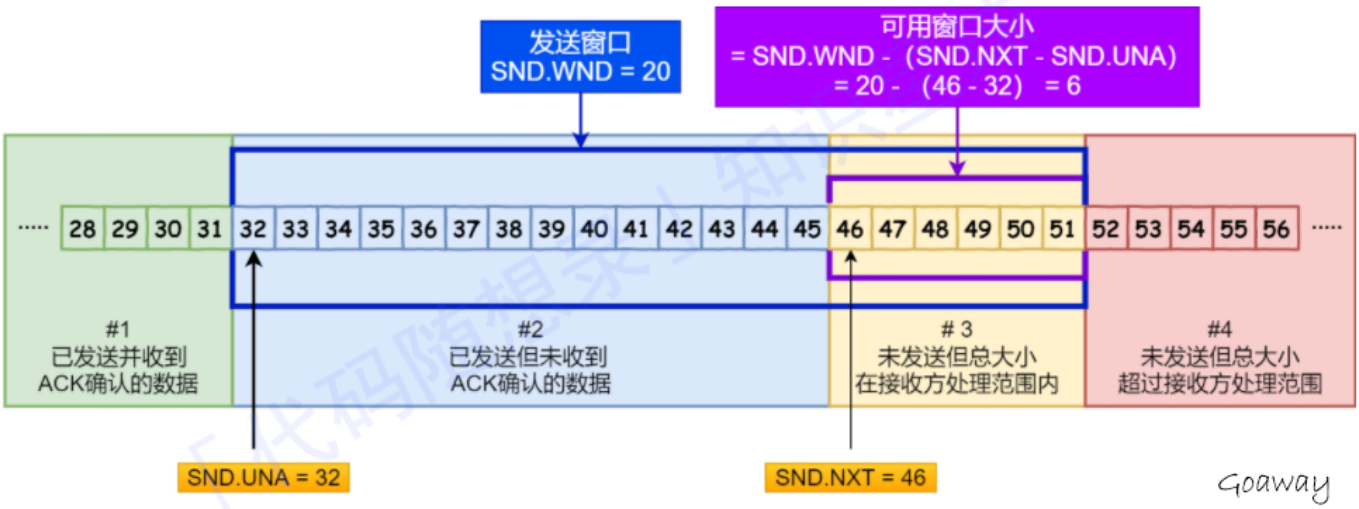
下图，当收到之前发送的数据 `32~36` 字节的 ACK 确认应答后，如果发送窗口的大小没有变化，则**滑动窗口往右边移动 5 个字节，因为有 5 个字节的数据被应答确认**，接下来 `52~56` 字节又变成了可用窗口，那么后续也就可以发送 `52~56` 这 5 个字节的数据了。



在收到32-36的ACK应答后，如果窗口大小不变，则滑动窗口先后移动5个字节，那么52-56变成可用窗口，可以继续发送数据。

4. 程序是如何表示发送方的四个部分的呢?

TCP 滑动窗口方案使用三个指针来跟踪在四个传输类别中的每一个类别中的字节。其中两个指针是绝对指针（指特定的序列号），一个是相对指针（需要做偏移）。



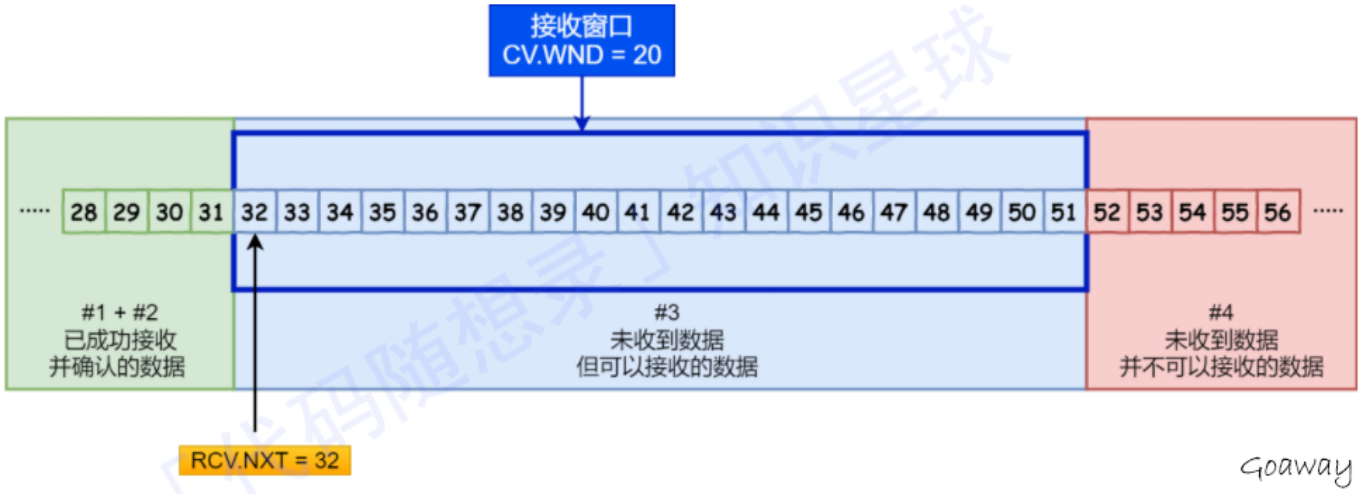
- `SND.WND`：表示发送窗口的大小（大小是由接收方指定的）；
- `SND.UNA`：是一个绝对指针，它指向的是已发送但未收到确认的第一个字节的序列号，也就是 #2 的第一个字节。
- `SND.NXT`：也是一个绝对指针，它指向未发送但可发送范围的第一个字节的序列号，也就是 #3 的第一个字节。
- 指向 #4 的第一个字节是个相对指针，它需要 `SND.UNA` 指针加上 `SND.WND` 大小的偏移量，就可以指向 #4 的第一个字节了。

可用窗口大小 = 发送窗口 - 已发送未确定`(SND.WND - (SND.NXT - SND.UNA))`

已发送未确认 = `(SND.NXT - SND.UNA)`

5. 接收方滑动窗口

接收窗口根据处理的情况划分成三个部分：



#1 + #2 是已成功接收并确认的数据（等待应用进程读取）

#3 是未收到数据但可以接收的数据

#4 未收到数据并不可以接收的数据

其中三个接收部分，使用两个指针进行划分:

- `RCV.WND`：表示接收窗口的大小，它会通告给发送方。
- `RCV.NXT`：是一个指针，它指向期望从发送方发送来的下一个数据字节的序列号，也就是 #3 的第一个字节。
- 指向 #4 的第一个字节是个相对指针，它需要 `RCV.NXT` 指针加上`RCV.WND`大小的偏移量，就可以指向 #4 的第一个字节了。

6. 接收窗口和发送窗口的大小是相等的吗?

并不是完全相等，接收窗口的大小是**约等于**发送窗口的大小的。

因为滑动窗口并不是一成不变的。比如，当接收方的应用进程读取数据的速度非常快的话，这样的话接收窗口可以很快的就空缺出来。那么新的接收窗口大小，是通过 TCP 报文中的 Windows 字段来告诉发送方。那么这个传输过程是存在时延的，所以接收窗口和发送窗口是约等于的关系。

TCP的流量控制机制

TCP 流量控制的基本原理是使用滑动窗口机制，其中接收方可以通过调整窗口大小来告诉发送方其当前处理数据的能力。

1. **接收窗口**：接收方维护一个接收窗口，表示可以接收的数据段的范围。窗口大小可以根据接收方的处理能力进行调整。
2. **通告窗口大小**：接收方通过 TCP 报文中的确认信息，通告当前的接收窗口大小给发送方。发送方会根据这个窗口大小来控制发送数据的速率。
3. **窗口滑动**：随着接收方处理数据的能力，窗口可以向前滑动。接收方可以通告更大的窗口，表示它可以接收更多的数据。
4. **发送速率控制**：发送方会根据接收方通告的窗口大小来控制发送数据的速率。如果接收窗口变小，表示接收方的处理能力减弱，发送方会减慢发送速率，避免数据拥塞。
5. **动态调整**：TCP 流量控制是动态的，适应网络和接收方的变化。如果网络拥塞或接收方的处理速度变慢，流量控制可以适时地减少发送速率。

TCP的拥塞控制机制

在网络出现拥堵时，如果继续发送大量数据包，可能会导致数据包时延、丢失等，这时 TCP 就会重传数据，但是一重传就会导致网络的负担更重，于是会导致更大的延迟以及更多的丢包，这个情况就会进入恶性循环被不断地放大....，于是，就有了**拥塞控制**，控制的目的就是**避免「发送方」的数据填满整个网络。**

拥塞控制通过`拥塞窗口`来防止过多的数据注入网络，使得网络中的路由器或者链路过载。

- 拥塞窗口`cwnd`是发送方维护的一个状态变量，根据网络拥塞程度而变化。
- 发送窗口的值是`swnd = min(cwnd, rwnd)`，也就是拥塞窗口和接收窗口中的最小值。
- 网络中没有出现拥塞，cwnd增大，出现拥塞，cwnd减小。

其实只要发送方没有在规定时间内接收到 ACK 应答报文，也就是发生了超时重传，就会认为网络出现了拥塞。

拥塞控制的常见算法:

1. 慢启动
2. 拥塞避免

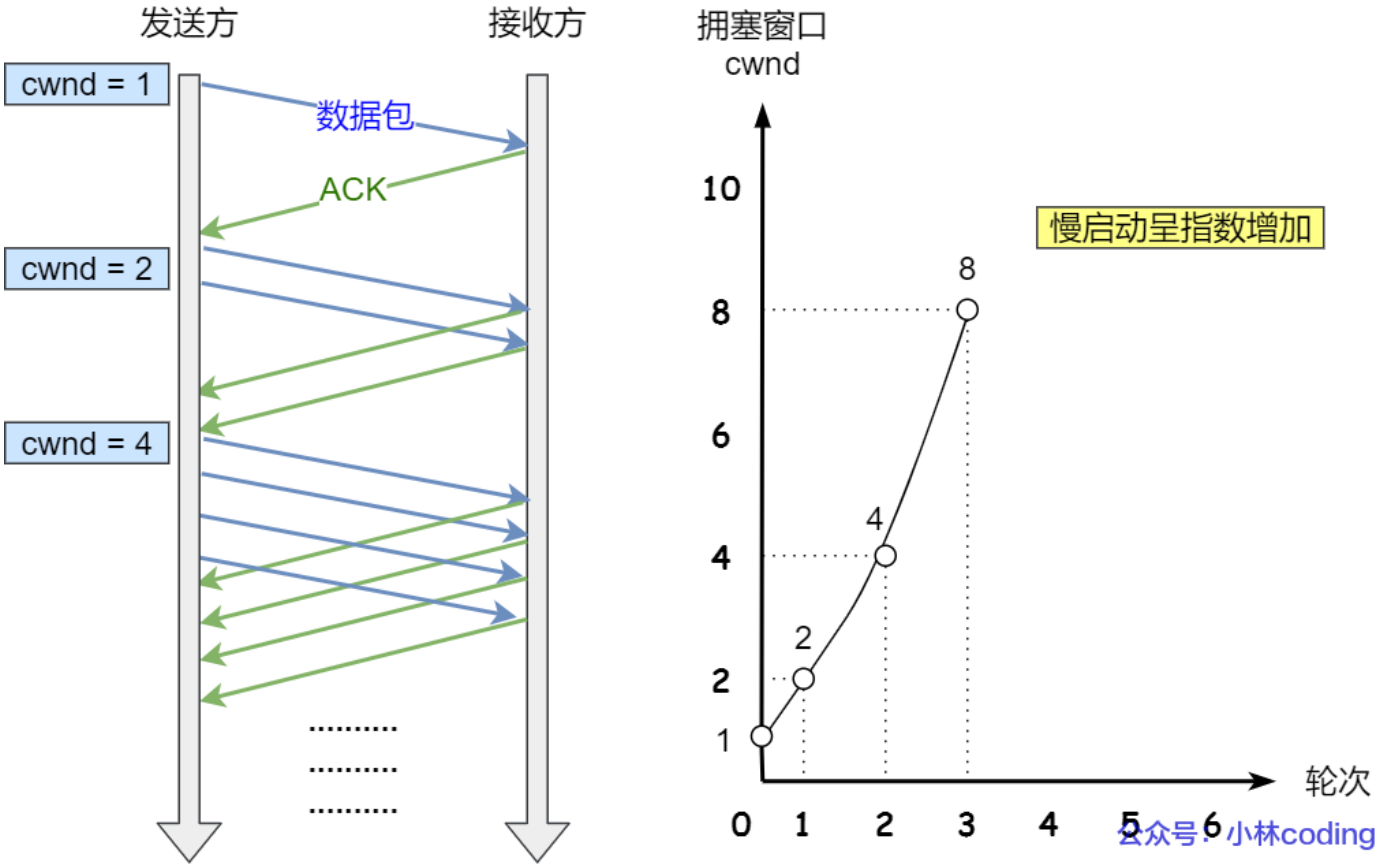
3. 拥塞发生
4. 快速恢复

1. 慢启动

TCP 在刚建立连接完成后，首先是有个慢启动的过程，这个慢启动的意思就是一点一点的提高发送数据包的数量。

慢启动的算法记住一个规则就行：**当发送方每收到一个 ACK，拥塞窗口 `cwnd` 的大小就会加 1。**

慢启动算法，发包的个数是指数性的增长。



不过，慢启动有一个门限 `ssthresh` 状态变量：

1. 当 cwnd < ssthresh 时，使用慢启动算法。
2. 当 cwnd >= ssthresh 时，就会使用拥塞避免算法。

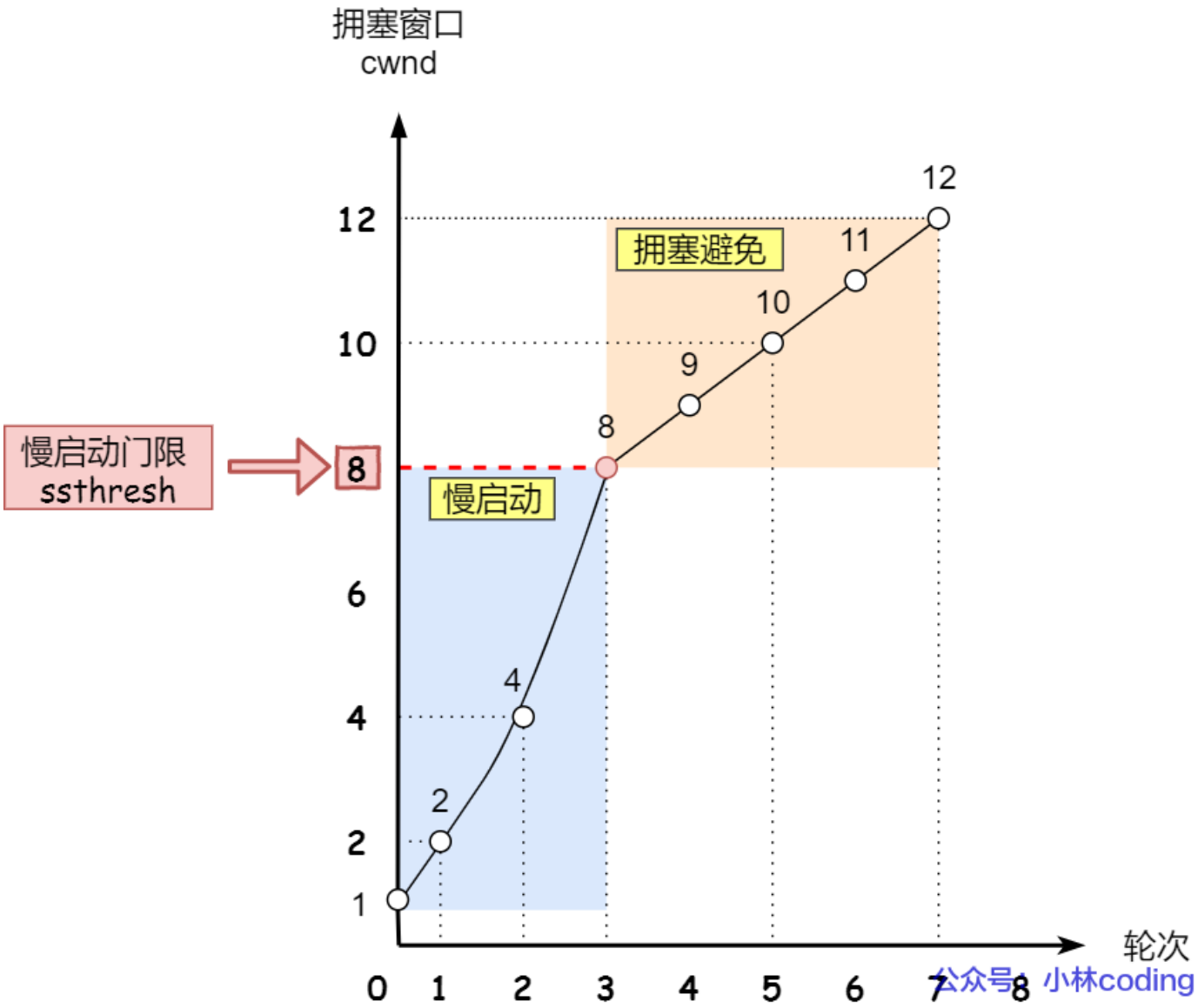
2. 拥塞避免

一般来说 `ssthresh` 的大小是65535字节。超过后会就会进入拥塞避免算法。

进入拥塞避免算法后，它的规则是：**每当收到一个 ACK 时，cwnd 增加1/cwnd。**

现假定 `ssthresh` 为 `8`：

- 当 8 个 ACK 应答确认到来时，每个确认增加 1/8，8 个 ACK 确认 cwnd 一共增加 1，于是这一次能够发送 9 个 `MSS` 大小的数据，变成了**线性增长。**



所以**拥塞避免算法就是将原本慢启动算法的指数增长变成了线性增长**，还是增长阶段，但是增长速度缓慢了一些。不过网络就会慢慢进入了拥塞的状况了，就会出现丢包现象，这时就需要对丢失的数据包进行重传。

当触发了重传机制，也就进入了拥塞发生算法。

3. 拥塞发生

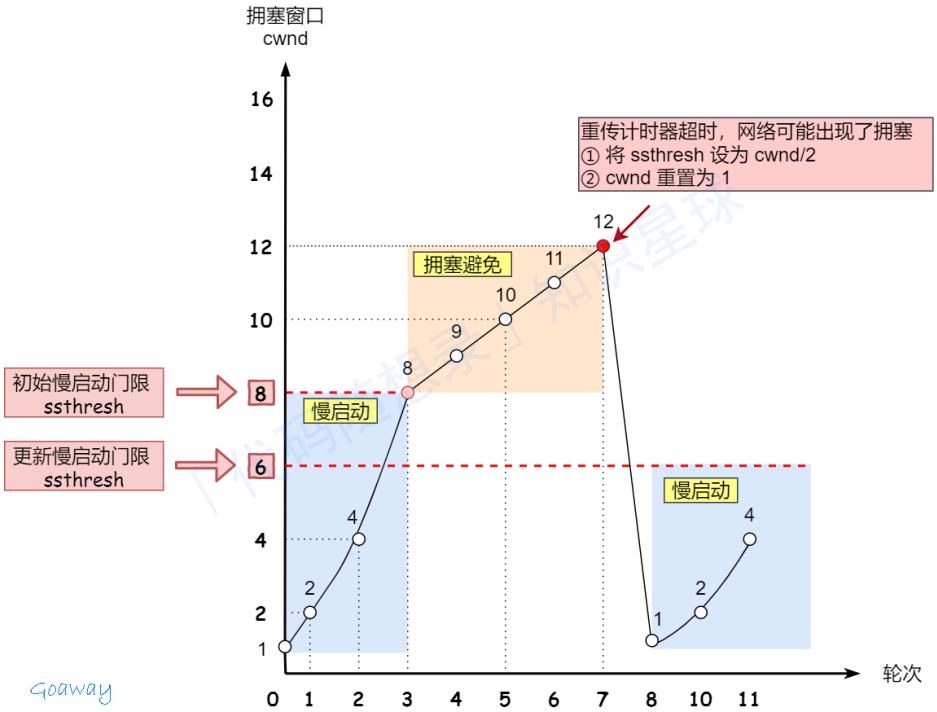
当网络出现拥塞，也就是会发生数据包重传，重传机制主要有两种：

- 超时重传
- 快速重传

（1）发生超时重传的拥塞发生算法：

这个时候，ssthresh 和 cwnd 的值会发生变化

1. `ssthresh` 设为 `cwnd/2`，
2. `cwnd` 重置为 1



接着，就重新开始慢启动，慢启动是会突然减少数据流的。这真是一旦「超时重传」，马上回到解放前。但是这种方式太激进了，反应也很强烈，会造成网络卡顿。

（2）快速重传拥塞算法：

TCP 认为这种情况不严重，因为大部分没丢，只丢了一小部分，则 ssthresh 和 cwnd 变化如下：

1. cwnd = cwnd/2 ，也就是设置为原来的一半
2. ssthresh = cwnd
3. 进入快速恢复算法

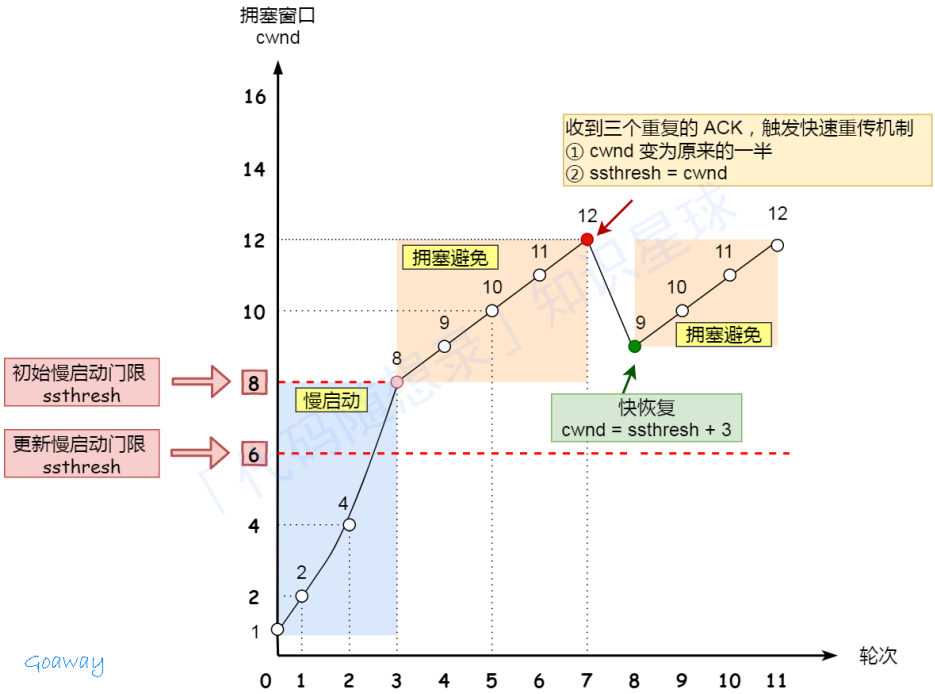
4. 快速恢复

快速重传和快速恢复算法一般同时使用，快速恢复算法是认为，还能收到 3 个重复 ACK 说明网络也不那么糟糕，所以没有必要像 RTO 超时那么强烈。

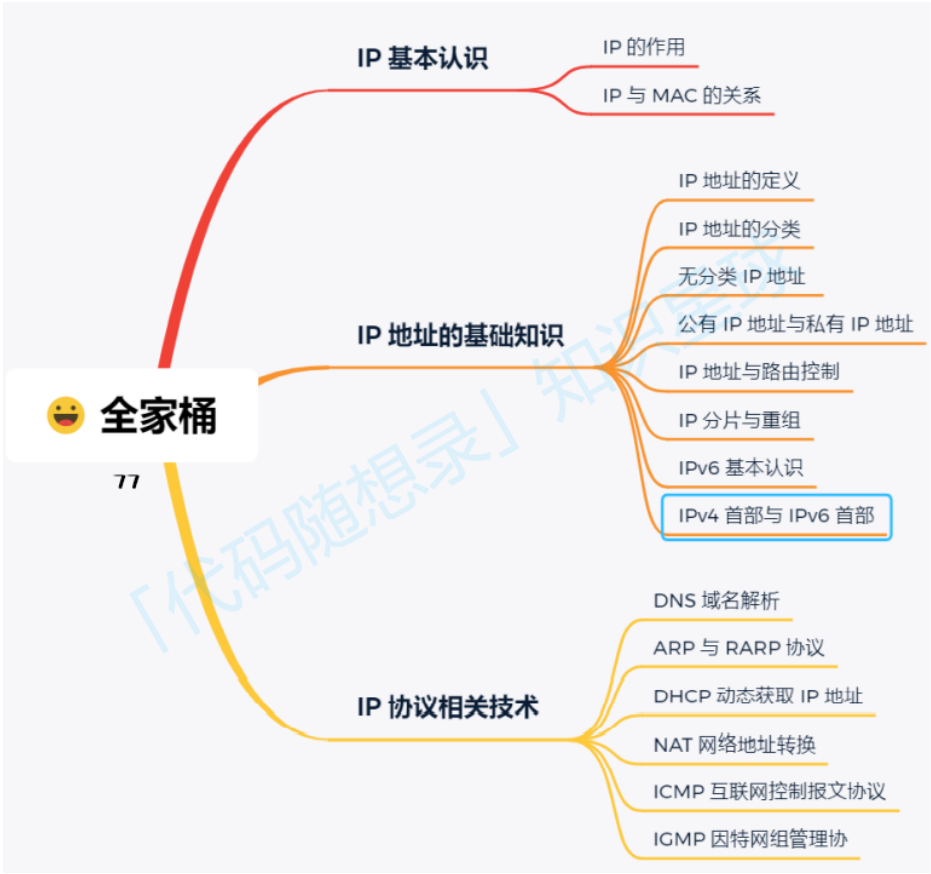
进入快速恢复：

1. 拥塞窗口 `cwnd = ssthresh + 3` （ 3 的意思是确认有 3 个数据包被收到了）；
2. 重传丢失的数据包；
3. 如果再收到重复的 ACK，那么 cwnd 增加 1；
4. 如果收到新数据的 ACK 后，把 cwnd 设置为第一步中的 ssthresh 的值，原因是该 ACK 确认了新的数据，说明从 duplicated ACK 时的数据都已收到，该恢复过程已经结束，可以回到恢复之前的状态了，也即再次进入拥塞避免状态；

而是还在比较高的值，后续呈线性增长。



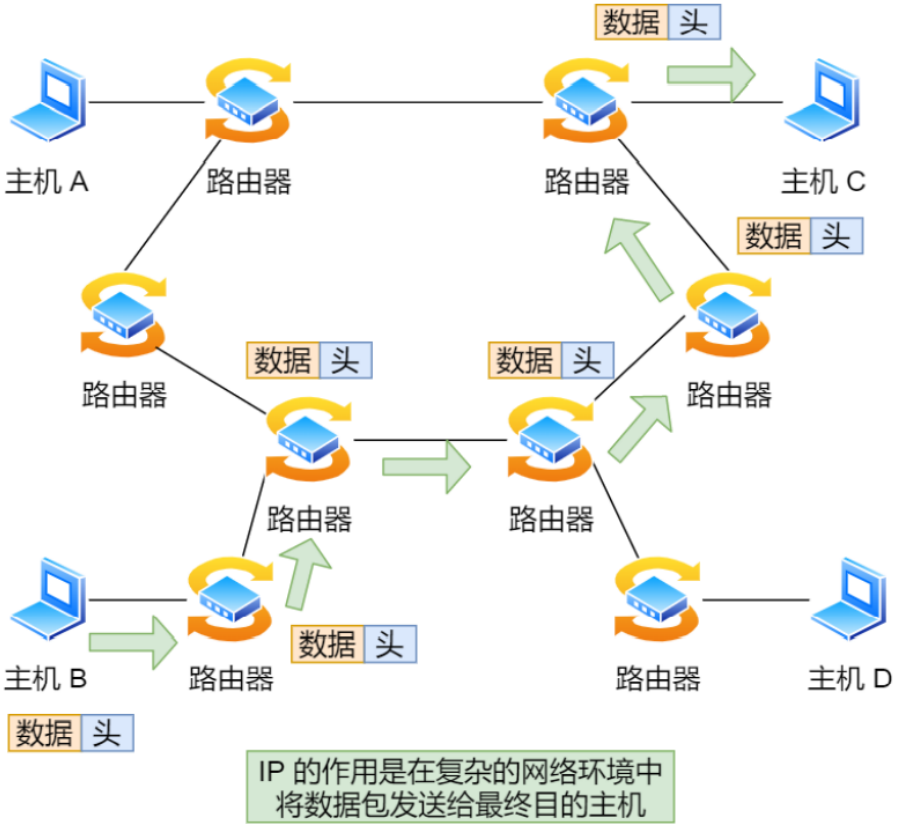
IP基础



IP基础

IP位于TCP/IP参考模型的第三层，也就是网络层。

网络层的主要作用是：实现主机与主机之间的通信，也叫点对点通信。



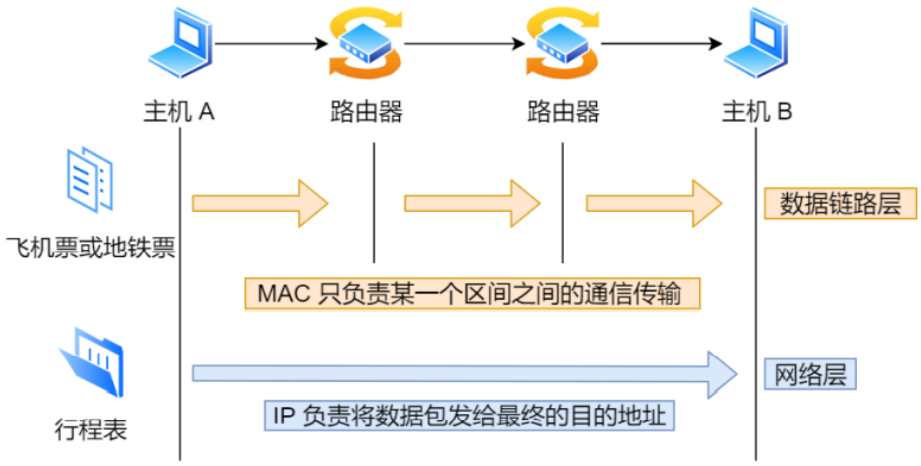
1、网络层（IP)与数据链路层(MAC)有什么关系呢?

MAC的作用:

实现【直连】的两个设备之间通信。

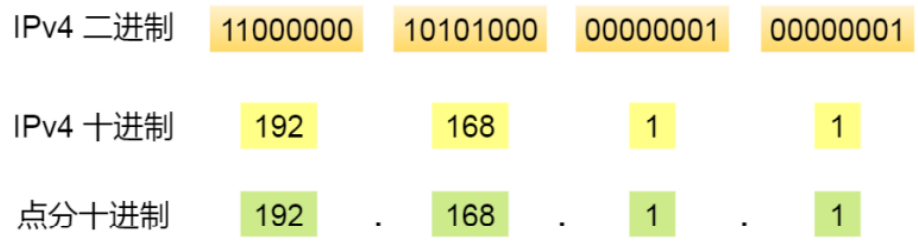
IP的作用:

负责在【没有直连】的两个网络之间进行通信传输。



在网络数据包传输中，源IP地址和目标IP地址在传输过程中是不会变的，只有源MAC地址和目标MAC一直在变化。

IP地址



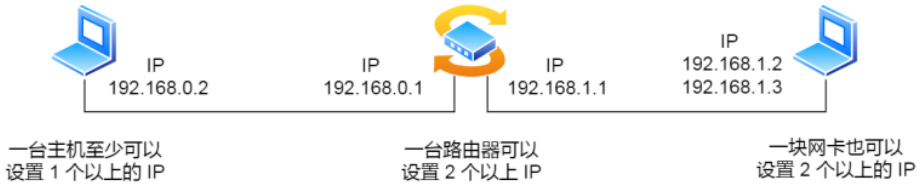
那么，IP 地址最大值也就是

$$2^{32} = 4294967296$$

也就说，最大允许 43 亿台计算机连接到网络。

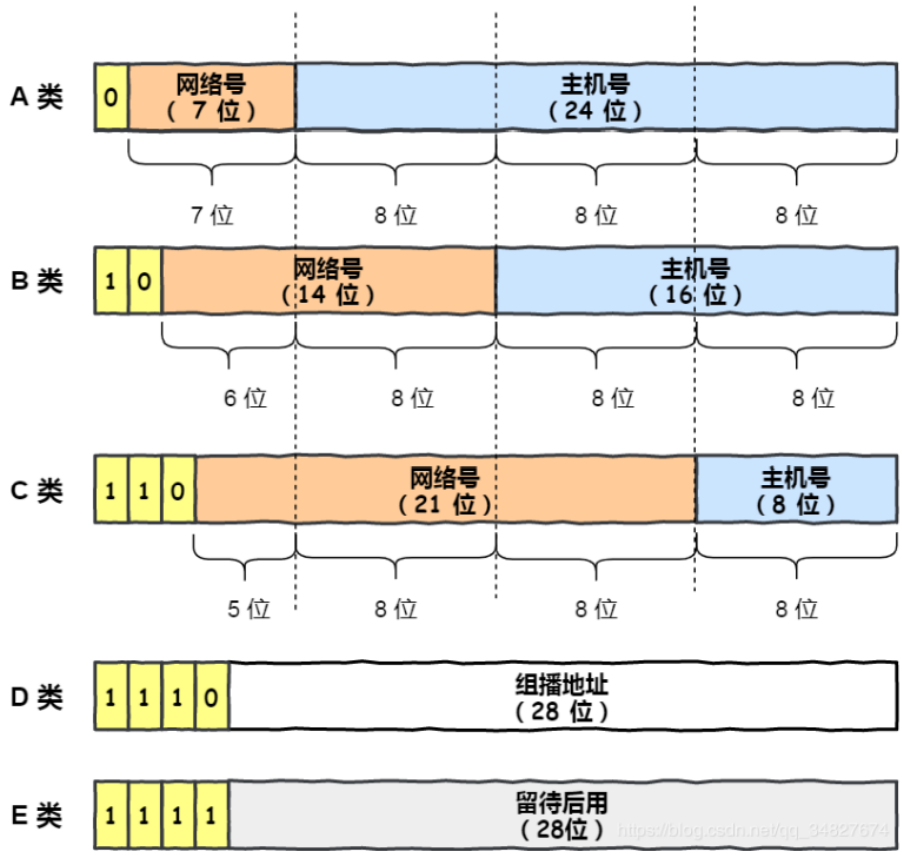
实际上让 43 亿台计算机全部连网其实是不可能的，更何况 IP 地址是由「网络标识」和「主机标识」这两个部分组成的，所以实际能够连接到网络的计算机个数更是少了很多。

但可以根据一种可以更换IP地址的技术NAT，使得可连接计算机数超过43亿台。



1、IP地址分类

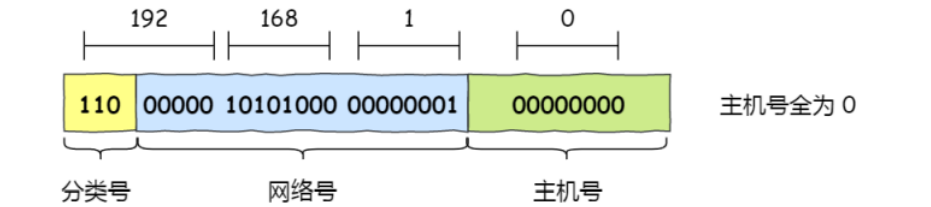
IP地址被分为了五种类型：

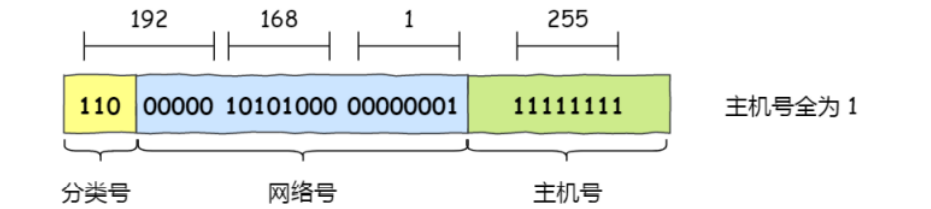


（1）A、B、C类地址

类别	IP 地址范围	最大主机数
A	0.0.0.0 ~ 127.255.255.255	16777214
B	128.0.0.0 ~ 191.255.255.255	65534
C	192.0.0.0 ~ 223.255.255.255	254

最大主机数=2^主机号位数-2，其中有两个地址，全0和全1是比较特殊的：



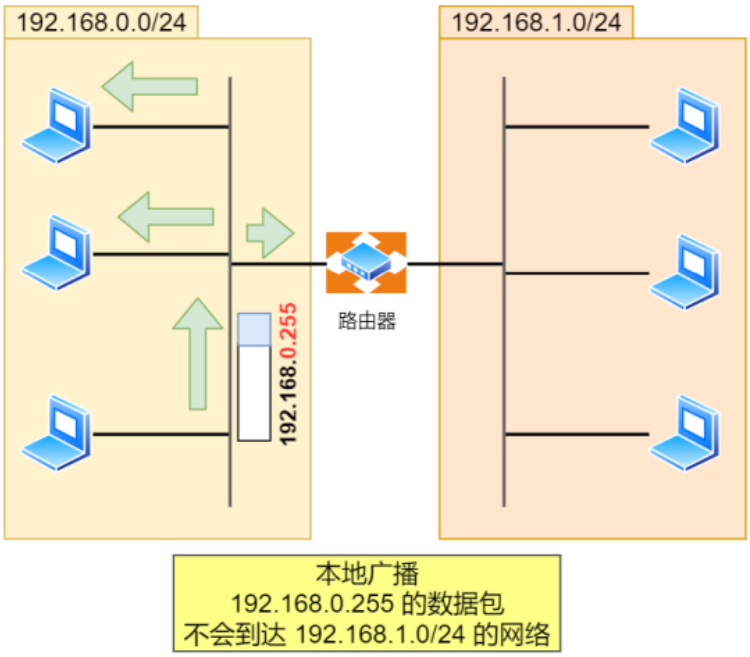


- 主机号全为 1 指定某个网络下的所有主机，用于广播
- 主机号全为 0 指定某个网络

广播地址用于在同一个链路中相互连接的主机之间发送数据包。广播地址可以分为本地广播和直接广播两种。

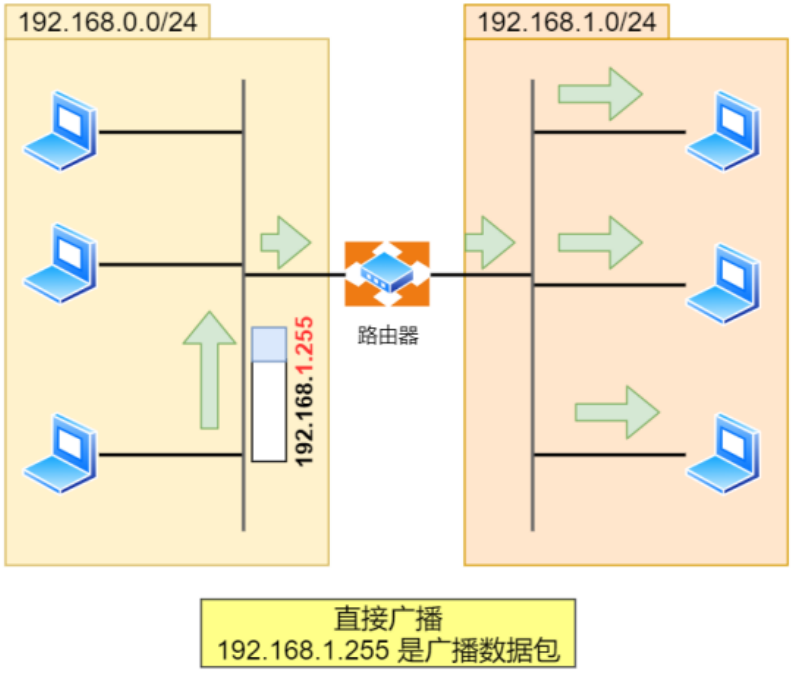
在本网络内广播的叫做本地广播。例如网络地址为 `192.168.0.0/24` 的情况下，广播地址是 `192.168.0.255`。

因为这个广播地址的 IP 包会被路由器屏蔽，所以不会到达 `192.168.0.0/24` 以外的其他链路上。



在不同网络之间的广播叫做直接广播。

例如网络地址为 `192.168.0.0/24` 的主机向 `192.168.1.255/24` 的目标地址发送 IP 包。收到这个包的路由器，将数据转发给 `192.168.1.0/24` 从而使得所有 `192.168.1.1~192.168.1.254` 的主机都能收到这个包（由于直接广播有一定的安全问题，多数情况下会在路由 器上设置为不转发。）

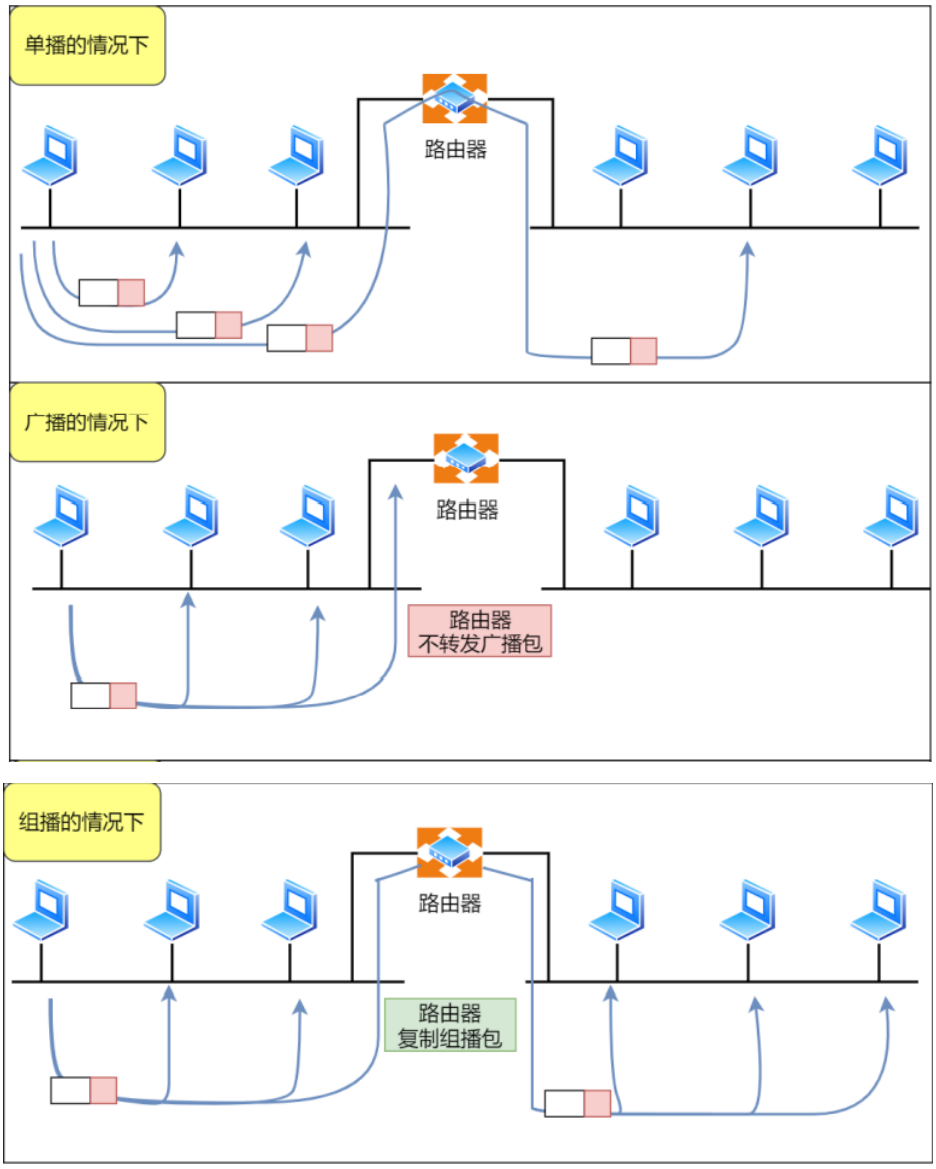


（2）什么是D、E类地址?

D 类和 E 类地址是没有主机号的，所以不可用于主机 IP，D 类常被用于多播，E 类是预留的分类，暂时未使用。

类别	IP 地址范围	用途
D	224.0.0.0 ~ 239.255.255.255	IP 多播
E	240.0.0.0 ~ 255.255.255.255	预留使用

多播地址用于将包发送给特定组内的所有主机。由于广播无法穿透路由，若想给其他网段发送同样的包，就可以使用可以穿透路由的多播。



多播使用的 D 类地址，其前四位是 1110 就表示是多播地址，而剩下的 28 位是多播的组编号。

从 224.0.0.0 ~ 239.255.255.255 都是多播的可用范围，其划分为以下三类：

224.0.0.0 ~ 224.0.0.255 为预留的组播地址，只能在局域网中，路由器是不会进行转发的。
224.0.1.0 ~ 238.255.255.255 为用户可用的组播地址，可以用于 Internet 上。
239.0.0.0 ~ 239.255.255.255 为本地管理组播地址，可供内部网在内部使用，仅在特定的本地范围内有效。

（3）IP分类的优缺点

优点：

可以根据IP地址的前四位来判别IP地址属于哪个类别。简单明了，选路简单。

缺点：

同一网络下没有地址层次，比如一个公司里用了 B 类地址，但是可能需要根据生产环境、测试环境、开发环境来划分地址层次，而这种 IP 分类是没有地址层次划分的功能，所以这就缺少地址的灵活性。

A、B、C类有个尴尬处境，就是不能很好的与现实网络匹配。

C 类地址能包含的最大主机数量实在太少了，只有 254 个，估计一个网吧都不够用。

而 B 类地址能包含的最大主机数量又太多了，6 万多台机器放在一个网络下面，一般的企业基本达不到这个规模，闲着的地址就是浪费。这两个缺点，都可以在 CIDR 无分类地址解决。

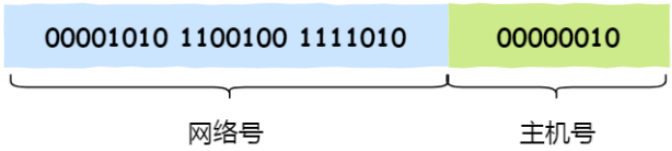
（4）CIDR无分类地址

这种方式不再有分类地址的概念，32 比特的 IP 地址被划分为两部分，前面是网络号，后面是主机号。

(i)如何划分网络号和主机号呢?

表示形式 a.b.c.d/x ，其中 /x 表示前 x 位属于网络号， x 的范围是 0 ~ 32 ，这就使得 IP 地址更加具有灵活性。

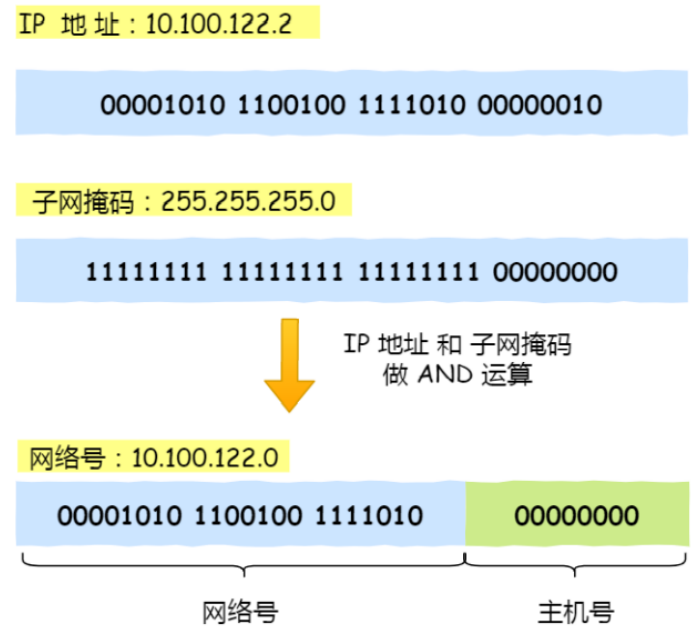
10.100.122.2/24



可用地址个数	254
子网掩码	255.255.255.0
网络号	10.100.122.0
第一个可用地址	10.100.122.1
最后可用地址	10.100.122.254
广播地址	10.100.122.255

还有另一种划分网络号与主机号形式，那就是子网掩码，掩码的意思就是掩盖掉主机号，剩余的就是网络号。

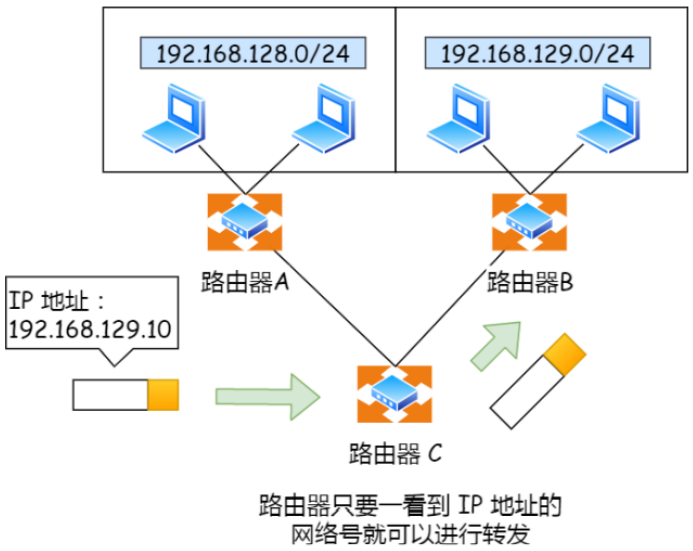
将子网掩码和 IP 地址按位计算 AND，就可得到网络号。



（ii）为什么要分离网络号和主机号?

因为两台计算机要通讯，首先要判断是否处于同一个广播域内，即网络地址是否相同。如果网络地址相同，表明接受方在本网络上，那么可以把数据包直接发送到目标主机。

路由器寻址工作中，也就是通过这样的方式来找到对应的网络号的，进而把数据包转发给对应的网络内。

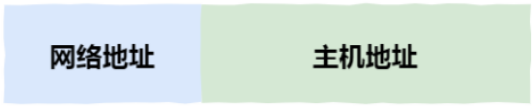


（iii）如何进行子网划分?

通过子网掩码划分出网络号和主机号，那实际上子网掩码还有一个作用，那就是划分子网。

子网划分实际上是将主机地址分为两个部分：子网网络地址和子网主机地址。形式如下：

未做子网划分的 ip 地址：



做子网划分后的 ip 地址 ：

- 未做子网划分的 ip 地址：网络地址＋主机地址
- 做子网划分后的 ip 地址：网络地址＋（子网网络地址＋子网主机地址）

假设对 C 类地址进行子网划分，网络地址 192.168.1.0，使用子网掩码 255.255.255.192 对其进行子网划分。

C 类地址中前 24 位是网络号，最后 8 位是主机号，根据子网掩码可知从 8 位主机号中借用 2 位作为子网号（对应4个子网）。

	网络号	主机号	
C 类网络地址	192.168.1	.0	
子网掩码 255.255.255.192	255.255.255	11	000000
子网划分	网络地址 24 位	子网网络地址 2 位	子网主机地址 6 位

划分后的4个子网如下：

子网号	网络地址	主机地址范围	广播地址
0	192.168.1.0	192.168.1.1 ~ 192.168.1.62	192.168.1.63
1	192.168.1.64	192.168.1.65 ~ 192.168.1.126	192.168.1.127
2	192.168.1.128	192.168.1.129 ~ 192.168.1.190	192.168.1.191
3	192.168.1.192	192.168.1.193 ~ 192.168.1.254	192.168.1.255

2、公有IP地址与私有IP地址

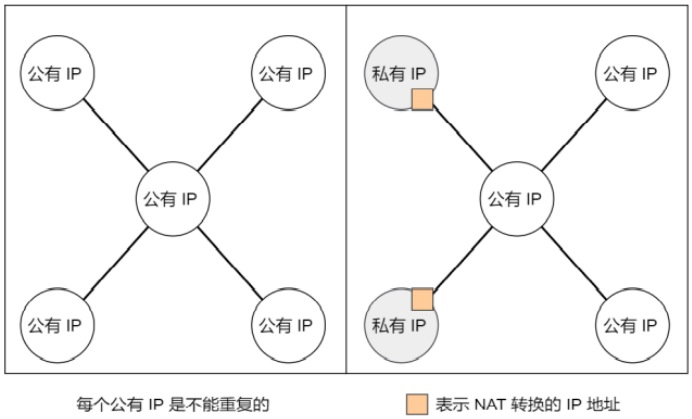
在A、B、C分类地址，实际又分公有IP地址和私有IP地址。

类别	IP 地址范围	最大主机数	私有 IP 地址范围
A	0.0.0.0 ~ 127.255.255.255	16777214	10.0.0.0 ~ 10.255.255.255
B	128.0.0.0 ~ 191.255.255.255	65534	172.16.0.0 ~ 172.31.255.255
C	192.0.0.0 ~ 223.255.255.255	254	192.168.0.0 ~ 192.168.255.255

平时我们办公室、家里、学校用的 IP 地址，一般都是私有 IP 地址。因为这些地址允许组织内部的 IT 人员自己管理、自己分配，而且可以重复。

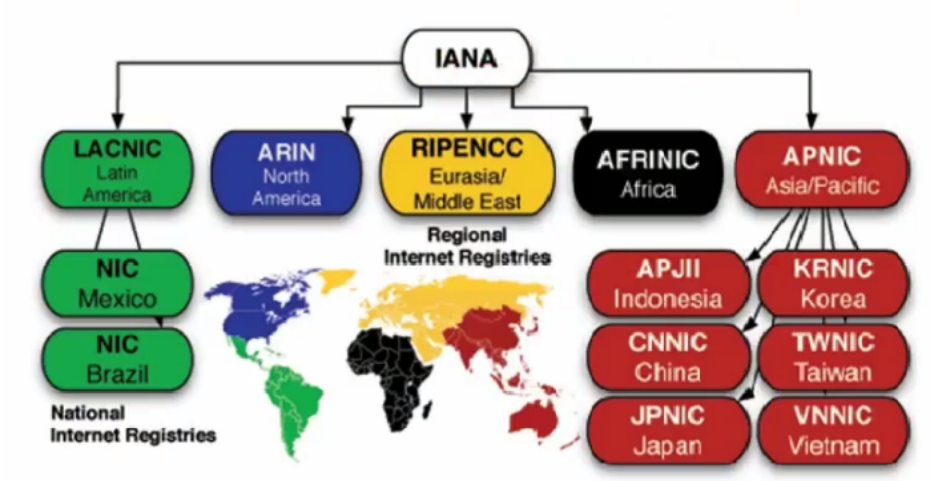
因此，你学校的某个私有 IP 地址和我学校的可以是一样的。 就像每个小区都有自己的楼编号和门牌号，你小区家可以叫 1 栋 101 号，我小区家也可以叫 1 栋 101，没有任何问题。但一旦出了小区，就需要带上中山路 666 号（公网 IP 地址），是国家统一分配的，不能两个小区都叫中山路 666。

所以，公有 IP 地址是有个组织统一分配的，假设你要开一个博客网站，那么你就需要去申请购买一个公有 IP，这样全世界的人才能访问。并且公有 IP 地址基本上要在整个互联网范围内保持唯一



那么公有IP地址由谁管理呢?

私有 IP 地址通常是内部的 IT 人员管理，公有 IP 地址是由 ICANN 组织管理，中文叫「互联网名称与数字地址分配机构」。 IANA 是 ICANN 的其中一个机构，它负责分配互联网 IP 地址，是按州的方式层层分配。

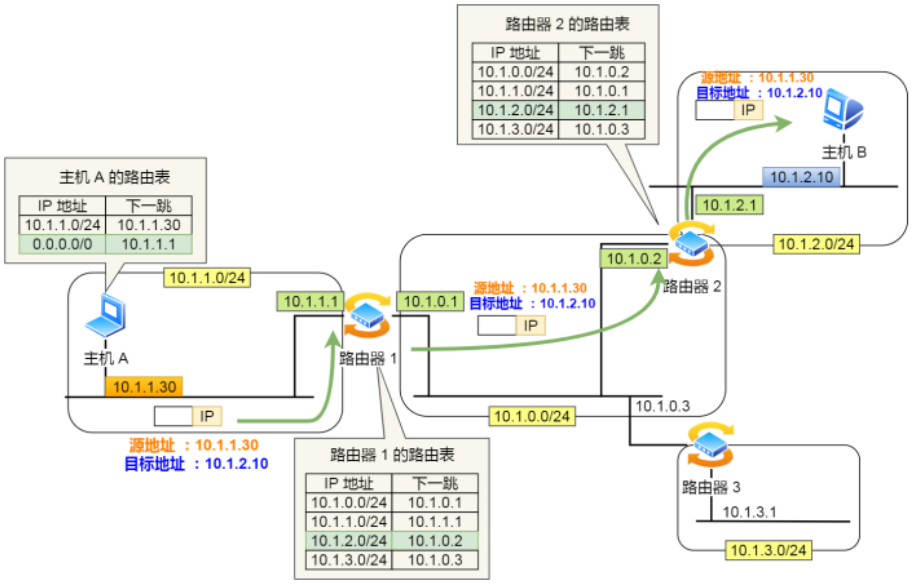


3、IP地址与路由控制

IP地址的网络地址这一部分是用于进行路由控制。路由控制表中记录着网络地址与下一步应该发送至路由器的地址。在主机和路由器上都会有各自的路由器控制表。

在发送 IP 包时，首先要确定 IP 包首部中的目标地址，再从路由控制表中找到与该地址具有相同网络地址的记录，根据该记录将 IP 包转发给相应的下一个路由器。如果路由控制表中存在多条相同网络地址的记录，就选择相同位数最多的网络地址，也就是最长匹配。

举例说明：



环回地址是不会流向网络：

环回地址是在同一台计算机上的程序之间进行网络通信时所使用的一个默认地址。 计算机使用一个特殊的 IP 地址 127.0.0.1 作为环回地址。

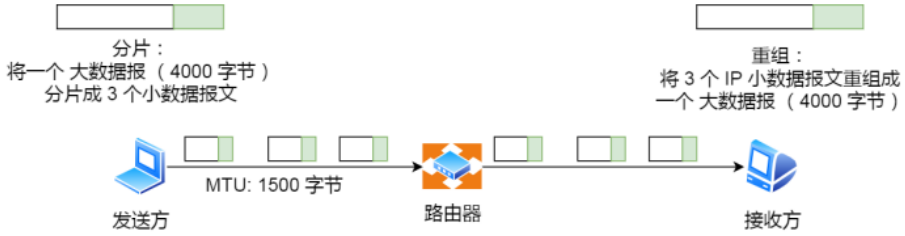
与该地址具有相同意义的是一个叫做 localhost 的主机名。使用这个 IP 或主机名时，数据包不会流向网络

4、IP分片与重组

每种数据链路的最大传输单元 MTU 都是不相同的，如 FDDI 数据链路 MTU 4352、以太网的 MTU 是 1500 字节等。

每种数据链路的 MTU 之所以不同，是因为每个不同类型的数据链路的使用目的不同。使用目的不同，可承载的 MTU 也就不同。

其中，我们最常见数据链路是以太网，它的 MTU 是 1500 字节。 那么当 IP 数据包大小大于 MTU 时， IP 数据包就会被分片。 经过分片之后的 IP 数据报在被重组的时候，只能由目标主机进行，路由器是不会进行重组的。



在分片传输中，一旦某个分片丢失，则会造成整个 IP 数据报作废，所以 TCP 引入了 MSS 也就是在 TCP 层进行分片不由 IP 层分片，那么对于 UDP 我们尽量不要发送一个大于 MTU 的数据报文。

5、IPV6基本认识

IPV6:更多的地址，更好的安全性和扩展性。

但IPV4和IPV6不能兼容。所以不但要我们电脑、手机之类的设备支持，还需要网络运营商对现有的设备进行升级，所以这可能是 IPv6 普及率比较慢的一个原因。

（1）IPV6的亮点

可分配地址变多

IPv6 可自动配置，即使没有 DHCP 服务器也可以实现自动分配IP地址，真是便捷到即插即用啊。

IPv6 包头包首部长度采用固定的值 40 字节，去掉了包头校验和，简化了首部结构，减轻了路由器负荷，大 大提高了传输的性能。

IPv6 有应对伪造 IP 地址的网络安全功能以及防止线路窃听的功能，大大提升了安全性。

（2）IPV6地址的标识方法

IPv4 地址长度共 32 位，是以每 8 位作为一组，并用点分十进制的表示方式。

IPv6 地址长度是 128 位，是以每 16 位作为一组，每组用冒号 「:」 隔开。

IPv6 用二进制数表示

```
1111111011011100：1011101010011000：0111011001010100：0011001000010000：1111111011011100：1011101010011000：0111011001010100：0011001000010000
```

IPv6 十六进制数表示

```
FEDC：BA98：7654：3210：FEDC：BA98：7654：3210
```

如果出现连续的 0 时还可以将这些 0 省略，并用两个冒号 「::」隔开。但是，一个 IP 地址中只允许出现一次两个连续的冒号。

IPv6 用二进制数表示

```
1111111011011100：1011101010011000：0111011001010100：0000000000000000：0000000000000000：0000000000000000：0000000000000000：0011001000010000
```

IPv6 十六进制数表示

```
FEDC:BA98:7654::3210
```

（3）IPV6地址的结构

IPv6 类似 IPv4，也是通过 IP 地址的前几位标识 IP 地址的种类。 IPv6 的地址主要有以下类型地址：

单播地址：用于一对一的通信；
组播地址：用于一对多的通信；
任播地址：用于通信最近的节点，最近的节点是由路由协议决定；
没有广播地址。

未定义	0000 ... 0000 (128 位)	::/128
环回地址	0000 ... 0001 (128 位)	::1/128
唯一本地地址	1111 110 ...	FC00::/7
链路本地单播地址	1111 1110 10 ...	FE80::/10
多播地址	1111 1111	FF00::/8
全局单播地址	其他	

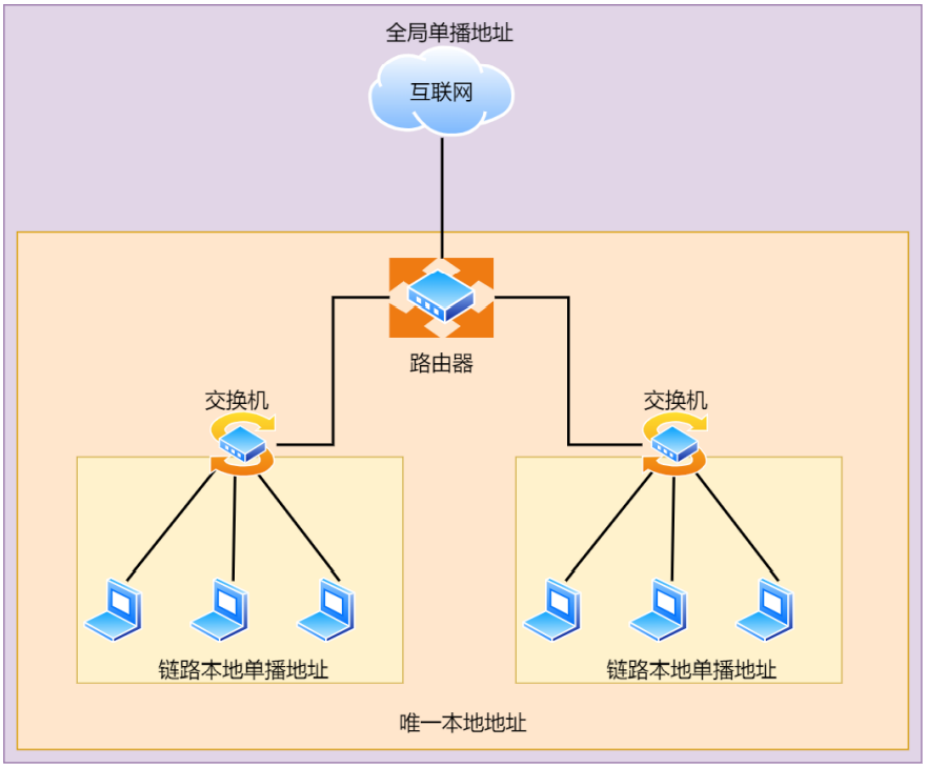
IPV6单播地址类型：

对于一对一通信的 IPv6 地址，主要划分了三类单播地址，每类地址的有效范围都不同。

在同一链路单播通信，不经过路由器，可以使用链路本地单播地址，IPv4 没有此类型

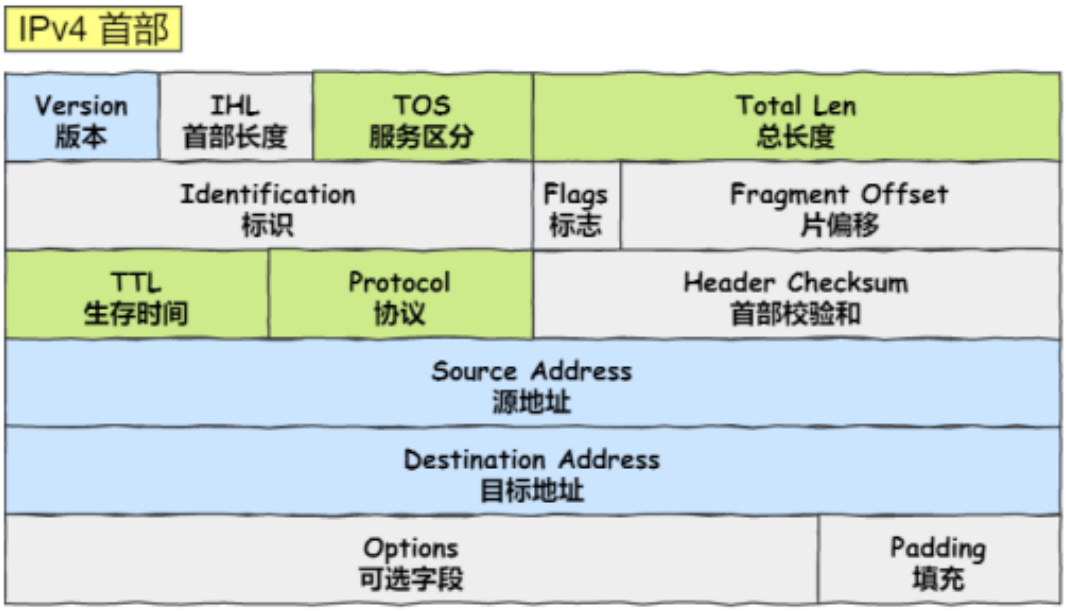
在内网里单播通信，可以使用唯一本地地址，相当于 IPv4 的私有 IP

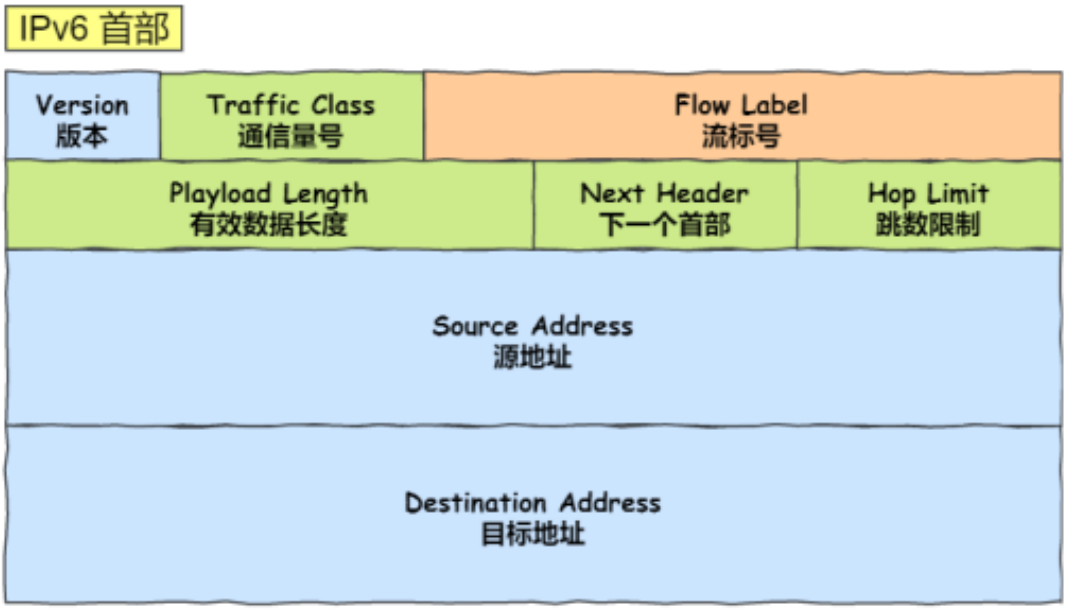
在互联网通信，可以使用全局单播地址，相当于 IPv4 的公有 IP



6、IPV4首部与IPV6首部

IPv4 首部与 IPv6 首部 IPv4 首部与 IPv6 首部的差异如下图：





其中：

IPV6相比IPV4的首部改进

取消了首部校验和字段：

因为在数据链路层和传输层都会校验，因此 IPv6 直接取消了 IP 的校验。

取消了分片/重新组装相关字段：

分片与重组是耗时的过程，IPv6 不允许在中间路由器进行分片与重组，这 种操作只能在源与目标主机，这将大大提高了路由器转发的速度。

取消选项字段：

选项字段不再是标准 IP 首部的一部分了，但它并没有消失，而是可能出现在 IPv6 首部中的 「下一个首部」指出的位置上。删除该选项字段使的 IPv6 的首部成为固定长度的 40 字节。

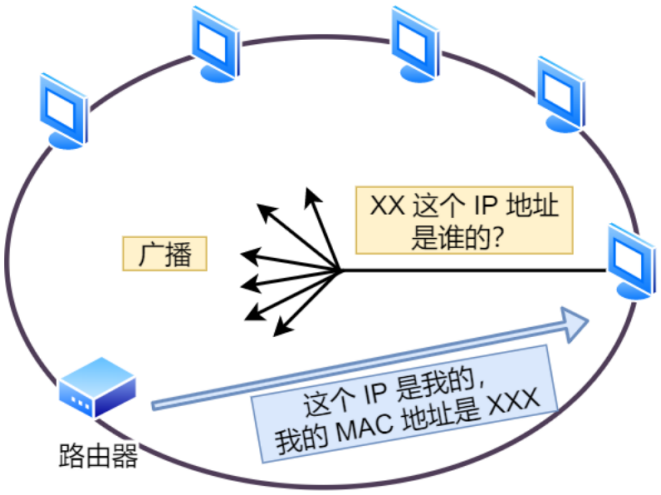
2、ARP与RARP协议

（1）ARP协议

在传输一个 IP 数据报的时候，确定了源 IP 地址和目标 IP 地址后，就会通过主机「路由表」确定 IP 数据包下一跳。然而，网络层的下一层是数据链路层，所以我们还要知道「下一跳」的 MAC 地址。

由于主机的路由表中可以找到下一跳的 IP 地址，所以可以通过 ARP 协议（Address Resolution Protocol，地址解析协议），求得下一跳的 MAC 地址。

ARP是如何知道对方的MAC地址的呢？ARP 是借助 ARP 请求与 ARP 响应两种类型的包确定 MAC 地址的。



同一个子网

主机会通过广播发送 ARP 请求，这个包中包含了想要知道的 MAC 地址的主机 IP 地址。

当同个链路中的所有设备收到 ARP 请求时，会去拆开 ARP 请求包里的内容，如果 ARP 请求包中的目标 IP 地址与自己的 IP 地址一致，那么这个设备就将自己的 MAC 地址塞入 ARP 响应包返回给主机。

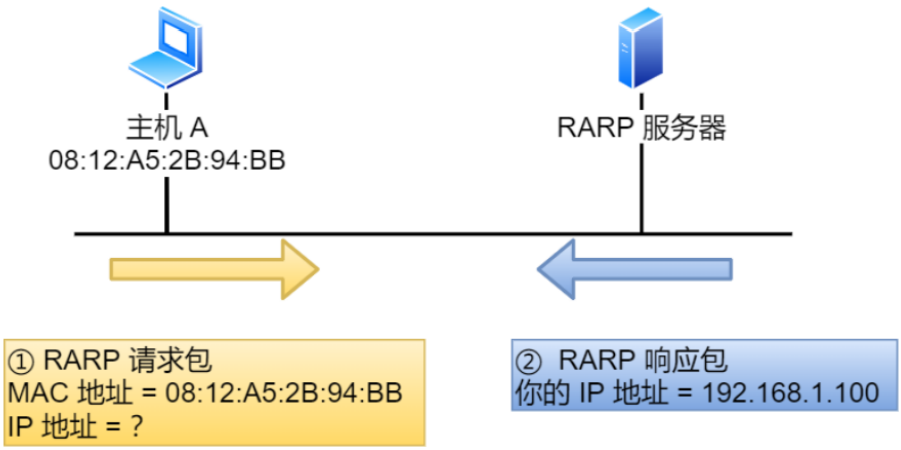
操作系统通常会把第一次通过 ARP 获取的 MAC 地址缓存起来，以便下次直接从缓存中找到对应 IP 地址的 MAC 地址。 不过，MAC 地址的缓存是有一定期限的，超过这个期限，缓存的内容将被清除。

（2）RARP协议

ARP 协议是已知 IP 地址求 MAC 地址，那 RARP 协议正好相反，它是已知 MAC 地址求 IP 地址。

例如将打印机服务器等小型嵌入式设备接入到网络时就经常会用得到。通常这需要架设一台 RARP 服务器，在这个服务器上注册设备的 MAC 地址及其 IP 地址。然后再将这个设备接入到网络

接着：



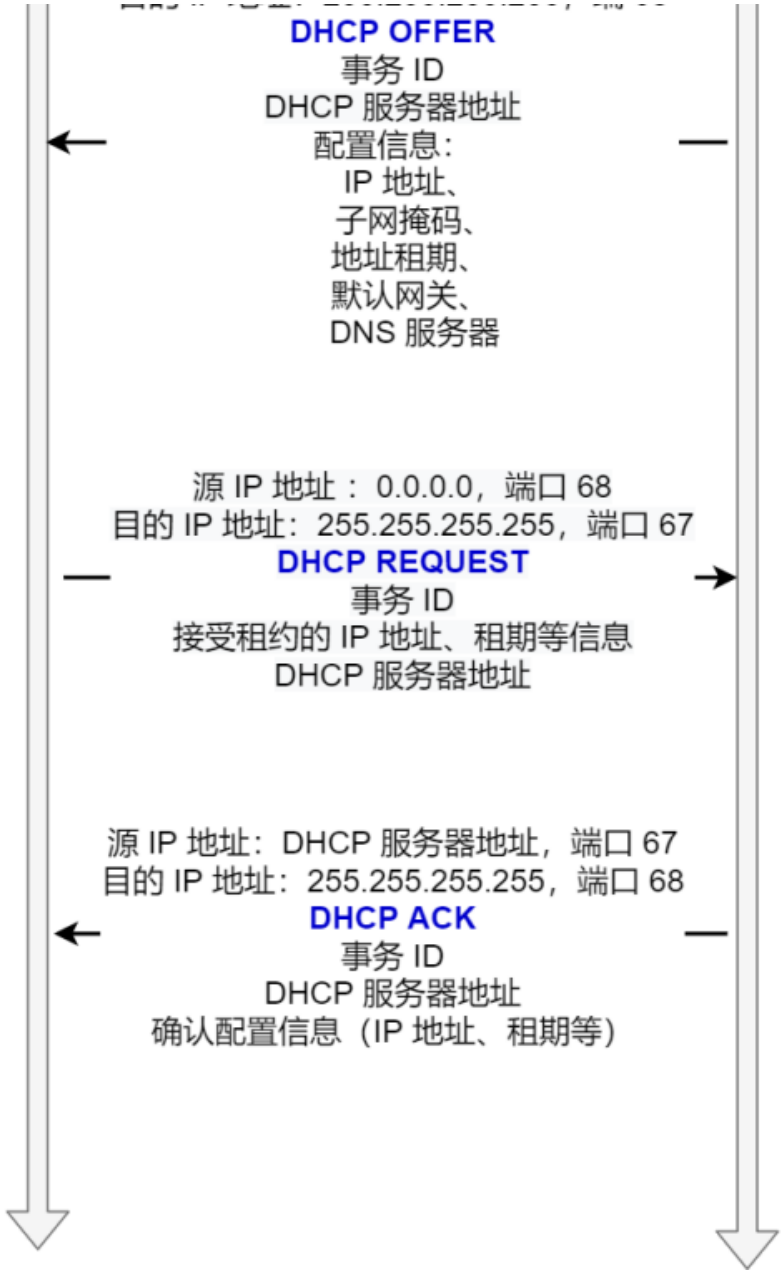
3、DHCP动态获取IP地址

DHCP(Dynamic Host Configuration Protocol,动态主机配置协议)。我们的电脑通常都是通过 DHCP 动态获取 IP 地址，大大省去了配 IP 信息繁琐的过程。

接下来，我们来看看我们的电脑是如何通过 4 个步骤的过程，获取到 IP 的：

- 客户端首先发起 **DHCP 发现报文（DHCP DISCOVER）** 的 IP 数据报，由于客户端没有 IP 地址，也不知道 DHCP 服务器的地址，所以使用的是 UDP **广播**通信，其使用的广播目的地址是 255.255.255.255（端口 67） 并且使用 0.0.0.0（端口 68） 作为源 IP 地址。DHCP 客户端将该 IP 数据报传递给链路层，链路层然后将帧广播到所有的网络中设备。
- DHCP 服务器收到 DHCP 发现报文时，用 **DHCP 提供报文（DHCP OFFER）** 向客户端做出响应。该报文仍然使用 IP 广播地址 255.255.255.255，该报文信息携带服务器提供可租约的 IP 地址、子网掩码、默认网关、DNS 服务器以及 **IP 地址租用期**。
- 客户端收到一个或多个服务器的 DHCP 提供报文后，从中选择一个服务器，并向选中的服务器发送 **DHCP 请求报文（DHCP REQUEST**进行响应，回显配置的参数。
- 最后，服务端用 **DHCP ACK 报文**对 DHCP 请求报文进行响应，应答所要求的参数。

一旦客户端收到 DHCP ACK 后，交互便完成了，并且客户端能够在租用期内使用 DHCP 服务器分配的 IP 地址。



说明： DHCP 客户端进程监听的是 68 端口号，DHCP 服务端进程监听的是 67 端口号。

如果租约的 DHCP IP 地址快期后，客户端会向服务器发送 DHCP 请求报文：

服务器如果同意继续租用：

则用 DHCP ACK 报文进行应答，客户端就会延长租期。

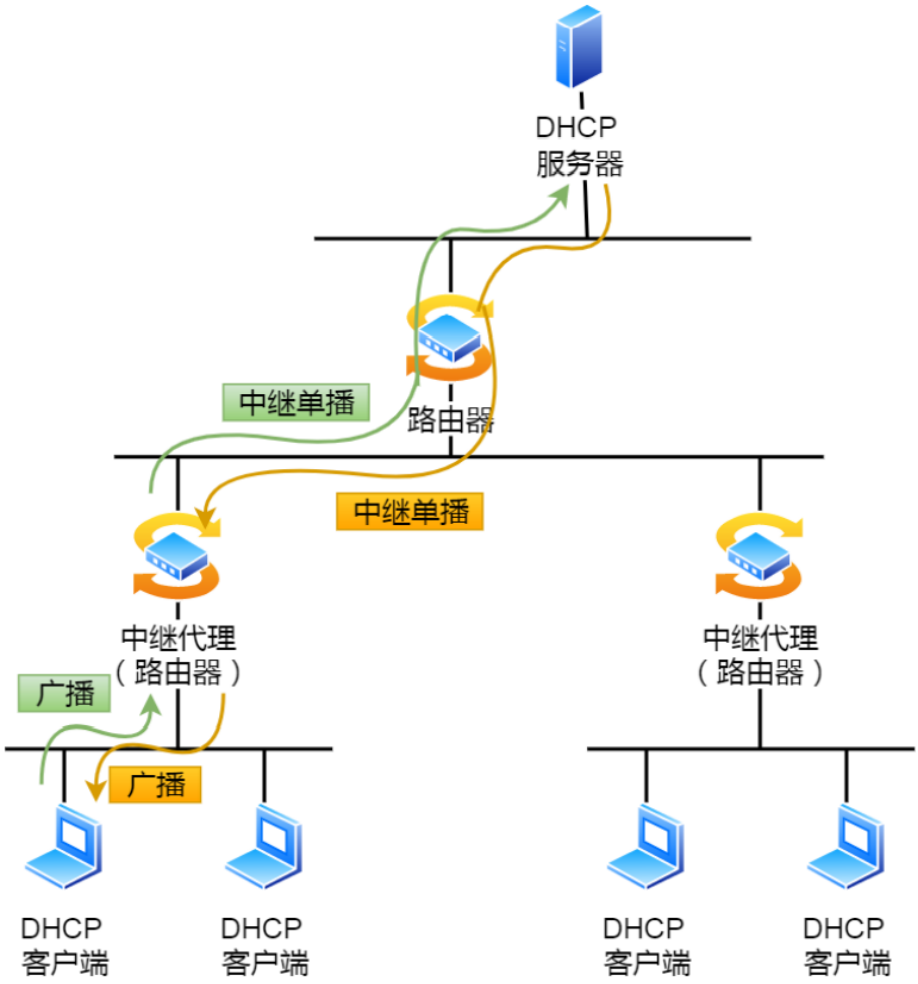
服务器如果不同意继续租用：

则用 DHCP NACK 报文，客户端就要停止使用租约的 IP 地址。 可以发现，DHCP 交互中，全程都是使用 UDP 广播通信。

疑问：

咦，用的是广播，那如果 DHCP 服务器和客户端不是在同一个局域网内，路由器又不会转发广播包，那不是每个网络 都要配一个 DHCP 服务器?

为了解决这一问题，就出现了 DHCP 中继代理。有了 DHCP 中继代理以后，对不同网段的 IP 地址分配也 可以由一个 DHCP 服务器统一进行管理。



DHCP 客户端会向 DHCP 中继代理发送 DHCP 请求包，而 DHCP 中继代理在收到这个广播包以后，再以单播的形式发给 DHCP 服务器。

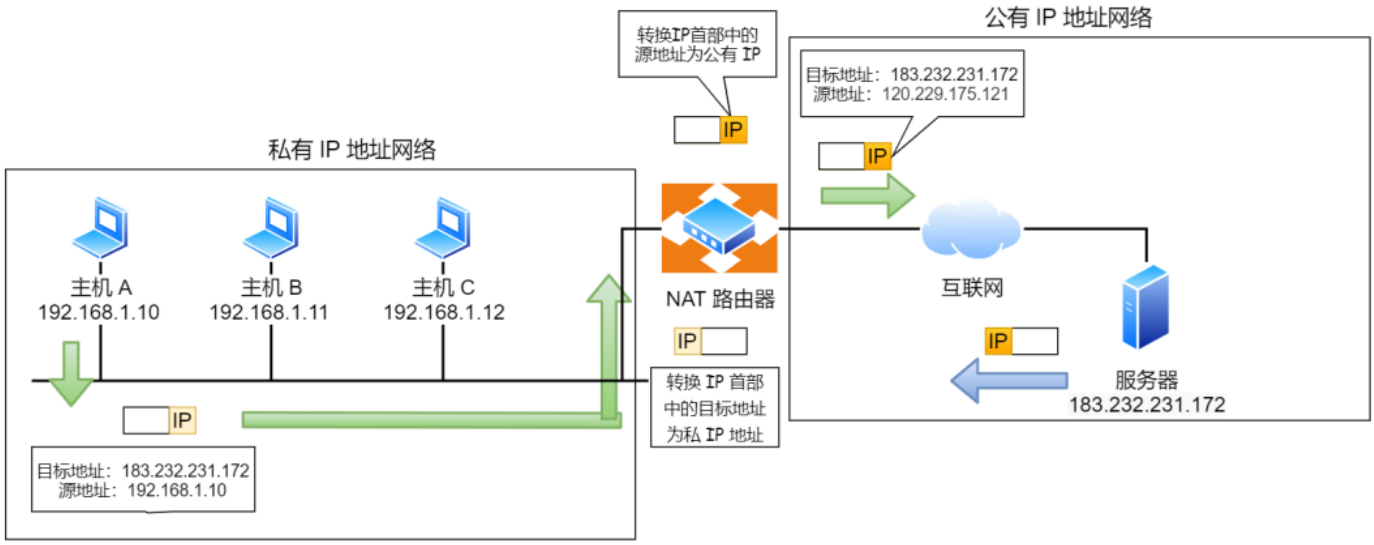
服务器端收到该包以后再向 DHCP 中继代理返回应答，并由 DHCP 中继代理将此包广播给 DHCP 客户端 。

因此，DHCP 服务器即使不在同一个链路上也可以实现统一分配和管理IP地址。

4、NAT(Network Address Translation, 网络地址转换)

IPv4 的地址是非常紧缺的，在前面我们也提到可以通过无分类地址来减缓 IPv4 地址耗尽的速度，但是互联网的用户增速是非常惊人的，所以 IPv4 地址依然有被耗尽的危险。

于是，提出了一种网络地址转换 NAT 的方法，再次缓解了 IPv4 地址耗尽的问题。 简单的来说 NAT 就是同个公司、家庭、教室内的主机对外部通信时，把私有 IP 地址转换成公有 IP 地址。

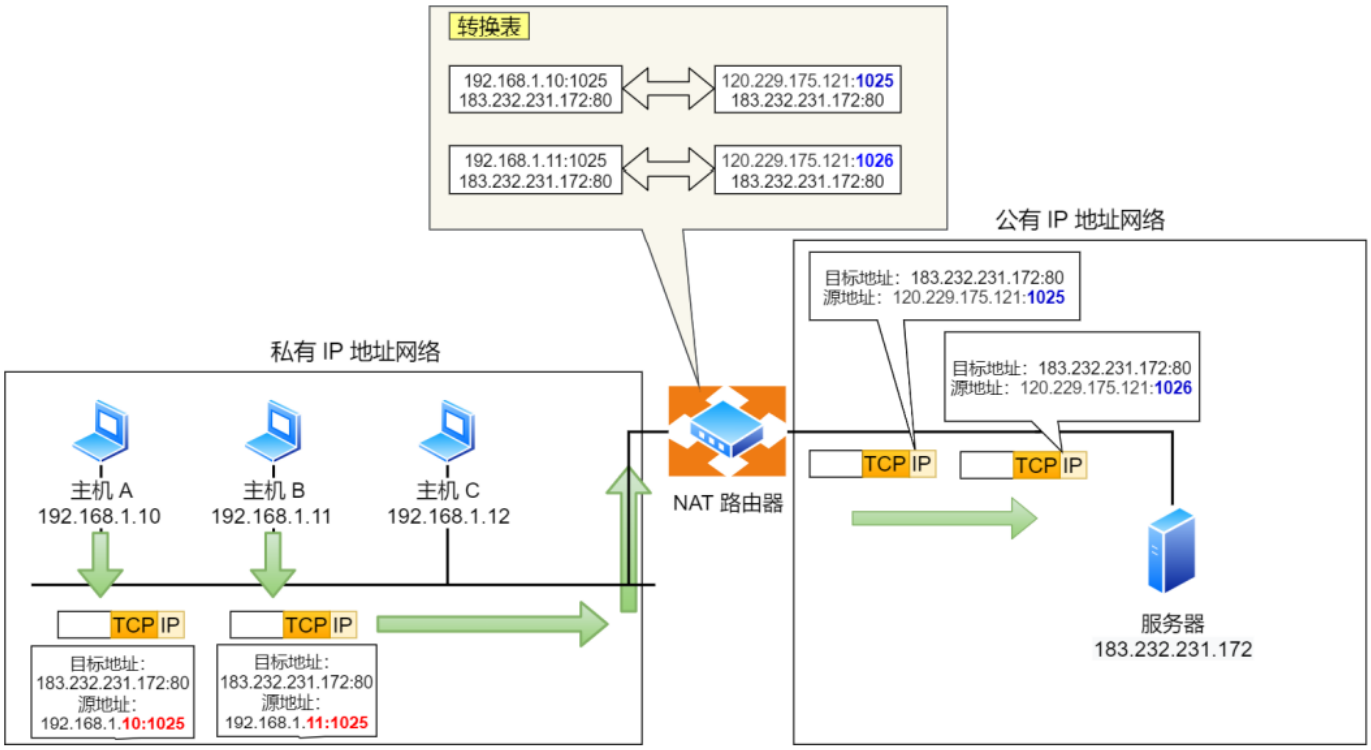


疑问：

那不是 N 个私有 IP 地址，你就要 N 个公有 IP 地址？这怎么就缓解了 IPv4 地址耗尽的问题？这不瞎扯吗？

确实是，普通的 NAT 转换没什么意义。 由于绝大多数的网络应用都是使用传输层协议 TCP 或 UDP 来传输数据的。 因此，可以把 IP 地址 + 端口号一起进行转换。 这样，就用一个全球 IP 地址就可以了，这种转换技术就叫网络地址与端口转换 NAPT。

来看看下面的图解：



图中有两个客户端 192.168.1.10 和 192.168.1.11 同时与服务器 183.232.231.172 进行通信，并且这两个客户端的本地端口都是 1025。 此时，两个私有 IP 地址都转换 IP 地址为公有地址 120.229.175.121，但是以不同的端口号作为区分。

于是，生成一个 NAPT 路由器的转换表，就可以正确地转换地址跟端口的组合，令客户端 A、B 能同时与服务器之间进行通信。 这种转换表在 NAT 路由器上自动生成。例如，在 TCP 的情况下，建立 TCP 连接首次握手时的 SYN 包一经发出， 就会生成这个表。而后又随着收到关闭连接时发出 FIN 包的确认应答从表中被删除。

（1）NAT有缺点吗?

由于 NAT/NAPT 都依赖于自己的转换表，因此会有以下的问题：

外部无法主动与 NAT 内部服务器建立连接，因为 NAPT 转换表没有转换记录。

转换表的生成与转换操作都会产生性能开销。

通信过程中，如果 NAT 路由器重启了，所有的 TCP 连接都将被重置。

（2）如何解决NAT潜在的问题呢?

改用IPV6：

IPv6 可用范围非常大，以至于每台设备都可以配置一个公有 IP 地址，就不搞那么多花里胡哨的地址转换了，但是 IPv6 普及速度还需要一些时间。

NAT 穿透技术：

NAT 穿越技术能够让网络应用程序主动发现自己位于 NAT 设备之后，并且会主动获得 NAT 设备的公有 IP，并为自己建立端口映射条目，注意这些都是 NAT设备后的应用程序自动完成的。 也就是说，客户端主动从 NAT 设备获取公有 IP 地址，然后自己建立端口映射条目，然后用这个条目对外通信， 就不需要 NAT 设备来进行转换了。

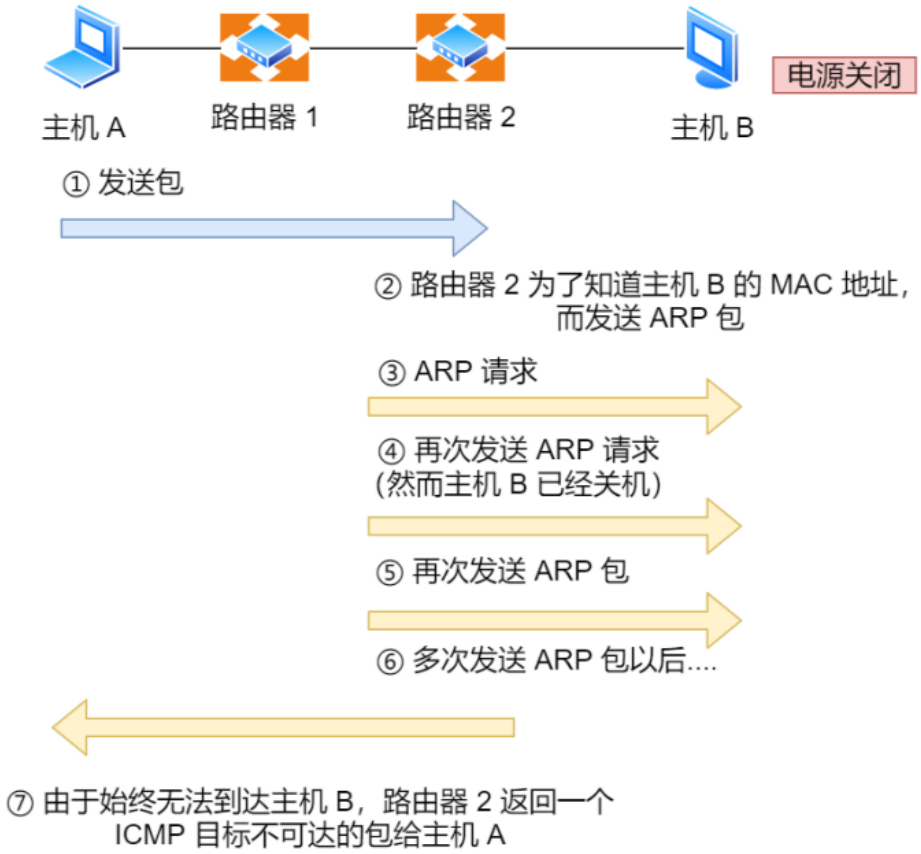
5、ICMP互联网控制报文协议

ICMP（ Internet Control Message Protocol，也就是互联网控制报文协议）。

网络包在复杂的网络传输环境里，常常会遇到各种问题。 当遇到问题的时候，总不能死个不明不白，没头没脑的作风不是计算机网络的风格。所以需要传出消息，报告遇到 了什么问题，这样才可以调整传输策略，以此来控制整个局面。

（1）ICMP功能都有啥

ICMP的主要功能包括：确认 IP 包是否成功送达目标地址、报告发送过程中 IP 包被废弃的原因和改善网络设置等。在 IP 通信中如果某个 IP 包因为某种原因未能达到目标地址，那么这个具体的原因将由 ICMP 负责通知。



ICMP 的这种通知消息会使用IP进行发送 。 因此，从路由器 2 返回的 ICMP 包会按照往常的路由控制先经过路由器 1 再转发给主机A 。收到该 ICMP 包的主机 A 则分解 ICMP 的首部和数据域以后得知具体发生问题的原因。

（2）ICMP的类型

ICMP大致可以分为两大类：

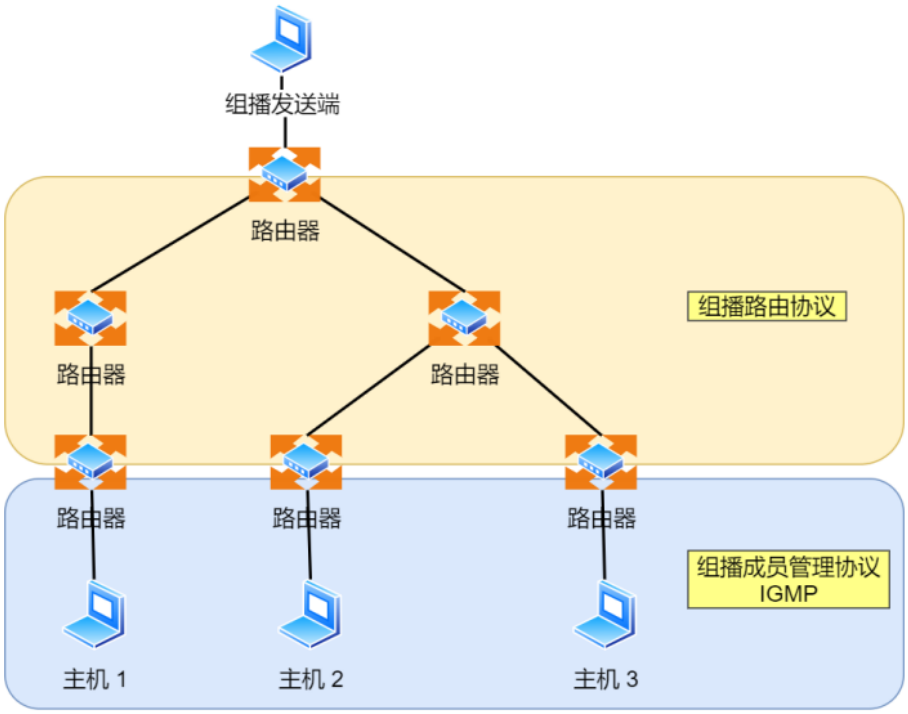
- 一类是用于诊断的查询消息，也就是「查询报文类型」
- 一类是通知出错原因的错误消息，也就是「差错报文类型」

ICMP 类型		
	内容	种类
0	回送应答（Echo Reply）	查询报文类型
3	目标不可达（Destination Unreachable）	差错报文类型
4	原点抑制（Source Quench）	差错报文类型
5	重定向或改变路由（Redirect）	差错报文类型
8	回送请求（Echo Request）	查询报文类型
11	超时（Time Exceeded）	差错报文类型

6、IGMP因特网组管理协议

在前面我们知道了组播地址，也就是 D 类地址，既然是组播，那就说明是只有一组的主机能收到数据包，不在一组的主机不能收到数组包，怎么管理是否是在一组呢?

那么，就需要 IGMP 协议了。



IGMP 是因特网组管理协议，工作在主机（组播成员）和最后一跳路由之间，如上图中的蓝色部分。

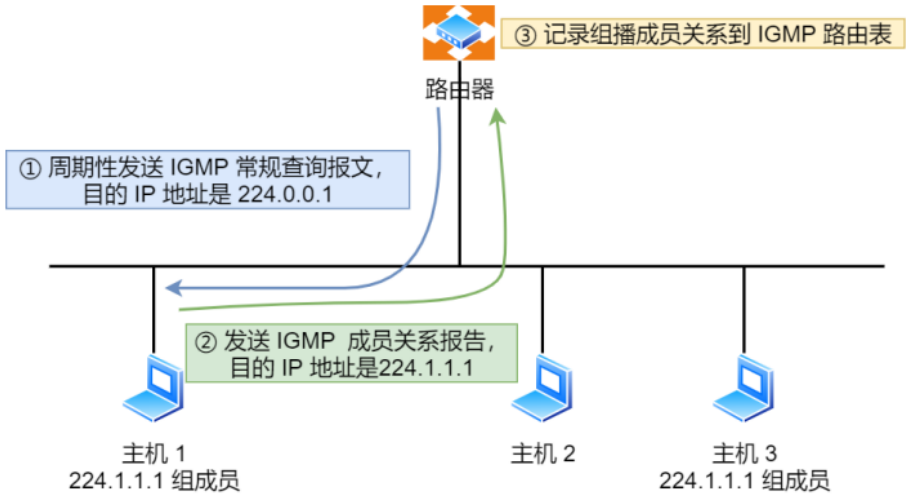
IGMP 报文向路由器申请加入和退出组播组，默认情况下路由器是不会转发组播包到连接中的主机，除非主机通过 IGMP 加入到组播组，主机申请加入到组播组时，路由器就会记录 IGMP 路由器表，路由器后续就会转 发组播包到对应的主机了。

IGMP 报文采用 IP 封装，IP 头部的协议号为 2，而且 TTL 字段值通常为 1，因为 IGMP 是工作在主机与连接的路由器之间。

（1）IGMP的工作机制

IGMP 分为了三个版本分别是，IGMPv1、IGMPv2、IGMPv3。 接下来，以 IGMPv2 作为例子，说说常规查询与响应和离开组播组这两个工作机制。

常规查询与响应机制：



（i）路由器会周期性发送目的地址为 224.0.0.1 （表示同一网段内所有主机和路由器） IGMP 常规查询报文。

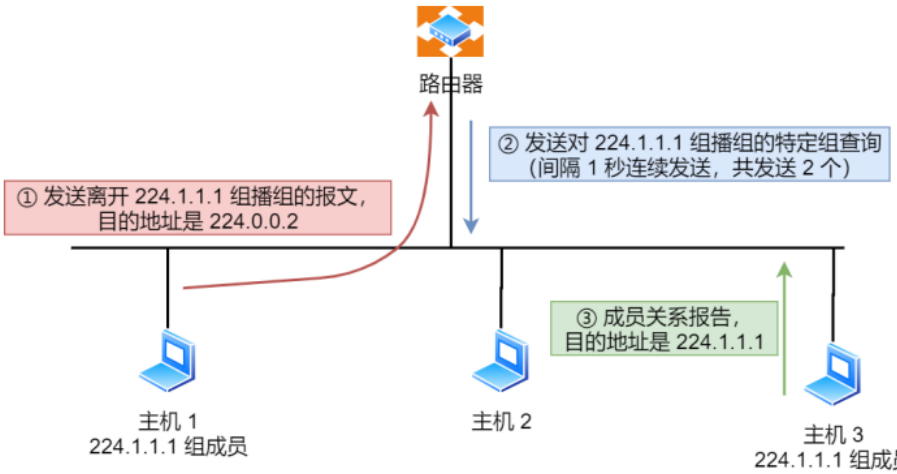
（ii）主机1 和 主机 3 收到这个查询，随后会启动「报告延迟计时器」，计时器的时间是随机的，通常是 0~10 秒，计时器超时后主机就会发送 IGMP 成员关系报告报文（源 IP 地址为自己主机的 IP 地址，目的 IP 地址为组播地址）。如果在定时器超时之前，收到同一个组内的其他主机发送的成员关系报告报文，则自己不再发 送，这样可以减少网络中多余的 IGMP 报文数。

（iii）路由器收到主机的成员关系报文后，就会在 IGMP 路由表中加入该组播组，后续网络中一旦该组播地址的数据到达路由器，它会把数据包转发出去

离开组播组工作机制：

情况1：

网段中仍有该组播组



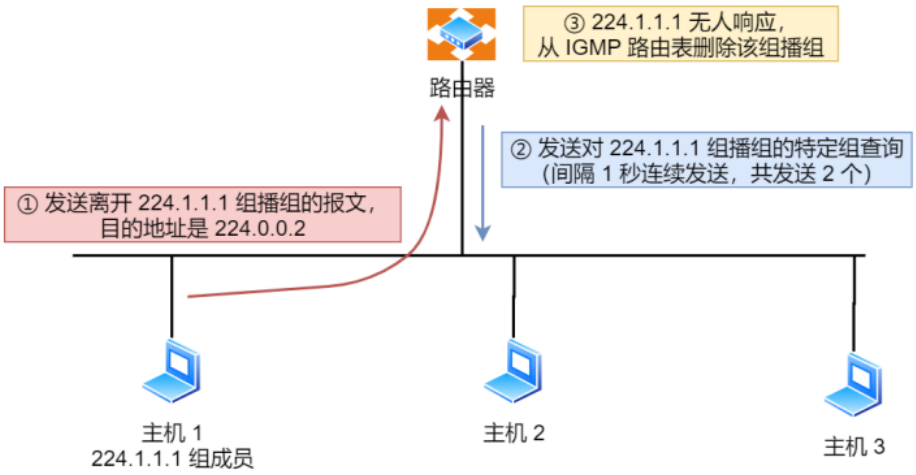
（i）主机 1 要离开组 224.1.1.1，发送 IGMPv2 离组报文，报文的目的地址是 224.0.0.2（表示发向网段内的所有路由器）

（ii）路由器 收到该报文后，以 1 秒为间隔连续发送 IGMP 特定组查询报文（共计发送 2 个），以便确认该网络是否还有 224.1.1.1 组的其他成员。

（iii）主机 3 仍然是组 224.1.1.1 的成员，因此它立即响应这个特定组查询。路由器知道该网络中仍然存在该组播组的成员，于是继续向该网络转发 224.1.1.1 的组播数据包。

情况2：

网段中没有该组播组



（i）主机 1 要离开组播组 224.1.1.1，发送 IGMP 离组报文。

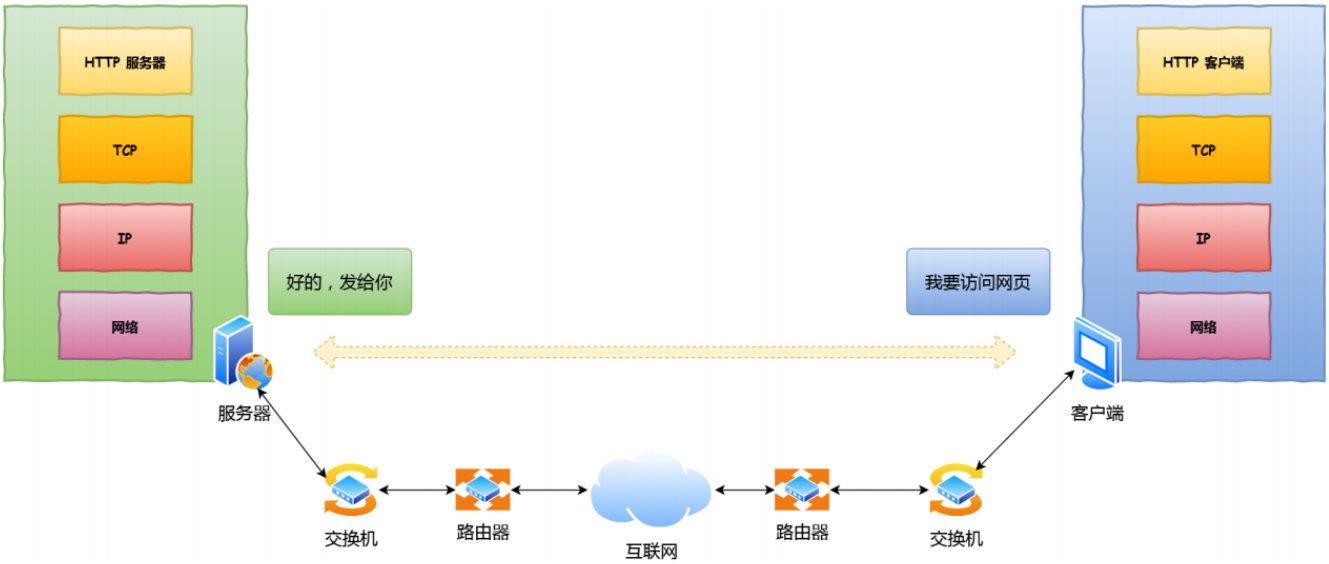
（ii）路由器收到该报文后，以 1 秒为间隔连续发送 IGMP 特定组查询报文（共计发送 2 个）。此时在该网段内，组 224.1.1.1 已经没有其他成员了，因此没有主机响应这个查询。

（iii）一定时间后，路由器认为该网段中已经没有 224.1.1.1 组播组成员了，将不会再向这个网段转发该组播地址的 数据包。

注：组播地址**不是**用于机器ip地址的，因为组播地址没有网络号和主机号，所以跟dhcp没关系。组播地址一般是用于 udp协议，机器发送UDP组播数据时，目标地址填的是组播地址，那么在组播组内的机器都能收到数据包。 是否加入组播组和离开组播组，是由socket一个接口实现的，主机ip是不用改变的。

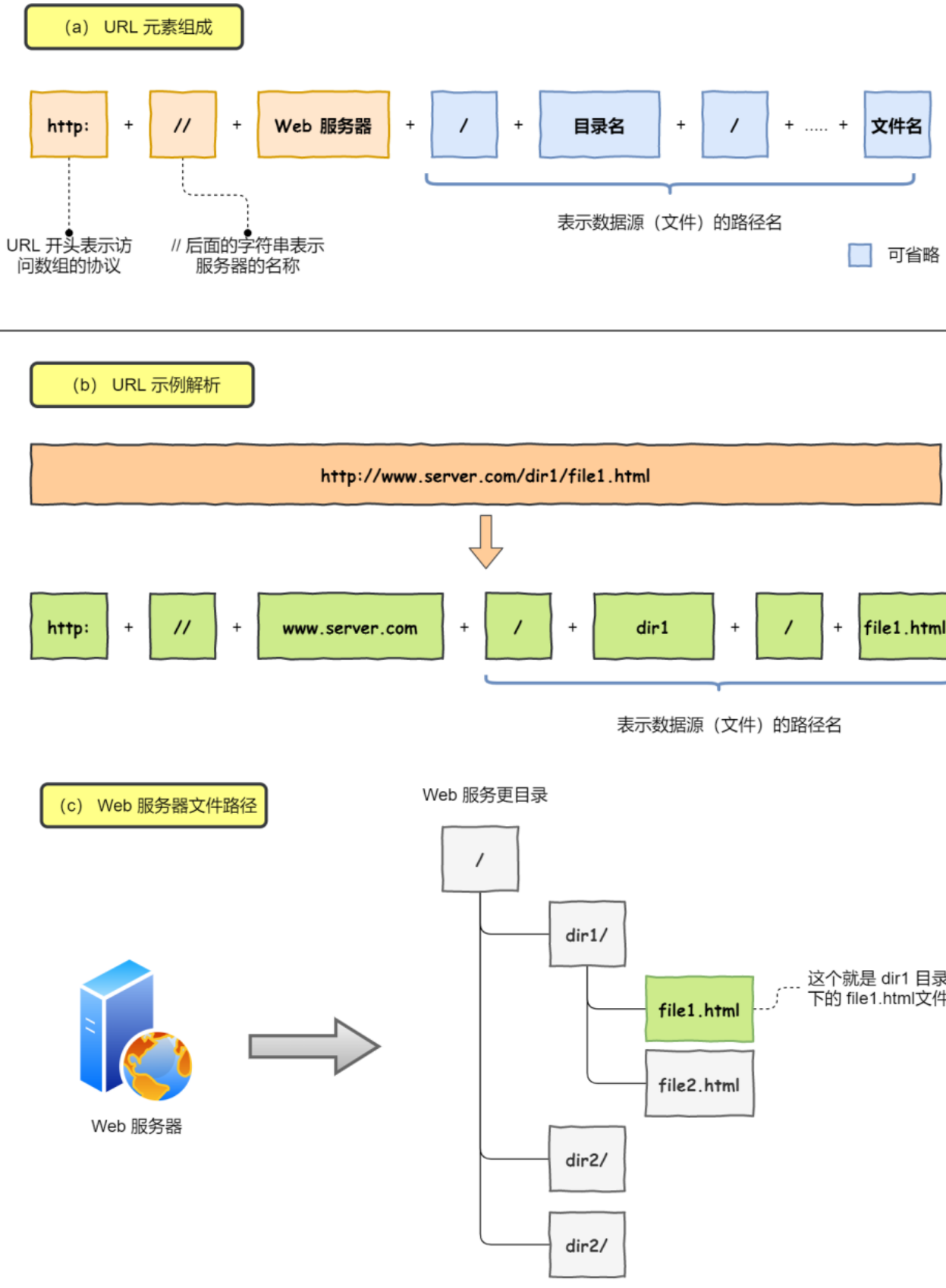
在浏览器中输入URL并按下回车之后会发生什么

> 经过了上面内容的学习之后，再看这道题目的剖析



解析URL，生成发送给WEB服务器的请求信息。

1、解析URL

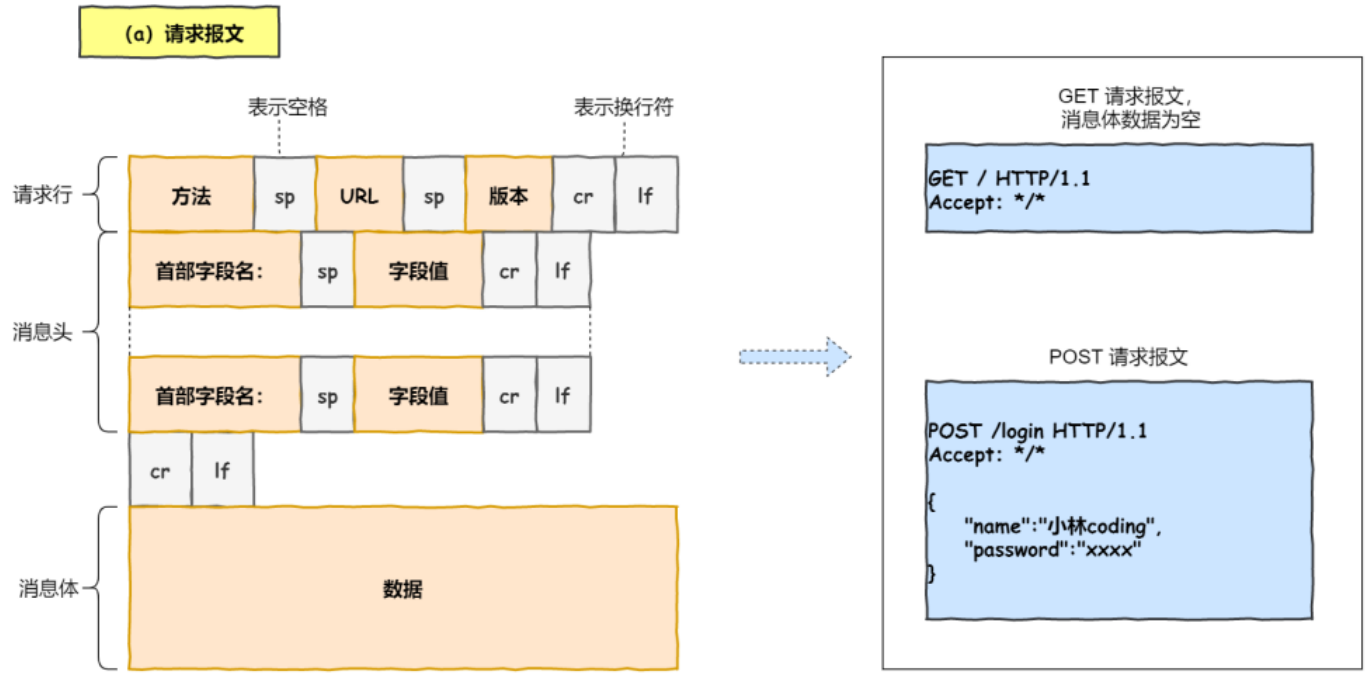


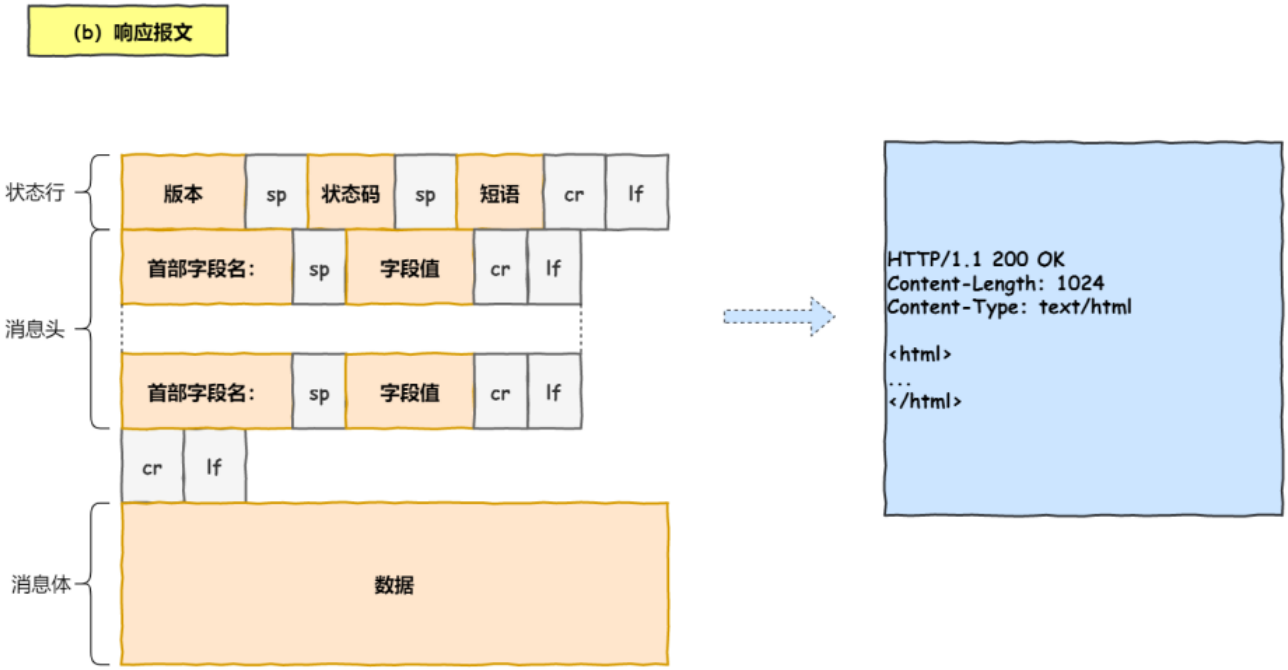
所以长长的 URL 实际上是请求服务器里的文件资源。

当图（a）示意图中的蓝色部分元素省略后（即：没有路径名），代表访问根目录下事先设置的默认文件，也就是`/index.html`或者`/default.html`这些文件，这样就不会发生混乱了。

2、生成HTTP请求信息

对 URL 进行解析之后，浏览器确定了 Web 服务器和文件名，接下来就是根据这些信息来生成 HTTP 请求消息了。





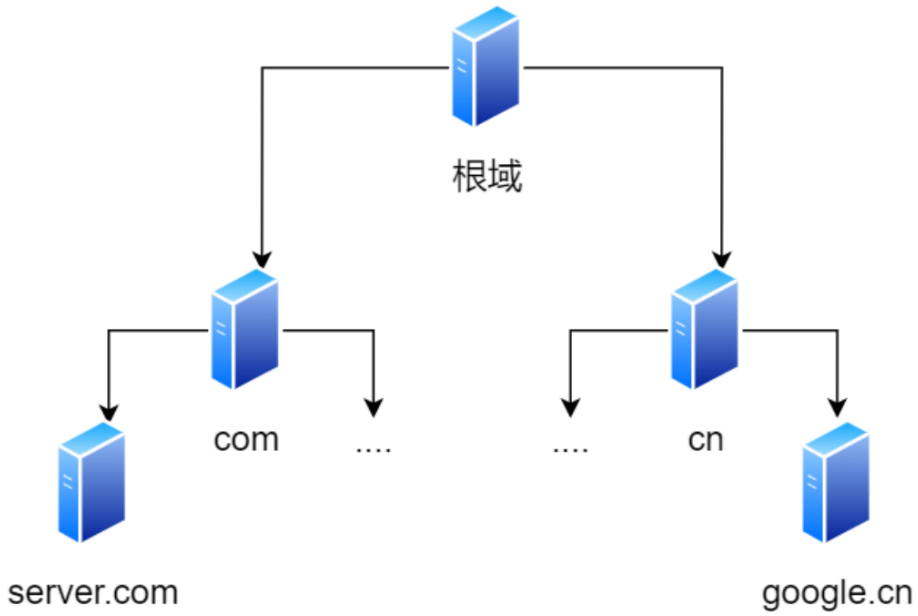
3、DNS域名解析

通过浏览器解析 URL 并生成 HTTP 消息后，需要委托操作系统将消息发送给 Web 服务器。

但在发送之前，还有一项工作需要完成，那就是查询服务器域名对应的 IP 地址，因为委托操作系统发送消息时，必须提供通信对象的 IP 地址。

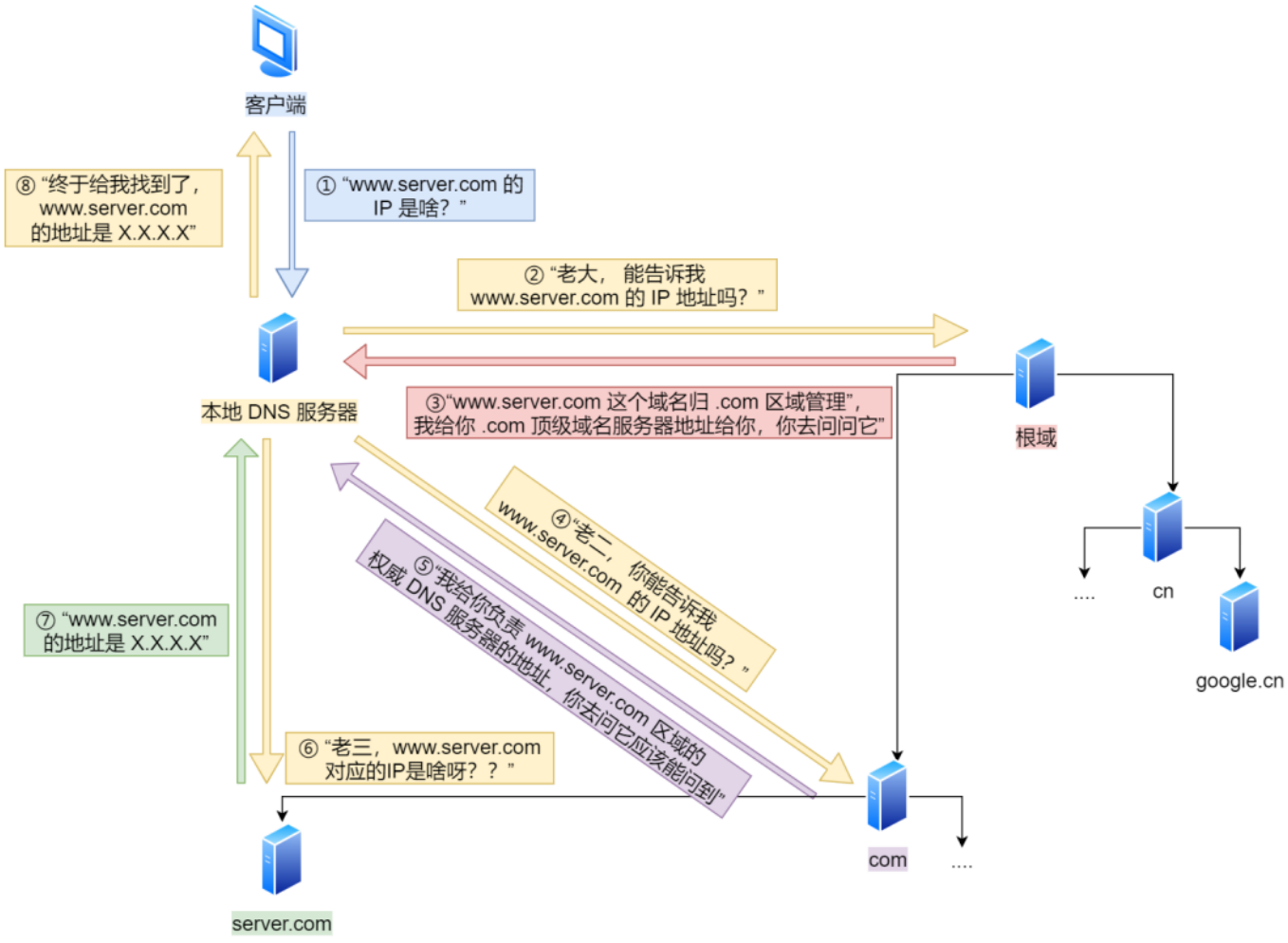
有一种服务器就专门保存了 Web 服务器域名与 IP 的对应关系，它就是 DNS 服务器。

（1）域名的层级关系



根域的 DNS 服务器信息保存在互联网中所有的 DNS 服务器中。这样一来，任何 DNS 服务器就都可以找到并访问根域 DNS 服务器了。 因此，客户端只要能够找到任意一台 DNS 服务器，就可以通过它找到根域 DNS 服务器，然后再一路顺藤摸瓜找到位于下层的某台目标 DNS 服务器。

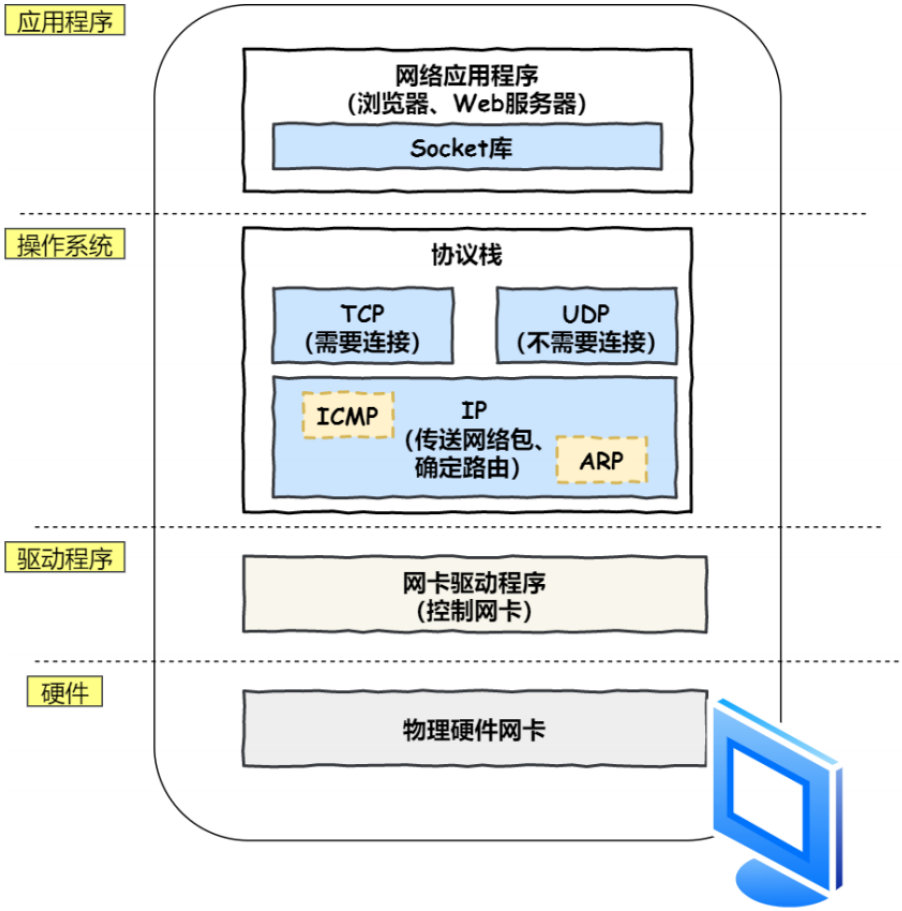
如下图所示：



4、协议栈：如何发送数据包

数据包表示：“DNS 老大哥厉害呀，找到了目的地了！我还是很迷茫呀，我要发出去，接下来我需要谁的帮助呢?”

通过 DNS 获取到 IP 后，就可以把 HTTP 的传输工作交给操作系统中的协议栈。协议栈的内部分为几个部分，分别承担不同的工作。上下关系是有一定的规则的，上面的部分会向下面的部分委托工作，下面的部分收到委托的工作并执行。



应用程序（浏览器）通过调用 `Socket` 库，来委托协议栈工作。协议栈的上半部分有两块，分别是负责收发数据的 TCP 和 UDP 协议，它们两会接受应用层的委托执行收发数据的操作。

协议栈的下面一半是用 IP 协议控制网络包收发操作，在互联网上传数据时，数据会被切分成一块块的网络包，而将网络包发送给对方的操作就是由 IP 负责的。 此外 IP 中还包括 ICMP 协议和 ARP 协议。

- ICMP 用于告知网络包传送过程中产生的错误以及各种控制信息
- ARP 用于根据 IP 地址查询相应的以太网 MAC 地址

IP 下面的网卡驱动程序负责控制网卡硬件，而最下面的网卡则负责完成实际的收发操作，也就是对网线中的信号执行发送和接收操作。

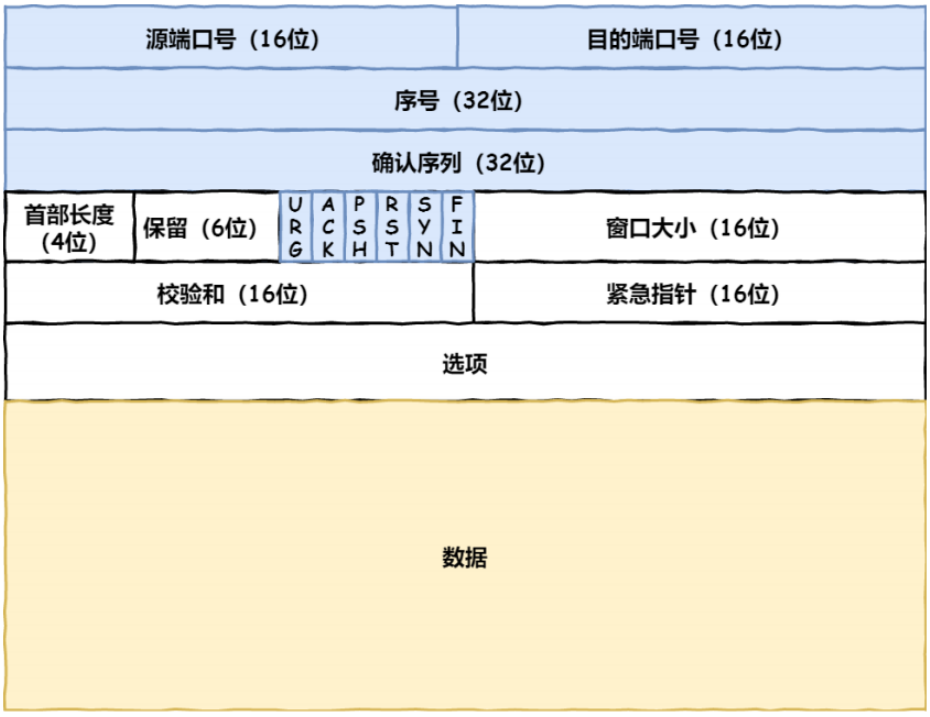
5、可靠传输—TCP

数据包看了这份指南表示：“原来我需要那么多大佬的协助啊，那我先去找找 TCP 大佬！”

（1）TCP包头格式

HTTP 是基于 TCP 协议传输的，所以在这我们先了解下 TCP 协议。

TCP包头格式：



首先，**源端口号** 和 **目标端口号**是不可少的，如果没有这两个端口号，数据就不知道应该发给哪个应用。

接下来有**包的序号**，这个是为了解决包乱序的问题。

还有应该有的是**确认号**，目的是确认发出去对方是否有收到。如果没有收到就应该重新发送，直到送达，这个是为了解决**不丢包**的问题。

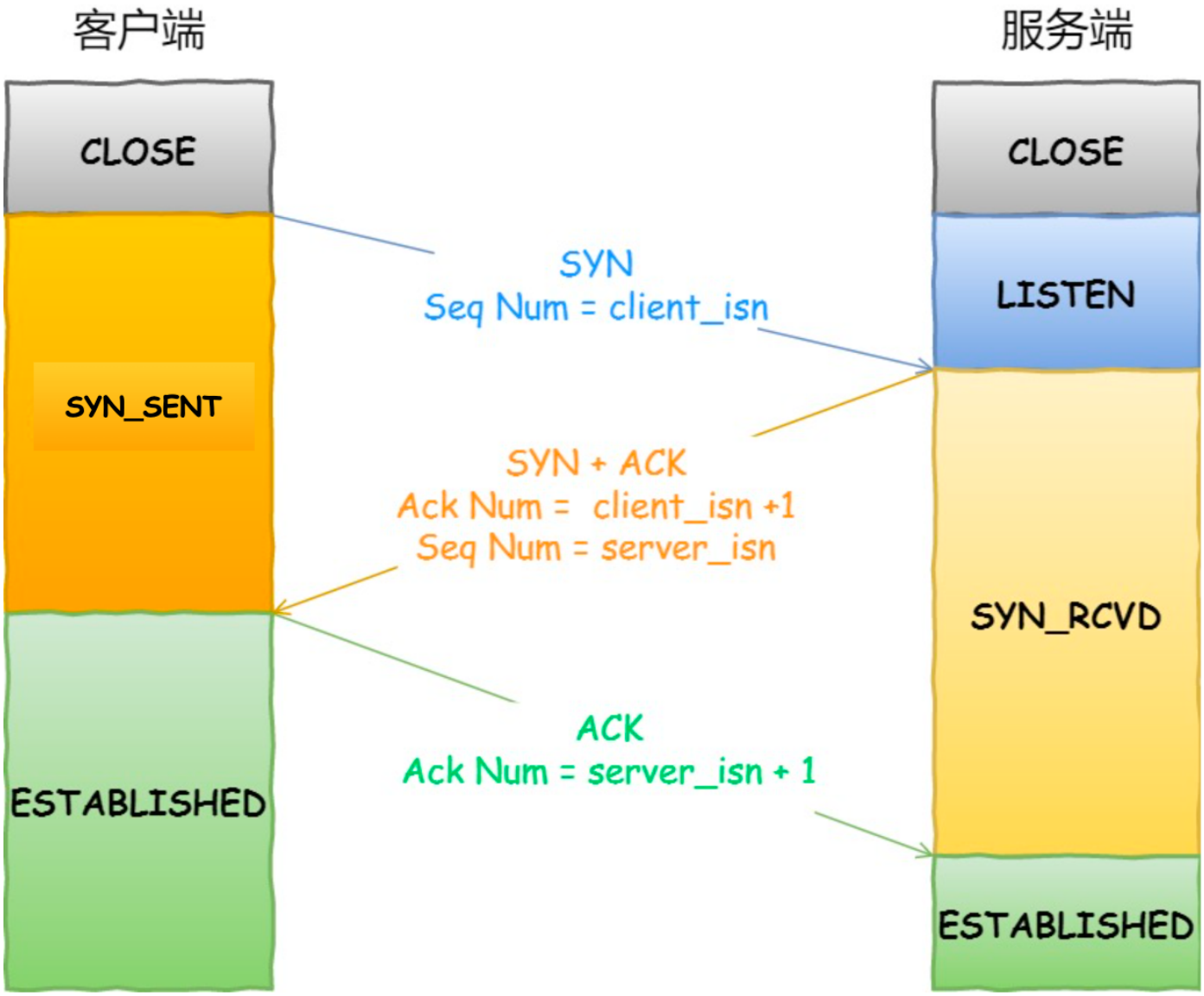
接下来还有一些**状态位**。例如 SYN 是发起一个连接， ACK 是回复， RST 是重新连接， FIN 是结束连接等。TCP 是面向连接的，因而双方要维护连接的状态，这些带状态位的包的发送，会引起双方的状态变更。

还有一个重要的就是**窗口大小**。TCP 要做**流量控制**，通信双方各声明一个窗口（缓存大小），标识自己当前能够的处理能力，别发送的太快，撑死我，也别发的太慢，饿死我。

除了做流量控制以外，TCP还会做**拥塞控制**，对于真正的通路堵车不堵车，它无能为力，唯一能做的就是控制自己，也即控制发送的速度。不能改变世界，就改变自己嘛。

（2）TCP 传输数据之前，要先三次握手建立连接

这个所谓的「连接」，只是双方计算机里维护一个状态机，在连接建立的过程中，双方的状态变化时序图就像这样。



一开始，客户端和服务端都处于 CLOSED 状态。先是服务端主动监听某个端口，处于 LISTEN 状态。

然后客户端主动发起连接 SYN ，之后处于 SYN-SENT 状态。

服务端收到发起的连接，返回 SYN ，并且 ACK 客户端的 SYN ，之后处于 SYN-RCVD 状态。

客户端收到服务端发送的 SYN 和 ACK 之后，发送 ACK 的 ACK ，之后处于 ESTABLISHED 状态， 因为它一发一收成功了。

服务端收到 ACK 的 ACK 之后，处于 ESTABLISHED 状态，因为它也一发一收了。

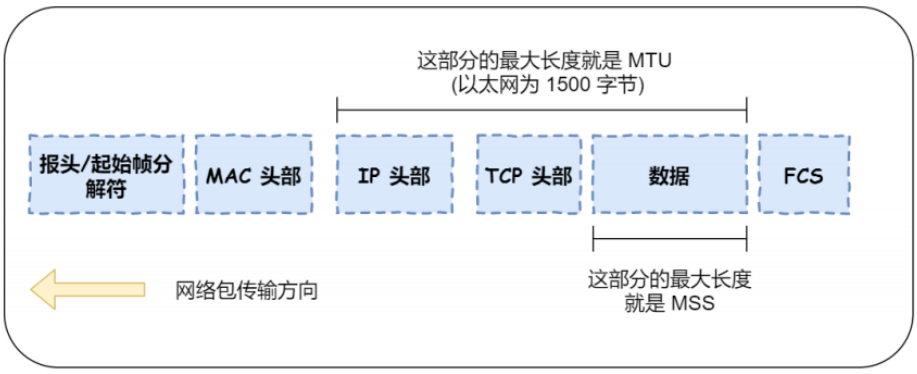
所以三次握手目的是**保证双方都有发送和接收的能力**。

（3）如何查看TCP的连接状态?

TCP 的连接状态查看，在 Linux 可以通过 netstat -napt 命令查看。

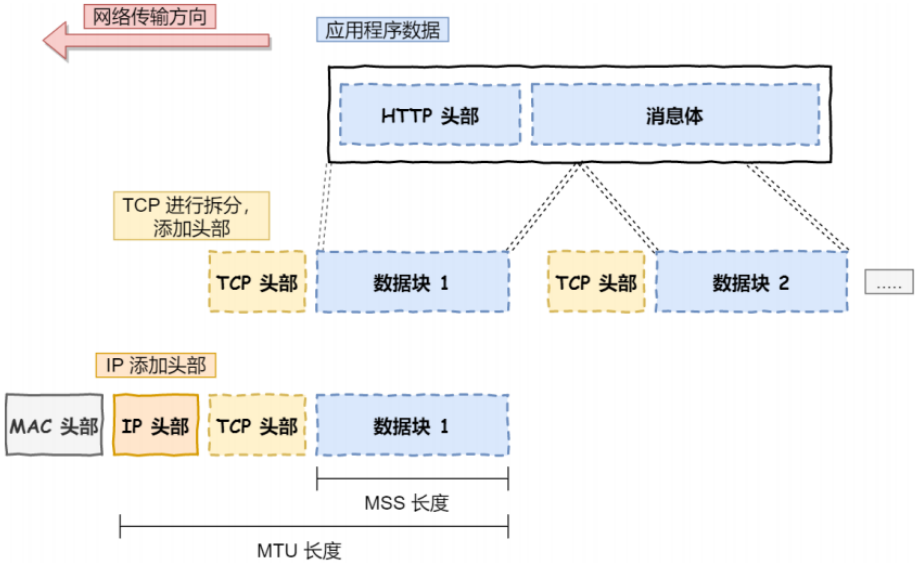
（4）TCP分割数据

如果 HTTP 请求消息比较长，超过了 MSS 的长度，这时 TCP 就需要把 HTTP 的数据拆解成一块块的数据发送，而不是一次性发送所有数据。



- MTU ： 一个网络包的最大长度，以太网中一般为 1500 字节。
- MSS ： 除去 IP 和 TCP 头部之后，一个网络包所能容纳的 TCP 数据的最大长度。

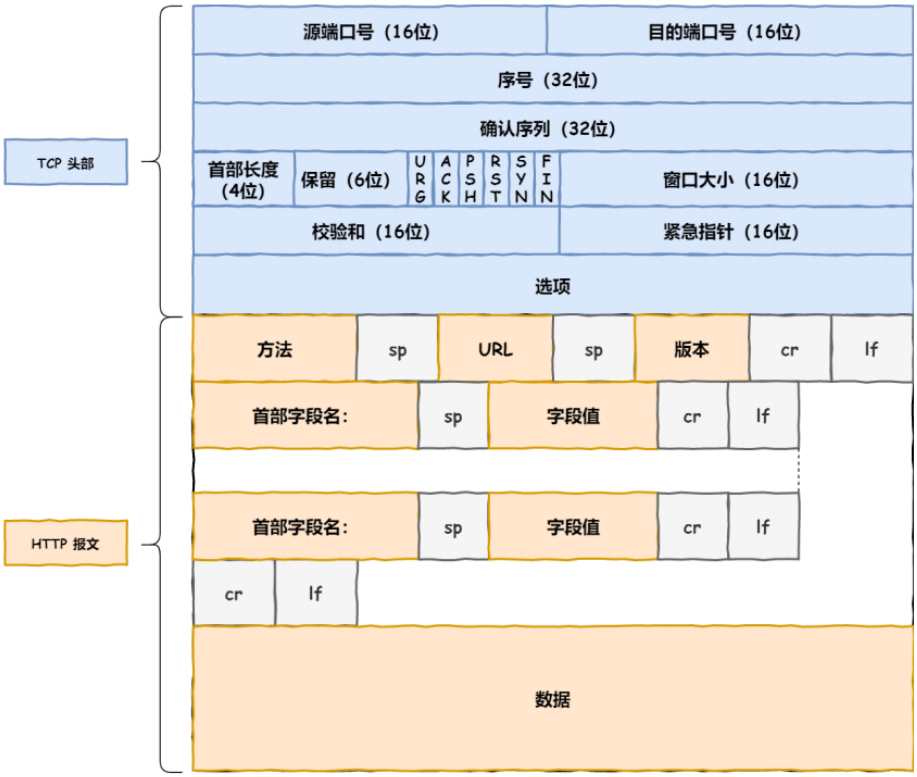
数据会被以 MSS 的长度为单位进行拆分，拆分出来的每一块数据都会被放进单独的网络包中。也就是在每个被拆分的数据加上 TCP 头信息，然后交给 IP 模块来发送数据。



（5）TCP报文生成

TCP 协议里面会有两个端口，一个是浏览器监听的端口（通常是随机生成的），一个是 Web 服务器监听的端口（HTTP 默认端口号是 80， HTTPS 默认端口号是 443 ）。

在双方建立了连接后，TCP 报文中的数据部分就是存放 HTTP 头部 + 数据，组装好 TCP 报文之后，就需交给下面的网络层处理。网络包的报文如下图：

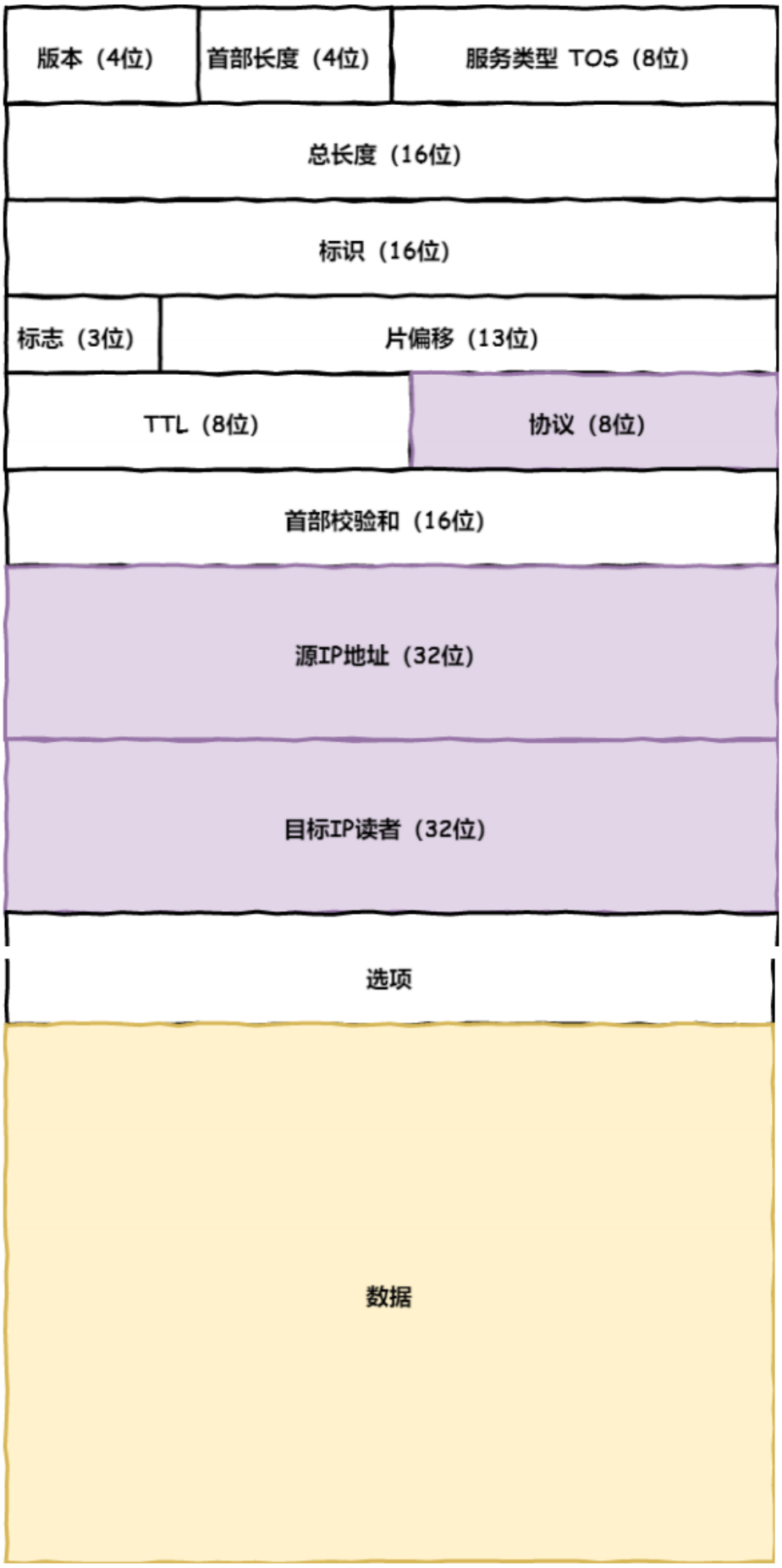


此时，遇上了 TCP 的数据包激动表示：“太好了，碰到了可靠传输的 TCP 传输，它给我加上 TCP 头部，我不再孤单了，安全感十足啊！有大佬可以保护我的可靠送达！但我应该往哪走呢？”（ 远程定位—IP ）

6、IP包头格式

TCP 模块在执行连接、收发、断开等各阶段操作时，都需要委托 IP 模块将数据封装成网络包发送给通信对象。

（1）IP包头格式如下：



在 IP 协议里面需要有源地址 IP 和 目标地址 IP：

- 源地址IP：即是客户端输出的 IP 地址

- 目标地址：即通过 DNS 域名解析得到的 Web 服务器 IP

因为 HTTP 是经过 TCP 传输的，所以在 IP 包头的协议号，要填写为 06 （十六进制），表示协议为 TCP。

（2）假设客户端有多个网卡

就会有多个 IP 地址，那 IP 头部的源地址应该选择哪个 IP 呢?

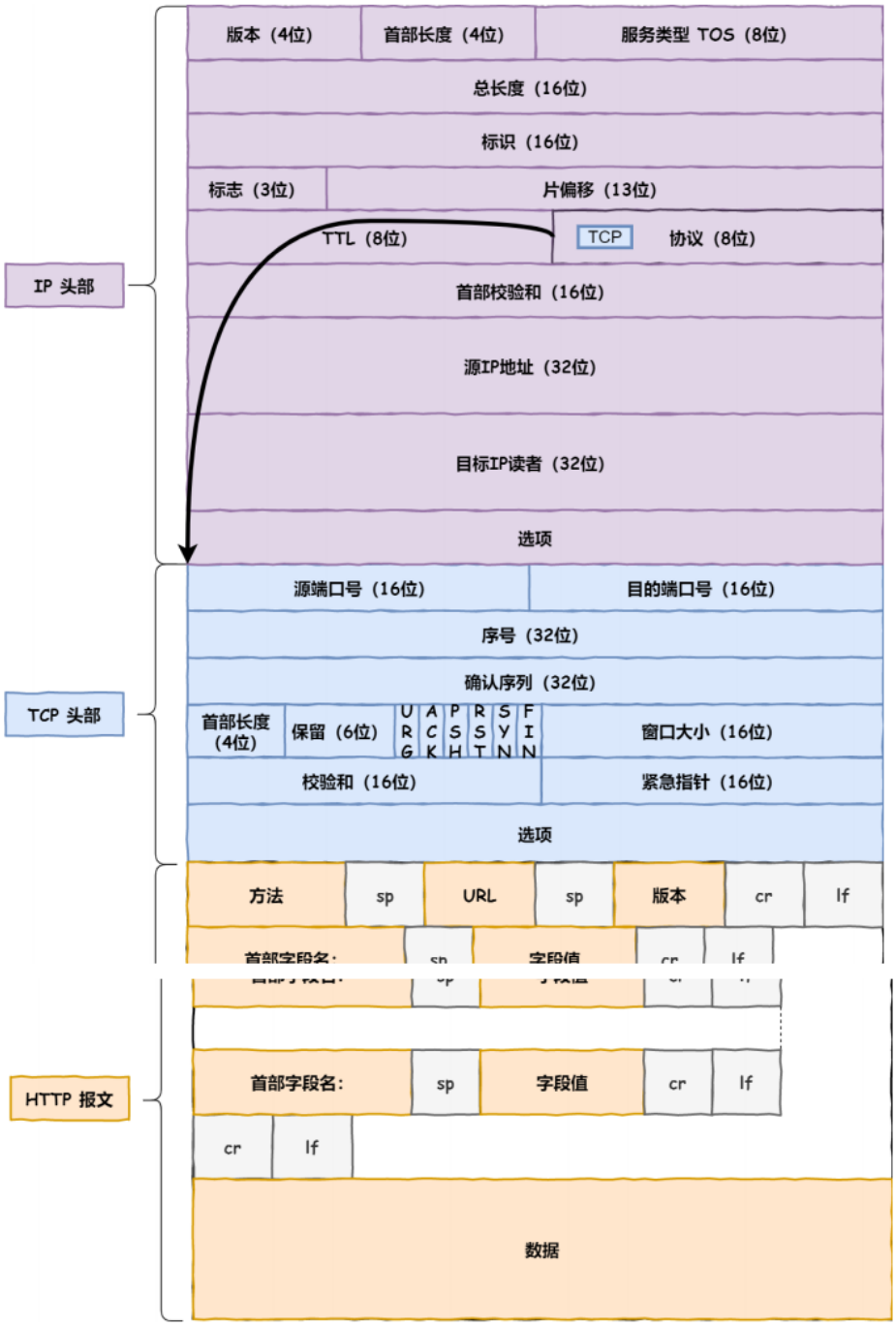
当存在多个网卡时，在填写源地址 IP 时，就需要判断到底应该填写哪个地址。这个判断相当于在多块网卡中判断应该使用哪个一块网卡来发送包。这个时候就需要根据路由表规则，来判断哪一个网卡作为源地址 IP。

在 Linux 操作系统，我们可以使用 route -n 命令查看当前系统的路由表。

```
[root@lincoding ~]# route -n

Kernel IP routing table
Destination     Gateway         Genmask         Flags Metric Ref    Use Iface
192.168.3.0     0.0.0.0         255.255.255.0   U     0      0        0 eth0
192.168.10.0    0.0.0.0         255.255.255.0   U     0      0        0 eth1
0.0.0.0         192.168.3.1     0.0.0.0         UG    0      0        0 eth0
```

第三条目比较特殊，它目标地址和子网掩码都是 0.0.0.0 ，这表示默认网关，如果其他所有条目都无法匹配，就会自动匹配这一行。并且后续就把包发给路由器，Gateway 即是路由器的 IP 地址。

（3）IP报文生成



7、两点传输-MAC

此时，加上了 IP 头部的数据包表示：“有 IP 大佬给我指路了，感谢 IP 层给我加上了 IP 包头，让我有了远程定位的能力！不会害怕在浩瀚的互联网迷茫了！可是目的地好远啊，我下一站应该去哪呢？”

生成了 IP 头部之后，接下来网络包还需要在 IP 头部的前面加上 MAC 头部。

（1）MAC包头格式

MAC 头部是以太网使用的头部，它包含了接收方和发送方的 MAC 地址等信息。

一般在 TCP/IP 通信里，MAC 包头的协议类型只使用：

- 0800：IP 协议
- 0806：ARP 协议

（2）MAC发送方和接收方如何确认

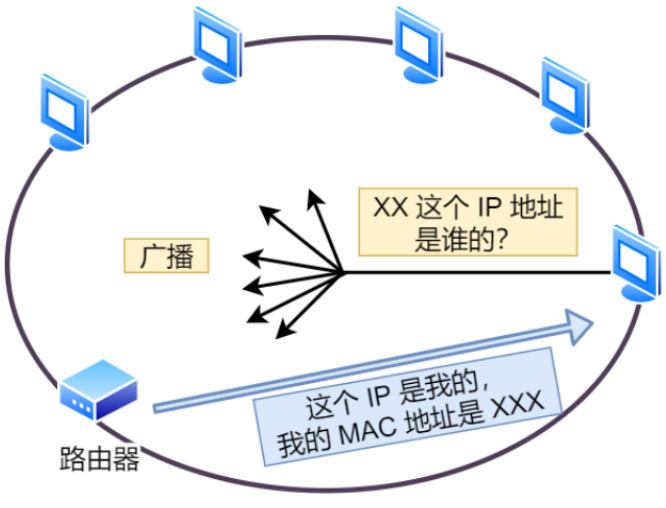
发送方的 MAC 地址获取就比较简单了，MAC 地址是在网卡生产时写入到 ROM 里的，只要将这个值读取出来写入到 MAC 头部就可以了。

接收方的 MAC 地址就有点复杂了，只要告诉以太网对方的 MAC 的地址，以太网就会帮我们把包发送过去，那么很显然这里应该填写对方的 MAC 地址。

所以先得搞清楚应该把包发给谁，这个只要查一下路由表就知道了。在**路由表**中找到相匹配的条目，然后把包发给 Gateway 列中的 IP 地址就可以了。

（3）既然知道要发给谁，按如何获取对方的 MAC 地址

不知道对方 MAC 地址？不知道就喊呗。此时就需要 ARP 协议帮我们找到路由器的 MAC 地址。



同一个子网

（4）如果每次都要广播获取，那不是很麻烦吗?

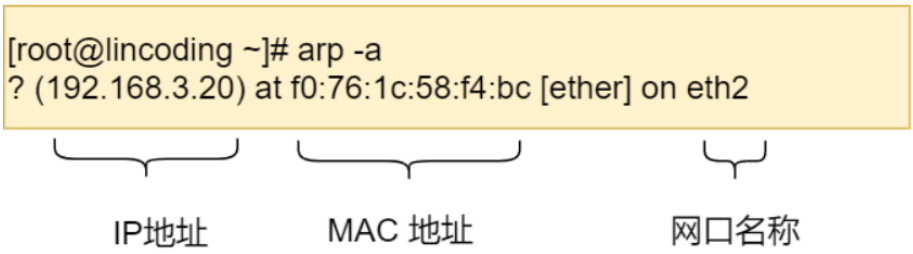
后续操作系统会把本次查询结果放到一块叫做 ARP 缓存的内存空间留着以后用，不过缓存的时间就几分钟。也就是说，在发包时：

先查询 ARP 缓存，如果其中已经保存了对方的 MAC 地址，就不需要发送 ARP 查询，直接使用 ARP 缓存中的地址。

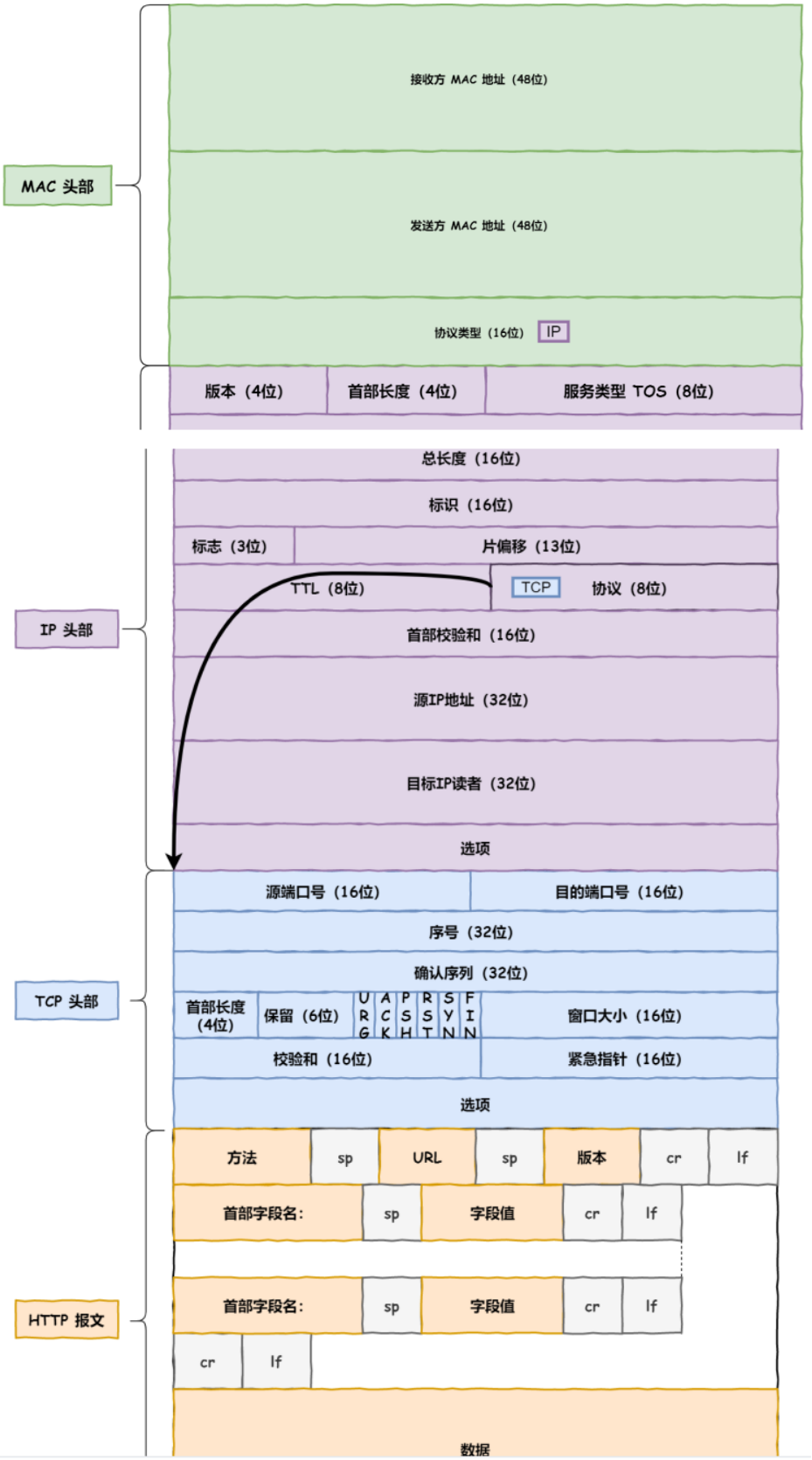
而当 ARP 缓存中不存在对方 MAC 地址时，则发送 ARP 广播查询。

（5）查看ARP缓存内容

在 Linux 系统中，我们可以使用 arp -a 命令来查看 ARP 缓存的内容。



（6）MAC报文生成

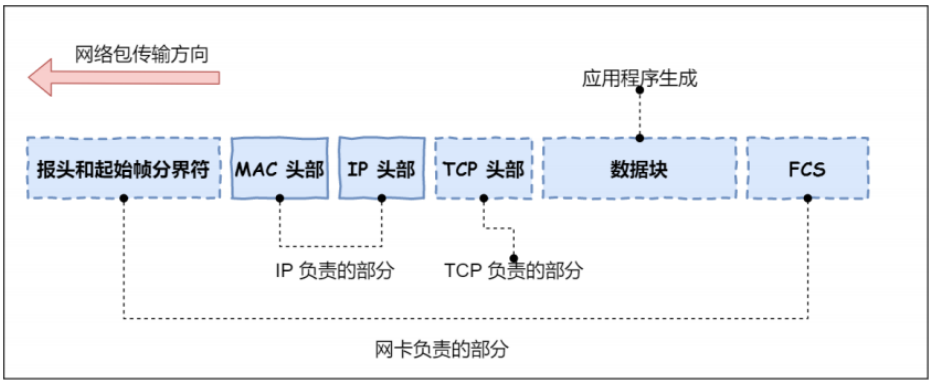


8、出口-网卡

此时，加上了 MAC 头部的数据包万分感谢，说道：“感谢 MAC 大佬，我知道我下一步要去哪了！我现在有很多头部兄弟，相信我可以到达最终的目的地！”。带着众多头部兄弟的数据包，终于准备要出门了。

网络包只是存放在内存中的一串二进制数字信息，没有办法直接发送给对方。因此，我们需要将数字信息转换为电信号，才能在网线上传输，也就是说，这才是真正的数据发送过程。负责执行这一操作的是网卡，要控制网卡还需要靠网卡驱动程序。

网卡驱动从 IP 模块获取到包之后，会将其复制到网卡内的缓存区中，接着会在其开头加上报头和起始帧分界符，在末尾加上用于检测错误的帧校验序列。



- 起始帧分界符是一个用来表示包起始位置的标记
- 末尾的 FCS （帧校验序列）用来检查包传输过程是否有损坏

最后网卡会将包转为电信号，通过网线发送出去。

9、送别者—交换机

唉，真是不容易，发一个包，真是历经千辛万苦。致此，一个带有许多头部的数据终于踏上寻找目的地的征途了！

下面来看一下包是如何通过交换机的？交换机的设计是将网络包原样转发到目的地。交换机工作在 MAC 层，也称为二层网络设备。

（1）交换机的包接收操作

首先，电信号到达网线接口，交换机里的模块进行接收，接下来交换机里的模块将电信号转换为数字信号。

然后通过包末尾的 FCS 校验错误，如果没问题则放到缓冲区。这部分操作基本和计算机的网卡相同，但交换机的工作方式和网卡不同。

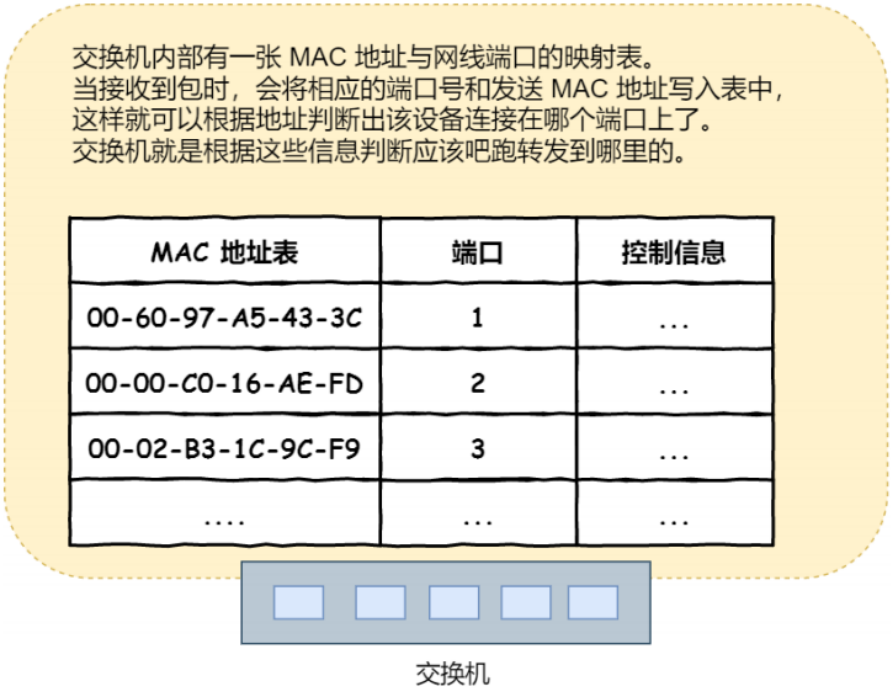
计算机的网卡本身具有 MAC 地址，并通过核对收到的包的接收方 MAC 地址判断是不是发给自己的，如果不是发给自己的则丢弃；相对地，交换机的端口不核对接收方 MAC 地址，而是直接接收所有的包并存放到缓冲区中。

因此，和网卡不同，交换机的端口不具有 MAC 地址。

将包存入缓冲区后，接下来需要查询一下这个包的接收方 MAC 地址是否已经在 MAC 地址表中有记录了。

交换机的 MAC 地址表主要包含两个信息：

- 一个是设备的 MAC 地址
- 一个是该设备连接在交换机的哪个端口上



交换机根据 MAC 地址表查找 MAC 地址，然后将信号发送到相应的端口。

（2）当 MAC 地址表找不到指定的 MAC 地址会怎么样？、

地址表中找不到指定的 MAC 地址。这可能是因为具有该地址的设备还没有向交换机发送过包，或者这个设备一段时间没有工作导致地址被从地址表中删除了。

这种情况下，交换机无法判断应该把包转发到哪个端口，只能将包转发到除了源端口之外的所有端口上，无论该设备连接在哪个端口上都能收到这个包。

这样做不会产生什么问题，因为以太网的设计本来就是将包发送到整个网络的，然后只有相应的接收者才接收包，而其他设备则会忽略这个包。

发送了包之后目标设备会作出响应，只要返回了响应包，交换机就可以将它的地址写入 MAC 地址表，下次也就不需要把包发到所有端口了。

此外，如果接收方 MAC 地址是一个广播地址，那么交换机会将包发送到除源端口之外的所有端口。

以下两个属于广播地址：

- MAC 地址中的 FF:FF:FF:FF:FF:FF
- IP 地址中的 255.255.255.255

10、出境大门—路由器

数据包通过交换机转发抵达了路由器，准备要离开土生土长的子网了。此时，数据包和交换机离别时说道：“感谢交换机兄弟，帮我转发到出境的大门，我要出远门啦！”

（1）路由器与交换机的区别

网络包经过交换机之后，现在到达了路由器，并在此被转发到下一个路由器或目标设备。

这一步转发的工作原理和交换机类似，也是通过查表判断包转发的目标。 不过在具体的操作过程上，路由器和交换机是有区别的。

- 因为路由器是基于 IP 设计的，俗称三层网络设备，路由器的各个端口都具有 MAC 地址和 IP 地址；

- 而交换机是基于以太网设计的，俗称二层网络设备，交换机的端口不具有 MAC 地址。

（2）路由器的基本原理

路由器的端口具有 MAC 地址，因此它就能够成为以太网的发送方和接收方；同时还具有 IP 地址，从这个意义上来说，它和计算机的网卡是一样的。

当转发包时，首先路由器端口会接收发给自己的以太网包，然后路由表查询转发目标，再由相应的端口作为发送方将以太网包发送出去。

（3）路由器的包接收操作

首先，电信号到达网线接口部分，路由器中的模块会将电信号转成数字信号，然后通过包末尾的 FCS 进行错误校验。如果没问题则检查 MAC 头部中的接收方 MAC 地址，看看是不是发给自己的包，如果是就放到接收缓冲区中，否则就否则就丢弃这个包。

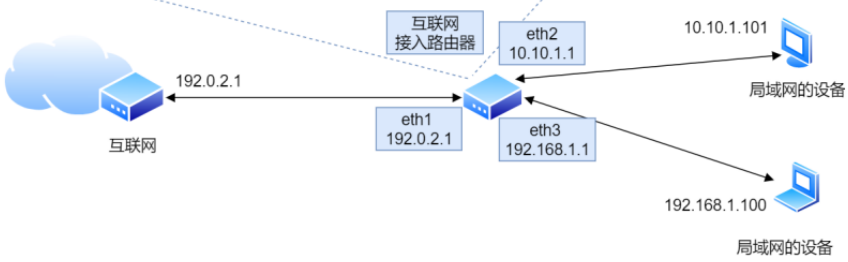
总的来说，路由器的端口都具有 MAC 地址，只接收与自身地址匹配的包，遇到不匹配的包则直接丢弃。

（4）查询路由表确定输出端口

完成包接收操作之后，路由器就会去掉包开头的 MAC 头部。 MAC 头部的作用就是将包送达路由器，其中的接收方 MAC 地址就是路由器端口的 MAC 地址。因此，当包到达路由器之后，MAC 头部的任务就完成了，于是 MAC 头部就会被丢弃。

接下来，路由器会根据 MAC 头部后方的 IP 头部中的内容进行包的转发操作。转发操作分为几个阶段，首先是查询路由表判断转发目标。

目标地址 (Destination)	子网掩码 (Netmask)	网关 (Gateway)	接口 (Interface)	跃点数 (Metric)
10.10.1.0	255.255.255.0	——	eth2	1
192.168.1.0	255.255.255.0	——	eth3	1
0.0.0.0	0.0.0.0	192.0.2.1	eth1	1



（5）路由器的发送操作

首先，我们需要根据路由表的网关列判断对方的地址。

如果网关是一个 IP 地址：

则这个IP 地址就是我们要转发到的目标地址，还未抵达终点，还需继续需要路由器转发。

如果网关为空：

则 IP 头部中的接收方 IP 地址就是要转发到的目标地址，也是就终于找到 IP 包头里的目标地址了，说明已抵达终点。

知道对方的 IP 地址之后，接下来需要通过 ARP 协议根据 IP 地址查询 MAC 地址，并将查询的结果作为接收方 MAC 地址。

路由器也有 ARP 缓存，因此首先会在 ARP 缓存中查询，如果找不到则发送 ARP 查询请求。

接下来是发送方 MAC 地址字段，这里填写输出端口的 MAC 地址。还有一个以太类型字段，填写 0800 （十六进制）表示 IP 协议。

网络包完成后，接下来会将其转换成电信号并通过端口发送出去。这一步的工作过程和计算机也是相同的。

发送出去的网络包会通过交换机到达下一个路由器。由于接收方 MAC 地址就是下一个路由器的地址，所以交换机会根据这一地址将包传输到下一个路由器。

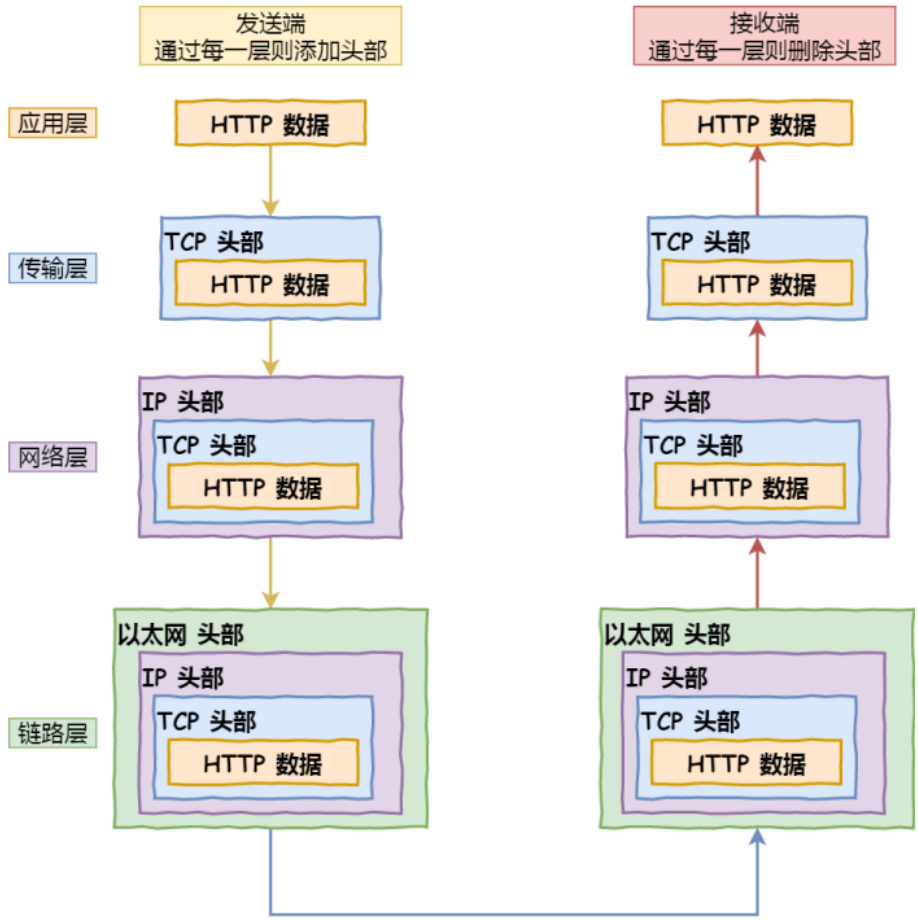
接下来，下一个路由器会将包转发给再下一个路由器，经过层层转发之后，网络包就到达了最终的目的地。

不知你发现了没有，在网络包传输的过程中，源 IP 和目标 IP 始终是不会变的，一直变化的是 MAC 地址，因为需要 MAC 地址在以太网内进行两个设备之间的包传输。

11、交换机转发

数据包通过多个路由器道友的帮助，在网络世界途经了很多路程，最终抵达了目的地的城门！城门值守的路由器，发 现了这个小兄弟数据包原来是找城内的人，于是它就将数据包送进了城内，再经由城内的交换机帮助下，最终转发到了目的地了。数据包感慨万千的说道：“多谢这一路上，各路大侠的相助！”

数据包抵达了服务器，服务器肯定高兴呀，开始扒数据包的皮!



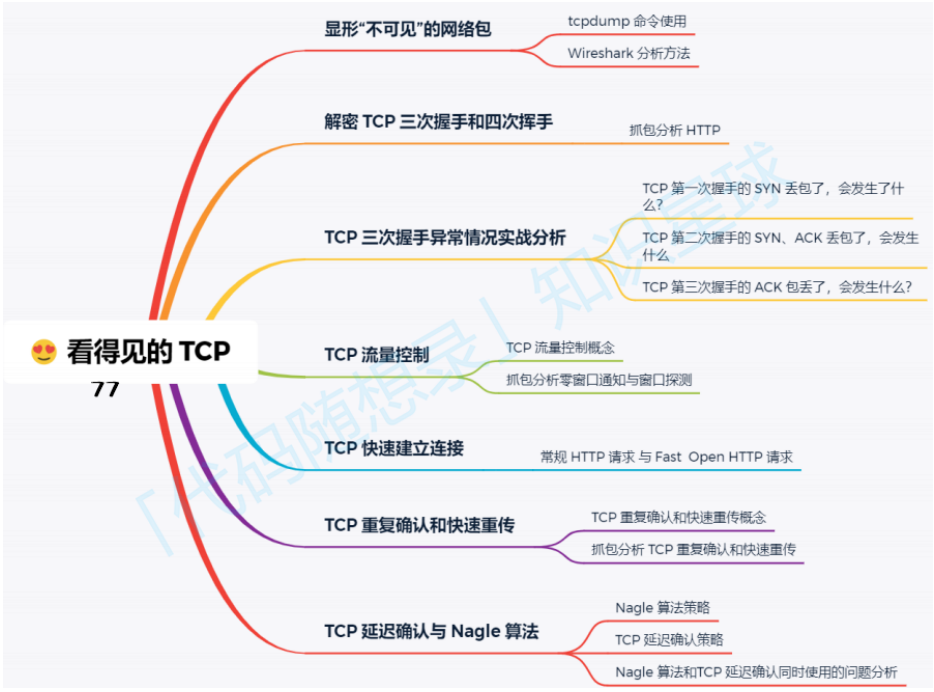
一个数据包臭不要脸的感受：

我一开始我虽然孤单、不知所措，但没有停滞不前。我依然满怀信心和勇气开始了征途。

我很庆幸遇到了各路神通广大的大佬，有可靠传输的 TCP、有远程定位功能的 IP、有指明下一站位置的 MAC 等这些大佬都给我前面加上了头部，使得我能在交换机和路由器的转发下，抵达到了目的地！

这一路上的经历，让我认识到了网络世界中各路大侠协作的〕要性，是他们维护了网络世界的秩序，感谢他们！

TCP 抓包实践



1、显形“不可见”的网络包

（1）最常用的网络抓包和分析工具：

tcpdump和Wireshark，那这两者有什么区别呢?

- tcpdump 仅支持命令行格式使用，常用在 Linux 服务器中抓取和分析网络包。
- Wireshark 除了可以抓包外，还提供了可视化分析网络包的图形页面。

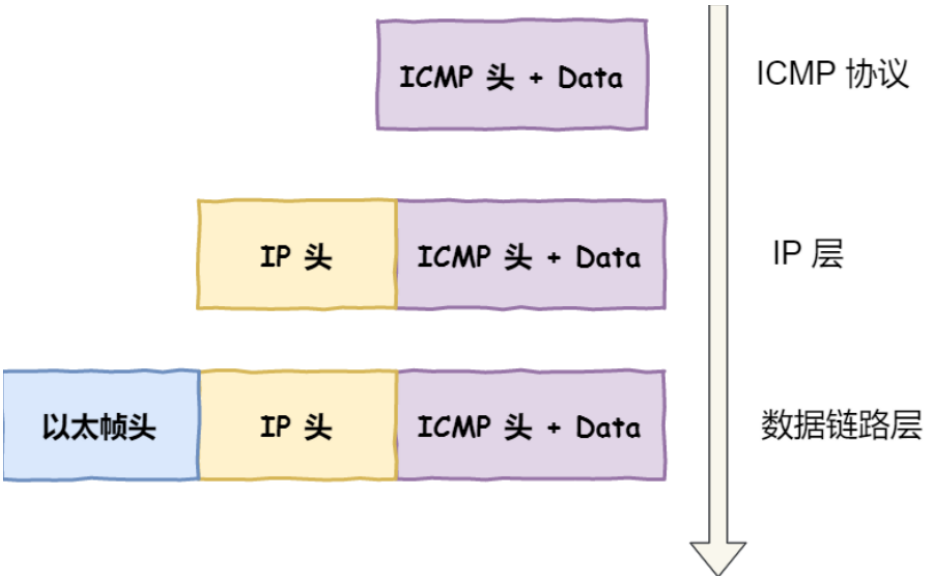
两者经常搭配使用，先用 tcpdump 命令在 Linux 服务器上抓包（保存成.pcap后缀格式的文件），接着把抓包的文件拖出到Windows 电脑后，用 Wireshark 可视化分析。

（2）tcpdump在Linux下如何抓包?

tcpdump 使用 —— 选项类		
选项	示例	说明
-i	tcpdump -i eth0	指定网络接口，默认是 0 浩接口（如 eth0 ），any 表示所有接口
-nn	tcpdump -nn	不解析 IP 地址和端口号的名称
-c	tcpdump -c 5	限制要抓取的网络包的个数
-w	tcpdump -w file.pcap	保持到文件中，文件名通常以 .pcap 为后缀

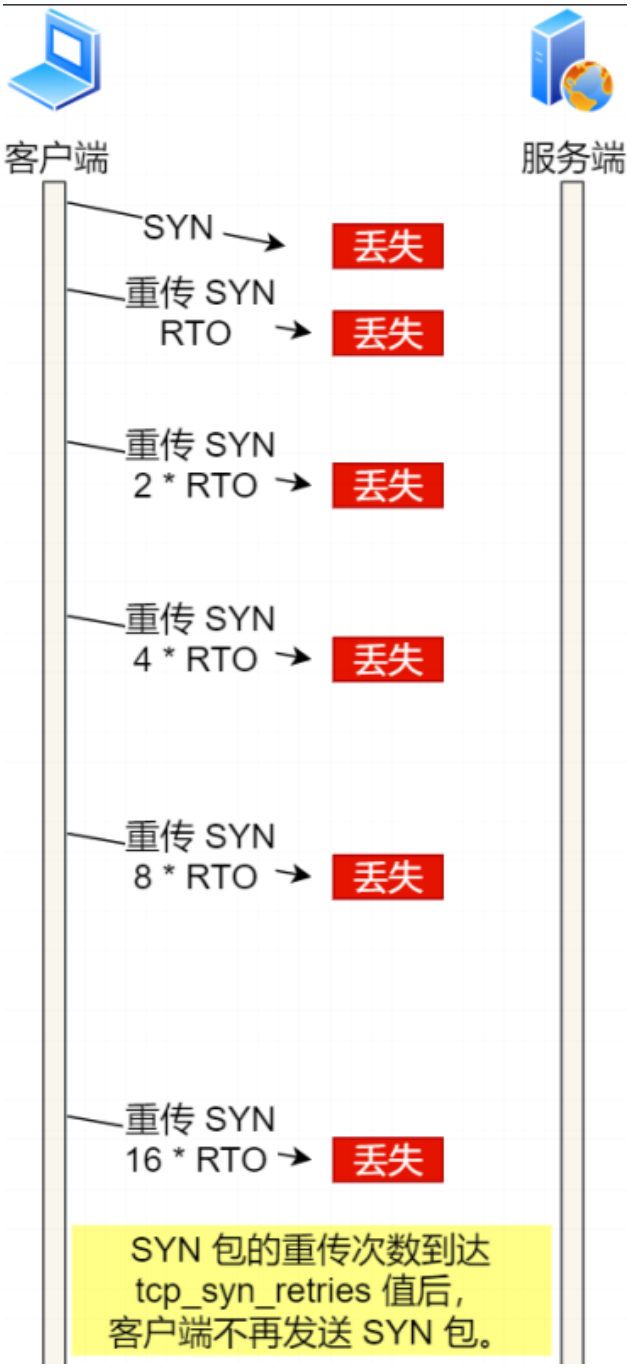
tcpdump 使用 —— 过滤表达式类		
选项	示例	说明
host、src host、dst host	tcpdump -nn host 192.168.1.100	主机过滤
port、src port、dst port	tcpdump -nn port 80	端口过滤
ip、ip6、arp、tcp、udp、icmp	tcpdump -nn tcp	协议过滤
and、or、not	tcpdump -nn host 192.168.1.100 and port 80	逻辑表达式
tcp[tcoflages]	tcpdump -nn "tcp[tcpflags] & tcp-syn != 0"	特定状态的 TCP 包

（3）Wireshark工具如何分析数据包?



2、TCP三次握手异常情况实战分析

（1）实验一：TCP第一次握手SYN 丢包



总结：

> 通过实验一的实验结果，我们可以得知，当客户端发起的 TCP 第一次握手 SYN 包，在超时时间内没收到服务端的 ACK，就会在超时重传 SYN 数据包，每次超时重传的 RTO 是翻倍上涨的，直到 SYN 包的重传次数到达 `tcp_syn_retries` 值后，客户端不再发送 SYN 包。

（2）实验二：TCP第二次握手SYN、ACK丢包

客户端设置了防火墙，粗暴地将来自服务端的数据都丢弃

> 通过实验二的实验结果，我们可以得知，当 TCP 第二次握手 SYN、ACK 包丢了后，客户端 SYN 包会发生超时重传，服务端 SYN、ACK 也会发生超时重传。
>
> 客户端 SYN 包超时重传的最大次数，是由 tcp_syn_retries 决定的，默认值是 5 次；服务端 SYN、ACK 包时重传的最大次数，是由 tcp_synack_retries 决定的，默认值是 5 次。

网络包进入主机后的顺序如下：

- 进来的顺序 Wire -> NIC -> **tcpdump -> netfilter/iptables**

- 出去的顺序 **iptables -> tcpdump** -> NIC -> Wire

（3）实验三：TCP第三次握手ACK丢包

在服务端配置防火墙，屏蔽客户端TCP报文中标志位是ACK的包。

如果客户端不发送数据，什么时候才会断开处于ESTABLISHED状态的连接？这就需要提到TCP的保活机制了：

定义一个时间段，在这个时间段内，如果没有任何连接相关的活动，TCP 保活机制会开始作用，每隔一个时间间隔，发送一个「探测报文」，该探测报文包含的数据非常少，如果连续几个探测报文都没有得到响应，则认为当前的 TCP 连接已经死亡，系统内核将错误信息通知给上层应用程序。

总结：

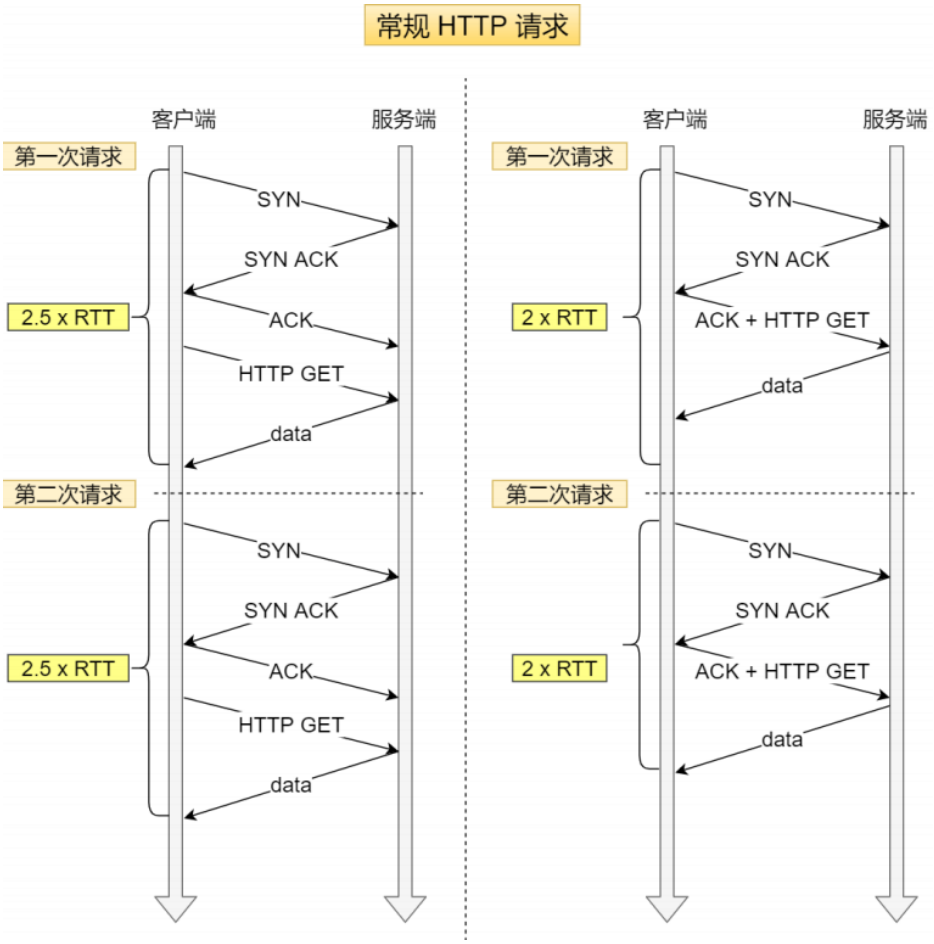
在建立 TCP 连接时，如果第三次握手的 ACK，服务端无法收到，则服务端就会短暂处于 `SYN_RECV` 状态，而客户端会处于 `ESTABLISHED` 状态。

由于服务端一直收不到 TCP 第三次握手的 ACK，则会一直重传 SYN、ACK 包，直到重传次数超过 `tcp_synack_retries` 值（默认值 5 次）后，服务端就会断开 TCP 连接。

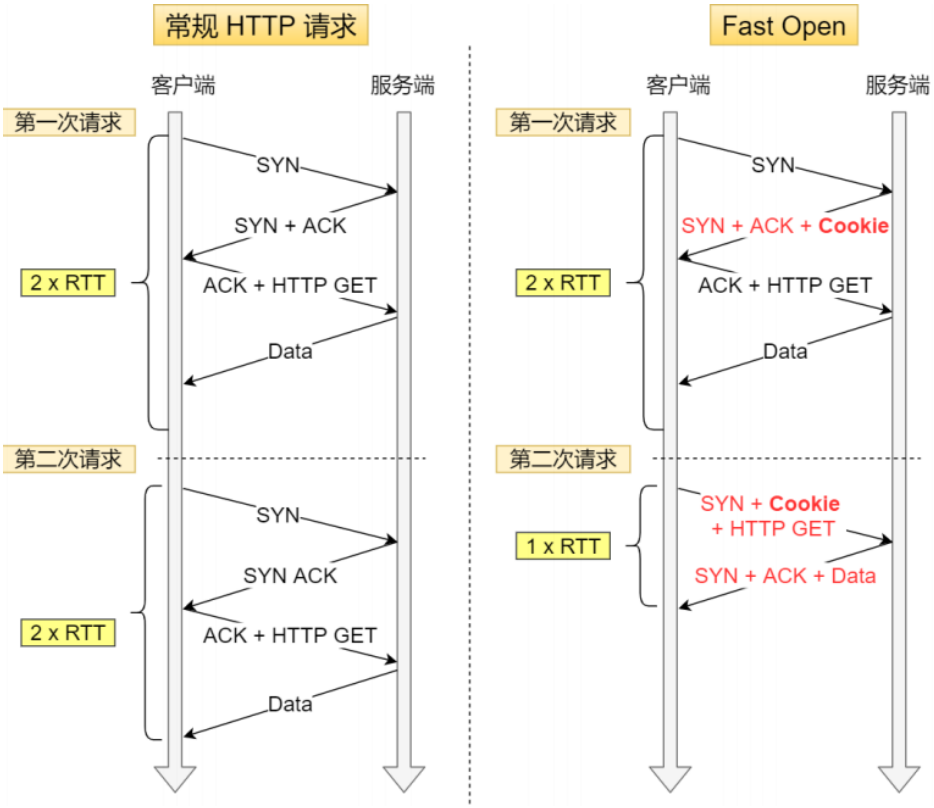
而客户端则会有两种情况：

- 如果客户端没发送数据包，一直处于 `ESTABLISHED` 状态，然后经过 2 小时 11 分 15 秒才可以发现一个「死亡」连接，于是客户端连接就会断开连接。
- 如果客户端发送了数据包，一直没有收到服务端对该数据包的确认报文，则会一直重传该数据包，直到重传次数超过 `tcp_retries2` 值（默认值 15 次）后，客户端就会断开 TCP 连接。

3、TCP快速建立连接



使用TCP Fast Open功能，可以减少TCP连接建立的时延。



- 在第一次建立连接的时候，服务端在第二次握手产生一个 `Cookie` （已加密）并通过 SYN、ACK 包一起发给客户端，于是客户端就会缓存这个 `Cookie` ，所以第一次发起 HTTP Get 请求的时候，还是需要 2 个 RTT 的时延；
- 在下次请求的时候，客户端在 SYN 包带上 `Cookie` 发给服务端，就提前可以跳过三次握手的过程，因为 `Cookie` 中维护了一些信息，服务端可以从 `Cookie` 获取 TCP 相关的信息，这时发起的 HTTP GET 请求就只需要 1 个 RTT 的时延；

注：客户端在请求并存储了 Fast Open Cookie 之后，可以不断重复 TCP Fast Open 直至服务器认为 Cookie 无效（通常为过期）

4、TCP流量控制

发送窗口决定了一次能发多少字节的数据，而MSS决定了这些字节要分多少包才能发送完。

5、TCP延迟确认与Nagle算法

当TCP报文承载的数据非常小的时候，整个网络的效率很低。就好像快递员开着大货车送一个小包裹一样浪费。所以就出现了常见的两种策略，来减少小报文的传输。

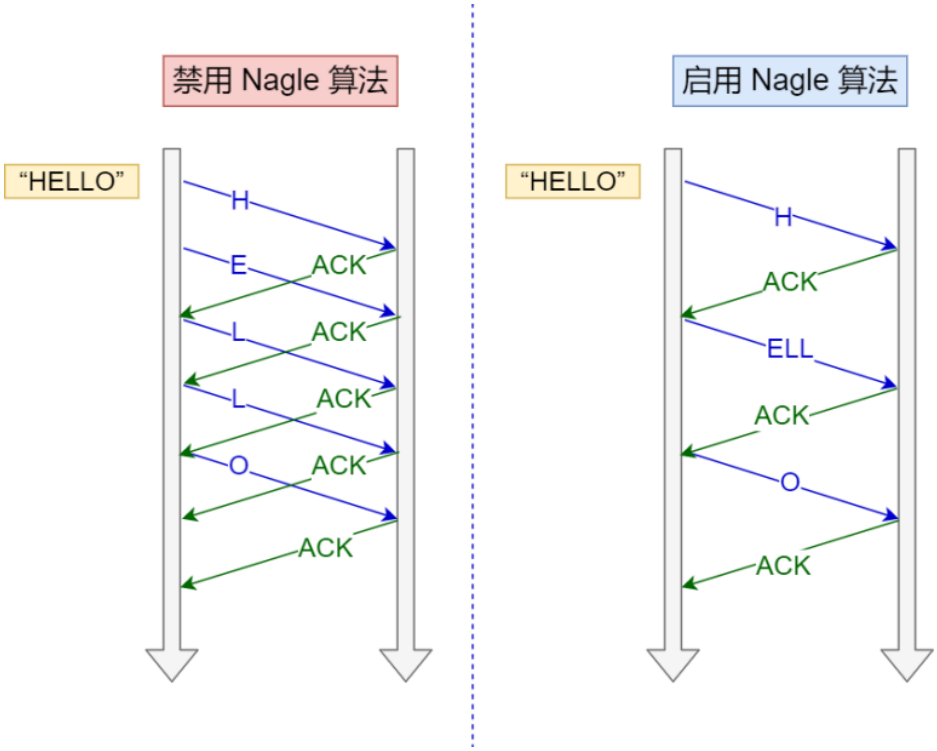
1. Nagle算法
2. 延迟确认

（1）Nagle算法是如何避免大量TCP小数据报文的传输的？

Nagle 算法的策略：

- 没有已发送未确认报文时，立刻发送数据。
- 存在未确认报文时，直到「没有已发送未确认报文」或「数据长度达到 MSS 大小」时，再发送数据。

只要没满足上面条件中的一条，发送方一直在囤积数据，直到满足上面的发送条件。

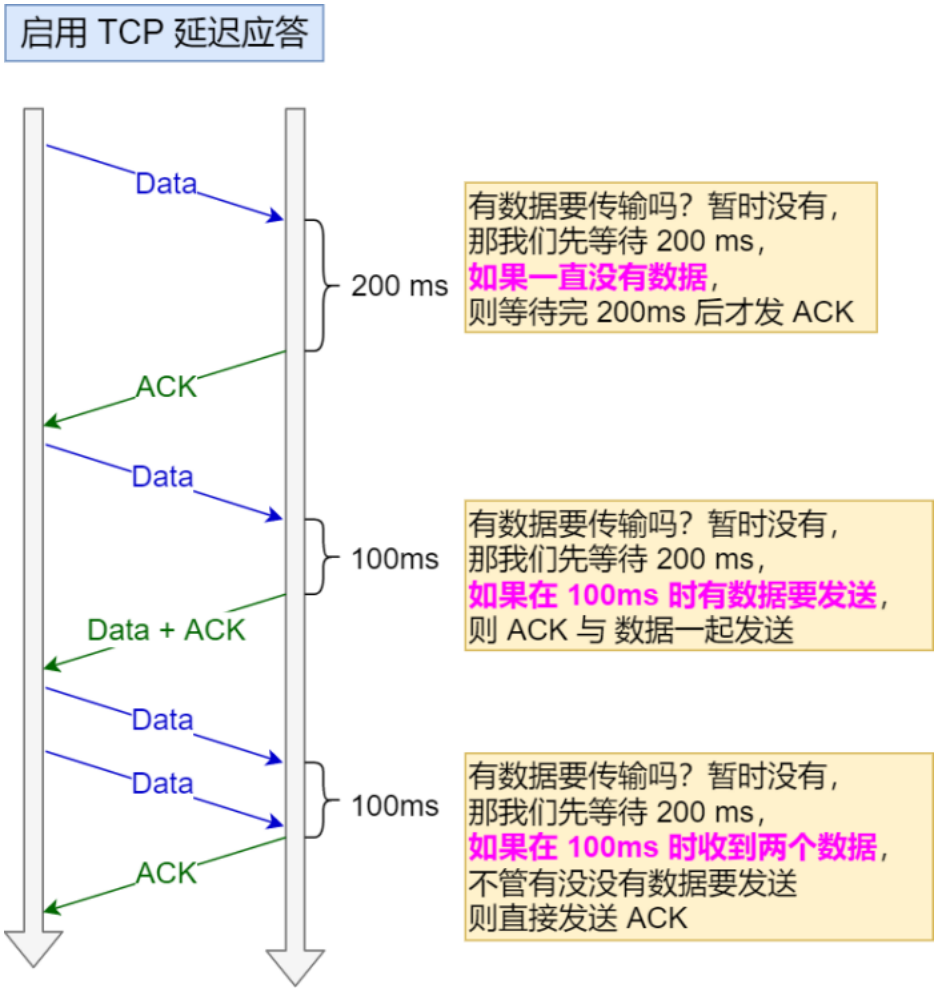


从上图可以看出：Nagle算法一定会有一个小报文，在最开始的时候。

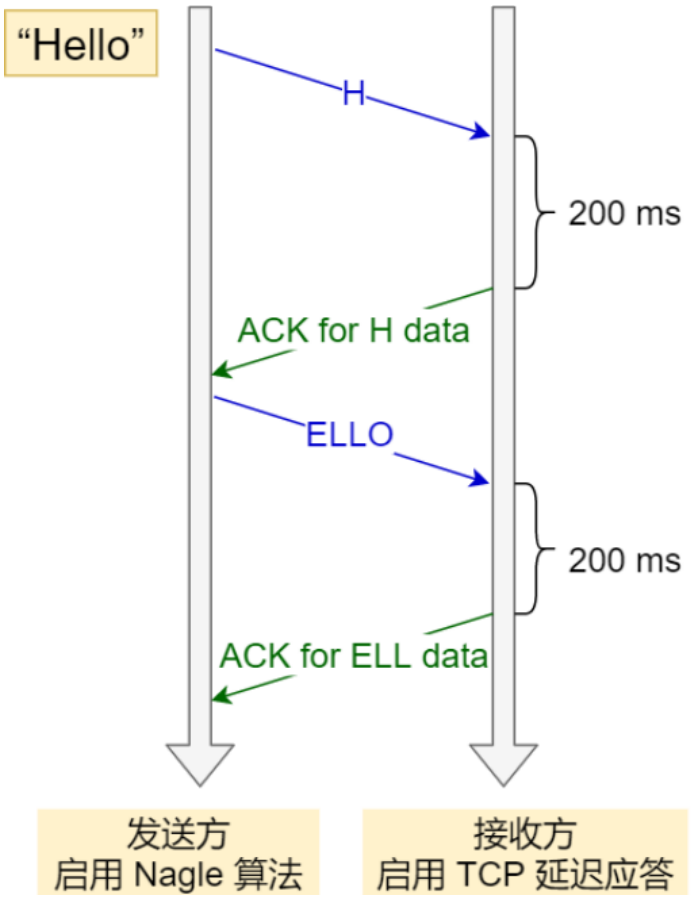
（2）延迟确认：为解决ACK传输效率低的问题，而提出的。

TCP 延迟确认的策略：

- 当有响应数据要发送时，ACK 会随着响应数据一起立刻发送给对方
- 当没有响应数据要发送时，ACK 将会延迟一段时间，以等待是否有响应数据可以一起发送
- 如果在延迟等待发送 ACK 期间，对方的第二个数据报文又到达了，这时就会立刻发送 ACK



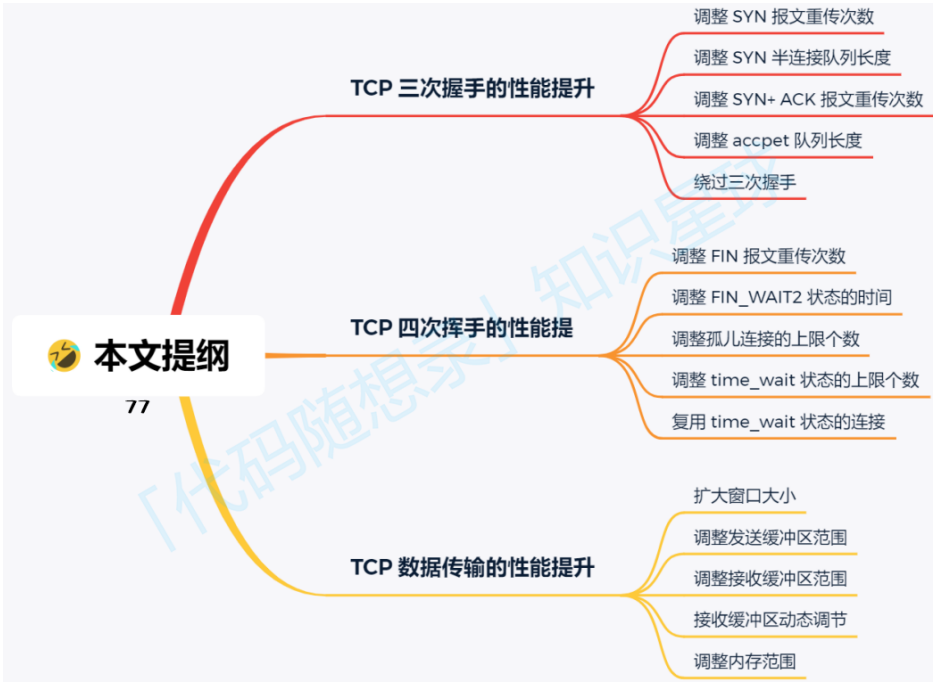
当延迟确认和Nagle混合使用时，会出现新的问题（导致时耗增长）



要解决上述问题，只有两个办法：

1. 发送方关闭Nagle算法
2. 接收方关闭TCP延迟确认

TCP连接与断开优化



关于优化TCP握手的几个TCP参数：

优化三次握手的策略	
策略	**TCP 内核参数**
调整 SYN 报文的重传次数	tcp_syn_retries
调整 SYN 半连接队列长度	tcp_max_syn_backlog、somaxconn、backlog
调整 SYN+ACK 报文的重传次数	tcp_synack_retries
调整 accpet 队列长度	min(backlog, somaxconn)
绕过三次握手	tcp_fastopen

1、客户端的优化

当客户端发起SYN包时，可以通过tcp_synb_retries控制其重传的次数。

2、服务端的优化

当服务端 SYN 半连接队列溢出后，会导致后续连接被丢弃，可以通过 `netstat -s` 观察半连接队列溢出的情况，如果 SYN 半连接队列溢出情况比较严重，可以通过 `tcp_max_syn_backlog、somaxconn、backlog` 参数来调整 SYN 半连接队列的大小。

服务端回复 SYN+ACK 的重传次数由 `tcp_synack_retries` 参数控制。如果遭受 SYN 攻击，应把 `tcp_syncookies` 参数设置为 1，表示仅在 SYN 队列满后开启 syncookie 功能，可以保证正常的连接成功建立。

服务端收到客户端返回的 ACK，会把连接移入 accpet 队列，等待进行调用 accpet() 函数取出连接。

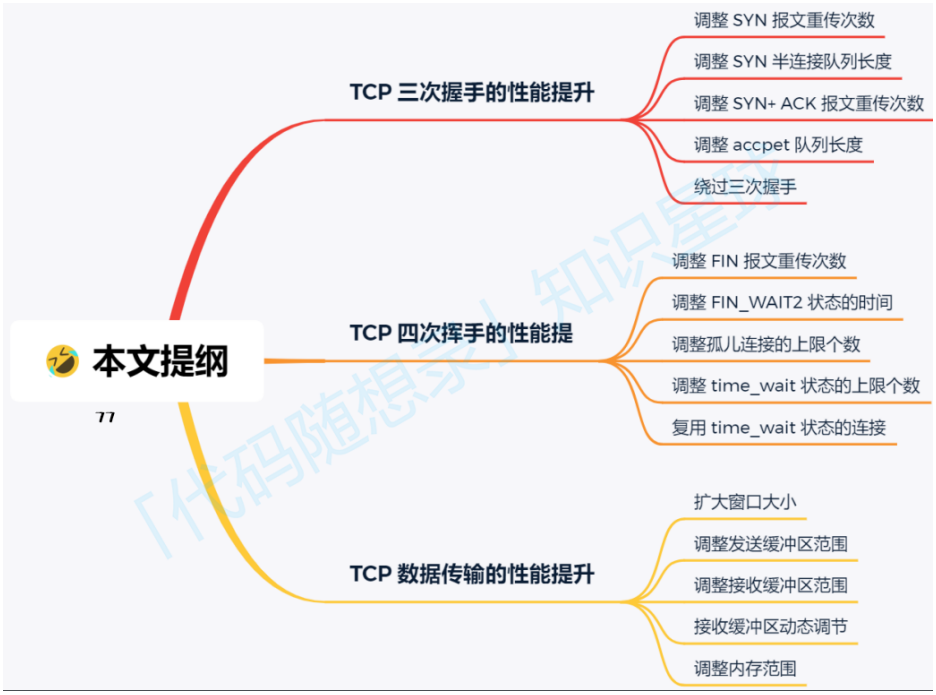
可以通过 `ss -lnt` 查看服务端进程的 accept 队列长度，如果 accept 队列溢出，系统默认丢弃 ACK，如果可以把 `tcp_abort_on_overflow` 设置为 1 ，表示用 RST 通知客户端连接建立失败。

如果 accpet 队列溢出严重，可以通过 listen 函数的 `backlog` 参数和 `somaxconn` 系统参数提高队列大小，accept 队列长度取决于 min(backlog, somaxconn)。

3、绕过三次握手

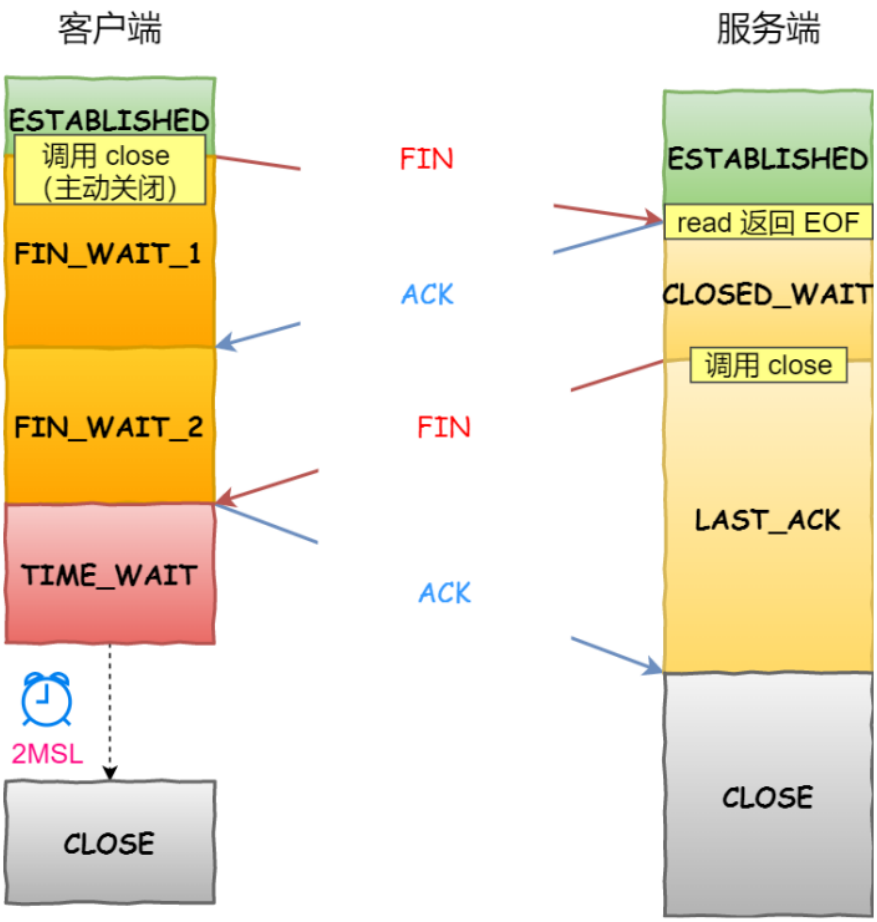
TCP Fast Open 功能可以绕过三次握手，使得 HTTP 请求减少了 1 个 RTT 的时间，Linux 下可以通过 `tcp_fastopen` 开启该功能，同时必须保证服务端和客户端同时支持。

TCP内核参数



1、TCP四次挥手的性能提升

TCP四次挥手的状态变迁如下：



注：主动关闭连接的一方才有TIME_WAIT状态。

主动关闭方和被动关闭方的优化思路也不同，下面将进行分别讨论：

主动方的优化

关闭连接的方式通常有两种，分别是 RST 报文关闭和 FIN 报文关闭。

如果进程异常退出了，内核就会发送 RST 报文来关闭，它可以不走四次挥手流程，是一个暴力关闭连接的方式。

安全关闭连接的方式必须通过四次挥手，它由进程调用 close 和 shutdown 函数发起 FIN 报文（shutdown 参数须传入 SHUT_WR 或者 SHUT_RDWR 才会发送 FIN）。

1、close 和 shutdown 有什么区别

调用了 close 函数意味着完全断开连接，完全断开不仅指无法接收数据，而且也不能发送数据。

此时，调用了 close 函数的一方的连接叫做「孤儿连接」，如果你用 netstat -p 命令，会发现连接对应的进程名为空。

使用 close 函数关闭连接是不优雅的。于是，就出现了一种优雅关闭连接的 shutdown 函数，它可以控制只关闭一个方向的连接。

```
int shutdown(int sock, int howto);
```

其中第二个参数决定断开连接的方式，主要有三种：

（1）SHUT_RD(0)

关闭连接的「读」这个方向，如果接收缓冲区有已接收的数据，则将会被丢弃，并且后续再收到新的数据，会对数据进行 ACK，然后悄悄地丢弃。也就是说，对端还是会接收到 ACK，在这种情况下根本不知道数据已经被丢弃了。

（2）SHUT_WR(1)

关闭连接的「写」这个方向，这就是常被称为「半关闭」的连接。如果发送缓冲区还有未发送的数据，将被立即发送出去，并发送一个 FIN 报文给对端。

（3）SHUT_RDWR(2)

相当于 SHUT_RD 和 SHUT_WR 操作各一次，关闭套接字的读和写两个方向。

2、FIN_WAIT1状态的优化

如果 FIN_WAIT1 状态连接很多，我们就需要考虑降低 tcp_orphan_retries 的值。当重传次数超过 tcp_orphan_retries 时，连接就会直接关闭掉（即：新增的孤儿连接将不再走四次挥手，而是直接发送 RST 复位报文强制关闭）。

注：tcp_max_orphans 参数，定义了「孤儿连接」的最大数量。

3、FIN_WAIT2状态的优化

当主动方收到 ACK 报文后，会处于 FIN_WAIT2 状态，就表示主动方的发送通道已经关闭，接下来将等待对方发送 FIN 报文，关闭对方的发送通道。

这时，如果连接是用 shutdown 函数关闭的，主动方连接可以一直处于 FIN_WAIT2 状态，因为它可能还可以发送或接收数据。

但对于 close 函数关闭的孤儿连接，由于无法再发送和接收数据，所以这个状态不可以持续太久，而 tcp_fin_timeout 控制了这个状态下连接的持续时长，默认值是 60 秒（与 TIME_WAIT 状态持续的时间是相同的）。它意味着对于孤儿连接（调用 close 关闭的连接），如果在 60 秒后还没有收到 FIN 报文，连接就会直接关闭。

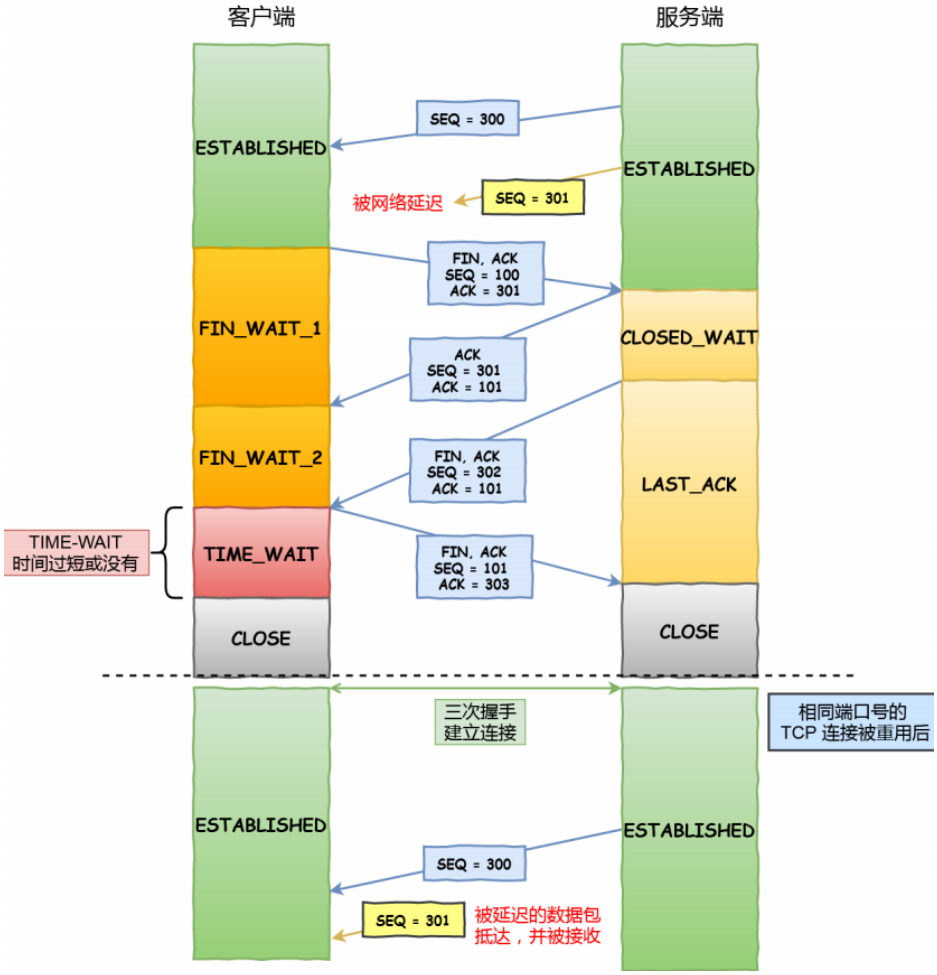
4、TIME_WAIT状态的优化

TIME_WAIT的状态尤其重要，主要是两个原因

- 防止具有相同「四元组」的「旧」数据包被收到；
- 保证「被动关闭连接」的一方能被正确的关闭，即保证最后的 ACK 能让被动关闭方接收，从而帮助其正常关闭；

（1）防止收到历史数据，从而导致数据错乱的问题

若TIME_WAIT等待时间过短，被延迟的数据包抵达后会发生什么呢?



TIME_WAIT设计为2MSL，足以让两个方向上的数据包都被丢弃，使得原来连接的数据包在网络中都自然消失，再出现的数据包一定都是新建立连接所产生的。

注：

MSL 全称是 Maximum Segment Lifetime，它定义了一个报文在网络中的最长生存时间（报文每经过一次路由器的转发，IP 头部的 TTL 字段就会减 1，减到 0 时报文 就被丢弃，这就限制了报文的最长存活时间）。

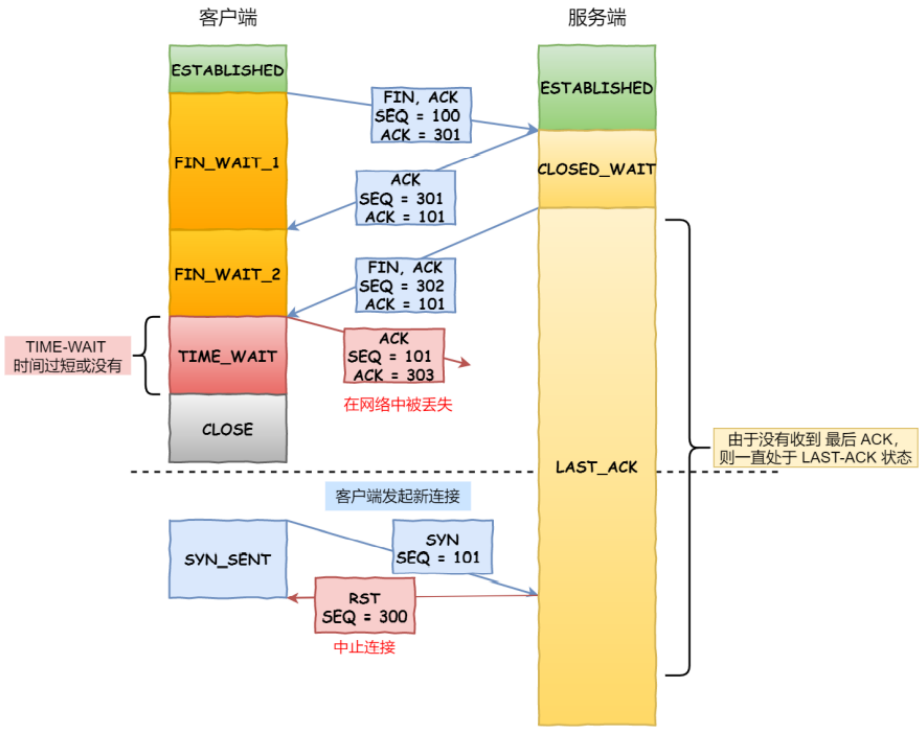
（2）为什么是 2 MSL 的时长呢?

这其实是相当于至少允许报文丢失一次。比如，若 ACK 在一个 MSL 内丢失，这样被 动方〕发的 FIN 会在第 2 个 MSL 内到达，TIME_WAIT 状态的连接可以应对。

在Linux系统中，MSL的值固定为30秒。

等待足够的时间以确保最后的ACK能让被动关闭方接收，从而帮助其正确关闭

（3）假设TIME_WAIT没有等待或等待的时间过短，断开连接会造成什么?



如上图红色框框客户端四次挥手的最后一个 ACK 报文如果在网络中被丢失了，此时如果客户端 TIMEWAIT 过短或没有，则就直接进入了 CLOSE 状态了，那么服务端则会一直处在 LAST-ACK 状态。

当客户端发起建立连接的 SYN 请求报文后，服务端会发送 RST 报文给客户端，连接建立的过程就会被终止。

注：

Linux 提供了 tcp_max_tw_buckets 参数，当 TIME_WAIT 的连接数量超过该参数时，新关闭的连接就不再经历 TIME_WAIT 而直接关闭。查看系统的TIME_WAIT的连接数量：

```
(base) [redacted]~$ cat /proc/sys/net/ipv4/tcp_max_tw_buckets
262144
```

但tcp_max_tw_buckets 也不是越大越好，毕竟内存和端口都是有限的。

还有一种方式是复用处于TIME_WAIT状态的连接，那就是打开 tcp_tw_reuse 参数。

但是需要注意，该参数是只用于客户端（建立连接的发起方），因为是在调用 connect() 时起作用的，而对于服务端（被动 连接方）是没有用的。

tcp_tw_reuse 从协议角度理解是安全可控的，可以复用处于 TIME_WAIT 的端口为新的连接所用。

什么是协议角度理解的安全可控呢?

主要有两点：

（1）只适用于连接发起方，也就是 C/S 模型中的客户端；

（2）对应的 TIME_WAIT 状态的连接创建时间超过 1 秒才可以被复用。

使用这个选项，还有一个前提，需要打开对 TCP 时间戳的支持（对方也要打开 ）。

由于引入了时间戳，带来了一些好处：

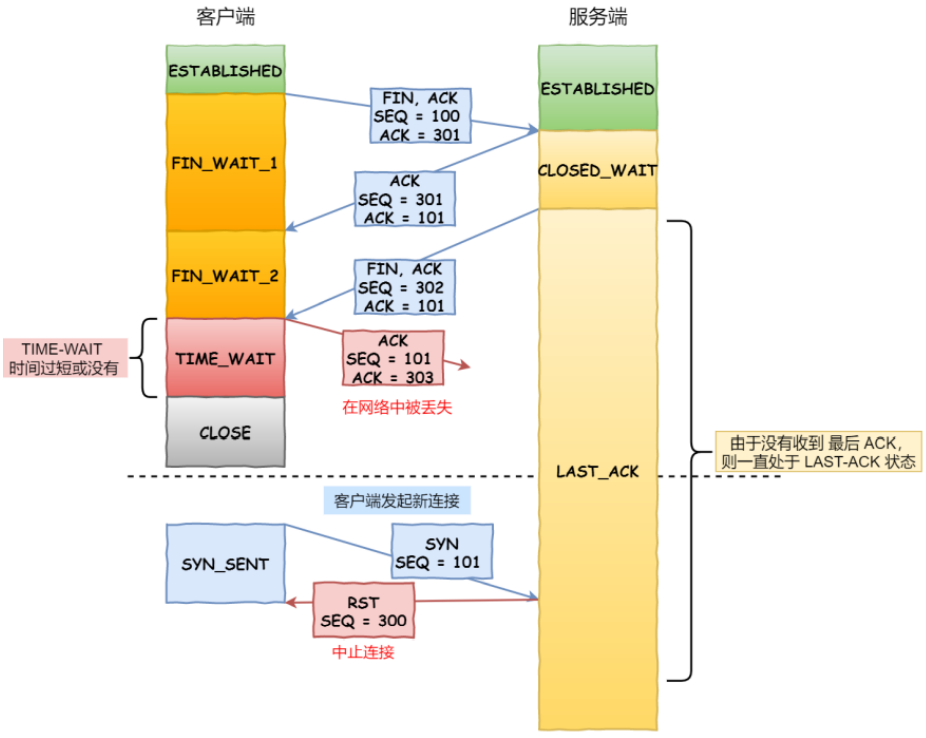
1. 我们在前面提到的 2MSL 问题就不复存在了，因为❩复的数据包会因为时间戳过期被自然丢弃；

2. 同时，它还可以防止序列号绕回，也是因为）复的数据包会由于时间戳过期被自然丢弃。

时间戳是在 TCP 的选项字段里定义的，开启了时间戳功能，在 TCP 报文传输的时候会带上发送报文的时间戳。

TCP option 字段			
类型	总长度（字节）	数据	描述
0	-	-	选项列表末尾标识
1	-	-	没有意义，用于 32 位对齐使用
2	4	MSS 值	三次握手时，发送端告知可以接收的最大报文段大小
3	3	窗口移位	指明最大窗口扩展后的大小
4	2	-	表明支持 SACK 选择性确认功能
5	可变	确认报文段	选择性确认窗口中间的报文段
8	10	Timestamps 时间戳	用于更精准的计算 RTT，以及解决序列号绕回的问题
14	3	校验和算法	双方认可后，可使用新的校验和算法
15	可变	校验和	当 16 位标准校验和放不下时，放置在这里
34	可变	FOC	TFO中Cookie

开启tcp_tw_reuse功能后，如果四次挥手中的最后一次ACK在网络中丢失了，会发生什么?

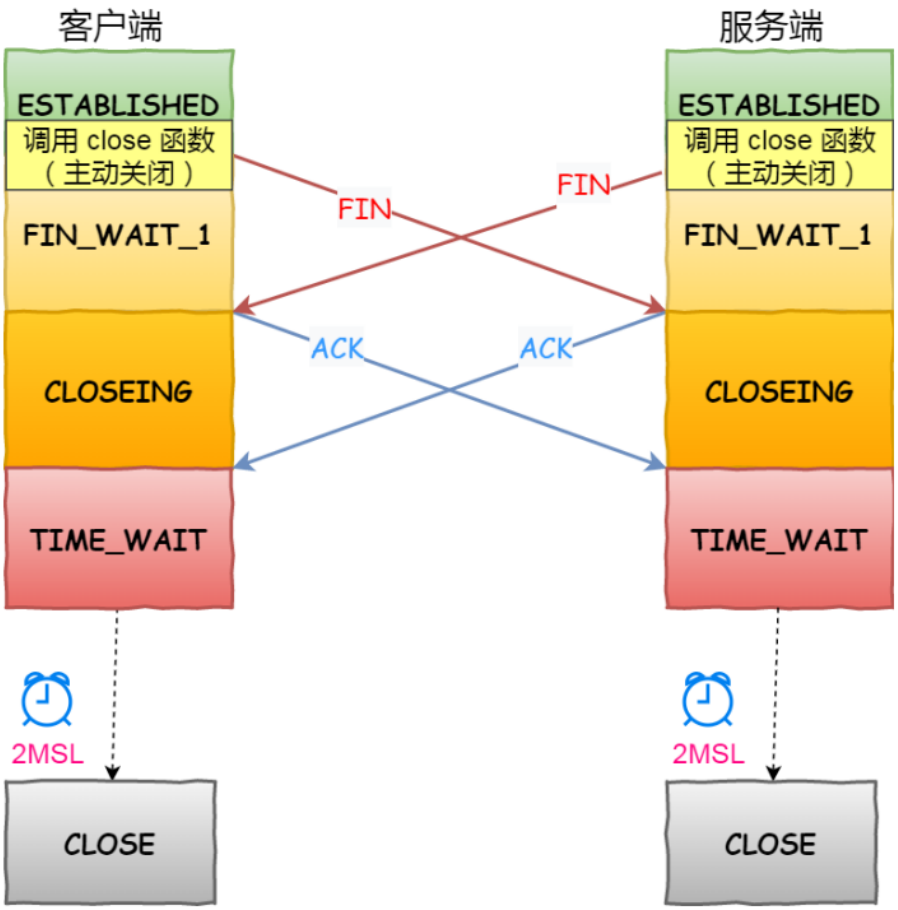


被动方的优化

1、如果连接双方同时关闭连接，会怎么样?

由于 TCP 是双全工的协议，所以是会出现两方同时关闭连接的现象，也就是同时发送了 FIN 报文。

此时，上面介绍的优化策略仍然适用。两方发送 FIN 报文时，都认为自己是主动方，所以都进入了 FIN_WAIT1 状态，FIN 报文的重发次数仍由 tcp_orphan_retries 参数控制。



接下来，双方在等待 ACK 报文的过程中，都等来了 FIN 报文。这是一种新情况，所以连接会进入一种叫做 CLOSING 的新状态，它替代了 FIN_WAIT2 状态。接着，双方内核回复 ACK 确认对方发送通道的关闭后，进入 TIME_WAIT 状态，等待 2MSL 的时间后，连接自动关闭。

小结：

针对 TCP 四次挥手的优化，我们需要根据主动方和被动方四次挥手状态变化来调整系统 TCP 内核参数。

优化四次挥手的策略	
策略	**TCP 内核参数**
调整 FIN 报文重传次数	tcp_orphan_retries
调整 FIN_WAIT2 状态的时间 (只适用 close 函数关闭的连接)	tcp_fin_timeout
调整孤儿连接的上限个数 (只适用 close 函数关闭的连接)	tcp_max_orphans
调整 time_wait 状态的上限个数	tcp_max_tw_buckets
复用 time_wait 状态的连接 (只适用客户端)	tcp_tw_reuse 、tcp_timestamps

2、TCP数据传输性能的提升

TCP 连接是由内核维护的，内核会为每个连接建立内存缓冲区：

如果连接的内存配置过小，就无法充分使用网络带宽，TCP 传输效率就会降低；
如果连接的内存配置过大，很容易把服务器资源耗尽，这样就会导致新连接无法建立；

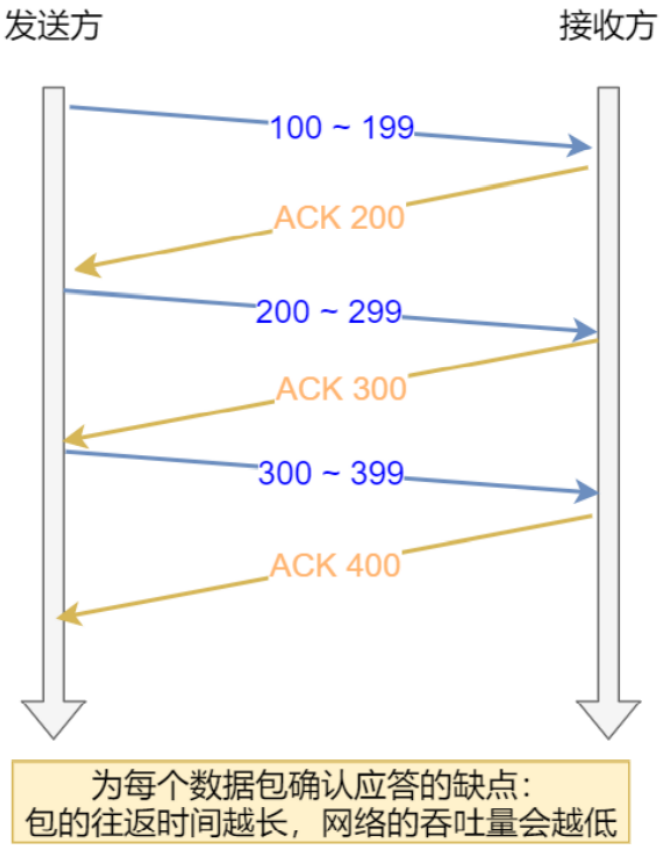
因此，我们必须理解 Linux 下 TCP 内存的用途，才能正确地配置内存大小。

（1）滑动窗口是如何影响传输速度的?

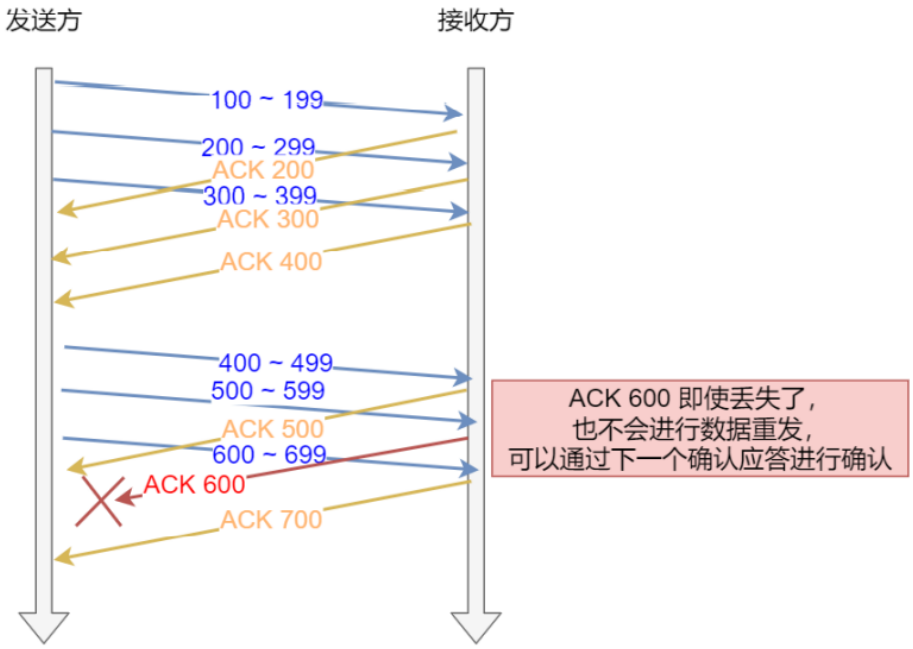
TCP 会保证每一个报文都能够抵达对方，机制如下：报文发出去后，必须接收到对方返回的确认报文 ACK，如果迟迟未收到，就会超时）发该报文，直到收到对方的 ACK 为止。

因此TCP报文发出去后，并不会立马从内存中删除，因为重传时还需要用到。由于TCP是由内核维护的，所以报文存放在内核缓冲区。

应答：



批量发送，批量应答：



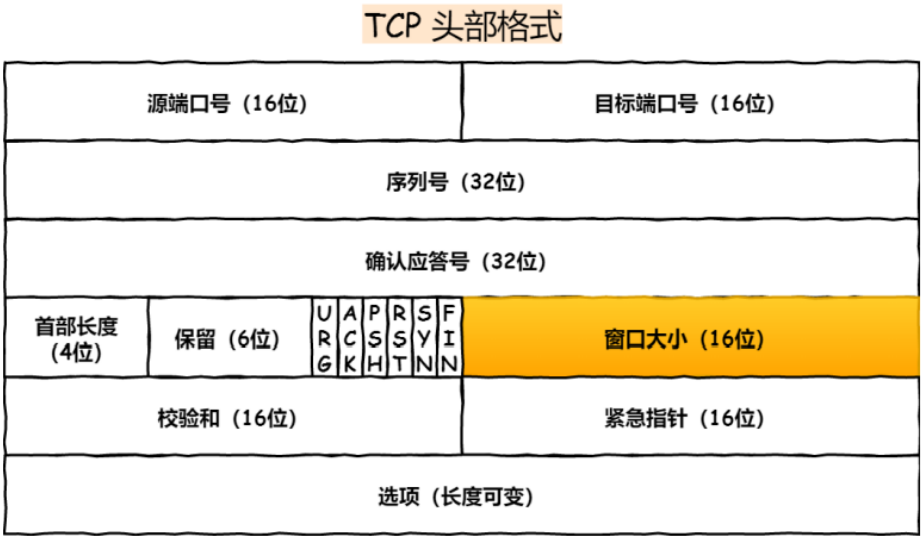
引发的问题：

当接收方硬件不如发送方，或者系统繁忙、资源紧张时，是无法瞬间处理这么多报文的。于是，这些报文只能被丢掉，使得网络效率非常低。

为了解决这种现象发生，TCP 提供一种机制可以让「发送方」根据「接收方」的实际接收能力控制发送的数据量，这就是滑动窗口的由来。

接收方根据它的缓冲区，可以计算出后续能够接收多少字节的报文，这个数字叫做接收窗口。当内核接收到报文时，必须用缓冲区存放它们，这样剩余缓冲区空间变小，接收窗口也就变小了；当进程调用 read 函数后，数据被读入了用户空间，内核缓冲区就被清空，这意味着主机可以接收更多的报文，接收窗口就会变大。

因此，接收窗口并不是恒定不变的，接收方会把当前可接收的大小放在 TCP 报文头部中的窗口字段，这样就可以起到窗口大小通知的作用。



从上图中可以看到，窗口字段只有 2 个字节，因此它最多能表达 65535 字节大小的窗口，也就是 64KB 大小。

这个窗口大小最大值，在当今高速网络下，很明显是不够用的。

所以后续有了扩充窗口的方法：

在 TCP 选项字段定义了窗口扩大因子，用于扩大 TCP 通告窗口，其值大小是 2^14，这样就使 TCP 的窗口大小从 16 位扩大为 30 位（2^16 * 2^ 14 = 2^30），所以此时窗口的最大值可以达到 1GB。

TCP option 字段			
类型	总长度（字节）	数据	描述
0	-	-	选项列表末尾标识
1	-	-	没有意义，用于 32 位对齐使用
2	4	MSS 值	三次握手时，发送端告知可以接收的最大报文段大小
3	3	窗口移位	指明最大窗口扩展后的大小
4	2	-	表明支持 SACK 选择性确认功能
5	可变	确认报文段	选择性确认窗口中间的报文段
8	10	Timestamps 时间戳	用于更精准的计算 RTT，以及解决序列号绕回的问题
14	3	校验和算法	双方认可后，可使用新的校验和算法
15	可变	校验和	当 16 位标准校验和放不下时，放置在这里
34	可变	FOC	TFO中Cookie

网络的传输能力是有限的，当发送方依据发送窗口，发送超过网络处理能力的报文时，路由器会直接丢弃这些报文。因此，缓冲区的内存并不是越大越好。

（2）如何确定最大传输速度?

TCP的传输速度，受限于发送窗口，接收窗口以及网络设备传输能力。

其中，窗口大小由内核缓冲区大小决定。如果缓冲区与网络传输能力匹配，那么缓冲区的利用率就达到了最大化。

（3）如何计算网络传输能力呢?

网络是有「带宽」限制的，带宽描述的是网络传输能力

它与内核缓冲区的计量单位不同:

带宽是单位时间内的流5，表达是「速度」，比如常见的带宽 100 MB/s；
缓冲区单位是字节，当网络速度乘以时间才能得到字节数。

带宽时延积BDP决定网络中飞行报文的大小。所谓【飞行】，是指它们就在网络线路、路由器等网络设备上。

带宽时延积 BDP = RTT * 带宽

其中：RTT为网络时延。

由于发送缓冲区大小决定了发送窗口的上限，而发送窗口又决定了「已发送未确认」的飞行报文的上限。因此，发送缓冲区不能超过「带宽时延积」。

发送缓冲区与带宽时延积的关系如下:

如果发送缓冲区「超过」带宽时延积，超出的部分就没办法有效的网络传输，同时导致网络过载，容易丢包；

如果发送缓冲区「小于」带宽时延积，就不能很好的发挥出网络的传输效率。

（4）怎样调整缓冲区大小?

发送缓冲区大小可自行调节，三个数值分别为：动态范围的最小值，初始默认值，动态范围的最大值。

```
(base)  ~$ cat /proc/sys/net/ipv4/tcp_wmem
4096    16384   4194304
```

接收缓冲区范围调节

```
~$ cat /proc/sys/net/ipv4/tcp_rmem
4096    87380   6291456
```

接收缓冲区可以根据系统空闲内存的大小来调节接收窗口：

如果系统的空闲内存很多，就可以自动把缓冲区增大一些，这样传给对方的接收窗口也会变大，因而提升发 送方发送的传输数据数

反之，如果系统的内存很紧张，就会减少缓冲区，这虽然会降低传输效率，可以保证更多的并发连接正常工作

发送缓冲区的调节功能是自动开启的，而接收缓冲区则需要配置 tcp_moderate_rcvbuf 为 1 来开启调节功能

调节TCP内存范围

上面三个数字单位不是字节，而是「页面大小」，1 页表示 4KB，它们分别表示：

- 当 TCP 内存小于第 1 个值时，不需要进行自动调节；
- 在第 1 和第 2 个值之间时，内核开始调节接收缓冲区的大小；
- 大于第 3 个值时，内核不再为 TCP 分配新内存，此时新连接是无法建立的；

根据实际场景调节的策略

在高并发服务器中，为了兼顾网速与大量的并发连接，**我们应当保证缓冲区的动态调整的最大值达到带宽时延积，而最小值保持默认的 4K 不变即可。而对于内存紧张的服务而言，调低默认值是提高并发的有效手段。**

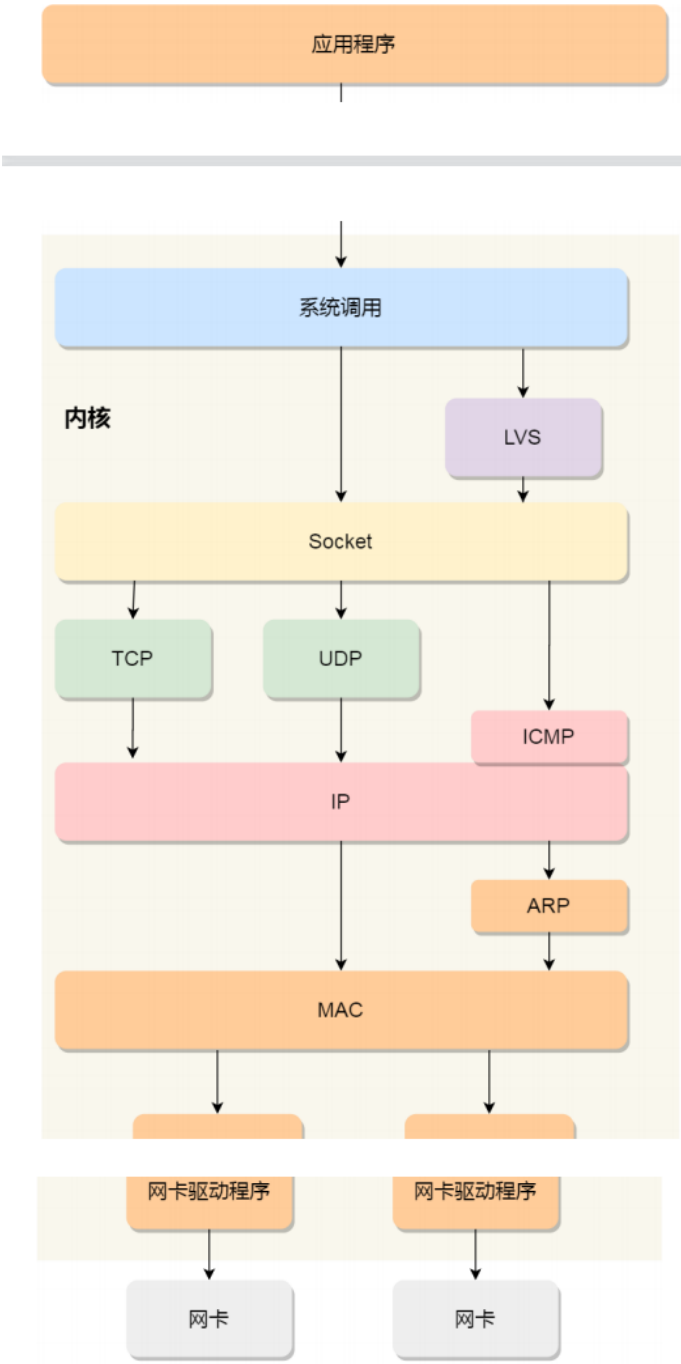
同时，如果这是网络 IO 型服务器，那么，**调大 tcp_mem 的上限可以让 TCP 连接使用更多的系统内存，这有利于提升并发能力**。需要注意的是，tcp_wmem 和 tcp_rmem 的单位是字节，而 tcp_mem 的单位是页面大小。而且，**千万不要在 socket 上直接设置 SO_SNDBUF 或者 SO_RCVBUF，这样会关闭缓冲区的动态调整功能。**

小结：

数据传输的优化策略	
策略	TCP 内核参数
扩大窗口大小	tcp_window_scaling
调整发送缓冲区范围	tcp_wmem
调整接收缓冲区范围	tcp_rmem
打开接收缓冲区动态调节	tcp_moderate_rcvbuf
调整内存范围	tcp_mem

Linux是如何收发网络包的

Linux网络协议栈



从上述网络协议栈，可以看出：

- 应用程序需要通过系统调用，来跟 Socket 层进行数据交互；
- Socket 层的下面就是传输层、网络层和网络接口层；
- 最下面的一层，则是网卡驱动程序和硬件网卡设备；

收发流程

网卡是计算机里的一个硬件，专门负责接收和发送网络包，当网卡接收到一个网络包后，会通过 DMA 技术，将网络包放入到 Ring Buffer，这个是一个环形缓冲区。

1、发送

首先，应用程序会调用 Socket 发送数据包的接口，由于这个是系统调用，所以会从用户态陷入到内核态中的 Socket 层，Socket 层会将应用层数据拷贝到 Socket 发送缓冲区中。

接下来，网络协议栈从 Socket 发送缓冲区中取出数据包，并按照 TCP/IP 协议栈从上到下逐层处理。

如果使用的是 TCP 传输协议发送数据，那么会在传输层增加 TCP 包头，然后交给网络层；

网络层会给数据包增加 IP 包，然后通过查询路由表确认下一跳的 IP，并按照 MTU 大小进行分片。 分片后的网络包，就会被送到网络接口层，在这里会通过 ARP 协议获得下一跳的 MAC 地址，然后增加帧头和帧尾，放到发包队列中。

这一些准备好后，会触发软中断告诉网卡驱动程序，这里有新的网络包需要发送，最后驱动程序通过 DMA，从发包队列中读取网络包，将其放入到硬件网卡的队列中，随后物理网卡再将它发送出去。

2、接收

为了解决频繁中断带来的性能开销，Linux 内核在 2.6 版本中引入了 NAPI 机制，它是混合「中断和轮询」的方式来接收网络包，它的核心概念就是不采用中断的方式读取数据，而是首先采用中断唤醒数据接收的服务程序，然后 poll 的方法来轮询数据。

比如，当有网络包到达时，网卡发起硬件中断，于是会执行网卡硬件中断处理函数，中断处理函数处理完需要「暂时屏蔽中断」，然后唤醒「软中断」来轮询处理数据，直到没有新数据时才恢复中断，这样一次中断处理多个网络包，于是就可以降低网卡中断带来的性能开销。

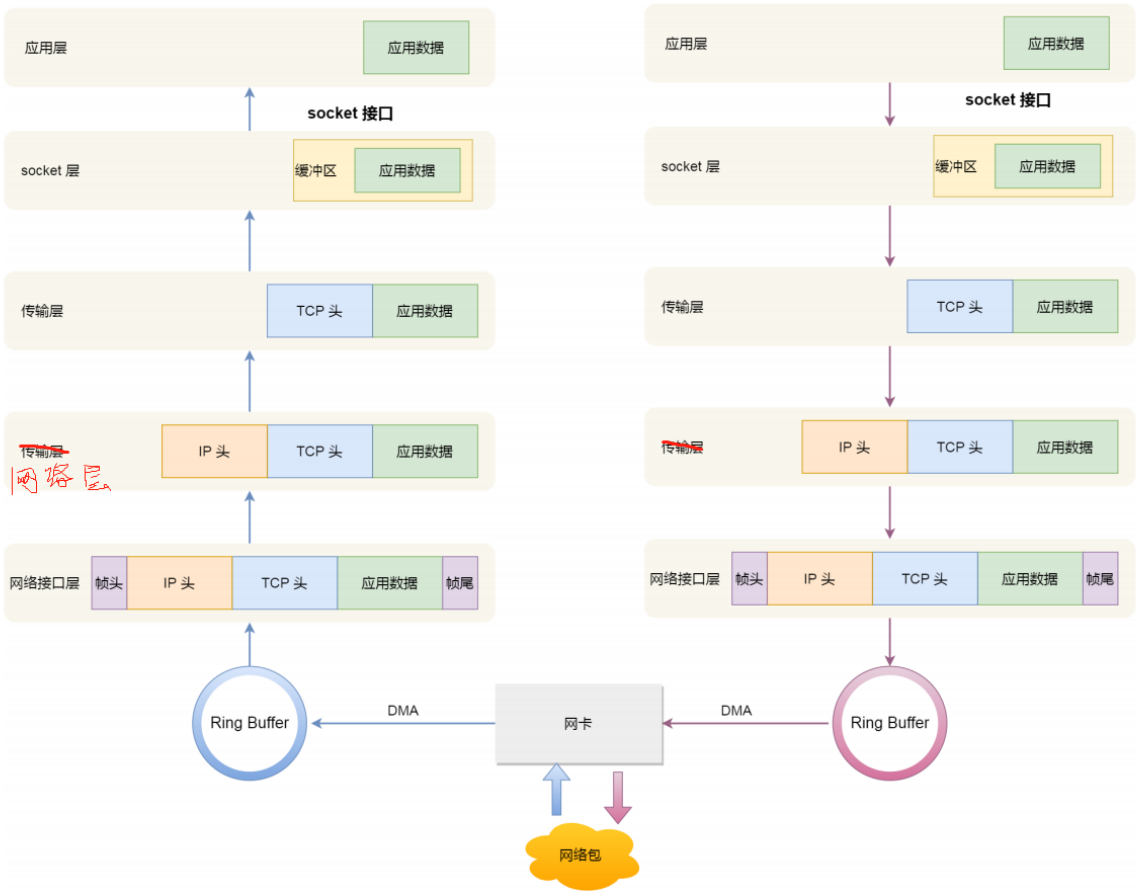
软中断会从 Ring Buffer 中拷贝数据到内核 struct sk_buff 缓冲区中，从而可以作为一个网络包交给网络协议栈进行逐层处理。

首先，会先进入到网络接口层，在这一层会检查报文的合法性，如果不合法则丢弃，合法则会找出该网络包的上层协议的类型，比如是 IPv4，还是 IPv6，接着再去掉帧头和帧尾，然后交给网络层。到了网络层，则取出 IP 包，判断网络包下一步的走向，比如是交给上层处理还是转发出去。

当确认这个网络包要发送给本机后，就会从 IP 头里看看上一层协议的类型是 TCP 还是 UDP，接着去掉 IP 头，然后交给传输层。

传输层取出 TCP 头或 UDP 头，根据四元组「源 IP、源端口、目的 IP、目的端口」 作为标识，找出对应的 Socket，并把数据拷贝到 Socket 的接收缓冲区。

最后，应用层程序调用 Socket 接口，从内核的 Socket 接收缓冲区读取新到来的数据到应用层。



面试题

> 本部分面试题仅做参考，读者可以结合前面的基础知识讲解形成自己的答案。

OSI 七层参考模型

OSI（Open System Interconnection）模型，即开放式系统互联，是国际标准化组织（ISO）制定的一个用于计算机或通信系统间互联的标准体系，旨在**将计算机网络通信划分为七个不同的层级，每个层级都负责特定的功能。每个层级都构建在其下方的层级之上，并为上方的层级提供服务。七层从下到上分别是物理层、数据链路层、网络层、传输层、会话层、表示层和应用层。可以简称为“物数网传会表应”。**

1. **物理层**：负责物理传输媒介的传输，例如电缆、光纤或无线信号。主要作用是传输比特流（就是由 1、0 转化为电流强弱来进行传输，到达目的地后再转化为 1、0，也就是我们常说的数模转换与模数转换）。**这一层的数据叫做比特。**
2. **数据链路层**：建立逻辑连接、进行硬件地址寻址、差错校验等功能。定义了如何让格式化数据以帧为单位进行传输，以及如何控制对物理介质的访问。将比特组合成字节进而组合成帧，用 MAC 地址访问介质，**传输单位是桢。**
3. **网络层**：**负责数据的路由和转发**，选择最佳路径将数据从源主机传输到目标主机。它使用IP地址来标识不同主机和网络，并进行逻辑地址寻址。**传输单位是数据报。常见的协议有ICMP、ARP、IP**
4. **传输层**：**提供端到端的数据传输服务。**它使用TCP（传输控制协议）和UDP（用户数据报协议）来管理数据传输。
5. **会话层**：建立、管理和终止应用程序之间的会话连接。它处理会话建立、维护和终止，以及处理会话过程中的异常情况。
6. **表示层**：负责数据的格式转换、加密和解密，确保数据在不同系统之间的正确解释和呈现，也就是把计算机能

够识别的东西转换成人能够能识别的东西（如图片、声音等）。
7. **应用层**：网络服务与最终用户的一个接口。这一层为用户的应用程序（例如电子邮件、文件传输和终端仿真）提供网络服务。**常见的协议有：FTP、SMTP、HTTP、DNS。**

OSI七层网络模型为网络通信的不同功能提供了逻辑上的划分，为网络协议的设计和实现提供了标准化的框架。然而，在实际网络通信中，常用的TCP/IP协议栈更为广泛和普遍。

TCP/IP四层网络模型

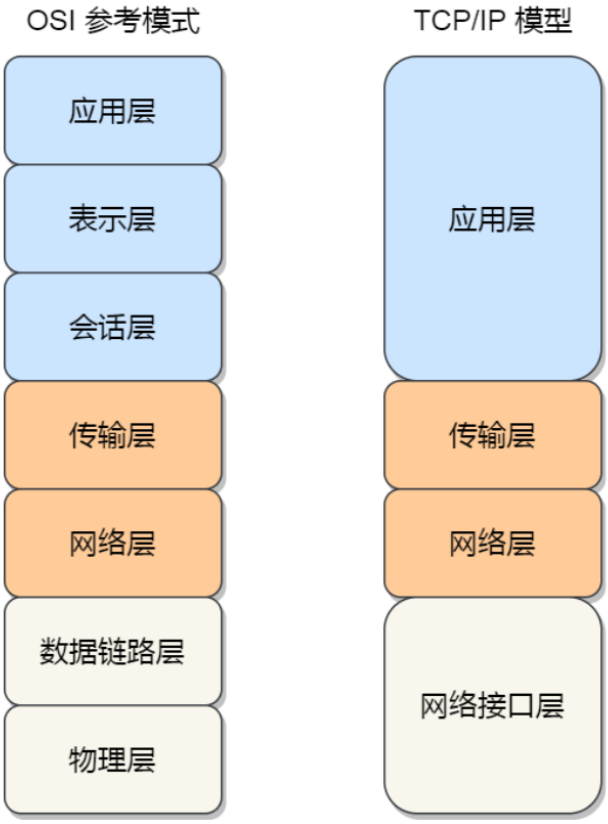
TCP/IP模型是一种用于组织和描述计算机网络通信的标准模型，**它是互联网最常用的协议栈。**TCP/IP模型由两个主要协议组成：TCP（Transmission Control Protocol）和IP（Internet Protocol）。它是互联网通信的基础，也被广泛用于局域网和广域网等各种网络环境。

TCP/IP模型分为四个层级，每个层级负责特定的网络功能。以下是TCP/IP模型的层级及其功能：

1. **应用层（Application Layer）：该层与OSI模型的应用层和表示层以及会话层类似，提供直接与用户应用程序交互的接口。**它为网络上的各种应用程序提供服务，如电子邮件（SMTP）、网页浏览（HTTP）、文件传输（FTP）等。
2. **传输层（Transport Layer）：该层对应OSI模型的传输层。它负责端到端的数据传输，提供可靠的、无连接的数据传输服务。**主要的传输层协议有TCP（Transmission Control Protocol）和UDP（User Datagram Protocol）。TCP提供可靠的数据传输，确保数据的正确性和完整性；而UDP则是无连接的，适用于不要求可靠性的传输，如实时音频和视频流。
3. **网际层（Internet Layer）：该层对应OSI模型的网络层。主要协议是IP（Internet Protocol），它负责数据包的路由和转发**，选择最佳路径将数据从源主机传输到目标主机。IP协议使用IP地址来标识主机和网络，并进行逻辑地址寻址。
4. **网络接口层（Link Layer）：该层对应OSI模型的数据链路层和物理层。**它负责物理传输媒介的传输，例如以太网、Wi-Fi等，并提供错误检测和纠正的功能。此外，网络接口层还包含硬件地址（MAC地址）的管理。

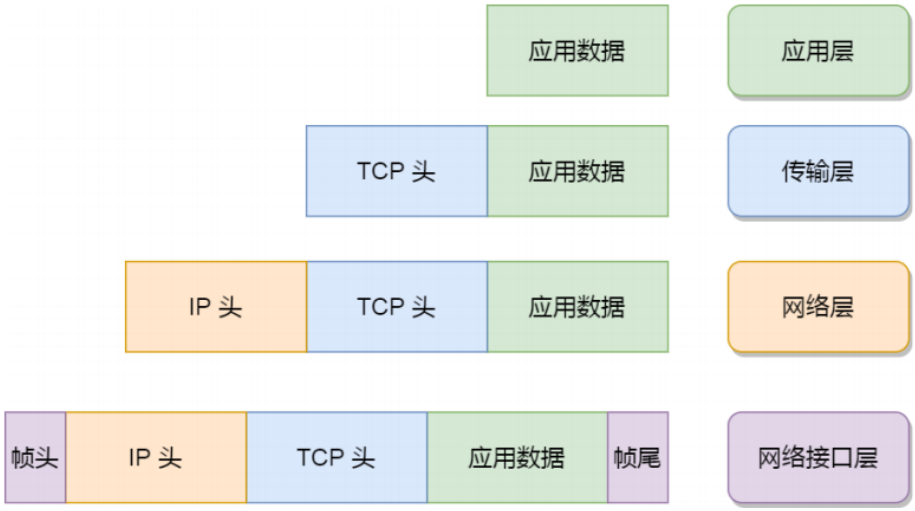
TCP/IP模型和OSI七层网络模型有些相似，但并不完全一样。TCP/IP模型较为简洁，只分为四个层级，而OSI模型分为七个层级。虽然OSI模型在理论上更全面，但在实际网络通信中，TCP/IP模型更为实用，并且成为了互联网通信的主要参考模型。

两种网络模型的对比



- 应用层，负责向用户提供一组应用程序，比如 HTTP、DNS、FTP 等；
- 传输层，负责端到端的通信，比如 TCP、UDP 等；
- 网络层，负责网络包的封装、分片、路由、转发，比如 IP、ICMP 等；
- 网络接口层，负责网络包在物理网络中的传输，比如网络包的封帧、 MAC 寻址、差错检测，以及通过网卡传输网络帧等；

对应的数据封装：



五层网络体系结构

> 五层网络体系结构是综合了OSI模型和TCP/IP模型所得来的。

五层网络体系结构分别为：**应用层、运输层、网络层、数据链路层、物理层**。各层功能分别如下：

1. 应用层（Application Layer）：与直接为用户的应用进程提供服务，是操作系统中的用户态，常见的有支持万维网应用的HTTP协议、支持电子邮件的SMTP协议，支持文件传送的FTP协议等等。
2. 传输层（Transport Layer）：负责向两个主机中**进程**之间的通信提供服务，是端（端口）到端的通信。传输层有两个传输协议。
    - TCP：面向连接的、可靠的传输控制协议
    - UDP: 无连接的，不提供可靠服务的用户数据报协议。
3. 网络层（Network Layer）：负责数据的路由和转发。它选择最佳路径将数据从源主机传输到目标主机，并使用逻辑地址（如IP地址）来标识主机和网络。
4. 数据链路层（Data Link Layer）：在直连网络中传输数据帧。它提供错误检测和纠正的功能，并负责数据的帧同步、地址寻址和流量控制。在这一层级上，通常会使用MAC地址来标识网络设备。
5. 物理层（Physical Layer）：负责物理传输媒介的传输。这包括电缆、光纤、无线信号等。该层级定义了传输数据位的形式、电压级别、传输速率等特性。

从输入URL到页面展示发生了什么?

1. **URL 输入**：用户在浏览器的地址栏中输入 URL，例如 "https://www.example.com"。
2. **域名解析**：浏览器通过域名系统（DNS）将域名解析为 IP 地址，以确定要连接的服务器位置。
3. **建立连接**：浏览器使用解析得到的 IP 地址，与服务器建立网络连接。这通常涉及使用 TCP 协议进行三次握手。
4. **发送请求**：浏览器向服务器发送 HTTP 请求，请求服务器的网页内容。请求中包含了要访问的路径、方法（GET、POST等）、头部信息等。
5. **服务器处理**：服务器接收到请求后，根据请求的内容和路径，处理请求并返回响应。服务器可能从数据库中获取数据，生成动态内容，然后将响应发送回浏览器。
6. **接收响应**：浏览器接收到服务器的响应，响应包含了 HTTP 状态码、头部信息和页面内容等。
7. **解析和渲染**：浏览器开始解析响应内容，构建文档对象模型（DOM）和渲染树。它解析 HTML、CSS 和 JavaScript，并确定页面的结构、样式和行为。
8. **页面渲染**：浏览器使用渲染树和样式信息，将页面内容绘制到屏幕上。这包括布局、绘制和显示页面元素。
9. **执行 JavaScript**：如果页面包含 JavaScript，浏览器会执行 JavaScript 代码，添加交互和动态行为。
10. **加载资源**：页面中可能包含外部资源，如图片、样式表、脚本文件等。浏览器会根据需要下载这些资源，以完整地呈现页面。
11. **完成页面加载**：页面的所有内容和资源加载完成后，浏览器显示完整的页面。

HTTP报文

HTTP请求报文

其主要由请求行、请求头、请求体构成

1、请求行：(请求方法 URI 协议版本号)

- **请求方法：** GET、POST、PUT、DELETE、PATCH、HEAD、OPTIONS、TRACE
- **URL：** `<协议>://<主机>:<端口>/<路径>?<参数>`

- **协议版本号：** HTTP版本号

例如：`POST /chapter17/user.html HTTP/1.1`

2、请求头

包含请求的附加信息，有key：value组成, 它可以包含很多不同的字段，用于告知服务器有关请求的详细信息。一些常见的请求头部字段包括：

- Host：指定服务器的主机名和端口号。
- User-Agent：标识客户端的用户代理（浏览器或其他工具）。
- Accept：指定客户端可以接受的响应的MIME类型。
- Content-Type：指定请求主体的MIME类型。
- Authorization：用于进行身份验证的凭据。

```
Host: example.com
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/88.0.4324.96 Safari/537.36
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8
Content-Type: application/json
Authorization: Bearer <token>
```

3、空行： 空行是请求头部和请求主体之间的空行，用于分隔请求头部和请求主体。

4、请求体

承载多个请求参数的数据，请求主体是可选的，通常在发送POST、PUT等请求时包含请求的实际数据。例如，在使用POST请求提交表单数据或上传文件时，请求主体会包含这些数据。

一个完整的HTTP请求报文示例如下：

```
GET /example/index.html HTTP/1.1
Host: example.com
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/88.0.4324.96 Safari/537.36
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8
```

HTTP响应报文

HTTP响应报文是服务器向客户端返回的数据格式，用于传达服务器对客户端请求的处理结果以及相关的数据。一个标准的HTTP响应报文通常包含状态行、响应头、响应体：

1、**状态行（Status Line）：** 状态行包括三个主要部分，用空格分隔：

- HTTP协议版本（通常是"HTTP/1.1"）
- 状态码（表示服务器处理结果的三位数字代码）
- 状态消息（对状态码的简要描述）

例如：`HTTP/1.1 200 OK`

2、**响应头部（Response Headers）：** 响应头部也是以键值对的形式提供的额外信息，类似于请求头部，用于告知客户端有关响应的详细信息。一些常见的响应头部字段包括：

- Content-Type：指定响应主体的MIME类型。
- Content-Length：指定响应主体的长度（字节数）。
- Server：指定服务器的信息。
- Location：在重定向时指定新的资源位置。
- Set-Cookie：在响应中设置Cookie。

```
HTTP/1.1 200 OK
Content-Type: text/html; charset=UTF-8
Content-Length: 1234
Server: Apache/2.4.38 (Unix)
Set-Cookie: session_id=abcd1234; Expires=Wed, 11 Aug 2023 00:00:00 GMT
```

3、**空行（Empty Line）：** 空行是响应头部和响应主体之间的空行，用于分隔响应头部和响应主体。

4、**响应主体（Response Body）：** 响应主体包含服务器返回给客户端的实际数据。例如，当请求一个网页时，响应主体将包含HTML内容。响应主体的存在与否取决于请求的性质以及服务器的处理结果。

一个完整的HTTP响应报文示例如下：

```
HTTP/1.1 200 OK
Content-Type: text/html; charset=UTF-8
Content-Length: 1234
Server: Apache/2.4.38 (Unix)
Set-Cookie: session_id=abcd1234; Expires=Wed, 11 Aug 2023 00:00:00 GMT

<!DOCTYPE html>
<html>
<head>
  <title>Example Page</title>
</head>
<body>
  <h1>Hello, World!</h1>
</body>
</html>
```

HTTP常见字段

通用头部字段（General Headers）：

- Cache-Control：指定缓存策略。
- Connection：控制连接的行为。
- Date：指定日期和时间。

请求头部字段（Request Headers）：

- Accept：指定客户端能够接受的响应的MIME类型。
- Accept-Encoding：指定客户端支持的内容编码方式。
- Authorization：用于进行身份验证的凭据。
- Host：指定请求的目标主机和端口。
- User-Agent：标识客户端的用户代理（浏览器或其他工具）。

什么是TCP粘包问题？如何解决TCP粘包问题？

- TCP粘包和拆包问题

  TCP是一个“流”协议，所谓流，就是**没有界限的一长串二进制数据**。TCP作为传输层协议并不了解上层业务数据的具体含义，它会根据TCP缓冲区的实际情况进行数据包的划分，所以在业务上认为是一个完整的包，可能会被TCP拆分成多个包进行发送，也有可能把多个小的包封装成一个大的数据包发送，这就是所谓的TCP粘包和拆包问题。

- 产生TCP粘包和拆包的原因

  我们知道TCP是以流动的方式传输数据的，传输的**最小单位**为一个报文段（Segment）。TCP Header中有个Options标识位。常见的标识位为MSS（Maximum Segment Size）指的是，连接层每次传输的数据有个最大限制MTU（Maximum Transmission Unit），一般是1500bit，超过这个量要分成多个报文段，MSS则是这个最大限制减去TCP的header，光是要传输的数据的大小，一般为1460bit。换算成字节，也就是180多字节。 TCP为提高性能，发送端会将需要发送的数据发送到缓冲区，等待缓冲区满了以后，再将缓冲中的数据发送到接收方。同理，接收方也有缓冲区这样的机制来接受数据。 发生TCP粘包、拆包主要是以下原因：（1）应用程序写入数据大于套接字缓冲区大小，会发生拆包；（2）应用程序写入数据小于套接字缓冲区大小，网卡将应用多次写入的数据发送到网络上，这将会发送粘包；（3）进行MSS（最大报文长度）大小的TCP分段，当TCP报文长度——TCP header长度>MSS 的时候会发生拆包；（4）接收方法不及时读取套接字缓冲区数据，这将发生粘包。

- 如何处理粘包和拆包

假设应用层协议是http

我从浏览器中访问了一个网站，网站服务器给我发了200k的数据。建立连接的时候，通告的MSS是50k，所以为了防止ip层分片，tcp每次只会发送50k的数据，一共发了4个tcp数据包。如果我又访问了另一个网站，这个网站给我发了100k的数据，这次tcp会发出2个包，问题是，客户端收到6个包，怎么知道前4个包是一个页面，后两个是一个页面。既然是tcp将这些包分开了，那tcp会将这些包重组吗，它送给应用层的是什么？这是我自己想的一个场景，正式一点讲的话，**这个现象叫拆包**。

我们再考虑一个问题。

tcp中有一个negal算法，用途是这样的：通信两端有很多小的数据包要发送，虽然传送的数据很少，但是流程一点没少，也需要tcp的各种确认，校验。这样小的数据包如果很多，会造成网络资源很大的浪费，negal算法做了这样一件事，当来了一个很小的数据包，我不急于发送这个包，而是等来了更多的包，将这些小包组合成大包之后一并发送，不就提高了网络传输的效率的嘛。这个想法收到了很好的效果，但是我们想一下，如果是分属于两个不同页面的包，被合并在了一起，那客户那边如何区分它们呢？**这就是粘包问题。**

从粘包问题我们更可以看出为什么tcp被称为流协议，因为它就跟水流一样，是没有边界的，没有消息的边界保护机制，所以**tcp只有流的概念，没有包的概念**。

我们还需要有两个概念：

（1）**长连接**： Client方与Server方先建立通讯连接，连接建立后不断开， 然后再进行报文发送和接收。

（2）**短连接**：Client方与Server每进行一次报文收发交易时才进行通讯连接，交易完毕后立即断开连接。此种方式常用于一点对多点 通讯，比如多个Client连接一个Server。

实际，我想象的**关于粘包的场景是不对的**，http连接是短连接，请求之后，收到回答，立马断开连接，不会出现粘包。 拆包现象是有可能存在的。

处理拆包这里提供两种方法：

（1）通过包头+包长+包体的协议形式，当服务器端获取到指定的包长时才说明获取完整。 （2） 指定包的结束标识，这样当我们获取到指定的标识时，说明包获取完整。

处理粘包我们从上面的分析看到，虽然像http这样的短连接协议不会出现粘包的现象，但是一旦建立了长连接，粘包还是有可能会发生的。处理粘包的方法如下：

（1）发送方对于发送方造成的粘包问题，可以通过关闭Nagle算法来解决，使用TCP_NODELAY选项来关闭算法。

（2）接收方没有办法来处理粘包现象，只能将问题交给应用层来处理。应用层的解决办法简单可行，不仅能解决接收方的粘包问题，还可以解决发送方的粘包问题。解决办法：循环处理，应用程序从接收缓存中读取分组时，读完一条数据，就应该循环读取下一条数据，直到所有数据都被处理完成，判断每条数据的长度的方法有两种：

a. 格式化数据：每条数据有固定的格式（开始符，结束符），这种方法简单易行，但是选择开始符和结束符时一定要确保每条数据的内部不包含开始符和结束符。

b. 发送长度：发送每条数据时，将数据的长度一并发送，例如规定数据的前4位是数据的长度，应用层在处理时可以根据长度来判断每个分组的开始和结束位置。

UDP会不会产生粘包问题呢?

TCP为了保证可靠传输并减少额外的开销（每次发包都要验证），采用了基于流的传输，基于流的传输不认为消息是一条一条的，是无保护消息边界的（保护消息边界：指传输协议把数据当做一条独立的消息在网上传输，接收端一次只能接受一条独立的消息）。UDP则是面向消息传输的，是有保护消息边界的，接收方一次只接受一条独立的信息，所以不存在粘包问题。

举个例子：有三个数据包，大小分别为2k、4k、6k，如果采用UDP发送的话，不管接受方的接收缓存有多大，我们必须要进行至少三次以上的发送才能把数据包发送完，但是使用TCP协议发送的话，我们只需要接受方的接收缓存有12k的大小，就可以一次把这3个数据包全部发送完毕。

HTTP请求方法

1. GET：申请获取资源，不对服务器产生影响
2. POST：POST请求通常用于发送数据，例如提交表单数据、上传文件等，会影响服务器，服务器可能动态创建新的资源或更新原有资源。
3. HEAD：类似GET，仅要求服务器返回头部信息，不返回实际的资源内容。
4. PUT：用于更新服务器上的资源或创建新资源。
5. DELETE：请求服务器删除指定的资源。
6. TRACE：用于测试。要求目标服务器返回原始的HTTP请求内容
7. PATCH： 用于对资源进行部分更新。
8. CONNECT：用于代理服务器
9. OPTIONS：用于获取服务器支持的HTTP方法列表，以及针对指定资源支持的方法

GET和POST请求的区别

概念

GET和POST都是HTTP请求方法

1、GET

申请获取资源，不对服务器产生影响

2、POST

客户端向服务器提交数据，会影响服务器，服务器可能动态创建新的资源或更新原有资源

请求数据：若为GET，则请求数据为空；其主要是在POST中进行使用，使用于需要填表单(Form)场景

参数传递

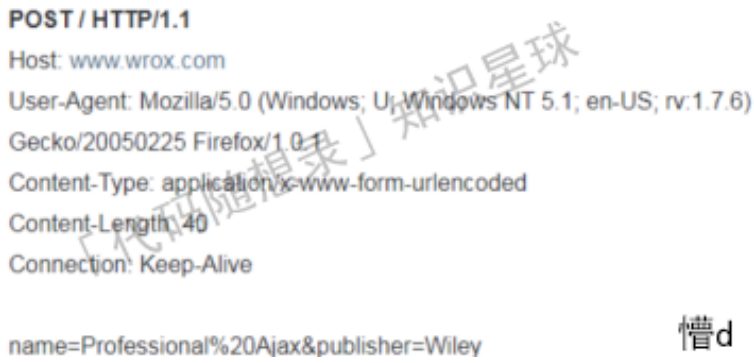
> **HTTP 协议没有 Body 和 URL 的长度限制，对 URL 限制的大多是浏览器和服务器的原因。**

- GET请求的参数一般写在URL中，所以GET传送的数据量较小，不能大于2KB，且只接受ASCII字符
- POST请求参数一般放在请求体中，所以其请求信息没有长度限制， 对于数据类型也没有限制

请求报文

1、GET请求报文：

2、POST请求报文：



请求行中请求方法为POST，URL为空，协议版本为HTTP1.1

与GET不同在于它的请求参数是位于请求数据中，最后一行name...为请求数据，且，请求数据和请求头之间必须空出一行

安全和幂等

安全：HTTP协议中，安全是指请求方法不会破坏服务器上的资源

幂等：多次执行相同的操作，结果都相同

- GET为安全幂等的，因为它为只读操作，无论操作多少次，服务器上的数据都是安全的，且每次的结果都是相同的
- **POST** 因为是「新增或提交数据」的操作，会修改服务器上的资源，所以是**不安全**的，且多次提交数据就会创建多个资源，所以**不是幂等**的。

缓存机制

- GET 请求会被浏览器主动cache，如果下一次传输的数据相同，那么就返回缓存中的内容，以求更快的展示数据，而 POST 不会，除非手动设置。
- GET 请求参数会被完整保留在浏览器历史记录里，而 POST 中的参数不会被保留。
- GET 产生的 URL 地址可以被 保存为书签，而 POST 不可以。
- GET 在浏览器回退时是无害的，而 POST 会再次提交请求。

时间消耗

- GET 产生一个 TCP 数据包：浏览器会把 header 和 data 一并发送出去，服务器响应 200（返回数据）
- POST 产生两个 TCP 数据包，对于 POST，浏览器先发送 Header，服务器响应 100 continue，浏览器再发送 data，服务器响应 200 ok（返回数据）

编码方式

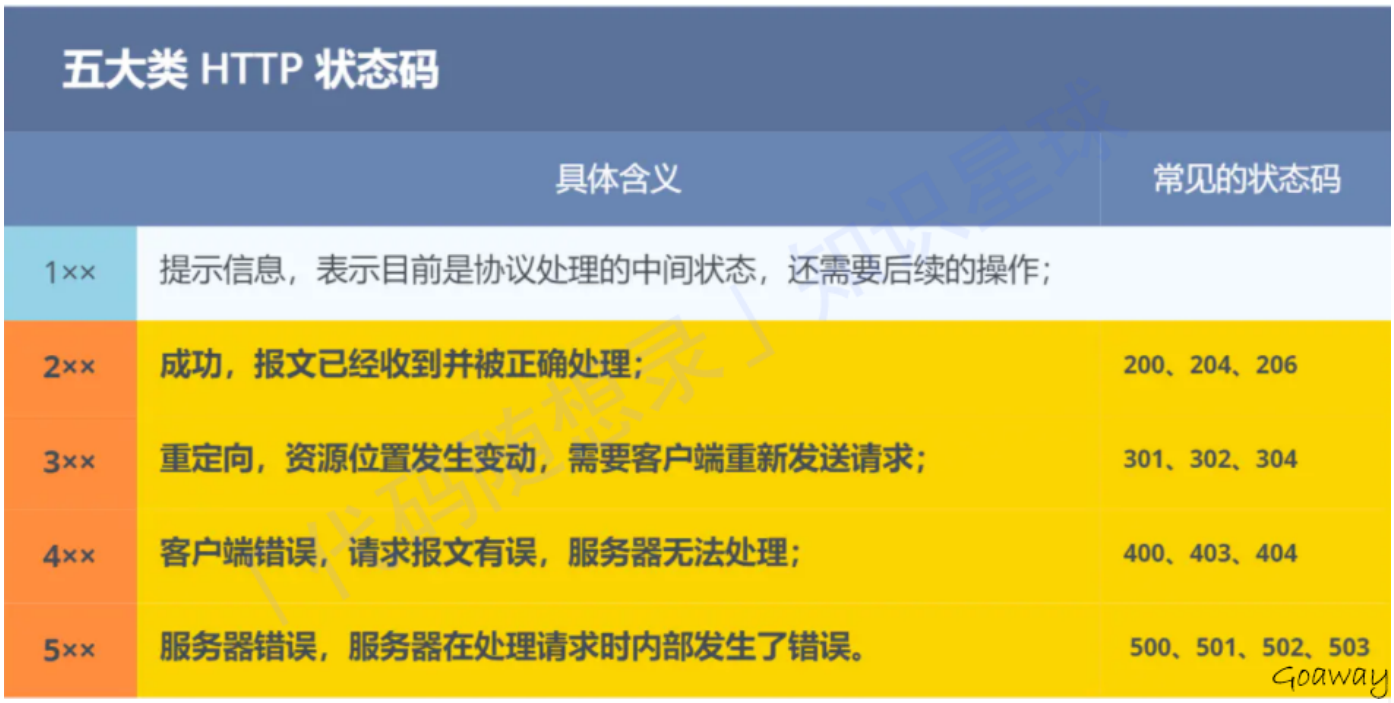
GET 请求只能进行 URL 编码`application/x-www-form-urlencoded`

POST 支持多种编码方式`application/x-www-form-urlencoded 或 multipart/form-data`。为二进制数据使用多种编码。）

实际上本质都是一样的，并无区别

1. 都是HTTP请求协议的请求方法，而HTTP又是基于TCP/IP的关于数据如何在万维网中如何通信的协议，所以GET/POST实际上都是TCP链接
2. 由于HTTP的规定以及浏览器/服务器的限制，导致它们在应用过程中可能会有所不同

HTTP状态码



200：客户端请求成功

206：partial content 服务器已经正确处理部分GET请求，实现断点续传或同时分片下载，该请求必须包含Range请求头来指示客户端期望得到的范围

301（永久重定向）：该资源已被永久移动到新位置，将来任何对该资源的访问都要使用本响应返回的若干个URL之一

302（临时重定向）：请求的资源现在临时从不同的URI中获得

304：如果客户端发送一个待条件的GET请求并且该请求以经被允许，而文档内容未被改变，则返回304,该响应不包含包体（即可直接使用缓存）

400：请求报文语法有误，服务器无法识别

401：请求需要认证

403：请求的对应资源禁止被访问

404：服务器无法找到对应资源

500：服务器内部错误

503：服务器正忙

HTTP缓存有几种

将资源（如网页、图像、脚本等）的副本存储在客户端或中间代理服务器上，以便将来的请求可以直接从缓存中获取，而不必重新从服务器下载资源。这有助于减少网络延迟，提高页面加载速度，并减轻服务器的负担。

缓存可以解决什么问题

- 减少不必要的网络传输，节约带宽
- 更快的加载页面
- 减少服务器负载，避免服务过载的情况出现

强制缓存

强缓存：浏览器判断请求的目标资源是否有效命中强缓存，如果命中，则可以直接从内存中读取目标资源，无需与服务器做任何通讯。

`Expires强缓存`：设置一个强缓存时间，此时间范围内，从内存中读取缓存并返回。

`Cache-Control强缓存`：`http1.1`中增加该字段，使用 `max-age` 指令，可以设置资源在缓存中的最长有效时间，单位为秒。例如，`Cache-Control: max-age=3600` 表示资源在缓存中保留 3600 秒

协商缓存

与强制缓存不同，协商缓存依赖于客户端和服务器之间的交互，在协商缓存中，服务器在响应中提供了资源的一些标识信息，客户端在后续请求中通过这些信息来判断资源是否发生了变化，进而判断是否需要重新传输资源。

下面是常用于协商缓存的一些头部字段

ETag 和 **If-None-Match**：

- `ETag` 是服务器为资源生成的唯一标识符，可以是根据文件内容计算出的哈希值。
- 客户端在请求头部的 `If-None-Match` 字段中携带上次响应的 `ETag` 值。
- 服务器比较请求中的 `If-None-Match` 值与当前资源的 `ETag` 值，如果匹配，表示资源未发生变化，返回状态码 `304 Not Modified`。

Last-Modified 和 If-Modified-Since：

- `Last-Modified` 是资源的最后修改时间，服务器在响应头部中返回。

- 客户端在请求头部的 `If-Modified-Since` 字段中携带上次响应的 `Last-Modified` 时间。
- 服务器比较请求中的 `If-Modified-Since` 值与当前资源的 `Last-Modified` 值，如果请求时间早于资源的最后修改时间，表示资源未发生变化，返回状态码 `304 Not Modified`。

HTTP1.0和HTTP1.1的区别?

1. 长连接

- `HTTP1.1` 支持长连接，每一个TCP连接上可以传送多个HTTP请求和响应，默认开启 `Connection:Keep-Alive`
- `HTTP1.0` 默认为短连接，每次请求都需要建立一个TCP连接。

2. 缓存

- `HTTP1.0` 主要使用 `If-Modified-Since/Expires` 来做为缓存判断的标准
- `HTTP1.1` 则引入了更多的缓存控制策略例如 `Entity tag / If-None-Match` 等更多可供选择的缓存头来控制缓存策略。

3. 管道化

- 基于 `HTTP1.1` 的长连接，使得请求管线化成为可能。管线化使得请求能够“并行”传输，但是响应必须按照请求发出的顺序依次返回，性能在一定程度上得到了改善。

4. 增加Host字段

使得一个服务器能够用来创建多个 Web 站点。

5. 状态码

新增了24个错误状态响应码

6. 带宽优化

- `HTTP1.0` 中，存在一些浪费带宽的现象，例如客户端只是需要某个对象的一部分，而服务器却将整个对象送过来了，并且不支持断点续传功能
- `HTTP1.1` 则在请求头引入了 `range` 头域，它允许只请求资源的某个部分，即返回码是 `206（Partial Content）`

HTTP1.1有什么特性

1、持久连接：只要客户端任意一端没有明确提出断开TCP连接，就一直保持连接，也称为“Keep-Alive”。

2、管线化：允许客户端在不等待前一个响应返回的情况下发送多个请求

3、增加了 PUT、DELETE、OPTIONS、PATCH 等新的方法

4、新增了24个错误状态响应码

5、新增了一些缓存的字段（`If-Modified-Since, If-None-Match`）

6、`HTTP1.1` 在请求头引入了 `range` 头域，它允许只请求资源的某个部分，即返回码是 `206（Partial Content）`

7、允许响应数据分块（chunked），利于传输大文件

8、增加 `Host` 字段：使得一个服务器能够用来创建多个 Web 站点。

HTPP1.1和HTTP1.0有什么区别

1. 长连接

- `HTTP1.1`支持长连接，每一个TCP连接上可以传送多个HTTP请求和响应，默认开启`Connection:Keep-Alive`
- `HTTP1.0`默认为短连接，每次请求都需要建立一个TCP连接。

2. 缓存

- `HTTP1.0`主要使用`If-Modified-Since/Expires`来做为缓存判断的标准
- `HTTP1.1`则引入了更多的缓存控制策略例如`Entity tag / If-None-Match`等更多可供选择的缓存头来控制缓存策略。

3. 管道化

- 基于`HTTP1.1`的长连接，使得请求管线化成为可能。管线化使得请求能够"并行"传输，但是响应必须按照请求发出的顺序依次返回，性能在一定程度上得到了改善。

4. 增加Host字段

使得一个服务器能够用来创建多个 Web 站点。

5. 状态码

新增了24个错误状态响应码

6. 带宽优化

- `HTTP1.0`中，存在一些浪费带宽的现象，例如客户端只是需要某个对象的一部分，而服务器却将整个对象送过来了，并且不支持断点续传功能
- `HTTP1.1`则在请求头引入了`range`头域，它允许只请求资源的某个部分，即返回码是`206（Partial Content）`

HTTP/2.0和HTTP1.1有什么区别

1、传输格式变化，采用了新的二进制格式

- `HTTP1.X`的解析都是基于文本，文本的表现形式多样，不利于健壮性考虑
- `HTTP2.0`采用二进制，只认0/1组合，实现更加快的方法，健壮性更加完善

2、多路复用

连接共享
一个请求对应一个ID，每个连接都可以有多个请求
接收方可以根据请求的ID将请求再归属到各自不同的服务端请求中
可以极大的提高效率

3、header压缩

在HTTP1.X中，header带有大量信息，而且每次都要重复发送

HTTP2.0通过encoder减少header大小，通讯双方会各自缓存一份header字段表

既可以避免重复header传输，又减小了需要传输的大小

4、服务端推送

把客户端所需要的资源伴随着index.html一起发送到客户端，省去了客户端重复请求的步骤

因为没有发起请求，建立连接等操作，所以静态资源通过服务器推送，可以极大的提升速度

HTTP/3

HTTPS和HTTP的区别

- HTTP 是超文本传输协议，信息是明文传输，存在安全风险的问题。HTTPS 则解决 HTTP 不安全的缺陷，在 TCP 和 HTTP 网络层之间加入了 SSL/TLS 安全协议，使得报文能够加密传输。
- HTTP 连接建立相对简单， TCP 三次握手之后便可进行 HTTP 的报文传输。而 HTTPS 在 TCP 三次握手之后，还需进行 SSL/TLS 的握手过程，才可进入加密报文传输。
- 两者的默认端口不一样，HTTP 默认端口号是 80，HTTPS 默认端口号是 443。
- HTTPS 协议需要向 CA（证书权威机构）申请数字证书，来保证服务器的身份是可信的。

SSL/TLS是什么

SSL：Secure Socket Layer 安全套接字

TSL：Transport Layer Security 安全传输层协议

HTTPS（HyperText Transfer Protocol Secure）是基于TLS/SSL的安全版本的HTTP协议

SSL和TLS协议通过以下方式确保安全通信：

- **加密：** 使用加密算法对传输的数据进行加密，防止第三方截取和读取敏感信息。
- **身份验证：** 使用数字证书验证通信双方的身份，确保数据传输的可信性。
- **数据完整性：** 通过使用消息摘要算法，确保传输的数据在传输过程中没有被篡改或损坏。

什么是对称加密和非对称加密

对称加密

对称加密也称为私钥加密，使用相同的密钥来进行加密和解密。

- 在加密过程中，明文数据通过应用特定的算法和密钥进行加密，生成密文数据。解密过程则是将密文数据应用同样的算法和密钥进行解密，恢复为明文数据。
- 由于加密和解密都使用相同的密钥，因此对称加密算法的速度通常较快，但密钥的安全性很重要。如果密钥泄漏，攻击者可以轻易地解密数据。

非对称加密

- 非对称加密也称为公钥加密，使用一对不同但相关的密钥：公钥和私钥。
- 公钥用于加密数据，私钥用于解密数据。如果使用公钥加密数据，只有拥有相应私钥的人才能解密数据；如果使用私钥加密数据，可以使用相应公钥解密。
- 除了加密和解密，非对称加密还用于【数字签名】，可以验证消息的来源和完整性。

HTTPS是如何建立连接的

HTTPS建立连接的一般过程：

1. **客户端发送连接请求**：
    - 当客户端想要与服务器建立HTTPS连接时，它会发送一个连接请求到服务器的443端口，表明它想要使用HTTPS进行通信。
2. **服务器响应**：
    - 服务器收到连接请求后，会发送一个`CA数字证书`给客户端。这个证书包含了服务器的公钥、证书的颁发者信息以及其他相关信息。
3. **客户端验证证书**：
    - 客户端接收到服务器发送的数字证书后，会**验证证书的合法性**。这个过程包括验证证书的签名、证书是否过期、是否与预期域名匹配等。
4. **生成会话密钥**：
    - 如果证书验证成功，**客户端会生成一个用于该连接的随机会话密钥（对称密钥）**。这个密钥将用于加密通信数据。
5. **用公钥加密会话密钥**：
    - 客户端使用**服务器的公钥，将生成的会话密钥进行加密**，并将加密后的会话密钥发送给服务器。
6. **服务器解密会话密钥**：
    - 服务器使用自己的私钥对客户端发送的加密会话密钥进行解密，获得会话密钥。
7. **建立安全通信**：
    - 从此时开始，客户端和服务器都有了相同的会话密钥，他们使用对称加密算法（如AES）来加密和解密通信数据，保证了通信的隐私性和完整性。

URL和URI是什么

URI（Uniform Resource Identifier）：统一资源标识符， URI是一个通用的术语，用于标识任何互联网上的资源，无论资源是什么类型。

URL（Uniform Resource Locator）：统一资源定位符， URL是URI的一种特殊形式，它不仅标识资源，还提供了资源的位置信息，即如何定位和获取资源。URL由以下几部分组成：

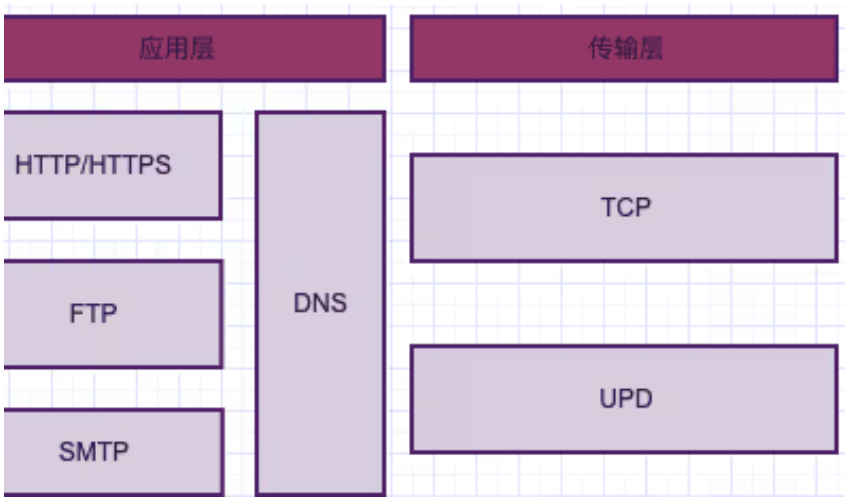
1. **协议（Protocol）**： 指定了访问资源的协议，例如HTTP、HTTPS、FTP等。
2. **域名或IP地址（Domain Name or IP Address）**： 标识了资源所在的主机或服务器的名称或IP地址。
3. **端口号（Port Number）**： 指定了服务器上监听资源请求的端口号，通常根据协议的默认端口。
4. **路径（Path）**： 描述了服务器上资源的具体路径或位置。
5. **查询参数（Query Parameters）**： 用于向服务器传递参数，以影响资源的获取或显示。
6. **片段标识（Fragment Identifier）**： 标识资源中的特定片段或位置。

以下是一个示例URL：https://www.example.com:8080/path/resource?param=value#section

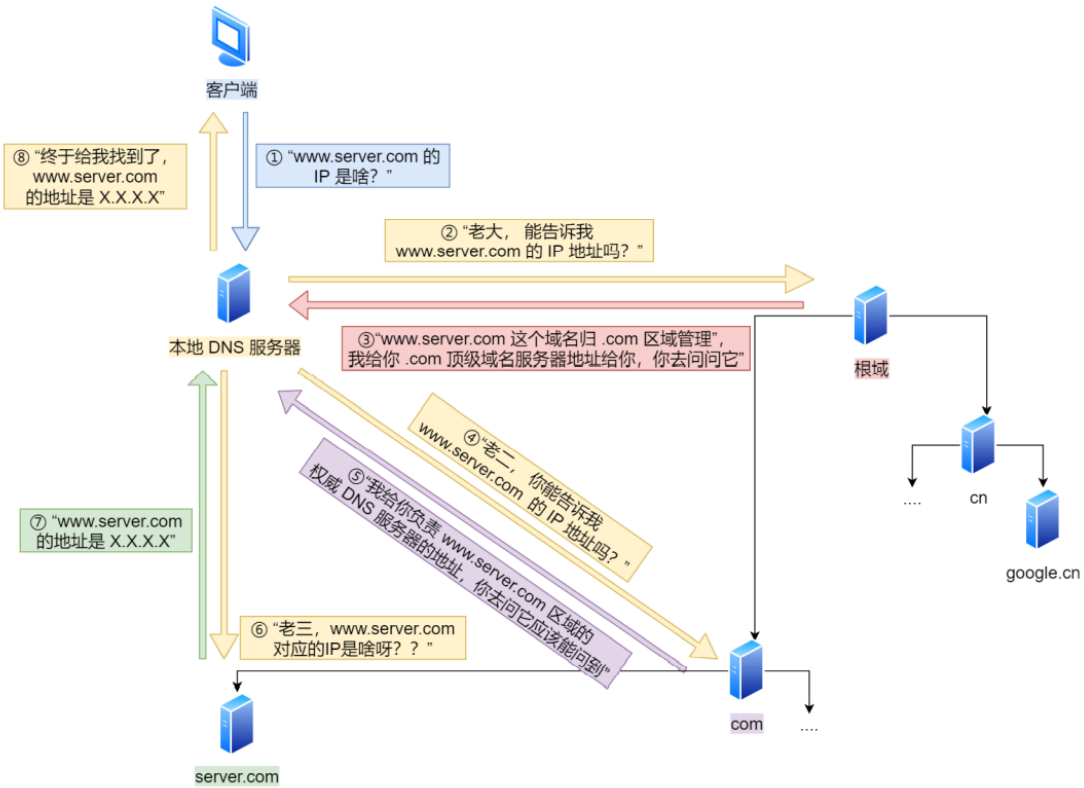
在这个示例URL中，协议是HTTPS，域名是www.example.com，端口号是8080，路径是/path/resource，查询参数是param=value，片段标识是section。

DNS是什么

`DNS(Domain Name System)`是域名“系统”的英文缩写,它作用于TCP/IP网络，它所提供的服务是用来**将主机名和域名（比如`www.baidu.com`转换为IP地址`128.13.45.56`的工作。**



DNS解析过程



1. 先查询浏览器缓存是否有该域名对应的IP地址。
2. 如果浏览器缓存中没有，会去计算机本地的Host文件中查询是否有对应的缓存。
3. 如果Host文件中也没有则会向**本地的DNS服务器**（通常由你的互联网服务提供商（ISP）提供， 比如中国移动）发送一个DNS查询请求。
4. 如果本地DNS解析器有该域名的ip地址，就会直接返回，如果没有缓存该域名的解析记录，它会向**根DNS服**

务器发出查询请求。根DNS服务器并不负责解析域名，但它能告诉本地DNS解析器应该向哪个顶级域（.com/.net/.org）的DNS服务器继续查询。

5. 本地DNS解析器接着向指定的**顶级域名DNS服务器**发出查询请求。顶级域DNS服务器也不负责具体的域名解析，但它能告诉本地DNS解析器应该前往哪个权威DNS服务器查询下一步的信息。
6. 本地DNS解析器最后向**权威DNS服务器**发送查询请求。 权威DNS服务器是负责存储特定域名和IP地址映射的服务器。当权威DNS服务器收到查询请求时，它会查找"example.com"域名对应的IP地址，并将结果返回给本地DNS解析器。
7. 本地DNS解析器将收到的IP地址返回给浏览器，并且还会将域名解析结果缓存在本地，以便下次访问时更快地响应。
8. **浏览器发起连接：** 本地DNS解析器已经将IP地址返回给您的计算机，您的浏览器可以使用该IP地址与目标服务器建立连接，开始获取网页内容。

递归查询和迭代查询

递归查询和迭代查询是在DNS解析过程中用于获取域名解析信息的两种不同方法。

1. 递归查询

在递归查询中，DNS客户端（通常是本地DNS解析器）向上层DNS服务器（如根域名服务器、顶级域名服务器）发起查询请求，并要求这些服务器直接提供完整的解析结果。递归查询的特点是，DNS客户端只需要发送一个查询请求，然后等待完整的解析结果。上层DNS服务器会自行查询下一级的服务器，并将最终结果返回给DNS客户端。

2. 迭代查询

在迭代查询中，DNS客户端向上层DNS服务器发起查询请求，但不要求直接提供完整的解析结果。相反，DNS客户端只是询问上层服务器一个更高级的域名服务器的地址，然后再自行向那个更高级的服务器发起查询请求，以此类推，直到获取完整的解析结果为止。

递归查询适合普通用户和客户端，而迭代查询适用于DNS服务器之间的通信。

TCP和UDP的概念和特点

TCP和UDP是两种常见的传输层协议，用于在计算机网络中进行数据传输。

1. TCP：TCP 是**面向连接的、可靠的、基于字节流**的传输层通信协议。

- **TCP是一种面向连接的协议**，在通信之前，TCP需要在发送和接收方之间建立连接，然后在通信完成后关闭连接。**这种连接的建立和关闭过程称为“三次握手”和“四次挥手”，确保可靠的数据传输。**
- **可靠性：TCP提供可靠的数据传输。**它使用序列号和确认机制来确保数据包的有序性和完整性。如果数据包丢失或损坏，TCP会重新发送丢失的数据包，直到接收方正确接收为止。
- **流量控制和拥塞控制**：TCP使用流量控制和拥塞控制算法，确保数据发送的速率不会超过接收方的处理能力，并防止网络拥塞。
- **有序传输：**TCP确保数据包按照发送的顺序到达接收方，并在接收方重新组装成正确的顺序。
- **适用于可靠数据传输的场景：TCP适用于那些对数据传输可靠性要求较高的应用**，如文件传输、电子邮件、网页浏览等。

2. UDP

- **UDP是一种无连接的协议**：与TCP不同，UDP在通信之前不需要建立连接，直接发送数据包。这使得UDP比TCP更加轻量级。

- **不可靠性：UDP不提供可靠的数据传输。**它发送数据包后不会关心数据包是否成功到达接收方。因此，如果数据包丢失或损坏，UDP不会重新发送，也不会提供确认机制。
- **速度较快**：由于没有连接建立和确认过程，UDP传输速度较快，适用于实时传输，如实时音频和视频流。
- **无序传输**：UDP不保证数据包的有序性，因此接收方接收到的数据包可能是无序的。
- **适用于实时传输的场景**：UDP适用于对数据传输可靠性要求不高的场景，如实时游戏、流媒体等，其中实时性比数据的准确性更为重要。

TCP与UDP的区别

1、连接

- **TCP是面向连接的，**在传输前需要三次握手建立连接。
- **UDP不需要连接，**直接发送数据包，没有连接建立和关闭的过程。

2、服务形式

- **TCP是一对一的通信。**在TCP连接中，一台客户端与一台服务器之间建立一条连接，进行双向通信。
- **UDP可以是一对一、一对多或多对多的通信。**UDP是无连接的，一个UDP包可以被广播到多个目标主机，或者从多个源主机接收UDP包。这使得UDP适用于多播和广播应用。

3、可靠性

- **TCP保证数据可靠交付**，拥有确认应答和重传机制，无重复、不丢失、按序到达;
- **UDP尽可能交付**，发送数据后不会关心数据包是否成功到达接收方，不会进行重传，不保证可靠性。

4、流量控制和拥塞控制

- TCP拥有流量控制、拥塞控制，确保数据发送的速率不会超过接收方的处理能力，并防止网络拥塞。
- UDP不进行流量控制和拥塞控制，数据发送的速率不受限制。

5、首部开销

- **TCP的首部大小通常为20字节**，但在选项字段被使用的情况下，可能会更大。TCP首部包含源端口号、目标端口号、序列号、确认号、窗口大小、校验和等字段。
- **UDP的首部大小固定为8字节**。UDP首部包含源端口号、目标端口号、包长度和校验和字段（各16位）。

6、传输方式

TCP基于字节流，没有边界，但是保证传输顺序和可靠性;
UDP继承了IP层特性，基于数据包，有边界可能出现乱序和丢包。

7、分片方式

TCP数据大于MSS时会在TCP层将数据进行分片传输，到达目的地后同样在传输层进行合并，如果有某个片丢失则只需要重传丢失的分片即可;

UDP数据大于MTU时会在IP层分片，则会在IP层合并，如果某个IP分片丢失，目标主机收到后，在 IP 层组装完数据，接着再传给传输层。

TCP和UDP分别在什么场景使用?

- UDP**适用于实时传输的场景**：UDP适用于对数据传输可靠性要求不高的场景，如实时游戏、流媒体等，其中实时性比数据的准确性更为重要。
- **TCP适用于可靠数据传输的场景**：TCP适用于那些对数据传输可靠性要求较高的应用，如文件传输、电子邮件、网页浏览等。

TCP连接如何确保可靠性

TCP协议保证数据传输可靠性的方式主要有：**校验和、序列号、确认应答、超时重传、连接管理、流量控制、拥塞控制**。

- 序列号和确认应答：TCP通过给每个发送的数据段分配一个序列号，以及使用确认（ACK）机制来跟踪数据的传输和接收。接收方会确认已成功接收的数据，发送方则根据收到的确认来确定哪些数据已经被成功传输，以及哪些需要重新发送。
- **超时和重传：** TCP使用超时机制来检测是否发生了数据包的丢失。如果发送方在一定时间内未收到确认，它会认为数据包丢失，并触发相应的重传。这确保了即使某个数据包在传输过程中丢失，它最终仍能够被成功传递。
- **流量控制：** TCP使用滑动窗口机制来进行流量控制，确保发送方不会以高于接收方处理速度的速率发送数据。这有助于防止接收方缓冲区溢出，并提高整个通信链路的效率。
- **拥塞控制：** TCP还具有拥塞控制机制，通过动态调整发送速率以适应网络状况。当网络出现拥塞时，TCP会减缓发送速率，以防止进一步加剧拥塞。

怎么用UDP实现可靠传输

1. 为什么需要可靠的UDP

   在弱网（2G、3G、信号不好）环境下，使用 TCP 连接的延迟很高，影响体验。使用 UDP 是很好的解决方案，既然把 UDP 作为弱网里面的 TCP 来使用，就必须保证数据传输能像 TCP 一样可靠

2. 如何实现可靠的UDP

   UDP它不属于连接型协议，因而具有资源消耗小，处理速度快的优点，所以通常音频、视频和普通数据在传送时使用UDP较多，因为它们即使偶尔丢失一两个数据包，也不会对接收结果产生太大影响。传输层无法保证数据的可靠传输，只能通过**应用层**来实现了。实现的方式可以参照tcp可靠性传输的方式，只是实现不在传输层，实现转移到了应用层。关键在于两点，从应用层角度考虑：

   （1）提供超时重传，能避免数据报丢失。

   （2）提供确认序列号，可以对数据报进行确认和排序。

   本端：首先在UDP数据报定义一个首部，首部包含确认序列号和时间戳，时间戳是用来计算RTT(数据报传输的往返时间)，计算出合适的RTO(重传的超时时间)。然后以等-停的方式发送数据报，即收到对端的确认之后才发送下一个的数据报。当时间超时，本端重传数据报，同时RTO扩大为原来的两倍，重新开始计时。

   对端：接受到一个数据报之后取下该数据报首部的时间戳和确认序列号，并添加本端的确认数据报首部之后发送给对端。根据此序列号对已收到的数据报进行排序并丢弃重复的数据报。

3. **已经实现的可靠UDP：**

（1）RUDP 可靠数据报传输协议；

（2）RTP 实时传输协议

为数据提供了具有实时特征的端对端传送服务；

Eg：组播或单播网络服务下的交互式视频、音频或模拟数据

（3）UDT

基于UDP的数据传输协议，是一种互联网传输协议；

主要目的是支持高速广域网上的海量数据传输，引入了新的拥塞控制和数据可靠性控制机制（互联网上的标准数据传输协议TCP在高带宽长距离的网络上性能很差）；

UDT是面向连接的双向的应用层协议，同时支持可靠的数据流传输和部分可靠的数据报服务；

应用：高速数据传输，点到点技术(P2P)，防火墙穿透，多媒体数据传输；

三次握手的过程

TCP（传输控制协议）的三次握手是建立网络连接的过程，确保通信双方能够正确地进行数据传输。

1. **第一次握手（SYN）**：

客户端（Client）向服务器（Server）发送一个带有 SYN（同步）标志位的包，表示客户端希望建立连接。该包同时指定客户端的初始序列号（Client Sequence Number）。

2. **第二次握手（SYN + ACK）**：

服务器收到客户端的 SYN 包后，会回复一个带有 SYN 和 ACK（确认）标志位的包，表示服务器接受了客户端的请求，并希望建立连接。服务器也会指定自己的初始序列号，以及对客户端序列号的确认。

3. **第三次握手（ACK）**：

客户端收到服务器的 SYN+ACK 包后，会发送一个带有 ACK 标志位的包作为确认回复。这个包的序列号会加一，表示客户端已经准备好与服务器进行数据传输。

此时，TCP 连接已经建立起来，通信双方可以开始进行数据传输。

四次挥手的过程

四次挥手是指在TCP连接的断开过程中，由客户端先断开，然后由服务器进行最后的断开。

具体的四次挥手步骤如下：

1. 客户端发送一个FIN（终止）报文给服务器，表示客户端不再发送数据。
2. 服务器收到FIN报文后，发送一个ACK（确认）报文给客户端，表示收到了客户端的终止请求。
3. 服务器发送一个FIN报文给客户端，表示服务器也不再发送数据。
4. 客户端收到服务器的FIN报文后，发送一个ACK报文给服务器，确认收到了服务器的终止请求，然后关闭连接。

这样，经过四次挥手，TCP连接才会完全关闭。因为TCP是全双工连接，双方都需要通知对方停止数据传输，所以需要四次握手来完成断开连接的过程。

Keep-Alive是什么

> HTTP1.0中需要配置长连接，在请求头中配置`Connection: Keep-Alive`，而HTTP1.1中默认开启了长连接

`Keep-Alive`是一种 HTTP 协议的机制，也被称为`HTTP长连接`

在启用 Keep-alive 的情况下，客户端和服务器在完成一个 HTTP 请求和响应后，并不立即关闭连接，而是继续保持连接处于打开状态。在连接保持打开的情况下，客户端可以继续发送其他请求，服务器可以继续发送响应，而无需重新建立连接，减少了连接的建立和关闭的开销，从而提高性能和效率。

Keep-Alive的优缺点

优点：

- TCP 连接的建立和关闭需要时间和资源，通过保持连接打开，可以减少这些开销，从而提高性能和效率。
- 客户端可以在同一个连接上同时发送多个请求，服务器可以并行地处理这些请求，提高并发性能。
- Keep-alive 连接中的多个请求共享同一个连接的头部信息（如用户代理、Cookie 等），减少了头部信息的重复传输。

缺点：

- 长时间的持久连接可能会占用服务器资源，特别是在高并发的情况下。为了平衡资源利用和性能，服务器和客户端通常会**设置 Keep-alive 的超时时间**，以便在一段时间内保持连接打开，超过该时间则关闭连接。

TCP KeepAlive是什么

`TCP Keep-Alive`是在操作系统和网络协议栈级别实现的，它通过发送特定的探测数据包来维护连接的活跃性。

1. 在启用 TCP Keep-Alive 的情况下，操作系统会定期发送一些特定的探测数据包到连接的另一端。这些数据包通常是空的，没有实际的数据内容。
2. 如果一端收到了探测数据包，它会回复一个确认（ACK）数据包。如果一段时间内没有收到确认数据包，发送端将认为连接可能已经断开，从而触发连接关闭。
3. TCP Keep-Alive 的主要目的是检测连接是否处于空闲状态，即没有实际数据传输。它不仅可以检测到连接断开，还可以在空闲连接超过一定时间时释放连接，从而释放资源。

TCP的TCP 的 Keepalive 和 HTTP 的 Keep-Alive 是一个东西吗?

1. HTTP 的 Keep-Alive，是由应用层实现的，称为 HTTP 长连接

每次请求都要经历这样的过程：建立 TCP -> 请求资源 -> 响应资源 -> 释放连接，这就是HTTP短连接，但是这样每次建立连接都只能请求一次资源，所以HTTP 的`Keep-Alive`实现了使用同一个 TCP 连接来发送和接收多个 HTTP 请求/应答，避免了连接建立和释放的开销，就就是 **HTTP 长连接**。

2. TCP 的 Keepalive，是由TCP 层（内核态）实现的，称为 TCP 保活机制，是一种用于在 TCP 连接上检测空闲连接状态的机制

通俗地说，就是TCP有一个定时任务做倒计时，超时后会触发任务，内容是发送一个探测报文给对端，用来判断对端是否存活。

CDN是什么

CDN，全称为内容分发网络`（Content Delivery Network）`，过将内容存储在分布式的服务器上，使用户可以从距离较近的服务器获取所需的内容，从而减少数据传输的时间和距离，提高内容的传输速度、减少延迟和提升用户体验。

CDN的工作流程

1. 当用户输入一个域名或点击一个链接时，首先会进行域名解析。如果网站启用了 CDN，DNS 解析会返回距离用户最近的 CDN 节点的 IP 地址，而不是原始源服务器的 IP 地址。
2. 用户的请求会被路由到距离最近的 CDN 节点，并且CDN 节点可以根据服务器的负载和可用性，动态地将请求分发到最适合的服务器节点上。
3. CDN 会首先检查是否已经缓存了该资源。如果有缓存，CDN 节点会直接返回缓存的资源，如果没有缓存所需资源，它会从源服务器（原始服务器）回源获取资源，并将资源缓存到节点中，以便以后的请求。

CDN是如何加速的

1. **就近访问**：CDN 在全球范围内部署了多个服务器节点，当用户请求访问一个网站时，CDN 会选择距离用户最近的节点来提供内容。这减少了数据传输的距离和时间，从而降低了延迟。
2. 内容缓存：CDN 节点会缓存静态资源，如图片、样式表、脚本等。当用户请求访问这些资源时，CDN 可以直接从缓存中返回，避免了从源服务器获取资源的延迟。
3. **前置缓存**：CDN 可以根据网站的配置，提前将热门的内容缓存在节点中，以备用户请求时快速响应。
4. **智能负载均衡**：CDN 会根据服务器的负载和可用性，动态地将请求分发到合适的服务器节点上，确保资源的快速获取。
5. **压缩技术**：CDN 使用压缩技术对内容进行压缩，减少传输数据的大小，从而加快内容的传输速度。
6. **并行下载**：由于 CDN 支持多路复用，用户可以在同一个连接上同时下载多个资源，从而提高并行下载的效率。

Cookie和Session是什么?

`Cookie`和 `Session`都用于管理用户的状态和身份，`Cookie`通过在客户端记录信息确定用户身份， `Session`通过在服务器端记录信息确定用户身份。

1. Cookie

- Cookie 是存储在用户浏览器中的小型文本文件，用于在用户和服务器之间传递数据。通常，服务器会将一个或多个 Cookie 发送到用户浏览器，然后浏览器将这些 Cookie 存储在本地。
- 服务器在接收到来自客户端浏览器的请求之后，就能够通过分析存放于请求头的Cookie得到客户端特有的信息，从而动态生成与该客户端相对应的内容。

2. Session

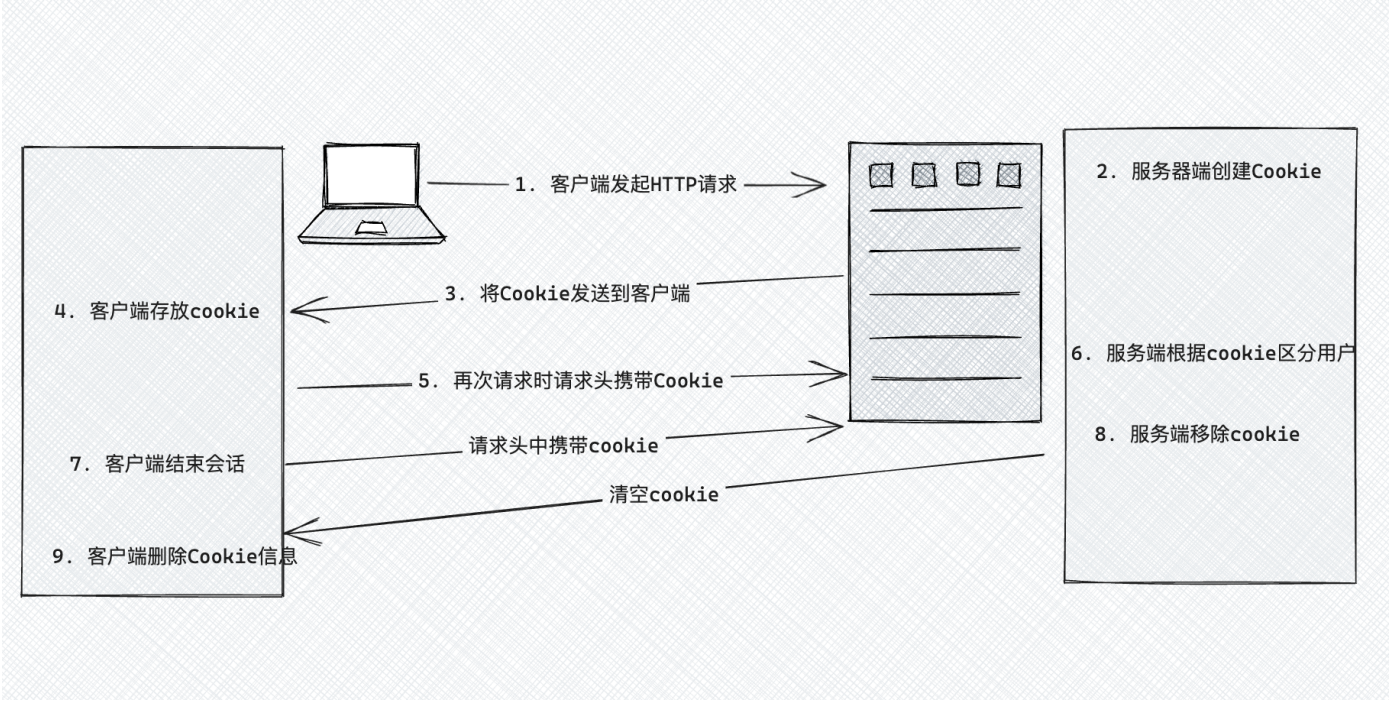
客户端浏览器访问服务器的时候，服务器把客户端信息以某种形式记录在服务器上。这就是`Session`。Session 主要用于维护用户登录状态、存储用户的临时数据和上下文信息等。

Cookie的工作原理

> 通过服务器在 HTTP 响应中设置 "Set-Cookie" 标头，然后浏览器将这些 Cookie 存储并在后续的请求中发送给服务器。这样服务器可以通过 Cookie 实现用户状态管理和数据传递。

1. 客户端（浏览器）第一次发送请求到服务器
2. 服务器可以通过 HTTP 响应的头部信息中的 `"Set-Cookie"`标头来创建一个 Cookie。这个标头指定了 Cookie 的名称、值和其他参数，如过期时间、域名、路径等。
3. 浏览器接收到 "Set-Cookie" 标头后，会将 Cookie 存储在用户的本地计算机上。
4. 当用户再次访问同一个网站时，浏览器会将与该域名相关的 Cookie 信息包括在 HTTP 请求的头部中。
5. 服务器收到包含 Cookie 的 HTTP 请求后，可以读取 Cookie 的值，根据其中的数据来判断用户的状态、偏好

等，并根据需要做出相应的响应，如根据用户登录状态提供个性化内容。
6. 服务器可以通过发送新的 "Set-Cookie" 标头来更新 Cookie 的值或设置新的参数。
7. Cookie 可以设置过期时间，可以是会话级的（浏览器关闭时失效）或持久性的（在一段时间后失效）。当过期时间到达后，浏览器不再发送该 Cookie。



Session的工作原理

Session 是一种在服务器端存储和管理用户状态和数据的机制，通常基于会话标识符（Session ID）进行操作。

1. 用户首次访问网站时，服务器会为该用户创建一个唯一的会话标识符（Session ID）。这个标识符可以是一个加密的字符串，通常以 Cookie 的形式存储在用户的浏览器中。
2. 每个会话标识符对应着服务器上的一个会话存储空间。这个存储空间用于存储该用户在会话期间的状态和数据。
3. 当用户与服务器交互时，服务器可以通过会话标识符来访问对应的会话存储空间。服务器可以将数据存储在会话中，如用户的登录状态、购物车内容、用户偏好等。
4. 服务器为每个会话设置一个超时时间，如果用户在一段时间内没有活动，会话会自动过期。一旦会话过期，会话数据将被清除。
5. 用户可以手动终止会话，例如通过退出登录操作。这会导致服务器删除与该用户相关的会话数据。

Cookie和Session有什么区别?

- 存储位置：Cookie 数据存储在用户的浏览器中，而 Session 数据存储在服务器上。
- 数据容量：Cookie 存储容量较小，一般为几 KB。Session 存储容量较大，通常没有固定限制，取决于服务器的配置和资源。
- 安全性：由于 Cookie 存储在用户浏览器中，因此可以被用户读取和篡改。相比之下，Session 数据存储在服务器上，更难被用户访问和修改。
- 传输方式：Cookie 在每次 HTTP 请求中都会被自动发送到服务器，而 Session ID 通常通过 Cookie 或 URL 参数传递。

TCP重传机制是怎么实现的?

当发送方的数据在传输过程中丢失、损坏或延迟，接收方可以请求发送方重新传输这些数据。

1. **序号与确认号**：在 TCP 通信中，每个发送的字节都有一个唯一的序号，而每个接收的字节都有一个确认号。发送方维护了一个发送窗口，接收方维护了一个接收窗口。发送方会持续发送数据，并等待接收方的确认。
2. **超时检测**：发送方为每个发送的数据段设置一个定时器，这个定时器的时长称为超时时间。发送方假设在这个超时时间内，数据能够到达接收方并得到确认。如果在超时时间内没有收到确认，发送方会认为数据丢失或损坏，触发重传。
3. **重传策略**：当发送方在超时时间内没有收到确认，它会认为数据丢失，然后重新发送相应的数据段。如果只有一个数据段丢失，发送方只会重传丢失的数据段。如果有多个数据段丢失，发送方可能会使用更复杂的算法来决定哪些数据需要重传。
4. **快速重传和快速恢复**：为了更快地发现丢失的数据，接收方可以使用快速重传策略。当接收方连续接收到相同的确认号时，它会立即向发送方发送冗余的确认，以触发发送方进行重传。此外，发送方在接收到快速重传的确认后，不需要等到超时再次发送，而是可以使用快速恢复算法继续发送未丢失的数据。

TCP流量控制是怎么实现的?

流量控制就是让发送发送速率不要过快，让接收方来得及接收。利用**滑动窗口机制**就可以实施流量控制，主要方法就是动态调整发送方和接收方之间数据传输速率。

- **滑动窗口大小：** 在TCP通信中，每个TCP报文段都包含一个窗口字段，该字段指示发送方可以发送多少字节的数据而不等待确认。这个窗口大小是动态调整的。
- **接收方窗口大小：** 接收方通过TCP报文中的窗口字段告诉发送方自己当前的可接收窗口大小。这是接收方缓冲区中还有多少可用空间。
- **流量控制的目标：** 流量控制的目标是确保发送方不要发送超过接收方缓冲区容量的数据。如果接收方的缓冲区快满了，它会减小窗口大小，通知发送方暂停发送，以防止溢出。
- **动态调整：** 发送方会根据接收方的窗口大小动态调整发送数据的速率。如果接收方的窗口大小增加，发送方可以加速发送数据。如果窗口大小减小，发送方将减缓发送数据的速率。
- **确认机制：** 接收方会定期发送确认（ACK）报文，告知发送方已成功接收数据。这也与流量控制密切相关，因为接收方可以通过ACK报文中的窗口字段来通知发送方它的当前窗口大小。

TCP拥塞控制是怎么实现的?

TCP拥塞控制可以在网络出现拥塞时动态地调整数据传输的速率，以防止网络过载。TCP拥塞控制的主要机制包括以下几个方面：

1. **慢启动（Slow Start）：** 初始阶段，TCP发送方会以较小的发送窗口开始传输数据。随着每次成功收到确认的数据，发送方逐渐增加发送窗口的大小，实现指数级的增长，这称为慢启动。这有助于在网络刚开始传输时谨慎地逐步增加速率，以避免引发拥塞。
2. **拥塞避免（Congestion Avoidance）：** 一旦达到一定的阈值（通常是慢启动阈值），TCP发送方就会进入拥塞避免阶段。在拥塞避免阶段，发送方以线性增加的方式增加发送窗口的大小，而不再是指数级的增长。这有助于控制发送速率，以避免引起网络拥塞。
3. **快速重传（Fast Retransmit）：** 如果发送方连续收到相同的确认，它会认为发生了数据包的丢失，并会快速重传未确认的数据包，而不必等待超时。这有助于更快地恢复由于拥塞引起的数据包丢失。
4. **快速恢复（Fast Recovery）：** 在发生快速重传后，TCP进入快速恢复阶段。在这个阶段，发送方不会回到慢启动阶段，而是将慢启动阈值设置为当前窗口的一半，并将拥塞窗口大小设置为慢启动阈值加上已确认但未被快速重传的数据块的数量。这有助于更快地从拥塞中恢复。

什么是SS0

普通的登录认证机制

普通登录认证机制在登录认证成功后，服务器把用户的登录信息写入 session，并为该用户生成一个 cookie，返回并写入浏览器；当用户再次访问这个系统的时候，请求中会带上这个 cookie，服务端会根据这个 cookie 找到对应的 session，通过session来判断这个用户是否已经登录。

普通的登录认证机制在多系统的环境下，在操作不同的系统时，需要多次登录，会变得很不方便。

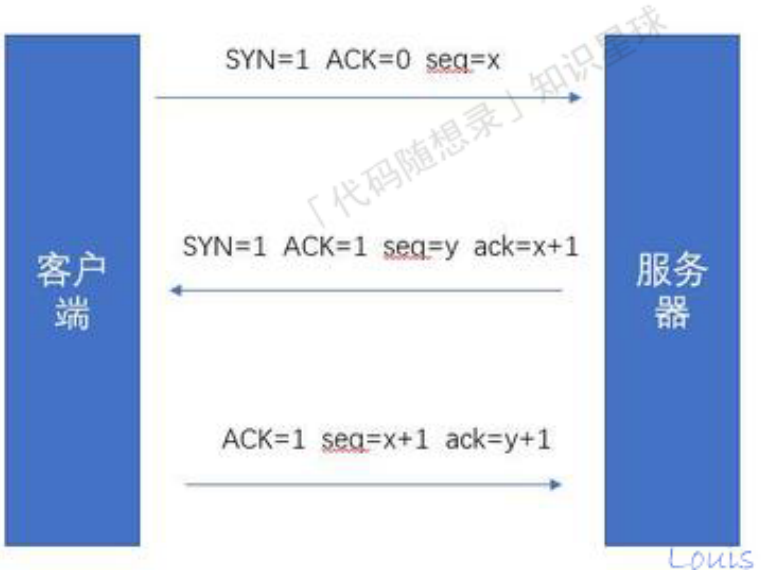
单点登录（`Single sign-on`，缩写为 SSO），允许用户在一次登录后访问多个关联的应用程序或服务，而无需再次输入其凭据。简而言之，用户只需一次登录，就能够无缝地访问多个应用。

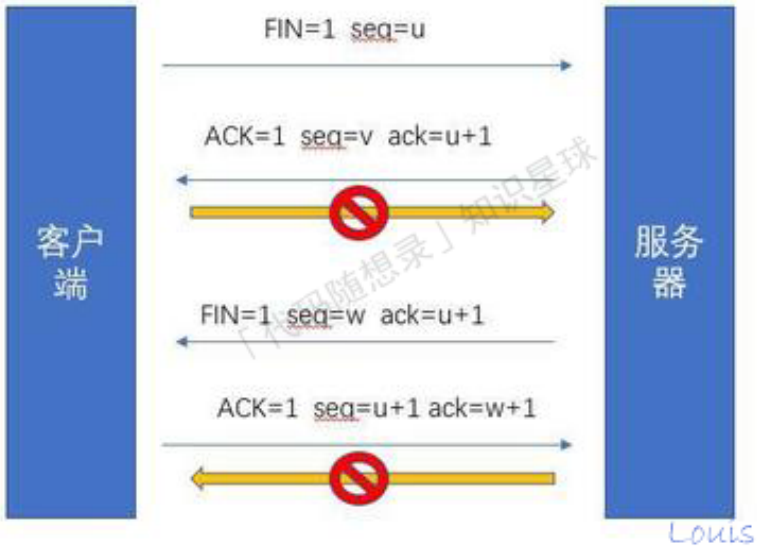
SSO的优点

- 降低密码重用风险： 因为用户只需一个凭据，不再需要在多个应用程序间重复使用相同的密码，从而减少了因密码泄露而引发的安全风险。
- 方便的用户体验： 用户只需要在一次登录过程中提供凭据，然后就可以无缝地访问多个应用程序，无需为每个应用程序输入用户名和密码。这大大简化了用户的登录过程，提供了更流畅的体验。
- 简化应用开发： 对于应用程序开发人员来说，他们可以将认证和授权的责任交给SSO系统，从而减少了在每个应用中实现这些功能的工作量。

HTTP请求过程

1. 首先，我们在浏览器地址栏中，输入要查找页面的URL，按下Enter
2. 浏览器依次在 浏览器缓存 -->>系统缓存 -->>路由器缓存中去寻找匹配的URL，若有，就会直接在屏幕中显示出页面内容。若没有，则跳到第三步操作
3. 发送HTTP请求前，浏览器需要先进行域名解析(即DNS解析)，以获取相应的IP地址;（浏览器DNS缓存、路由器缓存、DNS缓存）
4. 获取到IP地址之后，浏览器向服务器发起TCP连接，与浏览器建立TCP三次握手
5. 握手成功之后，浏览器就会向服务器发送HTTP请求，来请求服务器端的数据包
6. 服务器处理从浏览器端收到的请求，接着将数据返回给浏览器
7. 浏览器收到HTTP响应
8. 查询状态，状态成功则进行下一步，不成功则弹出相应指示
9. 再读取页面内容、进行浏览器渲染、解析HTML源码;（生成DOM树、解析CCS样式、处理JS交互，客户端和服务器交互）进行展示
10. 关闭TCP连接（四次挥手）





HTTP优化方案

1、TCP复用

将多个客户端的HTTP请求复用到一个服务端的TCP连接上

2、HTTP复用

将一个客户端的多个HTTP请求复用到一个TCP连接进行处理

3、内容缓存

将经常用到的内容进行缓存，在下一次获取的时候，就可以直接在内存中获取相应的数据了

4、压缩

文本数据压缩，减少带宽

5、SSL加速

6、TCP缓冲

通过采用TCP缓冲技术，提高服务端响应时间和处理效率

浏览器地址栏输入URL回车后涉及到的流程

1.1 查找DNS缓存

1. 先查找浏览器DNS缓存，看是否存放目标网络的IP地址;
2. 如果不在浏览器缓存，则浏览器将对操纵系统发起系统调用，查询操作系统本地缓存;
3. 如果不在操作系统本地缓存，则浏览器会查询与之相连的路由器缓存;
4. 如果不在路由器缓存，则浏览器会检查ISP【本地通信服务商】缓存;

若以上四步均没有查询到目标网络的IP地址，则发起DNS查询。

1.2 发起DNS查询

判断DNS服务器和我们的主机是否在同一子网内

1. 在同一子网，则采用 ARP 地址解析协议对 DNS 服务器进行 ARP 查询
2. 不在同一子网，则采用 ARP 地址解析协议对默认网关进行查询

若此时还是查询不到 IP 地址，则根据拿到 DNS 服务器或者默认网关的 IP 地址，继续进行 DNS 请求
使用53端口先向本地 DNS 服务器发送 UDP 请求包，此处一般使用 UDP 协议（如果响应包太大，则使用 TCP 协议）

没有查询到 IP 地址：

则它会发送一个递归查询请求，一层一层向高层DNS服务器查询，直到查询到 IP 地址，则将结果返回

【解释：DNS 是分布式域名服务器，每台服务器只维护一部分 IP 地址到网络地址的映射，没有任何一台服务器能够维持全部的映射关系】。

1.3 封装TCP数据包

拿到 IP 地址后，根据 URL 中的端口可知端口号【HTTP：80；HTTPS：443】，一般先会先尝试建立 HTTP 连接;

准备 TCP 数据包：

步骤：

1. 将应用层传递下来的实际数据，在传输层添加TCP首部;
2. 将传输层传下来的数据在网络层添加IP首部;
3. 将网络层传输下来的数据，在数据链路层添加以太网首部，并在传输介质中进行传输。

1.4 浏览器与目标服务器建立TCP连接

经过上述DNS和ARP查询流程后，浏览器会收到目标服务器的IP和MAC地址，然后经过三次握手后建立TCP连接;

1、使用HTTP协议

浏览器发送请求到服务器，如果使用的是HTTP协议，则服务器直接返回结果;

2、使用HTTPS协议

如果不是 HTTP 协议，则服务器会返回一个以 3 开头的重定向消息，告诉浏览器使用的 HTTPS，IP 没变，只是端口号变成 443；完成四次挥手；
重新建立 TCP 连接，将端口号修改为 443，同时沟通好双方的使用的认证算法、加密和解密算法，在次过程中也会检查对方的 CA 安全证书，采用 SSL 加密技术进行传输数据。

1.5 浏览器发送HTTP/HTTPS请求到web服务器

主要使用两种请求方式：

1. 浏览器发送get请求，要求目标服务器提供输入的网页;
2. 浏览器发送post请求，表示填写的是表单。

1.6 服务器处理请求并发挥一个响应

服务器会从浏览器接受请求并将其传递给请求处理程序并响应;

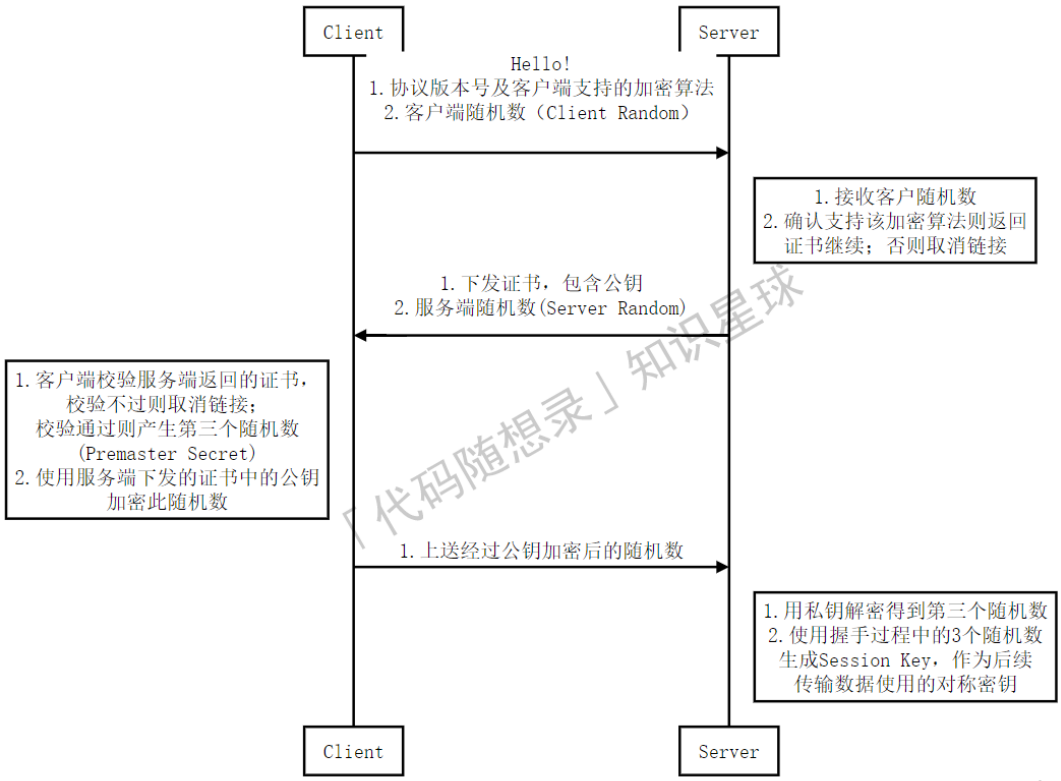
1.7 服务器发回一个HTTP响应

一般响应包包含：请求的网页以及状态码，压缩类型，如何缓存的页面，设置的cookie;

1.8 浏览器显示HTML页面

1. 渲染HTML骨架；涉及到Ajax技术;
2. 检查HTML标记并发送GET请求以获取网页上的其他元素【图像、CSS样式、JS文件等】，该静态文件一般由浏览器缓存，再次访问，不用重新请求;
3. 最后会看到请求色彩斑斓的网页。

2.3 HTTPS三次握手流程



壮

SYN攻击

1、原理

攻击者伪造不同IP地址的SYN报文请求连接，服务端收到连接请求后分配资源，回复ACK+SYN包，但是由于IP地址是伪造的，无法收到回应，久而久之造成服务端半连接队列被占满，无法正常工作。

2、避免方式

（1）修改半连接队列大小

使服务端能够容纳更多半连接。此外还可以修改服务端超时重传次数，使服务端尽早丢弃无用连接

（2）正常服务端行为是收到客户端SYN报文后

将其加入到内核半连接队列，接着发送ACK+SYN报文给客户端，当收到客户端ACK报文后把连接从半连接队列移动到accept队列。

当半连接队列满时，启动syn cookie,后续连接不进入半连接队列，而是计算一个cookie值，作为请求报文序列号发送给客户端，如果服务端收到客户端确认报文，会检查ack包合法性，如果合法直接加入到accept队列

TCP保活机制

1、概念

在一个定义的时间段内TCP连接无任何活动时，会启动TCP保活机制，每隔一定时间间隔发送一个探测报文，等待响应。

2、机制

1. 对端正常响应，重置保活时间;
2. 对端程序崩溃，响应一个RTS报文，将TCP连接重置;
3. 保活报文不可达，等待达到保活探测次数后关闭连接。

3、TCP为啥需要流量控制

1. 由于通讯双方网速不同，通讯方任意一方发送过快都会导致对方详细处理不过来，所以就需要把数据放到缓冲区中
2. 如果缓冲区满了，发送方还在疯狂发送，那接收方只能把数据包丢弃。因此我们需要控制发送速率
3. 我们缓冲区剩余大小称之为接收窗口，用变量win表示。如果win=0，则发送方停止发送

操作系统

基础知识讲解

操作系统基础

> 💦 本部分是一些基础的概念，仅做了解使用。

什么是操作系统

操作系统(Operating System)，是介于硬件资源和应用程序之间的一个系统软件 ，能控制和管理整个计算机系统的硬件和软件资源，调度计算机的工作与资源的分配，进而为用户和其他软件提供服务，**操作系统是计算机系统中最基本的系统软件。**

如果将它理解为“掌控计算机的系统”是否更能精确的描述OS所做的事情呢?

如果要更深入的掌握这个问题，可以问一问：如果没有了操作系统，你使用的PC机还能干什么？或者说，你能够使用你的PC机做什么呢?

操作系统的特征：

- 并发：并发指的是两个或多个事件在同一时间间隔内发生，计算机系统中同时存在多个运行的程序，因此具有处理和调度多个程序同时执行的能力。

> ⚠️注意：并行和并发的区别：并发指的是同一时间间隔，并行指的是同一时刻。

- 共享：系统中的资源可以供内存中多个并发执行的进程共同使用。
  - 互斥共享：一段时间内只允许一个进程访问该资源。**一段时间内只允许一个进程访问的资源称为临界资源。**
  - 同时访问：一段时间内允许多个进程“同时”访问，“同时”通常是宏观的，实际上是交替的对该资源进行访问。
- 虚拟：把一个物理上的实体变为若干逻辑上的对应物。
- 异步：进程的执行并不是一贯到底的，而是以不可预知的速度向前推进。

操作系统的功能

> 操作系统位于硬件资源之上，管理硬件资源; 应用程序之下，为应用程序提供服务，同时管理应用程序

1、资源分配，资源回收

计算机必要重要的硬件资源无非就是 CPU、内存、硬盘、I/O设备。
而这些资源总是有限的，因此需要有效管理，资源管理最终只有两个问题：资源分配、资源回收。

资源分配： 体现在CPU上，比如进程调度，多个进程同时请求CPU下，应该给哪一个进程呢？再比如内存分配，内存不够了怎么办？A进程非法访问了B进程的内存地址怎么办？内存内、外碎片问题等。

资源回收： 考虑内存回收后的合并等等。

2、为应用程序提供服务

操作系统将硬件资源的操作封装起来，提供相对统一的接口（系统调用）供开发者调用。

如果没有操作系统，应用程序将直接面对硬件，除去给开发者带来的编程困难不说，直接访问硬件，使用不当极有可能直接损坏硬件资源。

3、管理应用程序

即控制进程的生命周期：进程开始时的环境配置和资源分配、进程结束后的资源回收、进程调度等。

4、操作系统内核的功能

（1）**进程调度能力：** 管理进程、线程，决定哪个进程、线程使用CPU。

（2）**内存管理能力：** 决定内存的分配和回收。

（3）**硬件通信能力：** 管理硬件，为进程和硬件之间提供通信。

（4）**系统调用能力：** 应用程序进行更高限权运行的服务，需要系统调用，用户程序和操作系统之间的接口。

操作系统的角色

1、管理者

主要分为：CPU管理、内存管理、外存管理、IO管理；以及自己的健壮性和安全性管理。

> 健壮性，又称鲁棒性，即使很粗鲁的对待程序，它还是可以很好的运行。

2、魔术师:

比如操作系统会让每个进程都觉得自己独占CPU、独占整片物理内存，而实际上每个进程都只是在某一时间段内占用CPU，仅仅只是占用实际一点点物理内存。

用户程序与操作系统的关系

> 用户程序和操作系统之间是**相互调用**的关系

1、操作系统的角度

计算机启动后启动的第一个软件就是操作系统，随后启动的所有进程都运行在操作系统之上，使用操作系统提供的服务，同时被操作系统监控，进程结束后也由操作系统回收。

2、进程角度

调用操作系统提供的服务，实现自己的功能。

进程和线程

进程基础

1、进程的概念

我们编写的代码只是一个存储在硬盘的静态文件，通过编译后就会生成二进制可执行文件，当我们运行这个可执行文件后，它会被装载到内存中，接着 CPU 会执行程序中的每一条指令，那么这个**运行中的程序，就被称为「进程」（Process）**。

根据上面的过程，我们可以得到进程的其中一个定义：**进程是具有独立功能的程序在一个数据集合上运行的过程，是系统进行资源分配和调度的一个独立单位。**

2、进程控制块（PCB）

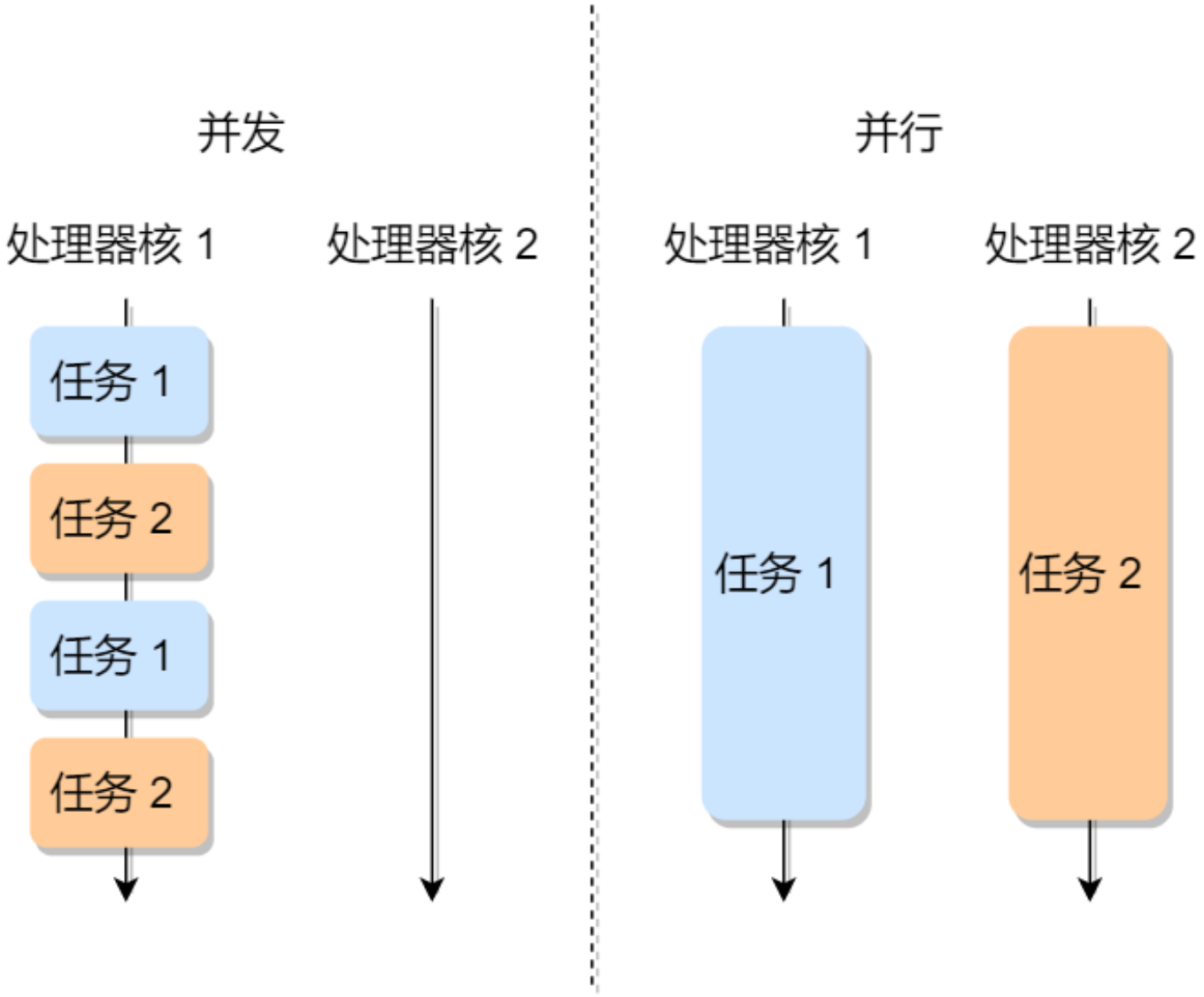
系统通过`进程控制块PCB`来描述进程的基本情况和运行状态，就进而控制和管理进程，**它是进程存在的唯一标识**，其包括以下信息：

1. 进程描述信息： 进程标识符、用户标识符
2. 进程控制和管理信息： 进程当前状态，进程优先级
3. 进程资源分配清单： 有关内存地址空间或虚拟地址空间的信息，所打开文件的列表和所使用的 I/O 设备信息。
4. CPU相关信息： 当进程切换时，CPU寄存器的值都被保存在相应PCB中，以便CPU重新执行该进程时能从断点处继续执行;

`PCB`通常是通过**链表**的方式进行组织，把具有**相同状态的进程链在一起，组成各种队列**。

3、并发与并行

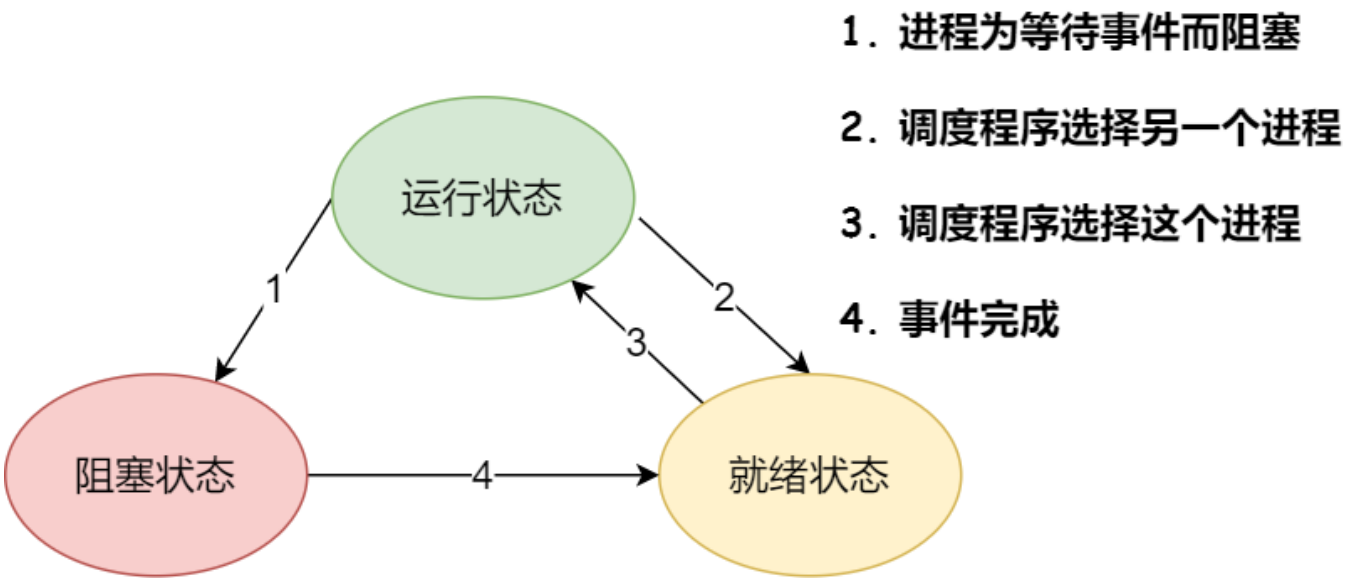
1. 单个处理核在很短时间内分别执行多个进程，称为并发
2. 多个处理核同时执行多个进程称为并行
3. 对于并发来说，CPU需要从一个进程切换到另一个进程，在切换前必须要记录**当前进程中运行的状态信息，以备下次切换回来的时候可以恢复执行。**



4、进程的状态切换

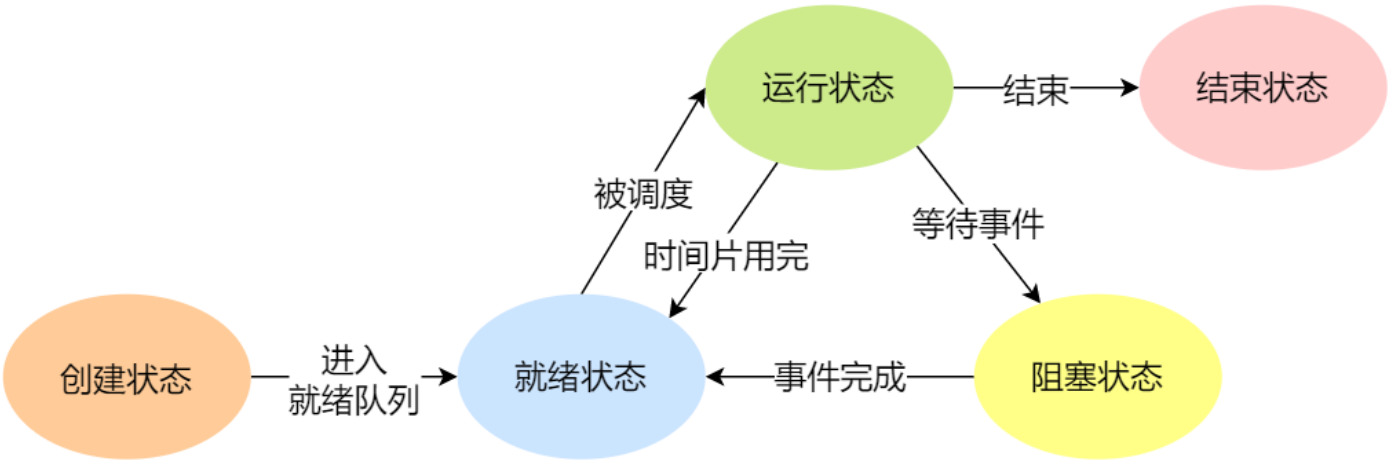
我们知道了并发会执行进程的切换，这就需要进程有运行状态和停止状态，实际上某个进程在某个时刻所处的状态分为以下几种状态：

- **运行态**： 该时刻进程占用CPU
- **就绪态**：可运行，由于其他进程处于运行状态而暂时停止运行
- **阻塞态**：该进程正在等待某一事件发生（如等待输入/输出操作的完成）而暂时停止运行



当然，进程还有另外两个基本状态：

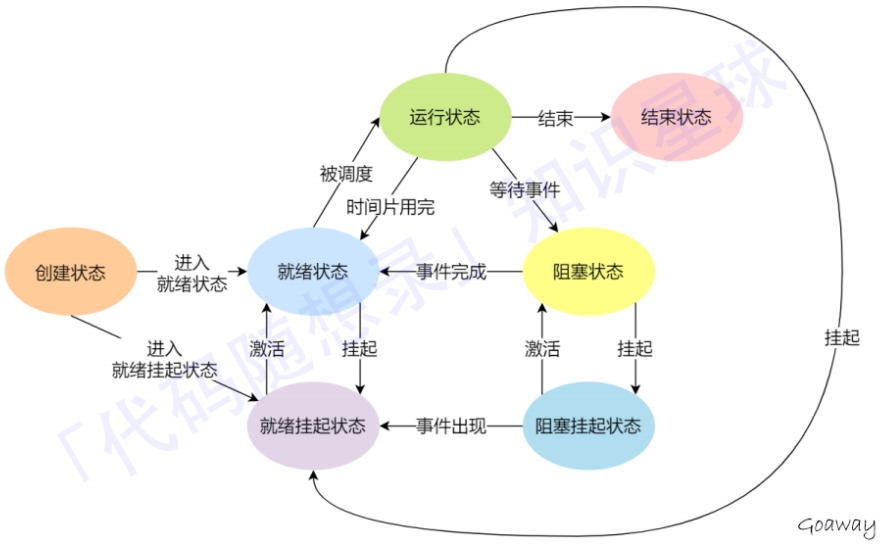
- 创建状态（*new*）：进程正在被创建时的状态；
- 结束状态（*Exit*）：进程正在从系统中消失时的状态；



如果有大量处于阻塞状态的进程，进程可能会占用着物理内存空间，所以系统通常会把阻塞状态的进程的物理内存空间换出到硬盘，等需要再次运行的时候，再从硬盘换入到物理内存，那么，就需要一个新的状态，来**描述进程没有占用实际的物理内存空间的情况，这个状态就是挂起状态**。这跟阻塞状态是不一样，阻塞状态是等待某个事件的返回。

挂起状态可以分为两种：

- 阻塞挂起状态：进程在外存（硬盘）并等待某个事件的出现；
- 就绪挂起状态：进程在外存（硬盘），但只要进入内存，即刻立刻运行；



特点：

1. **只有就绪态和运行态可以相互转换**，其它的都是单向转换。就绪态的进程通过调度算法从而获得CPU 时间，转为运行状态；
2. 运行态的进程，在分配给它的 CPU 时间片用完之后就会转为就绪状态，等待下一次调度。
3. 阻塞态是因缺少需要的资源从而由运行态转换而来，但是该资源不包括 CPU 时间，缺少 CPU 时间会从运行态转换为就绪态。
4. 当进程等待的外部事件发生时（如一些输入到达），则由阻塞态转换为就绪态，如果此时没有其他进程运行，则转换为运行态，否则该进程将处于就绪态，等待CPU空闲轮到它运行。

5、进程的上下文切换

一个进程切换到另一个进程运行，称为进程的上下文切换, **进程的上下文切换不仅包含了虚拟内存、栈、全局变量等用户空间的资源，还包括了内核堆栈、寄存器等内核空间的资源。**

> 发生进程上下文切换有哪些场景?

- 进程的时间片耗尽
- 阻塞等待
- 高优先级进程运行
- 中断处理
- 进程通过睡眠函数 sleep 这样的方法将自己主动挂起

6、进程的创建

一个进程可以创建另一个进程，此时创建者为父进程，被创建的进程为子进程，操作系统创建一个新进程的过程如下：

- 为新进程分配一个独特的进程控制块（PCB）
- 为新进程分配所需要的资源，比如内存、CPU时间
- 初始化进程控制块（PCB）的各种字段，包括状态、优先级、寄存器初始值等。
- 将其状态设置为就绪状态，使其能够被调度执行。进程进入就绪队列，等待分配处理器时间。

7、进程的终止

- 根据标识符，查找需要终止的进程的 PCB；
- 如果进程处于执行状态，则立即终止该进程的执行，然后将处理器资源分配给其他进程；
- 如果其还有子进程，则应将该进程的子进程交给 1 号进程接管；

- 将该进程所拥有的全部资源都归还给操作系统；
- 将其从 PCB 所在队列中删除；

8、进程的阻塞

- 找到被阻塞进程的标识符对应的PCB
- 如果该进程为运行状态，则保护其现场，将其状态转为阻塞状态，停止运行；
- 将该 PCB 插入到等待队列中，将处理机资源调度给其他就绪进程

9、进程的唤醒

- 在该事件的阻塞队列中找到相应进程的 PCB；
- 将其从阻塞队列中移出，并置其状态为就绪状态；
- 把该 PCB 插入到就绪队列中，等待调度程序调度；

线程基础

1、什么是线程?

线程是“轻量级线程”，是进程中的一个实体，是程序执行的最小单元，也是被**系统独立调度和分配的基本单位。**

线程是进程当中的一条执行流程，同一个进程内多个线程之间可以共享代码段、数据段、打开的文件等资源，但每个线程各自都有一套独立的寄存器和栈，这样可以确保线程的控制流是相对独立的。

2、线程的特点

- 线程是一个“轻量级线程”，一个进程中可以有多个线程，线程不拥有系统资源，但是也有PCB，创建线程使用的底层函数和进程一样，都是clone
- 各个线程之间可以并发执行
- 同一个进程中的各个线程共享该进程所拥有的资源。
- 进程可以蜕变成线程;

实际上，无论是创建进程的fork，还是创建线程的`pthread_create`，底层实现都是调用同一个内核函数 clone 。

1. 如果复制对方的地址空间，那么就产出一个“进程”;
2. 如果共享对方的地址空间，就产生一个“线程”。

Linux内核是不区分进程和线程的, 只在用户层面上进行区分。所以，线程所有操作函数`pthread_*`是库函数，而非系统调用。

3、进程和线程的比较

> 进程是资源（包括内存、打开的文件等）分配的单位，线程是 CPU 调度的单位；

- 资源：进程是系统中拥有资源的基本单位，而线程不拥有系统资源（仅有一点必不可少的能保证运行的资源，比如寄存器和栈），但线程可以访问隶属进程的系统资源。
- 调度：线程切换的代价远低于进程，在同一个进程中，线程的切换不会引起进程切换，而从一个进程中的线程切换到另一个进程中的线程中，会引起进程切换。
- 并发：进程可以并发执行，而一个进程中的多个线程之间也能并发执行，甚至不同进程中的线程也能并发执行，从而使得操作系统拥有更好的并发性，提高了系统资源的利用率和系统的吞吐量。
- 独立性：每个进程都拥有独立的地址空间和资源、除了共享全局变量，不允许其他进程访问。某进程中的线程对其他进程都不可见，同一进程中的不同线程是为了提高并发性以及进行相互之间的合作而创建的，它们共享进程的地址空间和资源。

- 系统开销：线程所需要的开销比进程小
    - 线程的创建时间比进程快，因为进程在创建的过程中，还需要资源管理信息，比如内存管理信息、文件管理信息，而线程在创建的过程中，不会涉及这些资源管理信息，而是共享它们；
    - 线程的终止时间比进程快，因为线程释放的资源相比进程少很多；
    - 同一个进程内的线程切换比进程切换快，因为线程具有相同的地址空间（虚拟内存共享），这意味着同一个进程的线程都具有同一个页表，那么在切换的时候不需要切换页表。而对于进程之间的切换，切换的时候要把页表给切换掉，而页表的切换过程开销是比较大的；
    - 由于同一进程的各线程间共享内存和文件资源，那么在线程之间数据传递的时候，就不需要经过内核了，这就使得线程之间的数据交互效率更高了；

所以，不管是时间效率，还是空间效率线程比进程都要高。

4、线程的状态：

- 执行状态：线程获得处理机正在执行
- 就绪状态：线程已经具备执行条件，只需要获得CPU就可以执行
- 阻塞状态：线程在执行中因事件受阻而处于暂停状态

5、线程的实现

（1）用户线程 （ULT）

用户线程是在用户空间实现的线程，不是由内核管理的线程，整个线程管理和调度，操作系统是不直接参与的，而是由用户级线程库函数来完成线程的管理，包括线程的创建、终止、同步和调度等。

优点：

1. 线程切换不需要切换到内核空间中，节省了模式切换的开销。
2. 调度算法可以是进程专用的，不同的进程可根据自身的需要，对自己的线程选择不同的调度算法。
3. 用户级线程的实现与操作系统平台无关，对线程管理的代码是属于用户程序的一部分。

缺点：

1. 由于不由操作系统调度，一旦用户线程发起系统调用而阻塞，那么此进程下用户线程都无法运行;
2. 一旦某个用户线程正在运行，只有当其交出CPU执行权，其他用户线程才可以运行，无法被打断，因为只有操作系统才有权限打断运行，但是操作系统不直接参与调度;
3. 由于时间片分配给进程，故与其他进程比，在多线程执行时，每个线程得到的时间片较少，执行会比较慢；

（2）内核线程（KLT）

由操作系统管理、调度，其对应的TCB是存放在内核中，这样线程的创建、终止和管理都是由操作系统负责。

优点：

1. 当一个内核线程发起系统调用阻塞时不会影响其它内核线程的执行;
2. 分配给线程，多线程的进程获得更多的 CPU 运行时间；

缺点：

1. 在支持内核线程的操作系统中，由内核来维护进程和线程的上下文信息，如 PCB 和 TCB；
2. 线程的创建、终止和切换都是通过系统调用的方式来进行，因此对于系统来说，系统开销比较大；

（3）轻量级线程 (LWP)：

轻量级进程是内核支持的用户线程，一个进程可有一个或多个 LWP，每个 LWP 是跟内核线程一对一映射的，也就是 LWP 都是由一个内核线程支持，而且 LWP 是由内核管理并像普通进程一样被调度。

6、线程共享资源

- 文件描述符表
- 每种信号的处理方式
- 当前工作目录
- 用户ID和组ID

7、 线程非共享资源

- 线程id
- 处理器现场和栈指针(内核栈)
- 独立的栈空间(用户空间栈)
- errno变量
- 信号屏蔽字
- 调度优先级

8、线程的优缺点

优点：

- 提高程序并发性
- 开销小
- 数据通信、共享数据方便

缺点：

- 库函数，不稳定
- 调试、编写困难、gdb不支持
- 对信号支持不好

9、线程如何减少开销

1. 线程创建快，进程创建需要资源管理信息，比如内存管理信息和文件管理信息，而线程创建后是共享其所属进程的资源管理信息;
2. 线程终止时间快，需要回收的仅有少量寄存器和私有的栈区;
3. 线程切换快，因为线程切换仅涉及到少量寄存器和栈区，而进程上下文切换有CPU寄存器和程序计数器(CPU上下文)、虚拟内存空间、页表切换等;
4. 线程因为创建时共享了其所属进程绝大多数资源，因此天生具有很好的线程间通信交互效率。

进程调度

当一个进程的状态发生改变时，操作系统需要考虑是否要换一个进程执行，这就需要用到“进程调度算法”

调度目标

不同的调度算法具有不同的特性，因为使用以下标准比较处理机调度算法的性能：

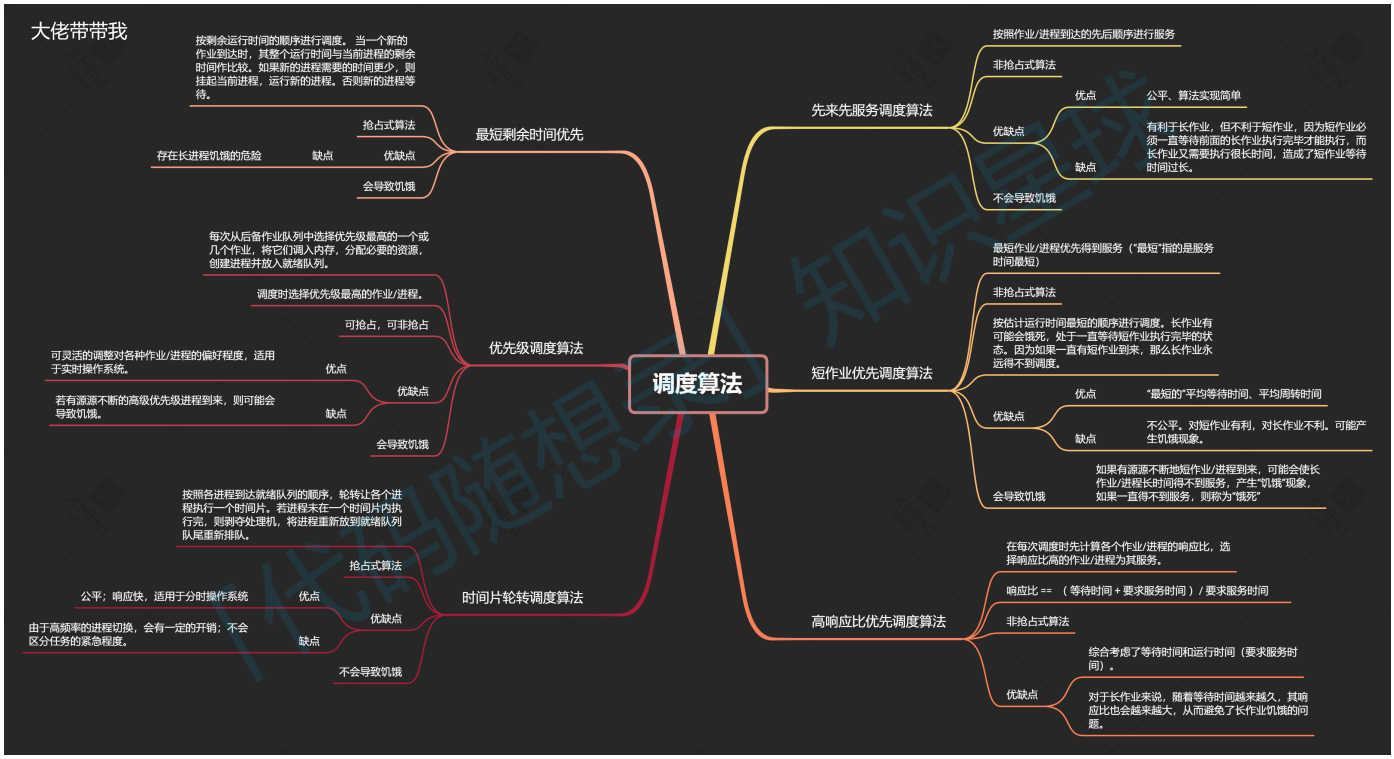
- CPU利用率：CPU是计算机系统中最重要和昂贵的资源之一，应该使CPU保持“忙碌”状态
- 系统吞吐量：单位时间内CPU完成作业的数量。
- 周转时间：作业从提交到作业完成所需要的时间，是作业等待、在就绪队列中排队、在处理机上运行及输入输出操作所花费时间的总和。

- 等待时间：进程处于等处理机（处于就绪队列）的时间之和，等待时间越长，用户满意度越低。
- 响应时间：用户提交请求到系统首次产生响应所需要的时间。

进程调度方式

- 非抢占调度方式：当一个进程正在处理中，即使有更为重要的进程进入到就绪队列中，仍然让正在执行的进程继续执行。
- 抢占调度方式：当一个进程正在处理中，如果有更为重要的进程进入到就绪队列中，则允许调度程序根据某种原则去暂停正在执行的进程，将处理机分配给这个更为重要或紧迫的进程。

调度算法

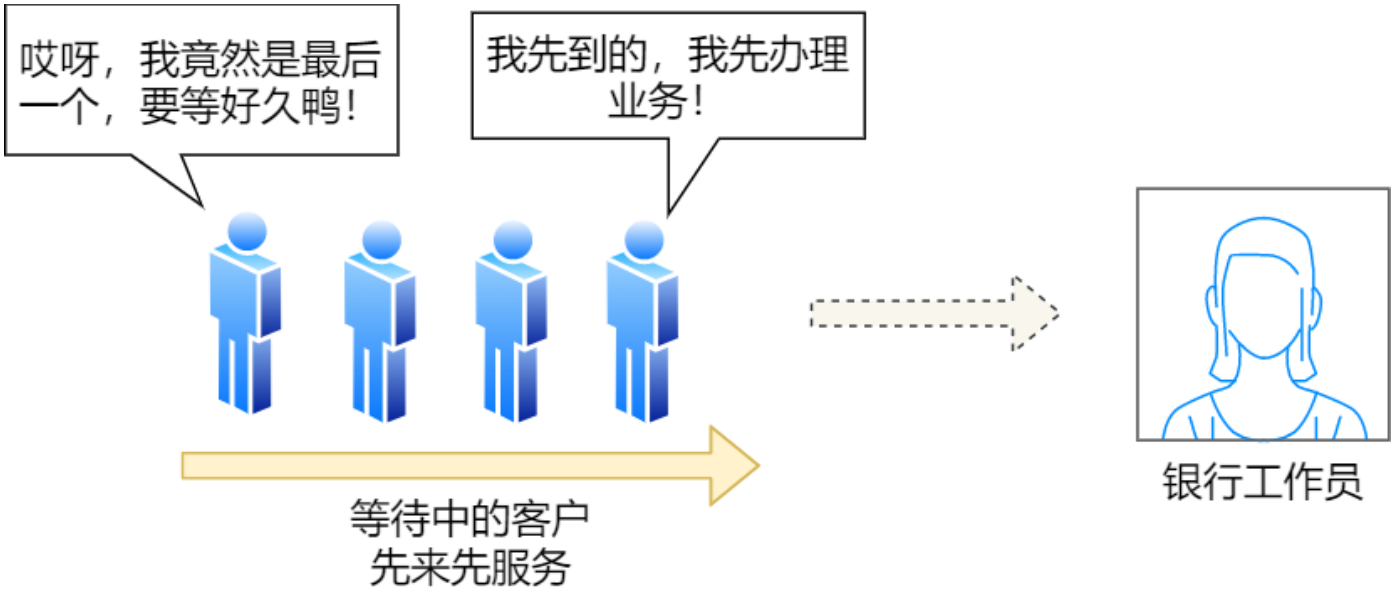


1. 先来先服务调度算法(FCFS)

每次从就绪队列选择最先进入队列的进程，然后一直运行，直到进程退出或被阻塞，才会继续从队列中选择第一个进程接着运行。

这种算法虽然看上去公平，但是如果有一个长作业需要处理，则后面的短作业需要处理很长时间。

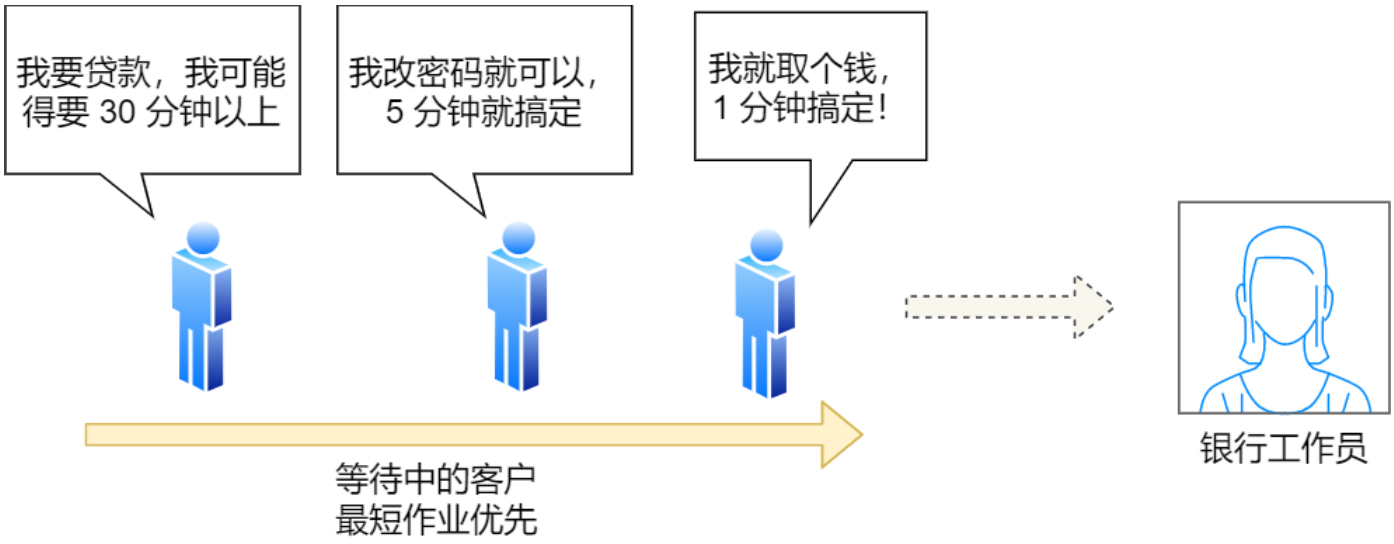
先来先调度算法的特点是算法简单，对长作业比较有利，对短作业不利，适用于 CPU 繁忙型作业的系统，而不适用于 I/O 繁忙型作业的系统。



2. 最短作业优先调度算法（SJF）

最短作业优先调度算法从就绪队列中选择一个估计运行时间最短的作业，将之调入到内存中运行，这有利于提高系统的吞吐量。

但是这对长作业十分不利，由于调度程序总是优先调度短作业，将会导致长作业长期不被调度，此外该算法也没有考虑到作业的紧迫程度，因此不能保证紧迫性作业会被及时处理。



3. 高响应比优先调度算法

每次进行进程调度时，先计算「响应比优先级」，然后把「响应比优先级」最高的进程投入运行

$$优先权 = \frac{等待时间 + 要求服务时间}{要求服务时间}$$

根据公式可以知道：

- 作业的等待时间相同时，如果要求服务时间越短，则响应比更高，有利于短作业执行
- 当要求服务时间相同时，响应比由等待时间决定，如果等待时间越长，则响应比越高
- 对于长作业，作业的响应比可以随着等待时间的增加而提高

4. 时间片轮转调度算法

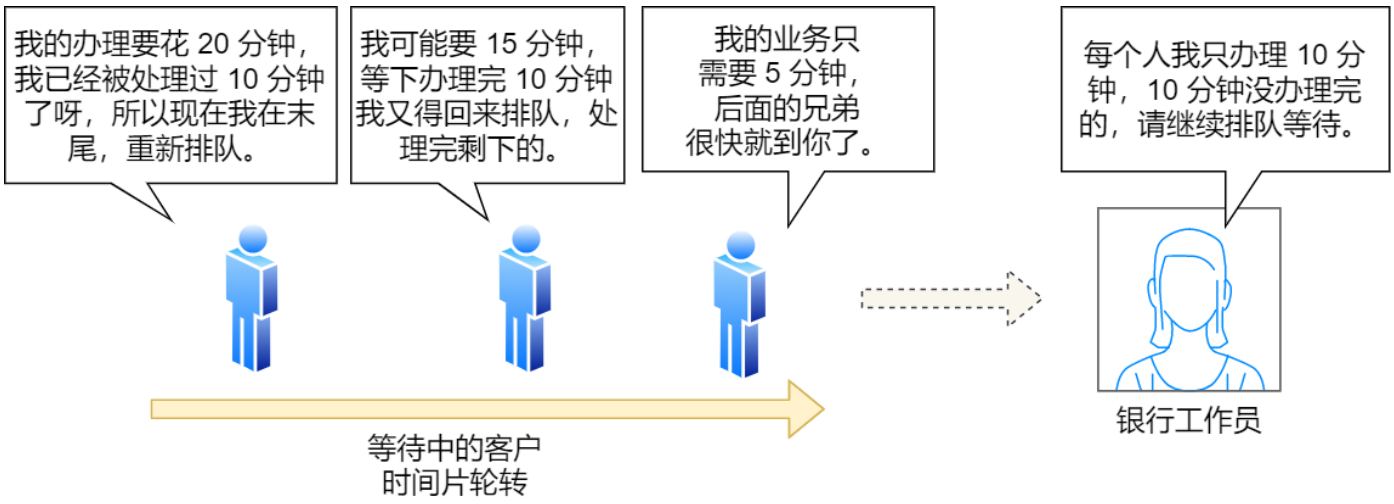
每个进程被分配一个时间段，称为时间片（Quantum），即允许该进程在该时间段中运行。

- 如果时间片用完，进程还在运行，那么将会把此进程从 CPU 释放出来，并把 CPU 分配给另外一个进程；
- 如果该进程在时间片结束前阻塞或结束，则 CPU 立即进行切换；

另外，时间片的长度就是一个很关键的点：

- 如果时间片设得太短会导致过多的进程上下文切换，降低了 CPU 效率；
- 如果设得太长又可能引起对短作业进程的响应时间变长。将

一般来说，时间片设为 `20ms~50ms` 通常是一个比较合理的折中值。

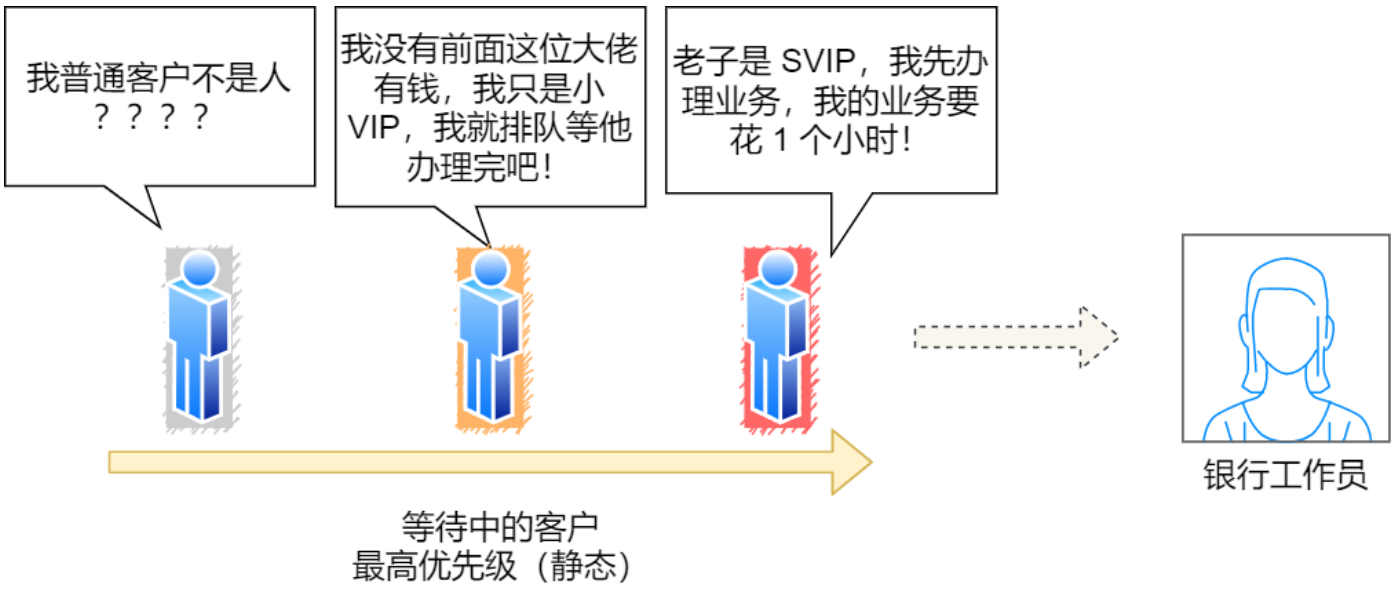


5. 最高优先级调度算法

从就绪队列中选择最高优先级的进程进行运行，但进程的优先级可以分为静态优先级和动态优先级

- 静态优先级：优先级在创建进程时已经确定，在进程运行期间保持不变，确定静态优先级的主要依据有进程类型，对资源的要求，用户要求。
- 动态优先级：进程运行过程中，根据进程运行时间和等待时间等因素调整进程的优先级

但是这种算法可能会导致低优先级的进程永远不被执行。



6. 多级队列调度算法

上面的各种调度算法是固定且单一的，无法满足系统中不同用户对进程调度策略的不同要求，多级队列调度算法在系统中设置多个就绪队列，将不同类型或性质的进程固定分配到不同的就绪队列，每个队列可以实施不同的调度算法。

7. 多级反馈队列调度算法

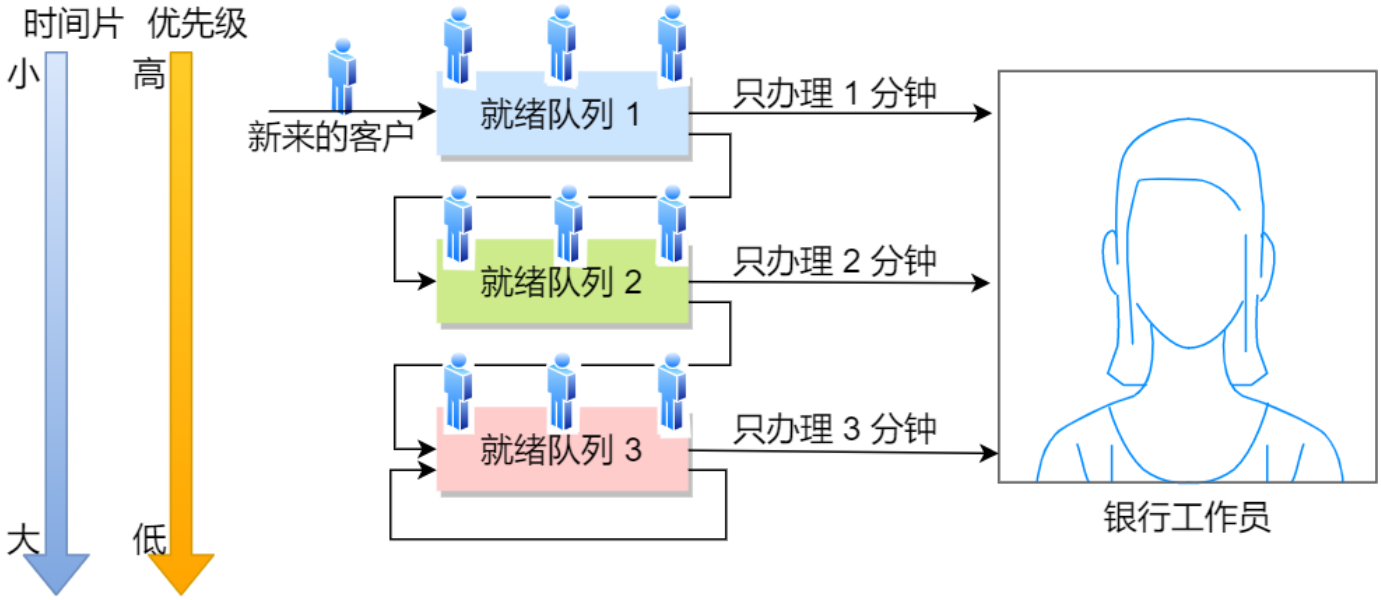
多级反馈队列调度算法融合了时间片轮转调度算法和优先级调度算法，通过动态调整进程的优先级和时间片大小，多级反馈队列调度算法可以兼顾多方面的系统目标

多级反馈队列调度算法的实现思想如下：

- 设置多个就绪队列，并为每个队列赋予不同的优先级。第1级队列的优先级最高，第2级队列的优先级次之，其余队列的优先级逐个降低。
- 赋予各个队列的进程运行时间片的大小各不相同。在优先级越高的队列中，每个进程的时间片就越小。例如，第`i+1`级队列的时间片要比第i级队列的时间片长1倍。
- 每个队列都采用FCFS算法。当新进程进入内存后，首先将它放入第1级队列的末尾，按FCFS原则等待调度。当轮到该进程执行时，如它能在该时间片内完成，便可撤离系统。若它在一个时间片结束时尚未完成，调度程序将其转入第2级队列的末尾等待调度：若它在第2级队列中运行一个时间片后仍未完成，再将它放入第3级队列...，依此类推。当进程最后被降到第n级队列后，在第n级队列中便采用时间片轮转方式运行。
- 按队列优先级调度。仅当第1级队列为空时，才调度第2级队列中的进程运行；仅当第`1~i-1`级队列均为空时，才会调度第i级队列中的进程运行。若处理机正在执行第i级队列中的某进程时，又有新进程进入任一优先级较高的队列，此时须立即把正在运行的进程放回到第级队列的末尾，而把处理机分配给新到的高优先级进程。

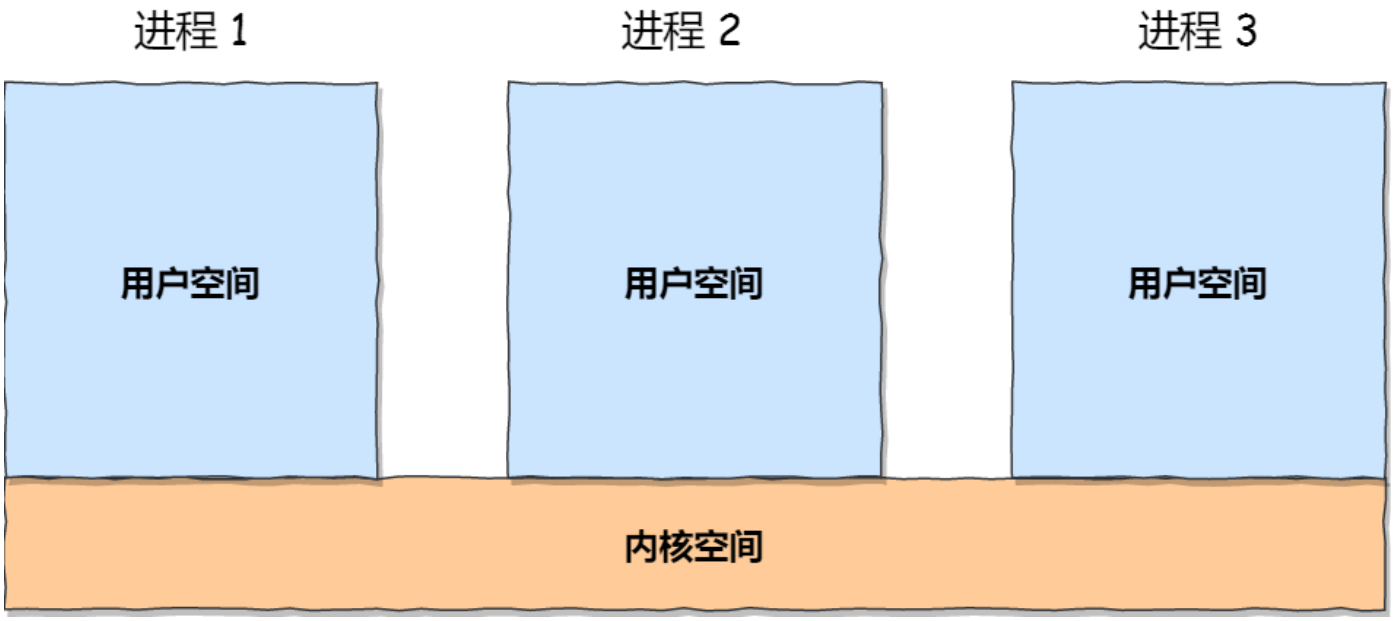
多级反馈队列的优势有以下几点：

- 终端型作业用户：短作业优先。
- 短批处理作业用户：周转时间较短。
- 长批处理作业用户：经过前面几个队列得到部分执行，不会长期得不到处理。



进程通信

进程通信指的是进程之间的信息交换，进程之间一般是相互独立的，但内核空间是每个进程都共享的，所以进程之间要通信必须通过内核。



共享存储

在通信的进程之间存在一块可直接访问的共享空间，通过对这片共享空间进行写/读操作实现进程之间的信息交换。

消息传递（消息队列）

若通信的进程之间不存在可直接访问的共享空间，则必须利用操作系统提供的消息传递方法进行进程通信，进程通过系统提供的发送消息和接收消息两个原语进行数据交换。

- 直接通信方式：发送进程直接把消息发送给接收进程，并将它挂在接收进程的消息缓冲队列上，接收进程从消息缓冲队列中取得消息。
- 间接通信方式：发送进程把消息发送到某个中间实体，接收进程从中间实体取得消息。这种中间实体一般称为信箱。

因为在内核中每个消息体都有一个最大长度的限制，所以消息队列不适合**比较大数据的传输**，而且通信也不是很及时。

管道

管道是指用于连接一个读进程和一个写进程以实现它们之间的通信的一个共享文件，又名pipe文件，向管道（共享文件）提供输入的发送进程（写进程），以字符流形式将大量的数据送入（写）管道：而接收管道输出的接收进程（即读进程）则从管道中接收（读）数据。

- 管道传输数据是半双工通信，某一时刻只能单向传输。
- 写入管道中的数据遵循先入先出的规则
- 管道所传送的数据是无格式的，这要求管道的读出方与写入方必须事先约定好数据的格式，如多少字节算一个消息等
- 管道不是普通的文件，不属于某个文件系统，其只存在于内存中
- 管道在内存中对应一个缓冲区, 不同的系统其大小不一定相同
- 从管道读数据是一次性操作，数据一旦被读走，它就从管道中被抛弃，释放空间以便写更多的数据
- 通信效率低，不适合进程间频繁地交换数据。

(1) 匿名管道和命名管道之间的区别

匿名管道：没有名字的管道，用完就销毁，`Linux`中的`|`就是一个匿名管道，只适用于父子进程之间的通信。

命名管道：提前创建了一个类型为管道的设备文件，在进程里只要使用这个设备文件，就可以相互通信，所以它可以在不相关的进程间进行通信。

(2) pipe函数

```
#include <unistd.h>
/**
 * 创建无名管道.
 * @param pipefd 为int型数组的首地址，其存放了管道的文件描述符
 * pipefd[0]、pipefd[1].
 * @return 创建成功返回0，创建失败返回-1.
 */
int pipe(int pipefd[2]);
/**
 * 当一个管道建立时，它会创建两个文件描述符 fd[0] 和 fd[1]。其中
 * fd[0] 固定用于读管道，而 fd[1] 固定用于写管道。
 * 一般文件 I/O的函数都可以用来操作管道(lseek() 除外。)
 */
```

（3）命名管道

命名管道（FIFO）不同于匿名管道之处在于它提供了一个路径名与之关联，以 FIFO 的文件形式存在于文件系统中，这样，即使与 FIFO 的创建进程不存在亲缘关系的进程，只要可以访问该路径，就能够彼此通过 FIFO 相互通信，因此，通过 FIFO 不相关的进程也能交换数据.

与无名管道(pipe)不同之处：

1. FIFO 在文件系统中作为一个特殊的文件而存在，但 FIFO 中的内容却存放在内存中;
2. 当使用 FIFO 的进程退出后，FIFO 文件将继续保存在文件系统中以便以后使用;
3. FIFO 有名字，不相关的进程可以通过打开命名管道进行通信.

通过命令创建有名管道 `mkfifo  myPipe`

通过函数创建有名管道

```
#include <sys/types.h>
#include <sys/stat.h>
/**
 * 命名管道的创建.
 * @param pathname 普通的路径名，也就是创建后 FIFO 的名字.
 * @param mode 文件的权限，
 * 与打开普通文件的 open() 函数中的 mode 参数相同。(0666).
 * @return 成功：0 状态码;
 *      失败：如果文件已经存在，则会出错且返回 -1.
 */
int mkfifo(const char *pathname, mode_t mode);
```

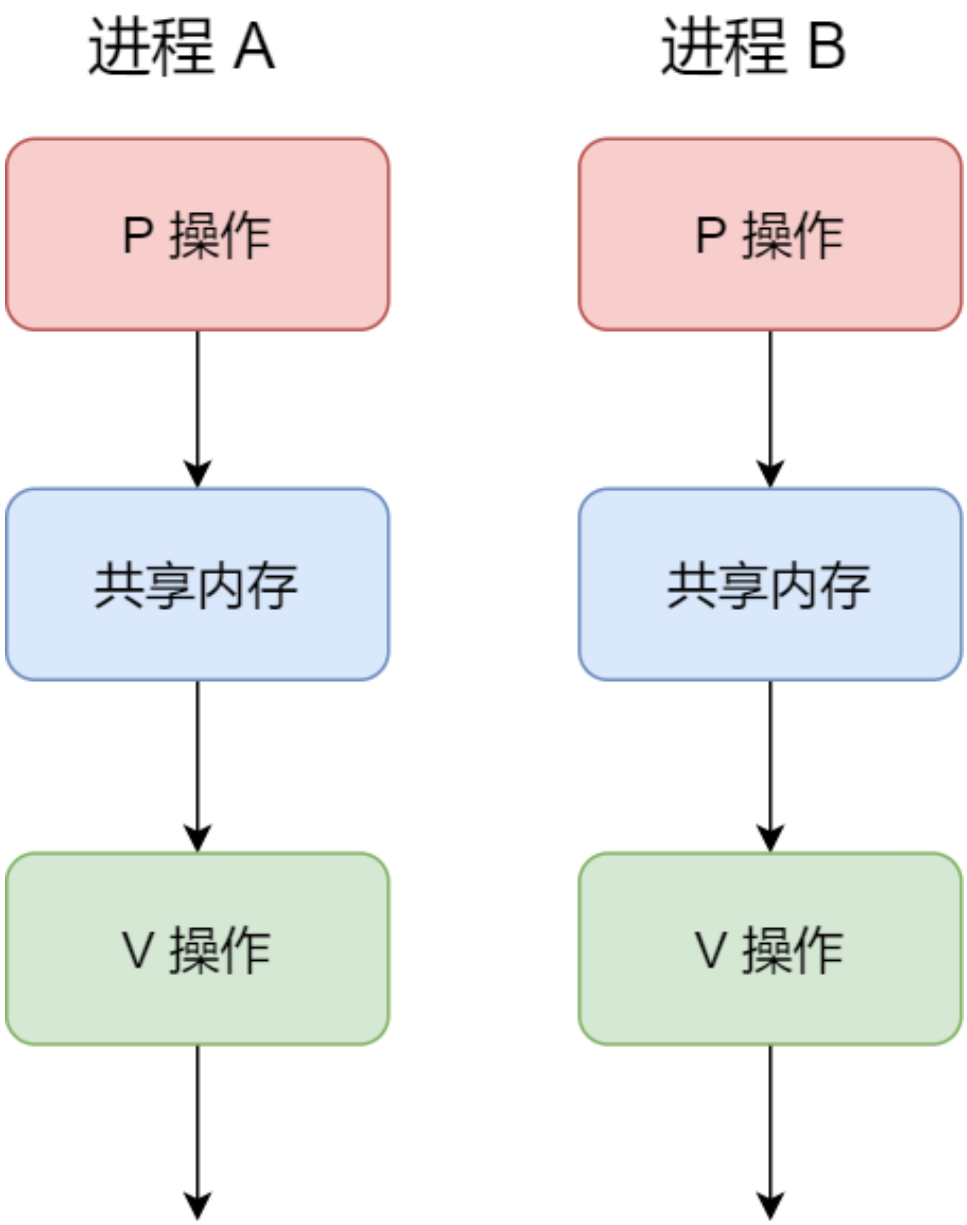
信号量

信号量用于控制多个进程对共享资源的访问，比如避免因为多个进程同时修改同一个共享内存造成冲突，信号量可以使共享的资源在任意时刻只能被一个进程访问。

信号量其实是一个整型的计数器，主要用于实现进程间的互斥与同步，而不是用于缓存进程间通信的数据。

信号量维护一个整数值，通常称为计数器。进程可以执行两种基本操作来操作信号量：

- **P（Wait）操作**：这个操作会把信号量减去 1，相减后如果信号量 < 0，则表明资源已被占用，进程需阻塞等待；相减后如果信号量 >= 0，则表明还有资源可使用，进程可正常继续执行。
- **V（Signal）操作**：这个操作会把信号量加上 1，相加后如果信号量 <= 0，则表明当前有阻塞中的进程，于是会将该进程唤醒运行；相加后如果信号量 > 0，则表明当前没有阻塞中的进程；

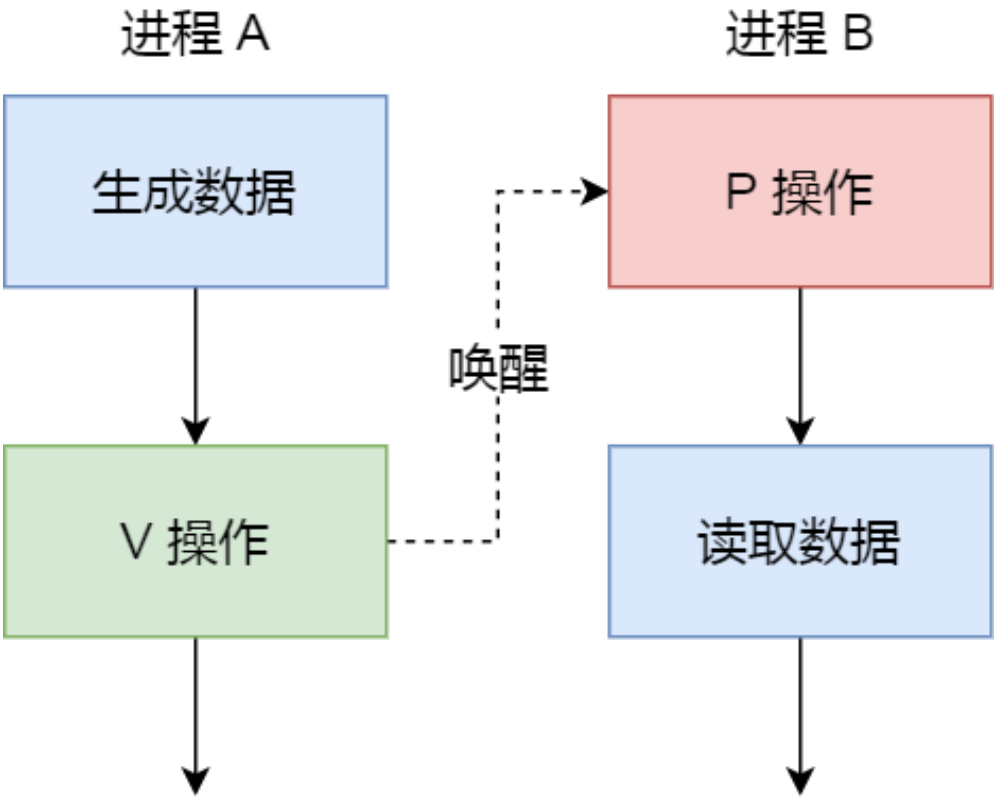


如果两个进程访问共享内存，我们可以初始化信号量为1

- 进程 A 在访问共享内存前，先执行了 P 操作，由于信号量的初始值为 1，故在进程 A 执行 P 操作后信号量变为 0，表示共享资源可用，于是进程 A 就可以访问共享内存。
- 若此时，进程 B 也想访问共享内存，执行了 P 操作，结果信号量变为了 -1，这就意味着临界资源已被占用，因此进程 B 被阻塞。
- 直到进程 A 访问完共享内存，才会执行 V 操作，使得信号量恢复为 0，接着就会唤醒阻塞中的线程 B，使得进程 B 可以访问共享内存，最后完成共享内存的访问后，执行 V 操作，使信号量恢复到初始值 1。

可以发现，信号初始化为 `1`，就代表着是**互斥信号量**，它可以保证共享内存在任何时刻只有一个进程在访问，这就很好的保护了共享内存。

还可以使用信号量实现多进程同步，比如可以初始化信号量为0



具体过程：

- 如果进程 B 比进程 A 先执行了，那么执行到 P 操作时，由于信号量初始值为 0，故信号量会变为 -1，表示进程 A 还没生产数据，于是进程 B 就阻塞等待；
- 接着，当进程 A 生产完数据后，执行了 V 操作，就会使得信号量变为 0，于是就会唤醒阻塞在 P 操作的进程 B；
- 最后，进程 B 被唤醒后，意味着进程 A 已经生产了数据，于是进程 B 就可以正常读取数据了。

可以发现，信号初始化为 `0`，就代表着是**同步信号量**，它可以保证进程 A 应在进程 B 之前执行。

信号

在 Linux 操作系统中， 为了响应各种各样的事件，提供了几十种信号，分别代表不同的意义。我们可以通过 `kill -l` 命令，查看所有的信号。

信号事件的来源主要有硬件来源（如键盘 Cltr+C ）和软件来源（如 kill 命令）。

信号是进程间通信机制中**唯一的异步通信机制**，它可以在一个进程中通知另一个进程发生了某种事件从而实现进程通信。

Socket通信

`Socket`通信是一种网络编程中常见的通信方式，但它也可以在同一台机器上的不同进程之间进行通信。

创建 socket 的系统调用：

```
int socket(int domain, int type, int protocal)
```

三个参数分别代表：

- domain 参数用来指定协议族，比如 AF_INET 用于 IPV4、AF_INET6 用于 IPV6、AF_LOCAL/AF_UNIX 用于本

机；
- type 参数用来指定通信特性，比如 SOCK_STREAM 表示的是字节流，对应 TCP、SOCK_DGRAM 表示的是数据报，对应 UDP、SOCK_RAW 表示的是原始套接字；
- protocal 参数原本是用来指定通信协议的，但现在基本废弃。因为协议已经通过前面两个参数指定完成，protocol 目前一般写成 0 即可；

根据创建 socket 类型的不同，通信的方式也就不同：

- 实现 TCP 字节流通信： socket 类型是 AF_INET 和 SOCK_STREAM；
- 实现 UDP 数据报通信：socket 类型是 AF_INET 和 SOCK_DGRAM；
- 实现本地进程间通信： 「本地字节流 socket 」类型是 AF_LOCAL 和 SOCK_STREAM，「本地数据报 socket 」类型是 AF_LOCAL 和 SOCK_DGRAM。另外，AF_UNIX 和 AF_LOCAL 是等价的，所以 AF_UNIX 也属于本地 socket；

线程通信

线程间的通信目的主要是用于线程同步。所以线程没有像进程通信中的用于数据交换的通信机制。

同一进程的不同线程共享同一份内存区域，所以线程之间可以方便、快速地共享信息。只需要将数据复制到共享（全局或堆）变量中即可。但是需要避免出现多个线程试图同时修改同一份信息。

线程属性

1、线程属性初始化和销毁

```
#include <pthread.h>
/**
 * 初始化线程属性函数，注意：应先初始化线程属性，再pthread_create创建线程.
 * @param attr 线程属性结构体.
 * @return 成功：0；失败：错误号.
 */
int pthread_attr_init(pthread_attr_t *attr);

/**
 * 销毁线程属性所占用的资源函数.
 * @param attr 线程属性结构体.
 * @return 成功：0；失败：错误号.
 */
int pthread_attr_destroy(pthread_attr_t *attr);
```

2、线程分离状态

（1）非分离状态：

线程的默认属性是非分离状态，这种情况下，原有的线程等待创建的线程结束。只有当pthread_join()函数返回时，创建的线程才算终止，才能释放自己占用的系统资源。

（2）分离状态：

分离线程没有被其他的线程所等待，自己运行结束了，线程也就终止了，马上释放系统资源。应该根据自己的需要，选择适当的分离状态。

注意：

1. 如果设置一个线程为分离线程，而这个线程运行又非常快，它很可能在pthread_create函数返回之前就终止了，它终止以后就可能将线程号和系统资源移交给其他的线程使用，这样调用pthread_create的线程就得到了错误的线程号。
2. 要避免这种情况可以采取一定的同步措施，最简单的方法之一是可以在被创建的线程里调用pthread_cond_timedwait函数，让这个线程等待一会儿，留出足够的时间让函数pthread_create返回。

```
#include <pthread.h>
/**
 * 设置线程分离状态.
 * @param attr 已初始化的线程属性.
 * @detachstate(分离状态)
 *      1. PTHREAD_CREATE_DETACHED（分离线程）;
 *      2. PTHREAD_CREATE_JOINABLE（非分离线程）.
 * @return 成功：0; 失败：非0.
 */

/**
 * 获取线程分离状态.
 * @param attr 已初始化的线程属性.
 * @detachstate(分离状态)
 *      1. PTHREAD_CREATE_DETACHED（分离线程）;
 *      2. PTHREAD_CREATE_JOINABLE（非分离线程）.
 * @return 成功：0; 失败：非0.
 */
int pthread_attr_getdetachstate(const pthread_attr_t *attr, int *detachstate);
```

3、线程栈地址

当进程栈地址空间不够用时，指定新建线程使用由malloc分配的空间作为自己的栈空间。

通过pthread_attr_setstack和pthread_attr_getstack两个函数分别设置和获取线程的栈地址。

```
#include <pthread.h>
/**
 * 设置线程的栈地址.
 * @param attr 指向一个线程属性的指针.
 * @param stackaddr 内存首地址.
 * @param stacksize 返回线程的堆栈大小.
 * @return 成功：0; 失败：错误号.
 */
int pthread_attr_setstack(pthread_attr_t *attr, void *stackaddr,  size_t stacksize);

/**
 * 获取线程的栈地址.
 * @param attr 指向一个线程属性的指针.
 * @param stackaddr 返回获取的栈地址.
 * @param stacksize 返回获取的栈大小.
```

```
 * @return 成功: 0; 失败: 错误号.
 */
int pthread_attr_getstack(const pthread_attr_t *attr, void **stackaddr,  size_t
*stacksize);-
```

4、线程栈大小

```
#include <pthread.h>
/**
 * 设置线程的栈大小.
 * @param attr 指向一个线程属性的指针.
 * @param stacksize 线程的堆栈大小.
 * @return 成功: 0; 失败: 错误号.
 */
int pthread_attr_setstacksize(pthread_attr_t *attr, size_t stacksize);

/**
 * 获取线程的栈大小.
 * @param attr 指向一个线程属性的指针.
 * @param stacksize 返回线程的堆栈大小.
 * @return 成功: 0; 失败: 错误号.
 */
int pthread_attr_getstacksize(const pthread_attr_t *attr, size_t *stacksize);
```

5、线程使用注意事项

（1）主线程退出其他线程不退出，主线程应调用pthread_exit

（2）避免僵尸线程

1. pthread_join
2. pthread_detach
3. pthread_create指定分离属性

被join线程可能在join函数返回前就释放完自己的所有内存资源，所以不应当返回被回收线程栈中的值

（3）malloc和mmap申请的内存可以被其他线程释放

（4）应避免在多线程模型中调用fork，除非马上exec，子进程中只有调用fork的线程存在，其他线程t在子进程中均pthread_exit

（5）信号的复杂语义很难和多线程共存，应避免在多线程引入信号机制

（6）Cache伪共享：

这种因为多个线程同时读写同一个 Cache Line 的不同变量时，而导致 CPU Cache 失效的现象。避免的方式一般有 Cache Line 大小字节对齐，以及字节填充等方法。在 Linux 内核中存在cachelinealignedin_smp 宏定义，是用于解决伪共享的问题。

多线程

用户态的多线程模型，同一个进程内部有多个线程，所有的线程共享同一个进程的内存空间，进程中定义的全局变量会被所有的线程共享；

i++在计算机中并不是原子操作，涉及内存取数，计算和写入内存几个环节，而线程的切换有可能发生在上述任何一个环节中间，所以不同的操作顺序很有可能带来意想不到的结果。

多线程的好处：

主要原因是许多应用中同时发生多个活动，某些活动随着时间推移而阻塞，将这些应用程序分解成并发运行的多个线程，简化设计模型。同时多线程有共享同一地址空间和可用数据的能力，这是多进程没有的。

线程比进程开销小，更容易创建和释放。

多个线程是IO密集型时，多线程可以使这些活动彼此重叠运行，可以加快程序执行的速度。

对于线程需要考虑：

线程之间有无先后访问顺序（线程依赖关系）

多个线程共享访问同一变量（同步互斥问题）

同一进程的多个线程共享进程的资源，除了标识线程的tid，每个线程还有自己独立的栈空间，线程彼此之间是无法访问其他线程栈上内容的。

进程表：

为了实现进程模型，操作系统维护着一张表格(一个结构数组)，即进程表。

每个进程占有一个进程表项。(有些著作称这些为进程控制块)

该表项包含了一个进程状态的重要信息

包括程序计数器、堆栈指针、内存分配状况、所打开文件的状态、账号的调度信息，以及其他在进程由运行态转换到就绪态或阻塞态时必须保存的信息，从而**保证该进程随后能再次启动**，就像从未中断过一样

7、进程切换为何比线程慢

涉及到虚拟内存的问题，进程切换涉及虚拟地址空间的切换而线程不会。

因为每个进程都有自己的虚拟地址空间，而线程是共享所在进程的虚拟地址空间的，所以同一个进程中的线程进行线程切换时不涉及虚拟地址空间的转换。

把虚拟地址转换为物理地址需要查找页表，页表查找是一个很慢的过程（至少访问2次内存），因此通常使用Cache来缓存常用的地址映射，这样可以加速页表查找，这个cache就是TLB（快表）。

由于每个进程都有自己的虚拟地址空间，那么显然每个进程都有自己的页表，那么当进程切换后页表也要进行切换，页表切换后TLB就失效了，cache失效导致命中率降低，那么虚拟地址转换为物理地址就会变慢，表现出来的就是程序运行会变慢，而线程切换则不会导致TLB失效，因为线程线程无需切换地址空间，这也就是进程切换要比同进程下线程切换慢的原因。

10、守护进程

守护进程是指在后台运行的，没有控制终端与它相连的进程。它独立于控制终端，周期性地执行某种任务。

Linux的大多数服务器就是用守护进程的方式实现的，如web服务器进程http等。

创建守护进程要点：

（1）让程序在后台执行。

方法是调用fork()产生一个子进程，然后使父进程退出。

（2）调用setsid()创建一个新对话期。

守护进程需要摆脱父进程的影响，方法是调用setsid()使进程成为一个会话组长。setsid()调用成功后，进程成为新的会话组长和进程组长，并与原来的登录会话、进程组和控制终端脱离。

（3）禁止进程重新打开控制终端。

经过1和2，进程已经成为一个无终端的会话组长，但是它可以重新申请打开一个终端。为了避免这种情况发生，可以通过使进程不再是会话组长来实现。再一次通过fork（）创建新的子进程，使调用fork的进程退出。

（4）关闭不再需要的文件描述符。

子进程从父进程继承打开的文件描述符。如不关闭，将会浪费系统资源，造成进程所在的文件系统无法卸下以及引起无法预料的错误。首先获得最高文件描述符值，然后用一个循环程序，关闭0到最高文件描述符值的所有文件描述符。

（5）将当前目录更改为根目录。

（6）子进程从父进程继承的文件创建屏蔽字可能会拒绝某些许可权。

为防止这一点，使用unmask(0)将屏蔽字清零。

（7）处理SIGCHLD信号。

对于服务器进程，在请求到来时往往生成子进程处理请求。如果子进程等待父进程捕获状态，则子进程将成为僵尸进程（zombie），从而占用系统资源。如果父进程等待子进程结束，将增加父进程的负担，影响服务器进程的并发性能。在Linux下可以简单地将SIGCHLD信号的操作设为SIG_IGN。这样，子进程结束时不会产生僵尸进程。

11、僵尸进程

多进程程序，父进程一般需要跟踪子进程的退出状态，当子进程退出，父进程在运行，子进程必须等到父进程捕获到了子进程的退出状态才真正结束。在子进程结束后，父进程读取状态前，此时子进程为僵尸进程。

设置僵尸进程的目的是维护子进程的信息，以便父进程在以后某个时候获取。这些信息至少包括进程ID，进程的终止状态，以及该进程使用的CPU时间。所以当终止子进程的父进程调用wait或waitpid时就可以得到这些信息。

但是子进程停止在僵尸态会占据内核资源，所以需要避免僵尸进程的产生或立即结束子进程的僵尸态。

1. 父进程调用wait/waitpid等函数等待子进程结束，如果尚无子进程退出wait会导致父进程阻塞。waitpid只会等待由pid参数指定的子进程，同时也是非阻塞，目标进程正常退出返回子进程PID，还没结束返回0。

2. 在事件已经发生情况下执行非阻塞调用可以提高程序效率。对waitpid，最好在子进程退出后调用。使用SIGCHLD信号通知父进程，子进程结束。

> 父进程中捕获信号，然后在信号处理函数中调用waitpid以彻底结束子进程

```
static void handle_child(int sig)
{
    pid_t pid;
    int stat;
    while((pid = waitpid(-1, &stat, WNOHANG)) > 0)
    {
        //结束子进程的处理;
    }
}
```

1. 通过signal(SIGCHLD, SIG_IGN)通知内核对子进程的结束不关心，由内核回收。如果不想让父进程挂起，可以在父进程中加入一条语句：signal(SIGCHLD,SIG_IGN);表示父进程忽略SIGCHLD信号，该信号是子进程退出的时候向父进程发送的。
2. 忽略SIGCHLD信号，这常用于并发服务器的性能的一个技巧因为并发服务器常常fork很多子进程，子进程终结之后需要服务器进程去wait清理资源。如果将此信号的处理方式设为忽略，可让内核把僵尸子进程转交给init进程去处理，省去了大量僵尸进程占用系统资源。

12、多进程

进程结构由以下几个部分组成：代码段、堆栈段、数据段。代码段是静态的二进制代码，多个程序可以共享。

父进程创建子进程之后，父、子进程除了pid外，几乎所有的部分几乎一样。

父、子进程共享全部数据，子进程在写数据时会使用写时复制技术将公共的数据重新拷贝一份，之后在拷贝出的数据上进行操作；不是对同一块数据进行操作；

如果子进程想要运行自己的代码段，还可以通过调用execv()函数重新加载新的代码段，之后就和父进程独立开了。

进程通信

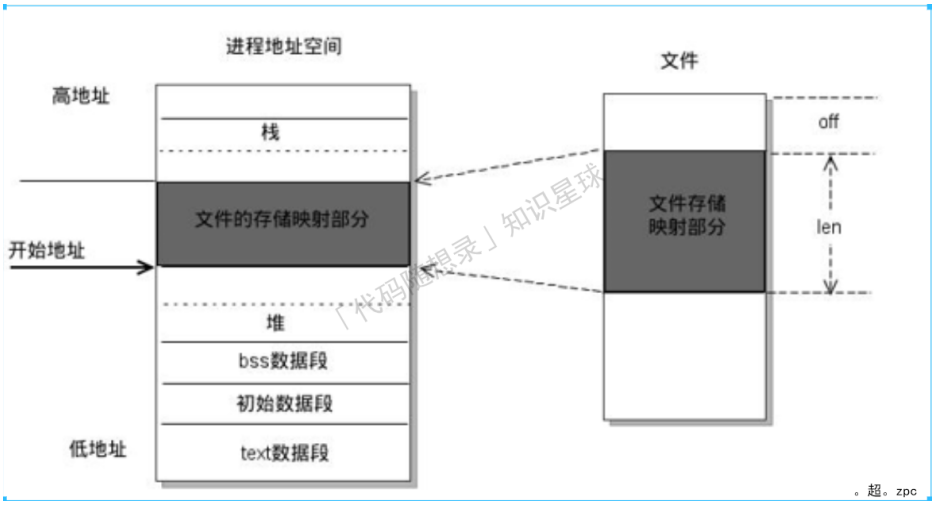
共享存储映射

存储映射I/O (Memory-mapped I/O) 使一个磁盘文件与存储空间中的一个缓冲区相映射.

于是当从缓冲区中取数据，就相当于读文件中的相应字节。于此类似，将数据存入缓冲区，则相应的字节就自动写入文件。

这样，就可在不适用read和write函数的情况下，使用地址（指针）完成I/O操作，进程就可以直接通过读写内存来操作文件.

共享内存可以说是最有用的进程间通信方式，也是最快的IPC形式, 因为进程可以直接读写内存，而不需要任何数据的拷贝.



1、存储映射函数：

（1）mmap函数

```
#include <sys/mman.h>
/**
 * 一个文件或者其它对象映射进内存.
 * @param addr 指定映射的起始地址, 通常设为NULL, 由系统指定.
 * @param length 映射到内存的文件长度.
 * @param prot 映射区的保护方式, 最常用的:
 *              (1) 读: PROT_READ;
 *              (2) 写: PROT_WRITE;
 *              (3) 读写: PROT_READ | PROT_WRITE.
 * @param flags 映射区的特性, 可以是:
 *              (1) MAP_SHARED: 写入映射区的数据会复制回文件,
 *                  且允许其他映射该文件的进程共享。
 *              (2) MAP_PRIVATE: 对映射区的写入操作
 *                   会产生一个映射区的复制(copy - on - write),
 *                   对此区域所做的修改不会写回原文件。
 * @param fd 由open返回的文件描述符, 代表要映射的文件.
 * @param offset 以文件开始处的偏移量,
 *          必须是4k的整数倍, 通常为0, 表示从文件头开始映射.
 * @return 成功: 返回创建的映射区首地址; 失败: MAP_FAILED宏.
 */
void *mmap(void *addr, size_t length,
    int prot, int flags, int fd, off_t offset);
```

> 内存是按照页来区别的，通常一页就是4K

总结：

1. 第一个参数写成NULL
2. 第二个参数要映射的文件大小 > 0
3. 第三个参数：PROT_READ 、PROT_WRITE
4. 第四个参数：MAP_SHARED 或者 MAP_PRIVATE
5. 第五个参数：打开的文件对应的文件描述符
6. 第六个参数：4k的整数倍，通常为0

（2）munmap函数

```
#include <sys/mman.h>
/**
 * 释放内存映射区.
 * @param addr 使用mmap函数创建的映射区的首地址.
 * @param length 映射区的大小.
 * @return 成功返回0; 失败返回-1.
 */
int munmap(void *addr, size_t length);
```

2、注意事项:

1. 创建映射区的过程中，隐含着一次对映射文件的读操作
2. 当MAP_SHARED时，要求：映射区的权限应 <=文件打开的权限(出于对映射区的保护)。而MAP_PRIVATE则无所谓，因为mmap中的权限是对内存的限制
3. 映射区的释放与文件关闭无关。只要映射建立成功，文件可以立即关闭
4. 特别注意，当映射文件大小为0时，不能创建映射区。所以，用于映射的文件必须要有实际大小。mmap使用时常常会出现总线错误，通常是由于共享文件存储空间大小引起的
5. munmap传入的地址一定是mmap的返回地址。坚决杜绝指针++操作
6. 如果文件偏移量必须为4K的整数倍
7. mmap创建映射区出错概率非常高，一定要检查返回值，确保映射区建立成功再进行后续操作

3、匿名映射实现父子进行通信

（1）为什么使用匿名的方式实现通信?

内存映射的需要依赖文件。而建立文件建立好了只会还要unlink close掉，比较麻烦;

（2）有什么好的不能办法进行解决?

直接使用匿名映射来代替;

（3）Linux系统给我们提供了创建匿名映射区的方法，无需依赖一个文件即可创建映射区。同样需要借助标志位参数flags来指定;

（4）使用MAP_ANONYMOUS (或MAP_ANON):

```
int *p = mmap(NULL, 4, PROT_READ|PROT_WRITE, MAP_SHARED|MAP_ANONYMOUS, -1, 0);
```

消息队列

基本原理：A 进程要给 B 进程发送消息，A 进程把数据放在对应的消息队列后就可以正常返回了，B 进程需要的时候再去读取数据就可以了。

特点：

1. 消息队列是保存在内核中的消息链表，每个消息体都是固定大小的存储块。如果进程从消息队列中读取了消息体，内核就会把这个消息体删除。
2. 如果没有释放消息队列或者没有关闭操作系统，消息队列会一直存在。

缺点：

1. 通信不及时，附件也有大小限制。
2. 消息队列不适合比较大数据的传输，每个消息体都有一个最大长度的限制，同时所有队列所包含的全部消息体的总长度也是有上限
3. 消息队列通信过程中，存在用户态与内核态之间的数据拷贝开销

信号

信号是linux进程通信的最古老的方式;

信号是软件中断，它是在软件层次上对中断机制的一种模拟，是一种异步通信的方式 。信号可以导致一个正在运行的进程被另一个正在运行的异步进程中断，转而处理某一个突发事件;

信号可以直接进行用户空间进程和内核空间进程的交互，内核进程可以利用它来通知用户空间进程发生了哪些系统事件.

1、信号的特点

1. 简单
2. 不能携带大量信息
3. 满足某个特定条件才发送

2、一个完整的信号周期

1. 信号的产生
2. 信号在进程种的注册，信号在进程种的注销
3. 执行信号处理函数

3、信号编号

（1）不存在编号为0的信号

1. 其中1-31号信号称之为常规信号（也叫普通信号或标准信号）
2. 34-64称之为实时信号，驱动编程与硬件相关。名字上区别不大。而前32个名字各不相同

（2）不存在编号为0的号，也没有32-33号

（3）比较重要的一些，需要记住的几个信号

1. SIGINT 当用户按下了<Ctrl+C>组合键时，用户终端向正在运行中的由该终端启动的程序发出此信号，终止进程
2. SIGQUIT 用户按下<ctrl+>组合键时产生该信号，用户终端向正在运行中的由该终端启动的程序发出些信号,终止进程
3. SIGSEGV 指示进程进行了无效内存访问(段错误), 终止进程并产生core文件
4. SIGPIPE Broken pipe向一个没有读端的管道写数据,终止进程
5. SIGCHLD 子进程结束时，父进程会收到这个信号,忽略这个信号

4、信号四要素

（1）编号: man 7 signal 查看文档帮助

（2）名称

（3）事件

（4）默认处理动作:

1. Term：终止进程
2. Ign： 忽略信号 (默认即时对该种信号忽略操作)
3. Core：终止进程，生成Core文件。(查验死亡原因，用于gdb调试)
4. Stop：停止（暂停）进程
5. Cont：继续运行进程

特别强调: 9) SIGKILL 和19) SIGSTOP信号，不允许忽略和捕捉，只能执行默认动作。甚至不能将其设置为阻塞.

5、信号的状态

（1）产生

1. 当用户按某些终端键时，将产生信号
2. 硬件异常将产生信号
3. 软件异常将产生信号
4. 调用系统函数(如：kill、raise、abort)将发送信号
5. 运行 kill /killall命令将发送信号

（2）未决状态：没有被处理

（3）递达状态：信号被处理了

6、阻塞信号集和未决信号集

（1）阻塞信号集

将某些信号加入集合，对他们设置屏蔽，当屏蔽x信号后，再收到该信号，该信号的处理将推后(处理发生在解除屏蔽后)

（2）未决信号集合

信号产生，未决信号集中描述该信号的位立刻翻转为1，表示信号处于未决状态。当信号被处理对应位翻转回为0。这一时刻往往非常短暂.

7、信号产生函数

（1）kill函数：

```
#include <sys/types.h>
#include <signal.h>
/**
 * 给指定进程发送指定信号(不一定杀死).
 * @param pid 取值有四种情况:
 *            pid > 0:  将信号传送给进程 ID 为pid的进程.
 *            pid = 0 :  将信号传送给当前进程所在进程组中的所有进程.
 *            pid = -1 : 将信号传送给系统内所有的进程.
 *            pid < -1 : 将信号传给指定进程组的所有进程，这个进程组号等于 pid 的绝对值.
 * @param sig 信号的编号，这里可以填数字编号，也可以填信号的宏定义.
 *            可以通过命令 kill - l("l" 为字母)进行相应查看.
 *            不推荐直接使用数字，应使用宏名，因为不同操作系统信号编号可能不同，但名称一致.
 * @return 成功: 0; 失败: -1.
 * 普通用户基本规则是: 发送者实际或有效用户ID == 接收者实际或有效用户ID.
 */
int kill(pid_t pid, int sig);
```

（2）raise函数

```
#include <signal.h>
/**
 * 给当前进程发送指定信号(自己给自己发)，等价于 kill(getpid(), sig).
 * @param sig 信号编号.
 * @return  成功：0；失败：非0值.
 */
int raise(int sig);
```

（3）abort函数

```
#include <stdlib.h>
/**
 * 给自己发送异常终止信号 6) SIGABRT，并产生core文件，等价于kill(getpid(), SIGABRT).
 */
void abort(void);
```

（4）alarm函数(闹钟)

```
#include <unistd.h>
/**
 * 设置定时器(闹钟)。在指定seconds后，内核会给当前进程发送14）SIGALRM信号。进程收到该信号，默认动作终止。每个进程都有且只有唯一的一个定时器；
 * 取消定时器alarm(0)，返回旧闹钟余下秒数.
 * @param seconds 指定的时间，以秒为单位.
 * @return 返回0或剩余的秒数.
 * /
unsigned int alarm(unsigned int seconds);
```

（5）setitimer函数（定时器）

```
#include <sys/time.h>
struct itimerval {
    struct timerval it_interval; // 闹钟触发周期
    struct timerval it_value;    // 闹钟触发时间
};
struct timeval {
    long tv_sec;            // 秒
    long tv_usec;           // 微秒
}
/**
 * 设置定时器(闹钟)。 可代替alarm函数。精度微秒us，可以实现周期定时.
 * @param which 指定定时方式：
 *              (1) 自然定时：ITIMER_REAL → 14) SIGALRM计算自然时间；
 *              (2) 虚拟空间计时(用户空间)：ITIMER_VIRTUAL → 26) SIGVTALRM  只计算进程占用cpu的时间；
```

```
 *              (3) 虚拟空间计时(用户空间)：ITIMER_VIRTUAL → 26) SIGVTALRM  只计算进程占用cpu
的时间.
 * @param new_value 负责设定timeout时间.
 * @param old_value 存放旧的timeout值，一般指定为NULL.
 * @return 成功：0；失败：-1.
 */
int setitimer(int which,  const struct itimerval *new_value, struct itimerval
*old_value);
// itimerval.it_value：  设定第一次执行function所延迟的秒数
// itimerval.it_interval：   设定以后每几秒执行function
```

8、自定义信号集函数

```
#include <signal.h>
int sigemptyset(sigset_t *set);    // 将set集合置空
int sigfillset(sigset_t *set);    // 将所有信号加入set集合
int sigaddset(sigset_t *set, int signo);    // 将signo信号加入到set集合
int sigdelset(sigset_t *set, int signo);    // 从set集合中移除signo信号
int sigismember(const sigset_t *set, int signo);    // 判断信号是否存在
```

除sigismember外，其余操作函数中的set均为传出参数。sigset_t类型的本质是位图;

9、阻塞信号集

（1）信号阻塞集也称信号屏蔽集、信号掩码;

（2）信号阻塞集用来描述哪些信号递送到该进程的时候被阻塞;

（3）sigprocmask函数

```
#include <signal.h>
/**
 * 检查或修改信号阻塞集，根据 how 指定的方法对进程的阻塞集合进行修改，
 * 新的信号阻塞集由 set 指定，而原先的信号阻塞集合由 oldset 保存.
 *
 * @param how 信号阻塞集合的修改方法，有 3 种情况：
 *            (1) SIG_BLOCK：向信号阻塞集合中添加 set 信号集，
 *                新的信号掩码是set和旧信号掩码的并集。相当于 mask = mask|set;
 *            (2) SIG_UNBLOCK：从信号阻塞集合中删除 set 信号集，
 *                从当前信号掩码中去除 set 中的信号。相当于 mask = mask & ~ set;
 *            (3) SIG_SETMASK：将信号阻塞集合设为 set 信号集，
 *                相当于原来信号阻塞集的内容清空，然后按照 set 中的信号重新设置信号阻塞集。相当于
mask = set.
 *
 * @set 要操作的信号集地址,若 set 为 NULL，则不改变信号阻塞集合，函数只把当前信号阻塞集合保存到
oldset 中.
 * @oldset 保存原先信号阻塞集地址.
 * @return 成功：0；失败：-1，失败时错误代码只可能是 EINVAL，表示参数 how 不合法.
 */
int sigprocmask(int how, const sigset_t *set, sigset_t *oldset);
```

（4）sigpending函数

```
#include <signal.h>
/**
 * 读取当前进程的未决信号集.
 * @param set 未决信号集.
 * @return 成功: 0; 失败: -1.
 */
int sigpending(sigset_t *set);
```

10、信号捕捉

> 注意: SIGKILL 和 SIGSTOP 不能更改信号的处理方式，因为它们向用户提供了一种使进程终止的可靠方法;

（1）sigaction函数

```
#include <signal.h>
/**
 * 检查或修改指定信号的设置（或同时执行这两种操作）.
 * @param signum 要操作的信号.
 * @param act 要设置的对信号的新处理方式（传入参数）.
 * @param oldact: 原来对信号的处理方式（传出参数）.
 *
 * 如果 act 指针非空，则要改变指定信号的处理方式（设置），
 * 如果 oldact 指针非空，则系统将此前指定信号的处理方式存入 oldact.
 *
 * @return 成功: 0; 失败: -1.
 */
int sigaction(int signum, const struct sigaction *act, struct sigaction *oldact);
```

（2）sa_handler、sa_sigaction：信号处理函数指针

和 signal() 里的函数指针用法一样，应根据情况给sa_sigaction、sa_handler 两者之一赋值

其取值如下：

1. SIG_IGN：忽略该信号
2. SIG_DFL：执行系统默认动作
3. 处理函数名：自定义信号处理函数

（3）sa_mask：

信号阻塞集，在信号处理函数执行过程中，临时屏蔽指定的信号;

（4）sa_flags：

用于指定信号处理的行为，通常设置为0，表使用默认属性。它可以是一下值的“按位或”组合：

1. A_RESTART：使被信号打断的系统调用自动重新发起（已经废弃）
2. SA_NOCLDSTOP：使父进程在它的子进程暂停或继续运行时不会收到 SIGCHLD 信号
3. SA_NOCLDWAIT：使父进程在它的子进程退出时不会收到 SIGCHLD 信号，这时子进程如果退出也不会成为僵尸进程

4. SA_NODEFER：使对信号的屏蔽无效，即在信号处理函数执行期间仍能发出这个信号
5. SA_RESETHAND：信号处理之后重新设置为默认的处理方式
6. SA_SIGINFO：使用 sa_sigaction 成员而不是 sa_handler 作为信号处理函数

11、struct sigaction结构体

```
struct sigaction {
    void(*sa_handler)(int); //旧的信号处理函数指针
    void(*sa_sigaction)(int, siginfo_t *, void *); //新的信号处理函数指针
    sigset_t   sa_mask;       //信号阻塞集
    int        sa_flags;      //信号处理的方式
    void(*sa_restorer)(void); //已弃用
};
```

12、信号处理函数

```
/**
 * @param signum 信号的编号.
 * @param info 记录信号发送进程信息的结构体.
 * @param context 可以赋给指向 ucontext_t 类型的一个对象的指针
 * 以引用在传递信号时被中断的接收进程或线程的上下文.
 */
void(*sa_sigaction)(int signum, siginfo_t *info, void *context);
```

13、不可重入、可重入函数

如果有一个函数不幸被设计成为这样：那么不同任务调用这个函数时可能修改其他任务调用这个函数的数据，从而导致不可预料的后果。

这样的函数是不安全的函数，也叫不可重入函数;

（1）不可重入函数:

1. 函数体内使用了静态的数据结构;
2. 函数体内调用了malloc() 或者 free() 函数(谨慎使用堆);
3. 函数体内调用了标准 I/O 函数;

（2）可重入函数:

1. 所谓可重入是指一个可以被多个任务调用的过程，任务在调用时不必担心数据是否会出错;
2. 在写函数时候尽量使用局部变量（例如寄存器、栈中的变量）；
3. 对于要使用的全局变量要加以保护（如采取关中断、信号量等互斥方法），这样构成的函数就一定是一个可重入的函数.

14、SIGCHLD信号

1. 子进程终止时;
2. 子进程接收到SIGSTOP信号停止时;
3. 子进程处在停止态，接受到SIGCONT后唤醒时。

15、如何避免僵尸进程

（1）最简单的方法:

父进程通过 wait() 和 waitpid() 等函数等待子进程结束，但是，这会导致父进程挂起;

（2）如果父进程要处理的事情很多，不能够挂起，通过 signal() 函数人为处理信号 SIGCHLD:

只要有子进程退出自动调用指定好的回调函数，因为子进程结束后， 父进程会收到该信号 SIGCHLD ，可以在其回调函数里调用 wait() 或 waitpid() 回收;

（3）如果父进程不关心子进程什么时候结束，那么可以用signal（SIGCHLD, SIG_IGN）通知内核:

自己对子进程的结束不感兴趣，父进程忽略此信号，那么子进程结束后，内核会回收，并不再给父进程发送信号;

守护进程

1、进程组概述

1. 代表一个或多个进程的集合;
2. 每个进程都属于一个进程组;
3. 是为了简化对多个进程的管理。

2、会话

1. 一个会话可以有一个控制终端。这通常是终端设备或伪终端设备;
2. 建立与控制终端连接的会话首进程被称为控制进程;
3. 一个会话中的几个进程组可被分为一个前台进程组以及一个或多个后台进程组;
4. 如果一个会话有一个控制终端，则它有一个前台进程组，其它进程组为后台进程组;
5. 如果终端接口检测到断开连接，则将挂断信号发送至控制进程（会话首进程。

3、创建会话注意事项

1. 调用进程不能是进程组组长，该进程变成新会话首进程(session header);
2. 该调用进程是组长进程，则出错返回;
3. 该进程成为一个新进程组的组长进程;
4. 需有root权限(ubuntu不需要);
5. 新会话丢弃原有的控制终端，该会话没有控制终端;
6. 建立新会话时，先调用fork, 父进程终止，子进程调用setsid。

4、API函数

（1）getsid 函数

```
#include <unistd.h>
/**
 * 获取进程所属的会话ID.
 * @param pid 进程号，pid为0表示查看当前进程session ID.
 * @return 成功：返回调用进程的会话ID; 失败：-1.
 */
pid_t getsid(pid_t pid);
```

（2）setsid函数

```
#include <unistd.h>
/**
 * 创建一个会话，并以自己的ID设置进程组ID，同时也是新会话的ID。
 * 调用了setsid函数的进程，既是新的会长，也是新的组长.
 *
 * @return 成功：  返回调用进程的会话ID; 失败: -1.
 */
pid_t setsid(void);
```

5、守护进程

守护进程（Daemon Process），也就是通常说的 Daemon 进程（精灵进程），是 Linux 中的后台服务进程

1. 它是一个生存期较长的进程，通常独立于控制终端并且周期性地执行某种任务或等待处理某些发生的事件
2. 一般采用以d结尾的名字
3. 所有的服务存在于 etc/init.d
4. 守护进程是个特殊的孤儿进程
5. 之所以脱离于终端是为了避免进程被任何终端所产生的信息所打断，其在执行过程中的信息也不在任何终端上显示
6. Linux 的大多数服务器就是用守护进程实现的

6、守护进程模型

（1）创建子进程，父进程退出(必须)

所有工作在子进程中进行形式上脱离了控制终端

（2）在子进程中创建新会话(必须)

1. setsid()函数
2. 使子进程完全独立出来，脱离控制

（3）改变当前目录为根目录(不是必须)

1. chdir()函数
2. 防止占用可卸载的文件系统
3. 也可以换成其它路径

（4）重设文件权限掩码(不是必须)

1. umask()函数
2. 防止继承的文件创建屏蔽字拒绝某些权限
3. 增加守护进程灵活性

（5）关闭文件描述符(不是必须)

继承的打开文件不会用到，浪费系统资源，无法卸载

（6）开始执行守护进程核心工作(必须)

守护进程退出处理程序模型

互斥与同步

互斥锁Mutex

互斥锁（mutex）：

也叫互斥量，互斥锁是一种简单的加锁的方法来控制对共享资源的访问，互斥锁只有两种状态,即加锁(lock)和解锁(unlock)

1. 在访问共享资源后临界区域前，对互斥锁进行加锁。
2. 在访问完成后释放互斥锁导上的锁。
3. 对互斥锁进行加锁后，任何其他试图再次对互斥锁加锁的线程将会被阻塞，直到锁被释放。

互斥锁的数据类型是： pthread_mutex_t

1. 创建互斥锁

pthread_mutex_init()

```
#include <pthread.h>
/**
 * 初始化一个互斥锁.
 * @param mutex 互斥锁地址。类型是 pthread_mutex_t.
 * @param attr 设置互斥量的属性，通常可采用默认属性，即可将 attr 设为 NULL.
 * @return 成功：0 成功申请的锁默认是打开的; 失败：非0(错误码).
 */
int pthread_mutex_init(pthread_mutex_t *restrict mutex,
const pthread_mutexattr_t *restrict attr);
// 这种方法等价于使用 NULL 指定的 attr 参数调用 pthread_mutex_init() 来完成动态初始化，
// 不同之处在于 PTHREAD_MUTEX_INITIALIZER 宏不进行错误检查。
```

2. 销毁互斥锁

pthread_mutex_destroy()

```
#include <pthread.h>
/**
 * 销毁指定的一个互斥锁。互斥锁在使用完毕后，必须要对互斥锁进行销毁，以释放资源.
 * @param mutex 互斥锁地址。类型是 pthread_mutex_t.
 * @return 成功：0; 失败：非0(错误码).
 */
int pthread_mutex_destroy(pthread_mutex_t *mutex);
```

3. 互斥锁上锁

pthread_mutex_lock()

```c
#include <pthread.h>
/**
 * 对互斥锁上锁，若互斥锁已经上锁，则调用者阻塞，直到互斥锁解锁后再上锁.
 * @param mutex 互斥锁地址.
 * @return 成功: 0; 失败: 非0(错误码).
 */
int pthread_mutex_lock(pthread_mutex_t *mutex);

/**
 * 调用该函数时，若互斥锁未加锁，则上锁，返回 0;
 * 若互斥锁已加锁，则函数直接返回失败，即 EBUSY.
 */
int pthread_mutex_trylock(pthread_mutex_t *mutex);
```

4. 互斥锁解锁

pthread_mutex_unlock()

```c
#include <pthread.h>
/**
 * 对指定的互斥锁解锁.
 * @param mutex 互斥锁地址.
 * @return 成功: 0; 失败: 非0(错误码)
 */
int pthread_mutex_unlock(pthread_mutex_t *mutex);
```

死锁（DeadLock）

如果一个进程集合中的每一个进程都在等待只能由该进程集合中的其他进程才能引发的事件，那么，该进程集合就是死锁

1. 资源

1、可抢占资源

可以从拥有它的进程中抢占而不会产生任何副作用，存储器就是一类可抢占资源

2、不可抢占资源

是指在不引起相关计算失败的情况下，无法把它从占有它的进程处抢占过来

2. 必要条件

1、互斥

每个资源要么已经分配给一个进程，要么就是可用的

2、占有和等待

已经得到了某个资源的进程可以再请求新的资源

3、不可抢占

已经分配给一个进程的资源不能强制性地被抢占，它只能被占有它的进程显式地释放

4、环路等待

死锁发生时，系统中一定有由两个或两个以上的进程组成的一条环路，该环路中的每个进程都在等待着下一个进程所占有的资源。

3. 处理方法

1、鸵鸟算法

把头埋在沙子里，假装根本没发生问题。

因为解决死锁问题的代价很高，因此鸵鸟算法这种不采取任务措施的方案会获得更高的性能。

当发生**死锁时不会对用户造成多大影响**，或发生死锁的概率很低，可以采用鸵鸟算法。

2、死锁检测与死锁恢复

（1）每种类型一个资源的死锁检测

检测算法：

通过检测有向图中是否存在环来实现，从一个节点出发进行深度优先搜索，对访问过的节点进行标记，如果访问了已经标记的节点，就表示有向图存在环，也就是检测到死锁发生

（2）每种类型多个资源的死锁检测

锁检测算法如下：

每个进程最开始时都不被标记，执行过程有可能被标记。当算法结束时，任何没有被标记的进程都是死锁进程。

1. 寻找一个没有标记的进程Pi，它所请求的资源小于或等于A
2. 如果真找到这样一个进程，那么将C矩阵的第i行向量加到A中，标记该进程，并转回第1步
3. 如果没有这样的进程，那么算法终止

3、从死锁中恢复

（1）利用抢占恢复

将进程挂起，强行取走资源给另一个进程使用，用完再放回

（2）利用回滚恢复

复位到更早的状态，那时它还没有取得所需的资源

（3）通过杀死进程恢复

杀掉环中的一个进程或多个，牺牲掉一个环外进程

4、死锁预防

（1）破坏互斥条件

例如假脱机打印机技术允许若干个进程同时输出，唯一真正请求物理打印机的进程是打印机守护进程。

（2）破坏占有个等待条件

1. 规定所有进程在开始执行前请求所需要的全部资源。
2. 要求当一个进程请求资源时，先暂时释放其当前占用的所有资源，然后在尝试一次获得所需的全部资源。

（3）破坏不可抢占条件

1. 保证每一个进程在任何时刻只能占用一个资源，如果请求另一个资源必须先释放第一个资源
2. 将所有的资源统一编号，进程可以在任何时刻提出资源请求，但是所有请求必须按照资源编号的顺序(升序)提出

（4）破坏环路等待

5、死锁避免

（1）安全状态

如果没有死锁发生，并且即使所有进程突然请求对资源的最大需求，也仍然存在某种调度次序**能够使得每一个进程运行完毕**，则称该状态是安全的。

（2）单个资源的银行家算法

一个小城镇的银行家，他向一群客户分别承诺了一定的贷款额度，算法要做的是判断对**请求的满足是否会进入不安全状态**，如果是，就拒绝请求；否则予以分配。

（3）多个资源的银行家算法

检查一个**状态是否安全**的算法

1. 查找右边的矩阵是否存在一行小于等于向量 A。如果不存在这样的行，那么系统将会发生死锁，状态是不安全的。
2. 假若找到这样一行，将该进程标记为终止，并将其已分配资源加到 A 中。
3. 重复以上两步，直到所有进程都标记为终止，则状态时安全的。

如果一个状态不是安全的，需要拒绝进入这个状态。

读写锁

在对数据的读写操作中，更多的是读操作，写操作较少，例如对数据库数据的读写应用。

为了满足当前能够允许多个读出，但只允许一个写入的需求，线程提供了读写锁来实现。

读写锁的特点

1、如果有其它线程读数据，则允许其它线程执行读操作，但不允许写操作

2、如果有其它线程写数据，则其它线程都不允许读、写操作

读写锁分为读锁和写锁，规则如下

1. 如果某线程申请了读锁，其它线程可以再申请读锁，但不能申请写锁。
2. 如果某线程申请了写锁，其它线程不能申请读锁，也不能申请写锁。

POSIX 定义的读写锁的数据类型是： pthread_rwlock_t。

1. 初始化读写锁

pthread_rwlock_init()

```
/**
 * 用来初始化 rwlock 所指向的读写锁.
 * @param rwlock 指向要初始化的读写锁指针.
 * @param attr: 读写锁的属性指针。如果 attr 为 NULL 则会使用默认的属性初始化读写锁，否则使用指定的
attr 初始化读写锁.
 * 可以使用宏 PTHREAD_RWLOCK_INITIALIZER 静态初始化读写锁，比如：
 * pthread_rwlock_t my_rwlock = PTHREAD_RWLOCK_INITIALIZER;
 * 这种方法等价于使用 NULL 指定的 attr 参数调用 pthread_rwlock_init() 来完成动态初始化，
 * 不同之处在于PTHREAD_RWLOCK_INITIALIZER 宏不进行错误检查.
 * @return 成功：0，读写锁的状态将成为已初始化和已解锁；失败：非 0 错误码.
 */
#include <pthread.h>
int pthread_rwlock_init(pthread_rwlock_t *restrict rwlock,
const pthread_rwlockattr_t *restrict attr);
```

2. 销毁读写锁

pthread_rwlock_destroy()

```
/**
 * 用于销毁一个读写锁，并释放所有相关联的资源（所谓的所有指的是由 pthread_rwlock_init() 自动申请的资
源.
 * @param rwlock 读写锁指针.
 * @return 成功：0；失败：非0错误码.
 */
#include <pthread.h>
int pthread_rwlock_destroy(pthread_rwlock_t *rwlock);
```

3. 读锁定

pthread_rwlock_rdlock()

```
#include <pthread.h>
/**
 * 以阻塞方式在读写锁上获取读锁（读锁定）.
 * 如果没有写者持有该锁，并且没有写者阻塞在该锁上，则调用线程会获取读锁.
 * 如果调用线程未获取读锁，则它将阻塞直到它获取了该锁。一个线程可以在一个读写锁上多次执行读锁定.
 * 线程可以成功调用 pthread_rwlock_rdlock() 函数 n 次，但是之后该线程必须调用
pthread_rwlock_unlock() 函数 n 次才能解除锁定.
 * @param rwlock 读写锁指针.
 * @return 成功：0；失败：非0错误码.
 */
int pthread_rwlock_rdlock(pthread_rwlock_t *rwlock);

/**
 * 用于尝试以非阻塞的方式来在读写锁上获取读锁.
 * 如果有任何的写者持有该锁或有写者阻塞在该读写锁上，则立即失败返回.
 */
int pthread_rwlock_tryrdlock(pthread_rwlock_t *rwlock);
```

4. 写锁定

pthread_rwlock_wrlock()

```
#include <pthread.h>
/**
 * 在读写锁上获取写锁（写锁定）.
 * 如果没有写者持有该锁，并且没有写者读者持有该锁，则调用线程会获取写锁.
 * 如果调用线程未获取写锁，则它将阻塞直到它获取了该锁.
 * @param rwlock 读写锁指针.
 * @return 成功: 0; 失败: 非0错误码.
 */
int pthread_rwlock_wrlock(pthread_rwlock_t *rwlock);

/**
 * 用于尝试以非阻塞的方式来在读写锁上获取写锁.
 * 如果有任何的读者或写者持有该锁，则立即失败返回.
 */
int pthread_rwlock_trywrlock(pthread_rwlock_t *rwlock);
```

5. 解锁

pthread_rwlock_unlock()

```
#include <pthread.h>
/**
 * 无论是读锁或写锁，都可以通过此函数解锁.
 * @param rwlock 读写锁指针.
 * @return 成功: 0; 失败: 非0错误码.
 */
int pthread_rwlock_unlock(pthread_rwlock_t *rwlock);
```

条件变量

与互斥锁不同，条件变量是用来等待而不是用来上锁的，条件变量本身不是锁!

条件变量用来自动阻塞一个线程，直到某特殊情况发生为止。通常条件变量和互斥锁同时使用。

条件变量的两个动作:

1. 条件不满, 阻塞线程
2. 当条件满足, 通知阻塞的线程开始工作

条件变量的类型: **pthread_cond_t**

1. 创建条件变量

pthread_cond_init()

```
#include <pthread.h>
/**
 * 初始化一个条件变量.
 * @param cond 指向要初始化的条件变量指针.
 * @param attr 条件变量属性，通常为默认值，传NULL即可.
 *              也可以使用静态初始化的方法，初始化条件变量;
 *              pthread_cond_t cond = PTHREAD_COND_INITIALIZER.
 * return 成功: 0; 失败: 非0错误号.
 */
int pthread_cond_init(pthread_cond_t *restrict cond,
const pthread_condattr_t *restrict attr);
```

2. 删除条件变量

pthread_cond_destroy()

```
#include <pthread.h>
/**
 * 销毁一个条件变量.
 * @param cond 指向要初始化的条件变量指针.
 * @return  成功: 0; 失败: 非0错误号.
 */
int pthread_cond_destroy(pthread_cond_t *cond);
```

3. 唤醒线程

pthread_cond_signal()

```
#include <pthread.h>
/**
 * 唤醒至少一个阻塞在条件变量上的线程.
 * @param cond 指向要初始化的条件变量指针.
 * @return  成功: 0; 失败: 非0错误号.
 */
int pthread_cond_signal(pthread_cond_t *cond);

/**
 * 唤醒全部阻塞在条件变量上的线程.
 * @param cond 指向要初始化的条件变量指针.
 * @return  成功: 0; 失败: 非0错误号.
 */
int pthread_cond_broadcast(pthread_cond_t *cond);
```

4. 阻塞线程

pthread_cond_wait()

```
#include <pthread.h>
struct timespec {
    time_t tv_sec;      /* seconds */ // 秒
    long   tv_nsec; /* nanosecondes*/ // 纳秒
}
// time_t cur = time(NULL);         //获取当前时间。
// struct timespec t;               //定义timespec 结构体变量t
// t.tv_sec = cur + 1;              // 定时1秒
// pthread_cond_timedwait(&cond, &t);
/**
 * 阻塞等待一个条件变量
 *      1. 阻塞等待条件变量cond（参1）满足
 *      2. 释放已掌握的互斥锁（解锁互斥量）相当于pthread_mutex_unlock(&mutex);
 *      ((1), (2)两步为一个原子操作)
 *      3. 当被唤醒，pthread_cond_wait函数返回时，解除阻塞并重新申请获取互斥锁
pthread_mutex_lock(&mutex);
 * @param cond 指向要初始化的条件变量指针.
 * @param mutex 互斥锁.
 * @return  成功: 0; 失败: 非0错误号.
 */
int pthread_cond_wait(pthread_cond_t *restrict cond,
    pthread_mutex_t *restrict mutex);

/**
 * 限时等待一个条件变量.
 * @param cond 指向要初始化的条件变量指针.
 * @param mutex 互斥锁.
 * @param abstime 绝对时间.
 * @return  成功: 0; 失败: 非0错误号.
 */
int pthread_cond_timedwait(pthread_cond_t *restrict cond,
    pthread_mutex_t *restrict mutex,
    const struct *restrict abstime);
```

5. 条件变量流程分析

1、条件变量的优缺点

相较于mutex而言，**条件变量可以减少竞争**。

如直接使用mutex，除了生产者、消费者之间要竞争互斥量以外，消费者之间也需要竞争互斥量;

但如果汇聚（链表）中没有数据，消费者之间竞争互斥锁是无意义的。
有了条件变量机制以后，只有生产者完成生产，才会引起消费者之间的竞争。提高了程序效率。

2、条件变量流程分析

场景: 你是个老板，招聘了三个员工，但是你不是有了活才去招聘员工，而是先把员工招来，没有活的时候员工需要在那里等着，一旦有了活，你要去通知他们，他们要去抢活干，干完了再等待，你再有活，再通知他们

代码如下：

```
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>
#define NUM_OF_TASKS 3
#define MAX_TASK_QUEUE 11
char tasklist[MAX_TASK_QUEUE]="ABCDEFGHIJ";
int head = 0;
int tail = 0;
int quit = 0;
pthread_mutex_t g_task_lock;
pthread_cond_t g_task_cv;

void *coder(void *notused)
{
    pthread_t tid = pthread_self();
    while(!quit){
        pthread_mutex_lock(&g_task_lock);
        while(tail == head){
            if(quit){
                pthread_mutex_unlock(&g_task_lock);
                pthread_exit((void *)0);
            }
            printf("No task now! Thread %u is waiting!\n", (unsigned int)tid);
            pthread_cond_wait(&g_task_cv, &g_task_lock);
            printf("Have task now! Thread %u is grabing the task !\n", (unsigned
int)tid);
        }
        char task = tasklist[head++];
        pthread_mutex_unlock(&g_task_lock);
        printf("Thread %u has a task %c now!\n", (unsigned int)tid, task);
        sleep(5);
        printf("Thread %u finish the task %c!\n", (unsigned int)tid, task);
    }
    pthread_exit((void *)0);
}

int main(int argc, char *argv[])
{
    pthread_t threads[NUM_OF_TASKS];
    int rc;
    int t;
    pthread_mutex_init(&g_task_lock, NULL);
    pthread_cond_init(&g_task_cv, NULL);
    for(t=0;t<NUM_OF_TASKS;t++){
```

```
        rc = pthread_create(&threads[t], NULL, coder, NULL);
        if (rc){
            printf("ERROR; return code from pthread_create() is %d\n", rc);
            exit(-1);
        }
    }
    sleep(5);
    for (t=1;t<=4;t++) {
        pthread_mutex_lock(&g_task_lock);
        tail+=t;
        printf("I am Boss, I assigned %d tasks, I notify all coders!\n", t);
        pthread_cond_broadcast(&g_task_cv);
        pthread_mutex_unlock(&g_task_lock);
        sleep(20);
    }
    pthread_mutex_lock(&g_task_lock);
    quit = 1;
    pthread_cond_broadcast(&g_task_cv);
    pthread_mutex_unlock(&g_task_lock);
    pthread_mutex_destroy(&g_task_lock);
    pthread_cond_destroy(&g_task_cv);
    pthread_exit(NULL);
}
```

下面是对于每个输出的分析：

```
No task now! Thread 1461831424 is waiting!
No task now! Thread 1453438720 is waiting!
No task now! Thread 1445046016 is waiting!
```

这是主线程新建的三个工作线程，这时候工作线程都是先获得锁，判断tail==head，阻塞，cond_wait释放了锁，三个工作线程都等待信号来唤醒

```
I am Boss, I assigned 1 tasks, I notify all coders!
```

主线程增加了一个任务，引起了head!=tail，这时候broadcast唤醒了所有工作线程，之后主线程释放锁，开始睡眠20s

```
Have task now! Thread 1453438720 is grabing the task !
```

1工作线程在竞争中抢到了锁，退出了while(tail==head)循环，并开始完成A任务，这时候把头加1，释放锁

```
Have task now! Thread 1445046016 is grabing the task !
No task now! Thread 1445046016 is waiting!
```

1工作线程时间片到了，2，3工作线程对锁进行竞争，2得到了锁，但是这时候已经没有工作了。while(tail==head)循环没有退出，因此释放锁，等待唤醒

```
Thread 1453438720 has a task A now!
```

调度程序调度到1线程执行，打印了这句话，开始睡眠5s

```
Have task now! Thread 1461831424 is grabing the task !
No task now! Thread 1461831424 is waiting!
```

3工作线程抢到了锁（虽然这时候只有它在抢锁），3得到了锁，但是这时候已经没有工作了。while(tail==head)循环没有退出，因此释放锁，等待唤醒

```
Thread 1453438720 finish the task A!
No task now! Thread 1453438720 is waiting!
```

20工作线程执行完任务，这时候还是没有工作。先上锁，后进入while(tail==head)循环没有退出，因此释放锁，等待唤醒

```
I am Boss, I assigned 2 tasks, I notify all coders!
```

主线程醒了，又找了两个工作B,C，唤醒了所有工作线程去做

```
Have task now! Thread 1461831424 is grabing the task !
Thread 1461831424 has a task B now!
```

24号线程抢到了锁，执行头加1，释放锁，然后睡眠了

```
Have task now! Thread 1445046016 is grabing the task !
Thread 1445046016 has a task C now!
```

16号线程抢到了锁，执行头加1，释放锁，然后睡眠了

```
Have task now! Thread 1453438720 is grabing the task !
No task now! Thread 1453438720 is waiting!
```

20号线程抢到了锁，没任务了，就释放锁，等待唤醒

```
Thread 1445046016 finish the task C!
No task now! Thread 1445046016 is waiting!
```

调度16号线程执行，16号线程完成任务，得到锁，发现没任务，释放锁，等待唤醒

```
Thread 1461831424 finish the task B!
No task now! Thread 1461831424 is waiting!
```

调度24号线程执行，24号线程完成任务，得到锁，发现没任务，释放锁，等待唤醒

```
I am Boss, I assigned 3 tasks, I notify all coders!
```

主线程醒了，又找了三个个工作D,E,F，唤醒了所有工作线程去做

```
Have task now! Thread 1461831424 is grabing the task !
Thread 1461831424 has a task D now!
```

24号线程抢到了锁，执行头加1，释放锁，然后睡眠了

```
Have task now! Thread 1445046016 is grabing the task !
Thread 1445046016 has a task E now!
```

16号线程抢到了锁，执行头加1，释放锁，然后睡眠了

```
Have task now! Thread 1453438720 is grabing the task !
Thread 1453438720 has a task F now!
```

20号线程抢到了锁，执行头加1，释放锁，然后睡眠了

```
Thread 1461831424 finish the task D!
No task now! Thread 1461831424 is waiting!
```

调度24号线程执行，24号线程完成任务，得到锁，发现没任务，释放锁，等待唤醒

```
Thread 1445046016 finish the task E!
No task now! Thread 1445046016 is waiting!
```

调度16号线程执行，16号线程完成任务，得到锁，发现没任务，释放锁，等待唤醒

```
Thread 1453438720 finish the task F!
No task now! Thread 1453438720 is waiting!
```

调度20号线程执行，20号线程完成任务，得到锁，发现没任务，释放锁，等待唤醒

```
I am Boss, I assigned 4 tasks, I notify all coders!
```

主线程醒了，又找了三个个工作G,H,I,J，唤醒了所有工作线程去做

```
Have task now! Thread 1453438720 is grabing the task !
Thread 1453438720 has a task G now!
```

20号线程抢到了锁，执行头加1，释放锁，然后睡眠了

```
Have task now! Thread 1461831424 is grabing the task !
Thread 1461831424 has a task H now!
```

24号线程抢到了锁，执行头加1，释放锁，然后睡眠了

```
Have task now! Thread 1445046016 is grabing the task !
Thread 1445046016 has a task I now!
```

16号线程抢到了锁，执行头加1，释放锁，然后睡眠了

```
Thread 1453438720 finish the task G!
Thread 1453438720 has a task J now!
```

20号线程执行完任务，下一轮while中，持有了锁，发现还有任务，就继续做J，执行头加1，释放了锁，开始睡眠5s

```
Thread 1461831424 finish the task H!
No task now! Thread 1461831424 is waiting!
```

调度24号线程执行，24号线程完成任务，得到锁，发现没任务，释放锁，等待唤醒

```
Thread 1445046016 finish the task I!
No task now! Thread 1445046016 is waiting!
```

调度16号线程执行，16号线程完成任务，得到锁，发现没任务，释放锁，等待唤醒

```
Thread 1453438720 finish the task J!
No task now! Thread 1453438720 is waiting!
```

20线程完成任务，下一轮while中持有锁，发现没任务了，释放锁，等待唤醒
主线程抢锁，设置退出标志quit为1，并通知三个工作线程

```
Have task now! Thread 1461831424 is grabing the task !
Have task now! Thread 1445046016 is grabing the task !
Have task now! Thread 1453438720 is grabing the task !
```

三个工作线程分别抢到了锁，发现quit为1，就释放锁，然后终止各自的线程
主线程销毁了锁和条件变量
终止了main线程

信号量

信号量广泛**用于进程或线程间的同步和互斥**，信号量本质上是一个非负的整数计数器，它被用来控制对公共资源的访问。

编程时可根据操作信号量值的结果判断是否对公共资源具有访问的权限，当信号量值大于 0 时，则可以访问，否则将阻塞。

PV 原语是对信号量的操作，一次 P 操作使信号量减 1 ，一次 V 操作使信号量加 1 。

信号量数据类型为：sem_t

1. 创建信号量

sem_init()

```
#include <semaphore.h>
/**
 * 创建一个信号量并初始化它的值。一个无名信号量在被使用前必须先初始化.
 * @param sem 信号量的地址.
 * @param pshared 等于 0，信号量在线程间共享（常用）；不等于0，信号量在进程间共享.
 * @param value 信号量的初始值.
 * @return 成功：0；失败：-1.
 */
int sem_init(sem_t *sem, int pshared, unsigned int value);
```

2. 删除信号量

sem_destroy()

```
#include <semaphore.h>
/**
 * 删除 sem 标识的信号量.
 * @return 成功：0；失败：-1.
 */
int sem_destroy(sem_t *sem);
```

3. P操作

sem_wait()(P操作)

```
#include <semaphore.h>
/**
 * 将信号量的值减 1。操作前，先检查信号量（sem）的值是否为 0；
 * 若信号量为 0，此函数会阻塞，直到信号量大于 0 时才进行减 1 操作。
 * @param sem 信号量的地址.
 * @return 成功：0；失败：-1.
 */
int sem_wait(sem_t *sem);

/**
```

```
 * 以非阻塞的方式来对信号量进行减 1 操作。
 * 若操作前，信号量的值等于 0，则对信号量的操作失败，函数立即返回。
 */
int sem_trywait(sem_t *sem);

/**
 * 限时尝试将信号量的值减 1
 * abs_timeout：绝对时间
 */
int sem_timedwait(sem_t *sem, const struct timespec *abs_timeout);
```

4. V操作

sem_post()(V操作)

```
#include <semaphore.h>
/**
 * 将信号量的值加 1 并发出信号唤醒等待线程(sem_wait()).
 * @param sem 信号量的地址.
 * @return 成功：0；失败：-1.
 */
int sem_post(sem_t *sem);
```

5. 获取信号量的值

sem_getvalue()

```
#include <semaphore.h>
/**
 * 获取 sem 标识的信号量的值，保存在 sval 中.
 * @param sem 信号量的地址.
 * @param sval 保存信号量值的地址.
 * @return 成功：0；失败：-1.
 */
int sem_getvalue(sem_t *sem, int *sval);\
```

管程

使用信号量机制实现的生产者消费者问题需要客户端代码做很多控制，
而管程把控制的代码独立出来，不仅不容易出错，也使得客户端代码调用更容易。

下面是经典的IPC问题

1. 哲学家进餐问题

下面是一种错误的解法，如果所有哲学家同时拿起左手边的筷子，那么所有哲学家**都在等待其它哲学家吃完并释放**自己手中的筷子，导致死锁。

```
#define N 5

void philosopher(int i) {
    while(TRUE) {
        think();
        take(i);        // 拿起左边的筷子
        take((i+1)%N); // 拿起右边的筷子
        eat();
        put(i);
        put((i+1)%N);
    }
}
```

为了防止死锁的发生，可以设置两个条件：

1. 必须同时拿起左右两根筷子
2. 只有在两个邻居都没有进餐的情况下才允许进餐

```
#define N 5
#define LEFT (i + N - 1) % N // 左邻居
#define RIGHT (i + 1) % N    // 右邻居
#define THINKING 0
#define HUNGRY   1
#define EATING   2
typedef int semaphore;
int state[N];                // 跟踪每个哲学家的状态
semaphore mutex = 1;         // 临界区的互斥，临界区是 state 数组，对其修改需要互斥
semaphore s[N];              // 每个哲学家一个信号量

void philosopher(int i) {
    while(TRUE) {
        think(i);
        take_two(i);
        eat(i);
        put_two(i);
    }
}

void take_two(int i) {
    down(&mutex);
    state[i] = HUNGRY;
    check(i);
    up(&mutex);
    down(&s[i]); // 只有收到通知之后才可以开始吃，否则会一直等下去
}

void put_two(i) {
    down(&mutex);
    state[i] = THINKING;
```

```
    check(LEFT); // 尝试通知左右邻居，自己吃完了，你们可以开始吃了
    check(RIGHT);
    up(&mutex);
}

void eat(int i) {
    down(&mutex);
    state[i] = EATING;
    up(&mutex);
}

// 检查两个邻居是否都没有用餐，如果是的话，就 up(&s[i])，使得 down(&s[i]) 能够得到通知并继续执行
void check(i) {
    if(state[i] == HUNGRY && state[LEFT] != EATING && state[RIGHT] !=EATING) {
        state[i] = EATING;
        up(&s[i]);
    }
}
```

2. 读者-写者问题

允许多个进程同时对数据进行读操作，但是不允许读和写以及写和写操作同时发生。

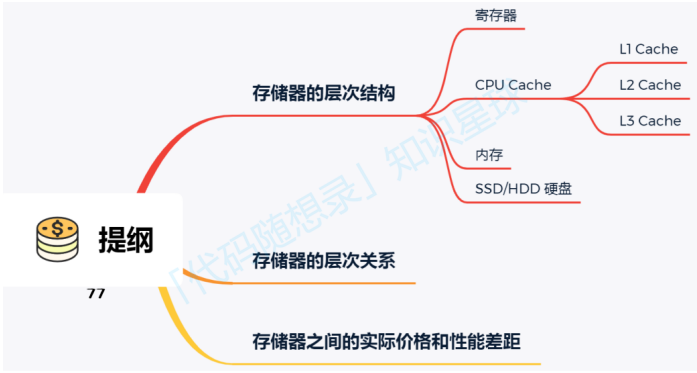
一个整型变量 count 记录在对数据进行读操作的进程数量，一个互斥量 count_mutex 用于对 count 加锁，一个互斥量 data_mutex 用于对读写的数据加锁。

```
typedef int semaphore;
semaphore count_mutex = 1;
semaphore data_mutex = 1;
int count = 0;

void reader() {
    while(TRUE) {
        down(&count_mutex);
        count++;
        if(count == 1) down(&data_mutex); // 第一个读者需要对数据进行加锁，防止写进程访问
        up(&count_mutex);
        read();
        down(&count_mutex);
        count--;
        if(count == 0) up(&data_mutex);
        up(&count_mutex);
    }
}

void writer() {
    while(TRUE) {
        down(&data_mutex);
        write();
        up(&data_mutex);
```

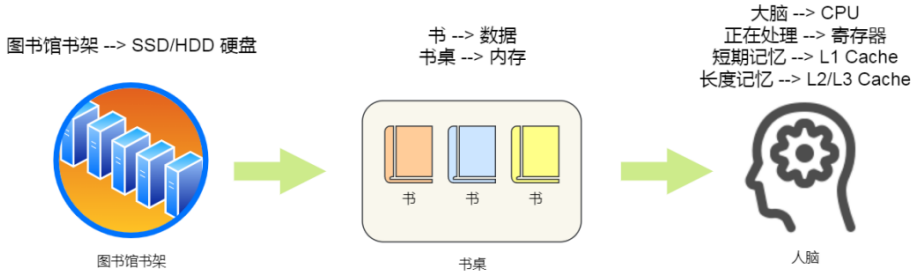
```
    }
}
```

存储系统

存储系统层级



1、存储器的层次结构



我们从图书馆书架取书，把书放到桌子上，再阅读书，我们大脑就会记忆知识点，然后再经过大脑思考，这一系列过程相当于，数据从硬盘加载到内存，再从内存加载到 CPU 的寄存器和 Cache 中，然后再通过 CPU 进行处理和计算。

对于存储器，它的速度越快、能耗会越高、而且材料的成本也是越贵的，以至于速度快的存储器的容量都比较小。

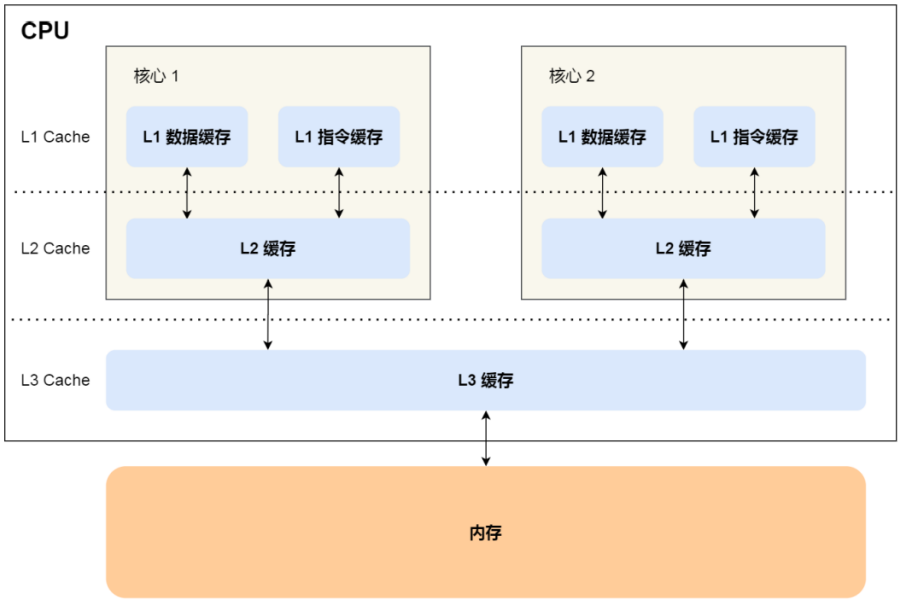
（1）寄存器

寄存器的访问速度非常快，一般要求在半个 CPU 时钟周期内完成读写，CPU 时钟周期跟 CPU 主频息息相关，比如 2 GHz 主频的 CPU，那么它的时钟周期就是 1/2G，也就是 0.5ns（纳秒）。

（2）CPU Cache（用的是一种叫 SRAM（Static Random-Access Memory，静态随机存储器）的芯片）

SRAM 之所以叫「静态」存储器，是因为只要有电，数据就可以保持存在，而一旦断电，数据就会丢失了。

CPU Cache 通常分为三层，如下图所示：



查看各级Cache的大小：

```
(base)  ~$ cat /sys/devices/system/cpu/cpu0/cache/index0/size
32K
(base) jiaoqq@ubuntu-GPU:~$ cat /sys/devices/system/cpu/cpu0/cache/index1/size
32K
(base) jiaoqq@ubuntu-GPU:~$ cat /sys/devices/system/cpu/cpu0/cache/index2/size
1024K
(base) jiaoqq@ubuntu-GPU:~$ cat /sys/devices/system/cpu/cpu0/cache/index3/size
14080K
```

（3）内存

内存用的芯片和 CPU Cache 有所不同，它使用的是一种叫作 DRAM （Dynamic Random Access Memory，动态随机存取存储器） 的芯片。相比 SRAM，DRAM 的密度更高，功耗更低，有更大的容量，而且造价比 SRAM 芯片便宜很多。

DRAM 存储一个 bit 数据，只需要一个晶体管和一个电容就能存储，但是因为数据会被存储在电容里，电容会不断漏电，所以需要「定时刷新」电容，才能保证数据不会被丢失，这就是 DRAM 之所以被称为「动态」存储器的原因，只有不断刷新，数据才能被存储起来。

DRAM 的数据访问电路和刷新电路都比 SRAM 更复杂，所以访问的速度会更慢，内存速度大概在 200~300 个 时钟周期之间。

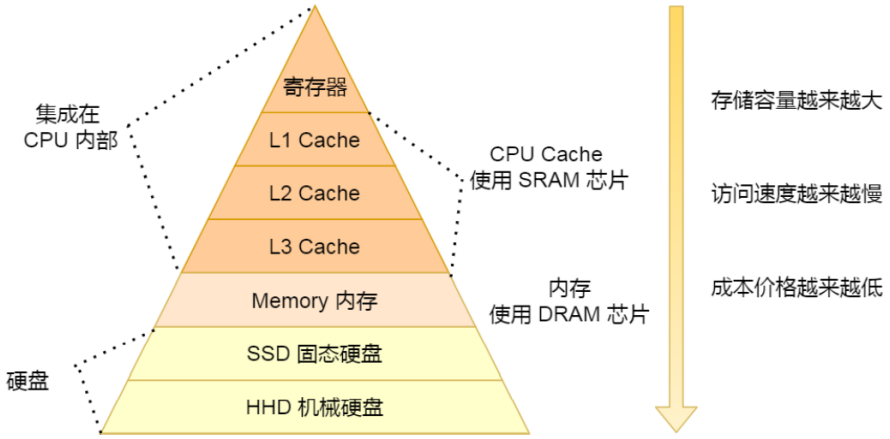
（4）SSD/HDD硬盘:

SSD（Solid-state disk） 就是我们常说的固体硬盘，结构和内存类似，但是它相比内存的优点是**断电后数据还是存在**的，而**内存、寄存器、高速缓存断电后数据都会丢失**。内存的读写速度比 SSD 大概快 10~1000 倍。

机械硬盘（Hard Disk Drive, HDD），它是通过物理读写的方式来访问 数据的，因此它访问速度是非常慢的，它的速度比内存慢 10W 倍左右。

2、存储器的层次关系

CPU 并不会直接和每一种存储器设备直接打交道，而是每一种存储器设备只和它相邻的存储器设备打交道。



所以，每个存储器只和相邻的一层存储器设备打交道，并且存储设备为了追求更快的速度，所需的材料成本必然也是更高，也正因为成本太高，所以 CPU 内部的寄存器、L1\L2\L3 Cache 只好用较小的容量，相反内存、硬盘则可用更大的容量，这就我们今天所说的存储器层次结构。

3、存储器之间的实际价格和性能差距

不同层级的存储器之间的成本对比图：

存储器	硬件介质	单位成本（美元/MB）	随机访问延时
L1 Cache	SRAM	7	1ns
L2 Cache	SRAM	7	4ns
Memory	DRAM	0.015	100ns
Disk	SSD（NAND）	0.0004	150μs
Disk	HDD	0.00004	10ms

机械硬盘（HDD)到底有多慢：

- **SSD 比机械硬盘快 70 倍左右；**
- **内存比机械硬盘快 100000 倍左右；**
- **CPU L1 Cache 比机械硬盘快 10000000 倍左右；**

虚拟内存

虚拟内存是计算机操作系统中的一种内存管理技术，它允许程序访问比物理内存（RAM）更大的地址空间，但首先我们要区分这两个概念：

虚拟内存地址：程序所使用的内存地址

物理内存地址：实际存在硬件里面的空间地址

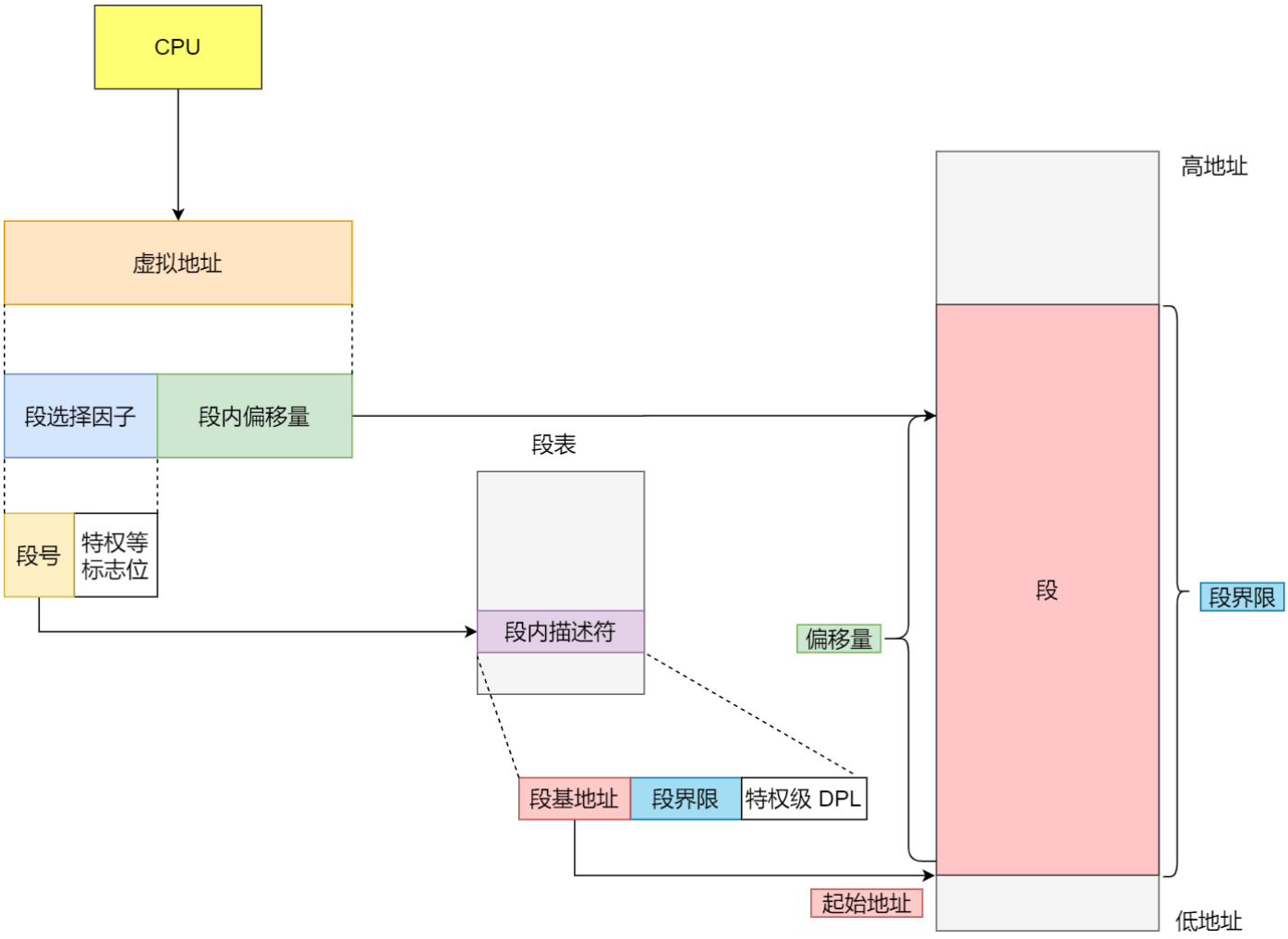
操作系统会提供一种机制，在物理内存和虚拟内存之间建立一个地址映射表，进程持有的虚拟地址会通过 CPU 芯片中的内存管理单元（MMU）的映射关系，来转换变成物理地址，然后再通过物理地址访问内存

操作系统主要通过**内存分段**和**内存分页**来管理虚拟内存和物理内存之间的关系。

内存分段

内存分段将物理内存划分成不同的逻辑段或区域，每个段用于存储特定类型的数据或执行特定类型的任务。每个段具有不同的大小和属性。常见的段包括代码段（存储程序执行代码）、数据段（存储程序数据）、堆栈段（存储函数调用和局部变量），以及其他自定义段，如共享库段等。

分段机制下的虚拟地址由两部分组成，**段选择因子**和**段内偏移量**，而虚拟地址是通过**段表**与物理地址进行映射的。



段选择因子中的段号与段表对应，作为段表的索引。**段表**里面保存的是这个**段的基地址、段的界限和特权等级**等。虚拟地址中的**段内偏移量**应该位于 0 和段界限之间，如果段内偏移量是合法的，就将段基地址加上段内偏移量得到物理内存地址。

内存碎片

内存碎片是指内存中的空闲空间被分割成多个不连续的小块，而不是一个连续的大块。这些小块空闲内存可能分布在整个内存地址空间中，导致内存资源的不充分利用。

外部碎片：虽然整体内存容量足够，但无法为较大的内存请求分配**连续**的内存块，因为已分配的内存块分散在内存中，它们之间的未分配空间太小。

内部碎片：有时分配的内存块大于进程实际需要的内存量，这意味着有一些内存浪费在内部碎片中。

内存分段

内存分页

页面置换算法

1、最佳页面置换算法(OPT)

置换在「未来」最长时间不访问的页面,但是实际系统中无法实现，因为程序访问页面时是动态的

我们是无法预知每个页面在「下一次」访问前的等待时间，因此作为实际算法效率衡量标准。

2、先进先出置换算法(FIFO)

顾名思义，将页面以队列形式保存，先进入队列的页面先被置换进入磁盘。

3、最近最久未使用的置换算法（LRU）

根据页面未被访问时长用升序列表将页面排列，每次将最久未被使用页面置换出去。

4、时钟页面置换算法

把所有的页面都保存在一个类似钟面的「环形链表」中，页面包含一个访问位。

当发生缺页中断时，顺时针遍历页面，如果访问位为1，将其改为0，继续遍历，直到访问到访问位为0页面，进行置换。

5、最不常用算法

记录每个页面访问次数，当发生缺页中断时候，将访问次数最少的页面置换出去，此方法需要对每个页面访问次数统计，额外开销。

分段

虚拟内存采用的是分页技术，也就是将地址空间划分成固定大小的页，每一页再与内存进行映射。

如果使用分页系统的一维地址空间，动态增长的特点会导致覆盖问题的出现。

分段的做法是把每个表分成段，一个段构成一个独立的地址空间。

每个段的长度可以不同，并且可以动态增长。

1、纯分段

分段和分页本质上是不同的，页面是定长的而段不是。

优点：

(1) 共享：有助于几个进程之间共享过程和数据。 比如共享库
(2) 保护：每个段都可以独立地增大或减小而不会影响其他的段

2、分段和分页结合

程序的地址空间划分成多个拥有独立地址空间的段，每个段上的地址空间划分成大小相同的页。

这样既拥有分段系统的共享和保护，又拥有分页系统的虚拟内存功能.

3、分段与分页的比较

对程序员的透明性：

分页透明，但是分段需要程序员显式划分每个段。

地址空间的维度：

分页是一维地址空间，分段是二维的。

大小是否可以改变：

页的大小不可变，段的大小可以动态改变。

出现的原因：

分页主要用于实现虚拟内存，从而获得更大的地址空间；分段主要是为了使程序和数据可以被划分为逻辑上独立的地址空间并且有助于共享和保护。

虚拟内存

简述

属于计算机系统内存管理的一种技术，虚拟地址空间构成虚拟内存，它使得应用程序认为自己拥有连续的可用内存空间，但实际上是被分隔的多个物理内存页、以及部分暂时存储在磁盘上的交换分区所构成的。虚拟内存的实现通过硬件异常、硬件地址翻译、主存、磁盘以及内核软件共同完成

1、地址空间：

地址空间是物理内存的抽象，是一个进程可用于寻址内存的一套地址集合。

2、分页：

地址空间被分割成多个块，每一块称作一页或页面(Page)。每一页有连续的地址范围，这些页被映射到连续的物理内存(页框)。

3、页表：

页表的目的是把虚拟页面(虚拟地址)映射为页框(物理地址)。页表给出了虚拟地址与物理地址的映射关系。从数学的角度说页表是一个函数，他的参数是虚拟页号，结果是物理页页框号

4、加速分页：

（1）TLB加速分页

概念：将虚拟地址直接映射到物理地址，而不必再访问页表，这种设备被称为转换检测缓冲区（TLB）、相联存储器或快表

工作过程：将一个虚拟地址放入MMU中进行转换时，硬件首先通过将该虚拟页号与TLB中所有表项同时进行匹配，判断虚拟页面是否在其中：

1. 虚拟页号在TLB中。如果MMU检测一个有效的匹配并且访问操作并不违反保护位，则将页框号直接从TLB中取出而不必访问页表。
2. 虚拟页号不在TLB中。如果MMU检测到没有有效的匹配项就会进行正常的页表查询。接着从TLB中淘汰一个表项，然后用新的页表项替换它。

（2）软件TLB管理

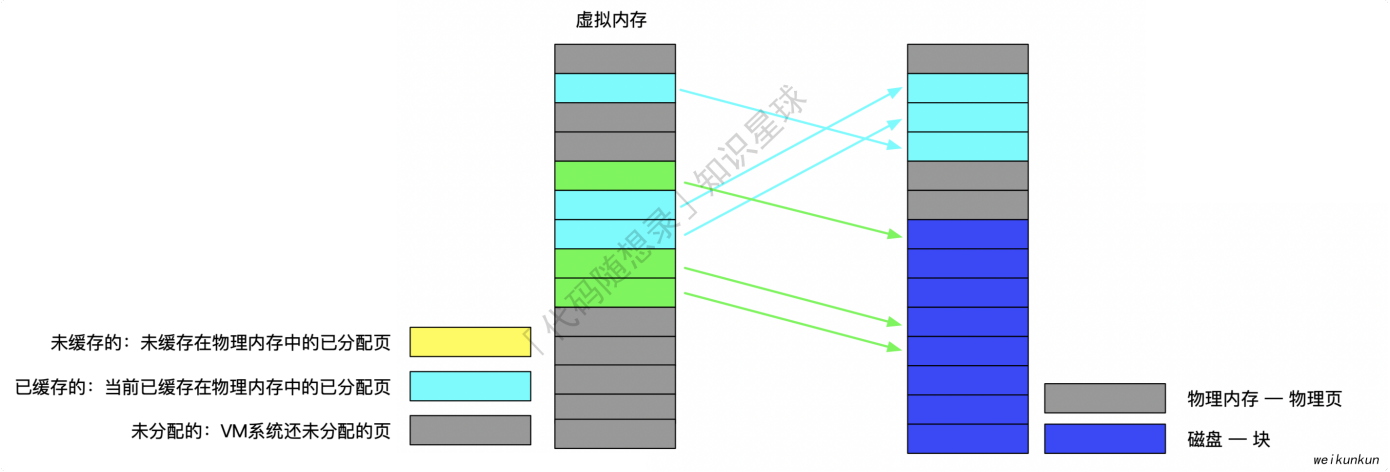
5、针对大内存的页表

（1）多级页表
（2）倒排页表

三个重要的能力：

1. 高速缓存

按照存储器层次结构中的缓存划分来看，将位于k层的更小更快的存储设备作为存储在k+1层的更大更慢的存储设备的缓存。具体如下：



在虚拟内存中的虚拟页共分为三种类型：

1. 未分配（没有被进程申请使用的，也就是空闲的虚拟内存， 不占用虚拟内存磁盘的任何空间）
2. 未缓存（仅仅加载到磁盘中的页）
3. 已缓存（已经加载到内存中的内存页（页框））

2. 内存管理

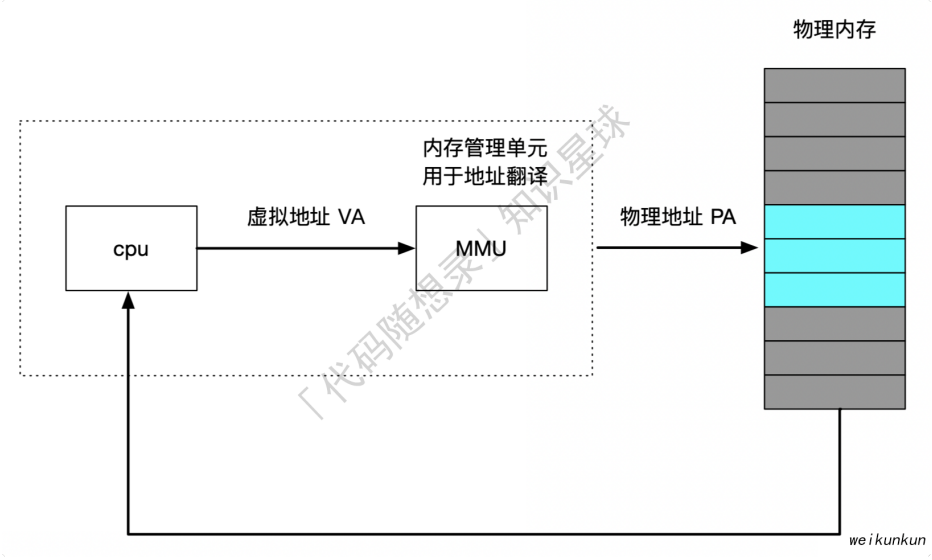
3. 内存保护

现代操作系统中，用户进程不应该被允许修改它的只读代码段;

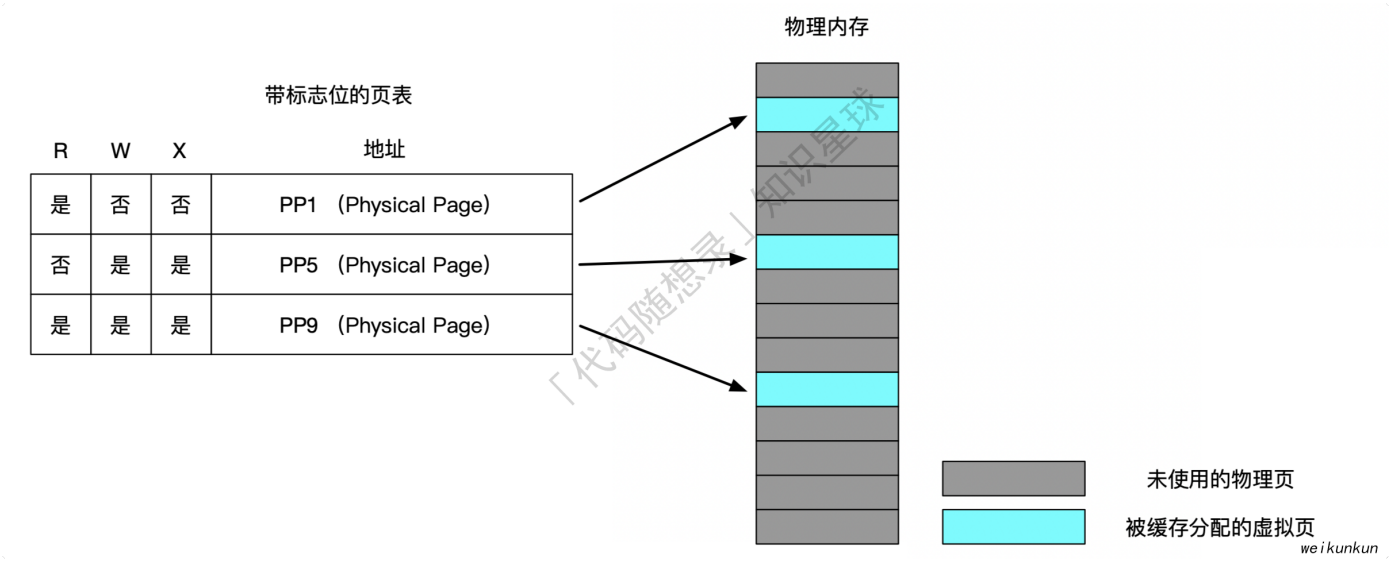
而且也不应该允许它读取或修改任何内核中的代码和数据结构;

并且也不允许其读取或者修改其他进程的私有内存，以及修改和其他进程共享的虚拟页面。

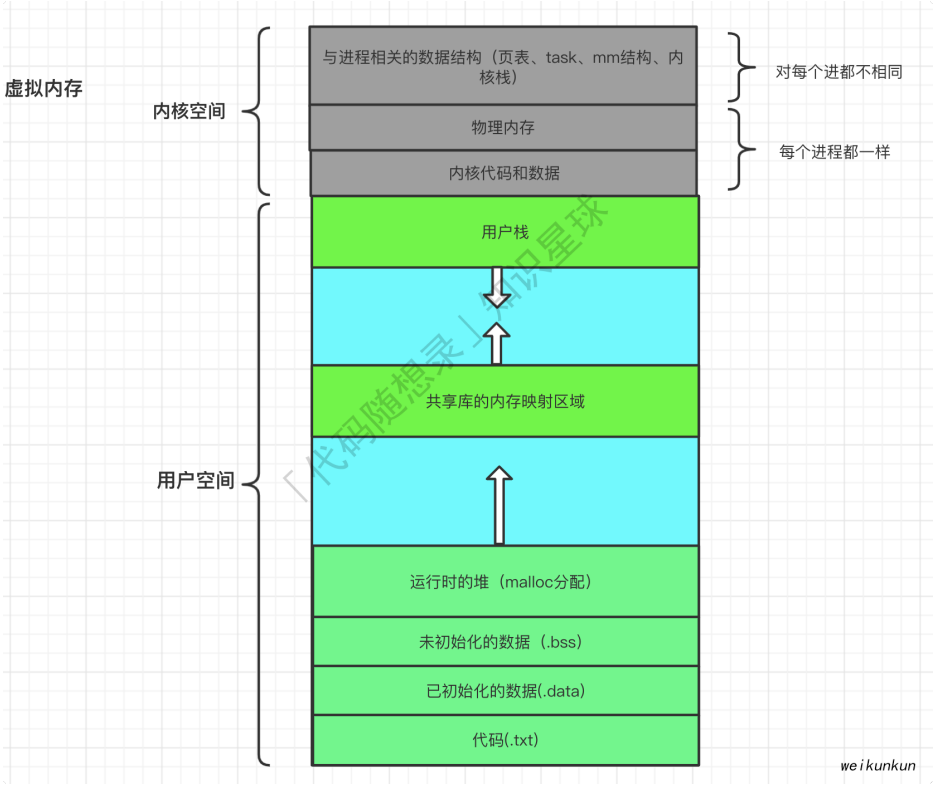
如果不对进程的内存访问进行限制，攻击者就能够访问和修改其他进程的私有内存，进而导致整个系统崩溃。大体的虚拟寻址如下：



通过MMU，每次都会读取页表中的一个页表条目(PTE)，通过在这些页表条目(PTE)中添加一些标志位，就能够实现对一个虚拟页的访问控制权限。譬如：



相关的其他概念



1. 用户态和内核态

为了使操作系统内核提供一个无懈可击的进程抽象，处理器必须提供一种机制，限制一个应用可以执行的指令以及他可以用来访问的地址空间范围。
处理器通常是用某个控制寄存器中的一个**模式位**来提供这种观功能的，该寄存器描述了进程当前享有的特权。

当设置了**模式位**时：
进程就运行在内核态中。
运行在内核态中的进程可以执行指令集中的任何指令，并且可以访问系统中的任何内存位置。

没有设置**模式位**时：
进程就运行在用户态。用户模式中的进程不允许执行**特权指令**。

比如：停止处理器、改变模式位、或者发起一个I/O操作。同时也不允许用户态下的进程直接引用地址空间中内核空间的代码和数据。
任何这样的尝试都会导致致命的保护故障。用户态下的程序必须通过系统调用接口间接的访问内核代码和数据。

2. 上下文切换 '

上下文切换属于一种较高层形式的异常控制流，进行上下切换涉及到的内容如下：

（1）各种寄存器，TLB 也是其中之一。

（2）程序计数器

（3）用户栈

（4）内核栈

（5）内核中的各种数据结构

1. 页表
2. task_struct
3. mm_struct

小总结

虚拟内存的思想，整体来看就是：**通过结合磁盘和内存各自的优势，利用中间层对资源进行更合理地调度，充分提高资源的利用率。并提供和谐以及统一的抽象。**

为什么需要虚拟内存：（摘自 为什么Linux需要虚拟内存一文)

1. 虚拟内存可以结合磁盘和物理内存的优势为进程提供看起来速度足够快并且容量足够大的存储
2. 虚拟内存可以为进程提供独立的内存空间并引入多层的页表结构将虚拟内存翻译成物理内存，进程之间可以共享物理内存减少开销，也能简化程序的链接、装载以及内存分配过程;
3. 虚拟内存可以控制进程对物理内存的访问，隔离不同进程的访问权限，提高系统的安全性。

文件系统

文件

1. 文件

文件是一种抽象机制，它提供了一种在磁上保存信息而且方便以后读取的方法。这种方法可以使用户不必了解存储信息的方法、位置和实际磁盘工作方式等有关细节

2. 文件命名

win95、win98用的都是MS-DOS的文件系统，即FAT-16， win98扩展了FAT-16成为FAT-32。

较新版的操作系统NTFS,win8配备ReFS。微软优化FAT,叫作exFAT。prog.c，圆点后面的部分称为文件扩展名。

3. 文件结构

1、字节结构

把文件看成字节序列为操作系统提供了最大的灵活度

2、记录序列

文件结构上的第一步改进，这种模型中，文件是具有固定长度记录的序列

3、树

文件在这种结构中由一棵记录树构成，每个记录不必具有相同的长度，记录的固定位置上有一个键字段。这棵树按“键”字段进行排序，从而可以对特定“键”进行快速查找。

4. 文件类型

1. 普通文件
2. 目录
3. 字符特殊文件（UNIX）
4. 块特殊文件（UNIX）

5. 文件访问

1、顺序访问

按顺序读取文件的全部字节，早期操作系统只有这种访问方式

2、随机访问文件

当用磁盘存储文件时，可以以任何次序读取其中字节或记录的文件。许多应用程序需要这种类型文件

6. 文件属性

除了文件名和数据外，所有操作系统还会保存其他的文件相关信息，如创建日期、时间和大小等，这些**附加的信息**称为文件属性

举例：

创建者：创建文件者ID
所有者：当前所有者
当前大小：文件字节数
… : …

7. 文件操作

使用文件的目的是**存储信息并方便以后检索**。对于存储和检索，不同系统提供了不同的操作。

常见的文件操作（系统调用）：

1、create

创建不包含任何数据的文件

2、delete

当不再需要某个文件时，必须删除该文件以释放磁盘空间

3、open

在使用文件之前，必须先打开文件

4、close

访问结束后，不再需要文件属性和磁盘地址，这时应该关闭文件以释放内部表空间

5、read

在文件中读取数据

6、write

向文件写数据，写操作一般也是从文件当前位置开始

7、append

此调用是write的限制形式，他只能在文件末尾添加数据

8、seek

对于随机访问文件，要指定从何处开始获取数据，通常的方法是用seek系统调用把当前位置指针指向文件中特定的位置。

9、get attributes

进程运行常需要读取文件属性

10、set attributes

某些属性是可由用户设置的，甚至在文件创建之后

11、rename

用户尝尝要改变已有的名字，rename系统调用用于这一目的

目录

文件系统通常提供目录或文件夹用于记录文件的位置，在很多操作系统中目录本身也是文件

1. 一级目录系统

在一个目录中包含所有文件，这有时称为根目录

2. 层级目录系统

当用户有着数以千计的文件，为了寻找方便。需要层次结构（即一个目录树）

3. 路径名

1、绝对路径名

它由从根目录到文件的路径组成。

Windows： `\usr\ast\maibox`
UNIX： `/usr/ast/mailbox`

路径名的第一个字符是分隔符，则这个路径是绝对路径

2、相对路径名

它常和工作目录(working directory)(也和当前目录(acurrent directory))一起使用

例：UNIX

```
cp /usr/ast/mailbox /usr/ast/mailbox/mailbox.bak
```

和命令

```
cp mailbox mailbox.bak # 具有相同的含义
```

3、特殊目录项

`. dot`：指当前目录
`.. dotdot` 指其父目录

eg:

```
cp ../lib/dictionay .
```

将usr/lib/下的dictionay复制到当前目录下

4. 目录操作

管理目录的系统调用

举例(Unix):

```
create      # 创建目录
delete      # 删除目录
opendir     # 目录内容可被读取
closedir    # 读目录结束
readdir     # 返回打开目录下一个目录项
rename      # 改变目录名
link        # 链接技术允许在多个目录中出现同一个文件
unlink      # 删除目录项
```

文件系统的实现

文件存储实现的关键问题是记录各个文件分别用到哪些磁盘块。不同的操作系统用到不同的方法

1. 文件系统布局

文件系统存放在磁盘上。多数磁盘划分为一个或多个分区，每个分区中国有一个独立的文件系统。

磁盘0号扇区称为主引导记录(MBR)，用来引导计算机。在MBR结尾是分区表。该表给出每个分区的起始结束地址。

表中的一个分区被标记为活动区。在计算机被引导时，BIOS读入并执行MBR。MBR做的第一件事是确定活动分区，读入它的第一个块，称为引导块，并执行之。

引导块中的程序将装载该分区中的操作系统。

2. 文件的实现

1、连续分配

最简单的分配是把每个文件作为一连串连续数据块存储在磁盘

优势：

（1）实现简单

记录每个文件用到磁盘块简化为只需记住两个数字即可：第一块的磁盘地址和文件的块数。给定了第一块编号，一个简单的加法就可以找到任何其他块的编号

（2）操作性能较好

因为单个操作中就能从磁盘上读出整个文件。只需一次寻找。

2、链表分配

为每个文件构造磁盘块链表。每一块的第一个字作为指向下一块的指针，块的其他部分存放数据。

优势：

（1）可以充分利用磁盘块，不会因为磁盘碎片浪费存储内存
（2）随机访问快。

缺点：

指针占用一些字节，每个磁盘块存储数据的字节数不再是2的整数次幂。怪异的大小降低了系统的运行效率，每个块前几个字节被指向下一个块的指针所占据，需要从两个磁盘中获取拼接信息，这就因复制而引发额外的开销。

3、采用内存中的表进行链表分配

取出每个磁盘块的指针字，把他放在内存的一个表中，解决链表分配的不足。内存中这样的表格称为文件分配表(File Allocation Table，FAT)

4、i节点

最后一个记录各文件分别包含哪些磁盘块的方法是给每个文件赋予一个称为i节点的数据结构，其中列出了文件属性和文件的磁盘地址。

3. 目录的实现

在读文件之前，必须先打开文件。打开文件时，操作系统利用用户给出的路径名找到相应的目录项。

简单目录：包含固定大小的目录，在目录项中有磁盘地址和属性

采用i节点的系统：把文件属性存放在i节点中而不是目录项中。这种情形下，目录项会更短。

4. 共享文件

当几个用户同在一个项目里工作时，他们常常需要共享文件。

共享文件与目录的联系称为一个链接（link）。这样文件系统本身就是一个有向无环图（DAG），而不是一棵树。

5. 日志结构文件系统

CPU运行速度越来越快，磁盘容量越来越大，价格越来越便宜（但磁盘速度并没有增快多少），同时内存容量也以指数形式增长。

而没有得到快速发展的参数是磁道的寻道时间。这些成为了文件系统性能的瓶颈，为了解决这一问题设计了全新的文件系统即日志结构文件系统（LFS）。

虽然是一个很吸引人的想法，由于它们和文件系统不匹配，该文件系统并没有被广泛应用。

6. 日志文件系统

基本的想法是保存一个用于记录系统下一步将要做什么的日志。

这样当系统在完成他们即将完成的任务前崩溃时，重新启动后，可以通过查看日志，获取崩溃前计划完成的任务，并完成它们。

这样的文件日志系统，并已经被实际应用。微软的NTFS、Linux的 ext3和ReiserFS文件系统都使用日志。

7. 虚拟文件系统

将多个文件系统整合到一个统一的结构中。

一个Linux系统可以用ext2作为根文件系统，ext3分区装载在/usr下，另一块采用ReiserFS文件系统的硬盘装载在/home下，以及一个ISO 9660的CD-ROM临时装载在/mnt下。从用户的观点来看，只有一个文件系统层级。

它们事实上是多种文件系统，对于用户和进程是不可见的。绝大多数Unix操作系统都在使用虚拟文件系统（Virtual File System, VFS）

文件系统的管理和优化

1. 磁盘空间管理

几乎所有文件系统都把文件分割成固定大小的块来存储，各块之间不一定相邻

1、块大小

从历史的观点上来说，文件系统将大小设在1~4KB之间，但现在随着磁盘超过了1TB，还是将块的大小提升到64KB并且接受浪费的磁盘孔空间，这样也许更好。磁盘空间几乎不再会短缺。

2、记录空闲块

（1）磁盘块链表

链表的每个块中包含尽可能多的空闲磁盘块号。
通常情况下，采用空闲块存放空闲表，这样不会影响存储器

（2）位图

在位图中，空闲块用1表示，已分配块用0表示。

（3）磁盘配额

为防止人们贪心而占有太多的磁盘空间，用户操作系统常常提供一种强制性磁盘配额机制。
其思想是系统管理员分给每个用户拥有文件和块的最大数量，操作系统确保每个用户不超过分给他们的配额。（配额表、打开文件表）

2. 文件系统备份

做磁盘备份主要是处理好两个潜在问题中的一个

1. 从意外的灾难中恢复
2. 从错误的操作中恢复

3. 文件系统的一致性

很多文件系统读取磁盘块，进行修改后，再写回磁盘。

如果在修改过的磁盘块全部写回之前系统崩溃，则文件系统有可能处于不一致状态。

如果一些未被写回的块是i节点块、目录块或者是包含有空闲表的块时，这个问题尤为严重

4. 文件系统性能

1、高速缓存

减少磁盘访问次数技术是块高速缓存（block cache）或者缓冲区高速缓存（buffer cache）。

本书中，高速缓存指的是一系列的块，它们在逻辑上属于磁盘，但实际上基于性能的考虑被保存在内存中。

2、块提取读

在需要用到块之前，试图提前将其写入高速缓存，从而提高命中率。

块提前读策略只适用于实际顺序读取的文件。对随机访问文件，提前读丝毫不起作用。

3、减少磁盘臂运动

把可能顺序访问的块放一起，当然最好是同一柱面上，从而减少磁盘臂的移动次数。

5. 磁盘碎片整理

移动文件使它们相邻，并把所有的空闲空间放在一个或多个大的连续区域内。

磁盘

生磁盘的使用

生磁盘：根据**盘块号**来使用磁盘；
熟磁盘：根据**文件**来使用磁盘。

磁盘如何使用?

1. 发送 out 指令；
2. 文件视图；
3. 发送 中断

磁盘的结构

盘面：类似于多个堆叠在一起的 DVD 光盘
磁道：每个 DVD 光盘上的同心圆
扇区：每个同心圆上的 一段儿 小区域

磁盘 IO 过程

寻道：将磁头移动到 指定 磁道
旋转：将磁头旋转到磁道中的指定 扇区
传输：对指定扇区 进行 磁盘数据 IO 读写

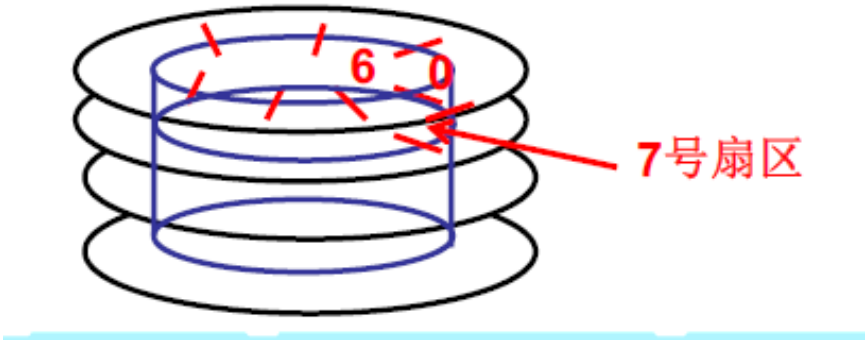
一层抽象

通过盘块号读写磁盘

对于用户来说，如果直接利用 盘/柱面 C、磁头 H、扇区 S 来进行 磁盘 IO 不太友好，所以通过一层抽象，即 用户只用利用 盘块号 block 来进行 磁盘IO，然后由系统根据 block 计算出 C、H、S 即可

磁盘编址

由于寻道时间 通常时间更长一些，所以，为了让 磁盘 IO 时间尽可能短一些，则需要使 寻道时间尽量段。即，把当一个 磁道编址用完后，向下一个 柱面 继续编号，而不是向 下一个 磁道继续编号。



计算公式：盘块号 block = C *(Heads* Sectors) + H * Sectors + S

二层抽象

多个进程通过队列使用磁盘

虽然一层抽象（利用 盘块号）可以达到使用磁盘的目的，但是如果有多个磁盘访问请求出现在请求队列怎么办？此时，操作系统会根据 磁盘调度算法 来挑选一个 磁盘请求服务

常见磁盘调度算法

1、FCFS：找最先来的请求

优点：公平
缺点：并不提供 最快的服务

FCFS 在寻道过程中，可能已经遇到一些以后可能需要访问的 磁道，但是会跳过，而造成 访问磁道 耗费时间较多。

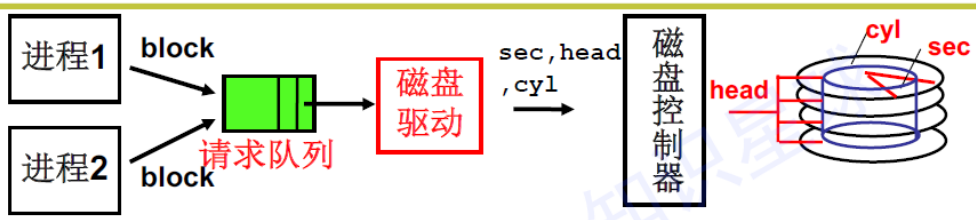
2、SSTF：类似于 SJF，每次选择距离当前磁头最近的待处理请求

优点：寻道时间最短
缺点：可能造成部分请求 “饥饿”（当某个请求的磁盘距离磁头较远，而一直有比其更近的请求时，这个请求一直无法执行）

3、SCAN 算法：类似电梯，先处理向上的请求，然后再处理反方向的请求

优点：不会造成部分请求的 “饥饿”

生磁盘(raw disk)的使用整理



- (1) 进程“得到盘块号”，算出扇区号(sector)
- (2) 用扇区号make req，用电梯算法add_request
- (3) 进程sleep_on
- (4) 磁盘中断处理

```
static void read_intr(void)
{   end_request(1);
    do_hd_request();
}
```

唤醒进程!

- (5) do_hd_request算出cyl,head,sector
- (6) hd_out调用outp(...)完成端口写

三层抽象

利用**文件**使用磁盘，就成了熟磁盘

让普通用户使用 raw disk： 许多人连扇区都不知道是什么? 要求他们根据盘块号来访问磁盘不太便利
需要在盘块上引入更高一层次的抽象概念! 文件

用户眼里的 “文件”：一堆字符序列（字符流）
磁盘上的 “文件”：多个**盘块**构成的集合
所以，如果要**利用文件来进行**磁盘IO的话，就需要建立字符流到盘块集合的映射关系

常见的文件映射关系（文件实现方式）有以下 3 种：

1. 顺序结构
2. 链式结构
3. 索引

顺序结构-实现文件

FCB：记录文件在磁盘中的起始块

文件->盘块号

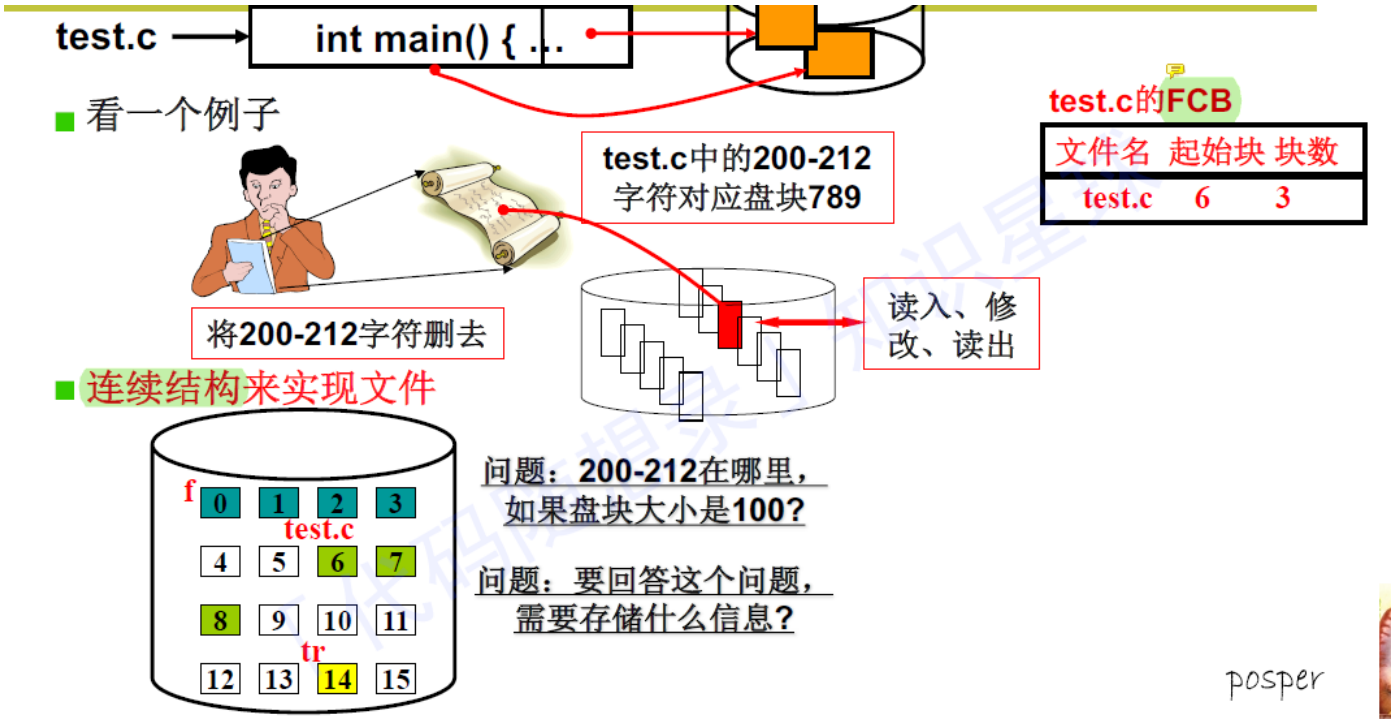
如何**根据 文件 找到 盘块号**?

将文件按照 连续结构存放；（下 左图）；
对每个文件建立一个 FCB，记录该文件在磁盘中的 起始块号；
根据文件（字符流）中的第 n 个待查字符 + FCB 计算 （n / 每个盘块的大小 + FCB中的 起始块号】出其在磁盘上对应的 盘块号 【得到盘块号，然后采用 SCAN 算法]

这种方法的缺点：

1. 利用顺序结构存储文件，不利用文件的修改（因为 顺序存储的 增/删 代价非常大）

2. 所以，当需要 动态修改 文件时，最好使用 链式存储文件



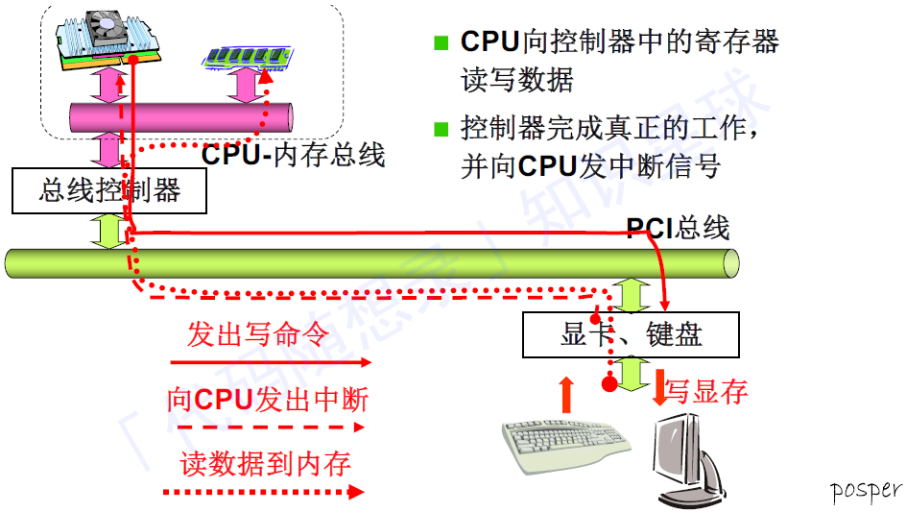
链式结构-实现文件

优点：利于 增/删
缺点：不利于查询

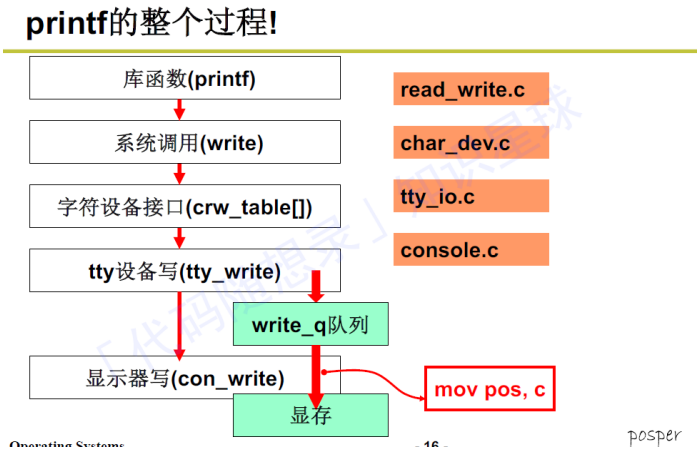
外设管理

如何让外设动起来

1. CPU 向 **外设的控制器**发送指令，即 out 指令
2. 形成 文件视图（为了统一 out 指令的形式）
3. 中断（外设处理完事后，需要通知 cpu 继续接手 下一步处理）

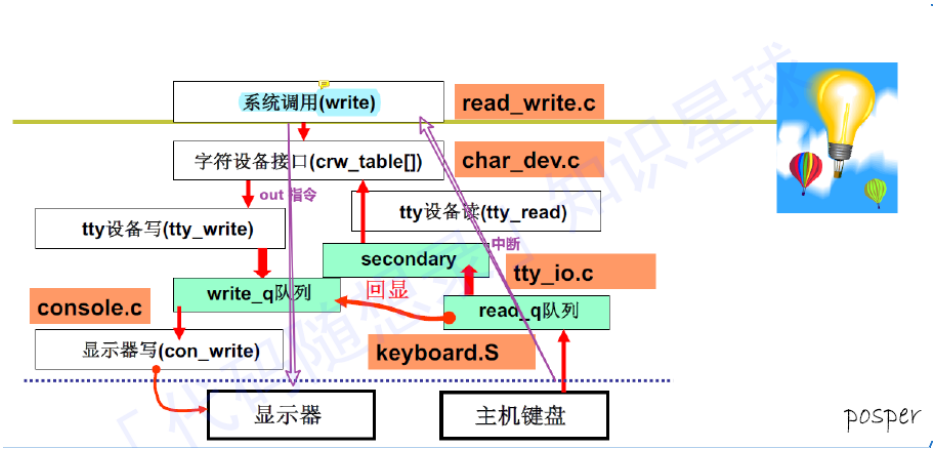


显示器如何工作? 即，printf 函数的工作流程



键盘如何工作?

中断处理（根据扫描码 获取 对应的 ascii 码）；
将对应的 ascii 码 加入 缓冲队列 read_que 中，等待上层程序调用



Linux

什么是IO多路复用

I/O多路复用是一种在单个线程或进程中处理多个输入和输出操作的机制。它允许单个进程同时监视多个文件描述符(通常是套接字)，当一个或多个文件描述符准备好读或写时，它就可以立即响应。

I/O多路复用通常通过select、poll、epoll等系统调用来实现。

- **select**： select是一个最古老的I/O多路复用机制，它可以监视多个文件描述符的可读、可写和错误状态。然而，但是它的效率可能随着监视的文件描述符数量的增加而降低。
- **poll**： poll是select的一种改进，它使用**轮询方式**来检查多个文件描述符的状态，避免了select中文件描述符数量有限的问题。但对于大量的文件描述符，poll的性能也可能变得不足够高效。
- **epoll**： epoll是Linux特有的I/O多路复用机制，相较于select和poll，它在处理大量文件描述符时更加高效。epoll使用事件通知的方式，只有在文件描述符就绪时才会通知应用程序，而不需要应用程序轮询。

I/O多路复用允许在一个线程中处理多个I/O操作，避免了创建多个线程或进程的开销，允许在一个线程中处理多个I/O操作，避免了创建多个线程或进程的开销。

select/poll/epoll的区别和联系

`select`，`poll`和`epoll`都是I/O多路复用技术，它们用于同时处理多个I/O操作，特别是在高并发网络编程中。

select

`select`是最早的I/O多路复用技术，它可以同时监视多个文件描述符（file descriptor, FD）的I/O状态（如可读、可写、异常等）。`select`函数使用一个文件描述符集合（通常是一个位图）来表示要监视的文件描述符，当有I/O事件发生时，`select`会返回对应的文件描述符集合。

select的主要限制如下：

- 文件描述符数量限制：`select`使用一个位图来表示文件描述符集合，这限制了它能够处理的文件描述符数量（通常是1024个）。
- 效率问题：当文件描述符数量较大时，`select`需要遍历整个文件描述符集合来查找就绪的文件描述符，这会导致较低的效率。
- 非实时性：每次调用`select`时，需要重新设置文件描述符集合，这会增加函数调用的开销。

poll

`poll`是为了克服`select`的限制而引入的一种I/O多路复用技术。`poll`使用一个文件描述符数组（通常是一个结构体数组）来表示要监视的文件描述符。与`select`类似，`poll`可以监视多个文件描述符的I/O状态。

poll的优点如下：

- 文件描述符数量不受限制：由于`poll`使用一个动态数组来表示文件描述符，因此它可以处理任意数量的文件描述符。
- 效率相对较高：`poll`在查找就绪的文件描述符时，只需要遍历实际使用的文件描述符数组，而不是整个文件描述符集合。

然而，`poll`仍然存在一些问题：

- 效率问题：尽管`poll`相对于`select`具有较高的效率，但当文件描述符数量很大时，它仍然需要遍历整个文件描述符数组。
- 非实时性：与`select`类似，每次调用`poll`时，需要重新设置文件描述符数组。

epoll

`epoll`是Linux特有的一种高效I/O多路复用技术，它克服了`select`和`poll`的主要限制。`epoll`使用一个事件驱动（event-driven）的方式来处理I/O操作，它只会返回就绪的文件描述符，而不是遍历整个文件描述符集合。

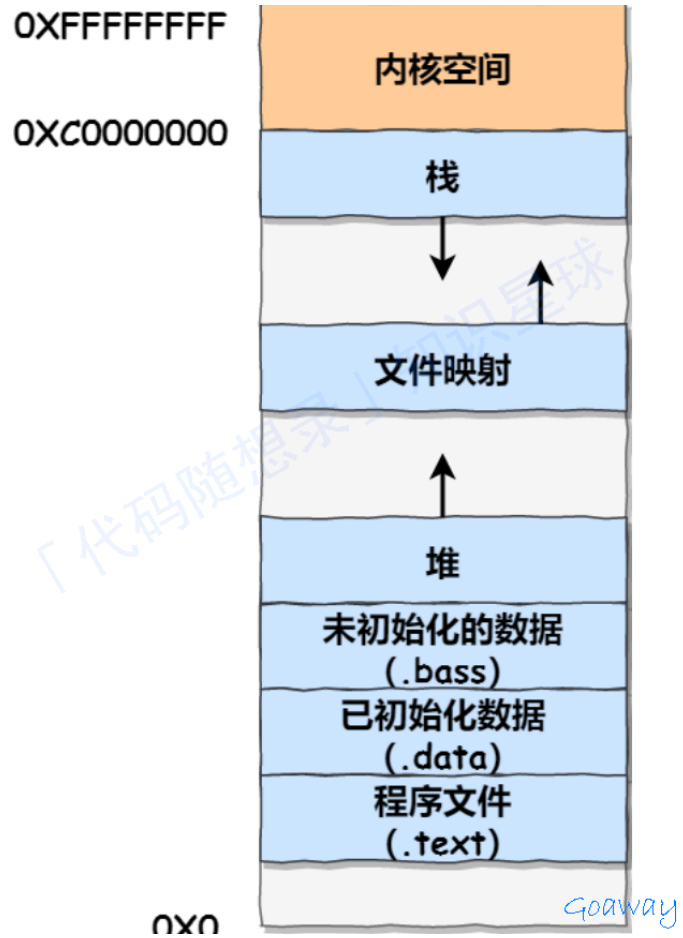
epoll的主要优点如下：

- 高效：`epoll`使用事件驱动的方式来处理I/O操作，因此它在处理大量文件描述符时具有很高的效率。当有I/O事件发生时，`epoll`可以立即得到通知，而无需遍历整个文件描述符集合。这使得`epoll`在高并发场景中具有更好的性能。
- 可扩展性：与`poll`类似，`epoll`可以处理任意数量的文件描述符，因为它使用一个动态数据结构来表示文件描述符。
- 实时性：`epoll`使用一个内核事件表来记录要监视的文件描述符和事件，因此在每次调用`epoll`时无需重新设置文件描述符集合。这可以减少函数调用的开销，并提高实时性。

`epoll`具有诸多优点，但它目前仅在Linux平台上可用。对于其他平台，可能需要使用类似的I/O多路复用技术，如BSD中的`kqueue`。

总结：`select`是最早的I/O多路复用技术，但受到文件描述符数量和效率方面的限制。`poll`克服了文件描述符数量的限制，但仍然存在一定的效率问题。`epoll`是一种高效的I/O多路复用技术，尤其适用于高并发场景，但它仅在Linux平台上可用。一般来说，epoll的效率是要比select和poll高的，但是对于活动连接较多的时候，由于回调函数触发的很频繁，其效率不一定比select和poll高。所以epoll在连接数量很多，但活动连接较小的情况性能体现的比较明显。

Linux 内存管理



1、程序文件段

包括程序的二进制可执行代码，只读；TEXT

2、已初始化数据段

包括已初始化的全局变量和静态常量；DATA

3、未初始化数据段

包括未初始化的静态变量和全局变量；BSS

4、堆段

包括动态分配的内存，从低地址开始向上增长；

5、文件映射段

包括动态库、共享内存等，从低地址开始向上增长（跟硬件和内核版本有关）；

6、栈段

包括函数的参数值和局部变量、函数调用的上下文等。栈的大小是固定的，一般是 8 MB 。当然系统也提供了参数，以便我们自定义大小；栈区是从高地址位向低地址位增长的

Linux虚拟内存

对32位处理器，虚拟内存空间为4G，每个进程都认为自己拥有4G的空间，实际上，在虚拟内存对应的物理内存上，可能只对应的一点点的物理内存。

进程得到的这4G虚拟内存是一个连续的地址空间（这也只是进程认为），而实际上，它通常是被分隔成多个物理内存碎片，还有一部分存储在外部磁盘存储器上，在需要时进行数据交换。

由于存在两个内存地址，因此一个应用程序从编写到被执行，需要**进行两次映射。**第一次是**映射到虚拟内存空间**，第二次时**映射到物理内存空间**。在计算机系统中，第两次映射的工作是由硬件和软件共同来完成的。承担这个任务的**硬件部分叫做存储管理单元 MMU**，**软件部分**就是操作系统的**内存管理模块**了。

1、如何处理虚拟地址和物理地址的关系

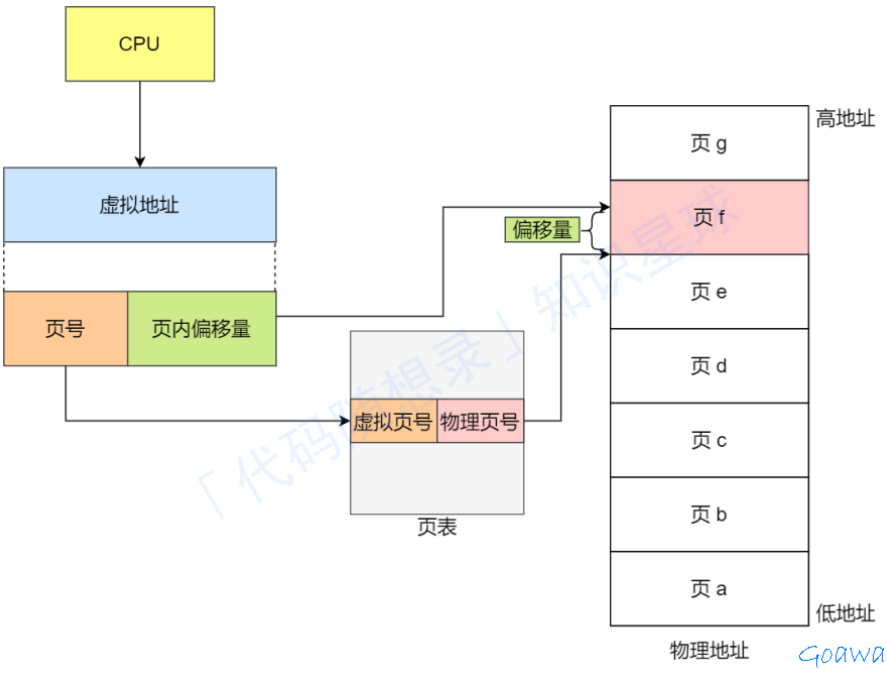
（1）内存分页

分页就是把整个虚拟和物理内存切成一段段固定大小的空间，连续且尺寸固定的内存空间叫页，Linux下每一页大小4KB。虚拟内存和物理内存之间通过页表来映射。

当进程访问的虚拟地址在页表中查不到时，系统会产生一个缺页异常，进入系统内核空间分配物理内存、更新进程页表，最后再返回用户空间，恢复进程的运行。

1. 采用了分页，那么释放的内存都是以页为单位释放的，也就不会产生无法给进程使用的小内存，解决了内存碎片的问题
2. 内存空间不足时，操作系统将正在运行的进程中，最近没使用的内存页面释放（暂时写入硬盘）需要的时候再加载进来，一次性只有少数的页，解决了交换效率低的问题
3. 分页使我们在加载程序时，不用一次性加载到物理内存，可以只有在程序运行中，需要用到对应虚拟内存页里面的指令和数据时，再加载到物理内存里面去

2、分页虚拟地址和物理地址是如何映射的



虚拟地址分为：页号和页内偏移。

页号作为页表的索引，页表包含物理页每页所在物理内存的基地址，这个基地址与页内偏移的组合就形成了物理内存地址。

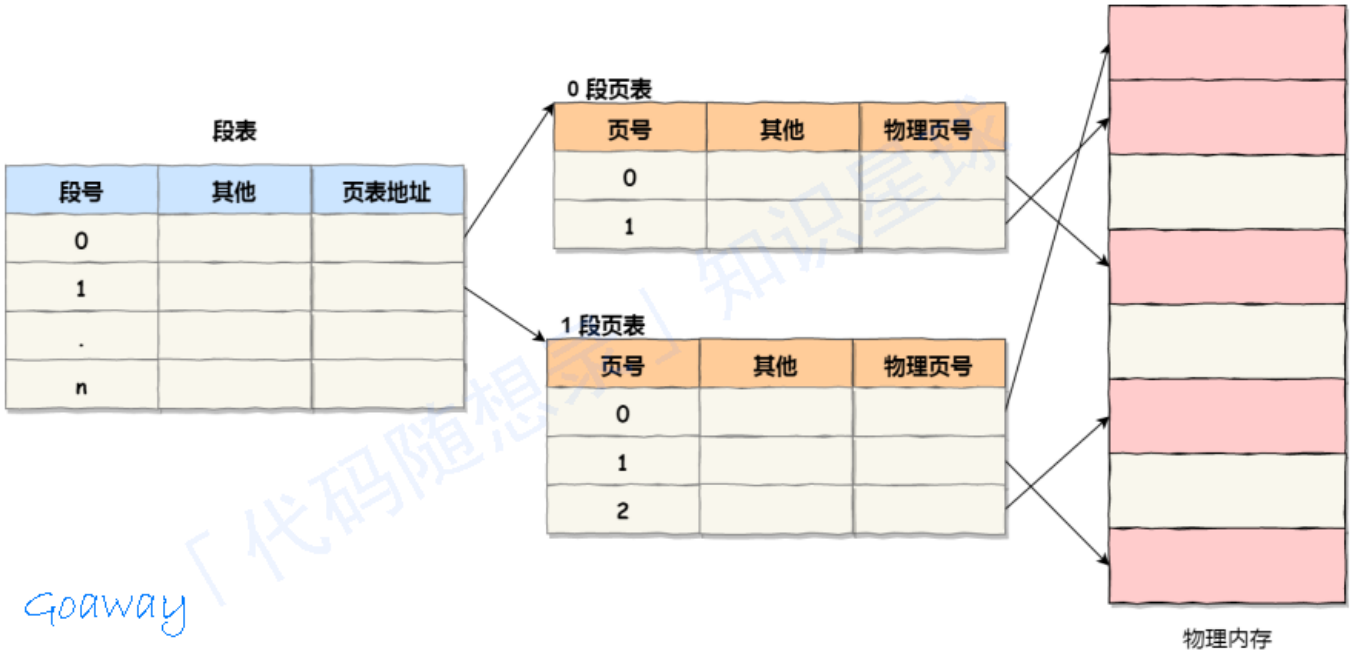
3、简单的分页缺陷

空间上的缺陷：

32位，单进程一个页4KB，虚拟内存4GB，就有2^20个页，一个页表项4字节，结果是4GB空间映射需要4*2^20 = 4MB存储页表。多进程100，需要400MB。

以页表一定要覆盖全部虚拟地址空间，不分级的页表就需要有 100 多万个页表项来映射，二级分页则只需要 1024 个页表项（此时一级页表覆盖到了全部虚拟地址空间，二级页表在需要时创建）。

4、段页式内存管理



地址结构就由段号、段内页号和页内位移三部分组成。

1. 第一次访问段表，得到页表起始地址
2. 第二次访问页表，得到物理页号
3. 第三次将物理页号与页内位移组合，得到物理地址

可用软、硬件相结合的方法实现段页式地址变换，这样虽然增加了硬件成本和系统开销，但提高了内存的利用率。

Linux 信号

信号是用户、系统、进程发送给目标进程的信息，通知某个状态的改变或系统异常。

linux常用信号：

1. SIGHUP控制终端挂起
2. SIGPIPE向读端关闭的通道或socket连接中写数据
3. SIGURG：socket连接上收到紧急数据

Linux设计

1、MutiTask 多任务

多个任务同时执行，同时可以是并发或并行。

2、SMP 对称多处理

每个CPU地位相等，使用权限相同，多个CPU共享一个内存，每个 CPU 都可以访问完整的内存和硬件资源。

3、ELF 可执行文件链接格式

Linux中可执行文件的存储格式。

4、Monolithic Kerne宏内核的特征是系统内核的所有模块

比如进程调度、内存管理、文件系统、设备驱动等，都运行在内核态。Linux 的内核是一个完整的可执行程序，且拥有最高的权限。

IO复用

1. I/O(数据交换过程)

1、背景

I/O即Input/Output（输入和输出），由于程序和运行时数据是在内存中驻留，由CPU这个超快的计算核心来执行，涉及到数据交换的地方，通常是磁盘、网络等，就需要IO接口。

IO编程中涉及到流，其是相对于内存而言的，所以，Input Stream就是数据从外面（磁盘、网络）流进内存，Output Stream就是数据从内存流到外面去。

2、概念

I/O操作就是在运行代码的过程中，可能需要对文件的读写，即将文件输入（Input）到内存和将代码执行结果产生的文件输出（Output）到外设（网络、磁盘）的过程。

3、分类

1. 网络I/O：通过网络进行数据的拉取和输出;
2. 磁盘I/O：主要是对磁盘进行读写工作。

2. 五种IO模型

1、输入阶段一般操作

1. 等待数据准备好
2. 从内核向进程复制数据

2、例如：

1. 等待数据从网络中到达：所等分组到达时，被复制到内核中的某个缓冲区
2. 把数据从内核缓冲区复制到应用进程缓冲区

3、分类

1. 阻塞式I/O
2. 非阻塞式I/O
3. I/O复用（select和poll）

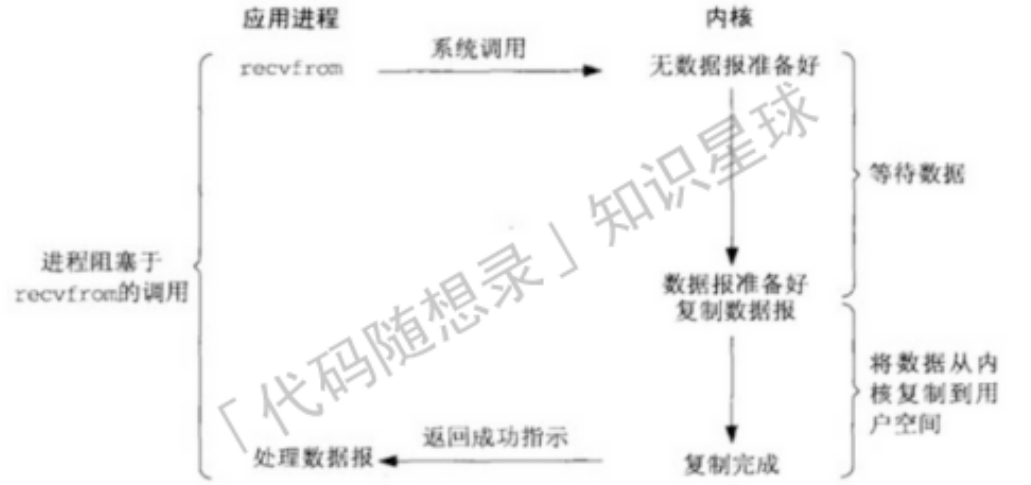
4. 信号驱动式I/O（SIGIO）
5. 异步I/O（POSIX的aio_系列函数）

4、同步I/O

（1）阻塞式I/O

进程或线程等待某个条件，如果条件不满足，则一直等下去。条件满足，则进行下一步操作。（默认情况下）

应用进程通过系统调用 recvfrom 接收数据，但由于内核还未准备好数据报，应用进程就会阻塞住，直到内核准备好数据报，recvfrom 完成数据报复制工作，应用进程才能结束阻塞状态。



「代码随想录」知识星球

图6-1 阻塞式I/O模型

槽d

优点：

设备文件不可操作时，可进入休眠状态，将CPU资源让出；当设备文件可以操作的时候就必须唤醒进程，一般在中断函数中完成唤醒工作;

缺点：

耗费时间，适合并发低，时效性要求低的情况。

（2）非阻塞式I/O

应用进程与内核交互，目的未达到之前，不再一味的等着，而是直接返回;

通过轮询的方式，不停的去问内核数据有没有准备好。如果某一次轮询发现数据已经准备好了，那就把数据拷贝到用户空间中。

前三次调用 recvfrom 时没有数据可返回，内核立即返回 EWOULDBLOCK 错误

第四次调用 recvfrom 时，已有一个数据报准备好，将其从内核复制到应用进程缓冲区， recvfrom 成功返回;

轮询（polling）：

应用进程持续轮询内核，以查看某个操作是否就绪;

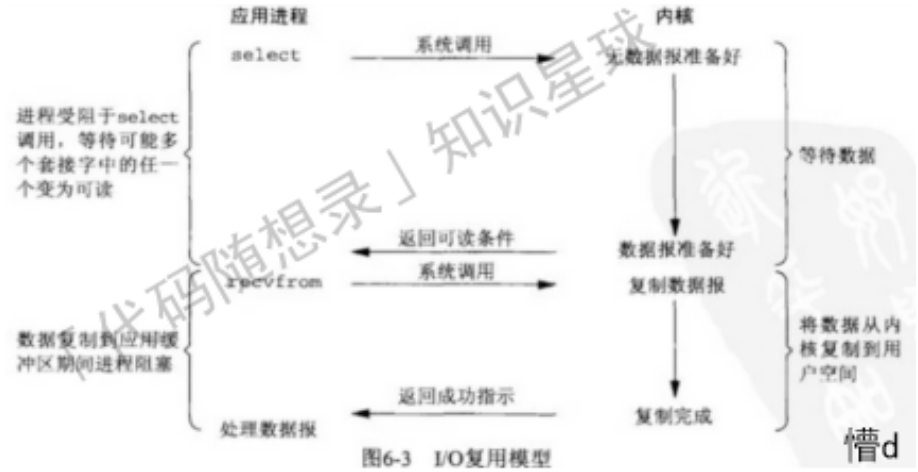
缺点：往往耗费大量CPU时间。

（3）IO复用

通过调用 select 或 poll，阻塞在这两个系统调用中的某一个之上，而不是阻塞在真正的I/O系统调用上。

阻塞于 select 调用，等待数据报套接字变为可读，当 select 返回套接字可读这一条件时，调用 recvfrom 把所读数据报复制到应用进程缓存区。

多个进程I/O注册到同一个 select，当用户进程调用 select，select 监听所有注册好的IO：



1. 若所有被监听的I/O需要的数据未准备好，则阻塞
2. 任意一个所需数据准备好后，select 调用返回
3. 用户进程通过 recvfrom 进行数据拷贝

优点：可以等待多个描述符就绪。

（4）信号驱动I/O

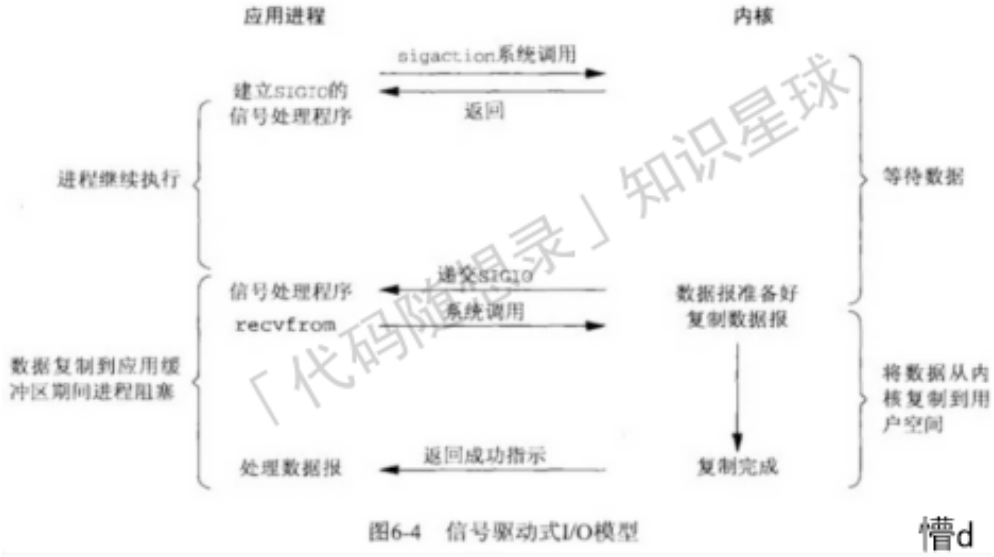


图6-4 信号驱动式I/O模型

「代码随想录」知识星球

开启套接字信号驱动I/O功能，通过 sigaction 系统调用安装一个信号处理函数，该系统函数立即返回，不阻塞;

数据报准备好后，内核为该进程产生一个 SIGIO 信号递交给进程;

可以在信号处理函数中调用 recvfrom 读取数据报，通知主循环数据已准备好待处理;

可以立即通知主循环，读取数据报。

优点：

等待数据报到达期间进程不被阻塞。主循环可以继续执行，等待来自信号处理函数的通知：既可以是数据已经准备好被处理，也可以是数据报已经准备好被读取。

（5）异步IO

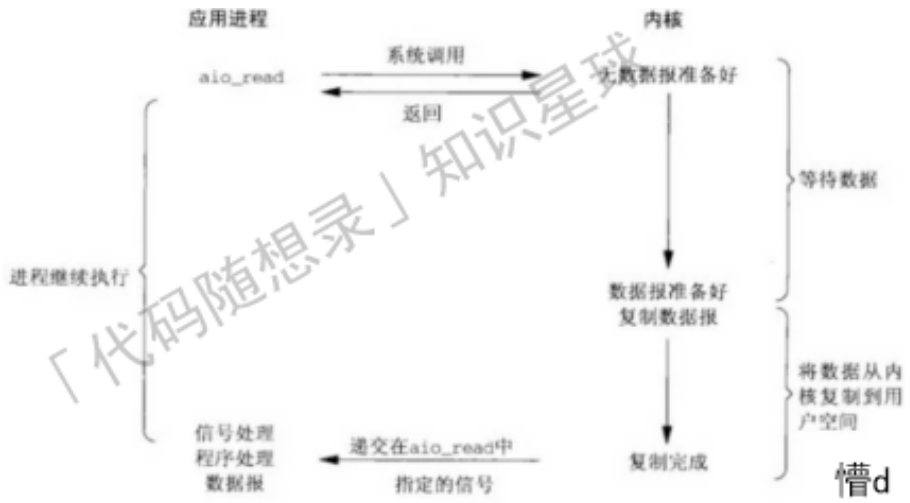
用户进程告知内核启动某个操作，并由内核在整个操作中完成后通知用户进程。

与信号驱动I/O的区别：

1. 信号驱动I/O是由内核通知我们何时可以启动一个I/O操作
2. 异步I/O是由内核通知我们I/O操作何时完成

步骤：

1. 用户进程调用aio_read函数，给内核传递描述符、缓冲区指针、缓冲区大小和文件偏移，告诉内核整个操作完成时如何通知我们，然后就立刻去做其他事情;
2. 当内核收到aio_read后，会立刻返回，然后内核开始等待数据准备，数据准备好以后，直接把数据拷贝到用户空间，然后再通知进程本次IO已经完成。



3. 各种IO模型的比较

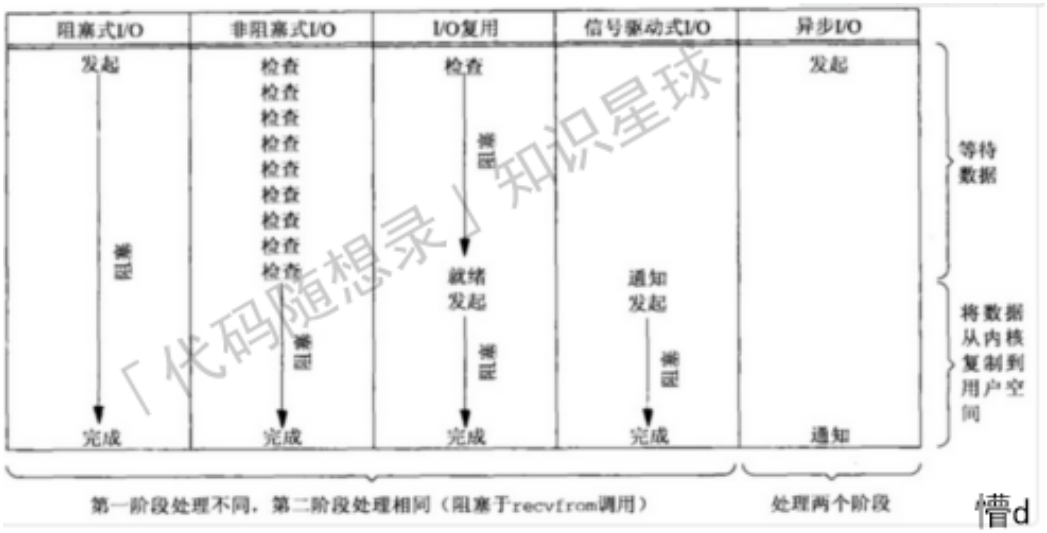
1、同步I/O

导致请求进程阻塞，直到I/O操作完成;

四种同步模型的**区别在于第一阶段等待数据的处理方式不同**，第二阶段均为将数据从内核空间复制到用户空间缓冲区期间，进程阻塞于recvfrom调用。

2、异步I/O

不导致请求进程阻塞。



3、I/O复用函数

多路复用接口 select/poll/epoll ，内核提供给用户态的多路复用系统调用，进程可以通过一个系统调用函数从内核中获取多个事件：

（1）**select-->两次遍历+两次拷贝**

把已连接的socket放在一个文件描述符集合，调用 select 函数将文件描述符集合拷贝到内核里，让内核来检查是否有网络事件产生；

通过遍历，有事件产生就把此socket标记为可读/可写，然后再整个拷贝回用户态；

用户态还需要遍历找到刚刚标记的socket。

（2）**poll**

动态数组，以链表形式来组织，相比于select，没有文件描述符个数限制，当然还会受到系统文件描述符限制。

（3）**epoll(event poll)-->红黑树**

在内核里使用红黑树来跟踪进程所有待检测的文件描述字。

调用epoll_ctl() 函数，把需要监控的 socket 加入内核中的红黑树里：（红黑树的增删查时间复杂度是O(logn)，不需要每次操作都传入整个集合，只需要传入一个待检测的socket，减少了内核和用户空间的大量数据拷贝和内存分配）

epoll 使用事件驱动的机制，内核里维护了一个链表来记录就绪事件（当某个 socket 有事件发生时，通过回调函数，内核会将其加入到这个就绪事件列表中）

当用户调用 epoll_wait() 函数时，只会返回有事件发生的文件描述符的个数，不需要像 select/poll 那样轮询扫描整个 socket 集合，大大提高了检测的效率。

（4）**epoll触发模式**

epoll支持的事件触发模式：

(1)边缘触发ET
(2)水平触发LT

边缘触发模式ET：

当被监控的 Socket 描述符上有可读事件发生时，服务器只会从 epoll_wait中苏醒一次，即使进程没有调用 read 函数从内核读取数据，也依然只苏醒一次，因此我们程序要保证一次性将内核缓冲区的数据读取完，只有第一次满足条件的时候才触发，之后就不会再传递同样的事件了。

水平触发模式LT：

当被监控的 Socket 上有可读事件发生时，服务器不断地从 epoll_wait中苏醒，直到内核缓冲区数据被 read 函数读完才结束，目的是告诉我们有数据，只要满足事件的条件，比如内核中有数据需要读，就一直不断地把这个事件传递给用户。

（5）**select和epoll的区别**

1. select 和 poll 采用轮询的方式检查就绪事件，每次都要扫描整个文件描述符，复杂度O(N);
2. epoll 采用回调方式检查就绪事件，只会返回有事件发生的文件描述符的个数，复杂度O(1);
3. select 只工作在低效的LT模式，epoll 可以在 ET 高效模式工作;
4. epoll 是 Linux 所特有，而 select 则应该是 POSIX 所规定，一般操作系统均有实现;
5. select 单个进程可监视的fd数量有限，即能监听端口的大小有限，64位是2048；epoll 没有最大并发连接的限制，能打开的 fd 的上限远大于2048（1G的内存上能监听约10万个端口）；
6. select 内核需要将消息传递到用户空间，都需要内核拷贝动作；epoll通过内核和用户空间共享一块内存来实现的。

Linux网络

> 如何在基本网络通信模型基础上提升服务器自身服务响应?

1、多进程服务器

> 每次的请求父进程fork出子进程进行业务处理，父进程用于处理请求连接

（1）父子进程的生命周期绝大多数都不相同，如何、何时进行销毁，才能确保不会大量出僵尸进程

（2）子父进程之间如何进行通讯?

进程的创建、切换都需要都会带来大量的开销，同时每个进程都具有自己独立的虚拟内存空间，所以需要使用到IPC机制，完成进程间的数据交换

2、I/O复用服务器

1. 在不额外创建进程的基础上同时为多个请求端提供请求
2. select、poll、epoll实现优缺点

3、多线程服务器

> 同一进程下的多个线程共享进程的系统资源，所以相比于多进程，同进程下的多线程通信更加方便，并且切换的开销小很多。

1. 如何创建、销毁线程
2. 如何保证线程的安全
3. 线程间的通信

Linux中的软链接和硬链接有什么区别?

- **物理实现：**
  - **软链接：** 软链接是一个独立的文件，它包含了指向目标文件或目录的路径。软链接实际上是一个特殊的文件，其中包含有关目标文件的引用。
  - **硬链接：** 硬链接是目标文件的一个额外的目录项。目录项指向相同的物理数据块，实际上只是文件系统中的两个或多个目录项指向相同的inode。
- **链接的目标：**
  - **软链接：** 软链接可以链接到文件或目录，甚至可以链接到不存在的文件。
  - **硬链接：** 硬链接只能链接到文件，而且必须是同一文件系统中的。
- **对链接的影响：**
  - **软链接：** 如果原始文件被删除或移动，软链接仍然存在，但链接将失效。软链接可以跨文件系统，但如果目标文件被删除，软链接将成为坏链接（dangling link）。
  - **硬链接：** 删除或移动原始文件并不会影响硬链接，因为硬链接只是inode的另一个引用。只有当所有硬链接都被删除后，inode的数据块才会被释放。
- **创建方式：**
  - **软链接：** 使用`ln -s`命令创建软链接。例如，`ln -s target_file link_name`。
  - **硬链接：** 使用`ln`命令创建硬链接。例如，`ln target_file link_name`。
- **链接数量：**
  - **软链接：** 软链接只会增加目标文件的链接计数，而不会增加inode的链接计数。
  - **硬链接：** 硬链接会增加目标文件的链接计数，也会增加inode的链接计数。当链接计数为零时，文件系统才会释放相关的数据块。

面试题

什么是中断和异常

中断和异常都会导致处理器暂停当前正在执行的任务，并转向执行一个特定的处理程序（中断处理程序或异常处理程序）。然后在处理完这些特殊情况后，处理器会返回到被打断的任务继续执行。

1. **中断是由计算机系统外部事件触发的，通常与硬件设备相关。**中断的目的是为了及时响应重要事件而暂时中断正常的程序执行。典型的中断包括时钟中断、I/O设备中断（如键盘输入、鼠标事件）和硬件错误中断等。操作系统通常会为每种类型的中断分配一个中断处理程序，用于处理相应的事件。
2. **异常是由计算机系统内部事件触发的，通常与正在执行的程序或指令有关，**比如程序的非法操作码、地址越界、运算溢出等错误引起的事件，异常不能被屏蔽，当出现异常时，计算机系统会暂停正常的执行流程，并转到异常处理程序来处理该异常。

用户态和核心态

1. 用户态和内核态的区别

用户态`（User Mode）`和内核态`（Kernel Mode）`是操作系统为了保护系统资源和实现权限控制而设计的两种不同的CPU运行级别，可以**控制进程或程序对计算机硬件资源的访问权限和操作范围。**

- 用户态：在用户态下，进程或程序只能访问受限的资源和执行受限的指令集，不能直接访问操作系统的核心部分，也不能直接访问硬件资源。
- 核心态：核心态是操作系统的特权级别，允许进程或程序执行特权指令和访问操作系统的核心部分。在核心态下，进程可以直接访问硬件资源，执行系统调用，管理内存、文件系统等操作。

2. 在什么场景下，会发生内核态和用户态的切换

- 系统调用：当用户程序需要请求操作系统提供的服务时，会通过系统调用进入内核态。
- 异常：当程序执行过程中出现错误或异常情况时，CPU会自动切换到内核态，以便操作系统能够处理这些异常。
- 中断：外部设备（如键盘、鼠标、磁盘等）产生的中断信号会使CPU从用户态切换到内核态。操作系统会处理这些中断，执行相应的中断处理程序，然后再将CPU切换回用户态。

并行和并发是什么

> - 并行是在同一时刻执行多个任务。
> - 并发是在相同的时间段内执行多个任务，任务可能交替执行，通过调度实现。

并行是指在同一时刻执行多个任务，这些任务可以同时进行，每个任务都在不同的处理单元（如多个CPU核心）上执行。在并行系统中，多个处理单元可以同时处理独立的子任务，从而加速整体任务的完成。

并发是指在相同的时间段内执行多个任务，这些任务可能不是同时发生的，而是交替执行，通过时间片轮转或者事件驱动的方式。并发通常与任务之间的交替执行和任务调度有关。

什么是内部碎片和外部碎片

说一说僵尸进程和孤儿进程

1. 孤儿进程：一个父进程退出，而它的一个或多个子进程还在运行，那么那些子进程将成为孤儿进程。孤儿进程将被init进程(进程号为1)所收养，并由init进程对它们完成状态收集工作。
2. 僵尸进程：一个进程使用fork创建子进程，如果子进程退出，而父进程并没有调用wait或waitpid获取子进程的状态信息，那么子进程的进程描述符仍然保存在系统中。这种进程称之为僵尸进程。

信号和信号量有什么区别

信号：一种处理异步事件的方式。信号是比较复杂的通信方式，用于通知接收进程有某种事件发生，除了用于进程外，还可以发送信号给进程本身。
信号量：进程间通信处理同步互斥的机制。是在多线程环境下使用的一种设施，它负责协调各个线程，以保证它们能够正确，合理的使用公共资源。

常用的Linux命令

```
ls：列出当前目录下的文件和子目录。

cd：切换目录。

pwd：显示当前工作目录的路径。

touch：创建新文件。

mkdir：创建新目录。

rm：删除文件或目录。

cp：复制文件或目录。

mv：移动文件或目录，也用于重命名。

cat：显示文件的内容。

vi：编辑文件

head 和 tail：查看文件的开头和结尾部分。

grep：查找文件或其他内容里符合条件的字符串

find：查找文件和目录。

chmod：更改文件或目录的权限。

chown：更改文件或目录的所有者。

ps：列出运行中的进程。

kill：终止进程。

df：显示磁盘空间使用情况。

tar：创建和提取归档文件（通常是.tar文件）。

ifconfig：查看ip地址
```

```
ping：测试网络连接。

ssh：通过SSH协议远程登录到其他计算机。

apt（Debian/Ubuntu）或 yum（Red Hat/CentOS）：包管理器，用于安装、更新和删除软件包。
```

Linux中如何查看一个进程

`ps`命令用于列出当前用户的进程信息如果要查找特定进程。

可以使用 `grep` 命令来过滤结果。例如，要查找名为 "myprocess" 的进程，可以执行以下命令：

```
ps aux | grep myprocess
```

`pgrep` 和`pidof` 命令用于通过进程名称查找进程的PID（进程ID）

```
pgrep myprocess
pidof myprocess
```

如何杀死一个进程

使用`kill`命令可以向一个进程发送信号，终止进程， `kill [options] PID`

`killall`命令用于根据进程名杀死所有匹配的进程。

杀死父进程并不会同时杀死子进程：每个进程都有一个父进程。可以使用 pstree 或 ps 工具来观察这一点。

杀死父进程后，子进程将会成为孤儿进程，而 init 进程将重新成为它的父进程。

局部性原理

在一段时间内，程序倾向于多次访问相同的数据或接近的数据，而不是随机地访问内存中的各个位置。局部性原理通常分为两种类型：时间局部性和空间局部性。

1. 时间局部性

如果一个数据被访问，那么在不久的将来它很可能会再次被访问。这意味着程序在短时间内倾向于反复使用相同的数据项，例如在循环中反复访问数组的元素。

通过利用时间局部性，程序可以将频繁使用的数据存储在缓存中，从而减少访问主内存的次数，提高程序的执行速度。

2. 空间局部性

如果一个数据被访问，那么它附近的数据也很可能会被访问。这意味着程序在访问一个数据时，通常会在接近该数据的附近访问其他数据，例如遍历数组时，往往会访问相邻的元素。

文件系统在磁盘上存储数据时，通常会将相关的数据块放在相邻的磁盘扇区上，以便在访问一个数据块时能够快速地访问相邻的数据块。

进程、线程的区别?

进程是系统进行资源分配和调度的基本单位。

线程是操作系统能够进行运算调度的最小单位，线程是进程的子任务，是进程内的执行单元。 一个进程至少有一个线程，一个进程可以运行多个线程，这些线程共享同一块内存。

资源开销：

- 进程：由于每个进程都有独立的内存空间，创建和销毁进程的开销较大。进程间切换需要保存和恢复整个进程的状态，因此上下文切换的开销较高。
- 线程：线程共享相同的内存空间，创建和销毁线程的开销较小。线程间切换只需要保存和恢复少量的线程上下文，因此上下文切换的开销较小。

通信与同步：

- 进程：由于进程间相互隔离，进程之间的通信需要使用一些特殊机制，如管道、消息队列、共享内存等。
- 线程：由于线程共享相同的内存空间，它们之间可以直接访问共享数据，线程间通信更加方便。

安全性：

- 进程：由于进程间相互隔离，一个进程的崩溃不会直接影响其他进程的稳定性。
- 线程：由于线程共享相同的内存空间，一个线程的错误可能会影响整个进程的稳定性。

进程的状态有哪些

进程的3种基本状态：**运行、就绪和阻塞。**

（1）就绪：当一个进程获得了除处理机以外的一切所需资源，一旦得到处理机即可运行，则称此进程处于就绪状态。就绪进程可以按多个优先级来划分队列。例如，当一个进程由于时间片用完而进入就绪状态时，排入低优先级队列；当进程由I / O操作完成而进入就绪状态时，排入高优先级队列。

（2）运行：当一个进程在处理机上运行时，则称该进程处于运行状态。处于此状态的进程的数目小于等于处理器的数目，对于单处理机系统，处于运行状态的进程只有一个。在没有其他进程可以执行时（如所有进程都在阻塞状态），通常会自动执行系统的空闲进程。

（3）阻塞：也称为等待或睡眠状态，一个进程正在等待某一事件发生（例如请求I/O而等待I/O完成等）而暂时停止运行，这时即使把处理机分配给进程也无法运行，故称该进程处于阻塞状态。

进程的五种状态

创建状态：进程在创建时需要申请一个空白PCB，向其中填写控制和管理进程的信息，完成资源分配。如果创建工作无法完成，比如资源无法满足，就无法被调度运行，把此时进程所处状态称为创建状态

就绪状态：进程已经准备好，已分配到所需资源，只要分配到CPU就能够立即运行

执行状态：进程处于就绪状态被调度后，进程进入执行状态

阻塞状态：正在执行的进程由于某些事件（I/O请求，申请缓存区失败）而暂时无法运行，进程受到阻塞。在满足请求时进入就绪状态等待系统调用

终止状态：进程结束，或出现错误，或被系统终止，进入终止状态。无法再执行

进程之间的通信方式

1. **管道：**是一种**半双工**的通信方式，数据只能单向流动而且只能在具有父子进程关系的进程间使用。
2. 命名管道： 也是半双工的通信方式，但是它允许无亲缘关系进程间的通信。
3. **信号量**：是一个计数器，可以用来控制多个进程对共享资源的访问，常作为一种锁机制，防止某进程正在访问共享资源时，其他进程也访问该资源。因此主要作为进程间以及同一进程内不同线程之间的同步手段。
4. **消息队列**：消息队列是消息的链表，存放在内核中并由消息队列标识符标识。消息队列克服了信号传递信息少、管道只能承载无格式字节流以及缓冲区大小受限等缺点。
5. 信号：用于通知接收进程某个事件已经发生，从而迫使进程执行信号处理程序。
6. **共享内存**：就是映射一段能被其他进程所访问的内存，这段共享内存由一个进程创建，但多个进程都可以访问。共享内存是最快的进程通信方式，它是针对其他进程间通信方式运行效率低而专门设计的。它往往与其他通信机制，比如信号量配合使用，来实现进程间的同步和通信。
7. `Socket`套接字：是支持TCP/IP 的网络通信的基本操作单元，主要用于在客户端和服务器之间通过网络进行通信。

线程之间的同步方式

线程同步机制是指在多线程编程中，为了保证线程之间的互不干扰，而采用的一种机制。常见的线程同步机制有以下几种：

1. 互斥锁：互斥锁是最常见的线程同步机制。它允许只有一个线程同时访问被保护的临界区（共享资源）
2. 条件变量：条件变量用于线程间通信，允许一个线程等待某个条件满足，而其他线程可以发出信号通知等待线程。通常与互斥锁一起使用。
3. 读写锁： 读写锁允许多个线程同时读取共享资源，但只允许一个线程写入资源。
4. 信号量：用于控制多个线程对共享资源进行访问的工具。

介绍一下你知道的锁

两个基础的锁：

- **互斥锁**：互斥锁是一种最常见的锁类型，用于实现互斥访问共享资源。在任何时刻，只有一个线程可以持有互斥锁，其他线程必须等待直到锁被释放。这确保了同一时间只有一个线程能够访问被保护的资源。
- 自旋锁：自旋锁是一种基于忙等待的锁，即线程在尝试获取锁时会不断轮询，直到锁被释放。

其他的锁都是基于这两个锁的

- 读写锁：允许多个线程同时读共享资源，只允许一个线程进行写操作。分为读（共享）和写（排他）两种状态。
- 悲观锁：认为多线程同时修改共享资源的概率比较高，所以访问共享资源时候要上锁
- 乐观锁：先不管，修改了共享资源再说，如果出现同时修改的情况，再放弃本次操作

什么情况下会产生死锁

死锁是指**两个或多个进程在争夺系统资源时，由于互相等待对方释放资源而无法继续执行的状态。**

死锁只有同时满足以下四个条件才会发生：

- 互斥条件：一个进程占用了某个资源时，其他进程无法同时占用该资源。
- 请求保持条件：一个线程因为请求资源而阻塞的时候，不会释放自己的资源。
- 不可剥夺条件：资源不能被强制性地从一个进程中剥夺，只能由持有者自愿释放。

- 环路等待条件：多个进程之间形成一个循环等待资源的链，每个进程都在等待下一个进程所占有的资源。

如何解除死锁

只需要破坏上面一个条件就可以破坏死锁。

- 破坏请求与保持条件：一次性申请所有的资源。
- 破坏不可剥夺条件：占用部分资源的线程进一步申请其他资源时，如果申请不到，可以主动释放它占有的资源。
- 破坏循环等待条件：靠**按序申请资源**来预防。让所有进程按照相同的顺序请求资源，释放资源则反序释放。

进程的调度算法有哪些?

1、批处理系统中的调度

（1）先来先服务：

非抢占式的调度算法，按照请求的顺序进行调度。

有利于长作业，但不利于短作业，因为短作业必须一直等待前面的长作业执行完毕才能执行，而长作业又需要执行很长时间，造成了短作业等待时间过长。

（2）最短作业优先：

非抢占式的调度算法，按估计运行时间最短的顺序进行调度。

长作业有可能会饿死，处于一直等待短作业执行完毕的状态。因为如果一直有短作业到来，那么长作业永远得不到调度。

（3）最短剩余时间优先：

最短作业优先的抢占式版本，按剩余运行时间的顺序进行调度。

当一个新的作业到达时，其整个运行时间与当前进程的剩余时间作比较。如果新的进程需要的时间更少，则挂起当前进程，运行新的进程。否则新的进程等待。

2、交互式系统中的调度

（1）时间片轮转调度

将所有就绪进程按 FCFS 的原则排成一个队列，每次调度时，把 CPU 时间分配给队首进程，该进程可以执行一个时间片。当时间片用完时，由计时器发出时钟中断，调度程序便停止该进程的执行，并将它送往就绪队列的末尾，同时继续把 CPU 时间分配给队首的进程。

（2）优先级调度

为每个进程分配一个优先级，按优先级进行调度。为了防止低优先级的进程永远等不到调度，可以随着时间的推移增加等待进程的优先级。

（3）多级队列

一个进程需要执行 100 个时间片，如果采用时间片轮转调度算法，那么需要交换 100 次。

多级队列是为这种需要连续执行多个时间片的进程考虑，它设置了多个队列，每个队列时间片大小都不同，例如 1,2,4,8,..。进程在第一个队列没执行完，就会被移到下一个队列。这种方式下，之前的进程只需要交换 7 次。

每个队列优先权也不同，最上面的优先权最高。因此只有上一个队列没有进程在排队，才能调度当前队列上的进程。

可以将这种调度算法看成是时间片轮转调度算法和优先级调度算法的结合。

（4）最短进程优先

如果我们将每一条命令的执行看作是一个独立的“作业”，则我们可以通过首先运行最短的作业来使响应事件最短

分段和分页的区别

分段

- **基本单位：** 地址空间被划分为不同的逻辑段，每个段具有独立的含义，如代码段、数据段等。
- **段的长度：** 每个段的长度可以动态变化，不同段的长度可以不同。
- **内部碎片：** 由于每个段的长度可以动态变化，可能会导致内部碎片，即段内部的未使用空间。
- **外部碎片：** 可能会导致外部碎片，即段之间的未使用空间。
- **逻辑地址：** 逻辑地址由两部分组成，一个是段号，另一个是段内偏移。

分页（Paging）：

- **基本单位：** 地址空间被划分为固定大小的页面，物理内存也被划分为相同大小的页面框。
- **页面的长度：** 页面的长度是固定的，由操作系统定义。
- **内部碎片：** 由于页面长度固定，可能会导致内部碎片，即页面内部的未使用空间。
- **外部碎片：** 由于页面长度固定，不同页面之间的未使用空间无法利用，可能导致外部碎片。
- **逻辑地址：** 逻辑地址由两部分组成，一个是页号，另一个是页内偏移。

页面置换算法

- **LRU（最近最少使用）算法**：每次选择最长时间没有被使用的角色进行切换。这种策略基于你对角色的喜好，认为最近被使用过的角色很可能还会被使用，而最久未被使用的角色很可能不会再被使用。LRU算法可以有效地减少切换次数，但是实现起来比较复杂，需要记录每个角色的使用时间或者维护一个使用顺序的列表。
- **FIFO（先进先出）算法**：每次选择最早进入内存的角色进行切换。这种策略很简单，只需要维护一个角色队列，每次淘汰队首的角色，然后把新的角色加入队尾。但是FIFO算法可能会淘汰一些经常被使用的角色，导致切换次数增加。而且FIFO算法有可能出现贝拉迪异常（Belady anomaly），即当分配给内存的空间增加时，切换次数反而增加。
- 最佳页面置换算法(OPT)

  置换在「未来」最长时间不访问的页面,但是实际系统中无法实现，因为程序访问页面时是动态的

  我们是无法预知每个页面在「下一次」访问前的等待时间，因此作为实际算法效率衡量标准。
- 时钟页面置换算法：把所有的页面都保存在一个类似钟面的「环形链表」中，页面包含一个访问位。

  当发生缺页中断时，顺时针遍历页面，如果访问位为1，将其改为0，继续遍历，直到访问到访问位为0页面，进行置换。
- 最不常用算法 ：记录每个页面访问次数，当发生缺页中断时候，将访问次数最少的页面置换出去，此方法需要对每个页面访问次数统计，额外开销。

IO多路复用

I/O多路复用通常通过select、poll、epoll等系统调用来实现。

- **select：** select是一个最古老的I/O多路复用机制，它可以监视多个文件描述符的可读、可写和错误状态。然而，但是它的效率可能随着监视的文件描述符数量的增加而降低。
- **poll：** poll是select的一种改进，它使用**轮询方式**来检查多个文件描述符的状态，避免了select中文件描述符数量有限的问题。但对于大量的文件描述符，poll的性能也可能变得不足够高效。
- **epoll：** epoll是Linux特有的I/O多路复用机制，相较于select和poll，它在处理大量文件描述符时更加高效。epoll使用事件通知的方式，只有在文件描述符就绪时才会通知应用程序，而不需要应用程序轮询。

I/O多路复用允许在一个线程中处理多个I/O操作，避免了创建多个线程或进程的开销，允许在一个线程中处理多个I/O操作，避免了创建多个线程或进程的开销。

数据库-MySQL

> 内容参考总结自网络资料，如小林coding、MySQL45讲

基础知识讲解

MySQL基础

主键、索引、外键

1、什么是主键

主键是一列，其值可以**唯一标识**表中的每一行数据，每个表只能有一个主键，而且主键的值不能重复，也不能包含NULL值，通常用来保证数据的唯一性和用于在表中查找特定的行。

2、主键、外键、索引的区别

定义：

主键：唯一标识一条记录，不允许重复，不允许为空
外键：外键是一个表中的字段，其值是另一个表的主键，用于建立两个表之间的关系。

索引：没有重复值，但可以有一个空值 ，用于快速查询到数据。

作用：

主键：用于唯一标识表中每一行的字段

外键：主要用于和其它表建立联系
索引：为了提高查询排序的速度

区别：

外键是一个表中的字段，它与另一个表的主键形成关联，用于建立表之间的关系。

主键和外键通常都与索引有关，但索引不一定是主键或外键。

MySQL表连接

连接和笛卡尔积

连接：

将各个表中的记录都取出来进行依次匹配，将匹配后的结果发给客户端

笛卡尔积：

连接查询的结果中包含一个表的每一条记录与另一个表中每一条记录的组合，这样的查询结果就是笛卡尔积

比如表a有5条记录；表b有6条记录；a和b的笛卡尔积就是30

连接过程

1、确定第一个需要查询的表，此表为`驱动表`

2、从驱动表中取每一条符合搜索条件的记录，到接下来的表中查找匹配的记录；驱动表之后的那个表就叫`被驱动表`

只需要访问驱动表一次，可能会多次访问被驱动表

每获得一条满足条件的驱动表记录，马上到被驱动表中寻找匹配的记录

内/外连接

内连接

驱动表中的记录在被驱动表中找不到匹配的记录，那么驱动表的这条记录不会加入到最后的结果中

```
select * from 驱动表, 被驱动表;
select * from 驱动表 join 被驱动表;
select * from 驱动表 inner join 被驱动表;
select * from 驱动表 cross join 被驱动表;
```

外连接：

驱动表中的记录在被驱动表中找不到匹配的记录，也仍需要加入到最后结果中

左外连接：语句左侧的表为驱动表

```
select * from 驱动表 left join 被驱动表 on 连接条件;
```

右外连接：语句右侧的表为驱动表

```
select * from 被驱动表 right join 驱动表 on 连接条件;
```

对于内连接，驱动表和被驱动表的顺序可以更换；对于外连接，这个顺序不能随意更换

过滤条件

where：

不论内外连接，只要是不符合 where 子句的记录都不会加入到最后的结果中

on：

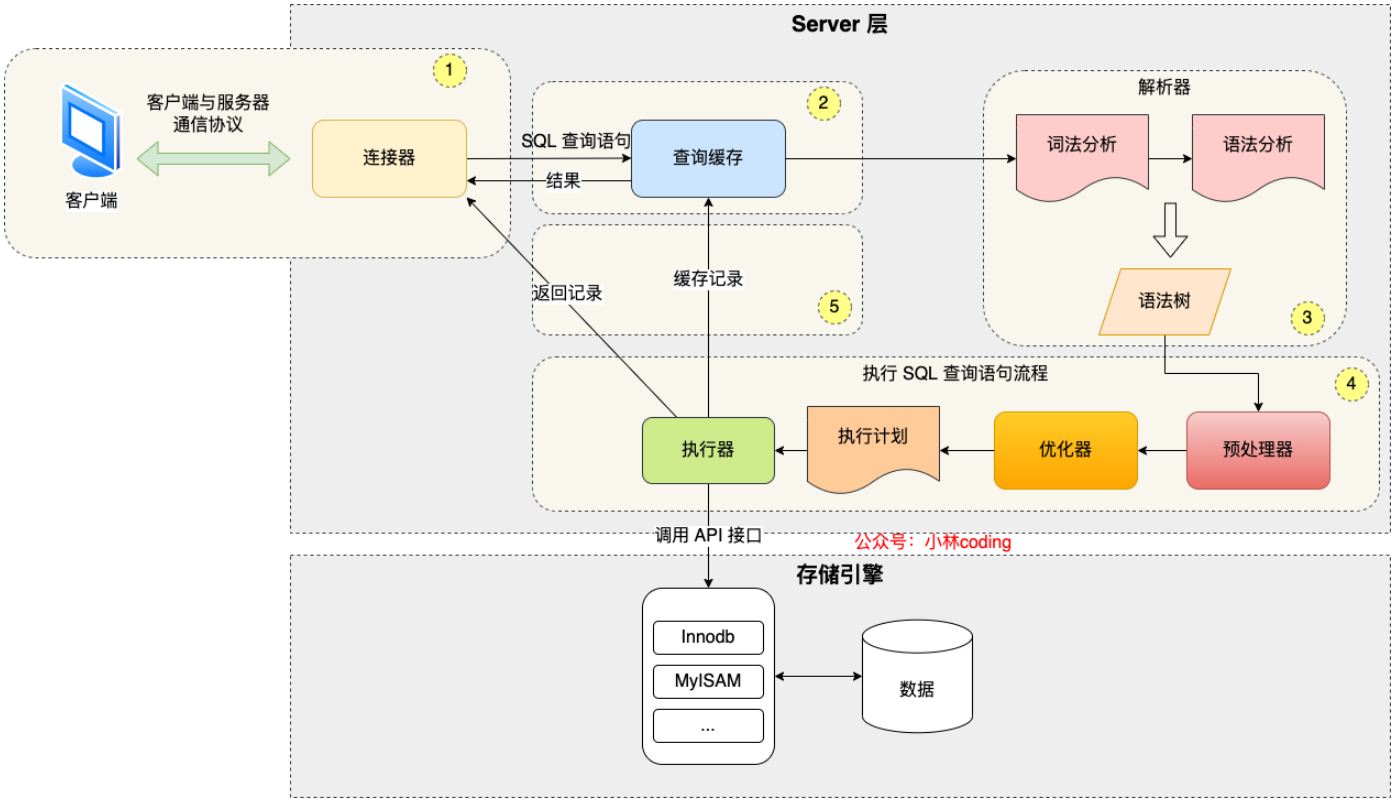
在内连接中与 where 等价；

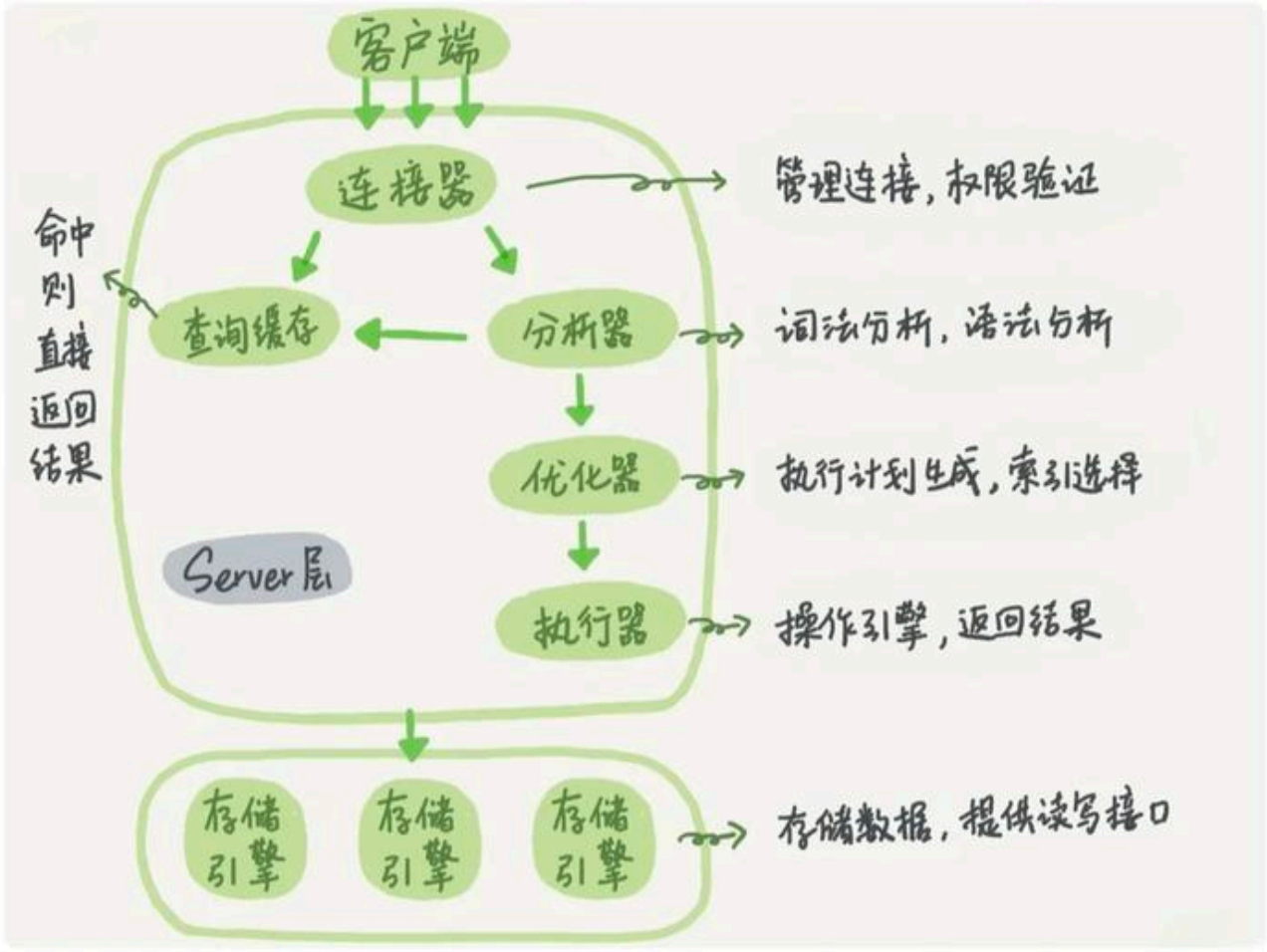
在外连接中，如果驱动表中的记录在被驱动表中没有记录可以匹配，该驱动表记录仍会加入到结果中，对应的被驱动表字段以 null 填充

嵌套循环连接

如果有3个表进行连接，那么表1和表2完成连接后的结果作为驱动表，将表3作为被驱动表进行连接查询

执行一次 select 语句，发生了什么





MySQL 的逻辑架构图

MySQL 的架构共分为两层：**Server 层和存储引擎层**

- **Server 层负责建立连接、分析和执行 SQL**
- **存储引擎层负责数据的存储和提取**, 支持 InnoDB、MyISAM、Memory 等多个存储引擎, 现在最常用的存储引擎是 InnoDB，其支持索引类型是 B+ 树索引。

1. 连接器：管理连接和权限验证

连接器跟客户端建立连接、获取权限、然后后面的权限逻辑判断都基于此时读取到的权限。

Mysql 会定期自动清理"空闲"连接，由参数 `wait_timeout` 控制的，默认值是 8 小时。
由于建立连接比较复杂，所以尽量使用长连接，而不是 短连接（少量查询后，就断开连接)
但是，当 长连接 过多时，可能导致内存占用太大，被系统强行杀掉（OOM），会导致 MySQL 异常重启

解决方案为:

- 定期断开长连接
- MySQL 5.7以后版本，可以通过执行 `mysql_reset_connection` 来重新初始化连接资源

2. 查询缓存

执行查询语句前，先看下查询缓存中是否有结果

- 如果有，则不必执行查询语句，直接取出缓存结果
- 如果没命中缓存，则执行查询语句，并将执行查询语句后的结果，放入查询缓存中

不建议使用查询缓存（当 数据表频繁更新时，最新查询结果可能和查询缓存中存放的结果不一致)

> MySQL 8.0 开始，执行一条 SQL 查询语句，不会再走到查询缓存这个阶段了。

3. 解析SQL

你输入的是由多个字符串和空格组成的一条SQL语句，MySQL需要识别出里面的字符串分别是什么，代表什么。

- 词法分析：根据你输入的字符串识别出关键字出来
- 语法分析：根据词法分析的结果判断是否符合SQL语法，并构建SQL语法课

4. 执行SQL

分为三个阶段：预处理阶段、优化阶段、执行阶段

预处理阶段：判断表和字段是否存在

优化阶段：**将 SQL 查询语句的执行方案确定下来**，比如在表里面有多个索引的时候，优化器会基于查询成本的考虑，来决定选择使用哪个索引, 或者在一个语句有多表关联（join）的时候，决定各个表的连接顺序。

执行阶段：MySQL通过分析器知道了你要做什么，通过优化器知道了该怎么做，于是就进入了执行器阶段，开始执行语句（执行语句时，首先会判断当前用户是否有执行权限）。

执行一次update语句，发生了什么

查询语句的流程，更新语句同样需要走一遍

1. 执行语句前先连接数据库
2. 数据库表有更新，跟表有关的查询缓存会失效，所以会清空之前的缓存结果
3. 分析器进行词法和语法解析，优化器进行优化，执行器负责执行，然后更新。

但是更新流程涉及到了两个重要的日志模块：`redo log（重做日志）`和`binlog(归档日志)`, 具体会在日志章节中讲解。

`redo log`和`binlog`有什么不同?

- `redo log` 是 InnoDB 引擎特有的；`binlog`是 MySQL 的 Server 层实现的，所有引擎都可以使用。
- `redo log` 是物理日志，记录的是“在某个数据页上做了什么修改”；`binlog` 是逻辑日志，记录的是这个语句的原始逻辑，比如“给 ID=2 这一行的 c 字段加 1
- `redo log` 是循环写的，空间固定会用完；`binlog` 是可以追加写入的。“追加写”是指 binlog 文件写到一定大小后会切换到下一个，并不会覆盖以前的日志。

MySQL语句

目的：为了保证MySql服务器的安全，每个MySql的用户应该对他们需要的数据具有适当的访问权;

例如:

1. 多数用户只能够用到读写权限，只有少数用户能够创建表格和删除表格
2. 一些用户只能访问某个数据库而不能访问其他的数据库
3. 一些用户可以管理其他用户的账号，但大多数人不需要管理别人的账号

每个用户需要用到什么权限，就只给他们什么权限

如果每个用户的权限都很高，那数据库会变得十分危险，随时都需要面临很多误操作，所以要进行访问权限控制

用户账号存储

在 MySql 下，所有的用户的账号，密码，对数据库的操作权限，都存储在 mysql 下（数据库的名字就叫 mysql）的 user 表格中。
user 表格有一个 user 属性，user 属性就是当前所有的用户名，通过以下操作可以查看当前 mysql 的用户一共有多少。

```
use mysql
select user from user
```

```
mysql> select user from user
    -> ;
+---------------+
| user          |
+---------------+
| root          |
| ysj           |
| mysql.session |
| mysql.sys     |
| root          |
+---------------+
5 rows in set (0.00 sec)
```

IT码随想录 / 知识星球

charon

用户账号管理

既然 mysql 中的 user 表格中存放着当前 mysql 的用户，那么新增用户只需要在 user 表中使用 insert 语句新增一条记录即可。

不过为了安全起见不建议这样做，mysql 数据库存放着很重要的数据，这样直接对 mysql 数据库直接操作可能会伤害到 mysql 服务器。

1. 新增用户

新增一个用户的语法：

```
CREATE USER [user_name] IDENTIFIED BY [user_pwd]
```

例如：新增用户 zhangsan，pwd=123456

```
CREATE USER zhangsan IDENTIFIED BY '123456'
```

2. 修改用户名

修改用户名的语法：

```
RENAME USER [user_name] TO [new_user_name]
```

例如：例如将账号名从 zhangsan 改为 lisi

```
RENAME USER zhangsan TO lisi
```

3. 修改用户密码

修改用户密码语法：

```
SET PASSWORD FOR [user_name] = Password('[new_pwd]')
```

这里使用 password 函数对新密码进行加密。

如果不指定用户名：

```
SET PASSWORD = Password('[new_pwd]')
```

默认为修改当前登陆的用户的密码。

例如：修改 lisi 账号的密码：

```
SET PASSWORD FOR lisi = Password('135790')
```

用户账号删除

删除用户账号语法：

```
DROP USER [user_name]
```

例如：删除 lisi 的账号

```
DROP USER lisi
```

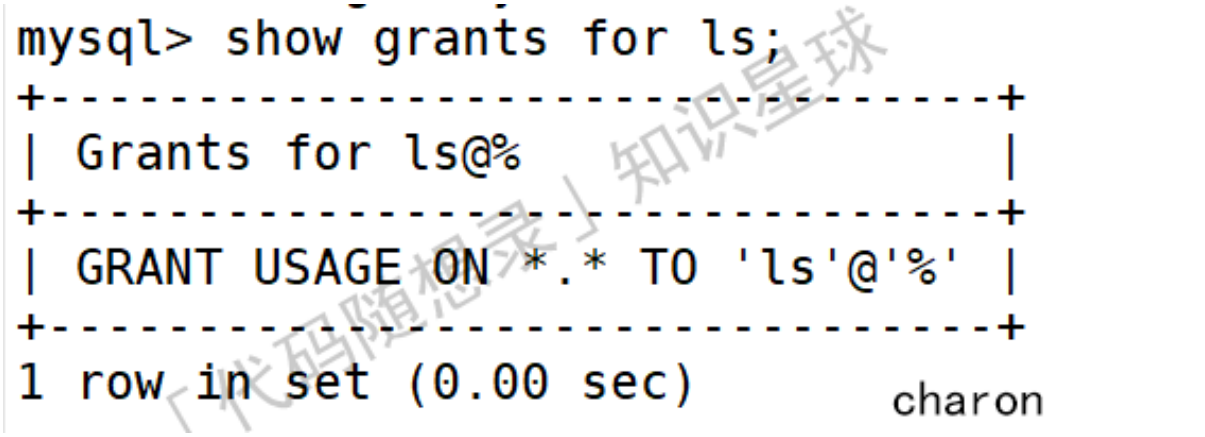
如果mysql版本低于mysql5，需要先revoke用户所有的权限，才能删除账号。更高版本则可以一次性删除权限和账号。

用户权限控制

1. 查看用户权限

刚创建的的用户是没有任何权限的，使用以下语法查看某用户所拥有的权限：

```
SHOW GRANTS FOR [user_name]
```



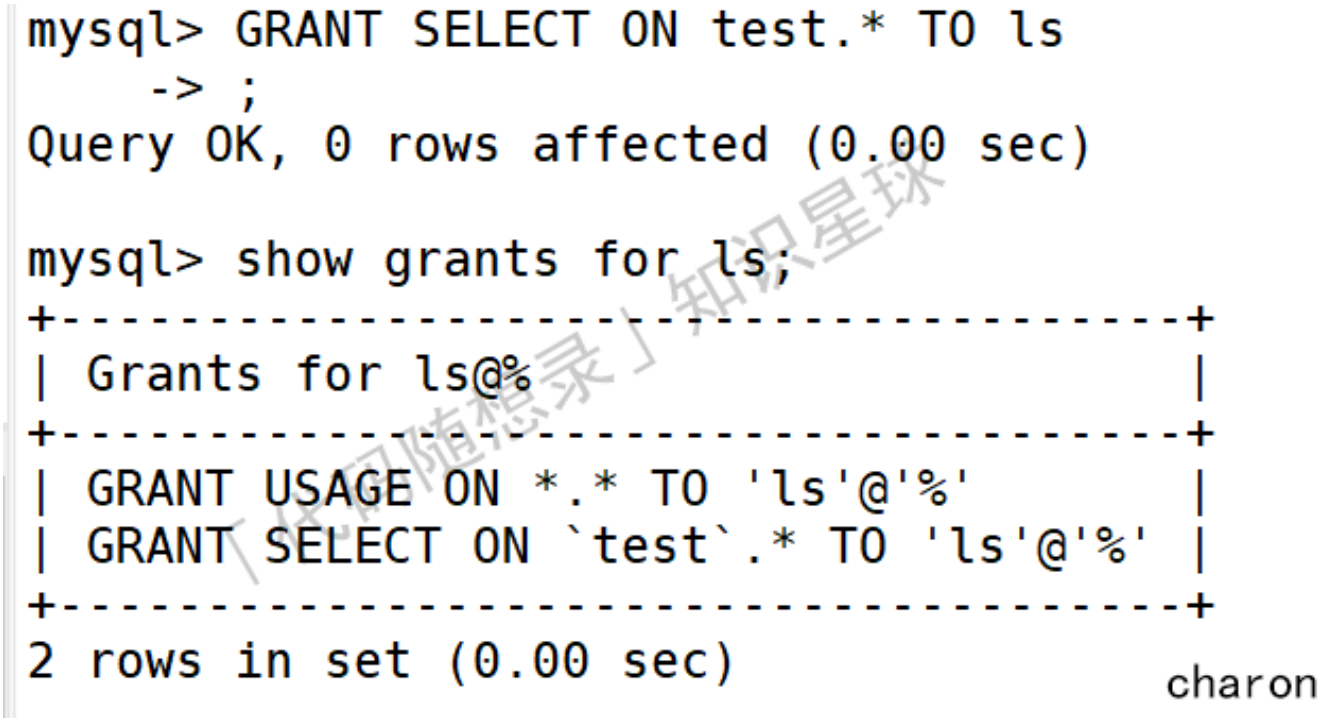
而`GRANT USAGE ON *.* TO 'ls'@'%'`则代表了用户ls没有任何权限。

2. 用户授权

给用户授权的语法：

```
GRANT [权限名] ON [数据库名].[表名] TO [用户名]
```

例如：用户 'ls' 查找权限 select



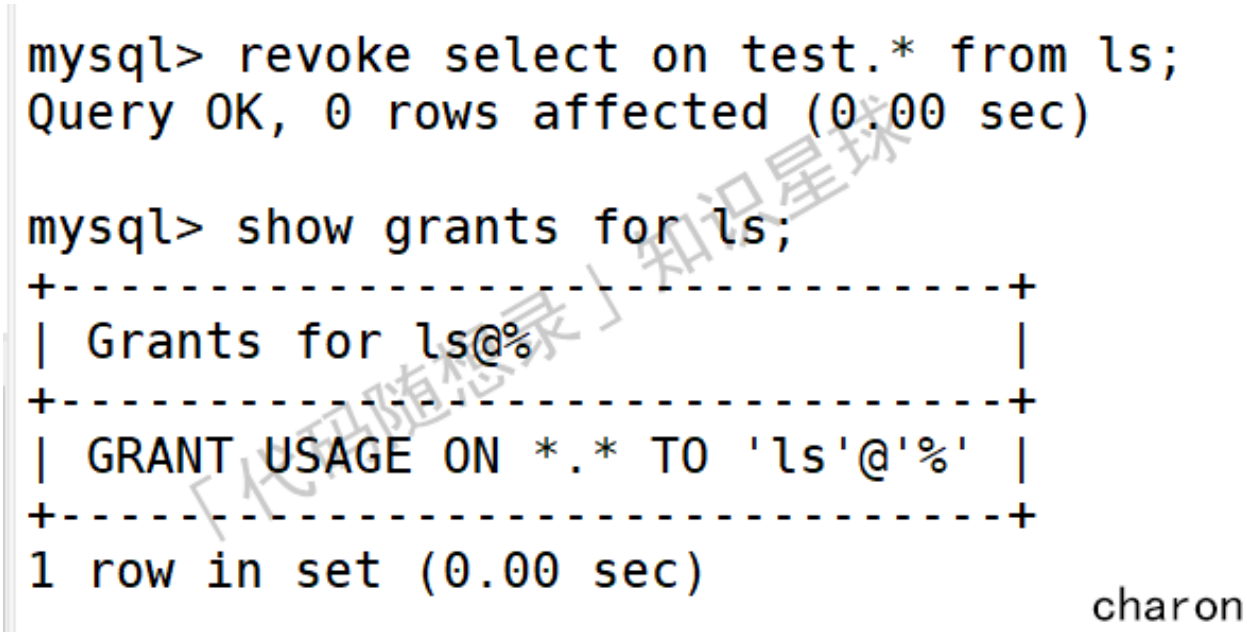
从表格中可以看到 'ls' 新增的权限。

3. 撤销授权

撤销用户权限的语法：

```
REVOKE [权限名] ON [数据库名].[表名] FROM [用户名]
```

收回用户 ls 查找权限 select



从表格中可以看初 ls 新增的权限消失了

索引

什么是索引

索引：索引的出现其实就是为了提高数据查询的效率，就像书的目录一样

MySQL 在查询方面主要就是**两种方式：**

1. 全表扫描（一个一个挨个找）
2. 根据索引检索

索引操作

1、创建索引

```
create index 索引名 on 表名(列名);
# 也可以有多个列名构成索引（即，组合索引）
```

2、删除索引

```
drop index 索引名 on 表名;
```

怎么查看一个SQL语句是否使用了索引进行检索

在 SQL 语句前，添加 explian 关键字

- 当 `type=ALL` 时，表示使用 全表查询（未使用索引）
- 当 `type=RES` 时，表示使用索引

索引有哪些类别

- 按「数据结构」分类：**B+tree索引、Hash索引、Full-text索引**。
- 按「物理存储」分类：**聚簇索引（主键索引）、二级索引（辅助索引）**。
- 按「字段特性」分类：**主键索引、唯一索引、普通索引、前缀索引**。
- 按「字段个数」分类：**单列索引、联合索引**。

1. 按照数据结构分类

- 哈希表：使用 key-value 对，存储数据 ，可能存在 hash 冲突（多个 value 对应同一个 key）
    - 优点：key 无序，插入数据时无需维护顺序（直接在最后一个元素后追加，即可），效率较高；
    - 缺点：因为不是有序的，所以哈希索引做区间查询的速度是很慢
    - 适用场景：适用于只有等值查询的场景，而不适用频繁 区间查找
- 有序数组：
    - 优点：等值查询 和 区间查询 性能都挺6 （有序数组 适合 查询）
    - 缺点：有序数组不适合 频繁 增/删 记录的场景 （但是，有序数组不适合 增/删）
    - 等值查询使用二分（前提是，有序 + 顺序存储），查询时间复杂度 O(log N)
    - 区间查询先用 二分查找 “左边界”（不存在的时，则找第一个比 它大的记录），然后，再向右扫描，直至 大于右边界
    - 适用场景：有序数组只适用于静态存储引擎，在 等值查询 和 范围（区间）查询 场景中的性能非常优秀
- 二叉搜索树
    - BST 是为了 保留了 “有序数组” 查询（二分，O(logN)）性能好的优点，同时解决“有序数组” 不适合 增/删 的缺点
    - BST 查询的时间复杂度：O(log N) ，但是，为了维持 O(log(N)) 的查询复杂度，就需要保持这棵树是平衡二叉树。（维护 BST 是 AVL 的时间复杂度也是 O(log(N))）
    - 二叉树是搜索效率最高的，但是实际上大多数的数据库存储却并不使用二叉树。其原因是，索引不止存在内存中，还要写到磁盘上
- N叉树

你可以想象一下一棵 100 万节点的平衡二叉树，树高 20。一次查询可能需要访问 20 个数据块。在机械硬盘时代，从磁盘随机读一个数据块需要 10 ms 左右的寻址时间。也就是说，对于一个 100 万行的表，如果使用二叉树来存储，单独访问一个行可能需要 20 个 10 ms 的时间，这个查询可真够慢的。

为了让一个查询尽量少地读磁盘，就必须让查询过程访问尽量少的数据块。那么，我们就不应该使用二叉树，而是要使用“N 叉”树。这里，“N 叉”树中的“N”取决于数据块的大小。

以 InnoDB 的一个整数字段索引为例，这个 N 差不多是 1200。这棵树高是 4 的时候，就可以存 1200 的 3 次方个值，这已经 17 亿了。考虑到树根的数据块总是在内存中的，一个 10 亿行的表上一个整数字段的索引，查找一个值最多只需要访问 3 次磁盘。其实，树的第二层也有很大概率在内存中，那么访问磁盘的平均次数就更少了。 posper

为了让一个查询尽量少地读磁盘，就必须让查询过程访问尽量少的数据块

那么，我们就不应该使用二叉树，而是要使用 “N 叉”树（N 取决于数据块的大小）

即，通过使用 N 叉树 来降低 树的高度，即 减少读取磁盘的次数（IO是很慢的），提高查询效率 ⭐

N 叉树特点：

折中 考虑了 “查询性能” 和 “读取磁盘的次数”，广泛应用于 数据库引擎中

MySQL 中，索引是在存储引擎层实现的，而且不同存储引擎的索引的工作方式并不一样。而即使多个存储引擎支持同一种类型的索引，其底层的实现也可能不同。

下面以InnoDB 为例，和你分析一下其中的索引模型

InnoDB 的索引模型:

InnoDB 使用了 B+ 树索引模型，每一个索引在 InnoDB 里面对应一棵 B+ 树

假设建表语句如下，则对应的 2 棵 索引树（主键 自动创建一颗；字段 k 创建 一颗索引树），如下所示

```
 create table T
(
    id int primary key, # 主键 自动创建索引
    k int not null,
    name varchar(16),
    index (k)
) # 给字段 k 条件 索引

engine=InnoDB;
```

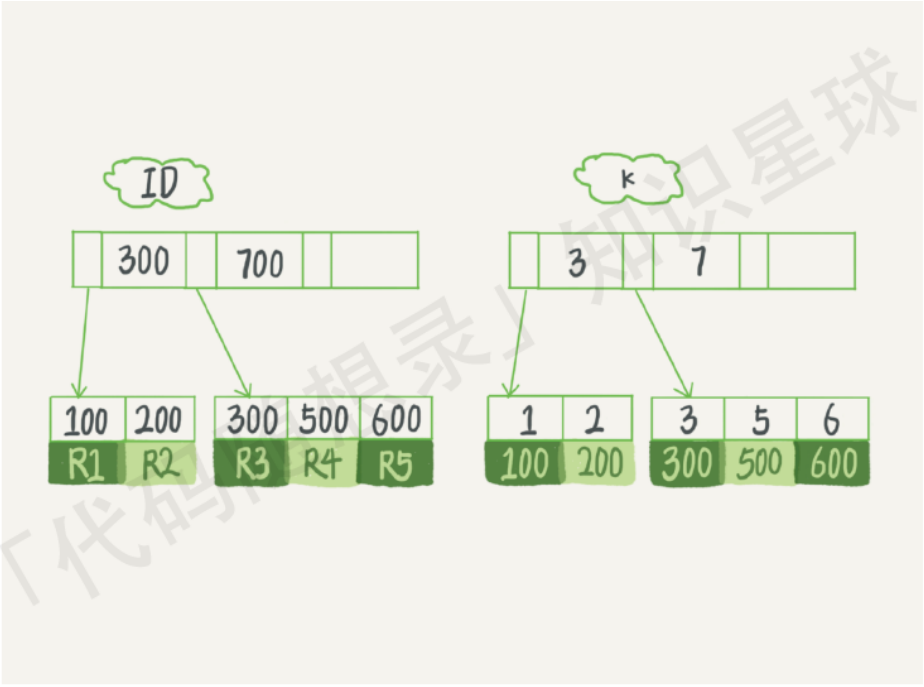


图 4 InnoDB 的索引组织结构

posper

2. 按照存储分类

根据叶子节点的内容，索引类型分为**主键索引（聚簇索引）和二级索引（辅助索引）**：

- 主键索引的 B+Tree 的叶子节点存放的是实际数据，所有完整的用户记录都存放在主键索引的 B+Tree 的叶子节点里；
- 二级索引的 B+Tree 的叶子节点存放的是主键值，而不是实际数据。

主键查询和二级索引查询

主键查询：直接在 主键索引 所在的 B+ 树中查询，然后直接返回查询到的叶子节点（此时，叶子节点里面就是整行记录）

二级索引查询：首先，在普通索引所在的 B+ 树中，查询到待查询记录的 主键； 然后，再根据这些查到的 主键，执行 “主键查询” （即，**回表**）

基于非主键索引的查询需要多扫描一棵索引树。因此，我们在应用中应该尽量使用主键查询。

不过当查询的数据是能在二级索引的 B+Tree 的叶子节点里查询到，这时就不用再查主键索引查，**这种在二级索引的 B+Tree 就能查询到结果的过程就叫作「覆盖索引」**

由于覆盖索引可以减少树的搜索次数，显著提升查询性能，所以使用覆盖索引是一个常用的性能优化手段。

3. 按照字段特性分类

- 主键索引：建立在主键字段上的索引，一张表最多只能有一个主键索引，不允许有空值。
- 唯一索引：建立在`UNIQUE`字段上的索引，一张表可以有多个唯一索引，索引列的值必须唯一，但是允许有空值。
- 普通索引：建立在普通字段上的索引
- 前缀索引：对字符类型字段的前几个字符建立的索引，而不是在整个字段上建立的索引，可以减小索引的大小，适用于较长列值的情况。

5. 按照字段个数分类

- 单列索引：建立在单列上的索引称为单列索引，比如主键索引；
- 联合索引：由多个列组合而成的索引。适用于多列的查询条件

什么是最左匹配原则

通过将多个字段组合成一个索引，该索引就被称为联合索引。

使用联合索引时，存在**最左匹配原则**，也就是按照最左优先的方式进行索引的匹配。

最左匹配原则要求查询条件中的列应该从索引的最左边的列开始，并且不能跳过中间的列。如果查询条件不按照索引的顺序进行匹配，那么索引可能会失效。

举个例子：

- 如果查询条件为 `WHERE column1 = 'value1'`，那么索引可以被有效使用。
- 如果查询条件为 `WHERE column1 = 'value1' AND column2 = 'value2'`，同样索引可以被有效使用。
- 但如果查询条件为 `WHERE column2 = 'value2'` 或 `WHERE column2 = 'value2' AND column3 = 'value3'`，则最左匹配原则不成立。

最左匹配时在遇到范围查询（如 >、<）的时候，就会停止匹配，范围查询的字段可以用到联合索引，但是范围查询字段的后面的字段无法用到联合索引。

索引下推

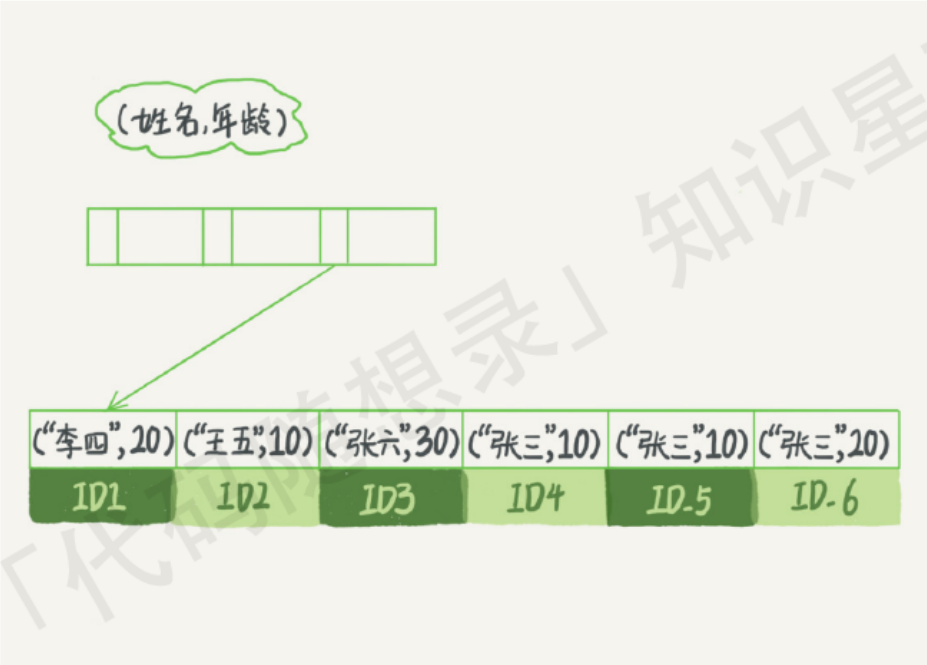


图 2 （ name，age ）索引示意图 posper

那些不符合最左前缀的部分，会怎么样呢?

比如，执行如下语句时候，则存在不符合最左前缀的部分：

```
select * from tuser
where name like '张 %'
and age=10
and ismale=1;
```

根据前缀索引规则，这个语句在搜索索引树的时候，只能用 “张”，找到第一个满足条件的记录 ID3

然后在 MySQL 5.6 之前，只能从 ID3 开始一个个回表，到主键索引上找出数据行，再对比字段值。

而 MySQL 5.6 引入了索引下推优化（index condition pushdown)

可以在联合索引遍历过程中，对联合索引中包含的字段先做判断，直接过滤掉不满足条件的记录，减少回表次数。

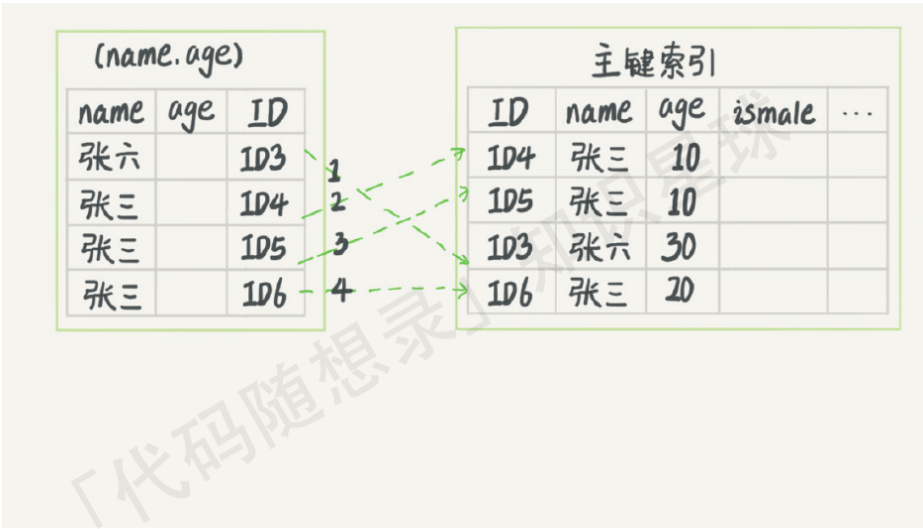


图 3 无索引下推执行流程

posper

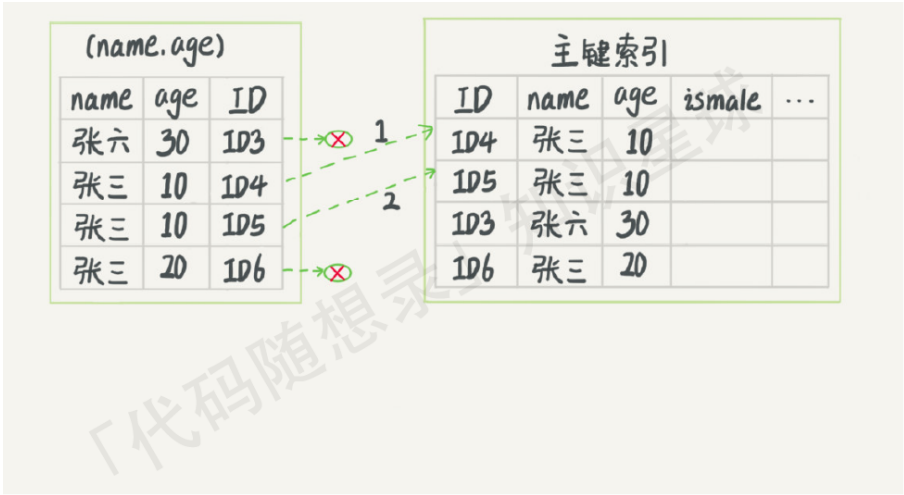


图 4 索引下推执行流程

posper

由图3可知，本例中 无索引优化时，需要 回表 4 次；

图4中，采用 索引下推 优化，直接在索引遍历的过程中，过滤了不符合条件的记录（ID3、ID6），只用回表 2 次

索引区分度

索引区分度表示某个字段不同值的个数占整个表的比例，**建立联合索引时，要把区分度大的字段排在前面**。

建立索引的注意事项

- 索引不是越多越好，虽然索引会提高 select 效率，但是也降低了insert以及update的效率
- 数据量小的表不需要建立索引，会增加额外的索引开销
- 不经常使用的列不要建立索引
- 频繁更新的列不要建立索引，会影响更新的效率

索引的优缺点

优点：

- 使用索引可以**大大加快数据的检索速度**（大大减少检索的数据量），这也是创建索引的最主要的原因。
- 但是注意使用索引不一定能够提高查询性能，因为如果数据库的数据量不大，那么使用索引也不一定能够带来很大提升。其余大多数情况下，索引查询比全表扫描要快。
- 通过创建唯一性索引，可以保证数据库表中每一行数据的唯一性。

缺点：

- 空间消耗，一个索引对应的就是一棵 B+树，每一个节点都是一个 16KB 大小的页。**占用的空间较大**。
- 创建索引和维护索引需要耗费许多时间，当对表中的数据进行增删改的时候，如果数据有索引，那么索引也需要动态的修改，会降低 SQL 执行效率。

什么时候需要创建索引

- 频繁用于查询的列需要创建索引
- 大表：对于很大的表，建立索引可以提高查询速度
- 唯一性要求：主键列和唯一性约束的列会自动创建唯一索引，但如果查询中经常包含唯一性条件，可以额外创建唯一性索引。
- 连接表的外键列：为外键列创建索引可以提高连接的效率。
- 频繁使用排序和分组的列：如果某列经常用于 ORDER BY 或 GROUP BY 子句可以创建索引

什么时候不需要创建索引

- 小表：在小表上，索引不会带来显著的性能提升，还会增加维护开销。
- 经常进行更新的字段不需要创建索引
- 很少用于查询的字段
- 字段中存在大量重复数据，不需要创建索引，比如性别字段

索引优化的方法

- 前缀索引优化：使用某个字段中字符串的前几个字符建立索引，从而减小索引字段大小
- 覆盖索引优化：从二级索引中可以查询得到记录，避免回表
- 主键索引最好是自增的；这样每次插入一条新记录，都是追加操作，不需要重新移动数据，而使用非自增主键会导致插入主键的索引值是随机的，这可能会插入到现有数据页的某个位置，导致其他数据的移动，造成页分裂。
- 避免过多的索引

索引什么时候会失效

- **使用左或者左右模糊匹配**：比如`LIKE '%abc'`这样的查询会导致索引失效。
- **在索引列上使用函数或表达式**： 索引列上参与计算，索引失效

```
SELECT * FROM table WHERE UPPER(column) = 'VALUE';
```

- 在 WHERE 子句中，如果在 OR 前的条件列是索引列，而在 OR 后的条件列不是索引列，那么索引会失效。
- 违背最左匹配原则，索引失效
- **数据分布不均匀：** 如果数据分布不均匀，例如某个索引列的大多数值都相同，选择性降低，导致索引失效。
- **隐式类型转换：** 如果查询中的条件涉及到隐式类型转换，例如将字符串与数字比较，索引可能无法被使用。

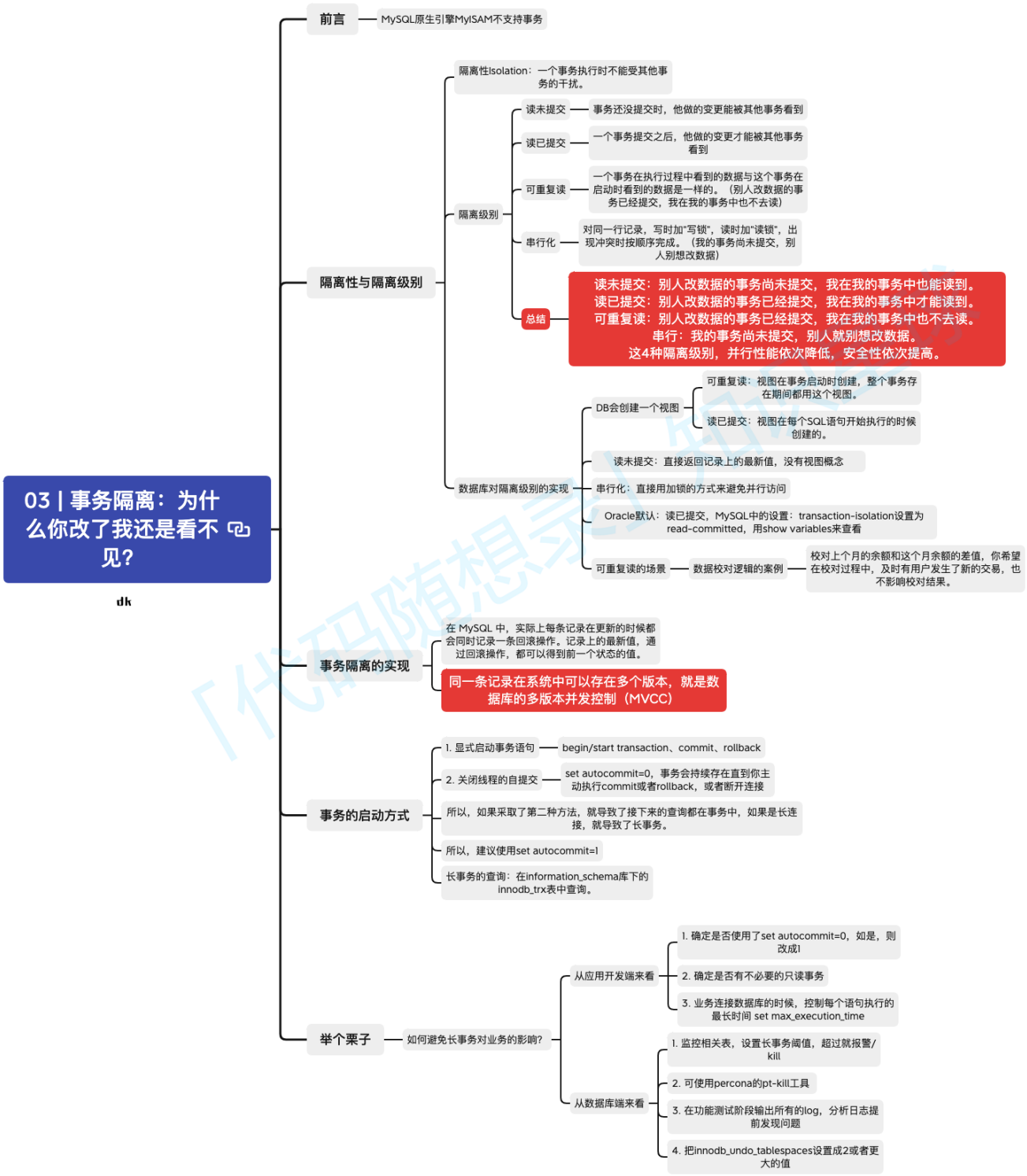
为什么使用B+树索引

- B+树的非叶子节点不存放实际的记录数据，仅存放索引，所以**数据量相同的情况下，相比存储即存索引又存记录的 B 树，B+树的非叶子节点可以存放更多的索引，因此 B+ 树可以比 B 树更「矮胖」，查询底层节点的磁盘 I/O次数会更少**。
- B+ 树有大量的冗余节点（所有非叶子节点都是冗余索引），这些冗余索引让 B+ 树在插入、删除的效率都更高，比如删除根节点的时候，不会像 B 树那样会发生复杂的树的变化；
- B+ 树叶子节点之间用链表连接了起来，有利于范围查询，而 B 树要实现范围查询，因此只能通过树的遍历来完成范围查询，这会涉及多个节点的磁盘 I/O 操作，范围查询效率不如 B+ 树。

事务

思维导图：

原图链接：「代码随想录」知识星球



MySQL原生引擎MyISAM不支持事务，所以被InnoDB取代。

事务的四大特性`ACID`

ACID（Atomicity、Consistency、Isolation、Durability，即原子性、一致性、隔离性、持久性）

1. 原子性

事务是一个不可分割的工作单元，要么完全执行，要么完全不执行。如果在事务执行的过程中发生了错误，系统会撤销事务中已经执行的操作，将数据库恢复到事务开始前的状态。原子性是通过 undo log（回滚日志） 来保证的。

2. 一致性

确保事务将数据库从一个一致的状态转变为另一个一致的状态。事务执行的结果必须满足数据库的完整性约束和规则，保持数据库的一致性。一致性则是通过持久性+原子性+隔离性来保证的。

3. 隔离性

多个事务并发执行时，每个事务都不能看到其他事务的中间状态。每个事务都应该感觉就像它是唯一在数据库上运行的事务一样。防止了多个事务之间的相互干扰。隔离性是通过 MVCC（多版本并发控制） 或锁机制来保证的。

4. 持久性

一旦事务被提交，其结果将永久保存在数据库中，即使系统发生故障。即使系统发生崩溃，事务的结果也不应该丢失，持久性是通过 redo log （重做日志）来保证的。

并行事务会出现什么问题

并行事务是指多个事务同时执行，这可以提高数据库系统的性能和吞吐量。但是并行事务也可能引发一些问题

1. **脏读**：读到其他事务未提交的数据

一个事务读取了另一个事务未提交的数据，如果另一个事务后来回滚，读取的数据就是无效的。读到了并一定最终存在的数据，这就是脏读。

2. **不可重复读**：前后读取的数据不一致

在一个事务内，同一查询可能返回不同的结果，因为在事务执行期间其他事务可能修改了数据。

3. **幻读**：前后读取的记录数量不一致

在一个事务内多次查询某个符合查询条件的「记录数量」，如果出现前后两次查询到的记录数量不一样的情况，就意味着发生了「幻读」现象。

这三个现象严重性排序如下：

隔离级别

隔离级别是指多个并发事务之间相互隔离的程度，SQL标准定义了4个隔离级别

1. 读未提交：

最低的隔离级别。在这个级别下，一个事务可以读取到另一个事务未提交的数据。这可能导致脏读（Dirty Reads）和不可重复读、幻读等问题。

2. 读提交：

在这个级别下，一个事务只能读取到已经提交的其他事务的数据。这解决了脏读的问题，但仍可能遇到不可重复读的问题。

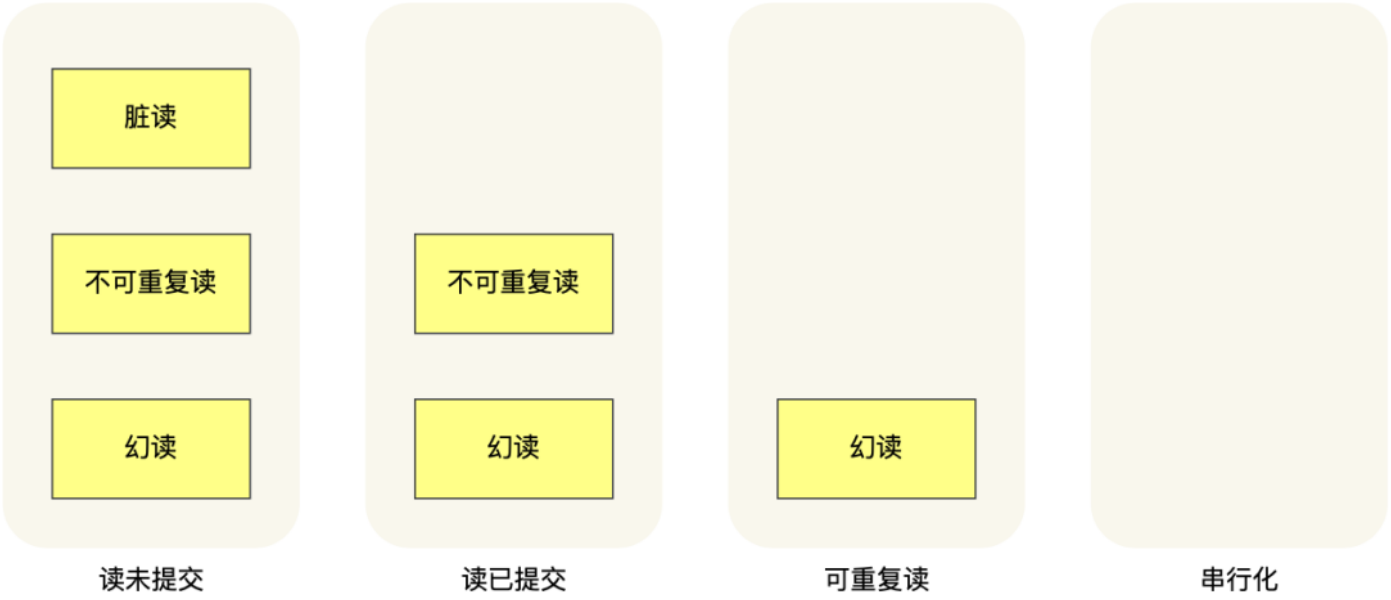
3. 可重复读

在这个级别下，一个事务在其生命周期内多次执行相同的查询，将始终看到相同的数据，但是，仍可能发生幻读。也是**MySQL InnoDB 引擎的默认隔离级别**；

4. 可串行化

提供了最高的隔离级别。会对记录加上读写锁，在多个事务对这条记录进行读写操作时，如果发生了读写冲突的时候，后访问的事务必须等前一个事务执行完成，才能继续执行，在这个级别下，事务的执行效果就好像它们是按顺序执行的，事务之间没有并发。这可以防止脏读、不可重复读和幻读，但也可能导致性能下降，因为并发性降低。

选择隔离级别需要根据应用程序的要求和性能需求进行权衡。较低的隔离级别提供更高的并发性能，但可能牺牲一致性。较高的隔离级别提供更强的一致性，但可能降低并发性能。



- 在「读未提交」隔离级别下，可能发生脏读、不可重复读和幻读现象；
- 在「读提交」隔离级别下，可能发生不可重复读和幻读现象，但是不可能发生脏读现象；
- 在「可重复读」隔离级别下，可能发生幻读现象，但是不可能脏读和不可重复读现象；
- 在「串行化」隔离级别下，脏读、不可重复读和幻读现象都不可能会发生。

幻读是如何解决的

MySQL InnoDB 引擎的默认隔离级别虽然是「可重复读」，但是它很大程度上可以避免幻读现象。解决的方案有两种：

- 针对**快照读**（普通 select 语句），是**通过 MVCC 方式解决了幻读**
- 针对当前读：（select ... for update 等语句），是**通过 next-key lock（记录锁+间隙锁）方式解决了幻读**，因为当执行 select ... for update 语句的时候，会加上 next-key lock，如果有其他事务在 next-key lock 锁范围内插入。

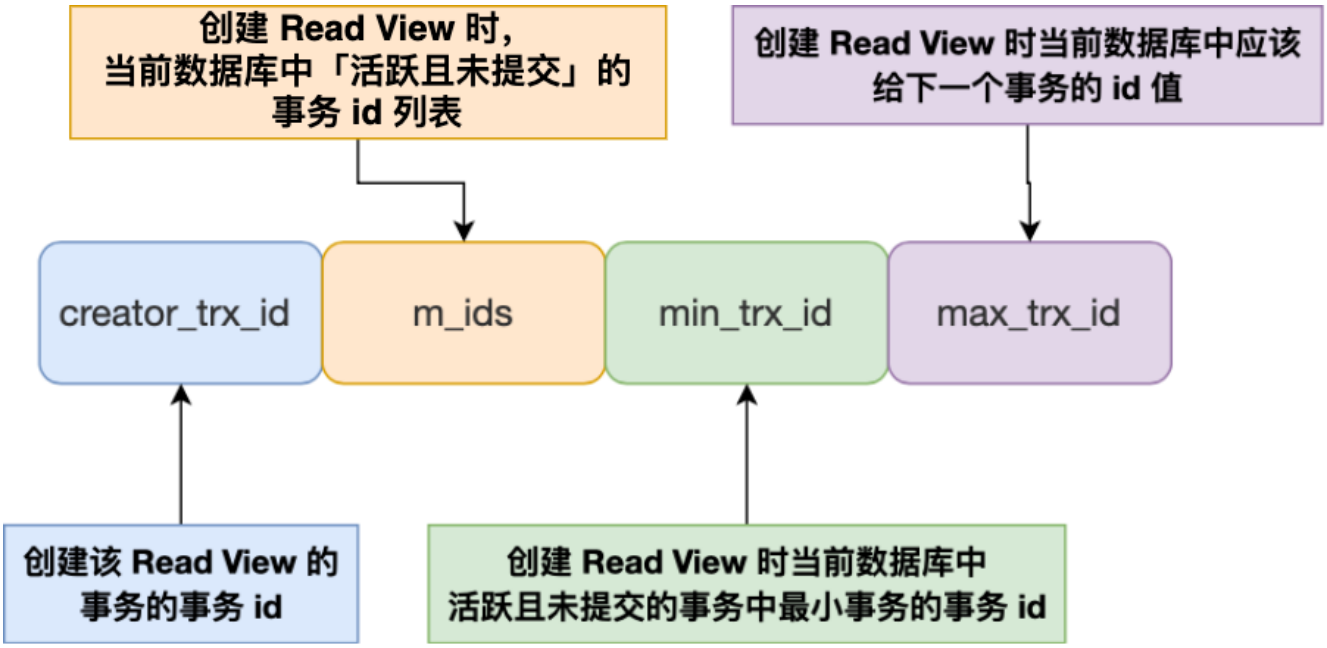
事务隔离的实现

隔离级别具体是如何实现的呢?

- 对于读未提交，可以读到未提交事务修改的数据，所以直接读取最新的数据就可以。
- 对于串行化：加读写锁的方式来避免并行访问
- 对于读提交和可重复读，通过 `Read View` 来实现的
    - 「读提交」隔离级别是在每个 select 都会生成一个新的 Read View，也意味着，事务期间的多次读取同一条数据，前后两次读的数据可能会出现不一致，因为可能这期间另外一个事务修改了该记录，并提交了事务。
    - 「可重复读」隔离级别是启动事务时生成一个 Read View，然后整个事务期间都在用这个 Read View，这样就保证了在事务期间读到的数据都是事务启动前的记录。

Read View在MVCC中是如何工作的

1. 事务启动：当一个事务启动时，系统会为该事务创建一个 Read View。
2. Read View 包含的信息



    - `m_ids`: 当前数据库中「活跃事务」的**事务 id 列表**, 表示启动了但还没有提交的事务
    - `min_trx_id`: 「活跃事务」中事务id 最小的事务
    - `max_trx_id`: 当前数据库中应该给下一个事务的 id 值
    - `creator_trx_id`: 创建该Read View的事务的id

除此之外，聚簇索引记录中都包含下面两个隐藏列：

- `trx_id`，当一个事务对某条聚簇索引记录进行改动时，就会**把该事务的事务 id 记录在 trx_id 隐藏列里**；
- `roll_pointer`，每次对某条聚簇索引记录进行改动时，都会把旧版本的记录写入到 undo 日志中，然后**这个隐藏列是个指针，指向每一个旧版本记录**，于是就可以通过它找到修改前的记录。

3. 事务读取数据：当一个事务要执行读操作时，系统会使用 Read View 来确定该事务能够看到哪些数据版本。具体步骤如下：
    - 如果记录的`trx_id`小于Read View 中的 `min_trx_id` 值，表示这个版本的记录是在创建 Read View **前**已经提交的事务生成的，所以该版本的记录对当前事务**可见**。

- 如果记录的 trx_id 值大于等于 Read View 中的 `max_trx_id` 值，表示这个版本的记录是在创建 Read View **后**才启动的事务生成的，所以该版本的记录对当前事务**不可见**。
- 如果记录的 trx_id 值在 Read View 的 `min_trx_id` 和 `max_trx_id` 之间，需要判断 trx_id 是否在 m_ids 列表中：
  - 如果记录的 trx_id **在** `m_ids` 列表中，表示生成该版本记录的活跃事务依然活跃着（还没提交事务），所以该版本的记录对当前事务**不可见**。
  - 如果记录的 trx_id **不在** `m_ids` 列表中，表示生成该版本记录的活跃事务已经被提交，所以该版本的记录对当前事务**可见**。

4. 事务提交：当一个事务提交时，它的事务ID会加入到已提交的事务ID集合中。这样，其他事务的 Read View 就能够看到该事务提交的数据版本。

读已提交是怎么实现的

使用`MVCC(多版本并法控制)`实现的

- 读操作：在MVCC中，每个事务在读取数据时都可以看到数据的一个版本。已提交的事务产生一个版本，而未提交的事务则不会对其他事务可见。因此，读已提交只能看到已提交事务的版本。
- 写操作： 写操作创建一个新的数据版本，而不是直接修改原始数据。这确保了正在进行的事务不会看到未提交事务的更改。

事务的启动方式

显式启动事务语句

```
begin/start transaction、commit、rollback
```

关闭线程的自提交

```
set autocommit=0;
```

事务会持续存在直到你主动执行commit或者rollback，或者断开连接，所以，如果采取了第二种方法，就导致了接下来的查询都在事务中，如果是长连接，就导致了长事务。

所以，建议使用 `set autocommit=1`

长事务的查询：

在informationschema库下的innodbtrx表中查询。

举个栗子

如何避免长事务对业务的影响?

从应用开发端来看:

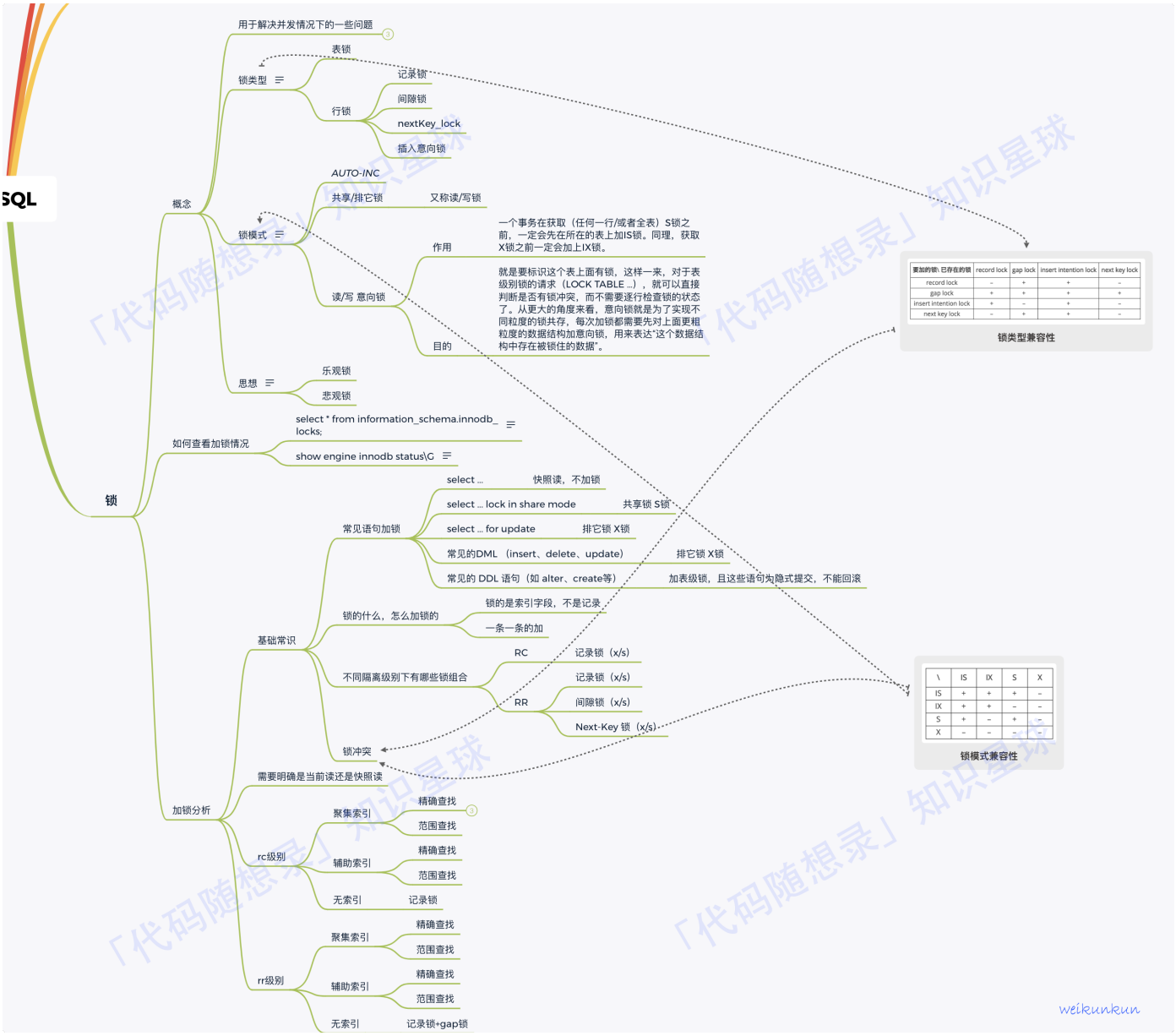
1. 确定是否使用了set autocommit=0，如是，则改成1
2. 确定是否有不必要的只读事务
3. 业务连接数据库的时候，控制每个语句执行的最长时间 set maxexecutiontime

从数据库端来看:

1. 监控相关表，设置长事务阈值，超过就报警/kill
2. 可使用percona的pt-kill工具
3. 在功能测试阶段输出所有的log，分析日志提前发现问题
4. 把innodbundotablespaces设置成2或者更大的值

锁

多个事物在【并发情况下】会出现一些经典的问题，如脏读、不可重复读、幻读、丢失更新。为了不同程度的解决这些问题，出现了不同的隔离级别，锁就是实现各种隔离级别的一种方式。



锁的种类

1、全局锁

顾名思义，全局锁就是**对整个数据库**实例加锁。

典型使用场景：全库逻辑备份，即把整个库的表都select出来存成文本

MySQL 提供了一个加全局读锁的方法，命令是

```
flush tables with read lock (FTWRL)。
```

当你需要让整个库处于只读状态的时候，可以使用这个命令，之后其他线程的**以下语句会被阻塞：**

- 数据更新语句（数据的增删改）
- 数据定义语句（包括建表、修改表结构等）
- 更新类事务的提交语句

2、表级锁

MySQL 里面表级别的锁有下面几种：

（1）表锁

特点：

- 每次操作锁住整张表
- 开销小，加锁快
- 并发度最低

表锁的语法是 `lock tables … read/write`。

与 FTWRL 类似，可以用 `unlock tables` 主动释放锁，也可以在客户端断开的时候自动释放。

需要注意： lock tables 语法除了会限制别的线程的读写外，也限定了本线程接下来的操作对象。

（2）元数据锁（meta data lock，MDL）

MDL 不需要显式使用，在访问一个表的时候会被自动加上。

MDL 的作用：保证读写的正确性。

你可以想象一下，如果一个查询正在遍历一个表中的数据，而执行期间另一个线程对这个表结构做变更，删了一列，那么查询线程拿到的结果跟表结构对不上，肯定是不行的。

在 MySQL 5.5 版本中引入了 MDL

当对一个表做增删改查操作的时候，加 MDL 读锁；
当要对表做结构变更操作的时候，加 MDL 写锁。

读锁之间不互斥，因此你可以有多个线程同时对一张表增删改查。读写锁之间、写锁之间是互斥的，用来**保证变更表结构操作的安全性**。因此，如果有两个线程要同时给一个表加字段，其中一个要等另一个执行完才能开始执行。

事务中的 MDL 锁，在语句执行开始时申请，但是语句结束后并不会马上释放，而会等到整个事务提交后再释放。（这可能会产生死锁的问题）

（3）意向锁

意向锁用于指示一个事务在未来可能会请求对某些资源（如数据行）的锁定

- 意向共享锁表示事务打算在资源上获得共享锁。其他事务可以继续获得共享锁，但不能获得排他锁。
- 意向排他（独占）锁： 表示事务打算在资源上获得排他锁。

对某些记录加上「共享锁」之前，需要先在表级别加上一个「意向共享锁」，对某些纪录加上「独占锁」之前，需要先在表级别加上一个「意向独占锁」。

意向共享锁和意向独占锁是表级锁，不会和行级的共享锁和独占锁发生冲突，意向锁之间也不会发生冲突，只会和共享表锁和独占表锁发生冲突。意向锁的目的是为了快速判断表里是否有记录被加锁。

（4）AUTO-INC 锁

作用：表里的主键通常都会设置成自增的，之后可以在插入数据时，可以不指定主键的值，数据库会自动给主键赋值递增的值通过 **AUTO-INC 锁**实现的。**在插入数据时，会加一个表级别的 AUTO-INC 锁**，然后为被 `AUTO_INCREMENT` 修饰的字段赋值递增的值，等插入语句执行完成后，才会把 AUTO-INC 锁释放掉。其他事务的如果要向该表插入语句都会被阻塞，从而保证插入数据时字段的值是连续递增的。

缺陷：对大量数据进行插入的时候，会影响插入性能，因为其他事务中的插入会被阻塞。

改进：InnoDB 存储引擎提供了一种**轻量级的锁**来实现自增。在插入数据的时候，会为被 `AUTO_INCREMENT` 修饰的字段加上轻量级锁，**然后给该字段赋值一个自增的值，就把这个轻量级锁释放了，而不需要等待整个插入语句执行完后才释放锁**。

3、行级锁

顾名思义，行锁就是针对数据表中行记录的锁（也有人称为记录锁）。

这很好理解，比如事务 A 更新了一行，而这时候事务 B 也要更新同一行，则必须等事务 A 的操作完成后才能进行更新。

类型：

- Record Lock，记录锁，仅仅把一条记录锁上，记录锁分为排他锁和共享锁。
- Gap Lock，间隙锁，锁定一个范围，但是不包含记录本身，只存在于可重复读隔离级别，目的是为了解决可重复读隔离级别下幻读的现象。间隙锁之间是兼容的，两个事务可以同时持有包含共同间隙范围的间隙锁，并不存在互斥关系。
- Next-Key Lock：Record Lock + Gap Lock 的组合，锁定一个范围，并且锁定记录本身。next-key lock 即能保护该记录，又能阻止其他事务将新纪录插入到被保护记录前面的间隙中。

特点：

- 每次操作锁住一行数据
- 开销大，加锁慢
- 发生锁冲突的概率是最低的，并发度是最高的

在 InnoDB 事务中，行锁是在需要的时候才加上的，但并不是不需要了就立刻释放，而是要等到事务结束时才释放。这个就是**两阶段锁协议**。

知道了这个设定，对我们使用事务有什么帮助呢？那就是，如果你的事务中需要锁多个行，要把最可能造成锁冲突、最可能影响并发度的锁尽量往后放。

加锁规则

加锁规则里面，包含了两个“原则”、两个“优化”和一个“bug”

两个原则：

1. 加锁的基本单位是 next-key lock。希望你还记得，next-key lock 是前开后闭区间。
2. 查找过程中访问到的对象才会加锁。

两个优化：

1. 索引上的等值查询，给唯一索引加锁的时候，next-key lock 退化为行锁。
2. 索引上的等值查询，向右遍历时且最后一个值不满足等值条件的时候，next-key lock 退化为间隙锁。

一个"bug"：

1. 唯一索引上的范围查询会访问到不满足条件的第一个值为止。（MySQL8.0之后的版本已经修复）

以上规则是在可重复读隔离级别 (repeatable-read) 下验证的。同时，可重复读隔离级别遵守两阶段锁协议，所有加锁的资源，都是在事务提交或者回滚的时候才释放的。如果切换到读提交隔离级别 (read-committed) 的话，就好理解了，过程中去掉间隙锁的部分，也就是只剩下行锁的部分。

锁的划分

1.、数据库角度

1、共享锁（共享锁也叫读锁或 S 锁 ）

共享锁锁定的资源可以被其他用户读取，但不能修改。

在进行SELECT的时候，会将对象进行共享锁锁定，当数据读取完毕之后，就会释放共享锁，这样就可以保证数据在读取时不被修改。

如果我们想要给某一行加上共享锁呢，比如想对 user_id=10的数据行加上共享锁，可以像下面这样：

```
SELECT user_id FROM product_comment WHERE user_id = 10 LOCK IN SHARE MODE;
```

2、排他锁（排它锁也叫独占锁、写锁或 X 锁）

排它锁锁定的数据只允许进行锁定操作的事务使用，其他事务无法对已锁定的数据进行查询或修改。

如果我们想要在某个数据行上添加排它锁，比如针对 user_id=10的数据行，则写成如下这样：

```
SELECT user_id FROM product_comment WHERE user_id = 10 FOR UPDATE;
```

另外当我们对数据进行更新的时候，也就是INSERT、DELETE或者UPDATE的时候，数据库也会自动使用排它锁，防止其他事务对该数据行进行操作。

3、共享锁与排他锁

共享锁和排他锁不仅可以锁住一行，也可以锁住一张表，如下所示：

比如我们想给 product_comment 在表上加共享锁，可以使用下面这行命令

```
LOCK TABLE product_comment READ;
```

如果我们想要对表上的共享锁进行解锁，可以使用下面这行命令：

```
UNLOCK TABLE;
```

如果我们想给 product_comment 数据表添加排它锁，可以使用下面这行命令

```
LOCK TABLE product_comment WRITE;
```

这时只有获得排它锁的事务可以对 product_comment 进行查询或修改，其他事务如果想要在 product_comment 表上查询数据，则需要等待。

你可以自己开两个 MySQL 客户端来模拟下。这时我们释放掉排它锁，使用这行命令即可

```
UNLOCK TABLE;
```

4、意向锁（Intent Lock）

简单来说就是给更大一级别的空间示意里面是否已经上过锁。

举例：

你可以给整个房子设置一个标识，告诉它里面有人，即使你只是获取了房子中某一个房间的锁。这样其他人如果想要获取整个房子的控制权，只需要看这个房子的标识即可，不需要再对房子中的每个房间进行查找。这样是不是很方便?

返回数据表的场景，如果我们给某一行数据加上了排它锁，数据库会自动给更大一级的空间，比如数据页或数据表加上意向锁，告诉其他人这个数据页或数据表已经有人上过排它锁了，这样当其他人想要获取数据表排它锁的时候，只需要了解是否有人已经获取了这个数据表的意向排他锁即可。

如果事务想要获得数据表中某些记录的共享锁，就需要在数据表上添加意向共享锁。同理，事务想要获得数据表中某些记录的排他锁，就需要在数据表上添加意向排他锁。这时，意向锁会告诉其他事务已经有人锁定了表中的某些记录，不能对整个表进行全表扫描。

2.、程序员角度

1、乐观锁（Optimistic Locking）

认为**对同一数据的并发操作不会总发生**，属于小概率事件，**不用每次都对数据上锁**，也就是不采用数据库自身的锁机制，而是通过程序来实现。在程序上，我们可以采用版本号机制或者时间戳机制实现。

2、悲观锁（Pessimistic Locking）

也是一种思想，对数据被其他事务的修改持**保守态度**，会通过数据库自身的锁机制来实现，从而保证数据操作的排它性。

3、乐观锁和悲观锁的适用场景

（1）乐观锁

适合读操作多的场景，相对来说写的操作比较少。它的优点在于程序实现，不存在死锁问题，不过适用场景也会相对乐观，因为它阻止不了除了程序以外的数据库操作。

（2）悲观锁

适合写操作多的场景，因为写的操作具有排它性。采用悲观锁的方式，可以在数据库层面阻止其他事务对该数据的操作权限，防止读 - 写和写 - 写的冲突。但是加锁的时间会比较长，可能会长时间限制其他用户的访问，也就是说他的并发访问性不好。

Innodb使用表锁还是行锁

对于Innodb，绝大部分情况应该使用行锁

使用表锁的情况
（1）表比较大，事务需要更新全部或者大部分数据
（2）事务涉及到多个表，比较复杂，可能引起死锁，造成大量的事务回滚

日志

在前面中，我们提到了【更新语句】会设计到日志，具体包含下面几种

- **undo log（回滚日志）**：是 Innodb 存储引擎层生成的日志，实现了事务中的**原子性**，主要**用于事务回滚和MVCC**。
- **redo log（重做日志）**：是 Innodb 存储引擎层生成的日志，实现了事务中的**持久性**，主要**用于掉电等故障恢复**；
- **binlog （归档日志）**：是 Server 层生成的日志，主要**用于数据备份和主从复制**；

什么是undo logo

当事务对数据库进行更新（插入、修改、删除）时，系统会记录相应的`undo log`，以便在事务回滚或系统崩溃时进行数据恢复, 主要用于事务回滚和MVCC。`undo log`记录的信息包括操作类型（插入、删除还是更新），修改前的数据值，被修改的数据的位置，事务标识id等，比如在**更新**一条记录时，要把被更新的列的旧值记下来，这样之后回滚时再把这些列**更新为旧值**就好了。

每一次更新操作产生的`undo log`格式都有一个 `roll_pointer` 指针(将undo log 串成一个链表, 链表也被成为版本链)和一个`trx_id`事务id(记录是被哪个事务修改的)

- 事务回滚：如果事务在执行过程中出现错误或被用户显式地回滚，系统可以使用`undo log`来还原事务所做的所有修改。通过`undo log`，数据库系统可以逆向执行事务的操作，将数据库还原到事务开始前的状态。
- **实现 MVCC（多版本并发控制）关键因素之一。**MVCC 是通过 ReadView + undo log 实现的。undo log 为每条记录保存多份历史数据，MySQL 在执行快照读（普通 select 语句）的时候，会根据事务的 Read View 里的信息，顺着 undo log 的版本链找到满足其可见性的记录。

什么是redo log

redo log 是物理日志，记录了某个数据页做了什么修改，比如**对 XXX 表空间中的 YYY 数据页 ZZZ 偏移量的地方做了AAA 更新**，每当执行一个事务就会产生这样的一条或者多条物理日志。在事务提交时，只要先将 redo log 持久化到磁盘即可

redo log和undo log的区别是什么

undo log 和 redo log 这两个日志都是 Innodb 存储引擎生成的。

- redo log 记录了此次事务**「完成后」**的数据状态，记录的是更新**之后**的值；
- undo log 记录了此次事务**「开始前」**的数据状态，记录的是更新**之前**的值；

事务提交之前发生了崩溃，重启后会通过 undo log 回滚事务，事务提交之后发生了崩溃，重启后会通过 redo log 恢复事务

什么是binlog

MySQL 在完成一条更新操作后，Server 层还会生成一条 binlog，等之后事务提交的时候，会将该事物执行过程中产生的所有 binlog 统一写 入 binlog 文件。

binlog 文件是记录了所有数据库表结构变更和表数据修改的日志，不会记录查询类的操作，比如 SELECT 和 SHOW 操作。

redo log和bin log有什么区别

- **适用对象不同**：binlog 是 MySQL 的 Server 层实现的，所有存储引擎都可以使用；redo log 是 Innodb 存储引擎实现的日志。
- **文件格式不同**：redo log 是物理日志，记录的是在某个数据页做了什么修改，比如对 XXX 表空间中的 YYY 数据页 ZZZ 偏移量的地方做了AAA 更新。而`binlog` 主要包括三种格式：`Statement`、`Row` 和 `Mixed`。
- **写入方式不同**：binlog 是追加写，写满一个文件，就创建一个新的文件继续写，不会覆盖以前的日志，保存的是全量的日志。redo log是循环写，日志空间大小是固定，全部写满就从头开始，保存未被刷入磁盘的脏页日志。
- **用途不同**：binlog 用于备份恢复、主从复制；redo log 用于掉电等故障恢复。

为什么需要两阶段提交

事务提交后，redo log 和 binlog 都要持久化到磁盘，但是这两个是独立的逻辑，可能出现半成功的状态，造成两份日志之间的逻辑不一致。

- **如果在将 redo log 刷入到磁盘之后， MySQL 突然宕机了，而 binlog 还没有来得及写入**。MySQL 重启后，通过 redo log 能将 Buffer Pool 恢复到新值，但是 binlog 里面没有记录这条更新语句，在主从架构中，binlog 会被复制到从库，由于 binlog 丢失了这条更新语句，从库的这一行是旧值，主从不一致。
- **如果在将 binlog 刷入到磁盘之后， MySQL 突然宕机了，而 redo log 还没有来得及写入**。由于 redo log 还没写，崩溃恢复以后这个事务无效，数据是旧值，而 binlog 里面记录了这条更新语句，在主从架构中，binlog 会被复制到从库，从库执行了这条更新语句，这一行字段是新值，与主库的值不一致性。

所以会造成主从环境的数据不一致性。因为 redo log 影响主库的数据，binlog 影响从库的数据，redo log 和 binlog 必须保持一致。

两阶段提交把单个事务的提交拆分成了 2 个阶段，分别是准备(Prepare)阶段和提交(Commit)阶段，每个阶段都由协调者(Coordinator)和参与者(Participant)共同完成。

两阶段提交的过程

在 MySQL 的 InnoDB 存储引擎中，开启 binlog 的情况下，MySQL 会同时维护 binlog 日志与 InnoDB 的 redo log，为了保证这两个日志的一致性，MySQL 使用了**内部 XA 事务**，内部 XA 事务由 binlog 作为协调者，存储引擎是参与者。

当客户端执行 commit 语句或者在自动提交的情况下，MySQL 内部开启一个 XA 事务，**分两阶段来完成 XA 事务的提交**。

事务的提交过程有两个阶段，**将 redo log 的写入拆成了两个步骤：prepare 和 commit，中间再穿插写入 binlog**：

- **prepare 阶段**：将 内部 XA 事务的 ID写入到 redo log，同时将 redo log 对应的事务状态设置为 prepare，然后将 redo log 持久化到磁盘。
- **commit 阶段**：把 内部 XA 事务的 ID写入到 binlog，然后将 binlog 持久化到磁盘，接着调用引擎的提交事务

接口，将 redo log 状态设置为 commit，此时该状态并不需要持久化到磁盘，只需要 write 到文件系统的 page cache 成功，只要 binlog 写磁盘成功，redo log 的状态还是 prepare 也没有关系，一样会被认为事务已经执行成功。

执行引擎

有哪些执行引擎

在MySQL中，可以通过`SHOW ENGINES;`命令查看当前数据库支持的存储引擎。InnoDB是较为通用和常用的存储引擎。

- `InnoDB`: MySQL默认的事务性存储引擎，支持事务的提交（commit）和回滚（rollback），提供了行级锁定，支持外键约束和MVCC
- `MyISAM`: MyISAM 使用表级锁定, 不支持事务，支持全文索引，适用于以读操作为主的应用
- `Memory`: 将数据放在内存中，数据处理速度很快，但是当数据库重启或崩溃时，存储在内存中的数据将丢失。

数据库引擎InnoDB与MyISAM的区别和适用场景?

1. 事务支持

- `InnoDB:` 支持事务，具有ACID（原子性、一致性、隔离性、持久性）特性，适合需要数据一致性和完整性的应用，如银行系统或在线购物平台。
- `MyISAM:` 不支持事务，不具备ACID特性，适用于读密集、写少的场景，如博客系统或新闻网站。

2. 锁

- `InnoDB:` 行级锁，多个事务可以同时访问同一表的不同行。
- `MyISAM:` 表级锁，对整个表进行锁定，导致并发性能下降，特别是在有大量写操作时。

3. 外键约束:

- InnoDB: 支持外键约束，确保数据的一致性和完整性。
- MyISAM: 不支持外键约束

4. 崩溃恢复(Crash Recovery):

- `InnoDB:` 支持崩溃恢复
- `MyISAM:` 在崩溃后恢复可能会导致数据损失。

5. 全文索引(Full-text Indexing):

- *InnoDB:* 支持全文索引。
- *MyISAM:* 也支持全文索引，并且在这方面表现的更加好。

总结：

InnoDB: 适用于需要事务支持、并发性能好、具有高写入需求的应用

MyISAM: 适用于读操作频繁、写入操作较少的应用

MySQL常见问题

1. 幻读连问

1-1：什么是幻读?

幻读是指在同一个事务中，存在前后两次查询同一个范围的数据，但是第二次查询却看到了第一次查询没看到的行。

注意，幻读出现的场景：

1. 事务的隔离级别为可重复读，且是当前读
2. 幻读仅专指新插入的行

1-2：幻读带来的问题?

1. 对行锁语义的破坏
2. 破坏了数据一致性

1-3：怎么避免幻读?

存储引擎采用**加间隙锁**的方式来避免出现幻读

1-4：为啥会出现幻读?

行锁只能锁定存在的行，针对新插入的操作没有限定

1-5：间隙锁是啥？它怎么避免出现幻读的？它引入了什么新的问题?

间隙锁：

是专门用于解决幻读这种问题的锁，它锁的了行与行之间的间隙，**能够阻塞新插入的操作**

间隙锁的引入也带来了一些新的问题，比如：降低并发度，可能导致死锁。

注意，读读不互斥，读写/写读/写写是互斥的，但是间隙锁之间是不冲突的，间隙锁会阻塞插入操作

另外，间隙锁在可重复读级别下才是有效的

间隙锁和 next-key lock：

行锁和间隙锁合称 next-key lock，这个锁是左开右闭的区间。

MySQL 为了解决幻读问题，在线程更新数据并 next-key lock 的过程中，首先必须在可重复读的隔离级别下，执行以下的原则和优化：

原则：

原则 1：加锁的基本单位是 next-key lock，next-key lock 是前开后闭区间。
原则 2：查找过程中访问到的对象才会加锁。

优化：

优化 1：索引上的等值查询，给唯一索引加锁的时候，next-key lock 退化为行锁，如果不存在这个索引，退化为间隙锁。

优化 2：索引上的等值查询，向右遍历时且最后一个值不满足等值条件的时候，next-key lock 退化为间隙锁。

注意：

1. 非唯一索引的范围查询：范围查询都会访问到不满足条件的第一个值，并且不会执行上述的两个优化。唯一索引的范围查询仍旧会执行上述的优化。
2. delete 语句的和查询的加锁方式相同
3. limit 语句，遍历到满足条件的 n 条数据后，之后不再加 next-key lock

死锁：不同线程可以给同一个间隙加锁。

意思时只有我这个线程可以操控这个间隙，其他线程不能使用，加锁时不会检测是否冲突的，但是如果两个都给同一个间隙上锁，之后两个线程都没办法在这个间隙上更新数据了，都会陷入等待另一个线程的间隙锁释放，也就是死锁。

如果使用读提交隔离级别，那么只加行锁，不加间隙锁，语句执行过程中加上的行锁，在语句执行完成后，就要把“不满足条件的行”上的行锁直接释放了，不需要等到事务提交才释放。

2. 事务

2-1：for update的使用场景

一般这些操作都是很长一串并且是开启事务的。

如果库存刚开始读的时候是1，而立马另一个进程进行了update将库存更新为0了，而事务还没有结束，会将错的数据一直执行下去，就会有问题。

需要 for upate 进行数据加锁防止高并发时候数据出错，即 **使事务保持 当前读** 状态。

```
SELECT * FROM table LIMIT [offset,] rows | rows OFFSET offset;
# 如果只给定一个参数，表示记录数。
```

```
mysql> SELECT * FROM orange LIMIT 5;    # 检索前5条记录(1-5)
```

相当于

```
mysql> SELECT * from orange LIMIT 0,5;
```

两个参数，第一个参数表示offset, 第二个参数为记录数。

```
mysql> SELECT * FROM orange LIMIT 10,15; // 检索记录11-25
```

如果你想要清除一些MySQL使用内部缓存，你应该使用FLUSH命令

2-2：事务隔离

innodb 支持 RC 和 RR 隔离级别实现是用的一致性视图(consistent read view)

事务在启动时会拍一个快照,这个快照是**基于整个库**的。

基于整个库的意思就是说一个事务内,整个库的修改对于该事务都是不可见的(对于快照读的情况)

如果在事务内 select t 表,另外的事务执行了DDL t表,根据发生时间,只有两种情况

1. 报错
2. 锁住

事务是如何实现mvcc的?

（1）每个事务都有一个事务ID,叫做transaction id(严格递增)

（2）事务在启动时,找到已提交的最大事务ID记为up_limit_id。

（3）事务在更新一条语句时

比如 id=1 改为了 id=2.会把 id=1 和该行之前的 row trx_id 写到 undo log 里, 并且在数据页上把 id 的值改为 2,并且把修改这条语句的transaction id 记在该行行头

（4）再定一个规矩

一个事务要查看一条数据时,必须先用该事务的 up_limit_id 与该行的transaction id 做比对

- 如果 up_limit_id >= transaction id,那么可以看
- 如果 up_limit_id < transaction id,则只能去 undo log 里去取。

去 undo log 查找数据的时候,也需要做比对,必须 up_limit_id > transaction id，才返回数据

2-3：什么是当前读

由于当前读都是先读后写,只能读当前的值,所以为当前读会更新事务内的 up_limit_id 为该事务的 transaction

你提到了隐藏列有一个DB_ROW_ID，是干嘛的？那假设有10个update，到第九个回滚了，DB_ROLL_PTR如何做的，那提交了是否更新DB_ROLL_PTR?

为什么 rr 能实现可重复读而 rc 不能,分两种情况

（1）快照读的情况下：

rr不能更新事务内的 up_limit_id ,而 rc 每次会把 up_limit_id 更新为快照读之前最新已提交事务的 transaction id,则 rc 不能可重复读

（2）当前读的情况下：

rr 是利用 record lock+gap lock来实现的,而 rc 没有 gap,所以 rc 不能可重复读

2-4：选择普通索引还是唯一索引?

1、对于查询过程来说：

（1）普通索引

查到满足条件的第一个记录后，继续查找下一个记录，知道第一个不满足条件的记录

（2）唯一索引

由于索引唯一性，查到第一个满足条件的记录后，停止检索

但是，两者的性能差距微乎其微。因为InnoDB根据数据页来读写的。

2、对于更新过程来说：

概念：change buffer

当需要更新一个数据页：

- 如果数据页在内存中就直接更新
- 如果不在内存中，在不影响数据一致性的前提下，InnoDB 会将这些更新操作缓存在change buffer 中。

下次查询需要访问这个数据页的时候，将数据页读入内存，然后执行 change buffer 中的与这个页有关的操作。

change buffer 是**可以持久化**的数据。在内存中有拷贝，也会被写入到磁盘上

purge：将 change buffer 中的操作应用到原数据页上，得到最新结果的过程，成为 purge

访问这个数据页会触发 purge，系统有后台线程定期 purge，在数据库正常关闭的过程中，也会执行 purge

唯一索引的更新不能使用 change buffer

change buffer 用的是 buffer pool 里的内存，change buffer 的大小，可以通过参数 innodb_change_buffer_max_size 来动态设置。这个参数设置为50的时候，表示 change buffer 的大小最多只能占用 buffer pool 的50%。

将数据从磁盘读入内存涉及随机IO的访问，是数据库里面成本最高的操作之一。

change buffer 因为减少了随机磁盘访问，所以对更新性能的提升很明显。

change buffer使用场景：

在一个数据页做purge之前，change buffer记录的变更越多，收益就越大。

对于写多读少的业务来说，页面在写完以后马上被访问到的概率比较小，此时change buffer的使用效果最好。这种业务模型常见的就是账单类、日志类的系统。

反过来，假设一个业务的更新模式是写入之后马上会做查询，那么即使满足了条件，将更新先记录在change buffer,但之后由于马上要访问这个数据页，会立即触发purge过程。

这样随机访问IO的次数不会减少，反而增加了change buffer的维护代价。所以，对于这种业务模式来说，change buffer反而起到了副作用。

索引的选择和实践：

尽可能使用普通索引。

redo log主要节省的是随机写磁盘的IO消耗(转成顺序写)，而change buffer主要节省的则是随机读磁盘的IO消耗。

2-5：char 和 varchar 的区别?

char 是固定长度类型，比如：定义 char(10)，当你输入”abc”三个字符的时候，它们占的空间还是 10 个字节，其他 7 个是空字节。varchar 长度可变

存储情况不同

以 compact 行格式为例：

一条完整的记录由两部分组成

1. 变长字段的真正数据内容
2. 该变长字段所占用的字节数

该变长字段所占用的字节数被放在变长字段列表中，并且按照列的顺序逆序存放。

而 char 类型的数据则只需要存储其真正的数据内容。

对于 类型为 char(M) 的列

如果采用的是变长编码的字符集：

那么该列的值占用的字节数也会被存储到变长字段列表中，采用 变长编码字符集的 CHAR(M) 类型的值要求至少占用 M 个字节，但是 varchar(M) 没有这个要求。

2-6：MySQL抖一下是什么意思?

因为运行的不正常了，或者不稳定了，要花费更多的资需源处理别的事情，会使SQL语句的执行效率明显变慢。

针对innoDB导致MySQL抖的原因，主要是InnoDB 会在后台刷脏页，而刷脏页的过程是要将内存页写入磁盘。所以，无论是你的查询语句在需要内存的时候可能要求淘汰一个脏页，还是由于刷脏页的逻辑会占用 IO 资源并可能影响到了你的更新语句，都可能是造成你从业务端感知MySQL“抖”了一下的原因。

2-7：MySQL抖一下有啥问题?

很明显系统不稳定，性能突然下降对业务端是很不友好的

2-8：怎么让MySQL不抖?

设置合理参数配配置，尤其是设置 好innodb_io_capacity 的值，并且平时要多关注脏页比例，不要让它经常接近 75%

3. 脏页

3-1：啥是脏页?

当内存数据页跟磁盘数据页内容不一致的时候，我们称这个内存页为“脏页”。

3-2：啥是干净页?

内存数据写入到磁盘后，内存和磁盘上的数据页的内容就一致了，称为“干净页”。

3-3：脏页是咋产生的?

因为使用了WAL技术，这个技术会把数据库的随机写转化为顺序写，但副作用就是会产生脏页。

3-4：啥是随机写？为啥那么耗性能?

随机写我的理解是，这次写磁盘的那个扇区和上一次没啥关系，需要重新定位位置，机械运动是很慢的即使不是机械运动重新定位写磁盘的位置也是很耗时的。

3-6：啥是顺序写?

顺序写我的理解是，这次写磁盘那个扇区就在上一次的下一个位置，不需要重新定位写磁盘的位置速度当然会快一些。

3-7：WAL怎么把随机写转化为顺序写的?

写redolog是顺序写的，先写redolog等合适的时候再写磁盘，间接的将随机写变成了顺序写，性能确实会提高不少。

3-8：为啥删除了表的一半数8据，表文文件大小没变化？ 、

因为delete 命令其实只是把记录的位置，或者数据页标记为了“可复用”，但磁盘文件的大小是不会变的。也可以认为是一种逻辑删除，所以物理空间没有实际释放，只是标记为可复用，表文件的大小当然是不变的啦！

4. 表结构

4-1：表的数据信息存在哪里?

表数据信息可能较小也可能巨大无比，可以存储在共享表空间里，也可以单独存储在一个以.ibd为后缀的文件里，由参数 innodb_file_per_table 来控制，建议总是作为一个单独的文件来存储，这样非常容易管理，并且在不需要的时候，使用 drop table 命令也能直接把对应的文件删除，如果存储在共享空间之中即使表删除了空间也不会释放。

4-2：表的结构信息存在哪里?

首先，表结构定义占有的存储空间比较小

MySQL8.0 之前：表结构的定义信息存在以.frm为后缀的文件里
MySQL8.0 之后：则允许把表结构的定义信息存在系统数据表之中

系统数据表，主要用于存储MySQL的系统数据，比如：数据字典、undo log(默认)等文件

4-3：如何才能删除表数据后，表文件大小就变小?

重建表，消除表因为进行大量的增删改操作而产生的空洞，使用如下命令：

```
alter table t engine=InnoDB
optimize table t( 等于 recreate+analyze)。
truntace table t (等于drop+create)
```

5. Count

5-1：空洞是啥？咋产生的?

空洞就是那些被**标记可复用但是还没被使用的存储空间**。

使用delete命令删除数据会产生空洞，标记为可复用

插入新的数据可能引起页分裂，也可能产生空洞

修改操作，有时是一种先删后插的动作也可能产生空洞

按照效率排序的话：

count(字段) < count(主键 id) < count(1) ≈ count(*)

5-2：count(*)这么慢，我该怎么办?

要么忍，要么自己动手记录。。。

5-3：`count()`的语义是啥?

首先，不同的存储引擎实现方式不同

MyISAM 引擎把一个表的总行数存在了磁盘上，因此执行 count(*) 的时候会直接返回这个数，效率很高；

而 InnoDB 引擎就麻烦了，它执行 count(*) 的时候，需要把数据一行一行地从引擎里面读出来，然后累积计数。

以下针对innodb来说

count() 是一个聚合函数，对于返回的结果集，一行行地判断，如果 count 函数的参数不是 NULL，累计值就加 1，否则不加，最后返回累计值。

5-4：count(字段)怎么计数?

1、如果这个“字段”是定义为 not null 的话

一行行地从记录里面读出这个字段，判断不能为 null，按行累加；

2、如果这个“字段”定义允许为 null

执行的时候，判断到有可能是 null，还要把值取出来再判断一下，不是 null 才累加。

从引擎返回的字段会涉及到解析数据行，以及拷贝字段值的操作。

5-5：count(主键 id)怎么计数?

对于 count(主键 id) 来说，InnoDB 引擎会遍历整张表，把每一行的 id 值都取出来，返回给 server 层。

server 层拿到 id 后，判断是不可能为空的，就按行累加。从引擎返回的 主键id 会涉及到解析数据行，以及拷贝字段值的操作。

5-5：count(1)怎么计数?

对于 count(1) 来说，InnoDB 引擎遍历整张表，但不取值。server 层对于返回的每一行，放一个数字“1”进去，判断是不可能为空的，按行累加。

5-6：count(*)怎么计数?

对于count(*)来说，并不会把全部字段取出来，而是专门做了优化，不取值。

count(*) 肯定不是 null，按行累加。

这些count()的原则如下：

server 层要什么就给什么；

InnoDB 只给必要的值；

现在的优化器只优化了 count(*) 的语义为“取行数”，其他“显而易见”的优化并没有做。

6. 查询效率

6-1：order by 是怎样工作的?

example：

```
select city,name,age from t where city='杭州' order by name limit 1000;
```

涉及到用户语句的排序，mysql 会给每个线程分配一块内存用于排序，也就是 sort_buffer。

这条语句的执行逻辑是：

1、先初始化 sort_bufer

2、然后放入 city,name,age 字段，不断地由主键id索引到整行再到三个字段的值，匹配查找的值存入 sort_buff

3、然后按 name 排序，返回前 1000 个值

但是如果 sort_buffer_size 设置的太小，无法存放所有匹配的字段，排序就无法在内存中完成

需要借鉴磁盘临时文件辅助排序，可以通过 number_of_tmp_files 这个标识来判断是否使用，其实这个原理和对超大数据的排序相同。

如果要记录的字段太长，这样内存里能够同时放下的行数很少，要分成很多个临时文件，排序的性能会很差。

这时会换一个算法，叫做rowid排序，顾名思义，就是对主键 id 以及排序字段进行存放，这样就节省了空间，但是最后需要通过主键 id 去找到之前未取出的字段。对比全字段排序，rowid 排序多访问了一次表 t 的主键索引。

结论：

如果 MySQL 实在是担心排序内存太小，会影响排序效率，才会采用 rowid 排序算法，这样排序过程中一次可以排序更多行，但是需要再回到原表去取数据。

如果 MySQL 认为内存足够大，会优先选择全字段排序，把需要的字段都放到 sort_buffer 中，这样排序后就会直接从内存里面返回查询结果了，不用再回到原表去取数据。

如果想要避免排序，可以建立对应字段的索引。

如果想要进一步避免主键 id 的回表查询，可以使用覆盖索引，这种情况的索引建立成本会比较大，需要你去自己权衡是否使用

6-2：如何正确地显示随机消息?

example：

英语学习 App 首页有一个随机显示单词的功能，也就是根据每个用户的级别有一个单词表，然后这个用户每次访问首页的时候，都会随机滚动显示三个单词。他们发现随着单词表变大，选单词这个逻辑变得越来越慢，甚至影响到了首页的打开速度

1、order by rand() 实现

创建临时表→按主键顺序取出所有的单词，并给他们一个随机小数→初始化 sort_buffer，从内存临时表取出数据放入 sort_buffer，按随机数排序，取出前三个，总扫描行数是2×n+3

2、磁盘临时表

如果 sort_buffer_size 设置的小，就会使用到磁盘临时表用于辅助排序，当然 mysql 高版本使用了优先队列排序的方法，就是只取三个值，构成一个堆，然后不断把剩下的值输入，用于更新队列。

这个过程不需要临时文件，因此对应的 number_of_tmp_files 是 0。当然，如果堆的大小设置的很大，sort_buffer 放不下，就会舍弃优先队列，重新使用临时表+归并排序来实现，number_of_tmp_files 就不再为 0。

3、随机id方法

我们实际上并不需要真正的为每行赋予一个随机值，然后进行排序，这样的成本就太高了。

可以借鉴主键id的唯一性，生成一个介于主键id最小值和最大值之间的随机数，然后取不小于这个随机数的主键id的那一行即可。总扫描行数是3，的那hi是如果主键id有空洞，就会导致选择不同行的概率不同，不是真正的随机

4、严格随机id方法

取出表的总行数，生成一个总行数对应的随机值，取不小于这个随机数的主键id的那一行，总扫描行数是C+Y+1

6-3：为什么SQL语句逻辑相同，性能却差异巨大?

案例一：条件字段函数操作

```
select count(*) from tradelog where month(t_modified)=7;
```

并且在 t_modified 上建立了索引

你以为对字段生成了索引，应该会返回的很快，但是mysql有一个规则：就是如果对字段做了函数计算，就用不上索引了。

实际上，mysql使用的B+ 树提供的对于索引的快速定位能力，来源于同一层兄弟节点的有序性。对索引字段做函数操作，可能会破坏索引值的有序性，因此优化器就决定放弃走树搜索功能，转而进行全表扫描，所以运行就变慢了

案例二：隐式类型转换：

如果在限制语句中错误的将字符串与数字进行比较，mysql 会将可以转化成数字的字符串转化成数字，没法转换的话，就会变成数字 0，因此如果出现这种输入的失误，相当于调用了字符串转数字的函数，还是会使得优化器会放弃走树搜索功能，进行全表扫描

案例三：隐式字符编码转换

如果执行两个表的联合查询，两个表通过外键进行联结，如果两个表使用的字符集不同，会对低水平的字符集执行升级转换函数，（相当于将int赋给double的这个思路）。

如果限制语句是 where d.tradeid=l.tradeid，如果d的字符集低水平，就会执行函数操作，使得优化器会放弃走树搜索功能，进行全表扫描

总结：对索引字段做函数操作，可能会破坏索引值的有序性，因此优化器就决定放弃走树搜索功能

6-4：为什么我只查一行的语句，也执行这么慢?

原因从大到小可分为三种情况

第一 MySQL 数据库本身被堵住了，比如：系统或网络资源不够

第二 SQL 语句被堵住了，比如：表锁，行锁等，导致存储引擎不执行对应的 SQL 语句

第三确实是索引使用不当，没有走索引，类比上一节

第四是表中数据的特点导致的，走了索引，但由于一致性读，需要回滚多次才能读取出当前事务对应的值。

6-5：为什么临时表可以重名?

在实际应用中，临时表一般用于处理比较复杂的计算逻辑。

由于临时表是每个线程自己可见的，所以不需要考虑多个线程执行同一个处理逻辑时，临时表的重名问题。在线程退出的时候，临时表也能自动删除，省去了收尾和异常处理的工作。

在 binlog_format='row'的时候，临时表的操作不记录到 binlog 中，也省去了不少麻烦，防止出现主备不一致的情况

这里说到的临时表是用户临时表，而不是内存临时表

6-5：group-by使用原则

如果对 group by 语句的结果没有排序要求，要在语句后面加 order by null；

尽量让 group by 过程用上表的索引，确认方法是 explain 结果里没有 Using temporary 和 Using filesort；

如果 group by 需要统计的数据量不大，尽量只使用内存临时表；也可以通过适当调大 tmp_table_size 参数，来避免用到磁盘临时表；

如果数据量实在太大，使用 SQL_BIG_RESULT 这个提示，来告诉优化器直接使用排序算法得到 group by 的结果，不要使用内存临时表。

6-6：一张表可以存储多少条信息

假设采用的是InnoDB引擎

那么数据结构的角度来看，本着**索引即数据**的思路，就是求一个 B+树 能够存储多少数据，根据 B+树 定义也就转变成了 **2 个子问题**：

1. 每个叶子结点存储能存储多少条记录
2. 一共有多少个叶子结点

1、每个叶子结点能存储多少条记录

假设一条记录的大小为1kb，InnoDB中默认每页为16kb，则一个叶子节点可存储16条数据

2、一共有多少个叶子结点

根据B+树的形式，确定了该B+树为几阶的B+树，以及树的高度，就能确定叶子节点有最多有多少个。

非叶子节点存储的数据为关键子+指针。关键字也就是我们常说的主键，如果为int类型则4字节，指针大小也假设为4字节。

以关键字+指针为基本单位，那么一个非叶子节点最多能够存储 16384 / 8 = 2048 个，2048个指针能够引出2048个子节点，如果该B+树的高度为2，则存储的记录数量为2048 *16 = 32768条记录，如果高度为3则有2048* 2048 * 16 = 67108864条记录。（也能反向得出，一般索引高度都在3左右吧）

6-5：有一个联合索引(A,B,C),select * from test where a,c会不会使用索引?

不会，根据索引的创建过程，这棵 b+ 树的叶子节点的记录以及 二级索引记录 是按照 abc 的规则来排序的，也就是说，先按照 a的大小来排序，a 相等的情况下，再按照 b 的大小来排序，b 相等的话，最后再是 c 。这样的话，符合 where 条件的 记录不一定相邻，

也就是说，并不能在某一个区间中找，还是得一条条找，查询效率并没有变高。

MySQL分库分表

原因

1、单库太大

单个数据库处理能力有限，所在的服务器上的磁盘空间也有限，单库存在I/O操作瓶颈。

主要方案：切分成更多更小的库

2、单表太大

CRUD都成问题，索引膨胀，查询超时。

主要方案：切分成多个数据集更小的表

垂直分表

需求：

商品表字段太多，每个字段访问频次不一样，浪费了IO资源，需要进行优化

垂直分表介绍：

也就是“大表拆小表”，基于列字段进行的，拆分原则一般是表中的字段较多，将不常用的或者数据较大，长度较长的拆分到“扩展表 如text类型字段，访问频次低、字段大的商品描述信息单独存放在一张表中，访问频次较高的商品基本信息单独放在一张表中。

垂直拆分原则：

1. 把不常用的字段单独放在一张表;
2. 把text，blob等大字段拆分出来放在附表中;
3. 业务经常组合查询的列放在一张表中

垂直分库

需求：

C端项目里面，单个数据库的CPU、内存长期处于90%+的利用率，数据库连接经常不够，需要进行优化

垂直分库讲解：

垂直分库针对的是一个系统中的不同业务进行拆分， 数据库的连接资源比较宝贵且单机处理能力也有限，没拆分之前全部都是落到单一的库上的，单库处理能力成为瓶颈，还有磁盘空间，内存，tps等限制

拆分之后，避免不同库竞争同一个物理机的CPU、内存、网络IO、磁盘，所以在高并发场景下，垂直分库一定程度上能够突破IO、连接数及单机硬件资源的瓶颈

垂直分库可以更好解决业务层面的耦合，业务清晰，且方便管理和维护

一般从单体项目升级改造为微服务项目，就是垂直分库

仍然存在的问题：

垂直分库分表可以提高并发，但是依然没有解决单表数据量过大的问题

水平分表

需求：

当一张表的数据达到几千万时，查询一次所花的时间长，需要进行优化，缩短查询时间

都是大表拆小表

垂直分表：表结构拆分
水平分表：数据拆分

水平分表：

把一个表的数据分到一个数据库的多张表中，每个表只有这个表的部分数据，核心是把一个大表，分割N个小表，每个表的结构是一样的，数据不一样，全部表的数据合起来就是全部数据

针对数据量巨大的单张表（比如订单表），按照某种规则（RANGE,HASH取模等），切分到多张表里面去，但是这些表还是在同一个库中，所以单数据库操作还是有IO瓶颈，主要是解决单表数据量过大的问题

减少锁表时间，没分表前，如果是DDL(create/alter/add等)语句，当需要添加一列的时候mysql会锁表，期间所有的读写操作只能等待

水平分库

需求：

高并发的项目中，水平分表后依旧在单个库上面，1个数据库资源瓶颈 CPU/内存/带宽等限制导致响应慢，需要进行优化

水平分库：

把同个表的数据按照一定规则分到不同的数据库中，数据库在不同的服务器上。

水平分库是把不同表拆到不同数据库中，它是对数据行的拆分，不影响表结构，每个库的结构都一样,但每个库的数据都不一样，没有交集，所有库的并集就是全量数据

水平分库的粒度，比水平分表更大

水平分库分表切分规则：

- **range：** 根据范围，比如0-1000一个表，1001到2000一个表
- **hash取模：** 比如取ID，进行hash取模，根据模数分配到不同的数据库中
- **地理区域：** 按照地理范围进行划分
- **时间范围：** 按照时间进行切分

分库分表总结

1、技术负责人前瞻性思维

1. 需要提前考虑系统一年到两年左右的业务增长情况
2. 对数据库服务器的QPS、连接数、容量等做合理评估和规划
3. 很多人觉得“分库分表”是宜早不宜迟-----其实不然

常规开发里面单表建议1千万内，推荐是百万级别单表存储，常规sql和索引优化先行，然后结合缓存+异步+nosql+mq

2、垂直角度（表结构不一样）

垂直分表：

将一个表字段拆分多个表，每个表存储部分字段

好处：

避免IO时锁表的次数，分离热点字段和非热点字段，避免大字段IO导致性能下降

原则：

业务经常组合查询的字段一个表；不常用字段一个表；text、blob类型字段作为附属表

垂直分库：

根据业务将表分类，放到不同的数据库服务器上

好处：

避免表之间竞争同个物理机的资源，比如CPU/内存/硬盘/网络IO

原则：

根据业务相关性进行划分，领域模型，微服务划分一般就是垂直分库

3、水平角度（表结构一样）

水平分库：

把同个表的数据按照一定规则分到不同的数据库中，数据库在不同的服务器上

好处：

多个数据库，降低了系统的IO和CPU压力

原则：

- 选择合适的分片键和分片策略，和业务场景配合
- 避免数据热点和访问不均衡、避免二次扩容难度大

水平分表：

- 同个数据库内，把一个表的数据按照一定规则拆分到多个表中，对数据进行拆分，不影响表结构
- 单个表的数据量少了，业务SQL执行效率高，降低了系统的IO和CPU压力

原则：

- 选择合适的分片键和分片策略，和业务场景配合
- 避免数据热点和访问不均衡、避免二次扩容难度大

4、互联网公司实际使用和跳槽面试

公司业务稳定发展，多数情况是为了解决【单库单表】数据量过多问题

重点是水平角度的【分库分表】

分库分表的问题

分库分表解决了问题，同时也带来了问题。

如跨库查询、分布式事务、分库之后的排序（翻页，函数的计算问题）、全局主键问题、容量规划（二次扩容问题）、技术选型问题

以下是一些常见的问题

1、跨节点数据库Join关联查询

数据库切分前：

多表关联查询，可以通过sql join进行实现

分库分表后：

数据可能分布在不同的节点上，sql join带来的问题就比较麻烦

2、分库操作带来的分布式事务问题

操作内容同时分布在不同库中，不可避免会带来跨库事务问题，即分布式事务

3、执行的SQL排序、翻页、函数计算问题

分库后：

数据分布再不同的节点上， 跨节点多库进行查询时，会出现limit分页、order by排序等问题

而且当排序字段非分片字段时，更加复杂了，要在不同的分片节点中将数据进行排序并返回，然后将不同分片返回的结果集进行汇总和再次排序（也会带来更多的CPU/IO资源损耗）

4、数据库全局主键重复问题

常规表的id是使用自增id进行实现，分库分表后，由于表中数据同时存在不同数据库中，如果用自增id，则会出现冲突问题

5、容量规划,分库分表后二次扩容问题

业务发展快，初次分库分表后，满足不了数据存储，导致需要多次扩容

6、分库分表技术选型问题

市场分库分表中间件相对较多，框架各有各的优势与短板，应该如何选择

数据库排序思路

1. 思路

整体的思路和海量数据排序类似，不过考虑更加好一些

正常思路：

在分割段的阶段，使用内部排序，生成n个大小等于可用内存的顺串，最后再进行归并，使得数据整体有序。

但是，为了**避免I/O操作带来的影响**，所以**使用替换-选择排序的方式**，可以使得在分割段阶段生成的顺串大小大于可用内存大小。

同时为了能够再次减小I/O开销，合并阶段，可适量的增加归并的路数

不过增大路数也就意味着内部每个缓冲区数量增加（一个缓冲区放一个顺串），每次要在更多的缓冲区中选取最小值，所以引入败者树（和堆排很像），减少增加归并路数，而影响内部归并。

2. 练习题与小总结

1. 练习内部排序的各种算法
   LC_排序数组
2. 练习二路归并
   LC_归并排序

总结：

为了提升效率，避免速度不匹配的问题，真的是精益求情，OS里面有各种高速缓存、缓冲区等，数据库会利用磁盘的预读，顺序性读，以及设计特殊的数据结构。

MySQL主从复制

1. 概念

数据可以从一个MySQL数据库服务器主节点复制到一个或者多个从节点。

MySQL默认采用异步复制方式，这样从节点就不用一直访问主服务器来更新最新数据。

从节点可以复制主节点数据库中的所有数据库、特定的数据库或者特定的表。

2. 用途

1、数据实时备份

当系统中某个节点发生故障时，可以方便故障切换

2、读写分离

在开发过程中，如果遇到某个sql语句需要锁表，导致暂时不能使用读的服务

使用主从复制，让主数据库负责写，从数据库负责读，即使主库出现锁表的情景，也可以通过从库正常读数据

3、架构扩展

随着系统中业务访问量的增加，如果是单机部署数据，会导致I/O访问频率过高

通过主从复制，增加多个数据存储结点，将负载分布在多个从节点上，降低单机的I/O访问频率，提高单机的I/O性能

3. 原理

MySQL主从复制涉及到三个线程

一个运行在主节点： binary log dump thread
两个运行在从节点： I/O thread、SQL thread

1、binary log dump

当从节点连接主节点的时候，主节点创建该线程，用于发送bin-log内容

2、I/O thread

当从节点执行“start slave”命令之后，从节点会创建一个I/O线程用来连接主节点，请求其中的数据。I/O线程接收到主节点binlog dump的更新数据之后，保存在本地的relay log中

3、SQL线程

该线程负责读取relay log中的内容，解析或具体的操作并执行，最终保证主从数据的一致性

4. 基本过程

1、从节点I/O进程连接主节点

请求指定日志文件的指定位置后面的内容

2、主节点接收到请求之后

通过负责复制的I/O进程根据请求的信息读取指定的日志位置之后的日志信息，返回给从节点。返回信息中除了日志所包含的指定日志信息还包含了本次返回信息的 bin-log file 以及 bin-log position

3、从节点的I/O线程接收到内容之后

将接收到的日志内容更新到本机的 relay log 中，并且把读取到的 binary log 文件名和位置保存到 master-info 文件中，方便下一次告知 master 从节点需要更新的位置

4、Slave 的 SQL 线程检测到 relay-log 中新增了内容

将 relay-log 的内容解析成在主节点上实际执行的操作，并在数据库中执行

5. 主从复制模式

1. 异步模式
2. 半同步模式
3. 全同步模式
4. GTID复制模式

6. MySQL高可用方案类型

（1）基于主从复制

- 一般情况下，采用双节点主从+ `keepalived/heartbeat` 方案
- 在master节点发生故障以后，复用 `keepalived/heartbeat` 的高可用机制实现快速切换到slave节点

（2）基于Galera协议
（3）基于NDB协议
（4）基于中间件/proxy
（5）基于共享存储
（6）基于主机高可用

MySQL架构升级之路

数据库的演变升级

1、单机

（1）请求量大查询慢
（2）单机故障导致业务不可用

2、主从

（1）数据库主从同步，从库可以水平扩展，满足更大读需求
（2）但单服务器TPS，内存，IO都是有限的
（3）可以考虑用主节点负责业务，从节点负责统计

3、双主

（1）用户量级上来后，写请求越来越多
（2）一个Master是不能解决问题的，添加多了个主节点进行写入
（3）多个主节点数据要保存一致性，写操作需要2个master之间同步更加复杂

4、分库和分表

自研工具类、tddl、shardingsphere、mycat等

【面试题】业务增长-数据库性能优化：

这边有个数据库-单表1千万数据，未来1年还会增长多500万，性能比较慢，说下你的优化思路?

思路：

千万不要一上来就说分库分表，这个是最忌讳的事项，一定要根据实际情况分析，两个角度思考

不分库分表：

（1）软优化：

1. 数据库参数调优
2. 分析慢查询SQL语句，分析执行计划，进行sql改写和程序改写
3. 优化数据库索引结构
4. 优化数据表结构优化
5. 引入NOSQL和程序架构调整

（2）硬优化

提升系统硬件（更快的IO、更多的内存）：带宽、CPU、硬盘

（3）分库分表

根据业务情况而定，选择合适的分库分表策略（没有通用的策略），外卖、物流、电商领域。

先看只分表是否满足业务的需求和未来增长，数据库分表能够解决单表数据量很大的时,数据查询的效率问题。

无法给数据库的并发操作带来效率上的提高，分表的实质还是在一个数据库上进行的操作，受数据库IO性能的限制如果单分表满足不了需求，再分库分表一起。

结论：在数据量及访问压力不是特别大的情况，首先考虑缓存、读写分离、索引技术等方案，如果数据量极大，且业务持续增长快，再考虑分库分表方案。

5、MySQL数据库分库分表后带来的优点

分库分表解决的现状问题：

- 可以解决数据库本身的瓶颈以及系统本身的瓶颈
- 解决数据库本身瓶颈：

连接数：连接数过多时，就会出现‘too many connections’的错误，访问量太大或者数据库设置的最大连接数太小的原因
Mysql默认的最大连接数为100可以修改，而mysql服务允许的最大连接数为16384。

- **数据库分表可以解决单表海量数据的查询性能问题**
- **数据库分库可以解决单台数据库的并发访问压力问题**
- 解决系统本身IO、CPU瓶颈

磁盘读写IO瓶颈，热点数据太多，尽管使用了数据库本身缓存，但是依旧有大量IO,导致sql执行速度慢

网络IO瓶颈，请求的数据太多，数据传输大，网络带宽不够，链路响应时间变长

CPU瓶颈，尤其在基础数据量大单机复杂SQL计算，SQL语句执行占用CPU使用率高，也有扫描行数大、锁冲突、锁等待等原因

可以通过 show processlist; 、show full processlist，发现 CPU 使用率比较高的SQL

常见的对于查询时间长，State 列值是 Sending data，Copying to tmp table，Copying to tmp table on disk，Sorting result，Using filesort 等都是可能有性能问题SQL，清楚相关影响问题的情况可以kill掉

也存在执行时间短，但是CPU占用率高的SQL，通过上面命令查询不到，这个时候最好通过执行计划分析explain进行分析

面试题

排它锁和共享锁是什么，有什么区别?

- 排它锁也叫独占锁、写锁或 X 锁，锁定的数据只允许进行锁定操作的事务使用，其他事务无法对已锁定的数据进行查询或修改。当我们对数据进行更新的时候，也就是INSERT、DELETE或者UPDATE的时候，数据库也会自动使用排它锁，防止其他事务对该数据行进行操作。
- 共享锁也叫读锁或 S 锁 ，锁定的资源可以被其他用户读取，但不能修改。在进行SELECT的时候，会将对象进行共享锁锁定，当数据读取完毕之后，就会释放共享锁，这样就可以保证数据在读取时不被修改。

区别：

- 排它锁是独占的，一次只能由一个事务持有。它阻止其他事务获取相同资源的任何锁。共享锁是非独占的，允许多个事务同时持有相同资源的共享锁。它允许多个事务同时读取相同的数据。
- 排它锁通常用于写操作，例如更新或删除数据，共享锁通常用于读操作，例如查询数据。

MySQL的行级锁有哪些？作用是什么?

- `Record Lock`记录锁，仅仅把一条记录锁上，记录锁分为排他锁和共享锁。
- `Gap Lock`间隙锁，锁定一个范围，但是不包含记录本身，只存在于可重复读隔离级别，目的是为了解决可重复读隔离级别下幻读的现象。间隙锁之间是兼容的，两个事务可以同时持有包含共同间隙范围的间隙锁，并不存在互斥关系。
- `Next-Key Lock`临键锁：Record Lock + Gap Lock 的组合，锁定一个范围，并且锁定记录本身。next-key lock 即能保护该记录，又能阻止其他事务将新纪录插入到被保护记录前面的间隙中。

数据库悲观锁和乐观锁的原理和使用场景有哪些?

1、乐观锁

认为**对同一数据的并发操作不会总发生**，属于小概率事件，**不用每次都对数据上锁**，也就是不采用数据库自身的锁机制，而是通过程序来实现。在程序上，我们可以采用**版本号机制或者时间戳机制**实现。

2、悲观锁（Pessimistic Locking）

也是一种思想，对数据被其他事务的修改持**保守态度**，会通过数据库自身的锁机制来实现，从而保证数据操作的排它性。

3、乐观锁和悲观锁的适用场景

（1）乐观锁

适合读操作多的场景，相对来说写的操作比较少。它的优点在于程序实现，不存在死锁问题，不过适用场景也会相对乐观，因为它阻止不了除了程序以外的数据库操作。

（2）悲观锁

适合写操作多的场景，因为写的操作具有排它性。采用悲观锁的方式，可以在数据库层面阻止其他事务对该数据的操作权限，防止读 - 写和写 - 写的冲突。但是加锁的时间会比较长，可能会长时间限制其他用户的访问，也就是说他的并发访问性不好。

执行一条select语句，发生了什么?

- 连接器：跟客户端建立连接、获取权限、维持和管理连接。
- 查询缓存：查询语句如果命中查询缓存则直接返回，否则继续往下执行。MySQL 8.0 已删除该模块；
- 解析 SQL，通过解析器对 SQL 查询语句进行词法分析、语法分析，然后构建语法树。
- 执行 SQL：执行 SQL 共有三个阶段：
    - 预处理阶段：检查表或字段是否存在；将 `select *` 中的 `*` 符号扩展为表上的所有列。
    - 优化阶段：基于查询成本的考虑， 选择查询成本最小的执行计划；
    - 执行阶段：根据执行计划执行 SQL 查询语句，从存储引擎读取记录，返回给客户端；

事务的四大特性有哪些？【常考】

事务的四大特性通常被称为 ACID 特性

1. 原子性：确保事务的所有操作要么全部执行成功，要么全部失败回滚，不存在部分成功的情况。
2. 一致性：事务在执行前后，数据库从一个一致性状态转变到另一个一致性状态。
3. 隔离性：多个事务并发执行时，每个事务都应该被隔离开来，一个事务的执行不应该影响其他事务的执行。
4. 持久性：一旦事务被提交，它对数据库的改变就是永久性的，即使在系统故障或崩溃后也能够保持。

数据库的事务隔离级别有哪些？【常考】

1. **读未提交（Read Uncommitted）：**
    - 允许一个事务读取另一个事务尚未提交的数据修改。
    - 最低的隔离级别，存在脏读、不可重复读和幻读的问题。
2. **读已提交（Read Committed）：**
    - 一个事务只能读取已经提交的数据。其他事务的修改在该事务提交之后才可见。
    - 解决了脏读问题，但仍可能出现不可重复读和幻读。
3. **可重复读（Repeatable Read）：**
    - 事务执行期间，多次读取同一数据会得到相同的结果，即在事务开始和结束之间，其他事务对数据的修改不可见。
    - 解决了不可重复读问题，但仍可能出现幻读。
4. **序列化（Serializable）：**
    - 最高的隔离级别，确保事务之间的并发执行效果与串行执行的效果相同，即不会出现脏读、不可重复读和幻读。

为什么MySQL索引使用B+树

- B+树的非叶子节点不存放实际的记录数据，仅存放索引，所以**数据量相同的情况下，相比存储即存索引又存记录的 B 树，B+树的非叶子节点可以存放更多的索引，因此 B+ 树可以比 B 树更「矮胖」，查询底层节点的磁盘 I/O次数会更少**。
- B+ 树有大量的冗余节点（所有非叶子节点都是冗余索引），这些冗余索引让 B+ 树在插入、删除的效率都更高，比如删除根节点的时候，不会像 B 树那样会发生复杂的树的变化；

- B+ 树叶子节点之间用链表连接了起来，有利于范围查询，而 B 树要实现范围查询，因此只能通过树的遍历来完成范围查询，这会涉及多个节点的磁盘 I/O 操作，范围查询效率不如 B+ 树。

你对关系范式有哪些了解

关系数据库设计中，关系范式是一种规范化的设计方法，它的目标是减少数据冗余、提高数据一致性和避免插入、更新和删除操作中的异常。最常见的是第一范式（1NF）、第二范式（2NF）和第三范式（3NF）。

第一范式(1NF)：用来确保每列的原子性，要求每列（或者每个属性值）都是不可再分的最小数据单元（也称为最小的原子单元）。

第二范式(2NF)：在第一范式的基础上更进一层，要求表中的每列都和主键相关，即要求实体的唯一性。如果一个表满足第一范式，并且除了主键以外的其他列全部都依赖于该主键，那么该表满足第二范式。

第三范式(3NF)：在第二范式的基础上更进一层，第三范式是确保每列都和主键列直接相关，而不是间接相关，即限制列的冗余性。如果一个关系满足第二范式，并且除了主键以外的其他列都依赖于主键列，列和列之间不存在相互依赖关系，则满足第三范式。

可以对数据库表做那些优化?

1. 合理使用数据库分表

对于一些特别大的表，可以考虑将其拆分成多个子表，从而更好地管理数据。

2. 建立索引

在经常被查询的列上建立索引，提高查询性能。但是也要注意过多的索引影响插入、更新和删除的性能。

3. 避免使用 `Select *`

只选择需要的列而不是使用`SELECT *`

4. 选择合适的数据类型

- 尽量使用TINYINT，SMALLINT，MEDIUM_INT替代INT类型，如果是非负则加上UNSIGNED
- `VARCHAR`的长度只分配真正需要的空间
- 尽量使用整数或者枚举替代字符串类型
- 时间类型尽量使用`TIMESTAMP`而非`DATETIME`
- 单表不要放太多字段
- 尽量少使用NULL，很难查询优化而且占用额外索引空间

了解MongoDB嘛，它和MySQL有哪些区别

数据库模型
MySQL：关系型数据库，数据以表格的形式存储，每个表包含多个行和列，其中每一行是一个记录。
MongoDB：非关系型数据库(文档型数据库)，数据以BSON（Binary JSON）文档的形式存储，文档可以包含嵌套结构和数组。每个文档都有一个唯一的_id字段作为主键。

查询语句
MySQL：使用结构化查询语言（SQL）进行查询
MongoDB：使用MongoDB自己的查询方式

模式设计

MySQL： MySQL是有模式的数据库，表结构需要在设计时明确定义，包括字段名、数据类型。

MongoDB： MongoDB是无模式的数据库，文档可以根据需要动态添加字段，没有固定的表结构。

存储引擎

MySQL：MySQL支持多种存储引擎，如InnoDB、MyISAM等，每个引擎有不同的特性和适用场景。

MongoDB： MongoDB使用存储引擎WiredTiger，默认提供高性能的读写能力和压缩特性。

适用场景

MySQL： 适用于需要处理结构化数据，支持事务处理和复杂查询的应用。

MongoDB： 适用于需要处理大量非结构化或半结构化数据

扩展性
MySQL： 通常使用主从复制和垂直分区来实现扩展性。

MongoDB： 具有较好的横向扩展性，通过分片可以在多台机器上分布数据。

总结：如果数据是结构化的且需要复杂的事务支持，MySQL更适合；如果数据是非结构化或半结构化的，且需要横向扩展性，MongoDB更适合。

数据库

什么是慢查询？原因是什么？可以怎么优化？【常考】

数据库查询的执行时间超过指定的超时时间时（long_query_time：默认10秒 ），就被称为慢查询。

原因：

- 查询语句比较复杂：查询涉及多个表，包含复杂的连接和子查询，可能导致执行时间较长。
- 查询数据量大：当查询的数据量庞大时，即使查询本身并不复杂，也可能导致较长的执行时间。
- 缺少索引：如果查询的表没有合适的索引，需要遍历整张表才能找到结果，查询速度较慢。
- 数据库设计不合理：数据库表设计庞大，查询时可能需要较多时间。
- 并发冲突：当多个查询同时访问相同的资源时，可能发生并发冲突，导致查询变慢。
- 硬件资源不足：如果MySQL服务器上同时运行了太多的查询，会导致服务器负载过高，从而导致查询变慢

优化：

1. 运行语句，找到慢查询的sql
2. 查询区分度最高的字段
3. explain：显示mysql如何使用索引来处理select语句以及连接表，可以帮助选择更好的索引、写出更优化的查询语句
4. `order by limit`形式的sql语句，让排序的表优先查
5. 考虑建立索引原则
6. 了解业务使用场景

数据库-Redis

基础知识讲解

Redis基础

Redis是什么，有哪些特点

Redis是一个开源的基于内存的数据库，**读写速度非常快，通常被用作缓存**、消息队列、分布式锁和键值存储数据库。它支持多种数据结构，如字符串、哈希表、列表、集合、有序集合等，除此之外，Redis 还支持事务 、持久化、Lua 脚本、多种集群方案（主从复制模式、哨兵模式、切片机群模式）、发布/订阅模式，内存淘汰机制、过期删除机制等。

具有以下特点：

- 基于内存： Redis 将数据存储在内存中，使得它具有快速的读写访问速度。这也使得 Redis 适用于需要高性能的应用场景，比如缓存。
- 持久性：虽然 Redis 主要是内存中的存储系统，但它可以通过将数据持久化到磁盘上的快照和日志文件来保证数据的持久性，以防止数据丢失。
- 多数据结构：Redis 不仅支持简单的键值对存储，还提供了丰富的数据结构，如列表、哈希表、集合等，使得它更灵活地适应不同的应用场景。
- 原子性操作： Redis 支持原子性操作，这意味着它可以保证一个操作是原子的，要么执行成功，要么完全不执行，这对于并发环境下的数据一致性很重要。
- **分布式**： Redis 提供了分布式特性，可以将数据分布在多个节点上，以提高可扩展性和可用性。

为什么要使用 Redis 而不仅仅依赖 MySQL

Redis 具备「高性能」和「高并发」两种特性

- Redis读写非常快速，对于需要频繁读写的数据，特别是**缓存数据，**Redis 的性能优势非常明显，将一部分常用的、频繁访问的数据存储在 Redis 中，可以有效减轻对 MySQL 等持久性数据库的压力。
- Redis 提供了丰富的数据结构，这使得 Redis 在处理特定类型数据和实现特定功能时更为灵活。
- Redis 能够更好地处理**高并发**请求，尤其适用于一些需要快速响应的场景。通过缓存和原子性操作，可以降低数据库的并发压力

Redis 更适合处理高速、高并发的数据访问，以及需要复杂数据结构和功能的场景，在实际应用中，很多系统会同时使用 MySQL 和 Redis。

Redis是单线程吗

Redis 是单线程的，但是**Redis 单线程指的是网络请求模块使用单线程进行处理，其他模块仍用多个线程**，Redis程序并不是单线程的，在启动的时候，会启动后台线程。

- **Redis 在 2.6 版本**，会启动 2 个后台线程，分别处理关闭文件、AOF 刷盘这两个任务；
- **Redis 在 4.0 版本之后**，新增了一个新的后台线程，用来异步释放 Redis 内存，也就是 lazyfree 线程。

之所以 Redis 为「关闭文件、AOF 刷盘、释放内存」这些任务创建单独的线程来处理，是因为这些任务的操作都是很耗时的，如果把这些任务都放在主线程来处理，那么 Redis 主线程就很容易发生阻塞，这样就无法处理后续的请求了。

为了提高网络 I/O 的并行度，Redis 6.0 **采用了多个 I/O 线程来处理网络请求**，但是对于命令的执行，Redis 仍然使用单线程来处理。

Redis单线程为什么还这么快

- 基于内存存储：Redis 将数据存储在内存中，而不是磁盘上。内存访问速度远远快于磁盘访问速度，因此 Redis 能够快速读取和写入数据。
- 非阻塞单线程： Redis 使用单线程模型，不需要进行多线程间的上下文切换，避免了多线程带来的竞争和同步开销。此外，Redis 使用了非阻塞 I/O，可以更高效地处理并发请求。
- 高效的数据结构：Redis专门设计了STRING、LIST、HASH等高效的数据结构，依赖各种数据结构提升了读写的效率。
- **I/O多路复用**：采用I/O多路复用机制同时监听多个Socket，内核会一直监听这些 Socket 上的连接请求或数据请求。一旦有请求到达，就会根据Socket上的事件来选择对应的事件处理器进行处理，这就实现了一个 Redis 线程处理多个 IO 流的效果。

Redis数据类型和数据结构

Redis五种数据结构和应用场景

Redis 提供了丰富的数据类型，常见的有五种数据类型：**String（字符串），Hash（哈希），List（列表），Set（集合）、Zset（有序集合）**。

结构类型	结构存储的值	结构的读写能力
String 字符串	可以是字符串、整数或浮点数	对整个字符串或字符串的一部分进行操作；对整数或浮点数进行自增或自减操作；
List列表	一个链表，链表上的每个节点都包含一个字符串	对链表的两端进行push和pop操作，读取单个或多个元素；根据值查找或删除元素；
Set集合	包含字符串的无序集合	字符串的集合，包含基础的方法有看是否存在添加、获取、删除；还包含计算交集、并集、差集等
Hash散列	包含键值对的无序散列表	包含方法有添加、获取、删除单个元素
Zset有序集合	和散列一样，用于存储键值对	字符串成员与浮点数分数之间的有序映射；元素的排列顺序由分数的大小决定；包含方法有添加、获取、删除单个元素以及根据分值范围或成员来获取元素

1. String

String类型的**底层的数据结构实现主要是SDS(简单动态字符串)**。应用场景主要有：

- 缓存对象：例如可以用STRING缓存整个对象的JSON。
- 计数：Redis处理命令是单线程，所以执行命令的过程是原子的，因此String数据类型适合计数场景，比如计算访问次数、点赞、转发、库存数量等等。
- 分布式锁：可以利用SETNX命令。
- 共享Session信息：服务器都会去同一个Redis获取相关的Session信息，解决了分布式系统下Session存储的问题。

命令：

get、set、del

2. List

List 类型的底层数据结构是由**双向链表或压缩列表**实现的：

- 如果列表的元素个数小于 512 个，列表每个元素的值都小于 64 字节，Redis 会使用**压缩列表**作为 List 类型的底层数据结构；
- 如果列表的元素不满足上面的条件，Redis 会使用**双向链表**作为 List 类型的底层数据结构；

在 Redis 3.2 版本之后，List 数据类型底层数据结构只由 quicklist 实现，替代了双向链表和压缩列表。在 Redis 7.0 中，压缩列表数据结构被废弃，由 listpack 来实现。应用场景主要有：

- **微信朋友圈点赞**：要求按照点赞顺序显示点赞好友信息，如果取消点赞，移除对应好友信息。
- **消息队列**：可以使用左进右出的命令组成来完成队列的设计。比如：数据的生产者可以通过Lpush命令从左边插入数据，多个数据消费者，可以使用BRpop命令阻塞的"抢"列表尾部的数据。

命令：

rpush、lrange、index、lpop

3. Set

Set 类型的底层数据结构是由**哈希表或整数集合**实现的：

- 如果集合中的元素都是整数且元素个数小于 512个，Redis 会使用**整数集合**作为 Set 类型的底层数据结构；
- 如果集合中的元素不满足上面条件，则 Redis 使用**哈希表**作为 Set 类型的底层数据结构。

应用场景主要有：

- **点赞**：key 是文章id，value 是用户id。
- **共同关注**：Set 类型支持交集运算，所以可以用来计算共同关注的好友、公众号等。key 可以是用户id，value 则是已关注的公众号的id。
- **抽奖活动**：存储某活动中中奖的用户名 ，Set 类型因为有去重功能，可以保证同一个用户不会中奖两次。

命令：

sadd、smember、sismember、srem

4. Hash

Hash 类型的底层数据结构是由**压缩列表或哈希表**实现的：

- 如果哈希类型元素个数小于 512 个，所有值小于 64 字节的话，Redis 会使用压缩列表作为 Hash 类型的底层数据结构；
- 如果哈希类型元素不满足上面条件，Redis 会使用哈希表作为 Hash 类型的底层数据结构。

在Redis 7.0 中，压缩列表数据结构被废弃，交由 listpack 来实现。应用场景主要有：

- **缓存对象**：一般对象用 String + Json 存储，对象中某些频繁变化的属性可以考虑抽出来用 Hash 类型存储。
- **购物车**：以用户 id 为 key，商品 id 为 field，商品数量为 value，恰好构成了购物车的3个要素。

命令：

hset、hget、hgetall、hdel

5. Zset

Zset类型(Sorted Set，有序集合)可以根据元素的权重来排序，可以自己来决定每个元素的权重值。比如说，可以根据元素插入Sorted Set 的时间确定权重值，先插入的元素权重小，后插入的元素权重大。

Zset 类型的底层数据结构是由**压缩列表或跳表**实现的, 应用场景主要有：

- 在面对需要展示最新列表、排行榜等场景时，如果数据更新频繁或者需要分页显示，可以优先考虑使用Zset。
- **排行榜**：有序集合比较典型的使用场景就是排行榜。例如学生成绩的排名榜、游戏积分排行榜、视频播放排名、电商系统中商品的销量排名等。

命令：

zadd、zrange、zrangebyscore、zrem

随着 Redis 版本的更新，后面又支持了四种数据类型： **BitMap（2.2 版新增）、HyperLogLog（2.8 版新增）、GEO（3.2 版新增）、Stream（5.0 版新增）**

BitMap

bit 是计算机中最小的单位，使用它进行储存将非常节省空间，特别适合一些数据量大且使用二值统计的场景。可以用于签到统计、判断用户登陆态等操作。

HyperLogLog

HyperLogLog用于基数统计，统计规则是基于概率完成的，不准确，标准误算率是 0.81%。优点是，在输入元素的数量或者体积非常非常大时，所需的内存空间总是固定的、并且很小。比如百万级网页 UV 计数等；

GEO

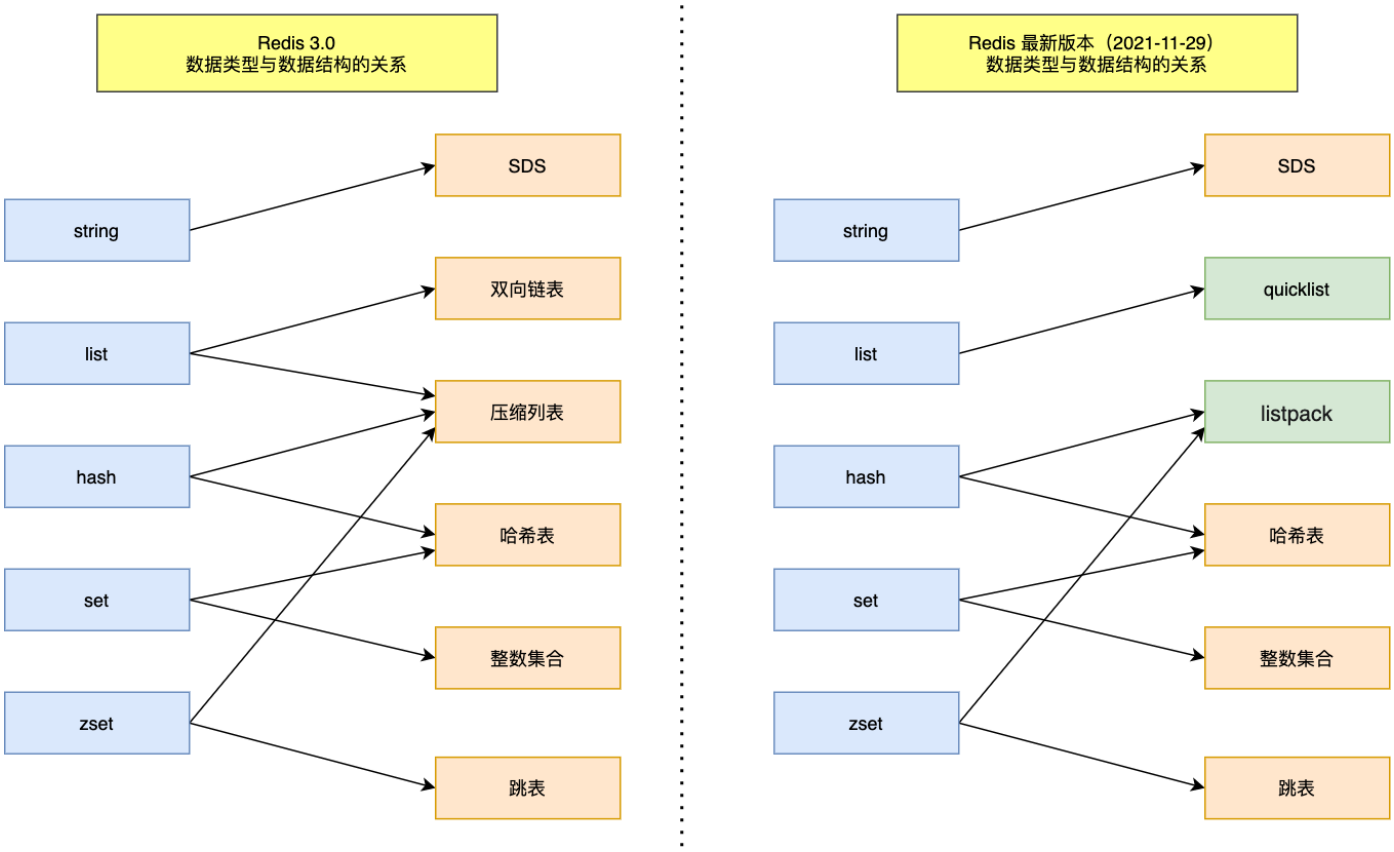
主要用于存储地理位置信息，并对存储的信息进行操作。底层是由Zset实现的，使用GeoHash编码方法实现了经纬度到Zset中元素权重分数的转换，这其中的两个关键机制就是「对二维地图做区间划分」和「对区间进行编码」。一组经纬度落在某个区间后，就用区间的编码值来表示，并把编码值作为Zset元素的权重分数。

Stream

Redis专门为消息队列设计的数据类型。相比于基于 List 类型实现的消息队列，有这两个特有的特性：自动生成全局唯一消息ID，支持以消费组形式消费数据。

之前方法缺陷：不能持久化，无法可靠的保存消息，并且对于离线重连的客户端不能读取历史消息。

Redis底层数据结构



1. SDS: 简单动态字符串

- `SDS`可以存储字符串，还可以存储二进制数据，包括空字符。这使得SDS在处理二进制数据时更为灵活，不受空字符的限制。
- 缓存长度信息：SDS在头部保存了字符串的长度信息，因此可以在O(1)的时间复杂度内获取字符串的长度。这样，不需要遍历整个字符串来计算长度，提高了获取长度的效率。
- 动态扩容：SDS可以根据实际存储的数据动态扩容。当字符串长度变长时，SDS会自动进行内存的扩展，而不需要像C语言中的传统字符串那样手动管理内存。

2. 双端链表

Redis中的`List`就是用双端链表实现的，双端链表的链表节点可以保存不同类型的值，支持在两端进行元素的快速插入和删除，并且链表结构提供了表头指针和表尾指针，获取链表的表头节点和表尾节点的时间复杂度只需O(1)；获取链表数量的时间复杂度也只需O(1);

缺陷：

- 链表每个节点之间的内存都是不连续的，意味着**无法很好利用 CPU 缓存**
- 保存一个链表节点的值都需要一个链表节点结构头的分配，**内存开销较大**。

3. 压缩列表

压缩列表是一种紧凑的、可变长度，**由连续内存块组成的顺序型数据结构**，类似于数组，被用于存储列表和哈希表的数据。压缩列表在内存使用效率上相对较高，它可以根据数据大小进行灵活的扩容和收缩。

缺陷：

- 空间扩展操作也就是重新分配内存，因此连锁更新一旦发生，就会导致压缩列表占用的内存空间要多次重新分配，直接影响到压缩列表的访问性能。
- 如果保存的元素数量增加了，或是元素变大了，会导致内存重新分配，会有连锁更新的问题。

- 压缩列表只会用于保存的节点数量不多的场景，只要节点数量足够小，即使发生连锁更新也能接受。

4. 哈希表

哈希表是一种保存键值对(key-value)的数据结构。优点在于能以O(1)的复杂度快速查询数据。Redis 采用了拉链法来解决哈希冲突，在不扩容哈希表的前提下，将具有相同哈希值的数据串起来，形成链接。

5. 整数集合

整数集合是一种专门用于存储整数值的数据结构，通过紧凑的二进制表示，提高了整数存储的效率。整数集合被用于存储Redis中的集合数据结构的整数元素。

6. 跳表

跳跃表是一种**在链表基础上改进过来的，实现了一种「多层」的有序链表**，当数据量很大时，跳表的查找复杂度就是O(logN)。用于实现有序集合（Sorted Set）。

跳表的查找过程?

查找一个跳表节点的过程时，跳表会从头节点的最高层开始，逐一遍历每一层。在遍历某一层的跳表节点时，会用跳表节点中的 SDS 类型的元素和元素的权重来进行判断：

- 如果当前节点的权重小于要查找的权重时，跳表就会访问该层上的下一个节点。
- 如果当前节点的权重等于要查找的权重时，并且当前节点的 SDS 类型数据小于要查找的数据时，跳表就会访问该层上的下一个节点。

如果上面两个条件都不满足，或者下一个节点为空时，跳表就会使用目前遍历到的节点的 level 数组里的下一层指针，然后沿着下一层指针继续查找。

7. `quicklist`

`quicklist` 就是双向链表 + 压缩列表组合，quicklist 就是一个链表，而链表中的每个元素又是一个压缩列表。`quicklist`解决办法，通过控制每个链表节点中的压缩列表的大小或者元素个数，来规避连锁更新的问题。因为压缩列表元素越少或越小，连锁更新带来的影响就越小，从而提供了更好的访问性能。

8. `listpack`

listpack 没有压缩列表中记录前一个节点长度的字段，listpack 只记录当前节点的长度，当我们向 listpack 加入一个新元素的时候，不会影响其他节点的长度字段的变化，从而避免了压缩列表的连锁更新问题。

Redis持久化

Redis如何实现数据不丢失

Redis 实现了数据持久化的机制，这个机制会把数据存储到磁盘，这样在 Redis 重启就能够从磁盘中恢复原有的数据。

Redis 共有三种数据持久化的方式：

- **AOF 日志**：每执行一条写操作命令，就把该命令以追加的方式写入到一个文件里；
- **RDB 快照**：将某一时刻的内存数据，以二进制的方式写入磁盘；
- **混合持久化方式**：Redis 4.0 新增的方式，集成了 AOF 和 RBD 的优点；

AOF日志

以日志的形式记录服务器所处理的每一个写操作，redis服务器启动之初，会读取该日志来重新构建数据库，以保证启动后的数据库是完整的。

AOF 日志是写后日志，"写后" 的意思是 Redis 是先执行命令，把数据写入内存，然后才记录日志，Redis是内存和日志(写后日志)，mysql是磁盘数据和日志(写前日志)。

Redis 使用写后日志这一方式的好处：

1. 可以避免出现记录错误命令的情况。
2. 它是在命令执行后才记录日志，所以不会阻塞当前的写操作。

AOF的三种写回测策略

- Always，同步写回；
- Everysec，每秒写回；
- No，操作系统控制的写回。

Always是每次写操作命令执行完后，同步将 AOF 日志数据写回硬盘；**Everysec**每次写操作命令执行完后，先将命令写入到 AOF 文件的内核缓冲区，然后每隔一秒将缓冲区里的内容写回到硬盘；**No**就是不控制写回硬盘的时机。每次写操作命令执行完后，先将命令写入到 AOF 文件的内核缓冲区，再由操作系统决定何时将缓冲区内容写回硬盘。

DPNO

配置项	写回时机	优点	缺点
Always	同步写回	可靠性高，数据基本不丢失	每个写命令都要落盘，性能影响较大
Everysec	每秒写回	性能适中	宕机时丢失1秒内的数据 最多是丢失2s
No	操作系统控制的写回	性能好	宕机时丢失数据较多

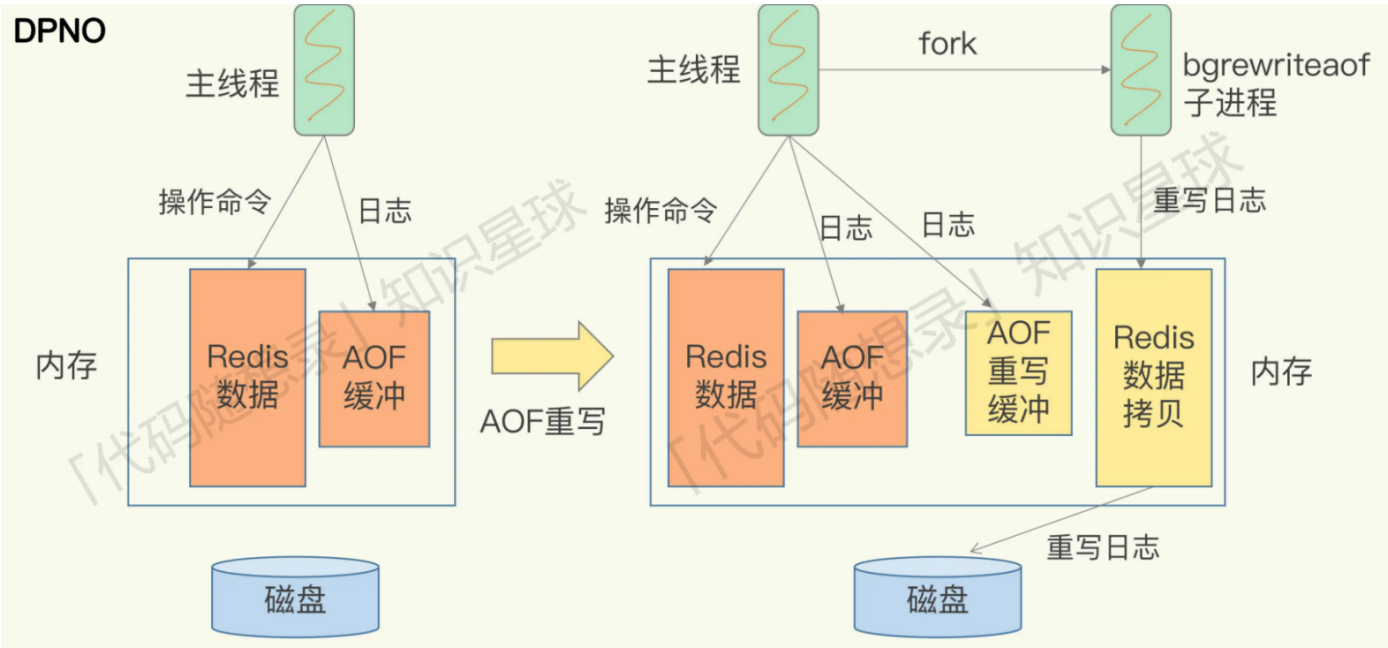
写回策略选择：

想要获得高性能，就选择 No 策略；如果想要得到高可靠性保证，就选择 Always 策略；如果允许数据有一点丢失，又希望性能别受太大影响的话，那么就选择 Everysec 策略。

AOF的磁盘重写机制

随着执行的命令越多，AOF 文件的体积自然也会越来越大，为了避免日志文件过大， Redis 提供了 AOF 重写机制，它会直接扫描数据中所有的键值对数据，然后为每一个键值对生成一条写操作命令，接着将该命令写入到新的 AOF 文件，重写完成后，就替换掉现有的 AOF 日志。重写的过程是由后台子进程完成的，这样可以使得主进程可以继续正常处理命令。

AOF重写过程：一处拷贝，两处日志



AOF 重写的四个触发时机：

- 时机一：bgrewriteaof 命令被执行。
- 时机二：主从复制完成 RDB 文件解析和加载（无论是否成功）。
- 时机三：AOF 重写被设置为待调度执行。
- 时机四：AOF 被启用，同时 AOF 文件的大小比例超出阈值，以及 AOF 文件的大小绝对值超出阈值。

另外，这里还需要注意，在这四个时机下，其实都**不能有正在执行的 RDB 子进程和 AOF 重写子进程**，否则的话，AOF 重写就无法执行了。

AOF 重写的基本执行过程。AOF 重写和 RDB 创建的过程类似，它也是创建了一个子进程来完成重写工作。这是因为 AOF 重写操作，实际上需要遍历 Redis server 上的所有数据库，把每个键值对以插入操作的形式写入日志文件，而日志文件又要进行写盘操作。所以，Redis 源码使用子进程来实现 AOF 重写，这就避免了阻塞主线程，也减少了对 Redis 整体性能的影响。

RDB快照

RDB 快照就是记录某一个瞬间的内存数据，记录的是实际数据。每次执行快照，都是把内存中的「所有数据」都记录到磁盘中。如果频率太频繁，可能会对 Redis 性能产生影响。如果频率太低，服务器故障时，丢失的数据会更多。通常可能设置至少 5 分钟才保存一次快照。

Redis 提供了两个命令来生成 RDB 文件，分别是 save 和 bgsave，他们的区别就在于是否在「主线程」里执行：

- 执行了 save 命令，就会在主线程生成 RDB 文件，由于和执行操作命令在同一个线程，所以如果写入 RDB 文件的时间太长，**会阻塞主线程**；
- 执行了 bgsave 命令，会创建一个子进程来生成 RDB 文件，这样可以**避免主线程的阻塞**；

混合持久化

RDB 优点是数据恢复速度快，但是快照的频率不好把握。频率太低，丢失的数据就会比较多，频率太高，就会影响性能。AOF 优点是丢失数据少，但是数据恢复不快。为了集成了两者的优点， Redis 4.0 提出了**混合使用 AOF 日志和内存快照**，也叫混合持久化，既保证了 Redis 重启速度，又降低数据丢失风险。

混合持久化工作在 **AOF 日志重写过程**，在 AOF 重写日志时，fork出来的重写子进程会先将与主线程共享的内存数据以 RDB 方式写入到 AOF 文件，然后主线程处理的操作命令会被记录在重写缓冲区里，重写缓冲区里的增量命令会以 AOF 方式写入到 AOF 文件，写入完成后通知主进程将新的含有 RDB 格式和 AOF 格式的 AOF 文件替换旧的的 AOF 文件。文件的前半部分是 RDB 格式的全量数据，后半部分是 AOF 格式的增量数据。

这样的好处在于，重启 Redis 加载数据的时候，由于前半部分是 RDB 内容，这样**加载的时候速度会很快**。加载完 RDB 的内容后，才会加载后半部分的 AOF 内容，这里的内容是 Redis 后台子进程重写 AOF 期间，主线程处理的操作命令，可以使得**数据更少的丢失**。

混合持久化的缺点是AOF文件的可读性变差了，以及兼容性比较差（Redis4.0之前版本不支持）

Redis功能

Redis集群

Redi如何实现高可用

1. 主从复制

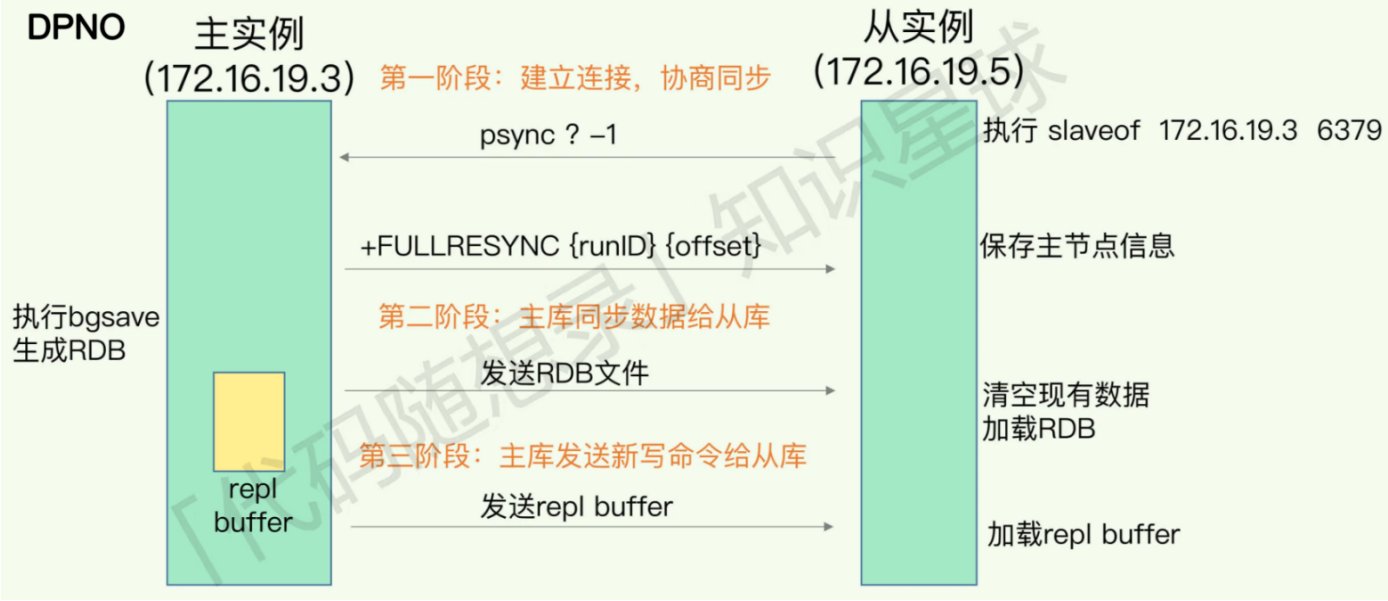
Redis 提供了主从库模式，以保证数据副本的一致，主从库之间采用的是读写分离的方式。

读操作：主库、从库都可以接收；

写操作：首先到主库执行，然后，主库将写操作同步给从库。

全量复制

主从库间进行第一次复制操作（全量复制）



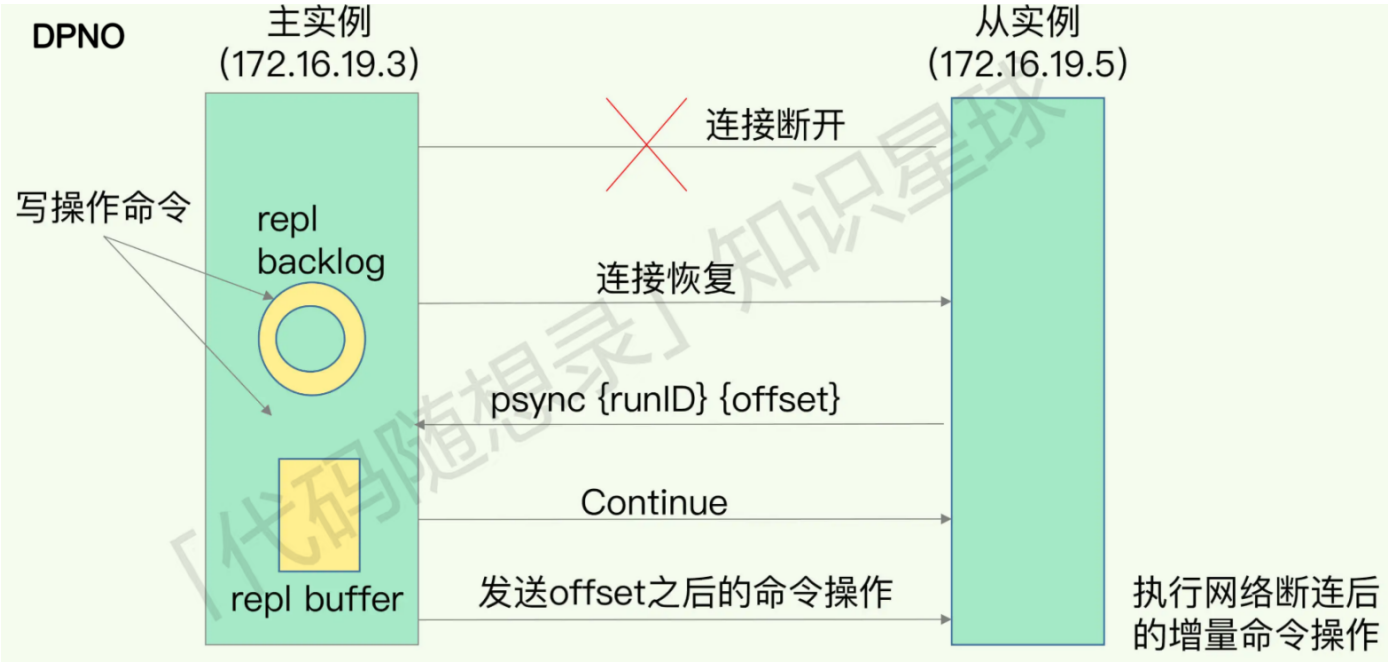
第一阶段是从库和主库建立起连接，并告诉主库即将进行同步，主库确认回复后，主从库间就可以开始同步了。

第二阶段主库收到 psync 命令后，会用 FULLRESYNC 响应命令带上两个参数：主库 runID 和主库目前的复制进度 offset，返回给从库。

第三个阶段，主库会把第二阶段执行过程中新收到的写命令，再发送给从库。

增量复制

从库发生宕机，重新连接后数据的同步操作（增量复制）



若从库发生宕机，主库会把断连期间收到的写操作命令，写到repl_backlog_buffer中，当从库重连后，从库首先会给主库发送 psync 命令，并把自己当前的 slave_repl_offset 发给主库，主库会判断自己的 master_repl_offset 和 slave_repl_offset 之间的差距。若**此时从库相差 > repl_backlog_buffer**说明可能一个从库如果和主库断连时间过长，不能够进行增量复制(因为前面的值被覆盖的了，会导致数据不一致)，所以直接进行全量复制。

而**从库相差 < repl_backlog_buffer**，则将master_repl_offset 和 slave_repl_offset 之间的差距发送给从库进行执行。

Redis 的主从库同步的基本原理，总结来说，有三种模式：全量复制、基于长连接的命令传播，以及增量复制。

- 第一次同步(本来就是全量复制)或者增量复制时master_repl_offset 和 slave_repl_offset差值 >repl_backlog_buffer时(repl_backlog_size这个配置参数) 就无法增量复制，就被迫导致全量复制。
- 主从库正常运行后的常规同步阶段，在这个阶段中，主从库之间通过命令传播实现同步。
- 从库断开重连后进行增量复制。

主从全量同步使用RDB而不使用AOF的原因：

1、RDB文件内容是经过压缩的二进制数据（不同数据类型数据做了针对性优化），文件很小。而AOF文件记录的是每一次写操作的命令，写操作越多文件会变得很大，其中还包括很多对同一个key的多次冗余操作。

2、打开AOF就要选择文件刷盘的策略，选择不当会严重影响Redis性能。而RDB只有在需要定时备份和主从全量同步数据时才会触发生成一次快照。

2. 哨兵模式

当 Redis 的主从服务器出现故障宕机时，需要手动进行恢复，为了解决这个问题，Redis 增加了哨兵模式，哨兵监控主从服务器，并且提供**主从节点故障转移的功能。**

监控：

监控主从是否正常

通知：

出现问题的时候，通知相关人员

故障迁移：

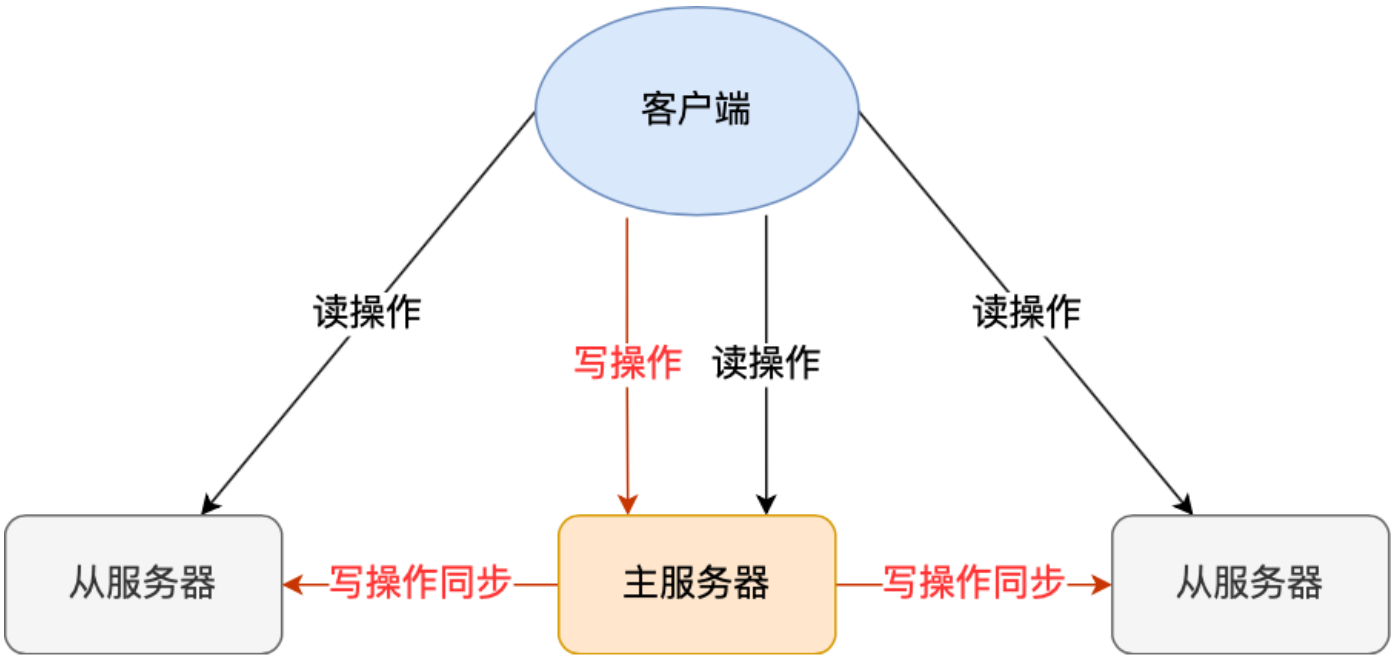
自动主从切换

统一的配置管理：获取主从地址

3. 切片集群

当数据量大到一台服务器无法承载，需要使用Redis切片集群方案，它将数据分布在不同的服务器上，以此来降低系统对单主节点的依赖，提高 Redis 服务的读写性能。

什么是Redis主从复制



主从复制将一个Redis服务器的数据复制到其他服务器，其中一个被视为主节点（master），而其他的是从节点（slave）

- 主服务器上可以进行读写操作，当发生写操作时自动将写操作同步给从服务器
- 从服务器一般只读，并接受主服务器同步过来写操作命令，然后执行这条命令。

主从复制的同步过程

1. 建立连接

从节点通过向主节点发送SYNC命令请求建立连接。如果是初次同步，主节点会创建一个专门的后台线程进行数据传输。

2. 主节点创建RDB快照

如果是初次同步，主节点会执行BGSAVE命令，创建一个RDB（Redis Database Dump）快照，将当前内存中的数据保存到一个文件中。

3. 主节点发送RDB文件和AOF缓冲区内容：

主节点在创建RDB快照后，会将这个RDB文件发送给从节点。同时，主节点将AOF缓冲区中的写命令也发送给从节点。这确保了从节点在收到RDB文件后，能够通过执行AOF缓冲区中的写命令将数据更新至主节点执行BGSAVE时的状态。

4. 从节点载入RDB文件和执行AOF缓冲区命令：

从节点收到RDB文件后，会载入这个文件，将自己的数据状态更新为主节点在执行BGSAVE时的状态。接着，从节点会执行AOF缓冲区中的写命令，以追赶主节点的最新状态。

5. **增量复制：**

一旦初次同步完成，从节点就会转入增量复制的阶段。主节点会实时将写入命令发送给从节点，从节点接收并执行这些写入命令，保持和主节点数据的一致性。

6. 心跳和命令传播

主节点和从节点之间会维持心跳连接，以检测对方的存活状态。主节点将写入命令通过命令传播机制发送给从节点，确保数据的实时同步。

Redis哨兵机制是什么

Redis的哨兵用于监控和管理Redis实例，它可以实现自动故障转移和高可用性。哨兵系统由一组独立运行的进程组成，这些进程定期检查Redis主节点和从节点的状态，并在主节点宕机时选择一个从节点升级为新的主节点。

如何避免主从数据的不一致

- 持久化和复制配置：确保在主节点和从节点上都启用了持久化（如RDB快照和AOF文件）以及复制配置。这样可以保证从节点在重启后能够通过持久化文件进行数据的完整恢复。
- 保证网络稳定性：确保主节点和从节点之间的网络连接是稳定的，比如让主从节点处于同一机房，降低网络延迟
- 监控和报警：使用监控系统来实时监测主从节点的状态，包括复制延迟、连接状态等。设置报警机制。

哨兵机制的工作原理

1. 监控

哨兵定期检查与其关联的所有Redis实例的健康状况，包括主节点和从节点。

2. 故障检测

当哨兵检测到主节点不可用（宕机）时，会尝试选举一个新的主节点。

3. 故障转移

选举新主节点后，哨兵会通知所有相关的从节点更新其配置，使其将选出的新主节点作为其新的主节点。

4. 自动恢复

一旦主节点重新可用，哨兵会将其重新添加到系统中，并根据需要将其配置为从节点，以提供故障恢复和负载分担。

哨兵机制使得Redis在主节点发生故障时能够快速进行自动故障转移，确保系统的高可用性。

Redis切片集群的工作原理

当 Redis 缓存数据量大到一台服务器无法缓存时，就需要使用 **Redis 切片集群**（Redis Cluster ）方案，它将数据分布在不同的服务器上，以此来降低系统对单主节点的依赖，从而提高 Redis 服务的读写性能。

1. 数据分片

切片集群将整个数据集划分为16384个槽（slot），每个槽对应一个整数值。每个节点负责管理其中一部分槽的数据。

2. 节点间通信

切片集群中的节点通过gossip协议相互通信，分享节点信息和集群状态。节点使用PING、PONG等消息保持通信，并通过定期的集群信息交换来保持一致性。

3. 数据分布

当一个新的键值对被添加到切片集群时，根据键的CRC16哈希值确定它属于哪个槽，然后将数据存储到负责该槽的节点上。这确保了数据在集群中均匀分布。

4. 故障检测

切片集群使用哨兵机制来检测节点的健康状态。当一个节点不可用时，其他节点会发起故障检测流程，重新分配槽到可用节点，并选举一个新的主节点。

5. 客户端路由

客户端连接到切片集群时，通过计算键的CRC16哈希值确定它所属的槽，然后将请求发送到负责该槽的节点上。这样，客户端可以直接与负责相应数据的节点通信，而无需中间代理。

6. 数据复制

切片集群使用主从复制机制，每个槽都有一个主节点和若干个从节点。数据在主节点上写入后，会被异步地复制到从节点上，确保数据的持久性和高可用性。

什么是集群脑裂

如果主节点的网络突然发生了问题与所有的从节点都失联了，但此时的主节点和客户端的网络是正常的，客户端不知道集群内部已经出现了问题，还在向这个失联的主节点写数据，此时这些数据被主节点缓存到了缓冲区里。哨兵也发现主节点失联了，就会在从节点中选举出一个leader作为主节点，会导致集群有两个主节点。

网络恢复后哨兵因为之前已经选举出一个新主节点了，它就会把旧主节点降级，然后从旧主节点会向新主节点请求数据同步，**因为第一次同步是全量同步的方式，旧主节点会清空掉自己本地的数据。客户端在过程之前写入的数据就会丢失了**。所以脑裂会导致集群数据的丢失。

集群脑裂带来数据丢失怎么办

当主节点发现从节点下线或者通信超时的总数量小于阈值时，那么禁止主节点进行写数据，直接把错误返回给客户端。**设置主节点连接的从节点中至少有 N 个从节点，并且主节点进行数据复制时的 ACK 消息延迟不能超过 T 秒**，否则，主节点就不会再接收客户端的写请求了。等到新主节点上线时，就只有新主节点能接收和处理客户端请求，此时，新写的数据会被直接写到新主节点中。而原主节点会被哨兵降为从节点，即使它的数据被清空了，也不会有新数据丢失。

Redis过期删除

在Redis中，可以为键设置过期时间，使得键在一定时间后自动过期并被删除。你可以使用`EXPIRE`命令或`SET`命令的`EX`选项来设置过期时间，因此需要有相应的机制将已过期的键值对删除，而做这个工作的就是过期键值删除策略。

过期删除策略有哪些

- 定时删除：**设置 key 的过期时间时，同时创建一个定时事件，当时间到达时，由事件处理器自动执行 key 的删除操作。**这样可以保证过期的 key 会被尽快删除，但删除过期 key 会占用一部分 CPU 时间，对CPU不友好。
- 惰性删除： **不主动删除过期键，每次从数据库访问 key 时，都检测 key 是否过期，如果过期则删除该 key。**这样会使用很少的系统资源，对CPU友好，但是可能会导致过期key长期不被删除，浪费内存空间。
- 定期删除：**每隔一段时间「随机」从数据库中取出一定数量的 key 进行检查，并删除其中的过期key。**这种方法减少了删除操作对cpu的影响，也减少了内存空间的浪费，但是效果也没有两者好，且难以确定删除操作执行的时长和频率。

Redis过期删除策略是什么

Redis 选择「惰性删除+定期删除」这两种策略配和使用，以求在合理使用 CPU 时间和避免内存浪费之间取得平衡。

1. 惰性删除依据`expireIfNeeded`函数来实现，每次返回或者修改key之前，都会调用该函数检查`key`是否过期
    - 如果过期，删除该key, `lazyfree_lazy_expire`参数决定是同步删除还是异步删除，然后返回`null`给客户端
    - 如果没有过期，则返回正常的键给客户端
2. 定期删除的做法是**每隔一段时间「随机」从数据库中取出一定数量的 key 进行检查，并删除其中的过期key。**
    - 默认每隔10s检查一次数据库，配置键为`hz: 10`
    - 随机抽查的数量是20个key

定期删除的流程：

- 从过期字典中随机抽取 20 个 key
- 检查这 20 个 key 是否过期，并删除已过期的 key；
- 如果本轮检查的已过期 key 的数量，超过 5 个（20/4），也就是「已过期 key 的数量」占比「随机抽取 key 的数量」大于 25%，则继续重复步骤 1；如果已过期的 key 比例小于 25%，则停止继续删除过期 key，然后等待下一轮再检查。

Redis内存淘汰

Redis通过参数 `maxmemory <bytes>` 来设定最大运行内存。当系统内存不足时，Redis会根据配置的淘汰策略来删除一些键以释放内存。

内存淘汰策略

1. 不进行数据淘汰

- **NoEviction**： 如果内存不足以执行写操作，Redis将返回错误，默认的淘汰策略。

2. 进行数据淘汰

- **VolatileTTL**：优先淘汰更早过期的键值。
- **VolatitleLRU:** 只对带有过期时间的键使用LRU策略，其他键使用NoEviction策略。
- **VolatitleRandom**： 只对带有过期时间的键使用随机淘汰，其他键使用NoEviction策略。
- **VolatitleLFU**: 只对带有过期时间的键使用LFU策略，其他键使用NoEviction策略。
- **AllKeysLRU**： 根据最近最少使用的原则淘汰最久未使用的键。

- **AllKeysRandom**： 随机选择一个键进行淘汰。
- **AllKeysLFU**： 根据最少频繁使用的原则淘汰最少使用的键。

可以使用 `config get maxmemory-policy` 命令，来查看当前 Redis 的内存淘汰策略

Redis的LRU算法和LFU算法有什么区别

1. **LRU（Least Recently Used）：**
    - LRU算法假设最近被访问的数据更有可能在未来被再次访问，所以选择**最近最少被使用的对象**进行淘汰。
    - Redis 实现的是一种**近似 LRU 算法，在 Redis 的对象结构体中添加一个额外的字段，用于记录此数据的最后一次访问时间**。当 Redis 进行内存淘汰时，会使用**随机采样的方式来淘汰数据**，它是随机取 5 个值（此值可配置），然后**淘汰最久没有使用的那个**。
2. **LFU（Least Frequently Used）：**
    - LFU算法假设使用频率较低的对象在未来仍然会较少被访问，所以选择使用频率最低的对象进行淘汰。
    - LFU维护一个使用计数，每当一个对象被访问时，其使用计数增加。在需要淘汰对象时，选择使用计数最低的对象淘汰。

区别：

- LRU关注的是最近的访问情况，认为最近被访问的对象更可能在未来被再次访问。
- LFU关注的是使用频率，认为使用频率较低的对象在未来仍然会较少被访问。

Redis缓存

因为用户请求访问数据库的请求数量比较多时，数据库很容易崩溃，所以一般将Re dis作为数据库的缓存层，

什么是缓存雪崩

缓存雪崩：在某个时间点，缓存中的大量数据同时失效，大量请求都需要重新计算或重新获取数据，数据库压力剧增。这可能是由于缓存的过期时间设置相近，导致在某个时间点同时失效。或者Redis故障宕机。

1. 大量数据同时过期

针对大量数据同时过期而引发缓存雪崩问题，常见的应对方法有以下几种：

- 均匀设置过期时间: 避免将大量数据设置成同一个过期时间，可以给这些数据的过期时间加一个随机数。
- 互斥锁：如果发现访问的数据不在 Redis 里，就加个互斥锁，保证同一时间内只有一个请求来构建缓存
- 后台更新缓存：让缓存“永久有效”，并将更新缓存的工作交由后台线程定时更新。

2. Redis故障宕机

针对 Redis 故障宕机而引发的缓存雪崩问题，常见的应对方法有下面这几种：

- 服务熔断或请求限流机制：
    - 服务熔断：暂停业务应用对缓存服务的访问，直接返回错误
    - 请求限流：只将少部分请求发送到数据库进行处理，再多的请求就在入口直接拒绝服务
- 构建Redis缓存高可靠集群：
    - 通过**主从节点的方式构建 Redis 缓存高可靠集群**，
    - 如果 Redis 缓存的主节点故障宕机，从节点可以切换成为主节点，继续提供缓存服务，避免了由于 Redis 故障宕机而导致的缓存雪崩问题。

什么是缓存击穿

缓存击穿：当某个“热点数据”过期时，大量并发请求查询这个缓存不存在的数据时，导致请求直接访问数据库，增加数据库的负载。缓存击穿是缓存雪崩的一个子集，可以使用前面说到的方案，添加互斥锁或者后台更新缓存。

什么是缓存穿透

缓存穿透要访问的数据**既不在缓存中，也不在数据库中**，这样就无法构建缓存数据，后续所有的请求都会向数据库查找，增加数据库的负载。

出现缓存穿透一般是因为以下两种原因：

- 业务误操作，缓存中的数据和数据库中的数据都被误删除了
- 恶意攻击，通过构造不存在的 key 大量访问缓存，导致对数据库的频繁查询。

对应的解决办法有3种：

- 对非法请求做限制
- 针对查询的数据，在缓存中设置一个空值或者默认值
- 使用布隆过滤器快速判断数据是否存在，避免通过查询数据库来判断数据是否存在。

缓存异常	产生原因	应对方案
缓存雪崩	大量数据同时过期	- 均匀设置过期时间，避免同一时间过期 - 互斥锁，保证同一时间只有一个应用在构建缓存 - 双 key 策略，主 key 设置过期时间，备 key 永久，主 key 过期时，返回备 key 的内容 - 后台更新缓存，定时更新、消息队列通知更新
	Redis 故障宕机	- 服务熔断 - 请求限流 - 构建 Redis 缓存高可靠集群
缓存击穿	频繁访问的热点数据过期	- 互斥锁 - 不给热点数据设置过期时间，由后台更新缓存
缓存穿透	访问的数据既不在缓存，也不在数据库	- 非法请求的限制； - 缓存空值或者默认值； - 使用布隆过滤器快速判断数据是否存在；

如何保证数据库和缓存的一致性

Cache Aside策略

细分为「读策略」和「写策略」

写策略的步骤：

- 更新数据库中的数据；
- 删除缓存中的数据。

读策略的步骤：

- 如果读取的数据命中了缓存，则直接返回数据；
- 如果读取的数据没有命中缓存，则从数据库中读取数据，然后将数据写入到缓存，并且返回给用户。

「先更新数据库 + 再删除缓存」的方案，是可以保证数据一致性的，但是每次更新数据的时候，缓存的数据都会被删除，这样会对缓存的命中率带来影响。

如果业务对缓存命中率有很高的要求，我们可以采用「更新数据库 + 更新缓存（不是删除缓存，而是更新缓存）」的方案，因为更新缓存并不会出现缓存未命中的情况，但是在两个更新请求并发执行的时候，会出现数据不一致的问题，可以用以下手段来解决。

- 在更新缓存前先加个**分布式锁**，保证同一时间只运行一个请求更新缓存，就会不会产生并发问题了。
- 在更新完缓存时，给缓存加上较短的**过期时间**

如何保证删除缓存操作一定能成功?

重试机制

引入消息队列

- 如果应用**删除缓存失败**，可以从消息队列中重新读取数据，然后再次删除缓存，这个就是**重试机制**。如果重试超过的一定次数，还是没有成功，就需要向业务层发送报错信息了。
- 如果**删除缓存成功**，就要把数据从消息队列中移除，避免重复操作，否则就继续重试。

订阅BINLog

订阅 binlog 日志，拿到具体要操作的数据，然后再执行缓存删除。可以让删除服务模拟自己伪装成一个 MySQL 的从节点，向 MySQL 主节点发送 dump 请求，主节点收到请求后，就会开始推送 BINLog ，删除服务解析 BINLog 字节流之后，转换为便于读取的结构化数据，再进行删除。

面试题

什么是Redis, 具有哪些特点

Redis是一个基于内存的数据库，**读写速度非常快，通常被用作缓存**、消息队列、分布式锁和键值存储数据库。它支持多种数据结构，如字符串、哈希表、列表、集合、有序集合等， Redis 还提供了分布式特性，可以将数据分布在多个节点上，以提高可扩展性和可用性。

Redis为什么快

- **基于内存操作**： 传统的磁盘文件操作相比减少了IO，提高了操作的速度。
- 高效的数据结构：Redis专门设计了STRING、LIST、HASH等高效的数据结构，依赖各种数据结构提升了读写的效率。
- 单线程：单线程操作省去了上下文切换带来的开销和CPU的消耗，同时不存在资源竞争，避免了死锁现象的发生。
- I/O多路复用：采用I/O多路复用机制同时监听多个Socket，根据Socket上的事件来选择对应的事件处理器进行处理。

为什么Redis是单线程

Redis单线程指的是：网络请求模块使用单线程进行处理，其他模块仍用多个线程，单线程减少了上下文切换的开销，也减少了资源竞争而导致的锁的问题，提升了整体性能。

官方答案是：因为CPU不是Redis的瓶颈，Redis的瓶颈最有可能是机器内存或者网络带宽。既然单线程容易实现，而且CPU不会成为瓶颈，那就顺理成章地采用单线程的方案了。

Redis为什么又要引入多线程呢

因为Redis的瓶颈不在内存，而是在网络I/O模块带来CPU的耗时，所以Redis6.0的多线程是用来处理网络I/O这部分，充分利用CPU资源，减少网络I/O阻塞带来的性能损耗。Redis引入的多线程 I/O 特性对性能提升至少是一倍以上。

Redis数据类型 【常问】

Redis 常见的五种数据类型：**String（字符串），Hash（哈希），List（列表），Set（集合）及 Zset(sorted set：有序集合)**。

1. 字符串`STRING`：存储字符串数据，最基本的数据类型。
2. 哈希表`HASH`：存储字段和值的映射，用于存储对象。
3. 列表`LIST`：存储有序的字符串元素列表。
4. 集合`SET`：存储唯一的字符串元素，无序。
5. 有序集合`ZSET`：类似于集合，但每个元素都关联一个分数，可以按分数进行排序。

Redis版本更新，又增加了几种数据类型，

- `BitMap`: 存储位的数据结构，可以用于处理一些位运算操作。
- `HyperLogLog`：用于基数估算的数据结构，用于统计元素的唯一数量。
- `GEO`： 存储地理位置信息的数据结构。
- `Stream`：专门为消息队列设计的数据类型。

Redis数据类型的应用场景

Redis 五种常见数据类型的应用场景：

- String 类型的应用场景：缓存对象、常规计数、分布式锁、共享session信息等。
- List 类型的应用场景：消息队列（有两个问题：1. 生产者需要自行实现全局唯一 ID；2. 不能以消费组形式消费数据）等。
- Hash 类型：缓存对象、购物车等。
- Set 类型：聚合计算（并集、交集、差集）场景，比如点赞、共同关注、抽奖活动等。
- Zset 类型：排序场景，比如排行榜、电话和姓名排序等。

Redis 后续版本又支持四种数据类型，它们的应用场景如下：

- BitMap（2.2 版新增）：二值状态统计的场景，比如签到、判断用户登陆状态、连续签到用户总数等；
- HyperLogLog（2.8 版新增）：海量数据基数统计的场景，比如百万级网页 UV 计数等；
- GEO（3.2 版新增）：存储地理位置信息的场景，比如滴滴叫车；
- Stream（5.0 版新增）：消息队列，相比于基于 List 类型实现的消息队列，有这两个特有的特性：自动生成全局唯一消息ID，支持以消费组形式消费数据。

Redis底层使用了什么数据结构

1. SDS: 简单动态字符串：用于存储二进制安全的字符串数据，包含字符串的长度信息以及字符数组，支持动态扩展。
2. 双端链表：Redis中的`List`就是用双端链表实现的，支持在两端进行元素的快速插入和删除。
3. 压缩列表：紧凑的由连续内存块组成的顺序型数据结构，可以在元素较少时减少内存占用。
4. 哈希表：用于实现 Redis 的哈希表数据类型。它使用两种不同的底层实现：ziplist 和哈希表，当哈希表的元素较少时，使用压缩列表（ziplist）来存储，而在元素较多时，切换到更为传统的哈希表结构。
5. 整数集合：整数集合是一种专门用于存储整数值的数据结构，通过紧凑的二进制表示，提高了整数存储的效率。整数集合被用于存储Redis中的集合数据结构的整数元素。
6. 跳表：跳跃表是一种**在链表基础上改进过来的，实现了一种「多层」的有序链表**，当数据量很大时，跳表的查找复杂度就是O(logN)。用于实现有序集合（Sorted Set）。
7. `quicklist`：`quicklist` 就是双向链表 + 压缩列表组合，quicklist 就是一个链表，而链表中的每个元素又是一个压缩列表。
8. `listpack`：listpack 没有压缩列表中记录前一个节点长度的字段，listpack 只记录当前节点的长度，当我们向 listpack 加入一个新元素的时候，不会影响其他节点的长度字段的变化，从而避免了压缩列表的连锁更新问题。

Redis与MySQL的区别是什么

- Redis基于键值对，支持多种数据结构；而MySQL是一种关系型数据库，使用表来组织数据。
- Redis将数据存在内存中，通过持久化机制将数据写入磁盘，MySQL通常将数据存储在磁盘上。
- Redis不使用SQL，而是使用自己的命令集，MySQL使用SQL来进行数据查询和操作。
- Redis以高性能和低延迟为目标，适用于读多写少的应用场景，MySQL 适用于需要支持复杂查询、事务处理、拥有大规模数据集的场景。

Redis 更适合处理高速、高并发的数据访问，以及需要复杂数据结构和功能的场景，在实际应用中，很多系统会同时使用 MySQL 和 Redis。

Redis与Memcached有什么区别

- Redis支持更多的数据结构，而Memcached主要关注于简单的键值存储。
- Redis支持数据的持久性，而Memcached重启服务会导致数据丢失。
- Redis通常比Memcached更快，部分原因是由于Redis采用单线程模型，而Memcached使用多线程来处理并发请求
- 总结来说Redis适用于需要更丰富数据结构和功能、需要数据持久性支持、以及对高可用性有要求的场景，而**Memcached**：适用于简单的键值对缓存。

Redis为什么不使用红黑树，而使用跳表

1. 跳表相对于红黑树来说实现更加简单
2. 跳表是一种相对简单的数据结构，更容易理解和调试。
3. 跳表对于范围查询的支持较好，这符合有序集合数据类型的特性。有序集合中的元素是有序的，而跳表天然支持按范围查找。
4. 跳表在插入和删除操作上的性能表现较好，尤其是在有序集合的场景下。

Redis实现分布式锁

使用`SETNX`命令，只有插入的key不存在才插入，如果SETNX的key存在就插入失败，key插入成功代表加锁成功，否则加锁失败；解锁的过程就是将key删除，**保证执行操作的客户端就是加锁的客户端**，加锁时候要设置`unique_value`，解锁的时候，要先判断锁的 `unique_value` 是否为加锁客户端，是才将 `lock_key` 键删除。此外要给锁设置一个过期时间，以免客户端拿到锁后发生异常，导致锁一直无法释放，可以指定 `EX/PX` 参数设置过期时间。

```
SET lock_key unique_value NX PX 10000
```

Redis持久化机制有哪些【常问】

- **AOF 日志**：每执行一条写操作命令，就把该命令以追加的方式写入到一个文件里；
- **RDB 快照**：将某一时刻的内存数据，以二进制的方式写入磁盘；
- **混合持久化方式**：Redis 4.0 新增的方式，集成了 AOF 和 RBD 的优点；

AOF的三种写回策略

Always、Everysec 和 No，这三种策略在可靠性上是从高到低，而在性能上从低到高。

Always是每次写操作命令执行完后，同步将 AOF 日志数据写回硬盘；**Everysec**每次写操作命令执行完后，先将命令写入到 AOF 文件的内核缓冲区，然后每隔一秒将缓冲区里的内容写回到硬盘；**No**就是不控制写回硬盘的时机。每次写操作命令执行完后，先将命令写入到 AOF 文件的内核缓冲区，再由操作系统决定何时将缓冲区内容写回硬盘。

什么是Redis事务

在 Redis 中，事务是一组命令的有序执行序列，它们被一起执行，而在执行期间不会被其他客户端的命令打断。Redis 使用 MULTI 和 EXEC 命令来实现事务。

基本流程如下：

- **MULTI：** 开始一个事务，标记事务的开始。
- 在 MULTI 和 EXEC 之间，可以输入多个 Redis 命令。这些命令并不会立即执行，而是放入一个队列中，等待执行。
- **EXEC：** 执行所有在 MULTI 和 EXEC 之间输入的命令。这些命令作为一个事务一起执行，而且是按照顺序执行的。如果在 EXEC 之前发生错误，整个事务会被取消。
- 如果事务执行成功，客户端会收到一个包含所有命令执行结果的数组。
- 如果事务执行失败（比如在 MULTI 和 EXEC 之间发生了错误），那么事务中的所有命令都不会被执行，Redis 将回滚到 MULTI 命令执行之前的状态。

在事务中，还可以使用 WATCH 命令来监视一个或多个键。如果在事务执行期间监视的键被其他客户端修改了，事务就会被取消，从而确保事务执行的原子性。

缓存中的热点数据和冷数据是什么

- 热数据指的就是访问次数较多的数据
- 冷数据就是访问很少或者从不访问的数据

只有热数据，缓存才有价值。

Redis集群模式有哪些

- 主从复制：将一个Redis实例的数据复制到其他实例，其中一个是主节点，其余是从节点。主节点将写操作传播到所有从节点。
- 哨兵：监控Redis实例的状态，发现主节点的故障并自动进行故障转移。
- 切片集群：将数据分布在不同的服务器上，以此来降低系统对单主节点的依赖

Redis的哨兵是用来做什么的

Redis哨兵用于监控Redis实例的状态，发现主节点的故障并自动进行故障转移。

- 监控：监控 Redis 主服务器和从服务器的状态，包括连接状态、是否能够执行命令、是否有持久性问题等
- 故障转移：当 Sentinel （哨兵）发现主服务器不可用时（比如宕机），它会通过一定的选举机制选择一个从服务器升级为新的主服务器，然后通知其他从服务器将其切换到新的主服务器。

主从模式的同步过程

- 连接协商：从服务器先发送命令给主服务器表示要进行数据同步，命令内容包括**主服务器的runID**和**复制进度**两个参数，主服务器收到命令之后会给从服务响应命令，响应包括**主服务器的runID**和**复制进度**。从服务器收到响应之后会记录这两个值。
- 主从数据同步：主服务器执行 `BGSAVE` 命令，生成 RDB 快照文件。这个文件包含了主服务器当前时刻的数据状态，将生成的 RDB 快照文件发送给从服务器。从服务器接收到 RDB 快照文件后，通过 `LOAD` 命令将其加载到内存中，从服务器的数据现在与主服务器的数据一致。
- **主服务器开启增量同步**：主服务器将开启 TCP 连接，等待从服务器连接，并开始记录所有的写命令，将它们发送给从服务器，从服务器连接到主服务器，执行 `PSYNC` 命令，开始接收主服务器发送的增量同步命令，从服务器按顺序执行主服务器发送的所有写命令，确保与主服务器同步。

哨兵的工作原理 【常问】

判断节点是否存活

每个哨兵定期向 Redis 服务器发送 PING 命令，以检测服务器是否处于活跃状态。如果哨兵连续一定次数未收到服务器的响应，就认为服务器主观下线。然后哨兵会从从节点中选择一个作为主节点。

选出新主节点

在发现主服务器下线后，哨兵会协调选举一个新的主服务器。这个过程中，哨兵会考虑每个可用的从服务器，选择一个作为新的主服务器，并将其他从服务器配置为复制新的主服务器。

具体过程：

- **选择候选从服务器：** 哨兵会从可用的从服务器中选择一组候选服务器，通常选择复制偏移量（replication offset）最大的从服务器。
- **计算投票：** 每个哨兵为每个候选从服务器投票。投票的考量因素包括从服务器的复制偏移量、连接质量、优

先级等。
- **达成共识：** 哨兵们根据投票结果达成共识，选择一个从服务器作为新的主服务器。这通常需要获得多数哨兵的同意。

更新配置信息

一旦新的主服务器被选出，哨兵会更新 Redis 集群的配置信息，包括将新的主服务器的地址和端口通知给其他哨兵和客户端。

通知客户端

哨兵会向客户端发送通知，告知客户端新的主服务器的位置，以便客户端能够重新连接。

缓存雪崩、击穿、穿透和解决办法【常问】

1. 缓存雪崩是指在某个时间点，大量缓存同时失效，导致请求直接访问数据库或其他后端系统，增加了系统负载。

   对于缓存雪崩，可以通过合理设置缓存的过期时间，分散缓存失效时间点，或者采用永不过期的策略，再结合定期更新缓存。

2. 缓存击穿是指一个缓存中不存在但是数据库中存在的数据，当有大量并发请求查询这个缓存不存在的数据时，导致请求直接访问数据库，增加数据库的负载。典型的场景是当一个缓存中的数据过期或被清理，而此时有大量请求访问这个缓存中不存在的数据，导致大量请求直接访问底层存储系统。

   对于缓存击穿，可以采用互斥锁（例如分布式锁）或者在查询数据库前先检查缓存是否存在，如果不存在再允许查询数据库，并将查询结果写入缓存。

3. 缓存穿透是指查询一个在缓存和数据库都不存在的数据，这个数据始终无法被缓存，导致每次请求都直接访问数据库，增加数据库的负载。典型的情况是攻击者可能通过构造不存在的 key 大量访问缓存，导致对数据库的频繁查询。

   对于缓存穿透，可以采用布隆过滤器等手段来过滤掉恶意请求，或者在查询数据库前先进行参数的合法性校验。

如何保证数据库和缓存的一致性【常问】

Cache Aside

- **原理**：先从缓存中读取数据，如果没有就再去数据库里面读数据，然后把数据放回缓存中，如果缓存中可以找到数据就直接返回数据；更新数据的时候先把数据持久化到数据库，然后再让缓存失效。
- **问题**：假如有两个操作一个更新一个查询，第一个操作先更新数据库，还没来及删除数据库，查询操作可能拿到的就是旧的数据；更新操作马上让缓存失效了，所以后续的查询可以保证数据的一致性；还有的问题就是有一个是读操作没有命中缓存，然后就到数据库中取数据，此时来了一个写操作，写完数据库后，让缓存失效，然后，之前的那个读操作再把老的数据放进去，也会造成脏数据。
- **可行性**：出现上述问题的概率其实非常低，需要同时达成读缓存时缓存失效并且有并发写的操作。数据库读写要比缓存慢得多，所以读操作在写操作之前进入数据库，并且在写操作之后更新，概率比较低。

Read/Write Through

- **原理**：Read/Write Through原理是把更新数据库（Repository）的操作由缓存代理，应用认为后端是一个单一的存储，而存储自己维护自己的缓存。
- **Read Through**：就是在查询操作中更新缓存，也就是说，当缓存失效的时候，Cache Aside策略是由调用方负责把数据加载入缓存，而Read Through则用缓存服务自己来加载，从而对调用方是透明的。

- **Write Through**：当有数据更新的时候，如果没有命中缓存，直接更新数据库，然后返回。如果命中了缓存，则更新缓存，然后再由缓存自己更新数据库（这是一个同步操作）。

Write Behind

- **原理**：在更新数据的时候，只更新缓存，不更新数据库，而缓存会异步地批量更新数据库。这个设计的好处就是让数据的I/O操作非常快，带来的问题是，数据不是强一致性的，而且可能会丢。
- **第二步失效问题**：这种可能性极小，缓存删除只是标记一下无效的软删除，可以看作不耗时间。如果会出问题，一般程序在写数据库那里就没有完成：故意在写完数据库后，休眠很长时间再来删除缓存。

Redis过期删除策略有哪些

- 定时删除：设置 key 的过期时间，当时间到达时，自动执行 key 的删除操作。
- 惰性删除： 每次从数据库访问 key 时，都检测 key 是否过期，如果过期则删除该 key。
- 定期删除：**每隔一段时间「随机」从数据库中取出一定数量的 key 进行检查，并删除其中的过期key。**

Redis 选择「惰性删除+定期删除」这两种策略配和使用

在应用中是怎么使用Redis的

- 可以用作缓存层，存储频繁访问的数据，提高访问速度。
- 用作消息队列，通过发布订阅模式传递消息。
- 存储会话数据、计数器等。

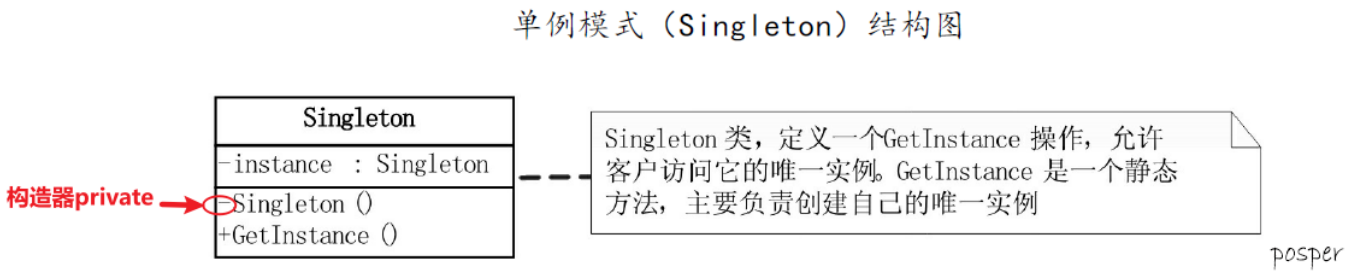
设计模式

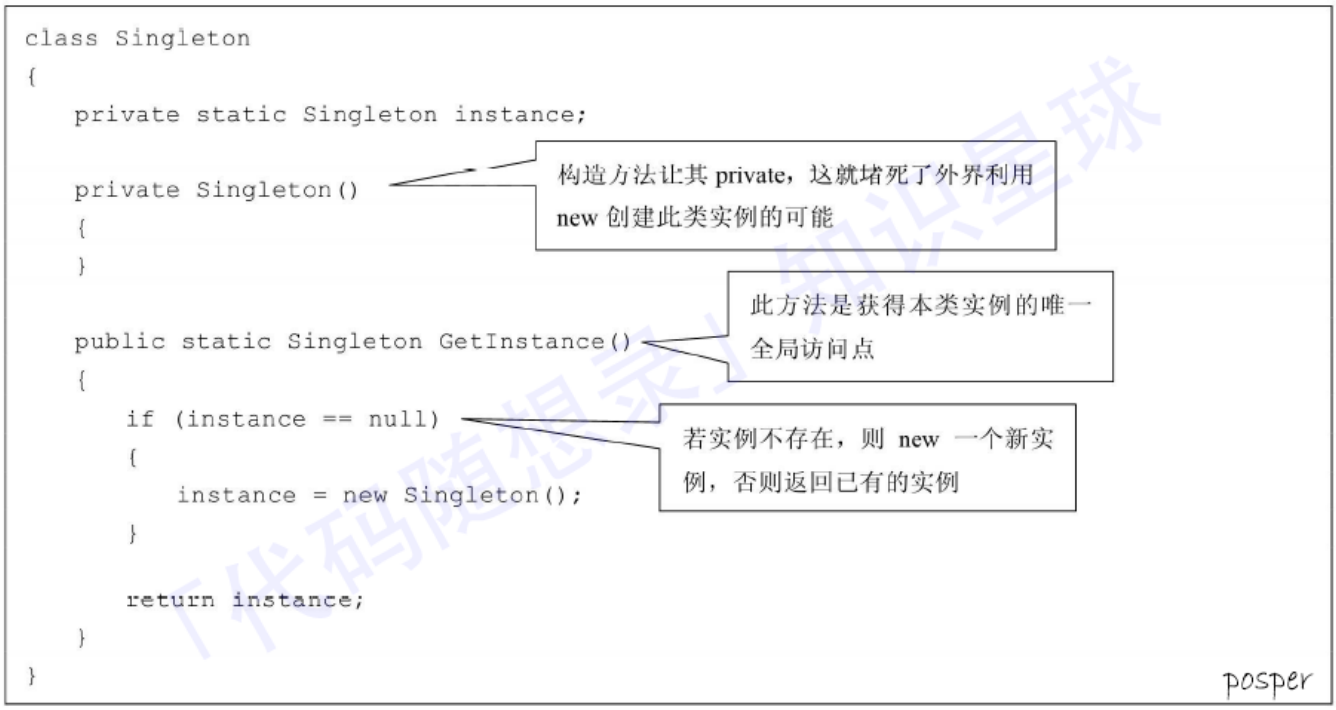
> 想要了解更多设计模式，可以查看卡码网设计模式精讲

常用设计模式

单例模式

单例设计模式是一种**确保一个类只有一个实例，并提供一个全局访问点来访问**该实例的创建模式。





优点：

有些实例，全局只需要一个就够了，使用单例模式就可以避免一个全局使用的类，频繁的创建与销毁，耗费系统资源。

关键概念

1. 一个私有构造函数（确保只能单例类自己创建实例）：

   单例类通常会将其构造函数设为私有，以防止外部代码直接实例化对象。

2. 一个私有静态变量（确保只有一个实例）

   单例类通常包含一个私有的静态变量，用于保存该类的唯一实例。

3. 一个公有静态函数（给使用者提供调用方法）

单例类提供一个公共的静态方法，用于获取该类的实例。如果实例不存在，则在该方法内部创建实例并返回。

简单来说就是，单例类的构造方法不让其他人修改和使用；并且单例类自己只创建一个实例，这个实例，其他人也无法修改和直接使用；然后单例类提供一个调用方法，想用这个实例，只能调用。这样就确保了全局只创建了一个实例。

单例模式的6种实现

单例模式6种实现及各实现的优缺点

1、懒汉式（线程不安全）

```
public class Singleton {
    // 类加载时就创建实例
    private static final Singleton instance = new Singleton();

    // 私有构造函数
    private Singleton() {}

    // 公共静态方法，返回实例
    public static Singleton getInstance() {
        return instance;
    }
}
```

说明：

先不创建实例，当第一次被调用时，再创建实例，所以被称为懒汉式。

优点：

延迟了实例化，如果不需要使用该类，就不会被实例化，只有在需要时才创建实例，避免了资源浪费。

缺点：

线程不安全，多线程环境下，如果多个线程同时进入了 if (uniqueInstance == null) ，若此时还未实例化，也就是 uniqueInstance == null，那么就会有多个线程执行 uniqueInstance = new Singleton(); ，就会实例化多个实例；

2、饿汉式（线程安全）

实现：

```
public class Singleton {
    // 类加载时就创建实例
    private static final Singleton instance = new Singleton();

    // 私有构造函数
    private Singleton() {}

    // 公共静态方法，返回实例
    public static Singleton getInstance() {
        return instance;
    }
}
```

说明：

先不管需不需要使用这个实例，直接先实例化好实例（饿死鬼一样，所以称为饿汉式），然后当需要使用的时候，直接调方法就可以使用了

优点：

提起实例化好了一个实例，避免了线程不安全问题的出现，

缺点：

直接实例化了实例，不再延迟实例化；若系统没有使用这个实例，或者系统运行很久之后才需要使用这个实例，都会使操作系统的资源浪费。

3、懒汉式（线程安全）

实现：

```
public class Singleton {
    private static Singleton uniqueInstance;
    private static singleton() {}
    private static synchronized Sing leton getUinqueInstance() {
        if (uniqueInstance == null) {
            uniqueInstance = new Singleton();
        }
        return uniqueInstance;
    }
}
```

说明：

实现和线程不安全的懒汉式 几乎一样，唯一不同的点是，在get方法上 加了一把锁。如此一来，多个线程访问，每次只有拿到锁的的线程能够进入该方法，避免了多线程不安全问题的出现。

优点：

延迟实例化，节约了资源，并且是线程安全的。

缺点：

虽然解决了线程安全问题，但是性能降低了。因为，即使实例已经实例化了，既后续不会再出现线程安全问题了，但是锁还在，每次还是只能拿到锁的线程进入该方法使线程阻塞，等待时间过长。

4、双重检查锁实现（线程安全）

实现：

```
public class Singleton {
    private volatile static Singleton uniqueInstance;

    private Singleton() {}
    //  使用双重检查锁定保证线程安全
    public static Singleton getUniqueInstance() {
        if (uniqueInstance == null) {
            synchronized (Singleton.class) {
                if (uniqueInstance == null) {
                    uniqueInstance = new Singleton();
                }
            }
        }
        return uniqueInstance;
```

```
    }
}
```

说明：

双重检查锁相当于是改进了线程安全的懒汉式。线程安全的懒汉式的缺点是性能降低了，造成的原因是因为即使实例已经实例化，依然每次都会有锁。

而现在，我们将锁的位置变了，并且多加了一个检查。也就是，先判断实例是否已经存在，若已经存在了，则不会执行判断方法内的有锁方法了。 而如果，还没有实例化的时候，多个线程进去了，也没有事，因为里面的方法有锁，只会让一个线程进入最内层方法并实例化实例。如此一来，最多最多，也就是第一次实例化的时候，会有线程阻塞的情况，后续便不会再有线程阻塞的问题。

为什么使用 volatile 关键字修饰了 uniqueInstance 实例变量?

`uniqueInstance = new Singleton();`

这段代码执行时分为三步：

1. 为 `uniqueInstance` 分配内存空间
2. 初始化 `uniqueInstance`
3. 将 `uniqueInstance` 指向分配的内存地址

正常的执行顺序当然是 1>2>3 ，但是由于 JVM 具有指令重排的特性，执行顺序有可能变成 1>3>2。

单线程环境时，指令重排并没有什么问题；多线程环境时，会导致有些线程可能会获取到还没初始化的实例。

例如：线程A 只执行了 1 和 3 ，此时线程B来调用 `getUniqueInstance()` ，发现 `uniqueInstance` 不为空，便获取 `uniqueInstance` 实例，但是其实此时的 `uniqueInstance` 还没有初始化。

解决办法就是加一个 `volatile` 关键字修饰 `uniqueInstance` ， `volatile` 会禁止 JVM 的指令重排，就可以保证多线程环境下的安全运行。

优点：

延迟实例化，节约了资源；线程安全；并且相对于线程安全的懒汉式，性能提高了。

缺点：

volatile 关键字，对性能也有一些影响。

5、静态内部类实现（线程安全）

实现：

```
public class Singleton {
    private Singleton() {}

    // 静态内部类持有实例
    private static class SingletonHolder {
        private static final Singleton instance = new Singleton();
    }

    // 公共静态方法，返回实例
    public static Singleton getInstance() {
        return SingletonHolder.instance;
    }
}
```

说明：

首先，当外部类 Singleton 被加载时，静态内部类 SingletonHolder 并没有被加载进内存。当调用 getUniqueInstance() 方法时，会运行 return SingletonHolder.INSTANCE;

触发了 SingletonHolder.INSTANCE ，此时静态内部类 SingletonHolder 才会被加载进内存，并且初始化 INSTANCE 实例，而且 JVM 会确保 INSTANCE 只被实例化一次。

优点：

延迟实例化，节约了资源，且线程安全，性能也提高了。

6、枚举类实现（线程安全）

实现：

```
public enum Singleton {
    INSTANCE;

    // 可以添加其他方法和属性
    public void doSomething() {
        // 实现
    }
}
```

说明：

默认枚举实例的创建就是线程安全的，且在任何情况下都是单例。

优点：

写法简单，线程安全，天然防止反射和反序列化调用。

防止反序列化

序列化：

把java对象转换为字节序列的过程；

反序列化：

通过这些字节序列在内存中新建java对象的过程；

说明：

反序列化将一个单例实例对象写到磁盘再读回来，从而获得了一个新的实例。

我们要防止反序列化，避免得到多个实例，枚举类天然防止反序列化。

其他单例模式 可以通过 重写 readResolve() 方法，从而防止反序列化，使实例唯一重写

```
readResolve();
private Object readResolve();
throws
ObjectStreamException {
    return singleton;
}
```

单例模式的应用场景

单例设计模式适用于以下一些场景

- 资源共享：当多个模块或系统需要共享某一资源时，可以使用单例模式确保该资源只被创建一次，避免重复创建和浪费资源。
- 控制资源访问：单例模式可以用于控制对特定资源的访问，例如数据库连接池、线程池等。
- 配置管理器：当整个应用程序需要共享一些配置信息时，可以使用单例模式将配置信息存储在单例类中，方便全局访问和管理。
- 日志记录器：单例模式可以用于创建一个全局的日志记录器，用于记录系统中的日志信息。
- 线程池：在多线程环境下，使用单例模式管理线程池，确保线程池只被创建一次，提高线程池的利用率。
- 缓存：单例模式可以用于实现缓存系统，确保缓存只有一个实例，避免数据不一致性和内存浪费。

应用场景举例：

1. 网站计数器
2. 应用程序的日志应用
3. Web项目中的配置对象的读取
4. 数据库连接池
5. 多线程池

……

使用场景总结：

（1）频繁实例化然后又销毁的对象，使用单例模式可以提高性能

（2）经常使用的对象，但实例化时耗费时间或者资源多，如数据库连接池，使用单例模式，可以提高性能，降低资源损坏

（3）使用线程池之类的控制资源时，使用单例模式，可以方便资源之间的通信

工厂模式

概述：

工厂模式是一种非常常用的创建型设计模式，其提供了创建对象的最佳方式。在创建对象时，不会对客户端暴露对象的创建逻辑，而是通过使用共同的接口来创建对象。其用来封装和管理类的创建，本质是对获取对象过程的抽象。

一般情况下，**工厂模式分为三种更加细分的类型：简单工厂、工厂方法和抽象工厂**。

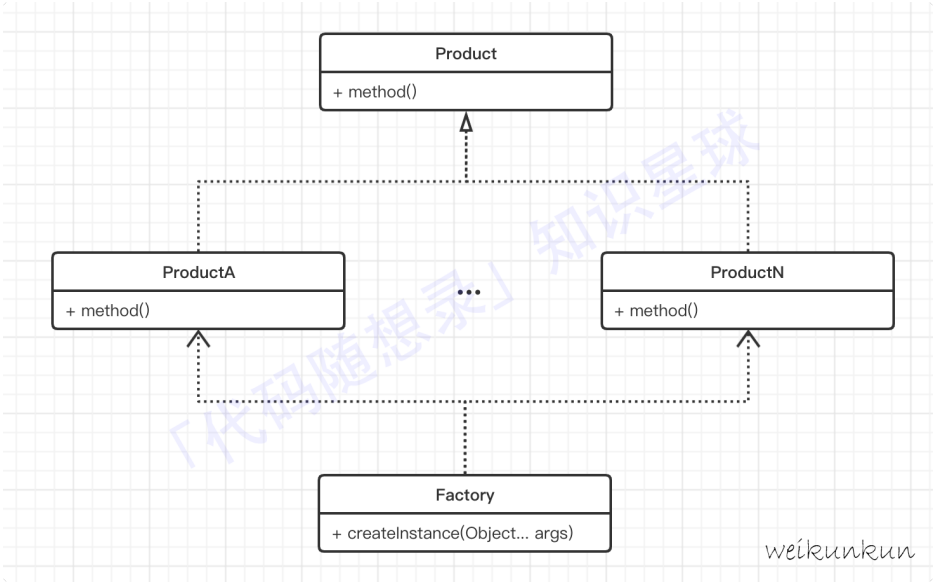
不过，在 GoF 的《设计模式》一书中，它将简单工厂模式看作是工厂方法模式的一种特例，所以工厂模式只被分成了工厂方法和抽象工厂两类

优点：

- 解耦：将对象的创建和使用进行分离，客户端代码与具体产品类的实例化过程解耦，客户端只需要知道工厂的接口和抽象产品的接口，而不需要关心具体的实现类。
- 可复用：对于创建过程比较复杂且在很多地方都使用到的对象，通过工厂模式可以提高对象创建的代码的复用性。
- 易于扩展：添加新的产品类时，只需要扩展相应的具体产品和具体工厂，而不需要修改已有的代码，符合开闭原则。
- 更符合面向对象的设计原则：通过工厂模式，将对象的创建封装在工厂类中，使得系统更符合单一职责原则。

简单工厂模式

UML



概念

简单工厂模式也被称为静态工厂方法模式。

简单工厂模式并不是一个标准的设计模式，更像是一种编程习惯。在简单工厂模式中，一个工厂类负责创建多个产品类的实例，通过传入不同的参数来决定创建哪种产品。

同时在简单工厂模式中会定义一个类负责创建其他类的实例，被创建的实例也通常具有共同的父类。

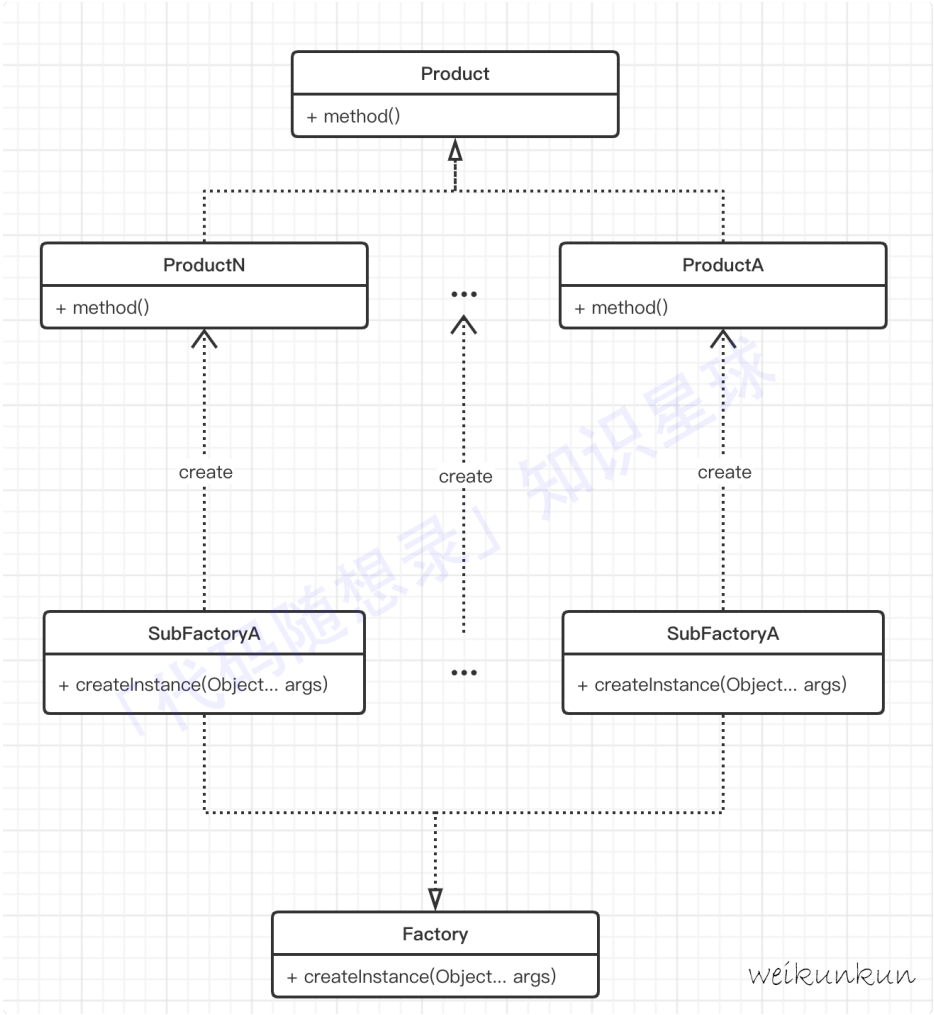
虽然实现了对象的创建和使用的分离，但是不够灵活，工厂类集合了所有产品的创建逻辑，职责过重，同时新增一个产品就需要在原工厂类内部添加一个分支，违反了开闭原则。**并且若是有多个判断条件共同决定创建对象，则后期修改会越来越复杂。**

实际应用

JDK中的DateFormate、Calendar类都有使用，通过不同参数返回我们需要的对象。

工厂方法模式

UML



概念

工厂方法模式中，将简单工厂中的工厂类变为一个抽象接口。负责给出不同工厂应该实现的方法，自身不再负责创建各种产品，而是将具体的创建操作交给实现该接口的子工厂类来做。

通过多态的形式解决了简单工厂模式过多的分支问题。虽然在新增产品时不仅要新增一个产品类还要实现与之对应的子工厂，但是相较于简单工厂模式更符合开闭原则

实际应用

JDK中的Collection接口中Iterator的实现。Collection中不同的实现类生产适合于自己的迭代器对象

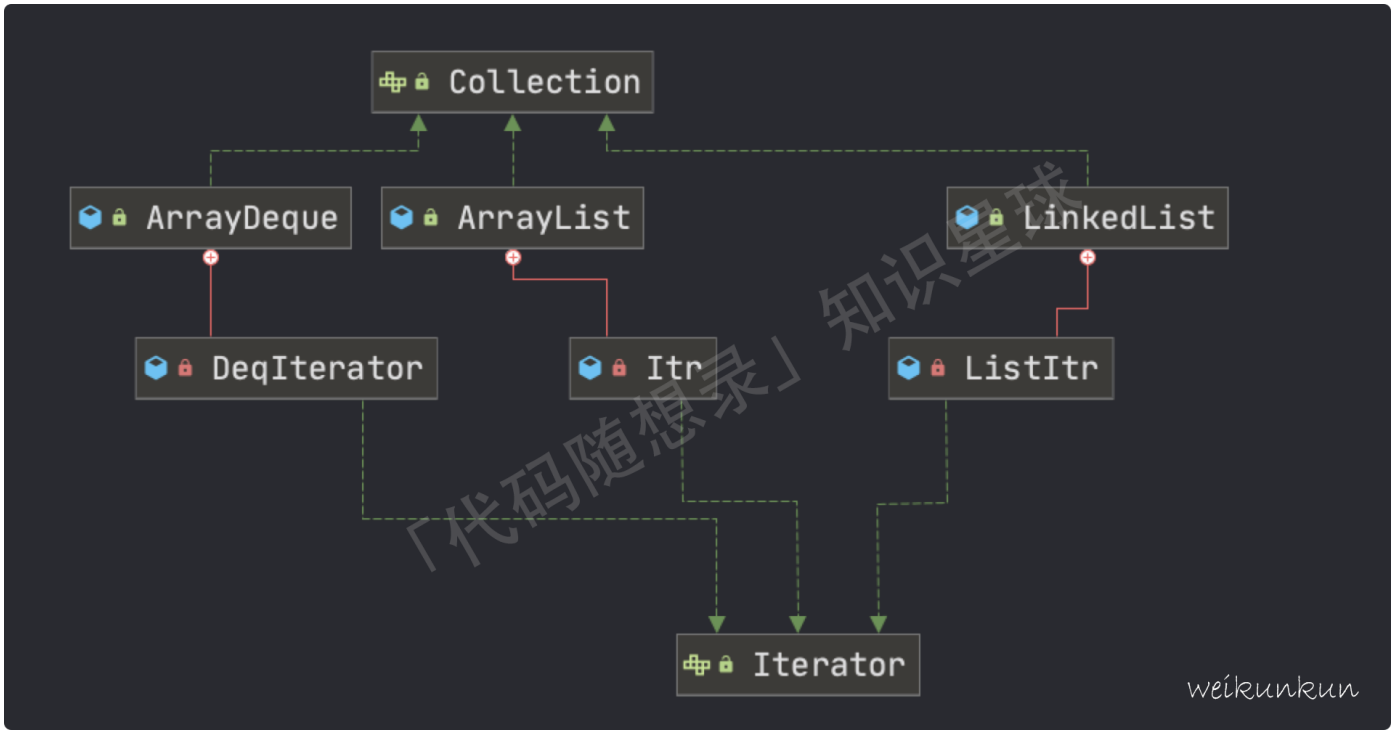
1、Factory：Collection

1. SubFactoryA：LinkedList

2. SubFactoryB：ArrayList

2、Product：Iterator

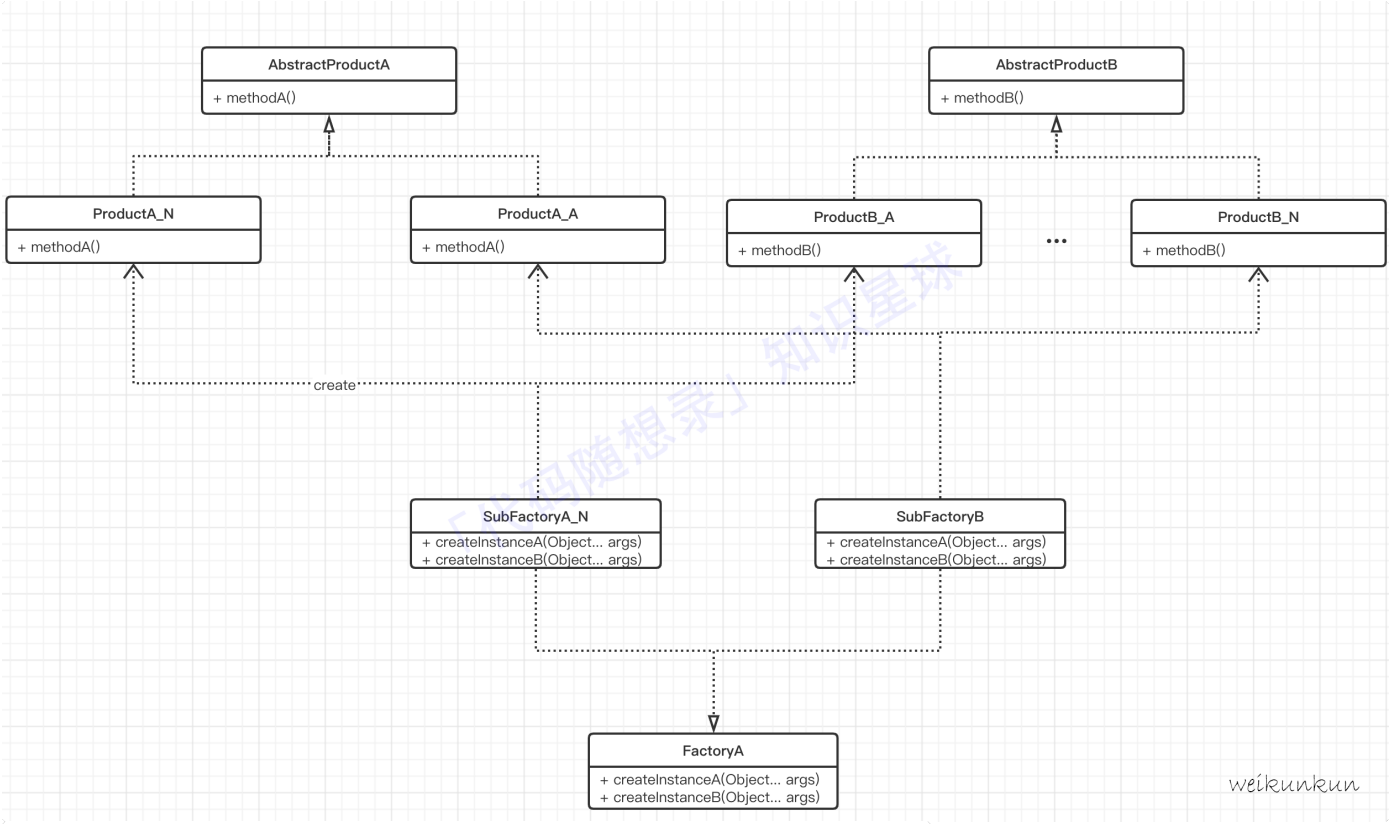
1. ProductA：ListItr
2. ProductB：Itr

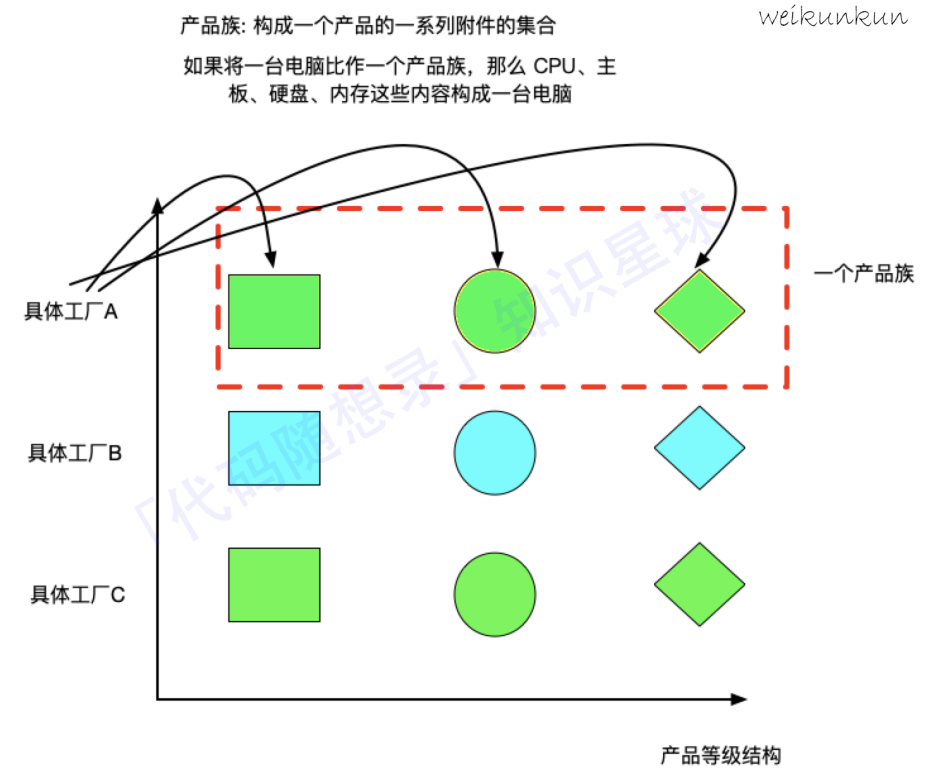


抽象工厂模式

抽象工厂模式提供一个创建一系列相关或相互依赖对象的接口，而无需指定它们的具体类。抽象工厂模式通常涉及多个抽象产品、多个具体产品和多个具体工厂。

UML





概念

工厂方法模式通过引入工厂等级结构，解决了简单工厂模式中工厂类职责太重的问题，但由于工厂方法模式中的每个工厂只生产一类产品，可能会导致系统中存在大量的工厂类，势必会增加系统的开销。抽象工厂模式为工厂方法模式的进一步延伸，其将一些相关的产品组成一个“产品族”，由同一个工厂来统一生产。虽然对于新增一个产品族很方面，并且也符合开闭原则，但是新增一个产品等级结构，会对整个工厂结构进行大改

应用

Spring中的BeanFactory

工厂设计模式的应用场景

- 当一个类不知道它所需要的类的时候。
- 当一个类希望通过其子类来指定创建对象的时候。
- 当类将创建对象的职责委托给多个帮助子类中的某一个，并且希望将哪一个帮助子类是代理者的信息局部化时。

举例来说：

- 在数据库操作中，通过工厂模式可以根据不同的数据库类型（MySQL、Oracle等）创建对应的数据库连接对象
- 通过工厂模式可以根据配置文件或其他条件选择不同类型的日志记录器，如文件日志记录器、数据库日志记录器等。
- 在图形用户界面（GUI）库中，可以使用工厂模式创建不同风格或主题的界面元素，如按钮、文本框等。
- 在加密算法库中，可以使用工厂模式根据需要选择不同的加密算法，例如对称加密、非对称加密等。
- 在文件解析过程中，可以使用工厂模式根据文件类型选择不同的解析器，如XML解析器、JSON解析器等。
- 在网络通信库中，可以使用工厂模式创建不同类型的网络连接对象，如TCP连接、UDP连接等。

观察者模式

属于行为型模式的一种，它定义了一种一对多的依赖关系，让多个观察者对象同时监听某一个主题对象。这个主题对象在状态变化时，会通知所有的观察者对象，使他们能够自动更新自己。

主要构成

Subject（主题）：

主题是被观察的对象，它包含了一组观察者对象，并提供了添加、删除和通知观察者的方法。主题通常有一个状态，当状态改变时，通知所有观察者。

Observer（观察者）：

观察者是依赖于主题的对象，当主题的状态发生改变时，观察者得到通知并进行相应的更新。观察者的具体实现类需要实现更新的方法。

ConcreteSubject（具体主题）：

具体主题是主题的具体实现类，它维护了一个状态，并在状态改变时通知观察者。

ConcreteObserver（具体观察者）：

具体观察者是观察者的具体实现类，实现了更新的方法，以便在接收到通知时进行相应的处理。

案例：

Carl哥的微信公众号 是被观察者，微信用户 是观察者，有多个微信用户关注了Carl哥的公众号，当Carl哥的公众号更新订阅时就会通知这些订阅了的微信用户。

优点：

解除耦合，让耦合的双方都依赖于抽象，从而使得各自的变换都不会影响另一边的变换

缺点：

调试复杂，而且在Java中消息的通知一般是顺序执行，那么一个观察者卡顿，会影响整体的执行效率，在这种情况下，一般会采用异步实现。

实现步骤

1. 定义主题接口（Subject），包括注册、移除和通知观察者的方法。
2. 定义观察者接口（Observer），包括更新的方法。
3. 创建具体主题类（Concrete Subject），维护一组观察者对象，并在状态改变时通知观察者。
4. 创建具体观察者类（Concrete Observer），实现更新的方法。
5. 客户端代码中创建主题对象和观察者对象，注册观察者到主题中，然后通过主题改变状态，观察者得到通知并进行更新。

使用场景

1. 事件处理：当一个对象的状态发生改变时，观察者模式可以用于通知和处理与该对象相关的事件。这在图形用户界面（GUI）开发中是很常见的，例如按钮点击事件、鼠标移动事件等。
2. 发布订阅：观察者模式可以用于实现发布-订阅模型，其中一个主题（发布者）负责发送通知，而多个观察者（订阅者）监听并响应这些通知。这种模型在消息队列系统、事件总线等场景中经常使用。
3. MVC架构：观察者模式常被用于实现MVC架构中的模型和视图之间的通信。当模型的状态发生改变时，所有相关的视图都会得到通知并更新显示。
4. 异步编程：观察者模式可以用于处理异步任务的完成事件。任务完成时，通知所有相关的观察者进行后续处理。

代理模式

结构图：

7.2 没有代理的代码

7.3 只有代理的代码

7.4 符合实际的代码

7.5 代理模式

7.6 代理模式应用

7.7 秀才让小六代其求婚

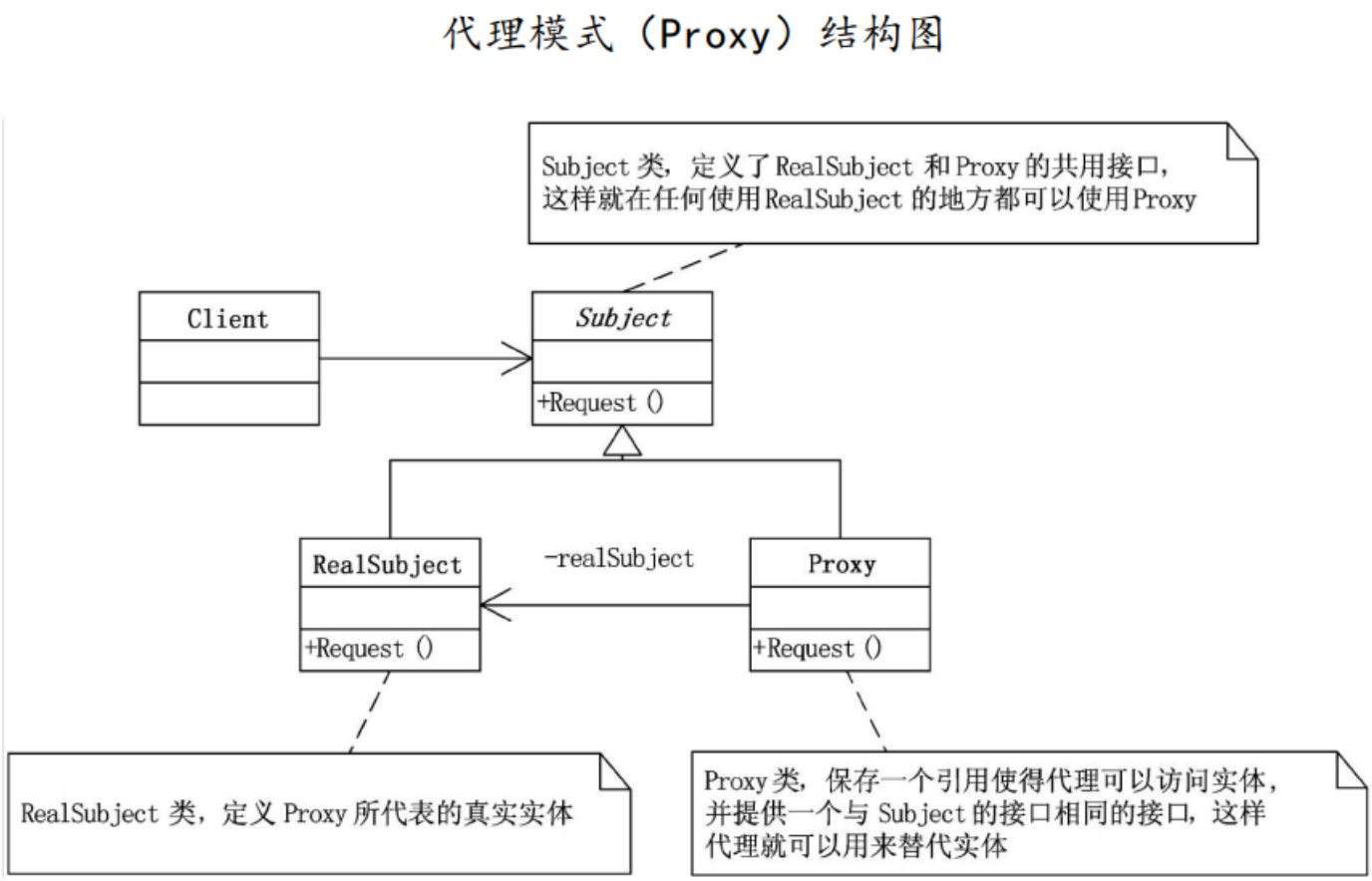
定义

代理模式（Proxy Pattern）是一种结构型设计模式，其主要目的是在访问某个对象时引入一种代理对象，通过代理对象控制对原始对象的访问。代理模式可以用于实现懒加载、控制访问、监控对象等场景。

主要角色

1. 抽象主题（`Subject`): 定义了代理对象和真实对象的共同接口，使得代理对象能够替代真实对象。
2. 真实主题`Real Subject`: 是实际执行业务逻辑的对象，是代理模式中的被代理对象。
3. 代理`Proxy`: 包含一个指向真实主题的引用，提供与真实主题相同的接口，可以控制对真实主题的访问，并在需要时负责创建或删除真实主题的实例。

代理模式结构图



（1）Subject类

```
abstract class Subject
{
    public abstract void Request（）；
}
```

（2）RealSubject 类，

定义Proxy 所代表的真实实体

```
static void Main（string[] args）
{
    Proxy proxy = new Proxy（）；
    proxy.Request（）；

    Console.Read（）；
}
```

（3）Proxy 类

保存一个引用使得代理可以访问实体，并提供一个与Subject的接口相同的接口，这样代理就可以用来替代实体。

```
class RealSubject : Subject
{
    public override void Request（）
    {
        Console.WriteLine（"真实的请求"）；
    }
}
```

（4）客户端代码

```
class Proxy : Subject
{
    RealSubject realSubject;
    public override void Request（）
    {
        if（realSubject == null）
        {
            realSubject = new RealSubject（）；
        }
        realSubject.Request（）；
    }
```

代理模式应用

（1）远程代理

一个对象在不同的地址空间提供局部代表。这样可以隐藏一个对象存在于不同地址空间的事实。

（2）虚拟代理

根据需要创建开销很大的对象。通过它来存放实例化需要很长时间的真实对象，这样就可以达到性能的最优化。比如说你打开一个很大的HTML网页时，里面可能有很多的文字和图片，但你还是可以很快打开它，此时你所看到的是所有的文字，但图片却是一张一张地下载后才能看到。那些未打开的图片框，就是通过虚拟代理来替代了真实的图片，此时代理存储了真实图片的路径和尺寸。

（3）安全代理

用来控制真实对象访问时的权限。一般用于对象应该有不同的访问权限的时候。

（4）智能指引

是指当调用真实的对象时，代理处理另外一些事。如计算真实对象的引用次数，这样当该对象没有引用时，可以自动释放它；或当第一次引用一个持久对象时，将它装入内存；或在访问一个实际对象前，检查是否已经锁定它，以确保其他对象不能改变它。它们都是通过代理在访问一个对象时附加一些内务处理。

策略模式

策略模式（Strategy Pattern）是一种行为设计模式，它定义了一系列算法，，把它们单独封装起来，并且使它们可以互相替换，使得算法可以独立于使用它的客户端而变化，也是说这些算法所完成的功能类型是一样的，对外接口也是一样的，只是不同的策略为引起环境角色表现出不同的行为。

相比于使用大量的if...else，使用策略模式可以降低复杂度，使得代码更容易维护。

缺点：可能需要定义大量的策略类，并且这些策略类都要提供给客户端。

主要构成

- 策略接口（Stragety）： 策略接口定义了算法的抽象，具体的策略类实现了这个接口。
- 具体策略类（ConcreteStragety）： 具体策略类实现了策略接口，封装了具体的算法。
- 环境类（Context): 用来操作策略的上下文环境类,环境类的构造函数包含了Strategy类，通过多态传进来不同的具体策略（ConcreteStrategyA。ConcreteStrategyB）来调用不同策略的方法

实现步骤：

1. 定义一个策略接口，声明算法的抽象方法。
2. 创建具体的策略类，实现策略接口，封装具体的算法。
3. 创建环境类，包含对策略接口的引用，以及一个用于设置具体策略对象的方法。
4. 在客户端中创建环境类的对象，并调用其方法来执行具体的算法。

优缺点：

优点：

使用策略模式可以避免使用多重条件转移语句。多重转移语句将算法或行为的逻辑混合在一起，不易维护

缺点：

客户端必须知道所有的策略类，并自行决定使用哪一个策略类，策略模式只适用于客户端知道所有的算法或行为的情况。

示例

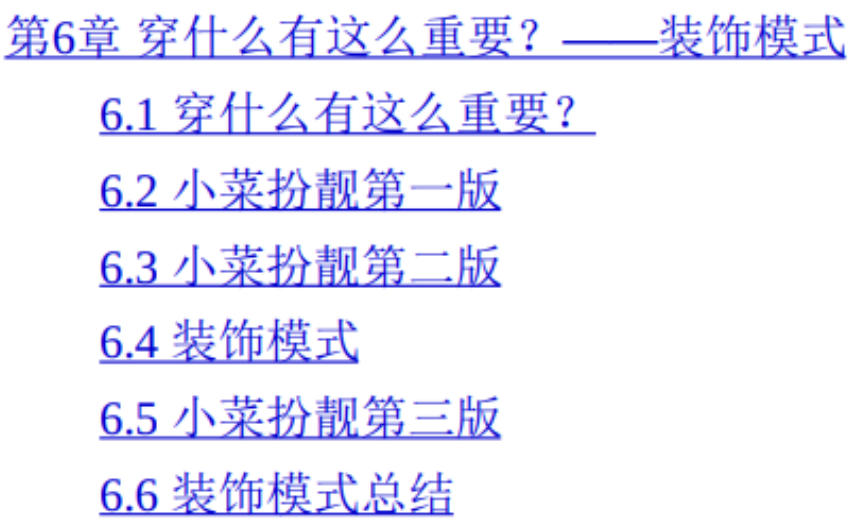
下面是一个简单的策略模式的示例，假设有一个商场销售系统，根据不同的促销策略计算最终价格：

```
// 策略接口
public interface PricingStrategy {
    double calculatePrice(double price);
}

// 具体策略类：无折扣
public class NoDiscountStrategy implements PricingStrategy {
    @Override
    public double calculatePrice(double price) {
        return price;
    }
}

// 具体策略类：打九折
public class DiscountStrategy implements PricingStrategy {
    @Override
    public double calculatePrice(double price) {
        return price * 0.9;
    }
}

// 具体策略类：满减
public class CashbackStrategy implements PricingStrategy {
    @Override
    public double calculatePrice(double price) {
        if (price >= 200) {
            return price - 50;
        } else {
            return price;
        }
    }
}

// 环境类
public class ShoppingCart {
    private PricingStrategy pricingStrategy;

    public void setPricingStrategy(PricingStrategy pricingStrategy) {
        this.pricingStrategy = pricingStrategy;
    }

    public double checkout(double totalPrice) {
        return pricingStrategy.calculatePrice(totalPrice);
    }
}
```

```
// 客户端代码
public class Client {
    public static void main(String[] args) {
        ShoppingCart cart = new ShoppingCart();

        // 选择无折扣策略
        cart.setPricingStrategy(new NoDiscountStrategy());
        double price1 = cart.checkout(100);
        System.out.println("Total Price: " + price1); // 输出: Total Price: 100.0

        // 选择打九折策略
        cart.setPricingStrategy(new DiscountStrategy());
        double price2 = cart.checkout(100);
        System.out.println("Total Price: " + price2); // 输出: Total Price: 90.0

        // 选择满减策略
        cart.setPricingStrategy(new CashbackStrategy());
        double price3 = cart.checkout(200);
        System.out.println("Total Price: " + price3); // 输出: Total Price: 150.0
    }
}
```

装饰模式

结构图



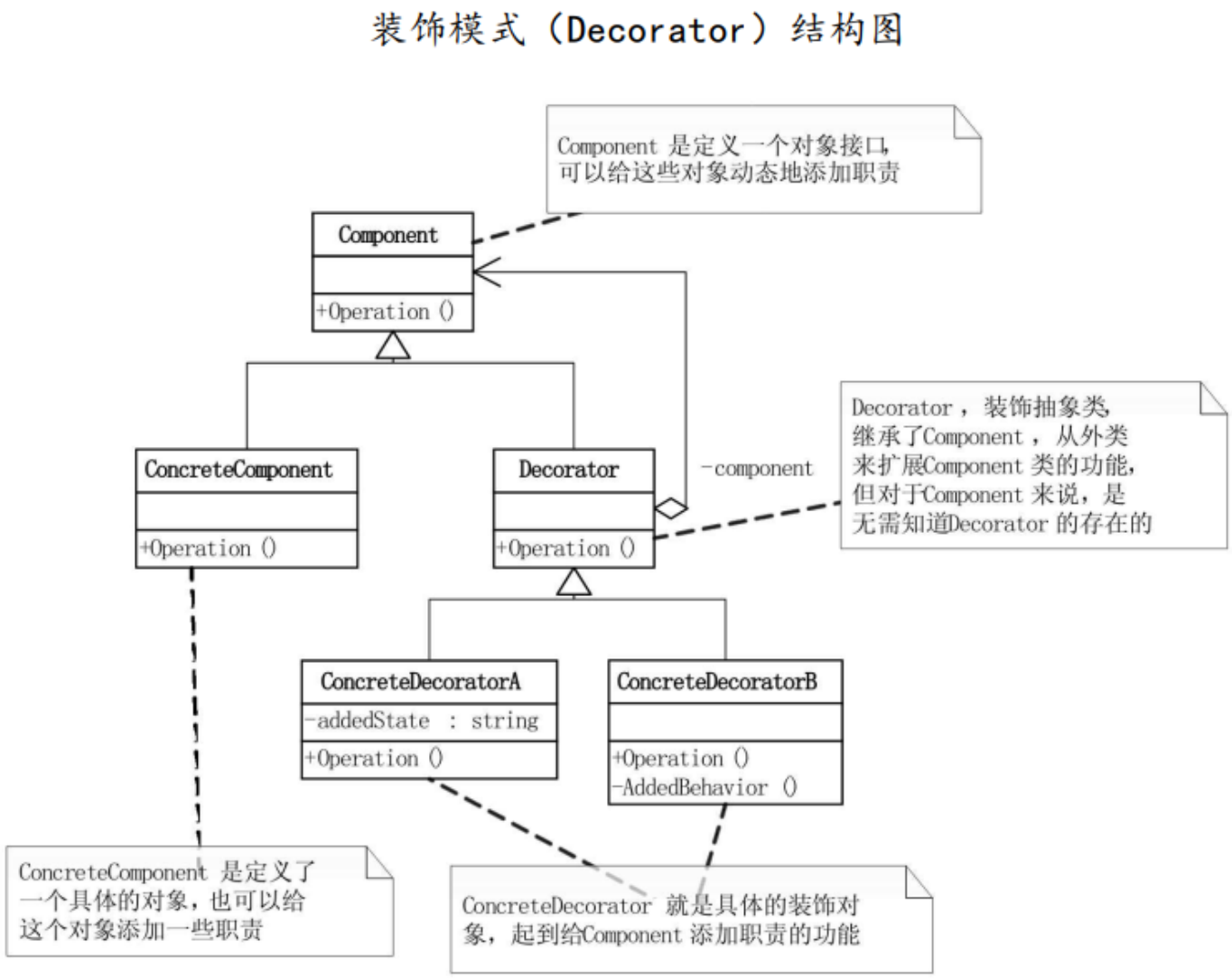
定义

装饰模式（Decorator Pattern）是一种结构型设计模式，它允许在不改变原始类接口的情况下，动态地添加功能或责任。装饰模式通过创建一个装饰类，包裹原始类的实例，并在保持原始类接口不变的情况下，提供额外的功能。，就增加功能来说，装饰模式比生成子类更为灵活。

主要角色

1. 组件接口：定义了具体组件和装饰器共同的接口，确保它们可以互相替换。
2. 具体组件：实现了组件接口，是被装饰的具体对象。
3. 装饰器：持有一个组件对象的引用，并实现了组件接口。装饰器通常是一个抽象类，它的具体子类实现具体的装饰逻辑。
4. 具体装饰器：继承自装饰器，实现了具体的装饰逻辑，并调用父类的方法以保持接口一致。

结构图



基本代码实现

（1）Component类

```
abstract class Component
{
    public abstract void Operation () ;
}
```

（2）ConcreteComponent类

```
class ConcreteComponent : Component
{
    public override void Operation（）
    {
        Console.WriteLine（"具体对象的操作"）；
    }
}
```

（3）Decorator类

```
abstract class Decorator : Component
{
    protected Component component;

    public void SetComponent(Component component)
    {
        this.component = component;
    }

    public override void Operation()
    {
        if (component != null)
        {
            component.Operation();
        }
    }
}
```

设置 Component

重写 Operation()，实际执行的是 Component 的 Operation()

（4）ConcreteDecoratorA类 和 ConcreteDecoratorB类

```
class ConcreteDecoratorA : Decorator
{
    private string addedState;

    public override void Operation()
    {
        base.Operation();
        addedState = "New State";
        Console.WriteLine("具体装饰对象 A 的操作");
    }
}

class ConcreteDecoratorB : Decorator
{
    public override void Operation()
    {
        base.Operation();
        AddedBehavior();
        Console.WriteLine("具体装饰对象 B 的操作");
    }

    private void AddedBehavior()
    {

    }
}
```

本类的独有功能，以区别于 ConcreteDecoratorB

首先运行原 Component 的 Operation()，再执行本类的功能，如 addedState，相当于对原 Component 进行了装饰

首先运行原 Component 的 Operation()，再执行本类的功能，如 AddedBehavior()，相当于对原 Component 进行了装饰

本类独有的方法，以区别于 ConcreteDecoratorA

（5）客户端代码

```
static void Main(string[] args)
{
    ConcreteComponent c = new ConcreteComponent();
    ConcreteDecoratorA d1 = new ConcreteDecoratorA();
    ConcreteDecoratorB d2 = new ConcreteDecoratorB();

    d1.SetComponent(c);
    d2.SetComponent(d1);
    d2.Operation();

    Console.Read();
}
```

装饰的方法是：首先用 ConcreteComponent 实例化对象 c，然后用 ConcreteDecoratorA 的实例化对象 d1 来包装 c，再用 ConcreteDecoratorB 的对象 d2 包装 d1，最终执行 d2 的 Operation()

4、装饰模式在什么情况下使用

装饰模式把每个要装饰的功能放在单独的类中，并让这个类包装它所要装饰的对象。因此，当需要执行特殊行为时，客户代码就可以在运行时根据需要有选择地、按顺序地使用装饰功能包装对象了。

23种设计模式的概念

创建型模式

1、工厂方法(factory method)模式：

定义一个创建对象的接口，但由子类决定需要实例化哪一个类。工厂方法使得子类实例化的过程推迟

2、抽象工厂(abstract factory)模式：

提供一个接口，可以创建一系列相关或相互依赖的对象，而无需指定他们具体的类

3、原型(prototype)模式：

用原型实例指定创建对象的类型，并且通过拷贝这个原型来创建新的对象

4、单例(singleton)模式：

保证一个类只有一个实例，并提供一个访问它的全局访问点

5、构建器(builder)模式：

将一个复杂类的表示与其构造相分离，使得相同的构建过程能够得出不同的表示

结构型模式

1、适配器(adapter)模式：

将一个类的接口转换成用户希望得到的另一个接口。它使原本不相容的接口得以协同工作——速记关键字:转换接口

2、桥接(bridge)模式：

将类的抽象部分和它的实现部分分离开来，使它们可以独立地变化——速记关键字：继承树拆分

3、组合(composite)模式：

将对象组合成树型结构以表示"整体-部分"的层次结构，使得用户对单个对象和组合对象的使用具有一致性——速记关键字：树形目录结构

4、装饰(decorator)模式：

动态地给一个对象添加一些额外的职责。它提供了用子类扩展功能的一个灵活的替代，比派生一个子类更加灵活——速记关键字：附加职责

5、外观(facade)模式：

定义一个高层接口，为子系统中的一组接口提供一个一致的外观，从而简化了该子系统的使用——速记关键字：对外统一接口

6、享元(flyweight)模式：

提供支持大量细粒度对象共享的有效方法

7、代理(proxy)模式：

为其他对象提供一种代理以控制这个对象的访问

行为型模式

1、职责链(chain of responsibility)模式：

通过给多个对象处理请求的机会，减少请求的发送者与接收者之间的耦合。将接收对象链接起来，在链中传递请求，直到有一个对象处理这个请求——速记关键字：传递职责

2、命令(command)模式：

将一个请求封装为一个对象，从而可用不同的请求对客户进行参数化，将请求排队或记录请求日志，支持可撤销的操作——速记关键字：日志记录，可撤销

3、解释器(interpreter)模式：

给定一种语言，定义它的文法表示，并定义一个解释器，该解释器用来根据文法表示来解释语言中的句子

4、迭代器(iterator)模式：

提供一种方法来顺序访问一个聚合对象中的各个元素而不需要暴露该对象的内部表示

5、中介者(mediator)模式：

用一个中介对象来封装一系列的对象交互。它使各对象不需要显式地相互调用，从而达到低耦合，还可以独立地改变对象间的交互——速记关键字：不直接引用

6、备忘录(memento)模式：

在不破坏封装性的前提下，捕获一个对象的内部状态，并在该对象之外保存这个状态，从而可用在以后将该对象恢复到原先保存的状态

7、观察者(observer)模式：

定义对象间的一种一对多的依赖关系，当一个对象的状态发生改变时，所有依赖于它的对象都得到通知并自动更新

8、状态(state)模式：

允许一个对象在其内部状态改变时改变它的行为——速记关键字：状态变成类

9、策略(strategy)模式：

定义一系列算法，把它们一个个封装起来，并且使它们之间可互相替换，从而让算法可以独立于使用它的用户而变化

10、模板方法(template method)模式：

定义一个操作中的算法骨架，而将一些步骤延迟到子类中，使得子类可以不改变一个算法的结构即可重新定义算法的某些特定步骤

11、访问者(visitor)模式：

表示一个作用于某对象结构中的各元素的操作，使得在不改变各元素的类的前提下定义作用于这些元素的新操作

设计原则

单一职责

在设计类的时候要尽量缩小粒度，使功能明确、单一，不要做多余的事情（高内聚，低耦合）

开闭原则

软件实体（类、模块、函数等）应该对扩展开放，对修改关闭。这意味着可以通过扩展来添加新功能，而不必修改现有代码。

里氏替换

子类型必须能够替换掉它们的基类型。在程序中，如果有一个基类和一个子类，那么可以用子类对象替换基类对象，而程序的行为仍然是正确的。

接口隔离

不应该强迫一个类实现它不需要的接口。一个类不应该对它用不到的方法负责。

依赖倒置

高层模块不应该依赖于低层模块，而是应该依赖于抽象。抽象不应该依赖于具体实现，具体实现应该依赖于抽象。

比如Java的操作数据库，Java定义了一组接口，由各个数据库去实现它，Java不依赖于他们，数据库依赖于Java

组合优于继承

继承耦合度高，组合耦合度低

继承基类是固定好的，但是组合通过组合类的指针，可以传入不同的类，避免高耦合

组合复用原则

1、概念：

优先使用组合 contains a（聚合 has a），而不是继承 is a 来达到目的

2、原因：

继承会将实现细节暴露给子类，继承复用破坏了封装性，是白箱复用
使用继承时需要考虑里氏替换原则

3、优点：

新类对象存取成员对象只通过成员对象的接口，是黑箱复用，系统更灵活，降低耦合度，可以在运行时动态进行，新对象可动态引用与成员对象类型相同的对象

4、缺点：

需要管理较多对象

迪米特法则

概念：

一个对象应当对其他对象有尽可能少的了解，即不和陌生人说话

“朋友圈”概念（以下情况是该对象朋友）：

1、this：

1. 该对象方法中的参数
2. 实例变量直接引用的对象
3. 实例变量如果是一个聚集（聚合对象），聚集中的元素
4. 该对象方法中创建的变量

2、要求：

1. 优先考虑将一个类设计成不变类
2. 尽量降低一个类的访问权限
3. 谨慎使用 Serializable（持久化，通过序列化一个对象，将其写入磁4. 盘，以后程序调用时重新恢复该对象）
4. 尽量降低成员的访问权限

3、优点： 降低类之间的耦合

4、缺点： 过多使用迪米特法则，会产生大量中介类，设计变复杂

面试题

手写单例设计模式

1. 饿汉式

```
public class EagerSingleton {

    // 类加载时就创建唯一实例
    private static final EagerSingleton instance = new EagerSingleton();

    // 私有构造方法
    private EagerSingleton() {
    }

    // 公有静态方法，获取唯一实例
    public static EagerSingleton getInstance() {
        return instance;
    }
}
```

2. 懒汉式

```
public class Singleton {
```

```
    // 私有静态成员变量，用于保存唯一的实例
    private static Singleton instance;

    // 私有构造方法，防止外部通过 new 创建实例
    private Singleton() {
    }

    // 公有静态方法，获取唯一的实例
    public static Singleton getInstance() {
        // 使用双重检查锁定，确保在多线程环境下也能正常工作，并且提高了性能
        if (instance == null) {
            synchronized (Singleton.class) {
                if (instance == null) {
                    instance = new Singleton();
                }
            }
        }
        return instance;
    }

}
```

3. 静态内部类单例模式

```
public class InnerStaticSingleton {

    // 私有静态内部类
    private static class SingletonHolder {
        private static final InnerStaticSingleton instance = new
InnerStaticSingleton();
    }

    // 私有构造方法
    private InnerStaticSingleton() {
    }

    // 公有静态方法，获取唯一实例
    public static InnerStaticSingleton getInstance() {
        return SingletonHolder.instance;
    }
}
```

4. 枚举单例模式

```
public enum EnumSingleton {

    // 定义一个枚举元素，它就是 Singleton 的唯一实例
    INSTANCE;

    // 其他业务方法
    public void doSomething() {

    }
}
```

你在实际开发中用过哪些设计模式

1. **单例模式：** 用于确保系统中某个类只有一个实例。在数据库连接、日志管理等场景中使用过。
2. **工厂模式：** 用于创建对象，将对象的创建过程封装在一个工厂类中。在创建复杂对象、实现类的选择时使用过。
3. **观察者模式：** 用于实现对象之间的一对多依赖关系，当一个对象的状态发生变化时，其所有依赖对象都会得到通知。在实现用户界面和数据模型之间的通信、事件处理等场景中使用过。

设计模式了解吗？有哪些设计原则需要遵守?

单一职责

在设计类的时候要尽量缩小粒度，使功能明确、单一，不要做多余的事情（高内聚，低耦合）

单一职责下的类会比较小而精悍，需要使用结构性模式组合复用他们

开闭原则

一个类应该对扩展开发，对修改关闭

关键是要做好封装，隐藏内部的实现细节，开发足够的接口，这样子外部的代码就只能通过接口去扩展功能，不需要侵入到类的内部

final关键字

不能说要实现某个功能必要要修改类内部的代码才能实现，需要能够通过扩展实现新的功能（低耦合）

里氏替换

子类能够完全替换父类，不会改变父类定义的行为

比如一个基类鸟类中有一个方法，能够飞行，所有的鸟类都必须继承它，但是企鹅、鸵鸟这些没办法飞行（使用接口代替继承）

接口隔离

不应该强迫客户依赖于它们不需要的接口

建立单一接口，不要建立庞大的接口，尽量细化，接口中的方法要建立少

类“犬科”依赖接口I中的方法：捕食（），行走（），奔跑（），宠物狗类是对类“犬科”的实现。对于具体的类：宠物狗来说，虽然存在着用不到的方法，但由于继承了接口，所以也必须要实现这些用不到的方法

依赖倒置

上层要避免依赖下层的实现细节，两者都应该依赖于抽象

比如Java的操作数据库，Java定义了一组接口，由各个数据库去实现它，Java不依赖于他们，数据库依赖于Java

组合优于继承

继承耦合度高，组合耦合度低

继承基类是固定好的，但是组合通过组合类的指针，可以传入不同的类，避免高耦合

说一说你了解的工厂模式

工厂模式提供了一种创建对象的接口，但由子类决定实例化的类是哪一个。工厂模式可以将复杂的对象创建过程封装在具体工厂类中，客户端只需关心使用工厂接口创建产品即可，适用于需要将对象创建过程进行解耦，或者在一个系统中有多个产品族、产品等级结构，需要统一管理的情况。

说一说你了解的观察者模式

定义了一种一对多的依赖关系，让多个观察者对象同时监听某一个主题对象，当主题对象状态发生变化时，会通知所有的观察者对象，使得它们能够自动更新。

考虑一个简单的气象站示例，气象站可以测量温度、湿度和气压，并在这些数据发生变化时通知观察者更新。

```
import java.util.ArrayList;
import java.util.List;

// 主题接口
interface Subject {
    void addObserver(Observer observer);
    void removeObserver(Observer observer);
    void notifyObservers();
}

// 具体主题类
class WeatherStation implements Subject {
    private List<Observer> observers = new ArrayList<>();
    private float temperature;

    public void setTemperature(float temperature) {
        this.temperature = temperature;
        notifyObservers();
    }

    @Override
    public void addObserver(Observer observer) {
        observers.add(observer);
```

```
    }

    @Override
    public void removeObserver(Observer observer) {
        observers.remove(observer);
    }

    @Override
    public void notifyObservers() {
        for (Observer observer : observers) {
            observer.update(temperature);
        }
    }
}

// 观察者接口
interface Observer {
    void update(float temperature);
}

// 具体观察者类
class Display implements Observer {
    @Override
    public void update(float temperature) {
        System.out.println("Display: Temperature is " + temperature);
    }
}

class Logger implements Observer {
    @Override
    public void update(float temperature) {
        System.out.println("Logger: Logging temperature data - " + temperature);
    }
}

// 客户端代码
public class Client {
    public static void main(String[] args) {
        WeatherStation weatherStation = new WeatherStation();

        Observer display = new Display();
        Observer logger = new Logger();

        weatherStation.addObserver(display);
        weatherStation.addObserver(logger);

        // 模拟温度变化
        weatherStation.setTemperature(25.5f);
        // 输出:
```

```
        // Display: Temperature is 25.5
        // Logger: Logging temperature data - 25.5
    }
}
```

代理模式和适配器模式有什么区别

代理模式（Proxy Pattern）和适配器模式（Adapter Pattern）都属于结构型设计模式

1. 目的不同

- 代理模式的主要目的是控制对对象的访问，可以添加额外的操作，而不改变原始对象的结构。
- 适配器模式的主要目的是使得原本由于接口不兼容而不能一起工作的类可以一起工作。

2. 角色不同

- 代理模式包含抽象主题、真实主题和代理。
- 适配器模式包含目标接口、适配器和被适配者

3. 应用场景不同

- 代理模式适用于控制对对象的访问、延迟加载、实现权限控制等场景。
- 适配器模式适用于解决两个已有接口不兼容的情况，使它们能够一起工作。

计算机系统&Linux

> 计算机体系结构内容较难，考察也较少，可以仅做了解，重点关注 Linux 相关的一些面试题

基础知识讲解

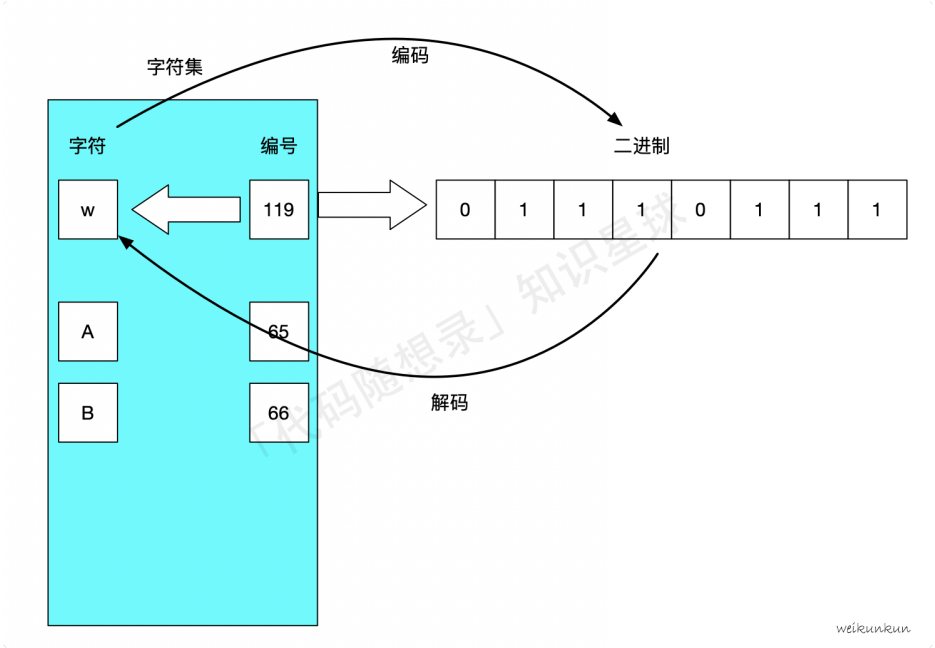
字符编码

我们知道一个字节有8位，8位在不考虑正负的情况下，能够表示256个数字。同理2个字节就能表示出65536个数字。那么对于字符该如何表示?

通过计算机领域的至理名言：**计算机科学领域的任何问题都可以通过增加一个间接的中间层来解决。**

我们通过在字符和二进制编码之间嵌套一个数字编号，通过数字编号，就能实现字符、二进制编码之间的关系绑定。

像这样收录了很多的字符并行一一编号得到的一个对照表，我们称之为字符集。从二进制 -> 字符 称为解码，从字符 -> 二进制 称为编码



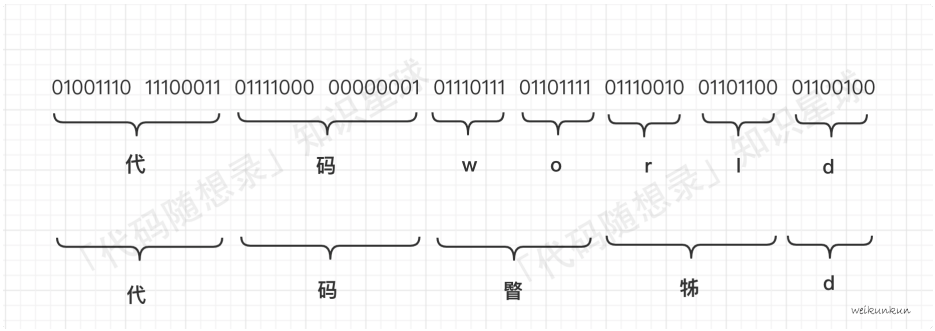
从ASCII字符集表示英文、控制符号这些，再到GB2312能表示出中文，目前存在着很多的字符集，这也就出现了一个问题，同一个二进制，在不同的字符集可能对应的字符不同，这也就导致了同一个文件，通过不同的字符集进行解码，呈现的结果不同（也就是我们所说的乱码）。如果有一种统一的编码形式（很全的字符集），对所有的字符都能提供唯一的二进制编码，那么就能解决上述的问题。Unicode就是这个很全的字符集。

虽然通过Unicode我们实现了对字符的唯一表示，但是随之而来的还有另一个问题，这些二进制编码该如何存储?

字符	编码	二进制
代	20195	01001110 11100011
码	30721	01111000 00000001
w	119	00000000 01110111
o	111	00000000 01101111
r	114	00000000 01110010
l	108	00000000 01101100
d	100	00000000 01100100

weikunkun

以代码world为例，将二进制直接照搬成一行，按照不同的步长进行解析，则会得出不同的结果，如何解决?



固定长度编码

最直接思路，就是固定长度，不管编号多大多小，统一根据最长位数来，位数不够，高位补零。

字符	编码	二进制
代	20195	00000000 00000000 01001110 11100011
码	30721	00000000 00000000 01111000 00000001
w	119	00000000 00000000 00000000 01110111
o	111	00000000 00000000 00000000 01101111
r	114	00000000 00000000 00000000 01110010
l	108	00000000 00000000 00000000 01101100
d	100	00000000 00000000 00000000 01100100

优点：高效、快速进行编码、解码

缺点：固定长度的形式，会导致内存的浪费，并且随着编码越来越大，编码之间的跨度也就越大，这样定长编码造成的浪费就更加显著

可变长度编码

根据固定长度编码的缺点，可以采用变长的形式进行解决，小编号少占用字节，大编号多占字节。不过如何划分边界是一个问题

一种解决方案：

编号	编码模板
[0, 127]	0XXXXXXX
[128, 2047]	110XXXXX 10XXXXXX
[2048, 65535]	1110XXXX 10XXXXXX 10XXXXXXXX

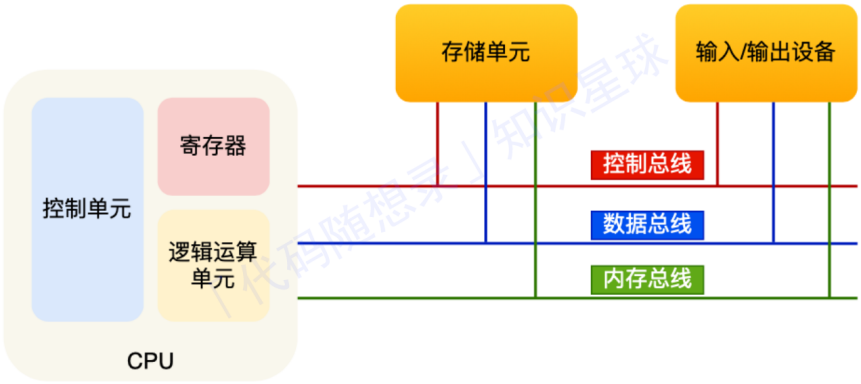
可以采用类似IPV4中的分类编制的形式，在二进制中占用几位作为标识。

w	119	01110111
代	228	11000011 10100100
码	30721	11100111 10100000 10000001

上述的解决方案，其实就是我们所熟知的UTF-8编码，同时也是Go默认的编码方式

冯诺依曼模型

冯诺依曼模型分为 5 大部件：内存、CPU、总线、输入设备、输出设备



PC 实质上是一个大小为一个字的存储区域。对于32位的机器，一个字是4个字节；对于64位的机器，一I/O设备个字就是8个字节。说白了，PC就是一个4字节或是8字节的存储空间，里面存放的是某一条指令的地址。

从系统上电的那一瞬间，直到系统断电，处理器就不断地在执行 PC 指向的指令，然后更新 PC，使其指向下一条要执行的指令（注意：这个下一条指令与刚刚执行的指令不一定是相邻的）

内存

从 0 开始编号，最先存储单位是字节（byte）

处理器在执行程序时，内存主要存放程序指令以及数据

内存和处理器之间通过总线来进行数据传递。实际上，总线贯穿了整个计算机系统，它负责将信息从一个部件传递到另外一个部件。

CPU

CPU 位宽： CPU 一次能处理多少字节的数据

32位CPU一次可以计算4个字节;
64位CPU一次可以计算8个字节;

为了能一次计算大数的运算，CPU需要支持多个byte一起计算，所以CPU位宽越大，可以计算的数值就越大。
比如说32位CPU能计算的最大整数是4294967295。

CPU内部还有一些组件，常见的有寄存器、控制单元和逻辑运算单元等

控制单元： 控制CPU工作；
逻辑运算单元： 负责计算；
寄存器： 在 CPU 中，计算速度快

通俗的讲，寄存器可以理解为一个临时存放数据的空间。

例如我们计算两个变量 a+b 的和，处理器从内存中读取 a 的值暂存在寄存器 X 中，读取 B 的值暂存在寄存器 Y 中，这个操作会覆盖寄存器中原来的数值，处理器完成加载的操作后，ALU（Arithmatic/logic Unit）会从复制寄存器 X 和 Y 中保存的数值，然后进行算术运算，得到的结果会保存到寄存器 X 或者寄存器 Y 中，此时寄存器中原来的数值会被新的数值覆盖

常见寄存器类型：

通用寄存器：放运算数据
程序记数器（PC）：用来存放 CPU 下一条要执行指令的 内存地址；
指令寄存器：用来存放 PC 执行的指令，即 指令本身（取值后放入到 指令寄存器中）

总线

用于 CPU 和内存以及其他设备之间的通信，**总线可分为 3 种:**

地址总线：

用于指定CPU将要操作的内存地址;

数据总线：

用于读写内存的数据;

控制总线：

用于发送和接收信号，比如中断、设备复位等信号，CPU收到信号后自然进行响应，这时也需要控制总线;

当CPU要读写内存数据的时候．一般需要通过两个总线:

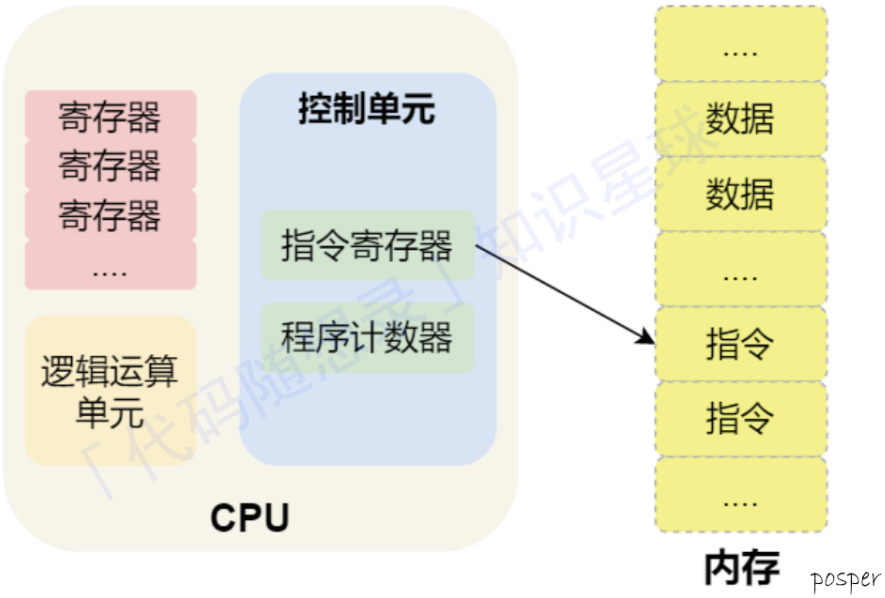
首先要通过「地址总线」来指定内存的地址;
再通过「数据总线」来传输数据;

程序的执行过程

现代大多数 CPU 都使用来流水线的方式来执行指令，
所谓的流水线就是把一个任务拆分成多个小任务，于是一条指令通常分为 4 个阶段，称为 4 级流水线

1. 取指令
2. CPU 的 控制单元向 地址总线 发送 PC 中的地址，从内存中找到 待执行指令；
3. 并将 待执行指令 通过 数据总线 传入到 CPU 中的 指令寄存器 中
4. 自动更新 PC，指向下一条待执行指令的地址
5. 译指（分析指令）
6. CPU分析「指令寄存器」中的指令，确定指令的类型和参数：
    1. 如果是计算类型的指令，就把指令交给「逻辑运算单元」运算;
    2. 如果是存储类型的指令，则交由「控制单元」执行
7. 执行指令

8. 根据 译指的结果，把指令交给 CPU 中响应的控件（逻辑运算单元/控制单元）执行
9. 数据回写
10. CPU 将计算结果存回寄存器或者将寄存器的值存入内存



不同的阶段其实是由计算机中的不同组件完成的：

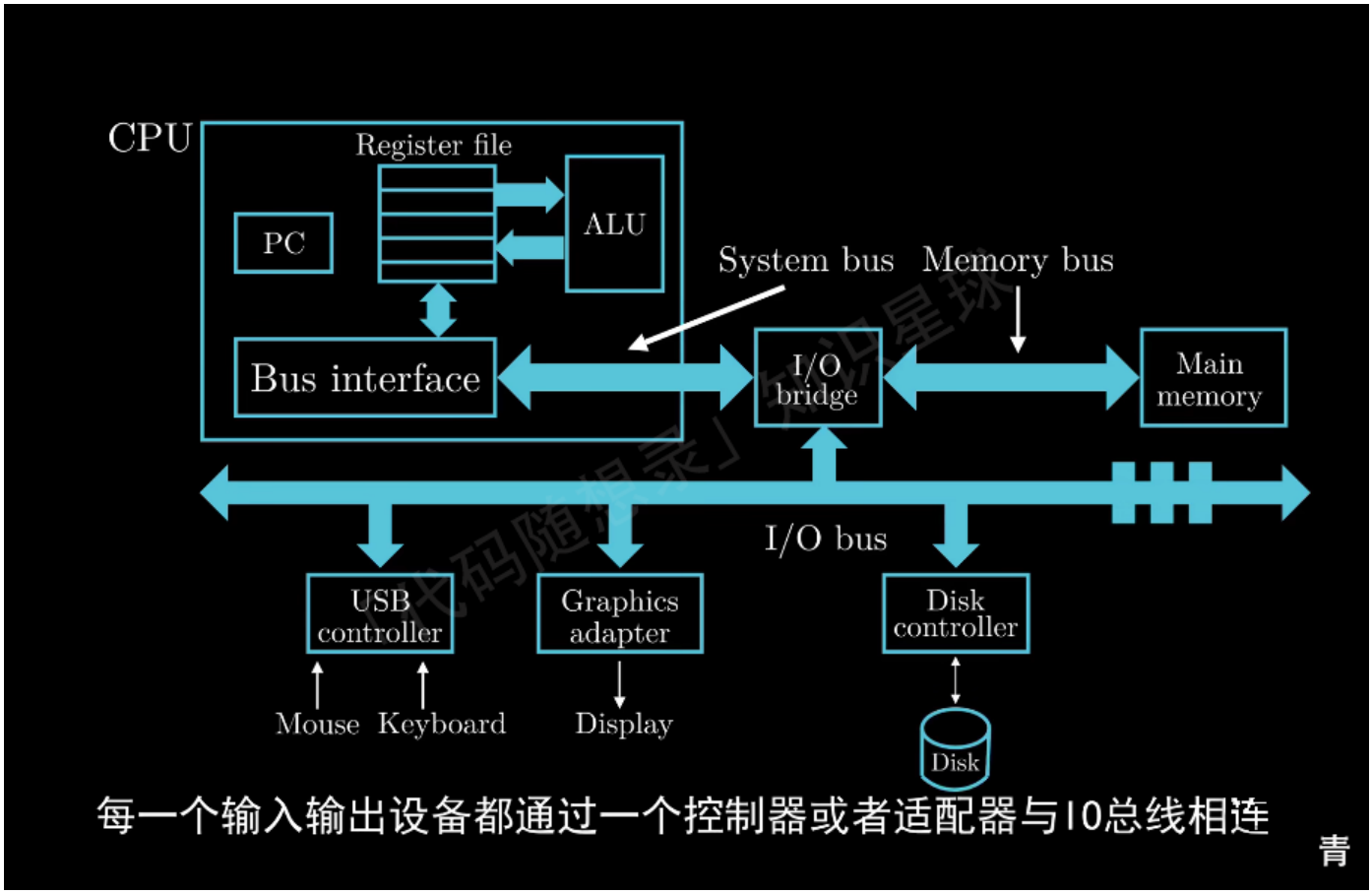
取指令、译指令：由 控制器 进行；

指令执行：

1. 算术操作、逻辑操作、数据传输、条件分支都是由 逻辑运算单元（运算器)完成
2. 跳转语句：由 控制器（控制单元）完成

输入/输出设备

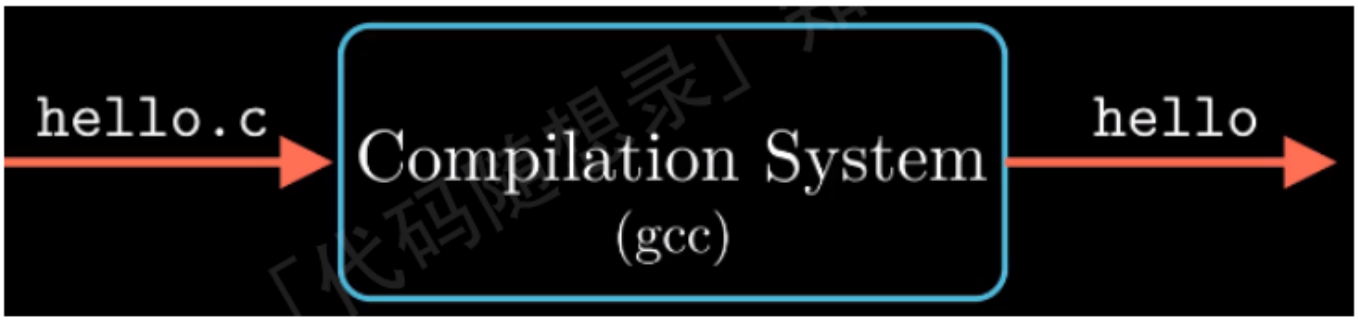
除了处理器，内存以及总线，计算机系统还包含各种输入输出设备，如键盘、鼠标、显示器以及磁盘等等。



控制器与适配器主要区别是在于它们的封装方式，无论是控制器还是适配器，它们的功能都是在 IO 设备与 IO 总线之间传递数据

编译系统如何工作

```
1 linux> gcc -o hello hell.c
```



1、理解编译系统可以优化程序的性能。

例如：一个switch语句是不是要比一连串的if-else要高效得多？函数调用的开销有多大？while循环比for循环更高效么？

2、理解编译系统可以帮助我们理解链接过程中出现的错误

例如：静态变量和全局变量的区别是什么？静态库和动态库的区别是什么？

3、避免安全漏洞

例如：缓冲区溢出（buffer overflow）是导致互联网安全漏洞的主要原因，如何避免写出的代码存在安全漏洞，第一步就是要理解数据和控制信息在程序栈上是如何存储的，了解不严谨不规范的书写方式会引起什么样的后果

处理器解释指令

处理器读并解释储存在内存中的指令

运行hello程序

hello.c 经过编译系统得到可执行目标文件 hello，此时可执行目标文件 hello 已经存放在系统的磁盘上，那么，如何运行这个可执行文件呢?

在 linux 系统上运行可执行程序：打开一个 shell 程序，然后在 shell 中输入相应可执行程序的文件名。

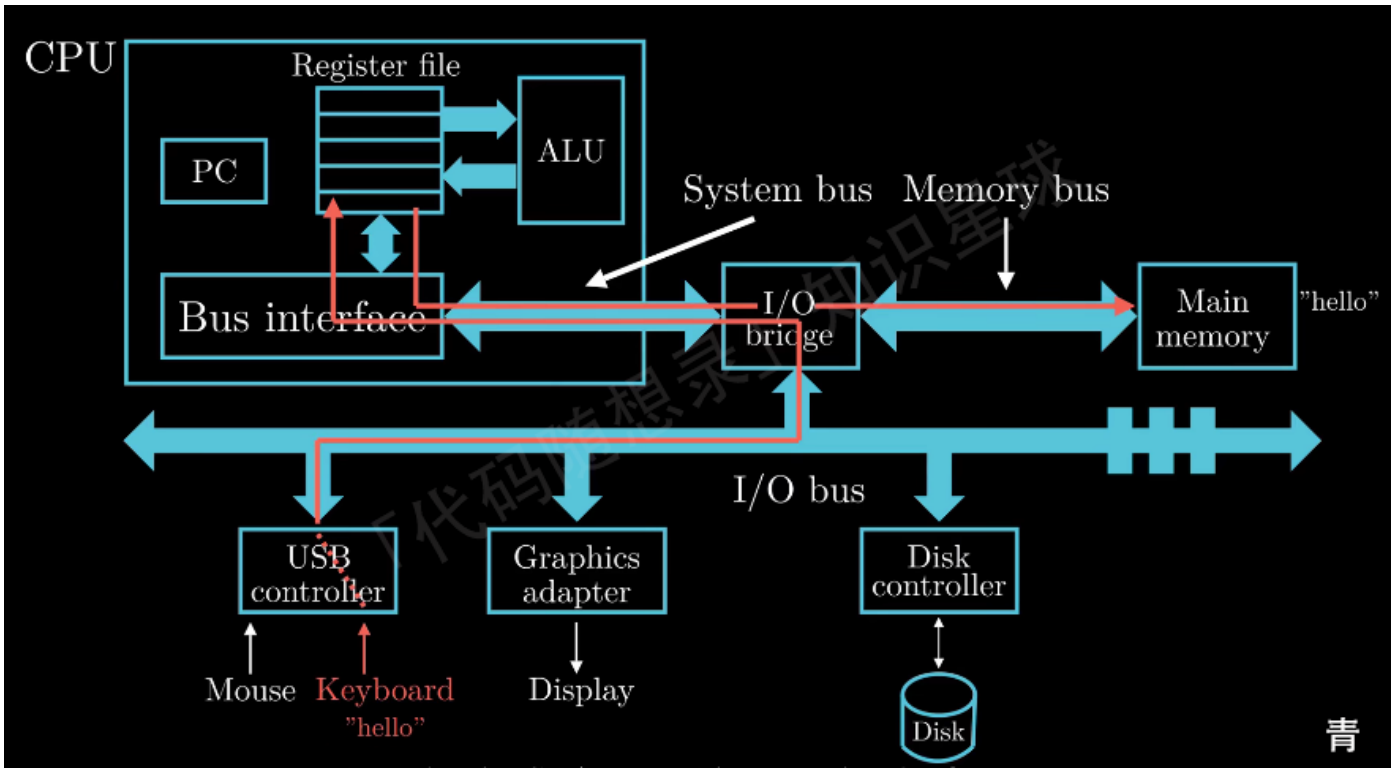
```
linux> ./hello
```

shell 是一个命令解释程序，它输出一个提示符 > 来等待一个命令行的输入，然后执行这个命令，如果该命令行的第一个单词不是内置的 shell 命令，那么 shell 就会假设这是一个可执行文件的名字，对这个文件进行加载并运行。

在这个例子中，shell 加载并且运行 hello 程序，屏幕上显示 hello,world 内容，hello 程序运行结束并退出，shell 继续等待下一个命令的输入。

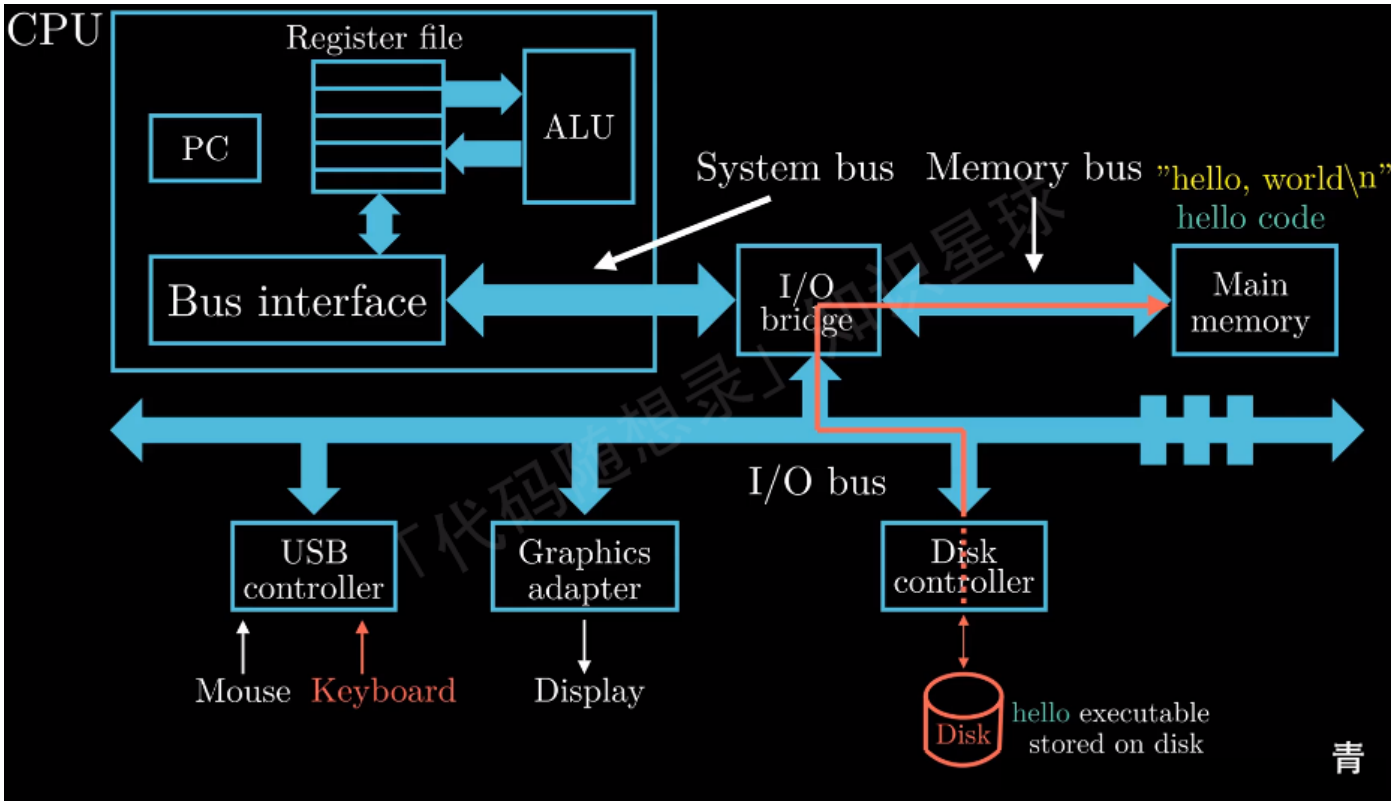
程序执行流程

首先我们通过键盘输入"./hello" 的字符串，shell 程序会将输入的字符逐一读入寄存器，处理器会把 hello 这个字符串放入内存中。

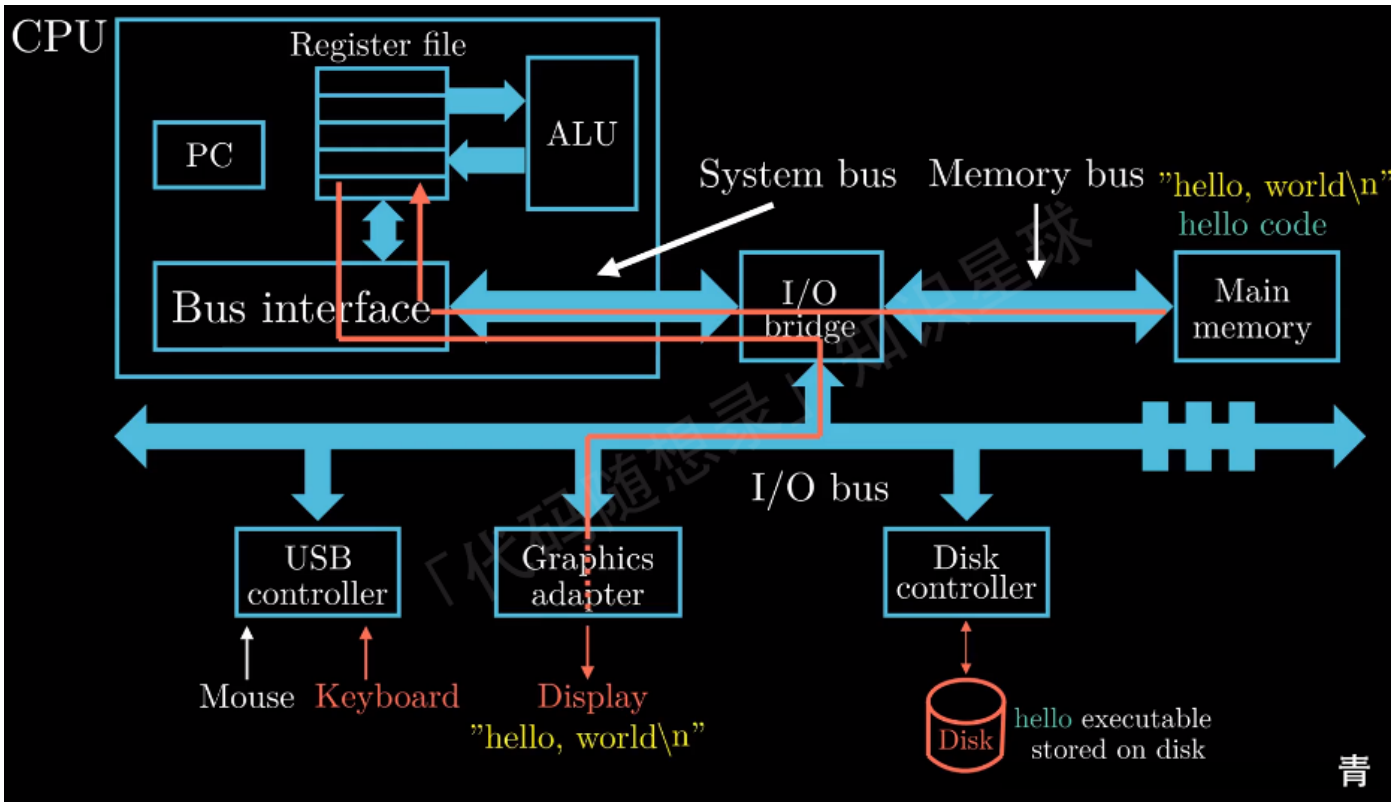


当我们完成输入，按下回车键时，shell 程序就知道我们已经完成了命令的输入，然后执行一系列的指令来来加载可执行文件 hello ，这些指令将 hello 中的数据和代码从磁盘复制到内存。

数据就是我们要显示输出的 "hello , world\n" ，这个复制的过程将利用DMA（Direct Memory Access）技术，数据可以不经过处理器，从磁盘直接到达内存。当可执行文件 hello 中的代码和数据被加载到内存中，处理器就开始执行 main 函数中的代码，main 函数非常简单，只有一个打印功能。



CPU会将 "hello , world\n" 这个字符串从内存复制到寄存器文件。然后再从寄存器文件复制到显示设备，最终hello , world显示在屏幕上。



从 hello 程序执行的过程来看，系统即使执行如此简单的程序，数据信息仍旧需要在磁盘、内存、处理器以及 IO 设备之间进行搬运。

数据从一个地方搬运到另外一个地方需要花费时间，系统设计人员的一个主要任务就是缩短信息搬运所花费的时间。

场景题 & 系统设计

系统设计

如何设计一个高性能的系统

后台服务架构高性能设计之道

如何设计一个秒杀系统

秒杀系统相关问题

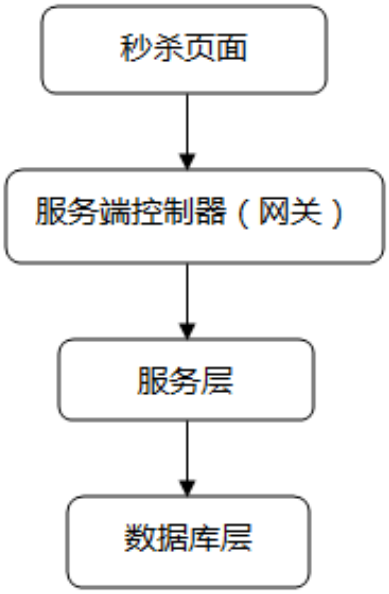
1. 秒杀系统场景特点

- 秒杀一般是访问请求数量远远大于库存数量，只有少部分用户能够秒杀成功
- 秒杀时大量用户会在同一时间同时进行抢购，网站瞬时访问流量激增
- 秒杀业务流程比较简单，一般就是下订单减库存

2. 秒杀架构设计理念

- 限流： 鉴于只有少部分用户能够秒杀成功，所以要限制大部分流量，只允许少部分流量进入服务后端秒杀程序。
- 削峰：对于秒杀系统瞬时会有大量用户涌入，所以在抢购一开始会有很高的瞬间峰值。高峰值流量是压垮系统很重要的原因，所以如何把瞬间的高流量变成一段时间平稳的流量也是设计秒杀系统很重要的思路。实现削峰的常用的方法有前端添加一定难度的验证码后端利用缓存和消息中间件等技术。
- 异步处理：秒杀系统是一个高并发系统，采用异步处理模式可以极大地提高系统并发量，其实异步处理就是削峰的一种实现方式。
- 内存缓存：秒杀系统最大的瓶颈一般都是数据库读写，由于数据库读写属于磁盘IO，性能很低，如果能够把部分数据或业务逻辑转移到内存缓存，效率会有极大地提升。
- 可拓展：当然如果我们想支持更多用户，更大的并发，最好就将系统设计成弹性可拓展的，如果流量来了，拓展机器就好了。像淘宝、京东等双十一活动时会增加大量机器应对交易高峰。

3. 架构方案
   一般的秒杀系统架构



设计思路

4. 设计思路

- 将请求拦截在系统上游，降低下游压力：秒杀系统特点是并发量极大，但实际秒杀成功的请求数量却很少，所以如果不在前端拦截很可能造成数据库读写锁冲突，甚至导致死锁，最终请求超时
- 充分利用缓存：利用缓存预减库存，拦截掉大部分请求
- 消息队列：这是一个异步处理过程，后台业务根据自己的处理能力，从消息队列中主动的拉取请求消息进行业务处理

5. 前端方案

- 页面静态化：将活动页面上的所有可以静态的元素全部静态化，并尽量减少动态元素。通过CDN来抗峰值。
- 禁止重复提交：用户提交之后按钮置灰，禁止重复提交
- 用户限流：在某一时间段内只允许用户提交一次请求，比如可以采取IP限流

6. 后端方案

服务端控制器层(网关层)

限制uid（UserID）访问频率：我们上面拦截了浏览器访问的请求，但针对某些恶意攻击或其它插件，在服务端控制层需要针对同一个访问uid，限制访问频率。

7. 服务层

上面只拦截了一部分访问请求，当秒杀的用户量很大时，即使每个用户只有一个请求，到服务层的请求数量还是很大。比如我们有100W用户同时抢100台手机，服务层并发请求压力至少为100W。

- 把需要秒杀的商品的主要信息以及库存初始化到redis缓存中
- 做请求合法性的校验（比如是否登录），如果请求非法，直接给前端返回错误码，进行相应的提示
- 进行内存标识的判断(true 已经秒杀结束，false 未秒杀结束，下面第4步会写入)，如果内存标识为true，直接返回秒杀结束
- redis中使用decr 进行预减库存操作，判断：如果decr后库存量小于0，则把内存标记置为true(已经秒杀结束，第3步会用到)，且返回秒杀结束
- 用redis的布隆过滤器来判断是否已经秒杀到了（下面第7步会写入），防止重复秒杀，如果重复秒杀，直接返回重复秒杀的错误码。具体做法是：先用redis的布隆过滤器来判断是否秒杀过，如果布隆过滤器判断已经秒杀过了， 则再次查库确认是否秒杀过了，之所以再次查库确认是因为布隆过滤器对可能存在的数据是有误判率的；但是它对不存在的数据的判断是百分百准确的，所以如果redis的布隆过滤器判断没秒杀过，就直接放

过去进行秒杀

- 发送成功秒杀到的MQ消息给相应的业务端进行处理，并给用户端返回排队中，如果客户端收到排队中的消息，则自动进行轮询查询，直到返回秒杀成功或者秒杀失败为止
- 相应的业务端进行处理：真正处理秒杀的业务端，再次进行校验（比如秒杀是否结束，库存是否充足等）、将用户和商品id作为key存入redis的布隆过滤器来标识该用户秒杀该商品成功（第5步会用到）、减库存（这里的是真正的减库存，操作数据库的库存）、生成秒杀订单、返回秒杀成功

注意：就算请求走到了真正处理业务的这一端，也有可能秒杀失败，比如秒杀结束，库存不足，真正减库存失败，秒杀单生成失败等等，一旦失败，则返回秒杀结束

优化：将秒杀接口隐藏：用户点击秒杀按钮的时候，根据用户id生成唯一的加密串存入缓存并返回给客户端，然后客户端再次请求的时候带着加密串过来，后端进行校验是否合法，若不合法，直接返回请求非法；

限制某个接口的访问频率：可以用拦截器配合自定义注解来实现，这么做可以和具体的业务分离减少入侵，使用起来也非常方便

8. 数据库层

- 数据库层是最脆弱的一层，一般在应用设计时在上游就需要把请求拦截掉，数据库层只承担“能力范围内”的访问请求。所以，上面通过在服务层引入队列和缓存，让最底层的数据库高枕无忧
- 为防止秒杀出现负数订单数大于真正的库存数，所以在真正减库存，update库存的时候应该加上where 库存>0，而且需要给秒杀订单表加上用户id和商品id联合的唯一索引

如何设计一个本地缓存

想要设计一个本地缓存，考虑点主要在数据用何种方式存储，能存储多少数据，多余的数据如何处理等几个点，下面我们来详细的介绍每个考虑点：

1. 数据结构

   首要考虑的就是数据该如何存储，用什么数据结构存储。最简单的就直接用Map来存储数据，或者复杂的如redis一样提供了多种数据类型哈希，列表，集合，有序集合等，底层使用了双端链表，压缩列表，集合，跳跃表等数据结构。

2. 对象上限

   因为是本地缓存，内存有上限，所以一般都会指定缓存对象的数量比如1024，当达到某个上限后需要有某种策略去删除多余的数据。

3. 清除策略

   上面说到当达到对象上限之后需要有清除策略，常见的比如有LRU(最近最少使用)、FIFO(先进先出)、LFU(最近最不常用)、SOFT(软引用)、WEAK(弱引用)等策略。

4. 过期时间

   除了使用清除策略，一般本地缓存也会有一个过期时间设置，比如redis可以给每个key设置一个过期时间，这样当达到过期时间之后直接删除，采用清除策略+过期时间双重保证。

5. 线程安全

   像redis是直接使用单线程处理，所以就不存在线程安全问题。而我们现在提供的本地缓存往往是可以多个线程同时访问的，所以线程安全是不容忽视的问题，并且线程安全问题是不应该抛给使用者去保证。

6. 简明的接口

提供一个傻瓜式的对外接口是很有必要的，对使用者来说使用此缓存不是一种负担而是一种享受，提供常用的get，put，remove，clear，getSize等方法即可。

7. 是否持久化

   这个其实不是必须的，是否需要将缓存数据持久化看需求。本地缓存如ehcache是支持持久化的，而guava是没有持久化功能的。分布式缓存如redis是有持久化功能的，memcached是没有持久化功能的。

8. 阻塞机制

   我们使用缓存的目的就是因为被缓存的数据生成比较费时，比如调用对外的接口，查询数据库，计算量很大的结果等等。这时候如果多个线程同时调用get方法获取的结果都为null，每个线程都去执行一遍费时的计算，其实也是对资源的浪费。最好的办法是只有一个线程去执行，其他线程等待，计算一次就够了。但是此功能基本上都交给使用者来处理，很少有本地缓存有这种功能。

如何设计一个RPC框架

1. **定义RPC接口：** 定义服务接口及其方法。这是RPC的核心，决定了客户端可以调用哪些方法。
2. **选择通信协议：** 选择合适的通信协议，如HTTP/REST、TCP、UDP等。不同的协议有不同的优劣势，取决于你的应用需求。
3. **选择序列化协议：** 选择序列化协议，将数据转换为字节流以在网络上传输。常见的有JSON、Protobuf、Thrift等。考虑效率、可读性和平台兼容性。
4. **实现远程调用：** 实现将RPC请求发送到远程服务并接收响应的机制。这可能涉及到网络通信、数据解析和错误处理。
5. **服务注册与发现：** 提供服务注册中心，使服务能够动态注册和发现。Zookeeper、Consul、etcd等工具可以用于服务注册与发现。
6. **负载均衡：** 实现负载均衡机制，确保请求在多个服务节点之间分布均匀。常见的负载均衡算法有轮询、随机、最小连接数等。
7. **安全性：** 考虑数据在传输中的安全性，可以使用加密协议（如TLS/SSL）来保护通信。
8. **错误处理：** 设计良好的错误处理机制，包括定义错误码、异常传播和日志记录。确保客户端能够得知发生了什么问题。
9. **性能优化：** 考虑并发性、延迟和吞吐量等性能方面的问题。使用连接池、异步IO等技术进行优化。
10. **监控和日志：** 实现监控和日志记录，以便及时发现问题并进行故障排除。集成日志系统和监控工具，如Prometheus、Grafana等。

微信红包怎么设计?

微信红包架构设计

扫码登录怎么设计?

扫码登录流程